

DevOps & AWS Complete Learning Guide

1 DevOps & AWS Learning Hub

[1.1 Download PDFs](#)

[1.2 Suggested study pace](#)

[1.3 Page convention](#)

2 DevOps Curriculum: Tools & Concepts (Beginner → Advanced)

[2.1 Level 0 — Prerequisites \(before DevOps\)](#)

[2.2 Level 1 — DevOps Foundations](#)

[2.3 Level 2 — Infrastructure & Automation](#)

[2.4 Level 3 — Containers & Orchestration](#)

[2.5 Level 4 — CI/CD Pipelines](#)

[2.6 Level 5 — Cloud Platforms \(cross-cutting\)](#)

[2.7 Level 6 — Observability & Reliability](#)

[2.8 Level 7 — Security \(DevSecOps\)](#)

[2.9 Level 8 — Data, Messaging & Databases \(Ops view\)](#)

[2.10 Level 9 — Platform Engineering & Internal Developer Platforms \(adv\)](#)

[2.11 Level 10 — Advanced & Specialized \(adv\)](#)

[2.12 Certifications \(optional milestones\)](#)

[2.13 Learning project ideas \(apply everything\)](#)

3 AWS Cloud Curriculum: Services & Tools (Beginner → Advanced)

[3.1 1. Account, Identity & Governance ★](#)

[3.2 2. Networking & Content Delivery ★](#)

[3.3 3. Compute ★](#)

[3.4 4. Storage ★](#)

[3.5 5. Databases ★](#)

[3.6 6. Application Integration & Messaging ★](#)

[3.7 7. Developer Tools & CI/CD ★](#)

[3.8 8. Observability & Management ★](#)

[3.9 9. Security, Identity & Compliance ★](#)

[3.10 10. Migration & Transfer ◆](#)

[3.11 11. Analytics & Big Data ◆](#)

[3.12 12. Machine Learning & AI ◇](#)

[3.13 13. IoT, Media & Specialized ◇](#)

[3.14 14. Cost & Billing ★](#)

[3.15 15. Well-Architected Framework \(conceptual ★\)](#)

[3.16 Certification path \(recommended order\)](#)

[3.17 Hands-on lab progression](#)

4 DevOps Handbook — Master Table of Contents

[4.1 How to read this handbook](#)

[4.2 Part I — Introduction & Mindset \(Pages 1–60\)](#)

[4.3 Part II — Linux & Shell Mastery \(Pages 61–140\)](#)

[4.4 Part III — Git & Collaboration \(Pages 141–200\)](#)

[4.5 Part IV — Networking for DevOps \(Pages 201–280\)](#)

[4.6 Part V — Scripting & Programming \(Pages 281–360\)](#)

[4.7 Part VI — Infrastructure as Code \(Pages 361–480\)](#)

[4.8 Part VII — Containers \(Pages 481–580\)](#)

[4.9 Part VIII — Kubernetes \(Pages 581–720\)](#)

[4.10 Part IX — CI/CD \(Pages 721–840\)](#)

[4.11 Part X — Observability & SRE \(Pages 841–960\)](#)

[4.12 Part XI — Security \(DevSecOps\) \(Pages 961–1080\)](#)

[4.13 Part XII — Advanced Topics \(Pages 1081–1200\)](#)

[4.14 Expansion status](#)

5 Chapter 1: What Is DevOps? History and Culture

[5.1 1.1 The problem DevOps solves](#)

[5.2 1.2 A working definition](#)

[5.3 1.3 Brief history](#)

[5.4 1.4 Culture before tools](#)

[5.5 1.5 DevOps practices \(overview\)](#)

[5.6 1.6 Organizational models](#)

[5.7 1.7 What DevOps is NOT](#)

[5.8 1.8 Your learning path in this handbook](#)

[5.9 1.9 Chapter summary](#)

[5.10 Lab 1.1 — Reflect on your current workflow](#)

[5.11 Review questions](#)

[6 Chapter 2: CALMS, Three Ways, and Team Topologies](#)

[6.1 2.1 The CALMS framework](#)

[6.2 2.2 The Three Ways \(Gene Kim\)](#)

[6.3 2.3 Team Topologies \(Skelton & Pais\)](#)

[6.4 2.4 Conway's Law and reverse Conway maneuver](#)

[6.5 2.5 Maturity model \(informal\)](#)

[6.6 2.6 Chapter summary](#)

[6.7 Lab 2.1 — CALMS assessment](#)

[6.8 Review questions](#)

[7 Chapter 3: DevOps vs SRE vs Platform Engineering](#)

[7.1 3.1 Three related disciplines](#)

[7.2 3.2 Site Reliability Engineering \(SRE\)](#)

[7.3 3.3 Platform Engineering](#)

[7.4 3.4 How they fit together](#)

[7.5 3.5 Job titles in the market](#)

[7.6 3.6 Chapter summary](#)

[7.7 Lab 3.1](#)

[8 Chapter 4: Career Paths, Certifications, and Learning Strategy](#)

[8.1 4.1 DevOps is a skill set, not a single job title](#)

[8.2 4.2 Common career paths](#)

[8.3 4.3 Certifications: value and limits](#)

[8.4 4.4 Building a learning strategy](#)

[8.5 4.5 Portfolio and interview preparation](#)

[8.6 4.6 Soft skills that separate good from great](#)

[8.7 4.7 Staying current without burnout](#)

[8.8 4.8 Chapter summary](#)

[8.9 Lab 4.1 — Personal learning plan](#)

[8.10 Lab 4.2 — Job description analysis](#)

[8.11 Review questions](#)

[9 Chapter 5: Linux Fundamentals — Filesystem, Users, Permissions](#)

[9.1 5.1 Why Linux dominates DevOps](#)

[9.2 5.2 Filesystem hierarchy \(FHS\)](#)

[9.3 5.3 Essential navigation commands](#)

[9.4 5.4 File operations](#)

[9.5 5.5 Users, groups, and permissions](#)

[9.6 5.6 Package management \(Debian/Ubuntu\)](#)

[9.7 5.7 Text processing \(daily DevOps tools\)](#)

[9.8 5.8 Environment and shell](#)

[9.9 5.9 SSH — remote access](#)

[9.10 5.10 Chapter summary](#)

[9.11 Lab 5.1 — Linux sandbox](#)

[9.12 Review questions](#)

[10 Chapter 6: Processes, systemd, and Service Management](#)

[10.1 6.1 Understanding Linux processes](#)

[10.2 6.2 systemd — modern service manager](#)

[10.3 6.3 Cron and timers](#)

[10.4 6.4 Resource limits and troubleshooting](#)

[10.5 6.5 Chapter summary](#)

[10.6 Lab 6.1](#)

11 Chapter 7: Networking on Linux — ip, ss, curl, dig

[11.1 7.1 Why networking skills matter for DevOps](#)

[11.2 7.2 The ip command suite](#)

[11.3 7.3 Sockets and connections with ss](#)

[11.4 7.4 HTTP debugging with curl](#)

[11.5 7.5 DNS with dig](#)

[11.6 7.6 Putting it together: diagnostic workflow](#)

[11.7 7.7 /etc/hosts, resolv.conf, and systemd-resolved](#)

[11.8 7.8 Firewalls at a glance](#)

[11.9 7.9 Chapter summary](#)

[11.10 Lab 7.1 — Local service bind address](#)

[11.11 Lab 7.2 — DNS and latency baseline](#)

[11.12 Review questions](#)

12 Chapter 8: Bash Scripting for Automation

[12.1 8.1 When bash is the right tool](#)

[12.2 8.2 Script anatomy and safety](#)

[12.3 8.3 Variables, quoting, and expansion](#)

[12.4 8.4 Conditionals and tests](#)

[12.5 8.5 Loops and functions](#)

[12.6 8.6 Exit codes and error handling](#)

[12.7 8.7 External commands and JSON](#)

[12.8 8.8 Logging and observability](#)

[12.9 8.9 Argument parsing](#)

[12.10 8.10 Script layout in repos](#)

[12.11 8.11 Security considerations](#)

[12.12 8.12 Chapter summary](#)

[12.13 Lab 8.1 — Health check script](#)

[12.14 Lab 8.2 — Batch service restart](#)

[12.15 Review questions](#)

13 Chapter 9: Git Internals — Objects, Refs, and the DAG

[13.1 9.1 Git is a content-addressable filesystem](#)

[13.2 9.2 How a commit is stored](#)

[13.3 9.3 Essential Git commands \(beyond basics\)](#)

[13.4 9.4 Branches and merging](#)

[13.5 9.5 Remote repositories](#)

[13.6 9.6 .gitignore and hooks](#)

[13.7 9.7 Chapter summary](#)

[13.8 Lab 9.1](#)

14 Chapter 10: Branching Strategies and Code Review

[14.1 10.1 Branches as a collaboration contract](#)

[14.2 10.2 Trunk-based development in depth](#)

[14.3 10.3 Git Flow and when it still fits](#)

[14.4 10.4 Pull requests and merge mechanics](#)

[14.5 10.5 Code review: purpose and culture](#)

[14.6 10.6 Size and speed metrics](#)

[14.7 10.7 Release tags and semantic versioning](#)

[14.8 10.8 Handling hotfixes](#)

[14.9 10.9 Chapter summary](#)

[14.10 Lab 10.1 — Strategy comparison](#)

[14.11 Lab 10.2 — Mock code review](#)

[14.12 Review questions](#)

15 Chapter 11: GitHub/GitLab Workflows — PRs, Issues, Actions Intro

[15.1 11.1 Platform-native Git workflows](#)

[15.2 11.2 Issues and project hygiene](#)

[15.3 11.3 Pull request lifecycle \(GitHub\)](#)

[15.4 11.4 CODEOWNERS and review routing](#)

[15.5 11.5 GitHub Actions introduction](#)

[15.6 11.6 GitLab CI introduction](#)

[15.7 11.7 Branch protection and environments](#)
[15.8 11.8 Fork and open-source workflows](#)
[15.9 11.9 Integrations DevOps teams use](#)
[15.10 11.10 Comparing Actions vs GitLab CI \(practical\)](#)
[15.11 11.11 CLI automation: gh and glab](#)
[15.12 11.12 Chapter summary](#)
[15.13 Lab 11.1 — First GitHub Actions workflow](#)
[15.14 Lab 11.2 — GitLab mirror \(optional\)](#)
[15.15 Review questions](#)

[16 Chapter 12: OSI Model, TCP/IP, DNS Deep Dive](#)

[16.1 12.1 Why models matter during incidents](#)
[16.2 12.2 OSI seven layers \(reference\)](#)
[16.3 12.3 TCP/IP four-layer model \(operational\)](#)
[16.4 12.4 IP addressing and CIDR](#)
[16.5 12.5 TCP vs UDP](#)
[16.6 12.6 DNS: the distributed phone book](#)
[16.7 12.7 DNS operations DevOps engineers perform](#)
[16.8 12.8 ICMP and path discovery](#)
[16.9 12.9 ARP and L2 in VPC context](#)
[16.10 12.10 Common failure patterns](#)
[16.11 12.11 Chapter summary](#)
[16.12 Lab 12.1 — DNS TTL experiment](#)
[16.13 Lab 12.2 — Layer mapping drill](#)
[16.14 Review questions](#)

[17 Chapter 13: HTTP/HTTPS, TLS, and Certificates](#)

[17.1 13.1 HTTP: the application protocol of the web](#)
[17.2 13.2 HTTP/1.1 vs HTTP/2 vs HTTP/3](#)
[17.3 13.3 HTTPS and TLS](#)
[17.4 13.4 Certificates and PKI](#)
[17.5 13.5 mTLS \(mutual TLS\)](#)
[17.6 13.6 Cipher suites and hardening](#)
[17.7 13.7 HTTP headers DevOps should know](#)
[17.8 13.8 Timeouts, keep-alive, and load balancers](#)
[17.9 13.9 Debugging TLS in production](#)
[17.10 13.10 Chapter summary](#)
[17.11 Lab 13.1 — Inspect a live certificate](#)
[17.12 Lab 13.2 — Broken chain simulation](#)
[17.13 Review questions](#)

[18 Chapter 14: Load Balancing, Proxies, and CDN](#)

[18.1 14.1 Why load balancing exists](#)
[18.2 14.2 Layer 4 vs Layer 7 load balancing](#)
[18.3 14.3 Load balancing algorithms](#)
[18.4 14.4 Reverse proxies vs forward proxies](#)
[18.5 14.5 Common proxy software](#)
[18.6 14.6 Session persistence \(stickiness\)](#)
[18.7 14.7 Content Delivery Networks \(CDN\)](#)
[18.8 14.8 Global load balancing and failover](#)
[18.9 14.9 Kubernetes ingress pattern](#)
[18.10 14.10 Observability for load balancers](#)
[18.11 14.11 Chapter summary](#)
[18.12 Lab 14.1 — nginx upstream failover](#)
[18.13 Lab 14.2 — CDN cache headers](#)
[18.14 Review questions](#)

[19 Chapter 15: Cloud Networking Patterns — VPC Overview](#)

[19.1 15.1 VPC: your private network in the cloud](#)
[19.2 15.2 CIDR design fundamentals](#)
[19.3 15.3 Public vs private subnets](#)
[19.4 15.4 Security groups vs NACLs](#)

[19.5 15.5 NAT, egress, and VPC endpoints](#)
[19.6 15.6 Hybrid and multi-VPC connectivity](#)
[19.7 15.7 Kubernetes networking in VPC](#)
[19.8 15.8 IaC example \(Terraform sketch\)](#)
[19.9 15.9 Operational checklist](#)
[19.10 15.10 Chapter summary](#)
[19.11 Lab 15.1 — Paper VPC design](#)
[19.12 Lab 15.2 — Terraform mini-VPC \(optional cloud\)](#)
[19.13 Review questions](#)

20 Chapter 16: Python for DevOps — boto3, requests, pathlib

[20.1 16.1 Why Python dominates DevOps automation](#)
[20.2 16.2 Project layout and virtual environments](#)
[20.3 16.3 pathlib: filesystem without footguns](#)
[20.4 16.4 requests: HTTP for APIs and health checks](#)
[20.5 16.5 boto3: AWS automation](#)
[20.6 16.6 Idempotency and dry-run patterns](#)
[20.7 16.7 CLI entry points with argparse](#)
[20.8 16.8 Logging instead of print](#)
[20.9 16.9 Testing Python automation](#)
[20.10 16.10 Security notes](#)
[20.11 16.11 Chapter summary](#)
[20.12 Lab 16.1 — EC2 inventory script](#)
[20.13 Lab 16.2 — Health gate for CI](#)
[20.14 Review questions](#)

21 Chapter 17: Go for CLI Tools and Operators

[21.1 17.1 Why Go in the DevOps toolchain](#)
[21.2 17.2 Module setup](#)
[21.3 17.3 A minimal CLI with cobra \(pattern\)](#)
[21.4 17.4 Error handling and exit codes](#)
[21.5 17.5 Concurrency for parallel probes](#)
[21.6 17.6 Kubernetes operators \(conceptual\)](#)
[21.7 17.7 Testing Go CLIs](#)
[21.8 17.8 Release and distribution](#)
[21.9 17.9 When not to use Go](#)
[21.10 17.10 Chapter summary](#)
[21.11 Lab 17.1 — Multi-target HTTP checker](#)
[21.12 Lab 17.2 — Cross-compile release](#)
[21.13 Review questions](#)

22 Chapter 18: JSON/YAML, Jinja2, and Templating

[22.1 18.1 Structured data in DevOps](#)
[22.2 18.2 JSON essentials](#)
[22.3 18.3 YAML essentials and pitfalls](#)
[22.4 18.4 JSON vs YAML: when to use which](#)
[22.5 18.5 Templating concepts](#)
[22.6 18.6 Jinja2 in Ansible](#)
[22.7 18.7 Helm and Go templates \(brief\)](#)
[22.8 18.8 Schema validation](#)
[22.9 18.9 Secrets and templating](#)
[22.10 18.10 Chapter summary](#)
[22.11 Lab 18.1 — Render nginx from Jinja2](#)
[22.12 Lab 18.2 — YAML lint pipeline](#)
[22.13 Review questions](#)

23 Chapter 19: Testing Automation Scripts

[23.1 19.1 Why test automation code?](#)
[23.2 19.2 Test pyramid for DevOps scripts](#)
[23.3 19.3 Testing bash scripts](#)
[23.4 19.4 Testing Python automation](#)
[23.5 19.5 Testing Go CLIs](#)

[23.6 19.6 Integration testing with containers](#)

[23.7 19.7 Testing Terraform and Ansible](#)

[23.8 19.8 CI pipeline layout for script repos](#)

[23.9 19.9 Test data and fixtures](#)

[23.10 19.10 When tests are enough](#)

[23.11 19.11 Chapter summary](#)

[23.12 Lab 19.1 — pytest suite for inventory script](#)

[23.13 Lab 19.2 — bats for bash healthcheck](#)

[23.14 Review questions](#)

[24 Chapter 20: IaC Principles — State, Idempotency, and Drift](#)

[24.1 20.1 What is Infrastructure as Code?](#)

[24.2 20.2 Declarative vs imperative](#)

[24.3 20.3 Idempotency](#)

[24.4 20.4 State management](#)

[24.5 20.5 Configuration drift](#)

[24.6 20.6 IaC workflow in teams](#)

[24.7 20.7 Testing IaC](#)

[24.8 20.8 Chapter summary](#)

[24.9 Lab 20.1](#)

[25 Chapter 21: Terraform — HCL, Providers, and Modules](#)

[25.1 21.1 Terraform overview](#)

[25.2 21.2 Basic configuration](#)

[25.3 21.3 Resources, data sources, and dependencies](#)

[25.4 21.4 Modules — reusable infrastructure](#)

[25.5 21.5 Lifecycle meta-arguments](#)

[25.6 21.6 Workspaces and environments](#)

[25.7 21.7 CI/CD integration](#)

[25.8 21.8 Chapter summary](#)

[25.9 Lab 21.1](#)

[26 Chapter 22: Terraform — Workspaces, Backends, and CI Integration](#)

[26.1 22.1 Why remote state and backends matter](#)

[26.2 22.2 Configuring S3 backend with locking](#)

[26.3 22.3 Terraform workspaces](#)

[26.4 22.4 State operations and safety](#)

[26.5 22.5 CI/CD integration patterns](#)

[26.6 22.6 Atlantis and Terraform Cloud](#)

[26.7 22.7 Testing and policy in the pipeline](#)

[26.8 22.8 Chapter summary](#)

[26.9 Lab 22.1](#)

[26.10 Lab 22.2](#)

[26.11 Review questions](#)

[27 Chapter 23: Ansible — Playbooks, Roles, and Inventories](#)

[27.1 23.1 Ansible in the IaC landscape](#)

[27.2 23.2 Inventory — defining targets](#)

[27.3 23.3 Playbooks — ordered automation](#)

[27.4 23.4 Modules, idempotency, and handlers](#)

[27.5 23.5 Roles — reusable bundles](#)

[27.6 23.6 Variables and facts precedence](#)

[27.7 23.7 Vault, collections, and project structure](#)

[27.8 23.8 Ansible in CI/CD](#)

[27.9 23.9 Chapter summary](#)

[27.10 Lab 23.1](#)

[27.11 Lab 23.2](#)

[27.12 Review questions](#)

[28 Chapter 24: Packer & Golden Images](#)

[28.1 24.1 Golden images and the immutable infrastructure pattern](#)

[28.2 24.2 Packer workflow](#)

[28.3 24.3 AWS EBS builder example](#)

[28.4 24.4 Provisioners](#)

[28.5 24.5 Multi-region and shared images](#)

[28.6 24.6 Versioning, promotion, and CI](#)

[28.7 24.7 Security and compliance](#)

[28.8 24.8 Chapter summary](#)

[28.9 Lab 24.1](#)

[28.10 Lab 24.2](#)

[28.11 Review questions](#)

[29 Chapter 25: Policy as Code — OPA, Checkov, and Sentinel](#)

[29.1 25.1 Why policy as code?](#)

[29.2 25.2 Policy evaluation points](#)

[29.3 25.3 Checkov — static IaC scanning](#)

[29.4 25.4 Open Policy Agent \(OPA\) and Rego](#)

[29.5 25.5 HashiCorp Sentinel \(Terraform Cloud/Enterprise\)](#)

[29.6 25.6 Comparison matrix](#)

[29.7 25.7 Building a policy program](#)

[29.8 25.8 CI pipeline integration](#)

[29.9 25.9 Chapter summary](#)

[29.10 Lab 25.1](#)

[29.11 Lab 25.2](#)

[29.12 Review questions](#)

[30 Chapter 26: Docker — Images, Dockerfile, and Multi-stage Builds](#)

[30.1 26.1 Containers in one paragraph](#)

[30.2 26.2 Core concepts](#)

[30.3 26.3 Essential commands](#)

[30.4 26.4 Dockerfile best practices](#)

[30.5 26.5 Docker Compose for local development](#)

[30.6 26.6 Security basics](#)

[30.7 26.7 Chapter summary](#)

[30.8 Lab 26.1](#)

[31 Chapter 27: Container Networking, Volumes, and Security](#)

[31.1 27.1 Container networking model](#)

[31.2 27.2 Docker network drivers](#)

[31.3 27.3 Container-to-container and host communication](#)

[31.4 27.4 Volumes and persistence](#)

[31.5 27.5 Linux capabilities and seccomp](#)

[31.6 27.6 Image and runtime security](#)

[31.7 27.7 Resource limits and health](#)

[31.8 27.8 Debugging networking issues](#)

[31.9 27.9 Chapter summary](#)

[31.10 Lab 27.1](#)

[31.11 Lab 27.2](#)

[31.12 Review questions](#)

[32 Chapter 28: Docker Compose and Local Dev Environments](#)

[32.1 28.1 What Docker Compose solves](#)

[32.2 28.2 Compose file structure](#)

[32.3 28.3 Essential commands](#)

[32.4 28.4 Environment-specific overrides](#)

[32.5 28.5 Networking in Compose](#)

[32.6 28.6 Development workflows](#)

[32.7 28.7 Testing with Compose](#)

[32.8 28.8 Compose vs production](#)

[32.9 28.9 Chapter summary](#)

[32.10 Lab 28.1](#)

[32.11 Lab 28.2](#)

[32.12 Review questions](#)

[33 Chapter 29: Container Registries and Image Signing](#)

[33.1 29.1 Container registries overview](#)

[33.2 29.2 Image naming and tagging strategy](#)
[33.3 29.3 Authenticating to registries](#)
[33.4 29.4 Pull policies in Kubernetes and beyond](#)
[33.5 29.5 Vulnerability scanning](#)
[33.6 29.6 Image signing and verification](#)
[33.7 29.7 Registry operations](#)
[33.8 29.8 Supply chain best practices](#)
[33.9 29.9 Chapter summary](#)
[33.10 Lab 29.1](#)
[33.11 Lab 29.2](#)
[33.12 Review questions](#)

[34 Chapter 30: Rootless Containers and Podman](#)

[34.1 30.1 Why rootless containers matter](#)
[34.2 30.2 Podman overview](#)
[34.3 30.3 Rootless setup](#)
[34.4 30.4 Pods and systemd integration](#)
[34.5 30.5 Podman Compose and Kubernetes YAML](#)
[34.6 30.6 Rootless Docker \(Moby\)](#)
[34.7 30.7 Security comparison](#)
[34.8 30.8 Buildah and Skopeo](#)
[34.9 30.9 When to use Podman vs Docker vs Kubernetes](#)
[34.10 30.10 Chapter summary](#)
[34.11 Lab 30.1](#)
[34.12 Lab 30.2](#)
[34.13 Review questions](#)

[35 Chapter 31: Kubernetes Architecture — Control Plane and Node Components](#)

[35.1 31.1 What Kubernetes provides](#)
[35.2 31.2 Cluster architecture overview](#)
[35.3 31.3 Control plane components](#)
[35.4 31.4 Node components](#)
[35.5 31.5 Kubernetes objects and namespaces](#)
[35.6 31.6 kubectl and kubeconfig](#)
[35.7 31.7 Managed vs self-managed clusters](#)
[35.8 31.8 Addons and the CNCF landscape](#)
[35.9 31.9 High availability and etcd sizing](#)
[35.10 31.10 Chapter summary](#)
[35.11 Lab 31.1](#)
[35.12 Lab 31.2](#)
[35.13 Review questions](#)

[36 Chapter 32: Kubernetes Workloads — Pod, Deployment, StatefulSet, DaemonSet](#)

[36.1 32.1 Pods — the atomic unit](#)
[36.2 32.2 Deployments — stateless apps](#)
[36.3 32.3 ReplicaSets and labels](#)
[36.4 32.4 StatefulSets — stable identity and storage](#)
[36.5 32.5 DaemonSets — one pod per node](#)
[36.6 32.6 Jobs and CronJobs](#)
[36.7 32.7 Choosing a workload type](#)
[36.8 32.8 Pod scheduling controls](#)
[36.9 32.9 Chapter summary](#)
[36.10 Lab 32.1](#)
[36.11 Lab 32.2](#)
[36.12 Review questions](#)

[37 Chapter 33: Kubernetes Services, Ingress, and Network Policies](#)

[37.1 33.1 Kubernetes networking model](#)
[37.2 33.2 Service types](#)
[37.3 33.3 Endpoints and EndpointSlices](#)
[37.4 33.4 Ingress — HTTP routing](#)
[37.5 33.5 LoadBalancer and NodePort in cloud](#)

[37.6 33.6 Gateway API \(modern alternative\)](#)

[37.7 33.7 Network Policies — zero trust inside the cluster](#)

[37.8 33.8 Debugging connectivity](#)

[37.9 33.9 Chapter summary](#)

[37.10 Lab 33.1](#)

[37.11 Lab 33.2](#)

[37.12 Review questions](#)

[38 Chapter 34: Kubernetes ConfigMaps, Secrets, and Storage \(PV/PVC\)](#)

[38.1 34.1 Separating config from images](#)

[38.2 34.2 ConfigMaps](#)

[38.3 34.3 Secrets](#)

[38.4 34.4 Storage basics — PV, PVC, StorageClass](#)

[38.5 34.5 StatefulSet storage pattern](#)

[38.6 34.6 Config reload and updates](#)

[38.7 34.7 Backup and snapshots](#)

[38.8 34.8 Security checklist](#)

[38.9 34.9 Chapter summary](#)

[38.10 Lab 34.1](#)

[38.11 Lab 34.2](#)

[38.12 Review questions](#)

[39 Chapter 35: Helm, Kustomize, and GitOps \(Argo CD\)](#)

[39.1 35.1 The packaging problem](#)

[39.2 35.2 Helm charts](#)

[39.3 35.3 Helm hooks and releases](#)

[39.4 35.4 Kustomize — patch without templates](#)

[39.5 35.5 GitOps principles](#)

[39.6 35.6 Argo CD](#)

[39.7 35.7 Promotion pipeline](#)

[39.8 35.8 Flux \(brief comparison\)](#)

[39.9 35.9 Secrets in GitOps](#)

[39.10 35.10 Chapter summary](#)

[39.11 Lab 35.1](#)

[39.12 Lab 35.2](#)

[39.13 Review questions](#)

[40 Chapter 36: Kubernetes RBAC, Pod Security, and Hardening](#)

[40.1 36.1 Defense in depth for Kubernetes](#)

[40.2 36.2 Authentication overview](#)

[40.3 36.3 RBAC — Roles and Bindings](#)

[40.4 36.4 Aggregated roles and default bindings](#)

[40.5 36.5 Pod Security Admission \(PSA\)](#)

[40.6 36.6 OPA Gatekeeper and Kyverno](#)

[40.7 36.7 Workload hardening checklist](#)

[40.8 36.8 Secrets, audit logs, and API security](#)

[40.9 36.9 Node and supply chain security](#)

[40.10 36.10 Incident response primitives](#)

[40.11 36.11 Chapter summary](#)

[40.12 Lab 36.1](#)

[40.13 Lab 36.2](#)

[40.14 Review questions](#)

[41 Chapter 37: CI/CD Concepts and Pipeline Design](#)

[41.1 37.1 Continuous Integration \(CI\)](#)

[41.2 37.2 Continuous Delivery vs Continuous Deployment](#)

[41.3 37.3 Pipeline stages](#)

[41.4 37.4 GitHub Actions example](#)

[41.5 37.5 Branching and trunk-based development](#)

[41.6 37.6 Deployment strategies](#)

[41.7 37.7 Pipeline secrets and artifacts](#)

[41.8 37.8 DORA metrics in your pipeline](#)

[41.9 37.9 Chapter summary](#)

[41.10 Lab 37.1](#)

[42 Chapter 38: GitHub Actions — Workflows, Secrets, Runners](#)

[42.1 38.1 GitHub Actions architecture](#)

[42.2 38.2 Workflow syntax fundamentals](#)

[42.3 38.3 Reusable workflows and composite actions](#)

[42.4 38.4 Secrets, variables, and security](#)

[42.5 38.5 Runners: hosted vs self-hosted](#)

[42.6 38.6 Matrix builds, caching, and artifacts](#)

[42.7 38.7 Environments, deployments, and approvals](#)

[42.8 38.8 Debugging and troubleshooting](#)

[42.9 38.9 Chapter summary](#)

[42.10 Lab 38.1 — Multi-environment deploy pipeline](#)

[42.11 Lab 38.2 — Self-hosted runner in Docker](#)

[42.12 Review questions](#)

[43 Chapter 39: Jenkins Pipelines as Code](#)

[43.1 39.1 Jenkins in the modern CI/CD landscape](#)

[43.2 39.2 Architecture: controller, agents, and plugins](#)

[43.3 39.3 Declarative Pipeline structure](#)

[43.4 39.4 Agents and parallelism](#)

[43.5 39.5 Credentials, parameters, and shared libraries](#)

[43.6 39.6 Multibranch and organization folders](#)

[43.7 39.7 Pipeline durability and best practices](#)

[43.8 39.8 Jenkins vs cloud-native CI \(when to stay\)](#)

[43.9 39.9 Chapter summary](#)

[43.10 Lab 39.1 — Declarative pipeline from scratch](#)

[43.11 Lab 39.2 — Kubernetes ephemeral agents](#)

[43.12 Review questions](#)

[44 Chapter 40: GitLab CI and Multi-platform Comparison](#)

[44.1 40.1 GitLab CI/CD overview](#)

[44.2 40.2 .gitlab-ci.yml fundamentals](#)

[44.3 40.3 Runners, executors, and isolation](#)

[44.4 40.4 Variables, environments, and child pipelines](#)

[44.5 40.5 GitLab-specific features](#)

[44.6 40.6 Multi-platform CI/CD comparison](#)

[44.7 40.7 Portable pipeline patterns](#)

[44.8 40.8 Migration considerations](#)

[44.9 40.9 Chapter summary](#)

[44.10 Lab 40.1 — GitLab pipeline with services](#)

[44.11 Lab 40.2 — Platform comparison exercise](#)

[44.12 Review questions](#)

[45 Chapter 41: Deployment Strategies — Blue/Green, Canary, Feature Flags](#)

[45.1 41.1 Why deployment strategy matters](#)

[45.2 41.2 Rolling deployment](#)

[45.3 41.3 Recreate deployment](#)

[45.4 41.4 Blue/green deployment](#)

[45.5 41.5 Canary deployment](#)

[45.6 41.6 Feature flags \(dark launches\)](#)

[45.7 41.7 Database migrations and deployment coupling](#)

[45.8 41.8 Choosing a strategy](#)

[45.9 41.9 Chapter summary](#)

[45.10 Lab 41.1 — Kubernetes rolling vs recreate](#)

[45.11 Lab 41.2 — Feature flag rollout](#)

[45.12 Review questions](#)

[46 Chapter 42: DORA Metrics and Continuous Improvement](#)

[46.1 42.1 What DORA measures](#)

[46.2 42.2 Why these four metrics matter](#)

[46.3 42.3 Measuring deployment frequency](#)

[46.4 42.4 Measuring lead time for changes](#)
[46.5 42.5 Change failure rate](#)
[46.6 42.6 Time to restore service \(MTTR\)](#)
[46.7 42.7 Performance clusters and benchmarking](#)
[46.8 42.8 Building a metrics program](#)
[46.9 42.9 Continuous improvement loops](#)
[46.10 42.10 Chapter summary](#)
[46.11 Lab 42.1 — Deployment event pipeline](#)
[46.12 Lab 42.2 — Lead time value stream map](#)
[46.13 Review questions](#)

47 Chapter 43: Monitoring Fundamentals — Metrics, Logs, Traces

[47.1 43.1 From monitoring to observability](#)
[47.2 43.2 Metrics: types and best practices](#)
[47.3 43.3 Logs: structure and aggregation](#)
[47.4 43.4 Traces: distributed request flow](#)
[47.5 43.5 The observability data pipeline](#)
[47.6 43.6 Alerting philosophy](#)
[47.7 43.7 Golden signals and SLIs](#)
[47.8 43.8 Instrumentation ownership](#)
[47.9 43.9 Chapter summary](#)
[47.10 Lab 43.1 — RED metrics for a sample API](#)
[47.11 Lab 43.2 — Correlate logs and traces](#)
[47.12 Review questions](#)

48 Chapter 44: Prometheus & Grafana Stack

[48.1 44.1 The Prometheus ecosystem](#)
[48.2 44.2 Prometheus architecture](#)
[48.3 44.3 Prometheus configuration](#)
[48.4 44.4 PromQL essentials](#)
[48.5 44.5 Alerting rules and Alertmanager](#)
[48.6 44.6 Grafana dashboards](#)
[48.7 44.7 Kubernetes deployment patterns](#)
[48.8 44.8 Exporters and instrumentation](#)
[48.9 44.9 Operations and troubleshooting](#)
[48.10 44.10 Chapter summary](#)
[48.11 Lab 44.1 — Local Prometheus + Grafana](#)
[48.12 Lab 44.2 — ServiceMonitor on Kubernetes](#)
[48.13 Review questions](#)

49 Chapter 45: ELK/EFK Log Aggregation

[49.1 45.1 Centralized logging rationale](#)
[49.2 45.2 Architecture overview](#)
[49.3 45.3 Elasticsearch fundamentals](#)
[49.4 45.4 Fluent Bit vs Logstash vs Filebeat](#)
[49.5 45.5 Logstash pipelines](#)
[49.6 45.6 Kibana for search and dashboards](#)
[49.7 45.7 EFK on Kubernetes \(ECK\)](#)
[49.8 45.8 Security and compliance](#)
[49.9 45.9 Loki as an alternative](#)
[49.10 45.10 Operational best practices](#)
[49.11 45.11 Chapter summary](#)
[49.12 Lab 45.1 — Docker Compose ELK](#)
[49.13 Lab 45.2 — Fluent Bit on Kubernetes](#)
[49.14 Review questions](#)

50 Chapter 46: OpenTelemetry Distributed Tracing

[50.1 46.1 The tracing problem](#)
[50.2 46.2 Traces, spans, and context propagation](#)
[50.3 46.3 OpenTelemetry architecture](#)
[50.4 46.4 Instrumentation: auto vs manual](#)
[50.5 46.5 OTel Collector configuration](#)

[50.6 46.6 Sampling strategies](#)

[50.7 46.7 Backends: Jaeger and Grafana Tempo](#)

[50.8 46.8 Correlating traces, logs, and metrics](#)

[50.9 46.9 Service mesh tracing](#)

[50.10 46.10 Adoption roadmap](#)

[50.11 46.11 Chapter summary](#)

[50.12 Lab 46.1 — OTEL + Jaeger locally](#)

[50.13 Lab 46.2 — Tail sampling](#)

[50.14 Review questions](#)

[51 Chapter 47: SLOs, Error Budgets, and Incident Response](#)

[51.1 47.1 Reliability as a product decision](#)

[51.2 47.2 Choosing SLIs and SLOs](#)

[51.3 47.3 Error budgets in practice](#)

[51.4 47.4 Incident severity and lifecycle](#)

[51.5 47.5 On-call and escalation](#)

[51.6 47.6 Incident response procedures](#)

[51.7 47.7 Blameless postmortems](#)

[51.8 47.8 Status communication](#)

[51.9 47.9 Tooling integration](#)

[51.10 47.10 Chapter summary](#)

[51.11 Lab 47.1 — Define SLOs for a sample service](#)

[51.12 Lab 47.2 — Tabletop incident exercise](#)

[51.13 Review questions](#)

[52 Chapter 48: Chaos Engineering and Game Days](#)

[52.1 48.1 Proactive resilience testing](#)

[52.2 48.2 Principles of chaos engineering](#)

[52.3 48.3 Failure modes to inject](#)

[52.4 48.4 Chaos tools landscape](#)

[52.5 48.5 Safe experimentation guardrails](#)

[52.6 48.6 Game days](#)

[52.7 48.7 From experiment to improvement](#)

[52.8 48.8 Organizational maturity](#)

[52.9 48.9 Chapter summary](#)

[52.10 Lab 48.1 — Pod delete experiment](#)

[52.11 Lab 48.2 — Mini game day](#)

[52.12 Review questions](#)

[53 Chapter 49: Shift-Left Security in the Pipeline](#)

[53.1 49.1 DevSecOps and shift-left](#)

[53.2 49.2 Security activities across the SDLC](#)

[53.3 49.3 Threat modeling fundamentals](#)

[53.4 49.4 Pipeline security gates](#)

[53.5 49.5 Secret scanning](#)

[53.6 49.6 Secure defaults and platform guardrails](#)

[53.7 49.7 Security champions program](#)

[53.8 49.8 Compliance integration](#)

[53.9 49.9 Culture and psychological safety](#)

[53.10 49.10 Chapter summary](#)

[53.11 Lab 49.1 — Pre-commit security stack](#)

[53.12 Lab 49.2 — Threat model exercise](#)

[53.13 Review questions](#)

[54 Chapter 50: SAST, DAST, SCA, and Container Scanning](#)

[54.1 50.1 Application security testing taxonomy](#)

[54.2 50.2 SAST — static analysis](#)

[54.3 50.3 DAST — dynamic testing](#)

[54.4 50.4 SCA — dependency analysis](#)

[54.5 50.5 Container and image scanning](#)

[54.6 50.6 IaC and Kubernetes scanning](#)

[54.7 50.7 Unified vulnerability management](#)

[54.8 50.8 Reducing noise](#)

[54.9 50.9 Chapter summary](#)

[54.10 Lab 50.1 — Full scan pipeline](#)

[54.11 Lab 50.2 — DAST against staging](#)

[54.12 Review questions](#)

[55 Chapter 51: Secrets Management — Vault, SOPS, External Secrets](#)

[55.1 51.1 The secrets problem](#)

[55.2 51.2 HashiCorp Vault architecture](#)

[55.3 51.3 Vault on Kubernetes](#)

[55.4 51.4 Mozilla SOPS — Git-friendly encryption](#)

[55.5 51.5 External Secrets Operator \(ESO\)](#)

[55.6 51.6 Cloud-native secret managers](#)

[55.7 51.7 CI/CD secrets patterns](#)

[55.8 51.8 Rotation and emergency response](#)

[55.9 51.9 Chapter summary](#)

[55.10 Lab 51.1 — SOPS with age](#)

[55.11 Lab 51.2 — External Secrets on Kubernetes](#)

[55.12 Review questions](#)

[56 Chapter 52: Supply Chain Security — SBOM, Sigstore, SLSA](#)

[56.1 52.1 Software supply chain threats](#)

[56.2 52.2 SBOM — Software Bill of Materials](#)

[56.3 52.3 SLSA — supply chain levels](#)

[56.4 52.4 Sigstore and cosign](#)

[56.5 52.5 Dependency and build hardening](#)

[56.6 52.6 in-toto and attestations](#)

[56.7 52.7 Organizational supply chain program](#)

[56.8 52.8 Chapter summary](#)

[56.9 Lab 52.1 — SBOM generation and CVE lookup](#)

[56.10 Lab 52.2 — Sign and verify with cosign](#)

[56.11 Review questions](#)

[57 Chapter 53: Runtime Security — Falco and OPA Gatekeeper](#)

[57.1 53.1 Defense beyond the pipeline](#)

[57.2 53.2 Falco — cloud-native runtime detection](#)

[57.3 53.3 Falco response actions](#)

[57.4 53.4 OPA and Gatekeeper](#)

[57.5 53.5 Policy libraries and Pod Security](#)

[57.6 53.6 Runtime security program](#)

[57.7 53.7 Chapter summary](#)

[57.8 Lab 53.1 — Falco shell detection](#)

[57.9 Lab 53.2 — Gatekeeper privileged block](#)

[57.10 Review questions](#)

[58 Chapter 54: Compliance Automation and Audit Trails](#)

[58.1 54.1 Compliance in continuous delivery](#)

[58.2 54.2 Control frameworks mapping](#)

[58.3 54.3 Audit trails — what to log](#)

[58.4 54.4 Policy as Code for compliance](#)

[58.5 54.5 Continuous control monitoring](#)

[58.6 54.6 Change management automation](#)

[58.7 54.7 Data privacy and retention](#)

[58.8 54.8 Audit preparation workflow](#)

[58.9 54.9 Chapter summary](#)

[58.10 Lab 54.1 — Kubernetes audit logging](#)

[58.11 Lab 54.2 — Compliance evidence pipeline](#)

[58.12 Review questions](#)

[59 Chapter 55: Platform Engineering and Internal Developer Platforms](#)

[59.1 55.1 From DevOps to platform engineering](#)

[59.2 55.2 Internal Developer Platform components](#)

[59.3 55.3 Backstage — reference IDP portal](#)

[59.4 55.4 Golden paths and paved roads](#)

[59.5 55.5 Platform as a product](#)

[59.6 55.6 Infrastructure abstractions](#)

[59.7 55.7 Developer experience metrics](#)

[59.8 55.8 Anti-patterns](#)

[59.9 55.9 Chapter summary](#)

[59.10 Lab 55.1 — Backstage software catalog](#)

[59.11 Lab 55.2 — Golden path template](#)

[59.12 Review questions](#)

[60 Chapter 56: Multi-Cloud and Hybrid Patterns](#)

[60.1 56.1 Why multi-cloud and hybrid exist](#)

[60.2 56.2 Architecture patterns](#)

[60.3 56.3 Portability strategies](#)

[60.4 56.4 Networking across clouds and on-prem](#)

[60.5 56.5 GitOps and multi-cluster management](#)

[60.6 56.6 Data and state in multi-cloud](#)

[60.7 56.7 Operational model](#)

[60.8 56.8 When NOT to multi-cloud](#)

[60.9 56.9 Chapter summary](#)

[60.10 Lab 56.1 — Terraform multi-provider module](#)

[60.11 Lab 56.2 — Argo CD ApplicationSet](#)

[60.12 Review questions](#)

[61 Chapter 57: FinOps and Cost-Aware Engineering](#)

[61.1 57.1 Cloud cost as engineering concern](#)

[61.2 57.2 Cost visibility and allocation](#)

[61.3 57.3 Right-sizing and waste elimination](#)

[61.4 57.4 Pricing models and savings](#)

[61.5 57.5 Cost in CI/CD and IaC](#)

[61.6 57.6 Unit economics](#)

[61.7 57.7 FinOps maturity stages](#)

[61.8 57.8 Chapter summary](#)

[61.9 Lab 57.1 — Cost allocation tags](#)

[61.10 Lab 57.2 — Infracost in CI](#)

[61.11 Review questions](#)

[62 Chapter 58: MLOps Overview for DevOps Engineers](#)

[62.1 58.1 Why DevOps engineers need MLOps](#)

[62.2 58.2 ML lifecycle vs traditional SDLC](#)

[62.3 58.3 Key MLOps concepts](#)

[62.4 58.4 Reproducible training pipeline](#)

[62.5 58.5 Model deployment patterns](#)

[62.6 58.6 Feature stores and training-serving skew](#)

[62.7 58.7 Monitoring ML in production](#)

[62.8 58.8 Governance and responsible ML](#)

[62.9 58.9 DevOps + MLOps collaboration](#)

[62.10 58.10 Chapter summary](#)

[62.11 Lab 58.1 — MLflow end-to-end](#)

[62.12 Lab 58.2 — Model monitoring dashboard](#)

[62.13 Review questions](#)

[63 Chapter 59: Capstone — End-to-End Production System Design](#)

[63.1 59.1 Capstone objectives](#)

[63.2 59.2 Architecture overview](#)

[63.3 59.3 Repository and team structure](#)

[63.4 59.4 CI/CD pipeline design](#)

[63.5 59.5 Infrastructure as Code](#)

[63.6 59.6 Security architecture](#)

[63.7 59.7 Observability and SRE](#)

[63.8 59.8 Data, DR, and scaling](#)

[63.9 59.9 FinOps and platform metrics](#)

[63.10 59.10 Capstone delivery roadmap](#)

[63.11 59.11 Design review checklist](#)

[63.12 59.12 Chapter summary](#)

[63.13 Capstone Lab — Mini ShopStream](#)

[63.14 Review questions](#)

[64 Chapter 60: Appendix — Cheatsheets, Glossary, and Index](#)

[64.1 60.1 kubectl quick reference](#)

[64.2 60.2 Docker quick reference](#)

[64.3 60.3 Terraform quick reference](#)

[64.4 60.4 Git essentials](#)

[64.5 60.5 Linux troubleshooting](#)

[64.6 60.6 PromQL cheatsheet](#)

[64.7 60.7 CI/CD YAML snippets](#)

[64.8 60.8 Security tool commands](#)

[64.9 60.9 Observability checklist](#)

[64.10 60.10 SLO error budget reference](#)

[64.11 60.11 Glossary](#)

[64.12 60.12 Handbook chapter index](#)

[64.13 60.13 Recommended reading](#)

[64.14 60.14 Certification paths \(optional\)](#)

[64.15 60.15 Final review questions \(handbook-wide\)](#)

[64.16 60.16 Chapter summary](#)

[65 AWS Handbook — Master Table of Contents](#)

[65.1 Part I — Cloud Foundations \(Pages 1–80\)](#)

[65.2 Part II — Identity & Access Management \(Pages 81–160\)](#)

[65.3 Part III — Networking \(Pages 161–280\)](#)

[65.4 Part IV — Compute \(Pages 281–400\)](#)

[65.5 Part V — Storage \(Pages 401–480\)](#)

[65.6 Part VI — Databases \(Pages 481–560\)](#)

[65.7 Part VII — Application Integration \(Pages 561–620\)](#)

[65.8 Part VIII — DevOps on AWS \(Pages 621–700\)](#)

[65.9 Part IX — Observability & Operations \(Pages 701–760\)](#)

[65.10 Part X — Security & Compliance \(Pages 761–800\)](#)

[65.11 Expansion status](#)

[66 Chapter 1: Introduction to AWS & the Shared Responsibility Model](#)

[66.1 1.1 What is Amazon Web Services \(AWS\)?](#)

[66.2 1.2 Cloud service models](#)

[66.3 1.3 The Shared Responsibility Model](#)

[66.4 1.4 Core AWS concepts](#)

[66.5 1.5 AWS console, CLI, and IaC](#)

[66.6 1.6 Pricing fundamentals](#)

[66.7 1.7 Well-Architected Framework \(preview\)](#)

[66.8 1.8 Chapter summary](#)

[66.9 Lab 1.1 — Create your AWS account](#)

[66.10 Review questions](#)

[67 Chapter 2: AWS Global Infrastructure — Regions, AZs, and Edge](#)

[67.1 2.1 Global infrastructure overview](#)

[67.2 2.2 Regions](#)

[67.3 2.3 Availability Zones \(AZs\)](#)

[67.4 2.4 Local Zones, Wavelength, and Outposts](#)

[67.5 2.5 Edge locations and CloudFront](#)

[67.6 2.6 Fault isolation design principles](#)

[67.7 2.7 Service quotas \(limits\)](#)

[67.8 2.8 Chapter summary](#)

[67.9 Lab 2.1 — Explore regions in CLI](#)

[67.10 Review questions](#)

[68 Chapter 3: AWS Account Setup, Billing, and Cost Basics](#)

[68.1 3.1 Creating a secure AWS account](#)

[68.2 3.2 AWS Free Tier](#)

[68.3 3.3 Billing and Cost Management](#)

[68.4 3.4 Cost optimization fundamentals](#)

[68.5 3.4 Consolidated billing with Organizations](#)

[68.6 3.5 Support plans \(overview\)](#)

[68.7 3.6 Chapter summary](#)

[68.8 Lab 3.1](#)

[69 Chapter 4: AWS CLI, CloudShell, and SDK Overview](#)

[69.1 4.1 Why the CLI matters for DevOps](#)

[69.2 4.2 Installing and configuring the AWS CLI](#)

[69.3 4.3 AWS CloudShell](#)

[69.4 4.4 CLI essentials: structure and patterns](#)

[69.5 4.5 AWS SDK overview](#)

[69.6 4.6 Terraform provider configuration](#)

[69.7 4.7 CLI security best practices](#)

[69.8 4.8 Useful CLI productivity tips](#)

[69.9 4.9 Chapter summary](#)

[69.10 Lab 4.1 — CLI and CloudShell workflow](#)

[69.11 Lab 4.2 — Boto3 script](#)

[69.12 Review questions](#)

[70 Chapter 5: IAM Users, Groups, Roles, and Policies](#)

[70.1 5.1 What is IAM?](#)

[70.2 5.2 IAM identity types](#)

[70.3 5.3 Policies — the core of permissions](#)

[70.4 5.4 Users and groups](#)

[70.5 5.5 Roles and trust policies](#)

[70.6 5.6 AWS managed vs customer managed policies](#)

[70.7 5.7 Least privilege in practice](#)

[70.8 5.8 Common IAM actions for DevOps](#)

[70.9 5.9 IAM Access Analyzer & CloudTrail](#)

[70.10 5.10 Chapter summary](#)

[70.11 Lab 5.1 — IAM hands-on](#)

[70.12 Review questions](#)

[71 Chapter 6: IAM Best Practices and Permission Boundaries](#)

[71.1 6.1 The IAM security baseline](#)

[71.2 6.2 Prefer roles over users for applications](#)

[71.3 6.3 Permission boundaries](#)

[71.4 6.4 Policy writing tips](#)

[71.5 6.5 Service-linked roles](#)

[71.6 6.6 Cross-account access](#)

[71.7 6.7 Chapter summary](#)

[71.8 Lab 6.1](#)

[72 Chapter 7: IAM Identity Center \(SSO\) & Federation](#)

[72.1 7.1 The problem with IAM users at scale](#)

[72.2 7.2 IAM Identity Center architecture](#)

[72.3 7.3 Permission sets](#)

[72.4 7.4 Federation with external identity providers](#)

[72.5 7.5 Accessing AWS with Identity Center](#)

[72.6 7.6 IAM federation without Identity Center](#)

[72.7 7.7 Multi-account access patterns](#)

[72.8 7.8 Security best practices](#)

[72.9 7.9 Chapter summary](#)

[72.10 Lab 7.1 — Identity Center setup](#)

[72.11 Lab 7.2 — Audit permission assignments](#)

[72.12 Review questions](#)

[73 Chapter 8: AWS Organizations & Service Control Policies](#)

[73.1 8.1 Why AWS Organizations?](#)

[73.2 8.2 Organization structure](#)

[73.3 8.3 Consolidated billing](#)

[73.4 8.4 Service Control Policies \(SCPs\)](#)

[73.5 8.5 Common SCP strategies](#)

[73.6 8.6 Account vending with Control Tower](#)

[73.7 8.7 Delegated administration](#)

[73.8 8.8 SCP troubleshooting](#)

[73.9 8.9 Chapter summary](#)

[73.10 Lab 8.1 — Explore your organization](#)

[73.11 Lab 8.2 — Create a sandbox SCP](#)

[73.12 Review questions](#)

[74 Chapter 9: VPC Fundamentals — CIDR, Subnets, and Route Tables](#)

[74.1 9.1 What is a VPC?](#)

[74.2 9.2 CIDR and IP planning](#)

[74.3 9.3 Subnets](#)

[74.4 9.4 Route tables](#)

[74.5 9.5 Internet Gateway and NAT \(preview\)](#)

[74.6 9.6 Security Groups vs NACLs](#)

[74.7 9.7 Terraform VPC snippet](#)

[74.8 9.8 Chapter summary](#)

[74.9 Lab 9.1](#)

[75 Chapter 10: Internet Gateway, NAT Gateway, and Routing](#)

[75.1 10.1 Internet Gateway \(IGW\)](#)

[75.2 10.2 NAT Gateway](#)

[75.3 10.3 VPC Endpoints — avoid NAT costs](#)

[75.4 10.4 Elastic IP \(EIP\)](#)

[75.5 10.5 Troubleshooting connectivity](#)

[75.6 10.6 High availability networking](#)

[75.7 10.7 Chapter summary](#)

[75.8 Lab 10.1](#)

[76 Chapter 11: Security Groups vs Network ACLs](#)

[76.1 11.1 Two layers of VPC security](#)

[76.2 11.2 Security groups in depth](#)

[76.3 11.3 Network ACLs in depth](#)

[76.4 11.4 Security groups vs NACLs — comparison](#)

[76.5 11.5 Defense in depth pattern](#)

[76.6 11.6 Common mistakes and troubleshooting](#)

[76.7 11.7 Security group limits and quotas](#)

[76.8 11.8 Chapter summary](#)

[76.9 Lab 11.1 — Security group tiers](#)

[76.10 Lab 11.2 — NACL deny rule](#)

[76.11 Review questions](#)

[77 Chapter 12: Load Balancers — ALB, NLB, and GWLB](#)

[77.1 12.1 Why load balancers?](#)

[77.2 12.2 Load balancer comparison](#)

[77.3 12.3 Application Load Balancer \(ALB\)](#)

[77.4 12.4 Network Load Balancer \(NLB\)](#)

[77.5 12.5 Gateway Load Balancer \(GWLB\)](#)

[77.6 12.6 Shared ELB concepts](#)

[77.7 12.7 SSL/TLS with load balancers](#)

[77.8 12.8 Monitoring and troubleshooting](#)

[77.9 12.9 Chapter summary](#)

[77.10 Lab 12.1 — Deploy an ALB](#)

[77.11 Lab 12.2 — NLB with static IP](#)

[77.12 Review questions](#)

[78 Chapter 13: Route 53 — DNS Records and Routing Policies](#)

[78.1 13.1 What is Amazon Route 53?](#)

[78.2 13.2 Core DNS concepts](#)

[78.3 13.3 DNS record types](#)

[78.4 13.4 Routing policies](#)

[78.5 13.5 Health checks](#)

[78.6 13.6 Private DNS and hybrid resolution](#)

[78.7 13.7 Route 53 with other AWS services](#)

[78.8 13.8 DNS security](#)

[78.9 13.9 Chapter summary](#)

[78.10 Lab 13.1 — Public hosted zone](#)

[78.11 Lab 13.2 — Weighted routing](#)

[78.12 Lab 13.3 — Failover routing](#)

[78.13 Review questions](#)

[79 Chapter 14: VPC Peering, Transit Gateway, and PrivateLink](#)

[79.1 14.1 Connecting VPCs and services](#)

[79.2 14.2 VPC Peering](#)

[79.3 14.3 AWS Transit Gateway](#)

[79.4 14.4 AWS PrivateLink](#)

[79.5 14.5 Choosing the right connectivity](#)

[79.6 14.6 Security considerations](#)

[79.7 14.7 Chapter summary](#)

[79.8 Lab 14.1 — VPC Peering](#)

[79.9 Lab 14.2 — S3 Gateway Endpoint](#)

[79.10 Lab 14.3 — Transit Gateway \(advanced\)](#)

[79.11 Review questions](#)

[80 Chapter 15: EC2 — Instance Types, AMIs, and Launch Templates](#)

[80.1 15.1 Amazon EC2 overview](#)

[80.2 15.2 Instance families](#)

[80.3 15.3 AMIs and launch templates](#)

[80.4 15.4 User data — bootstrap scripts](#)

[80.5 15.5 Auto Scaling Groups \(ASG\)](#)

[80.6 15.6 EC2 best practices](#)

[80.7 15.7 Chapter summary](#)

[80.8 Lab 15.1](#)

[81 Chapter 16: Auto Scaling & Elastic Load Balancing Integration](#)

[81.1 16.1 Why Auto Scaling?](#)

[81.2 16.2 Auto Scaling components](#)

[81.3 16.3 Launch templates vs launch configurations](#)

[81.4 16.4 Auto Scaling group with ALB](#)

[81.5 16.5 Scaling policies](#)

[81.6 16.6 Instance refresh](#)

[81.7 16.7 Lifecycle hooks](#)

[81.8 16.8 Mixed instances and Spot](#)

[81.9 16.9 Warm pools](#)

[81.10 16.10 Monitoring and troubleshooting](#)

[81.11 16.11 Chapter summary](#)

[81.12 Lab 16.1 — ASG with ALB](#)

[81.13 Lab 16.2 — Instance refresh](#)

[81.14 Review questions](#)

[82 Chapter 17: Lambda — Events, Concurrency, and Layers](#)

[82.1 17.1 What is AWS Lambda?](#)

[82.2 17.2 Lambda fundamentals](#)

[82.3 17.3 Creating a Lambda function](#)

[82.4 17.4 Event sources and triggers](#)

[82.5 17.5 Concurrency](#)

[82.6 17.6 Cold starts and performance](#)

[82.7 17.7 Lambda layers](#)

[82.8 17.8 Lambda in a VPC](#)

[82.9 17.9 Error handling and retries](#)

[82.10 17.10 Monitoring](#)

[82.11 17.11 Chapter summary](#)

[82.12 Lab 17.1 — S3-triggered processor](#)

[82.13 Lab 17.2 — API with provisioned concurrency](#)

[82.14 Review questions](#)

[83 Chapter 18: ECS & Fargate](#)

[83.1 18.1 Container orchestration on AWS](#)

[83.2 18.2 ECS architecture](#)

[83.3 18.3 EC2 vs Fargate launch types](#)

[83.4 18.4 Task definitions](#)

[83.5 18.5 ECS services and load balancing](#)

[83.6 18.6 Auto Scaling ECS services](#)

[83.7 18.7 Amazon ECR](#)

[83.8 18.8 Networking modes](#)

[83.9 18.9 Service Connect and Service Discovery](#)

[83.10 18.10 ECS Exec](#)

[83.11 18.11 Chapter summary](#)

[83.12 Lab 18.1 — Fargate web service](#)

[83.13 Lab 18.2 — ECS Exec debugging](#)

[83.14 Review questions](#)

[84 Chapter 19: EKS — Managed Kubernetes on AWS](#)

[84.1 19.1 Why Amazon EKS?](#)

[84.2 19.2 EKS architecture](#)

[84.3 19.3 Cluster creation](#)

[84.4 19.4 Node groups](#)

[84.5 19.5 Networking](#)

[84.6 19.6 IAM integration](#)

[84.7 19.7 Essential add-ons](#)

[84.8 19.8 Storage](#)

[84.9 19.9 Security](#)

[84.10 19.10 Upgrades and operations](#)

[84.11 19.11 EKS vs ECS decision matrix](#)

[84.12 19.12 Chapter summary](#)

[84.13 Lab 19.1 — EKS cluster deployment](#)

[84.14 Lab 19.2 — Persistent storage](#)

[84.15 Review questions](#)

[85 Chapter 20: S3 — Buckets, Objects, Versioning, and Lifecycle](#)

[85.1 20.1 Amazon S3 overview](#)

[85.2 20.2 Core concepts](#)

[85.3 20.3 Storage classes](#)

[85.4 20.4 Versioning and lifecycle](#)

[85.5 20.5 Security](#)

[85.6 20.6 Static website hosting](#)

[85.7 20.7 S3 performance tips](#)

[85.8 20.8 Chapter summary](#)

[85.9 Lab 20.1](#)

[86 Chapter 21: S3 Security, Encryption, and Static Websites](#)

[86.1 21.1 S3 security model](#)

[86.2 21.2 Block Public Access](#)

[86.3 21.3 Bucket policies](#)

[86.4 21.4 IAM policies for S3](#)

[86.5 21.5 Encryption](#)

[86.6 21.6 Access Points and Object Lambda](#)

[86.7 21.7 Static website hosting](#)

[86.8 21.8 S3 access logging and monitoring](#)

[86.9 21.9 Pre-signed URLs and temporary access](#)

[86.10 21.10 Chapter summary](#)

[86.11 Lab 21.1 — Secure bucket](#)

[86.12 Lab 21.2 — Static website with CloudFront](#)

[86.13 Review questions](#)

87 Chapter 22: EBS & EFS

[87.1 22.1 Block vs file storage on AWS](#)

[87.2 22.2 Amazon EBS deep dive](#)

[87.3 22.3 EBS snapshots](#)

[87.4 22.4 Amazon EFS deep dive](#)

[87.5 22.5 EBS vs EFS comparison](#)

[87.6 22.6 Instance Store \(ephemeral\)](#)

[87.7 22.7 EBS optimization and monitoring](#)

[87.8 22.8 Chapter summary](#)

[87.9 Lab 22.1 — EBS volume lifecycle](#)

[87.10 Lab 22.2 — Shared EFS](#)

[87.11 Review questions](#)

88 Chapter 23: AWS Backup & Disaster Recovery Patterns

[88.1 23.1 Why backup and disaster recovery matter](#)

[88.2 23.2 AWS Backup](#)

[88.3 23.3 DR strategies](#)

[88.4 23.4 Cross-region and cross-account backup](#)

[88.5 23.5 Service-specific DR patterns](#)

[88.6 23.6 Failover automation](#)

[88.7 23.7 Testing DR](#)

[88.8 23.8 Compliance and retention](#)

[88.9 23.9 Chapter summary](#)

[88.10 Lab 23.1 — AWS Backup plan](#)

[88.11 Lab 23.2 — Cross-region replication](#)

[88.12 Review questions](#)

89 Chapter 24: RDS & Aurora

[89.1 24.1 Managed relational databases](#)

[89.2 24.2 RDS fundamentals](#)

[89.3 24.3 High availability](#)

[89.4 24.4 Amazon Aurora](#)

[89.5 24.5 Backups and recovery](#)

[89.6 24.6 Security](#)

[89.7 24.7 Parameter and option groups](#)

[89.8 24.8 Monitoring](#)

[89.9 24.9 Terraform example](#)

[89.10 24.10 Chapter summary](#)

[89.11 Lab 24.1 — Aurora cluster](#)

[89.12 Lab 24.2 — Point-in-time recovery](#)

[89.13 Review questions](#)

90 Chapter 25: DynamoDB — Tables, Indexes, and Streams

[90.1 25.1 What is DynamoDB?](#)

[90.2 25.2 Core concepts](#)

[90.3 25.3 Capacity modes](#)

[90.4 25.4 Indexes](#)

[90.5 25.5 Query and Scan](#)

[90.6 25.6 DynamoDB Streams](#)

[90.7 25.7 Advanced features](#)

[90.8 25.8 Single-table design](#)

[90.9 25.9 Terraform example](#)

[90.10 25.10 Monitoring](#)

[90.11 25.11 Chapter summary](#)

[90.12 Lab 25.1 — Orders table](#)

[90.13 Lab 25.2 — Streams + Lambda](#)

[90.14 Review questions](#)

91 Chapter 26: ElastiCache & Caching Patterns

[91.1 26.1 Why caching?](#)

[91.2 26.2 Redis vs Memcached](#)

[91.3 26.3 ElastiCache deployment options](#)

[91.4 26.4 Common caching patterns](#)
[91.5 26.5 Redis data structure use cases](#)
[91.6 26.6 Session management](#)
[91.7 26.7 ElastiCache security](#)
[91.8 26.8 Monitoring and maintenance](#)
[91.9 26.9 Caching anti-patterns](#)
[91.10 26.10 Chapter summary](#)
[91.11 Lab 26.1 — Redis cache-aside](#)
[91.12 Lab 26.2 — Leaderboard](#)
[91.13 Review questions](#)

92 Chapter 27: Database Migration (DMS) & DR

[92.1 27.1 Database migration challenges](#)
[92.2 27.2 AWS DMS overview](#)
[92.3 27.3 Migration workflow](#)
[92.4 27.4 DMS replication instance](#)
[92.5 27.5 Schema Conversion Tool \(SCT\)](#)
[92.6 27.6 Migration strategies](#)
[92.7 27.7 Data validation](#)
[92.8 27.8 Ongoing replication for DR](#)
[92.9 27.9 Troubleshooting](#)
[92.10 27.10 Chapter summary](#)
[92.11 Lab 27.1 — Homogeneous migration](#)
[92.12 Lab 27.2 — Migration assessment](#)
[92.13 Review questions](#)

93 Chapter 28: API Gateway

[93.1 28.1 What is Amazon API Gateway?](#)
[93.2 28.2 API types](#)
[93.3 28.3 REST API architecture](#)
[93.4 28.4 HTTP API with Lambda](#)
[93.5 28.5 Authentication and authorization](#)
[93.6 28.6 Throttling and quotas](#)
[93.7 28.7 Request/response transformation](#)
[93.8 28.8 Caching](#)
[93.9 28.9 Custom domains and TLS](#)
[93.10 28.10 WebSocket API](#)
[93.11 28.11 Monitoring](#)
[93.12 28.12 Chapter summary](#)
[93.13 Lab 28.1 — HTTP API + Lambda CRUD](#)
[93.14 Lab 28.2 — Throttling and usage plans](#)
[93.15 Review questions](#)

94 Chapter 29: SQS, SNS, and EventBridge

[94.1 29.1 Event-driven architecture on AWS](#)
[94.2 29.2 Amazon SQS](#)
[94.3 29.3 Amazon SNS](#)
[94.4 29.4 Amazon EventBridge](#)
[94.5 29.5 Choosing the right service](#)
[94.6 29.6 Error handling patterns](#)
[94.7 29.7 Monitoring](#)
[94.8 29.8 Chapter summary](#)
[94.9 Lab 29.1 — Order processing pipeline](#)
[94.10 Lab 29.2 — EventBridge automation](#)
[94.11 Review questions](#)

95 Chapter 30: Step Functions Workflows

[95.1 30.1 What are Step Functions?](#)
[95.2 30.2 Workflow types](#)
[95.3 30.3 State types](#)
[95.4 30.4 Example: Order processing workflow](#)
[95.5 30.5 Service integrations](#)

[95.6 30.6 Error handling](#)

[95.7 30.7 Parallel and Map states](#)

[95.8 30.8 Terraform](#)

[95.9 30.9 Monitoring and debugging](#)

[95.10 30.10 Chapter summary](#)

[95.11 Lab 30.1 — Order workflow](#)

[95.12 Lab 30.2 — Parallel processing](#)

[95.13 Review questions](#)

96 Chapter 31: CloudFormation Deep Dive

[96.1 31.1 Infrastructure as Code with CloudFormation](#)

[96.2 31.2 Template structure](#)

[96.3 31.3 Stack operations](#)

[96.4 31.4 Nested stacks](#)

[96.5 31.5 StackSets](#)

[96.6 31.6 IAM in CloudFormation](#)

[96.7 31.7 Drift detection](#)

[96.8 31.8 Custom resources](#)

[96.9 31.9 CloudFormation Hooks](#)

[96.10 31.10 Best practices](#)

[96.11 31.11 Chapter summary](#)

[96.12 Lab 31.1 — Multi-tier stack](#)

[96.13 Lab 31.2 — Drift detection](#)

[96.14 Review questions](#)

97 Chapter 32: AWS CDK

[97.1 32.1 What is the AWS CDK?](#)

[97.2 32.2 CDK concepts](#)

[97.3 32.3 Getting started \(TypeScript\)](#)

[97.4 32.4 CDK workflow](#)

[97.5 32.5 Reusable constructs](#)

[97.6 32.6 CDK Pipelines](#)

[97.7 32.7 Testing CDK](#)

[97.8 32.8 CDK vs CloudFormation vs Terraform](#)

[97.9 32.9 Best practices](#)

[97.10 32.10 Chapter summary](#)

[97.11 Lab 32.1 — CDK web application](#)

[97.12 Lab 32.2 — Reusable construct](#)

[97.13 Review questions](#)

98 Chapter 33: CodePipeline, CodeBuild, and CodeDeploy

[98.1 33.1 CI/CD on AWS](#)

[98.2 33.2 CodePipeline](#)

[98.3 33.3 CodeBuild](#)

[98.4 33.4 CodeDeploy](#)

[98.5 33.5 Deployment strategies comparison](#)

[98.6 33.6 GitHub Actions alternative](#)

[98.7 33.7 Security best practices](#)

[98.8 33.8 Chapter summary](#)

[98.9 Lab 33.1 — CodePipeline for ECS](#)

[98.10 Lab 33.2 — Blue/green ECS deployment](#)

[98.11 Review questions](#)

99 Chapter 34: Systems Manager & Parameter Store

[99.1 34.1 AWS Systems Manager overview](#)

[99.2 34.2 Session Manager](#)

[99.3 34.3 Run Command](#)

[99.4 34.4 Patch Manager](#)

[99.5 34.5 State Manager](#)

[99.6 34.6 Parameter Store](#)

[99.7 34.7 Parameter Store vs Secrets Manager](#)

[99.8 34.8 Inventory and Compliance](#)

[99.9 34.9 VPC endpoints for SSM](#)

[99.10 34.10 Chapter summary](#)

[99.11 Lab 34.1 — Session Manager access](#)

[99.12 Lab 34.2 — Parameter Store configuration](#)

[99.13 Review questions](#)

[100 Chapter 35: CloudWatch — Metrics, Alarms, and Dashboards](#)

[100.1 35.1 Amazon CloudWatch overview](#)

[100.2 35.2 Metrics](#)

[100.3 35.3 Alarms](#)

[100.4 35.4 Dashboards](#)

[100.5 35.5 CloudWatch Logs \(overview\)](#)

[100.6 35.6 Observability best practices on AWS](#)

[100.7 35.7 Chapter summary](#)

[100.8 Lab 35.1](#)

[101 Chapter 36: CloudWatch Logs & Insights](#)

[101.1 36.1 CloudWatch Logs overview](#)

[101.2 36.2 Log groups and streams](#)

[101.3 36.3 Ingesting logs](#)

[101.4 36.4 CloudWatch Logs Insights](#)

[101.5 36.5 Metric filters](#)

[101.6 36.6 Log encryption and access control](#)

[101.7 36.7 Log export and archival](#)

[101.8 36.8 Best practices](#)

[101.9 36.9 Chapter summary](#)

[101.10 Lab 36.1 — Logs Insights queries](#)

[101.11 Lab 36.2 — Error alerting pipeline](#)

[101.12 Review questions](#)

[102 Chapter 37: X-Ray & Distributed Tracing](#)

[102.1 37.1 Why distributed tracing?](#)

[102.2 37.2 X-Ray concepts](#)

[102.3 37.3 How X-Ray works](#)

[102.4 37.4 Enabling X-Ray](#)

[102.5 37.5 SDK instrumentation](#)

[102.6 37.6 Service map](#)

[102.7 37.7 Trace analysis](#)

[102.8 37.8 X-Ray with OpenTelemetry](#)

[102.9 37.9 Sampling](#)

[102.10 37.10 X-Ray groups and insights](#)

[102.11 37.11 Chapter summary](#)

[102.12 Lab 37.1 — Lambda tracing](#)

[102.13 Lab 37.2 — Custom annotations](#)

[102.14 Review questions](#)

[103 Chapter 38: CloudTrail & AWS Config](#)

[103.1 38.1 Audit and compliance foundations](#)

[103.2 38.2 AWS CloudTrail](#)

[103.3 38.3 CloudTrail security](#)

[103.4 38.4 AWS Config](#)

[103.5 38.5 Config aggregators](#)

[103.6 38.6 CloudTrail vs Config](#)

[103.7 38.7 Compliance reporting](#)

[103.8 38.8 Chapter summary](#)

[103.9 Lab 38.1 — CloudTrail investigation](#)

[103.10 Lab 38.2 — Config compliance](#)

[103.11 Review questions](#)

[104 Chapter 39: KMS, Secrets Manager, and ACM](#)

[104.1 39.1 Encryption and certificate management](#)

[104.2 39.2 AWS Key Management Service \(KMS\)](#)

[104.3 39.3 AWS Secrets Manager](#)

[104.4 39.4 AWS Certificate Manager \(ACM\)](#)

[104.5 39.5 Choosing the right service](#)

[104.6 39.6 Security best practices](#)

[104.7 39.7 Chapter summary](#)

[104.8 Lab 39.1 — KMS encryption](#)

[104.9 Lab 39.2 — Secrets Manager rotation](#)

[104.10 Review questions](#)

[105 Chapter 40: GuardDuty, Security Hub, WAF, and Shield](#)

[105.1 40.1 AWS security services landscape](#)

[105.2 40.2 Amazon GuardDuty](#)

[105.3 40.3 AWS Security Hub](#)

[105.4 40.4 AWS WAF](#)

[105.5 40.5 AWS Shield](#)

[105.6 40.6 Defense-in-depth architecture](#)

[105.7 40.7 Incident response integration](#)

[105.8 40.8 Chapter summary](#)

[105.9 Lab 40.1 — GuardDuty and Security Hub](#)

[105.10 Lab 40.2 — WAF protection](#)

[105.11 Review questions](#)

[106 Chapter 41: Well-Architected Review & Capstone Design](#)

[106.1 41.1 The AWS Well-Architected Framework](#)

[106.2 41.2 The six pillars](#)

[106.3 41.3 Capstone scenario](#)

[106.4 41.4 Proposed architecture](#)

[106.5 41.5 Pillar-by-pillar review](#)

[106.6 41.6 Well-Architected Tool review process](#)

[106.7 41.7 Anti-patterns to avoid](#)

[106.8 41.8 Handbook journey recap](#)

[106.9 41.9 Chapter summary](#)

[106.10 Lab 41.1 — Well-Architected review](#)

[106.11 Lab 41.2 — Architecture documentation](#)

[106.12 Review questions](#)

[106.13 Final thoughts](#)

1 DevOps & AWS Learning Hub

A structured path from zero to advanced, plus two handbook series:

Resource	Target size	Status
DevOps Curriculum	Reference list	Complete
AWS Curriculum	Reference list	Complete
DevOps Handbook	~1,200 pages	TOC + Part I-II written; Parts III-XII outlined
AWS Handbook	~800 pages	TOC + Part I-II written; Parts III-X outlined

1.1 Download PDFs

Pre-built PDFs are in the [pdf/](#) folder:

PDF	Description
devops-handbook.pdf	DevOps handbook — 11 chapters + full TOC
aws-handbook.pdf	AWS handbook — 11 chapters + full TOC
devops-curriculum.pdf	DevOps tools & concepts reference
aws-curriculum.pdf	AWS services reference
devops-aws-complete-guide.pdf	All content in one PDF

Rebuild PDFs after editing markdown:

```
./scripts/build-pdfs.sh
```

Requires `pandoc` and `wkhtmltopdf`.

- 1. **Start with the curricula** — they list every tool, concept, and AWS service in learning order.
- 2. **Follow the handbook TOCs** — each chapter has a page budget; work through sequentially.
- 3. **Expand on demand** — ask an AI assistant or instructor to flesh out any chapter number from the TOC.

1.2 Suggested study pace

Phase	Duration (self-paced)	Focus
Foundations	4-8 weeks	Linux, Git, networking, scripting
Core DevOps	8-12 weeks	CI/CD, containers, IaC, cloud basics
Intermediate	8-12 weeks	Kubernetes, observability, security
Advanced	12+ weeks	SRE, platform engineering, multi-cloud, FinOps

1.3 Page convention

Each handbook “page” $\approx$ 400 words of instructional content + diagrams/commands. Labs and exercises add extra pages in the TOC estimates.

2 DevOps Curriculum: Tools & Concepts (Beginner → Advanced)

Complete reference of what to learn, in recommended order. Items marked **(adv)** are advanced topics.

2.1 Level 0 — Prerequisites (before DevOps)

2.1.1 Concepts

- Software development lifecycle (SDLC)
- Agile, Scrum, Kanban
- Waterfall vs iterative delivery
- Client-server architecture
- APIs (REST, GraphQL basics)
- Databases (SQL vs NoSQL overview)
- Environment tiers: dev, staging, production

2.1.2 Tools & Skills

Category	Tools / Skills
Operating systems	Linux (Ubuntu, RHEL), Windows Server basics
Shell	Bash, Zsh, PowerShell basics
Text editing	Vim, Nano, VS Code
Networking	TCP/IP, DNS, HTTP/HTTPS, TLS, ports, firewalls
Version control	Git (clone, branch, merge, rebase, cherry-pick)
Scripting	Python, Bash, Go (pick one primary)
Documentation	Markdown, diagrams (draw.io, Mermaid)

2.2 Level 1 — DevOps Foundations

2.2.1 Concepts

- What is DevOps? Culture, CALMS (Culture, Automation, Lean, Measurement, Sharing)
- DevOps vs SysAdmin vs SRE vs Platform Engineering
- Continuous Integration (CI) vs Continuous Delivery (CD) vs Continuous Deployment
- Build artifacts, immutable infrastructure
- Configuration vs secrets
- The Twelve-Factor App
- DevSecOps introduction
- GitOps introduction

2.2.2 Tools

Category	Tools
Git hosting	GitHub, GitLab, Bitbucket, Azure DevOps Repos
SSH & keys	OpenSSH, ssh-agent, key pairs
Package managers	apt, yum/dnf, Homebrew

Build tools	Make, Maven, Gradle, npm/yarn/pnpm
Virtualization	VirtualBox, VMware, Hyper-V
Cloud CLI basics	AWS CLI, gcloud, az

2.3 Level 2 — Infrastructure & Automation

2.3.1 Concepts

- Infrastructure as Code (IaC) principles
- Idempotency, declarative vs imperative
- Server provisioning & bootstrapping
- Configuration management
- Immutable vs mutable infrastructure
- Golden images / AMIs
- Inventory & state management

2.3.2 Tools

Category	Tools
IaC — Terraform	Terraform, OpenTofu, Terragrunt
IaC — Cloud-native	AWS CloudFormation, Azure Bicep/ARM, Google Deployment Manager, Pulumi, CDK (AWS/Azure/GCP)
Configuration mgmt	Ansible, Chef, Puppet, SaltStack
Image building	Packer
Secrets (intro)	HashiCorp Vault, AWS Secrets Manager, SOPS
Cloud VMs	EC2, Azure VMs, GCE

2.4 Level 3 — Containers & Orchestration

2.4.1 Concepts

- Containers vs VMs
- Images, layers, registries
- Dockerfile best practices
- Container networking & storage
- Orchestration primitives (Pod, Service, Deployment)
- Service mesh concepts
- Sidecar pattern
- Helm charts & package management

2.4.2 Tools

Category	Tools
Runtime	Docker, containerd, CRI-O, Podman
Compose / local	Docker Compose, Dev Containers
Registries	Docker Hub, ECR, GCR, ACR, Harbor, Artifactory

Orchestration	Kubernetes (K8s), Amazon ECS, Amazon EKS, GKE, AKS
K8s tooling	kubectl, k9s, Lens, Helm, Kustomize
Service mesh	Istio, Linkerd, Consul Connect
Local K8s	minikube, kind, k3s, Docker Desktop K8s

2.5 Level 4 — CI/CD Pipelines

2.5.1 Concepts

- Pipeline stages: source → build → test → scan → deploy
- Branching strategies (GitFlow, trunk-based)
- Artifact versioning & promotion
- Blue/green, canary, rolling deployments
- Feature flags
- Pipeline as Code
- DORA metrics (deployment frequency, lead time, MTTR, change failure rate)

2.5.2 Tools

Category	Tools
CI/CD platforms	Jenkins, GitHub Actions, GitLab CI, CircleCI, Travis CI, Azure Pipelines, AWS CodePipeline, Tekton, Argo CD (GitOps)
Build agents	Jenkins agents, GitHub-hosted/self-hosted runners
Artifact repos	Nexus, Artifactory, GitHub Packages
GitOps	Argo CD, Flux, Fleet
Deployment	Spinnaker, Harness, Octopus Deploy

2.6 Level 5 — Cloud Platforms (cross-cutting)

2.6.1 Concepts

- Shared responsibility model
- Regions, AZs, edge locations
- IAM (users, roles, policies, least privilege)
- VPC networking (subnets, routing, NAT, peering)
- Load balancing (L4 vs L7)
- Object storage vs block vs file
- Managed vs self-managed services
- Well-Architected Framework pillars
- Cost optimization & tagging
- Multi-account / landing zone strategy

2.6.2 Tools (by provider)

Provider	Core services to learn
AWS	See curriculum-aws.md
Azure	Resource Manager, AAD, VNet, AKS, App Service

GCP	IAM, VPC, GKE, Cloud Run, Cloud Build
-----	---------------------------------------

2.7 Level 6 — Observability & Reliability

2.7.1 Concepts

- Monitoring vs observability (metrics, logs, traces)
- SLI, SLO, SLA, error budgets
- Alerting philosophy (symptom vs cause)
- On-call, incident management, postmortems
- Chaos engineering
- Capacity planning
- Distributed tracing

2.7.2 Tools

Category	Tools
Metrics	Prometheus, Grafana, Datadog, New Relic, CloudWatch, Azure Monitor
Logging	ELK/EFK (Elasticsearch, Logstash/Fluentd, Kibana), Loki, Splunk, CloudWatch Logs
Tracing	Jaeger, Zipkin, OpenTelemetry, AWS X-Ray, Honeycomb
APM	Datadog APM, Dynatrace, AppDynamics
Incident mgmt	PagerDuty, Opsgenie, incident.io
Status	Statuspage
Chaos	Chaos Monkey, LitmusChaos, Gremlin

2.8 Level 7 — Security (DevSecOps)

2.8.1 Concepts

- Shift-left security
- OWASP Top 10
- Supply chain security (SBOM, signing)
- Zero trust networking
- CIS benchmarks
- Compliance frameworks (SOC 2, PCI-DSS, HIPAA overview)
- Threat modeling
- Principle of least privilege

2.8.2 Tools

Category	Tools
SAST	SonarQube, Semgrep, CodeQL, Checkmarx
DAST	OWASP ZAP, Burp Suite
Container scan	Trivy, Gype, Snyk, Clair
IaC scan	Checkov, tfsec, KICS, cfn-lint

Secrets scan	git-secrets, TruffleHog, Gitleaks
Policy as code	OPA, Gatekeeper, Kyverno, Sentinel
WAF / edge	AWS WAF, Cloudflare, ModSecurity
PKI / certs	cert-manager, Let's Encrypt, ACM
SIEM	Splunk, Elastic Security, AWS Security Hub

2.9 Level 8 — Data, Messaging & Databases (Ops view)

2.9.1 Concepts

- Backup & restore strategies (RPO/RTO)
- Replication, failover, DR
- Connection pooling
- Caching layers
- Message queues & event-driven architecture
- Database migrations in CI/CD

2.9.2 Tools

Category	Tools
RDBMS	PostgreSQL, MySQL, MariaDB, SQL Server
NoSQL	MongoDB, DynamoDB, Redis, Cassandra
Messaging	Kafka, RabbitMQ, SQS, SNS, Pub/Sub
Streaming	Kinesis, Flink, Spark Streaming
Backup	Velero (K8s), AWS Backup, pg_dump, Percona XtraBackup
Migrations	Flyway, Liquibase, Alembic

2.10 Level 9 — Platform Engineering & Internal Developer Platforms (adv)

2.10.1 Concepts

- Platform as a product
- Developer experience (DevEx)
- Self-service portals
- Golden paths & paved roads
- Service catalogs
- Backstage software templates

2.10.2 Tools

Category	Tools
IDP	Backstage, Port, Cortex
API gateways	Kong, Apigee, AWS API Gateway, Ambassador
Service discovery	Consul, Eureka, CoreDNS

Feature flags	LaunchDarkly, Unleash, Flagsmith
---------------	----------------------------------

2.11 Level 10 — Advanced & Specialized (adv)

2.11.1 Concepts

- Site Reliability Engineering (SRE) — toil reduction, automation
- FinOps & cloud cost governance
- Multi-cloud & hybrid cloud
- Edge computing & CDN strategy
- WebAssembly in ops (adv)
- eBPF for observability (adv)
- ML/LLM ops (MLOps) pipelines

2.11.2 Tools

Category	Tools
FinOps	Kubecost, CloudHealth, AWS Cost Explorer, Infracost
CDN	CloudFront, Cloudflare, Fastly, Akamai
DNS	Route 53, Cloudflare DNS, ExternalDNS
Workflow	Airflow, Temporal, Step Functions
MLOps	MLflow, Kubeflow, SageMaker (AWS)
eBPF	Cilium, Pixie, Falco

2.12 Certifications (optional milestones)

Track	Certifications
Linux	LPIC, RHCSA/RHCE
Cloud	AWS SAA, AWS DVA, AWS SOA, AWS DOP, AWS SAP; Azure AZ-104, AZ-400; GCP PCA
Kubernetes	CKA, CKAD, CKS
Terraform	HashiCorp Terraform Associate
Security	CISSP (broad), AWS Security Specialty
SRE	Google SRE book + practice (no single cert)

2.13 Learning project ideas (apply everything)

1. **Static site pipeline** — S3 + CloudFront + GitHub Actions + Terraform
2. **Microservices on EKS** — Helm, Prometheus, Argo CD
3. **Full observability stack** — app + OpenTelemetry + Grafana stack
4. **Secure CI/CD** — SAST/DAST/SCA in pipeline, signed images
5. **Multi-env IaC** — dev/stage/prod with Terragrunt or Terraform workspaces
6. **Incident simulation** — chaos tests + runbooks + PagerDuty
7. **Internal developer portal** — Backstage with one golden-path service

3 AWS Cloud Curriculum: Services & Tools (Beginner → Advanced)

All major AWS services grouped by domain, with learning priority.

Legend: ★ = core (learn first) · ◆ = intermediate · ◇ = advanced/specialized

3.1 1. Account, Identity & Governance ★

Service / Tool	Purpose
AWS Organizations ★	Multi-account structure, SCPs
IAM ★	Users, groups, roles, policies, MFA
IAM Identity Center (SSO) ★	Workforce federation, permission sets
AWS Control Tower ◆	Landing zone automation
AWS Config ◆	Resource compliance & history
AWS CloudTrail ★	API audit logging
AWS Service Catalog ◇	Governed self-service provisioning
AWS Resource Access Manager (RAM) ◆	Cross-account resource sharing
AWS Account Factory ◆	Automated account vending (Control Tower)
AWS CLI / CloudShell ★	Command-line & browser shell
AWS Management Console ★	Web UI
AWS CloudFormation ★	Infrastructure as Code (native)
AWS CDK ◆	IaC in TypeScript/Python/etc.
Terraform + AWS Provider ★	Industry-standard IaC
AWS Tagging & Resource Groups ★	Cost & ops organization

3.1.1 Concepts

- Shared responsibility model
 - Root account hygiene (MFA, no daily use)
 - Least privilege & permission boundaries
 - SCP vs IAM policy
 - Cross-account roles & assume-role
 - AWS global vs regional services
-

3.2 2. Networking & Content Delivery ★

Service	Purpose
VPC ★	Virtual network, subnets, route tables
Internet Gateway / NAT Gateway ★	Public & private egress
VPC Endpoints (Gateway & Interface) ◆	Private AWS API access

Security Groups & NACLs ★	Stateful vs stateless firewalls
Elastic IP ★	Static public IPs
Route 53 ★	DNS, health checks, routing policies
CloudFront ★	CDN, edge caching
ALB / NLB / GWLB ★	Layer 7, 4, and gateway load balancers
VPC Peering / Transit Gateway ◆	Inter-VPC & hybrid connectivity
AWS VPN / Direct Connect ◆	On-prem hybrid networking
PrivateLink ◆	Private service exposure
AWS Global Accelerator ◇	Anycast for global apps
Route 53 Resolver ◆	Hybrid DNS
AWS Network Firewall ◇	Managed perimeter firewall
VPC Lattice ◇	Service-to-service networking

3.2.1 Tools

- **Reachability Analyzer, VPC Flow Logs, Traffic Mirroring**

3.3 3. Compute ★

Service	Purpose
EC2 ★	Virtual machines, AMIs, instance types
Auto Scaling Groups ★	Horizontal scaling
Elastic Beanstalk ◆	PaaS for web apps
Lambda ★	Serverless functions
ECS / Fargate ★	Container orchestration (AWS-native)
EKS ★	Managed Kubernetes
AWS Batch ◆	Batch/HPC jobs
Lightsail ◆	Simplified VPS
App Runner ◆	Container PaaS
EC2 Image Builder ◆	Automated AMI pipelines
AWS Outposts / Wavelength / Local Zones ◇	Edge & hybrid compute
Serverless Application Repository ◇	Share Lambda apps

3.3.1 Tools

- **AWS Systems Manager** (Session Manager, Run Command, Patch Manager)
- **EC2 Instance Connect**

3.4 4. Storage ★

Service	Purpose

S3 ★	Object storage, versioning, lifecycle
EBS ★	Block storage for EC2
EFS ◆	Managed NFS
FSx ◆	Windows/Lustre/ONTAP/OpenZFS file systems
S3 Glacier ◆	Archive tiers
Storage Gateway ◇	Hybrid storage
AWS Backup ◆	Centralized backup
DataSync / Transfer Family ◆	Large data migration

3.4.1 Concepts

- Storage classes (S3 Standard, IA, Glacier)
- Encryption at rest (SSE-S3, SSE-KMS, SSE-C)
- S3 bucket policies vs ACLs
- EBS volume types (gp3, io2, st1)

3.5 5. Databases ★

Service	Purpose
RDS ★	Managed MySQL, PostgreSQL, MariaDB, Oracle, SQL Server
Aurora ★	AWS-native relational, serverless option
DynamoDB ★	Serverless NoSQL key-value/document
ElastiCache ◆	Redis / Memcached
DocumentDB ◆	MongoDB-compatible
Neptune ◇	Graph database
Timestream ◇	Time-series
Keyspaces ◇	Cassandra-compatible
Redshift ◆	Data warehouse
QLDB ◇	Ledger database
MemoryDB for Redis ◆	Durable Redis
RDS Proxy ◆	Connection pooling

3.6 6. Application Integration & Messaging ★

Service	Purpose
SQS ★	Managed message queues
SNS ★	Pub/sub notifications
EventBridge ★	Event bus, scheduling
Step Functions ◆	Workflow orchestration
MQ ◆	Managed ActiveMQ/RabbitMQ
AppSync ◆	GraphQL API

API Gateway ★	REST/WebSocket/HTTP APIs
----------------------	--------------------------

3.7 7. Developer Tools & CI/CD ★

Service	Purpose
CodeCommit ◆	Git repos (legacy; many use GitHub)
CodeBuild ★	Managed build
CodeDeploy ◆	Deployment automation
CodePipeline ★	CI/CD orchestration
CodeArtifact ◆	Package repository
Cloud9 ◆	Cloud IDE
CDK / SAM / Copilot ◆	IaC & serverless/container tooling
X-Ray ◆	Distributed tracing
CloudWatch Synthetics ◆	Canary monitoring

3.8 8. Observability & Management ★

Service	Purpose
CloudWatch ★	Metrics, alarms, dashboards, logs
CloudWatch Logs Insights ★	Log query
AWS X-Ray ◆	Tracing
Systems Manager ★	Ops hub (Parameter Store, Patch, Session)
CloudTrail ★	Audit
AWS Health Dashboard ★	Service events
Trusted Advisor ◆	Best-practice checks
Personal Health Dashboard ★	Account-specific events
Service Quotas ◆	Limit management
OpsWorks ◇	Legacy Chef/Puppet (avoid for new projects)

3.9 9. Security, Identity & Compliance ★

Service	Purpose
IAM ★	(see section 1)
KMS ★	Encryption key management
Secrets Manager ★	Rotating secrets
Certificate Manager (ACM) ★	TLS certificates
WAF & Shield ◆	Web firewall & DDoS

GuardDuty ★	Threat detection
Inspector ◆	Vulnerability scanning
Macie ◆	S3 data classification
Security Hub ◆	Central security findings
Detective ◇	Security investigation
Firewall Manager ◆	Central WAF/rule management
Artifact ◆	Compliance reports
Audit Manager ◇	Continuous audit evidence
CloudHSM ◇	Dedicated HSM
Cognito ◆	User pools & identity for apps
Verified Permissions ◇	Cedar policy engine
IAM Access Analyzer ◆	External access findings

3.10 10. Migration & Transfer ◆

Service	Purpose
Migration Hub	Migration tracking
Application Migration Service (MGN)	Lift-and-shift
Database Migration Service (DMS)	DB replication
Server Migration Service	(legacy → MGN)
Transfer Family	SFTP/FTPS/FTP to S3
Snow Family	Physical data transfer appliances

3.11 11. Analytics & Big Data ◆

Service	Purpose
Athena ◆	SQL on S3
Glue ◆	ETL & data catalog
EMR ◆	Hadoop/Spark
Kinesis ◆	Streaming (Data Streams, Firehose, Analytics)
QuickSight ◆	BI dashboards
Lake Formation ◆	Data lake governance
MSK ◆	Managed Kafka
OpenSearch Service ◆	Search & analytics (Elasticsearch fork)
Data Pipeline ◇	Legacy orchestration
Clean Rooms ◇	Privacy-preserving analytics

3.12 12. Machine Learning & AI ◇

Service	Purpose
SageMaker ◇	End-to-end ML platform
Bedrock ◇	Foundation models API
Comprehend / Rekognition / Textract / Polly / Transcribe ◇	AI APIs
Forecast / Personalize / Kendra ◇	Specialized ML
CodeWhisperer / Q Developer ◆	AI-assisted development

3.13 13. IoT, Media & Specialized ◇

Service	Purpose
IoT Core / Greengrass	Device connectivity
MediaConvert / Live / Package	Video processing
GameLift	Game server hosting
WorkSpaces / AppStream	Virtual desktops
Connect	Contact center

3.14 14. Cost & Billing ★

Service / Tool	Purpose
Cost Explorer ★	Spend analysis
Budgets ★	Alerts & limits
Cost & Usage Report (CUR) ◆	Detailed billing export
Pricing Calculator ★	Estimate costs
Savings Plans / Reserved Instances ◆	Commitment discounts
Compute Optimizer ◆	Right-sizing recommendations
Billing Conductor ◇	Reseller/MSP billing

3.14.1 Concepts

- Pay-as-you-go vs reserved vs spot
- Tagging strategy for cost allocation
- FinOps culture & unit economics

3.15 15. Well-Architected Framework (conceptual ★)

Six pillars — learn alongside every service:

1. **Operational Excellence** — run & monitor systems
2. **Security** — protect data & assets

3. **Reliability** — recover from failure
4. **Performance Efficiency** — use resources well
5. **Cost Optimization** — avoid unnecessary spend
6. **Sustainability** — minimize environmental impact

Tool: AWS Well-Architected Tool (workload reviews)

3.16 Certification path (recommended order)

Order	Certification	Focus
1	Cloud Practitioner (CLF)	Broad AWS vocabulary
2	Solutions Architect Associate (SAA)	Design & core services
3	Developer Associate (DVA)	CI/CD, serverless, APIs
4	SysOps Administrator (SOA)	Operations & monitoring
5	DevOps Engineer Professional (DOP)	Advanced CI/CD & automation
6	Solutions Architect Professional (SAP)	Complex multi-account design
7	Security Specialty	Deep security services
8	Networking Specialty	VPC, hybrid, TGW

3.17 Hands-on lab progression

1. Create IAM user, MFA, CLI profile
2. VPC with public/private subnets + NAT
3. EC2 + ALB + Auto Scaling
4. S3 static site + CloudFront + Route 53
5. RDS Multi-AZ + Secrets Manager
6. Lambda + API Gateway + DynamoDB
7. ECS Fargate or EKS cluster with CI/CD
8. CloudWatch dashboards + alarms + X-Ray
9. Terraform modules for reusable infra
10. Multi-account landing zone (Organizations + Control Tower)

4 DevOps Handbook — Master Table of Contents

Target length: ~1,200 pages (~480,000 words)
Audience: Beginner → Advanced
Format: Each numbered section ≈ one “page” of ~400 words in the final edition

4.1 How to read this handbook

Work through **Parts I-XII** in order. Labs (marked) add hands-on time beyond page counts. Cross-reference the [AWS Handbook](#) for cloud-specific depth.

4.2 Part I — Introduction & Mindset (Pages 1-60)

Ch	Title	Pages
1	What Is DevOps? History and Culture	1-15
2	CALMS, Three Ways, and Team Topologies	16-30
3	DevOps vs SRE vs Platform Engineering	31-42
4	Career Paths, Certifications, and Learning Strategy	43-60

Written: [Chapters 1-4](#) (Introduction complete)

4.3 Part II — Linux & Shell Mastery (Pages 61-140)

Ch	Title	Pages
5	Linux Fundamentals: Filesystem, Users, Permissions	61-85
6	Processes, systemd, and Service Management	86-105
7	Networking on Linux: ip, ss, curl, dig	106-120
8	Bash Scripting for Automation	121-140

Written: [Chapter 5](#), [Chapter 6](#), [Chapter 7](#), [Chapter 8](#)

4.4 Part III — Git & Collaboration (Pages 141-200)

Ch	Title	Pages
9	Git Internals: Objects, Refs, and the DAG	141-158
10	Branching Strategies and Code Review	159-175
11	GitHub/GitLab Workflows: PRs, Issues, Actions intro	176-200

Written: [Chapter 9](#), [Chapter 10](#), [Chapter 11](#)

4.5 Part IV — Networking for DevOps (Pages 201-280)

Ch	Title	Pages
12	OSI Model, TCP/IP, DNS Deep Dive	201-225
13	HTTP/HTTPS, TLS, and Certificates	226-245
14	Load Balancing, Proxies, and CDN Concepts	246-265
15	Cloud Networking Patterns (VPC overview)	266-280

Written: [Chapter 12](#) through [Chapter 15](#)

4.6 Part V — Scripting & Programming (Pages 281-360)

Ch	Title	Pages
16	Python for DevOps: boto3, requests, pathlib	281-305
17	Go for CLI Tools and Operators	306-325
18	JSON/YAML, Jinja2, and Templating	326-345
19	Testing Automation Scripts	346-360

Written: [Chapter 16](#) through [Chapter 19](#)

4.7 Part VI — Infrastructure as Code (Pages 361-480)

Ch	Title	Pages
20	IaC Principles: State, Idempotency, Drift	361-380
21	Terraform: HCL, Providers, Modules	381-410
22	Terraform: Workspaces, Backends, CI Integration	411-430
23	Ansible: Playbooks, Roles, Inventories	431-450
24	Packer & Golden Images	451-465
25	Policy as Code: OPA, Checkov, Sentinel	466-480

4.8 Part VII — Containers (Pages 481-580)

Ch	Title	Pages
26	Docker: Images, Dockerfile, Multi-stage Builds	481-505
27	Container Networking, Volumes, and Security	506-525
28	Docker Compose and Local Dev Environments	526-540
29	Container Registries and Image Signing	541-560
30	Rootless Containers and Alternatives (Podman)	561-580

4.9 Part VIII — Kubernetes (Pages 581-720)

Ch	Title	Pages
31	K8s Architecture: Control Plane & Node Components	581-605
32	Workloads: Pod, Deployment, StatefulSet, DaemonSet	606-630
33	Services, Ingress, and Network Policies	631-655
34	ConfigMaps, Secrets, and Storage (PV/PVC)	656-675
35	Helm, Kustomize, and GitOps (Argo CD)	676-700
36	RBAC, Pod Security, and Hardening	701-720

4.10 Part IX — CI/CD (Pages 721-840)

Ch	Title	Pages
37	CI/CD Concepts and Pipeline Design	721-745
38	GitHub Actions: Workflows, Secrets, Runners	746-770
39	Jenkins: Pipelines as Code (Declarative)	771-790
40	GitLab CI and Multi-platform Comparison	791-810
41	Deployment Strategies: Blue/Green, Canary, Feature Flags	811-825
42	DORA Metrics and Continuous Improvement	826-840

4.11 Part X — Observability & SRE (Pages 841-960)

Ch	Title	Pages
43	Monitoring Fundamentals: Metrics, Logs, Traces	841-865
44	Prometheus & Grafana Stack	866-890
45	ELK/EFK and Log Aggregation	891-910
46	OpenTelemetry and Distributed Tracing	911-930
47	SLOs, Error Budgets, and Incident Response	931-945
48	Chaos Engineering and Game Days	946-960

4.12 Part XI — Security (DevSecOps) (Pages 961-1080)

Ch	Title	Pages
49	Shift-Left Security in the Pipeline	961-980
50	SAST, DAST, SCA, and Container Scanning	981-1000
51	Secrets Management: Vault, SOPS, External Secrets	1001-1020
52	Supply Chain: SBOM, Sigstore, SLSA	1021-1040

53	Runtime Security: Falco, OPA Gatekeeper	1041-1060
54	Compliance Automation and Audit Trails	1061-1080

4.13 Part XII — Advanced Topics (Pages 1081-1200)

Ch	Title	Pages
55	Platform Engineering & Internal Developer Platforms	1081-1100
56	Multi-Cloud and Hybrid Patterns	1101-1115
57	FinOps and Cost-Aware Engineering	1116-1130
58	MLOps Overview for DevOps Engineers	1131-1145
59	Capstone: End-to-End Production System Design	1146-1180
60	Appendix: Command Cheatsheets, Glossary, Index	1181-1200

4.14 Expansion status

Part	Status
I	Ch 1-4 complete
II	Ch 5-8 complete
III	Ch 9-11 complete
IV	Ch 12-15 complete
V	Ch 16-19 complete
VI-XII	Ch 20-21, 26, 37 partial (~remaining chapters)

To request a chapter: *“Expand DevOps Handbook Chapter 21 (Terraform modules)”*

5 Chapter 1: What Is DevOps? History and Culture

DevOps Handbook — Part I, Pages 1–15

5.1 1.1 The problem DevOps solves

Before DevOps, software organizations often split into two camps:

- **Development** — wrote features quickly, measured by release velocity.
- **Operations** — kept systems stable, measured by uptime and incident count.

These goals frequently conflicted. Developers threw code “over the wall”; operators resisted change because every deployment risked an outage. Releases were rare, painful, and manual. When something broke in production, blame flowed in both directions.

DevOps is not a single tool or job title. It is a **cultural and technical movement** that unifies development and operations around shared goals: deliver value to users **fast** and **reliably**.

5.2 1.2 A working definition

***DevOps** is the practice of aligning people, processes, and technology so that software can be built, tested, released, and operated continuously—with feedback loops that shorten the time from idea to production.*

Key outcomes:

Outcome	What it means
Shorter lead time	Hours or days from commit to production, not months
Higher deployment frequency	Many small releases instead of rare big-bang releases
Lower change failure rate	Most changes succeed without rollback
Faster recovery	When failures happen, restore service quickly

These four metrics are known as **DORA metrics** (DevOps Research and Assessment) and are the standard way to measure DevOps maturity.

5.3 1.3 Brief history

Year	Milestone
2007–2009	Flickr, Etsy, and others share “10+ deploys per day” stories
2009	First DevOpsDays conference in Ghent, Belgium (Patrick Debois)
2010	“The Phoenix Project” novel popularizes DevOps narrative
2013	“The DevOps Handbook” (Gene Kim et al.) codifies practices
2016+	Cloud-native, containers, and Kubernetes accelerate adoption
2020s	Platform engineering, GitOps, and AI-assisted ops emerge

The term **DevOps** blends **Development + Operations**. It intentionally breaks down silos rather than creating a third silo called “DevOps team” that sits between dev and ops.

5.4 1.4 Culture before tools

A common mistake is buying Jenkins, Kubernetes, or Terraform and calling it “DevOps.” Tools enable DevOps; they do not create it.

Cultural pillars:

1. **Shared ownership** — The team that builds a service also runs it (“you build it, you run it”).
 2. **Blameless postmortems** — Focus on systems and processes, not individuals, after incidents.
 3. **Automation of toil** — Repetitive manual work is eliminated or codified.
 4. **Continuous learning** — Experimentation, feedback, and improvement are expected.
 5. **Customer focus** — Every decision ties back to user value.
-

5.5 1.5 DevOps practices (overview)

You will study each in depth later in this handbook:

Practice	Summary
Version control	Everything in Git: code, config, IaC, docs
CI/CD	Automated build, test, deploy pipelines
Infrastructure as Code	Reproducible environments (Terraform, Ansible)
Monitoring & observability	Know what production is doing in real time
Microservices & containers	Smaller deployable units (optional but common)
Security integration (DevSecOps)	Security checks in the pipeline, not at the end

5.6 1.6 Organizational models

5.6.1 Functional silos (anti-pattern for DevOps)

Separate dev, QA, ops, and security teams with handoffs.

5.6.2 Cross-functional product teams (recommended)

One team owns a product or service end-to-end: code, infra, on-call, roadmap.

5.6.3 DevOps enablement / platform team

A **platform team** builds internal tools (CI templates, K8s clusters, golden paths) so product teams move faster. This is **not** a ticket-taking ops team—it is a **product for developers**.

5.7 1.7 What DevOps is NOT

- **Not “NoOps”** — Operations expertise is more important at scale, not less.
 - **Not only for startups** — Enterprises adopt DevOps for compliance and speed.
 - **Not replacing developers with operators** — It merges responsibilities skillfully.
 - **Not a certification checkbox** — Culture and practice matter more than badges.
-

6 Chapter 2: CALMS, Three Ways, and Team Topologies

DevOps Handbook — Part I, Pages 16–30

6.1 2.1 The CALMS framework

CALMS is a mnemonic for assessing DevOps maturity across five dimensions:

Letter	Dimension	Description
C	Culture	Collaboration, trust, shared goals
A	Automation	Eliminate manual, error-prone work
L	Lean	Reduce waste, small batches, flow
M	Measurement	Data-driven decisions, DORA metrics
S	Sharing	Knowledge, tools, failures, successes

6.1.1 Culture (C)

Healthy DevOps culture means developers understand production constraints and operators understand business priorities. Psychological safety allows people to report problems early.

6.1.2 Automation (A)

Automate builds, tests, deployments, infrastructure provisioning, and compliance checks. Manual runbooks become code. **Toil**—repetitive operational work that scales linearly with service growth—should be targeted for elimination.

6.1.3 Lean (L)

Inspired by Toyota Production System: optimize **flow**, limit work in progress (WIP), and deliver **small batches**. Large releases hide defects; small releases surface them quickly.

6.1.4 Measurement (M)

You cannot improve what you do not measure. Track deployment frequency, lead time, MTTR, change failure rate, plus business KPIs (revenue, user satisfaction).

6.1.5 Sharing (S)

Internal conferences, shared Slack channels, communal dashboards, and open postmortems spread expertise. Tools and patterns are reused, not reinvented per team.

6.2 2.2 The Three Ways (Gene Kim)

From *The Phoenix Project* and *The DevOps Handbook*, the **Three Ways** describe the principles of DevOps flow:

6.2.1 First Way: Flow (left to right)

Accelerate delivery from **development** → **operations** → **customer**.

- Make work visible (Kanban boards, pipeline status).

- Reduce batch sizes.
- Build quality in (automated tests at every stage).
- Never pass defects downstream.

Example: A commit triggers CI; only green builds reach staging; only approved artifacts reach production.

6.2.2 Second Way: Feedback (right to left)

Amplify feedback from **production** → **development**.

- Monitor user experience and system health.
- Swarm on problems; shorten feedback loops.
- Developers get production access (with guardrails).

Example: Error rate spike in Grafana pages the on-call engineer; linked traces show the bad deploy; rollback is automated.

6.2.3 Third Way: Continuous learning and experimentation

Create a culture of **improvement** and **risk-taking**.

- Blameless postmortems.
- Reserve time for improvement (20% ops time, hack days).
- Rehearse failures (game days, chaos engineering).

6.3 2.3 Team Topologies (Skelton & Pais)

Modern DevOps organizations structure teams intentionally:

Team type	Purpose
Stream-aligned	Delivers product features; primary value stream
Platform	Provides self-service infra (K8s, CI, databases)
Enabling	Coaches stream teams on new tech (temporary)
Complicated-subsystem	Owns deep specialty (e.g., video encoding, ML)

6.3.1 Interaction modes

- **Collaboration** — Two teams work closely for discovery.
- **X-as-a-Service** — Platform provides API/portal; stream teams consume.
- **Facilitating** — Enabling team mentors others.

Anti-pattern: One ops team serves 50 dev teams via tickets.

Better: Platform team offers paved roads; stream teams deploy autonomously.

6.4 2.4 Conway's Law and reverse Conway maneuver

“Organizations design systems that mirror their communication structure.” — *Melvin Conway*

If dev and ops are separate departments, you will get separate deployment artifacts and fragile handoffs.

Reverse Conway maneuver: Structure teams the way you want the architecture to look. Want autonomous microservices? Give teams end-to-end ownership of each service.

6.5 2.5 Maturity model (informal)

Level	Characteristics
1 — Initial	Manual deploys, silos, hero culture
2 — Managed	Some CI, version control, basic monitoring
3 — Defined	IaC, automated testing, standardized pipelines
4 — Measured	SLOs, DORA tracking, blameless postmortems
5 — Optimizing	Continuous experimentation, platform self-service

Use this for self-assessment, not as a rigid certification ladder.

6.6 2.6 Chapter summary

- **CALMS** evaluates Culture, Automation, Lean, Measurement, Sharing.
 - The **Three Ways** optimize flow, feedback, and learning.
 - **Team Topologies** align org structure with delivery goals.
 - Apply **Conway’s Law** consciously when designing teams.
-

6.7 Lab 2.1 — CALMS assessment

Score your team 1–5 on each CALMS letter. Pick the lowest score and draft one actionable improvement per dimension.

6.8 Review questions

1. What does the “L” in CALMS stand for, and what Lean concept supports it?
 2. Describe the First Way in one sentence.
 3. Name the four team types in Team Topologies.
 4. What is a reverse Conway maneuver?
-

Next: Part II — [Chapter 5: Linux Fundamentals](#)

7 Chapter 3: DevOps vs SRE vs Platform Engineering

DevOps Handbook — Part I, Pages 31-42

7.1 3.1 Three related disciplines

DevOps, **Site Reliability Engineering (SRE)**, and **Platform Engineering** are often confused because they share goals: reliable systems, fast delivery, and reduced toil. They are complementary, not competing.

Discipline	Origin	Primary focus
DevOps	Movement (2009+)	Culture + practices bridging dev and ops
SRE	Google (2003+, public ~2016)	Engineering approach to operations at scale
Platform Engineering	Industry evolution (2020s)	Internal platforms that accelerate product teams

7.2 3.2 Site Reliability Engineering (SRE)

Google introduced **SRE** as “what happens when you ask a software engineer to design an operations function.”

7.2.1 Core SRE principles

1. **Embrace risk** — 100% uptime is impossible and expensive; use **error budgets** to balance reliability and velocity.
2. **Service level objectives (SLOs)** — Define measurable targets (e.g., 99.9% availability).
3. **Eliminate toil** — Manual, repetitive work that scales with traffic must be automated.
4. **Monitoring** — Systems must be observable; alerts fire on symptoms users feel.
5. **Release engineering** — Safe, gradual rollouts with fast rollback.
6. **Simplicity** — Prefer boring, well-understood technology.

7.2.2 SRE team responsibilities

- Define and track SLOs/SLIs
- On-call rotation and incident response
- Capacity planning
- Performance and reliability testing
- Building automation to replace toil

7.2.3 Error budget example

If SLO = 99.9% monthly availability, you have ~43 minutes of downtime budget. When budget is exhausted, **freeze feature releases** and focus on reliability until budget recovers.

7.3 3.3 Platform Engineering

Platform engineering treats the internal developer platform as a **product** whose customers are other engineers.

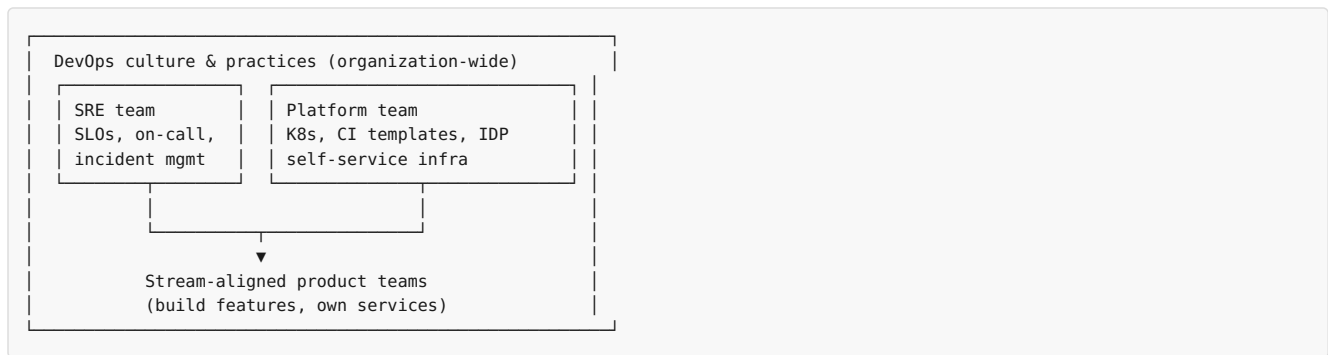
7.3.1 What platform teams build

- Kubernetes clusters with guardrails
- CI/CD templates and golden pipelines
- Self-service databases, caches, and queues
- Developer portals (Backstage) with docs and service catalogs
- Standardized observability (dashboards, tracing, logging)

7.3.2 Platform vs traditional ops

Traditional ops	Platform engineering
Ticket queue for infra requests	Self-service APIs and portals
Snowflake servers	Standardized golden paths
“Call us to deploy”	Teams deploy autonomously with paved roads

7.4 3.4 How they fit together



- **DevOps** sets the cultural north star.
- **SRE** ensures reliability is measurable and engineered.
- **Platform** removes friction so product teams ship faster.

7.5 3.5 Job titles in the market

Titles vary widely. Focus on **responsibilities**, not labels:

Title	Often means
DevOps Engineer	CI/CD, IaC, cloud infra, automation
SRE	SLOs, on-call, reliability tooling, toil reduction
Platform Engineer	Internal platform, K8s, developer experience
Cloud Engineer	AWS/Azure/GCP architecture and implementation
Infrastructure Engineer	Servers, networking, IaC

Many roles blend all three disciplines.

7.6 3.6 Chapter summary

- **SRE** engineers reliability with SLOs, error budgets, and toil automation.
- **Platform engineering** provides self-service paved roads for developers.
- **DevOps** is the umbrella culture; SRE and platform are specialized implementations.

7.7 Lab 3.1

Draft SLOs for a sample API: pick one SLI (latency or availability), set a target, and calculate the monthly error budget in minutes.

Next: [*Chapter 6 — Processes and systemd*](#)

8 Chapter 4: Career Paths, Certifications, and Learning Strategy

DevOps Handbook — Part I, Pages 43-60

8.1 4.1 DevOps is a skill set, not a single job title

Organizations use many titles for similar work: **DevOps Engineer**, **Site Reliability Engineer (SRE)**, **Platform Engineer**, **Cloud Engineer**, **Release Engineer**, **Infrastructure Engineer**, and **Build/Release Manager**. The labels differ; the underlying competencies overlap heavily.

What employers actually hire for:

Competency area	What you must demonstrate
Linux & shell	Debug servers, write automation, read logs
Networking	DNS, HTTP, TLS, load balancers, firewalls
Version control & CI/CD	Git workflows, pipelines, deployment strategies
Infrastructure as Code	Terraform, Ansible, reproducible environments
Containers & orchestration	Docker, Kubernetes, service mesh basics
Observability	Metrics, logs, traces, incident response
Security	Secrets, scanning, least privilege, compliance
Communication	Docs, postmortems, cross-team collaboration

A junior role might emphasize scripting and pipeline maintenance; a senior role adds architecture, reliability design, and mentoring. **Titles follow scope and seniority**, not a separate “DevOps career ladder” disconnected from software engineering.

8.2 4.2 Common career paths

8.2.1 Path A: Developer → DevOps / Platform

Developers who enjoy deployment, performance, and production debugging often move toward platform or DevOps roles. Strengths: application mindset, code quality, API design. Gaps to fill: Linux depth, networking, IaC, on-call culture.

8.2.2 Path B: Sysadmin / NOC → DevOps / SRE

Traditional operations professionals bring production instincts, incident handling, and infrastructure knowledge. Gaps to fill: Git-centric workflows, programming (Python/Go), cloud-native patterns, automation-first thinking.

8.2.3 Path C: QA / Release → CI/CD specialist

Test and release engineers often own pipelines and quality gates. Natural progression: expand into IaC, container platforms, and observability.

8.2.4 Path D: Security → DevSecOps

Security engineers who embed controls in pipelines and infrastructure are increasingly critical. Requires comfort with CI/CD, containers, and cloud IAM.

8.2.5 Senior and leadership trajectories

Level	Typical focus
Junior	Execute runbooks, fix pipeline failures, learn the stack
Mid	Own services end-to-end, design automation, participate in on-call
Senior	Architecture, SLO design, cross-team standards, incident command
Staff / Principal	Org-wide platforms, technical strategy, golden paths
Engineering Manager	People, hiring, delivery—still technical enough to review designs

DevOps careers rarely stay in one lane. The best engineers combine **breadth** (full stack of delivery) with **depth** in one or two areas (e.g., Kubernetes, Terraform, observability).

8.3 4.3 Certifications: value and limits

Certifications **validate baseline knowledge** and help pass HR filters. They do **not** replace hands-on experience or cultural fit.

Certification	Provider	Best for
AWS Certified DevOps Engineer - Professional	AWS	AWS-heavy shops, CI/CD + IaC on AWS
AWS Solutions Architect / SysOps	AWS	Foundational cloud before DevOps Pro
CKA / CKAD / CKS	CNCF	Kubernetes administration and security
Terraform Associate	HashiCorp	IaC fundamentals
LPIC-1 / LPIC-2	Linux Professional Institute	Linux fundamentals (less common in job reqs)
GitHub Actions / GitLab certifications	Vendors	Pipeline-specific credentials
Professional Cloud DevOps Engineer	Google	GCP-centric organizations

How to use certs effectively:

1. **Align with your target stack** — CKA helps for K8s roles; AWS DevOps Pro helps for AWS platform teams.
2. **Study by doing** — Spin up labs; exams that include performance tasks reward practice, not memorization.
3. **Pair with a portfolio** — GitHub repos with Terraform modules, CI workflows, and READMEs beat cert walls alone.
4. **Avoid cert collecting** — Two relevant certs plus projects outweigh five unrelated badges.

Remember Chapter 1: DevOps is **not a certification checkbox**. Hiring managers ask *“What have you shipped and operated?”*

8.4 4.4 Building a learning strategy

8.4.1 The 70/20/10 model (adapted for DevOps)

Allocation	Activity
70%	Hands-on labs, homelab, work projects, break/fix exercises
20%	Peers, communities, code review, conference talks

10%	Courses, books, documentation reading

Reading this handbook without a terminal open builds **false confidence**. Every chapter includes labs; treat them as required.

8.4.2 Spaced repetition and depth-first learning

Avoid “tutorial hell”—watching endless videos without retention. Instead:

1. **Pick one vertical** (e.g., Linux networking) for two weeks.
2. **Complete labs** until commands are muscle memory.
3. **Teach it** — write a short internal doc or blog post.
4. **Move to the next topic** in the handbook sequence.

8.4.3 Homelab options (low to high cost)

Option	Cost	Good for
Local VM (VirtualBox, UTM, KVM)	Free	Linux, Ansible, K8s (kind/minikube)
Cloud free tier	Low	Real AWS/GCP/Azure patterns (watch billing!)
Raspberry Pi cluster	Moderate	Physical networking, k3s
Second-hand server	Moderate	Bare-metal, storage, RAID

Document your homelab in Git: `README.md`, architecture diagram, and teardown scripts prove discipline to interviewers.

8.5 4.5 Portfolio and interview preparation

8.5.1 What belongs in a DevOps portfolio

- **laC repository** — Terraform or OpenTofu provisioning VPC + EKS/GKE/AKS or a simpler stack.
- **CI/CD pipeline** — Build, test, scan, deploy a small app (GitHub Actions or GitLab CI).
- **Observability demo** — Prometheus + Grafana or OpenTelemetry traces on a sample service.
- **Runbook or postmortem sample** — Shows operational maturity (sanitize any employer data).

8.5.2 STAR stories for behavioral interviews

Prepare stories for:

- **Production incident** you helped resolve (what broke, how you mitigated, what changed after).
- **Automation** that saved time (before/after metrics).
- **Disagreement** with dev or security resolved constructively.
- **Learning something hard** under time pressure.

8.5.3 Technical interview topics

Expect live exercises or take-homes involving: read a broken pipeline YAML, explain Git merge vs rebase, sketch a CI/CD diagram, debug `curl / kubectl` output, or write a small Python script against an API.

8.6 4.6 Soft skills that separate good from great

DevOps engineers spend as much time with **people and processes** as with servers.

Skill	
-------	--

	Why it matters
Written communication	Runbooks, RFCs, and Slack updates must be clear under stress
Blameless mindset	Incidents improve systems when fear is removed
Prioritization	Not every alert deserves a 2 a.m. page
Empathy for developers	Platform work succeeds when users adopt it willingly
Security partnership	Say “no” with alternatives, not blockers

8.7 4.7 Staying current without burnout

The toolchain changes constantly (Kubernetes releases, new CI features, AI-assisted ops). Sustainable habits:

1. **Follow a few high-quality sources** — official docs, CNCF blog, vendor release notes for *your* stack only.
2. **Quarterly learning goals** — one cert attempt OR one major project, not both every month.
3. **On-call boundaries** — chronic burnout kills careers faster than missing a new tool launch.
4. **Contribute upstream** — small doc fixes to open source count as learning and networking.

8.8 4.8 Chapter summary

- DevOps careers span multiple titles; **competencies** matter more than labels.
- Paths from dev, ops, QA, or security are all valid with targeted gap-filling.
- Certifications **help hiring filters** but must pair with **hands-on projects**.
- Use **70% practice / 20% collaboration / 10% theory** and depth-first study.
- Portfolio, STAR stories, and communication skills differentiate senior candidates.

8.9 Lab 4.1 — Personal learning plan

1. Write your current role (or target role) and list five skill gaps from Section 4.1.
2. Choose **one** certification OR **one** portfolio project aligned with your target stack.
3. Block **5 hours per week** on your calendar for hands-on labs (not video watching).
4. Create a private Git repo `devops-learning-journal` with a `WEEKLY.md` log; add your first entry.

8.10 Lab 4.2 — Job description analysis

1. Collect three real DevOps/SRE job postings from companies you admire.
2. Highlight repeated requirements (tools, years, responsibilities).
3. Build a spreadsheet: requirement → handbook chapter → lab to complete.
4. Identify the **top three** gaps and schedule them for the next 30 days.

8.11 Review questions

1. Name four competency areas common to DevOps job descriptions.
2. Why is “cert collecting” a weak strategy on its own?
3. What is the 70/20/10 learning split recommended in this chapter?
4. Describe two valid career entry paths into DevOps.
5. What three items would you include in a minimal DevOps portfolio?

Next: [Chapter 5 — Linux Fundamentals: Filesystem, Users, Permissions](#)

9 Chapter 5: Linux Fundamentals — Filesystem, Users, Permissions

DevOps Handbook — Part II, Pages 61–85

9.1 5.1 Why Linux dominates DevOps

Most servers, containers, and cloud instances run **Linux**. As a DevOps engineer you will SSH into machines, debug production issues, write shell scripts, and configure services daily. Windows matters too (Active Directory, .NET shops), but Linux fluency is non-negotiable.

Common distributions:

Distro	Typical use
Ubuntu	Cloud default, tutorials, desktops
Debian	Stable base for Ubuntu and containers
RHEL / Rocky / Alma	Enterprise, regulated industries
Amazon Linux	AWS-optimized

9.2 5.2 Filesystem hierarchy (FHS)

Path	Purpose
/	Root of entire tree
/home	User home directories (/home/alice)
/root	root user’s home
/etc	Configuration files
/var	Variable data: logs (/var/log), caches
/tmp	Temporary files (often cleared on reboot)
/usr	User programs and libraries
/bin , /sbin	Essential binaries
/opt	Optional third-party software
/proc , /sys	Virtual filesystems for kernel info

Everything is a **file** in Unix—including devices and sockets.

9.3 5.3 Essential navigation commands

```
pwd           # Print working directory
ls -lah      # List files (human-readable, including hidden)
cd /etc/nginx # Change directory
cd ..        # Parent directory
cd ~         # Home directory
tree -L 2 /etc # Tree view (install: apt install tree)
```

Tab completion saves time: type partial names and press `Tab`.

9.4 5.4 File operations

```
cp source dest           # Copy
cp -r dir1 dir2          # Copy directory recursively
mv old new               # Move or rename
rm file                  # Delete (careful!)
rm -rf directory         # Delete directory recursively (very careful!)
mkdir -p path/to/dir     # Create nested directories
touch file.txt           # Create empty file or update timestamp
cat file                 # Print file contents
less /var/log/syslog     # Paginated view (q to quit)
head -n 20 file          # First 20 lines
tail -f /var/log/app.log # Follow log in real time
```

9.4.1 Finding files

```
find /var/log -name "*.log" -mtime -7 # Logs modified in 7 days
find / -type f -size +100M 2>/dev/null # Files larger than 100MB
locate nginx.conf                     # Fast index search (updatedb)
```

9.5 5.5 Users, groups, and permissions

9.5.1 Users and groups

```
id                # Current user UID, GID, groups
whoami
cat /etc/passwd   # User accounts
cat /etc/group    # Groups
sudo command      # Run as root (if permitted)
su - username     # Switch user
```

9.5.2 Permission model

Each file has three permission sets for **user (u)**, **group (g)**, **others (o)**:

```
-rwxr-xr-- 1 alice developers 4096 Jan 15 10:00 deploy.sh
| | | | |
| | | | | others: read
| | | | |
| | | | | group: read, execute
| | | | |
| | | | | user: read, write, execute
```

Symbol	Meaning
<code>r</code> (4)	Read
<code>w</code> (2)	Write
<code>x</code> (1)	Execute (directories: enter)

```
chmod 755 script.sh # rwxr-xr-x
chmod u+x script.sh # Add execute for user
chown alice:developers file
chgrp developers file
```

9.5.3 Special permissions

- setuid** — Run as file owner (e.g., `passwd`)
- setgid** — Run as group; dirs inherit group

- **sticky bit** — On `/tmp`, only owner can delete own files

9.6 5.6 Package management (Debian/Ubuntu)

```
sudo apt update           # Refresh package index
sudo apt upgrade -y       # Upgrade installed packages
sudo apt install nginx -y # Install package
apt search nginx          # Search
apt show nginx            # Package info
sudo apt remove nginx     # Remove
dpkg -l | grep nginx     # List installed
```

RHEL/Rocky equivalent: `dnf install`, `dnf update`.

9.7 5.7 Text processing (daily DevOps tools)

```
grep -r "ERROR" /var/log/app/ # Search recursively
grep -E "timeout|refused" log.txt # Extended regex
awk '{print $1}' access.log     # Column extraction
sed 's/old/new/g' file.txt     # Stream editor
sort file | uniq -c | sort -rn | head # Count unique lines
wc -l file                    # Line count
```

Pipes (|) chain commands: stdout of one → stdin of next.

9.8 5.8 Environment and shell

```
echo $PATH                # Executable search path
export MY_VAR=hello       # Set environment variable
env                       # All environment variables
which python3             # Path to executable
alias ll='ls -lah'        # Command shortcut
```

Shell config files: `~/.bashrc`, `~/.profile` (sourced on login).

9.9 5.9 SSH — remote access

```
ssh user@hostname
ssh -i ~/.ssh/mykey.pem ec2-user@1.2.3.4
scp file user@host:/remote/path
rsync -avz ./local/ user@host:/remote/ # Efficient sync
```

`~/.ssh/config` simplifies connections:

```
Host prod-web
  HostName 10.0.1.50
  User ubuntu
  IdentityFile ~/.ssh/prod.pem
```

Then: `ssh prod-web`

9.10 5.10 Chapter summary

- Linux FHS organizes `/etc`, `/var`, `/home`, and more.
 - Master `ls`, `cd`, `cp`, `mv`, `rm`, `find`, `grep`, `tail -f`.
 - Permissions: user/group/other + `chmod` / `chown`.
 - Use `apt` / `dnf` for packages; SSH for remote ops.
-

9.11 Lab 5.1 — Linux sandbox

1. Create user `devops-lab` with home directory.
 2. Create `~/scripts/deploy.sh` with `#!/bin/bash` and `echo "Deploying..."`.
 3. Set permissions to `755`; run it.
 4. Write a one-liner to count ERROR lines in a sample log file.
 5. Configure SSH key auth to a local VM or cloud instance.
-

9.12 Review questions

1. What directory holds system configuration files?
 2. What does permission `644` mean in symbolic notation?
 3. How do you follow a log file in real time?
 4. What is the difference between `cp` and `mv`?
-

Continue: Chapter 6 — Processes, systemd, and Service Management (outline in TOC)

10 Chapter 6: Processes, systemd, and Service Management

DevOps Handbook — Part II, Pages 86–105

10.1 6.1 Understanding Linux processes

A **process** is a running instance of a program. Each has a **PID** (Process ID), owner, state, and resource usage.

```
ps aux          # All processes (BSD style)
ps -ef          # All processes (POSIX style)
top             # Interactive process viewer
htop            # Enhanced top (install separately)
pgrep nginx     # Find PIDs by name
pkill -HUP nginx # Send signal to processes by name
```

10.1.1 Process states

State	Code	Meaning
Running	R	Executing or runnable
Sleeping	S	Waiting for event
Zombie	Z	Terminated but not reaped by parent
Stopped	T	Paused (e.g., Ctrl+Z)

10.1.2 Signals

```
kill -15 <pid> # SIGTERM – graceful shutdown (default)
kill -9 <pid>  # SIGKILL – force kill (last resort)
kill -HUP <pid> # SIGHUP – reload config (many daemons)
```

10.2 6.2 systemd — modern service manager

Most Linux distros use **systemd** to boot the system and manage services (units).

```
systemctl status nginx
systemctl start nginx
systemctl stop nginx
systemctl restart nginx
systemctl reload nginx      # Reload config without full restart
systemctl enable nginx     # Start on boot
systemctl disable nginx
systemctl list-units --type=service --state=running
journalctl -u nginx -f      # Follow service logs
journalctl -u nginx --since "1 hour ago"
```

10.2.1 Unit files

Service definitions live in `/etc/systemd/system/` or `/lib/systemd/system/`.

Example `/etc/systemd/system/myapp.service`:

```
[Unit]
Description=My Application
After=network.target
```

```
[Service]
Type=simple
User=appuser
WorkingDirectory=/opt/myapp
ExecStart=/opt/myapp/bin/server
Restart=on-failure
RestartSec=5
Environment=PORT=8080

[Install]
WantedBy=multi-user.target
```

After creating or editing a unit:

```
sudo systemctl daemon-reload
sudo systemctl enable --now myapp
```

10.3 6.3 Cron and timers

10.3.1 cron — scheduled tasks

```
crontab -e      # Edit user crontab
crontab -l      # List crontab
```

Cron format: `minute hour day month weekday command`

```
# Every day at 2:30 AM – backup database
30 2 * * * /opt/scripts/backup.sh >> /var/log/backup.log 2>&1
```

System-wide jobs: `/etc/cron.d/`, `/etc/cron.daily/`, etc.

10.3.2 systemd timers (preferred on modern systems)

Timers are more flexible and integrate with journald logging.

```
# /etc/systemd/system/backup.timer
[Unit]
Description=Daily backup

[Timer]
OnCalendar=*-*-* 02:30:00
Persistent=true

[Install]
WantedBy=timers.target
```

```
systemctl enable --now backup.timer
systemctl list-timers
```

10.4 6.4 Resource limits and troubleshooting

```
# CPU and memory per process
ps -o pid,user,%cpu,%mem,cmd -p <pid>

# Open files by process
lsof -p <pid>
ss -tlnp      # Listening TCP ports with process
df -h         # Disk usage
free -h       # Memory usage
dmesg | tail  # Kernel messages
```

10.4.1 Common production issues

Symptom	Likely cause	Check
Service won't start	Bad config, missing binary	<code>journalctl -u service -n 50</code>
Port in use	Another process bound	<code>ss -tlnp \ grep :80</code>
Disk full	Logs, temp files	<code>df -h</code> , <code>du -sh /var/log/*</code>
OOM killed	Memory exhaustion	<code>dmesg \ grep -i oom</code>

10.5 6.5 Chapter summary

- Manage processes with `ps`, `kill`, and signals.
- Use **systemctl** and **journalctl** for services on systemd systems.
- Schedule jobs with cron or systemd timers.
- Check logs first when debugging service failures.

10.6 Lab 6.1

1. Create a simple bash script that writes a timestamp to a log every minute.
2. Run it as a systemd service + timer.
3. Practice `systemctl status`, `journalctl -f`, and graceful stop/start.

Next: [Chapter 9 — Git Internals](#)

11 Chapter 7: Networking on Linux — ip, ss, curl, dig

DevOps Handbook — Part II, Pages 106–120

11.1 7.1 Why networking skills matter for DevOps

Every deployment, health check, and incident touches the network: DNS resolution, TCP connections, TLS handshakes, load balancer health probes, and firewall rules. When an application is “down,” the failure is often **not** the app—it is DNS, routing, a security group, or a saturated socket backlog.

Linux provides modern tools that replaced much of the legacy `ifconfig` / `netstat` era:

Tool	Purpose
<code>ip</code>	Interfaces, addresses, routes, rules
<code>ss</code>	Socket statistics (connections, listening ports)
<code>curl</code>	HTTP/HTTPS client for APIs and debugging
<code>dig</code>	DNS queries and troubleshooting
<code>traceroute</code> / <code>tracert</code>	Path discovery
<code>tcpdump</code> / <code>tshark</code>	Packet capture (advanced)

This chapter focuses on daily diagnostic workflows.

11.2 7.2 The `ip` command suite

The `ip` utility from **iproute2** is the standard for network configuration and inspection on modern Linux.

11.2.1 View interfaces and addresses

```
ip link show           # Layer 2: interfaces, MAC, state
ip addr show           # Layer 3: IPv4/IPv6 addresses
ip -br addr            # Brief one-line-per-interface summary
ip addr show dev eth0  # Single interface
```

Example brief output interpretation:

lo	UNKNOWN	127.0.0.1/8 ::1/128
eth0	UP	10.0.1.25/24 fe80::5054:ff:fe12:3456/64

- **UP** — interface administratively enabled and carrier detected (for physical links).
- **127.0.0.1/8** — loopback; always present.
- **/24** — subnet prefix; 256 addresses in this example.

11.2.2 Routing table

```
ip route show          # Main routing table
ip route get 8.8.8.8    # Which path would a packet take?
ip rule show            # Policy routing rules (multi-table)
```

If `ip route get` shows an unexpected interface, compare with your cloud subnet route tables and default gateway.

11.2.3 Temporary address assignment (lab only)

```
sudo ip addr add 192.168.99.10/24 dev eth0
sudo ip link set eth0 up
sudo ip route add default via 192.168.99.1
```

Changes made with `ip` are **not persistent** across reboot unless saved via netplan, NetworkManager, or distro-specific config under `/etc`.

11.3 7.3 Sockets and connections with `ss`

`ss` replaces `netstat` for inspecting listening ports and established connections. It reads from `/proc` and is faster on busy servers.

11.3.1 Common patterns

```
ss -tuln           # TCP/UDP listening ports, numeric
ss -tunap         # Include process names (needs root for all PIDs)
ss -tn state established # Established TCP connections
ss -tn sport = :443  # Connections from local port 443
ss -tn dst 10.0.1.50 # Connections to specific destination
```

Flags:

Flag	Meaning
-t	TCP
-u	UDP
-l	Listening sockets
-n	Numeric (no DNS reverse lookup)
-a	All (listening + non-listening)
-p	Show process

11.3.2 DevOps debugging scenario

A service fails health checks but the process is running:

```
sudo ss -tlnp | grep 8080
curl -v http://127.0.0.1:8080/health
curl -v http://10.0.1.25:8080/health # bind address matters!
```

If `ss` shows `127.0.0.1:8080` but not `0.0.0.0:8080`, the app binds **localhost only**—external load balancers cannot reach it.

11.4 7.4 HTTP debugging with `curl`

`curl` is the universal CLI for HTTP/S, file transfers, and API testing.

11.4.1 Essential options

```
curl -I https://example.com           # HEAD request (headers only)
curl -v https://api.example.com/v1/users # Verbose: TLS, headers, redirects
curl -sS -o /dev/null -w '%{http_code}\n' https://example.com # Status only
curl -H 'Authorization: Bearer TOKEN' https://api.example.com/me
curl -X POST -H 'Content-Type: application/json' \
  -d '{"name": "svc"}' https://api.example.com/items
curl --connect-timeout 5 --max-time 30 https://slow.example.com
curl -k https://self-signed.local      # Skip TLS verify (debug only!)
```

```
curl -o out.bin https://example.com/file # Save body to file
curl -L https://short.url/abc           # Follow redirects
```

11.4.2 Timing breakdown

```
curl -w '\nnlookup:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect} ttfb:%{time_starttransfer} total:%{time_total}\n' \
-o /dev/null -sS https://example.com
```

Phase	Indicates
time_namelookup	DNS delay
time_connect	TCP handshake to server
time_appconnect	TLS handshake complete
time_starttransfer	Time to first byte (server processing)

High time_namelookup → DNS issue. High time_connect → routing/firewall. High time_starttransfer → backend overload.

11.5 7.5 DNS with dig

DNS failures cause mysterious outages: apps cache stale records, TTLs expire during migrations, and split-horizon DNS confuses internal vs external views.

11.5.1 Basic queries

```
dig example.com           # Default A record query
dig example.com AAAA      # IPv6
dig example.com MX        # Mail exchangers
dig example.com NS        # Name servers
dig @8.8.8.8 example.com  # Query specific resolver
dig +short example.com    # Answer only
dig +trace example.com    # Full delegation trace (education/debug)
```

11.5.2 Useful flags for SRE work

```
dig example.com +noall +answer +stats
dig -x 93.184.216.34 # Reverse DNS (PTR)
dig @1.1.1.1 app.internal.company.com # Test internal resolver
```

Compare answers from **corporate resolver**, **public resolver (8.8.8.8)**, and **authoritative NS** when debugging propagation.

11.6 7.6 Putting it together: diagnostic workflow

When `curl https://api.myapp.com/health` fails from a server:

1. `dig api.myapp.com` → Does DNS resolve? Correct IP?
2. `ip route get <that-ip>` → Expected egress path?
3. `curl -v` (with IP / Host) → TLS? HTTP status? Timeout?
4. `ss -tn dst <ip>:443` → Connection state stuck?
5. Check firewall (nftables/iptables, cloud SG)
6. `tcpdump` (if permitted) → Packets leaving/returning?

Document findings in the incident channel as you go—future you will thank present you.

11.7 7.7 /etc/hosts , resolv.conf , and systemd-resolved

File / service	Role
/etc/hosts	Static local overrides (break-glass, testing)
/etc/resolv.conf	Resolver configuration (may be managed)
systemd-resolved	Caching stub resolver on many distros

```
resolvectl status           # Current DNS servers and domains
resolvectl query example.com # Query through systemd-resolved
```

On containers, DNS is often injected by Docker/Kubernetes—always check `/etc/resolv.conf` **inside the pod**, not only on the node.

11.8 7.8 Firewalls at a glance

```
sudo nft list ruleset      # nftables (modern default on many distros)
sudo iptables -L -n -v     # Legacy iptables
sudo ufw status verbose    # Ubuntu uncomplicated firewall
```

DevOps engineers rarely invent firewall rules from scratch—they align **host firewall** with **cloud security groups** and **Kubernetes NetworkPolicies**. Still, verifying `DROP` rules locally saves hours of cloud-console hunting.

11.9 7.9 Chapter summary

- Use `ip` for interfaces, addresses, and routes; prefer it over deprecated `ifconfig`.
- Use `ss` to see listening ports, bind addresses, and connection states.
- Use `curl -v` and timing flags to separate DNS, TCP, TLS, and application latency.
- Use `dig` to validate DNS from multiple resolvers during cutovers.
- Follow a **structured workflow** instead of random command guessing during incidents.

11.10 Lab 7.1 — Local service bind address

1. Run `python3 -m http.server 9090` on a VM.
2. Use `ss -tlnp` to see which address it binds.
3. From another host (or curl to the VM’s private IP), confirm reachability.
4. Restart bound to `127.0.0.1` only (if supported) and observe external failure.
5. Write two sentences explaining why bind address matters for load balancers.

11.11 Lab 7.2 — DNS and latency baseline

1. Pick three production-like hostnames (or use `example.com`, `cloudflare.com`, `github.com`).
2. For each, run `dig +short` and `curl -w` timing from your lab machine.
3. Record results in a table: hostname, IP, nlookup, connect, ttfb, total.
4. Change resolver to `8.8.8.8` temporarily and repeat—note differences.

11.12 Review questions

1. What does `ss -tuln` show, and what does each letter in the flags mean?
2. How do you test whether an app listens on localhost only vs all interfaces?
3. Which `curl -w` timing field isolates DNS delay?
4. Why run `dig @8.8.8.8` during an internal DNS incident?
5. Name two files/services that affect name resolution on Linux.

Next: [*Chapter 8 — Bash Scripting for Automation*](#)

12 Chapter 8: Bash Scripting for Automation

DevOps Handbook — Part II, Pages 121–140

12.1 8.1 When bash is the right tool

Bash excels at **gluing** existing CLI tools: deploy hooks, cron jobs, CI steps, and quick operational scripts. Reach for Python or Go when you need complex logic, libraries, or testability—but bash remains the **lingua franca** of Linux automation.

Rules of thumb:

Use bash	Use Python/Go instead
Wrapping <code>kubectl</code> , <code>aws</code> , <code>curl</code> , <code>tar</code>	Parsing complex JSON at scale
CI job steps under 50 lines	Long-running services
One-off migration with tight deadline	Shared libraries across teams
Systemd <code>ExecStart</code> wrappers	Heavy unit testing requirements

12.2 8.2 Script anatomy and safety

Every production script should start with:

```
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'
```

Option	Effect
<code>set -e</code>	Exit on first command failure
<code>set -u</code>	Error on unset variables
<code>set -o pipefail</code>	Pipeline fails if any stage fails
<code>IFS</code>	Safer word splitting for loops

Optional: `set -x` for trace mode during debugging (remove before committing secrets to logs).

12.2.1 Shebang and portability

```
#!/usr/bin/env bash    # Preferred: finds bash in PATH
#!/bin/bash            # OK on most Linux; macOS may be old bash
```

Avoid bashisms in scripts that must run on **Alpine** `/bin/sh` (ash)—use `#!/bin/sh` and POSIX syntax, or explicitly require bash.

12.3 8.3 Variables, quoting, and expansion

```
name="api-server"
```

```
count=3
echo "Deploying ${name} x${count}"
echo 'Literal ${name} not expanded'

# Command substitution
today=$(date -u +%Y-%m-%d)
files=$(find /var/log -name '*.log' | wc -l)

# Default values
region="${AWS_REGION:-us-east-1}"
```

Always quote variable expansions unless you intentionally want word splitting:

```
rm -rf "$tmpdir"      # Good
rm -rf $tmpdir        # Dangerous if empty or contains spaces
```

Arrays (bash-specific):

```
services=(nginx api worker)
for svc in "${services[@]}; do
    systemctl is-active --quiet "$svc" || echo "$svc down"
done
```

12.4 8.4 Conditionals and tests

```
if [[ -f /etc/nginx/nginx.conf ]]; then
    echo "nginx config present"
elif [[ -d /opt/myapp ]]; then
    echo "app directory exists"
else
    echo "neither found" >&2
    exit 1
fi
```

Common tests:

Test	Meaning
<code>[[-f path]]</code>	Regular file exists
<code>[[-d path]]</code>	Directory exists
<code>[[-z "\$var"]]</code>	String empty
<code>[["\$a" == "\$b"]]</code>	String equality
<code>[["\$n" -gt 0]]</code>	Numeric greater than

Use `[[ ]]` in bash (not single-bracket `[ ]` unless POSIX required).

12.5 8.5 Loops and functions

```
for env in staging production; do
    echo "Deploying to $env"
    ./deploy.sh "$env"
done

while read -r line; do
    echo "Log: $line"
done < /var/log/app.log

deploy() {
    local env="$1"
    local version="$2"
```

```
    echo "deploy $version -> $env"
  }

  deploy staging "v1.2.3"
```

Use `local` inside functions to avoid polluting global scope.

12.6 8.6 Exit codes and error handling

Exit code `0` = success; non-zero = failure. CI systems and `set -e` depend on this.

```
if ! kubectl apply -f manifest.yaml; then
  echo "apply failed" >&2
  exit 1
fi

# Or with trap for cleanup
cleanup() {
  rm -f "$lockfile"
}
trap cleanup EXIT

lockfile=$(mktemp)
# ... critical section ...
```

`trap` ensures cleanup runs on exit, interrupt, or error—essential for temp files and lock release.

12.7 8.7 External commands and JSON

Prefer structured tools over fragile `grep` of command output when possible:

```
# jq for JSON (install in CI images)
instance_id=$(curl -s http://169.254.169.254/latest/meta-data/instance-id)
aws ec2 describe-instances --instance-ids "$instance_id" \
  | jq -r '.Reservations[0].Instances[0].PrivateIpAddress'

# Avoid parsing human tables with awk unless no JSON API exists
```

For simple key=value files:

```
# shellcheck disable=SC1091
source /etc/default/myapp
echo "$APP_PORT"
```

Never `source` untrusted files.

12.8 8.8 Logging and observability

```
log() {
  printf "[%s] %s\n" "$(date -u +%Y-%m-%dT%H:%M:%SZ)" "$*" >&2
}

log "Starting backup"
```

- Send logs to **stderr**; reserve stdout for data piped to other commands.
 - Include **timestamps in UTC** for correlating with centralized logging.
 - Avoid echoing secrets; redact tokens in debug output.
-

12.9 8.9 Argument parsing

```
usage() {
  echo "Usage: $0 -e ENV -v VERSION" >&2
  exit 1
}

env=""
version=""

while getopts ":e:v:h" opt; do
  case "$opt" in
    e) env="$OPTARG" ;;
    v) version="$OPTARG" ;;
    h) usage ;;
    *) usage ;;
  esac
done

[[ -n "$env" && -n "$version" ]] || usage
```

For long options (`--env`), consider `getopt` or a thin Python wrapper; keep bash CLIs simple.

12.10 8.10 Script layout in repos

```
scripts/
  deploy.sh
lib/
  common.sh      # shared functions, sourced by other scripts
README.md       # usage, env vars, examples
```

```
# deploy.sh
SCRIPT_DIR=$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)
# shellcheck source=lib/common.sh
source "$SCRIPT_DIR/lib/common.sh"
```

Run `shellcheck` in CI on all shell scripts—it catches quoting bugs and portability issues early.

12.11 8.11 Security considerations

Risk	Mitigation
Command injection	Never pass untrusted input to <code>eval</code> ; quote variables
Secrets in args	Use env vars or secret managers; avoid <code>ps</code> exposure
World-writable paths	Check <code>TMPDIR</code> ; use <code>mktemp</code>
<code>curl \</code> <code>bash</code>	Pin checksums; avoid in production install flows
SUID / excessive sudo	Run with least privilege

12.12 8.12 Chapter summary

- Start scripts with `set -euo pipefail` and quote expansions.
- Use functions, `local` , and `trap` for maintainable automation.
- Prefer **JSON APIs + jq** over parsing human-formatted CLI tables.
- Log to **stderr** with UTC timestamps; never leak secrets.
- Run **shellcheck** and document usage in README files.

12.13 Lab 8.1 — Health check script

1. Write `healthcheck.sh` that accepts `-u URL` and exits 0 only if HTTP status is 200.
 2. Use `curl -sS -o /dev/null -w '%{http_code}'`.
 3. Add `--timeout SECONDS` optional flag.
 4. Run `shellcheck` and fix all warnings.
-

12.14 Lab 8.2 — Batch service restart

1. Create `restart-services.sh` reading service names from a file (one per line).
 2. For each service: check active with `systemctl`, restart if failed, log result.
 3. Use `trap` to write a summary count on exit.
 4. Test with dummy unit names and intentional failures.
-

12.15 Review questions

1. What do `set -e`, `set -u`, and `pipefail` each prevent?
 2. Why quote `"$variable"` when passing paths to commands?
 3. When should you choose Python over bash for automation?
 4. What is the purpose of `trap cleanup EXIT`?
 5. How do you safely share functions across multiple bash scripts in a repo?
-

Next: [*Chapter 9 — Git Internals: Objects, Refs, and the DAG*](#)

13 Chapter 9: Git Internals — Objects, Refs, and the DAG

DevOps Handbook — Part III, Pages 141-158

13.1 9.1 Git is a content-addressable filesystem

At its core, Git stores **snapshots** of your project, not just diffs. Every piece of content is identified by a **SHA-1 hash** (Git is migrating to SHA-256).

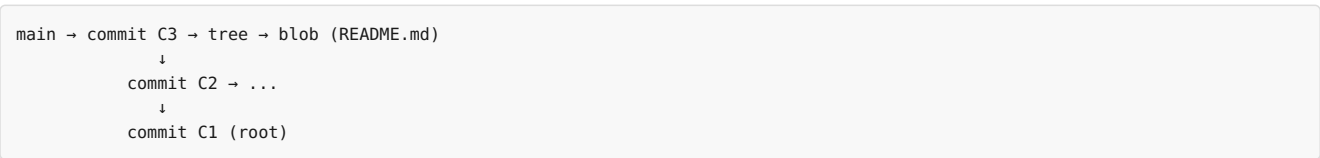
Four object types:

Type	Contains
blob	File content
tree	Directory listing (names → blobs/trees)
commit	Snapshot + metadata (author, message, parent)
tag	Named pointer to a commit (annotated tags)

13.2 9.2 How a commit is stored

When you `git add` and `git commit`:

1. Git compresses file content into **blobs**.
2. Directory structure becomes **trees**.
3. A **commit** object points to the root tree and parent commit(s).
4. Branch name (e.g., `main`) is a **ref** — a pointer to the latest commit.



This forms a **DAG** (directed acyclic graph). Merge commits have two parents.

13.3 9.3 Essential Git commands (beyond basics)

```
git log --oneline --graph --all
git show <commit>           # Commit details + diff
git diff HEAD~1             # Compare with previous commit
git stash push -m "wip"
git stash pop
git cherry-pick <commit>
git rebase main             # Replay commits on top of main
git reflog                  # History of HEAD movements (recovery!)
```

13.3.1 Undoing mistakes

```
git restore file.txt        # Discard working tree changes
git restore --staged file.txt # Unstage
git reset --soft HEAD~1     # Undo commit, keep changes staged
```

```
git reset --hard HEAD~1      # Undo commit, discard changes (dangerous)
git revert <commit>          # New commit that undoes a commit (safe for shared branches)
```

13.4 9.4 Branches and merging

```
git branch feature/login
git switch feature/login      # or: git checkout feature/login
git merge feature/login       # Merge into current branch
git merge --no-ff feature/login # Always create merge commit
```

13.4.1 Merge vs rebase

Merge	Rebase
Preserves history	Linear history
Creates merge commits	Rewrites commits
Safe for shared branches	Use only on local/feature branches

Golden rule: Never rebase commits that others have pulled.

13.5 9.5 Remote repositories

```
git remote -v
git fetch origin      # Download objects, don't merge
git pull origin main  # fetch + merge
git push origin main
git push -u origin feature/x # Set upstream
```

13.5.1 Tracking branches

`origin/main` is a **remote-tracking branch** — your local copy of what's on the server.

13.6 9.6 .gitignore and hooks

13.6.1 .gitignore

```
node_modules/
*.log
.env
dist/
.terraform/
```

13.6.2 Git hooks (`.git/hooks/`)

Scripts that run on events: `pre-commit`, `commit-msg`, `pre-push`. Used for linting, tests, and commit message format.

Example with **pre-commit** framework:

```
pip install pre-commit
# .pre-commit-config.yaml defines hooks
pre-commit install
```

13.7 9.7 Chapter summary

- Git stores blobs, trees, commits, and tags; branches are movable pointers.
 - Use `reflog` to recover “lost” commits.
 - Prefer **merge** on shared branches; **rebase** to clean up local work.
 - Automate quality with hooks and `.gitignore`.
-

13.8 Lab 9.1

1. Create a repo with 5 commits on `main`.
 2. Create a feature branch, make 2 commits, merge with `--no-ff`.
 3. Use `git log --graph --oneline --all` to visualize the DAG.
 4. Reset hard to an earlier commit, then recover using `git reflog`.
-

Next: [Chapter 20 — IaC Principles](#)

14 Chapter 10: Branching Strategies and Code Review

DevOps Handbook — Part III, Pages 159-175

14.1 10.1 Branches as a collaboration contract

Git branches are movable pointers to commits. How your team uses branches defines **release cadence**, **review culture**, and **incident risk**. There is no universal “best” strategy—only trade-offs aligned with deployment frequency, team size, and compliance needs.

Common strategies:

Strategy	Summary	Typical deployment
Trunk-based development (TBD)	Short-lived branches; merge to <code>main</code> daily	Continuous (many times/day)
GitHub Flow	<code>main</code> + feature branches + PR	On merge to <code>main</code>
Git Flow	<code>main</code> , <code>develop</code> , release/hotfix branches	Scheduled releases
Release branches	Stabilize on <code>release/x.y</code> before prod	Versioned releases

DevOps maturity correlates with **shorter branch lifetime** and **smaller changesets**.

14.2 10.2 Trunk-based development in depth

Trunk-based development keeps `main` **always releasable**. Developers integrate frequently—ideally at least once per day.

Practices:

1. **Feature flags** hide incomplete work instead of long-lived branches.
2. **Small PRs** (< 400 lines changed is a common guideline) review faster and fail less.
3. **Branch lifetime** measured in hours or days, not weeks.
4. **Automated gates** — CI must pass before merge; no manual “it works on my machine.”

```
developer → short feature branch → PR → CI → review → merge to main → deploy
```

Anti-patterns for TBD:

- “Integration branch” that lives for a sprint.
- Merging without CI green because “we’ll fix later.”
- Shared long-lived branches per environment (`dev`, `qa`, `prod` in Git)—prefer **promote artifacts**, not cherry-pick commits.

14.3 10.3 Git Flow and when it still fits

Git Flow (Vincent Driessen) defines:

- `main` — production releases only (tags).
- `develop` — integration branch for next release.
- `feature/*` — from develop, merge back to develop.
- `release/*` — stabilize version, bugfix only.
- `hotfix/*` — emergency fix from `main`, back-merge to develop.

It suits **shrink-wrapped software**, mobile apps with store review, or regulated release windows. Costs:

- Merge complexity between `develop`, `release`, and `main`.
- Slower feedback loops vs trunk-based.
- Temptation to batch large releases.

Many cloud-native teams **simplified Git Flow to GitHub Flow + tags** for semver.

14.4 10.4 Pull requests and merge mechanics

14.4.1 Merge commit vs squash vs rebase

Method	History	Pros	Cons
Merge commit	Preserves branch topology	True audit trail	Noisy graph
Squash merge	One commit on main	Clean main history	Loses granular commits
Rebase merge	Linear history	Readable log	Rewrites SHAs on branch

Platform teams often standardize on **squash to main** for product repos and **merge commits** for open-source where contributor attribution matters.

14.4.2 Protected branches

Configure on GitHub/GitLab:

- Require PR before merge to `main`.
- Require **status checks** (CI jobs) to pass.
- Require **approving reviews** (1–2 depending on risk).
- Block force-push and deletion.
- Optional: require signed commits.

14.5 10.5 Code review: purpose and culture

Code review is **not** a gate for senior engineers to flex syntax preferences. It is a **risk reduction and knowledge sharing** mechanism.

Goals:

1. Catch defects, security issues, and operational gaps before production.
2. Spread context so bus factor > 1.
3. Enforce team standards (tests, observability, docs) consistently.

14.5.1 What reviewers should examine

Area	Questions
Correctness	Does it do what the ticket says? Edge cases?
Tests	Adequate coverage for changed behavior?

Observability	Logs, metrics, traces for new paths?
Security	Injection, authz, secrets, dependencies?
Operability	Rollback, config, migrations, feature flags?
Performance	N+1 queries, unbounded loops, cache behavior?

14.5.2 Author responsibilities

- Keep PRs **focused**—one logical change.
- Write a **clear description**: problem, approach, test plan, rollout notes.
- Self-review the diff before requesting others.
- Respond to feedback with commits or comments; don't take review personally.

14.5.3 Reviewer etiquette

- Distinguish **blockers** from **nits** (nit: prefix).
- Approve when you'd be comfortable **on-call** for the change.
- Time-box reviews—slow reviews stall delivery and hurt morale.

14.6 10.6 Size and speed metrics

Google's research and DORA reports align: **small batches** and **fast review** improve stability.

Track informally:

Metric	Healthy signal
PR size (lines)	Median under ~200–400
Time to first review	< 4 business hours
Time to merge	< 1 day for most PRs
Revert rate	Low; postmortems when reverts happen

If review latency exceeds a day, fix process (rotation, alerts) before adding more tools.

14.7 10.7 Release tags and semantic versioning

Even with continuous deployment, **tags** mark releases for support and changelogs:

```
git tag -a v1.4.0 -m "Release 1.4.0: payment retry logic"
git push origin v1.4.0
```

Semantic versioning (semver):

- **MAJOR** — incompatible API change.
- **MINOR** — backward-compatible feature.
- **PATCH** — backward-compatible bug fix.

CI can build container images as `myapp:1.4.0` and `myapp:1.4.0-<git-sha>` for traceability.

14.8 10.8 Handling hotfixes

With trunk-based flow:

1. Branch from `main` (or fix forward on main directly if policy allows).
2. Minimal fix + test; fast-track review.
3. Merge; deploy; monitor.
4. Postmortem if customer-impacting.

With Git Flow:

1. `hotfix/*` from tagged `main`.
2. Merge to `main` and `develop`.
3. Tag patch release.

Either way: **deploy the same artifact** built from the merged commit—never hand-edit production.

14.9 10.9 Chapter summary

- Match branching strategy to **release frequency** and compliance—not hype.
 - Trunk-based + feature flags supports **continuous delivery**; Git Flow suits **scheduled releases**.
 - Protect `main` with **CI gates and reviews**; prefer small PRs.
 - Code review reduces risk and spreads knowledge when done with **respect and clarity**.
 - Tag releases with **semver** for supportability even if deploys are continuous.
-

14.10 Lab 10.1 — Strategy comparison

1. For a hypothetical SaaS (daily deploys) and a mobile app (monthly store release), pick a branching strategy for each.
 2. Write one paragraph justifying each choice using trade-offs from this chapter.
 3. List three protected-branch rules you would enable on `main`.
-

14.11 Lab 10.2 — Mock code review

1. Open a small PR in a practice repo (or use a provided diff).
 2. Leave three comments: one blocker, one suggestion, one nit.
 3. As author, respond and push a fix for the blocker.
 4. Record time from PR open to merge; aim under 30 minutes for this exercise.
-

14.12 Review questions

1. What is the main goal of trunk-based development?
 2. Name the five branch types in classic Git Flow.
 3. Compare squash merge vs merge commit for mainline history.
 4. List four areas a DevOps-minded code reviewer should check beyond syntax.
 5. When might Git Flow be preferable to trunk-based development?
-

Next: [Chapter 11 — GitHub/GitLab Workflows](#)

15 Chapter 11: GitHub/GitLab Workflows — PRs, Issues, Actions Intro

DevOps Handbook — Part III, Pages 176-200

15.1 11.1 Platform-native Git workflows

Git is the protocol; **GitHub** and **GitLab** are collaboration platforms. They add pull/merge requests, issue tracking, CI/CD, permissions, and audit trails. DevOps engineers live in these UIs and APIs daily.

Concept mapping:

Concept	GitHub	GitLab
Pull request	Pull Request (PR)	Merge Request (MR)
CI system	GitHub Actions	GitLab CI/CD
Issue	Issue	Issue
Code owners	CODEOWNERS	CODEOWNERS
Environments	Environments + rules	Environments + protected branches
Package registry	GHCR	GitLab Container Registry

Master one platform deeply; skills transfer with vocabulary swaps.

15.2 11.2 Issues and project hygiene

Issues are the **system of record** for work: bugs, features, tech debt, incidents.

Effective issue template fields:

Field	Purpose
Summary	One-line title
Context	User impact, links to alerts
Steps to reproduce	For bugs
Acceptance criteria	For features
Rollout / rollback	For infra changes

Link PRs to issues (`Fixes #123`) so merges auto-close tickets and audit trails stay connected.

Labels (`bug` , `infra` , `security` , `P1`) and milestones (sprints, releases) help prioritization—avoid label sprawl beyond ~15 types.

15.3 11.3 Pull request lifecycle (GitHub)

Typical flow:

```
1. Create branch from main    git checkout -b feat/retry-webhook
2. Commit with clear messages git commit -m "feat: add webhook retry"
```

3. Push and open PR `gh pr create`
4. CI runs on PR event
5. Reviewers approve
6. Merge (squash/merge/rebase per policy)
7. Deployment pipeline triggers on main

15.3.1 PR description template

```
## Summary
Brief description of change.

## Test plan
- [ ] Unit tests pass locally
- [ ] Staging deploy verified

## Rollout
Feature flag `webhook_retry` default off; enable 10% canary.

## Screenshots / logs
(if applicable)
```

Store templates in `.github/pull_request_template.md`.

15.4 11.4 CODEOWNERS and review routing

`CODEOWNERS` auto-requests reviewers based on path:

```
# .github/CODEOWNERS
* @platform-team
/terraform/ @infra-team @security
/apps/payments/ @payments-squad
```

GitLab equivalent lives at `.gitlab/CODEOWNERS` or project settings with **approval rules**.

This scales review without a central gatekeeper bottleneck—**domain experts** review their turf.

15.5 11.5 GitHub Actions introduction

GitHub Actions runs CI/CD in YAML workflows stored in `.github/workflows/`.

Minimal workflow:

```
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Run tests
```

```
run: pytest -q
```

Key concepts:

Term	Meaning
Workflow	YAML file; one or more jobs
Job	Runs on a runner VM; parallelizable
Step	Shell command or <code>uses:</code> action
Action	Reusable unit (checkout, setup-node)
Runner	GitHub-hosted or self-hosted machine

Secrets: Settings → Secrets and variables → Actions . Reference as `{{ secrets.API_TOKEN }}` —never commit secrets.

15.6 11.6 GitLab CI introduction

GitLab CI uses `.gitlab-ci.yml` at repo root:

```
# .gitlab-ci.yml
stages:
  - test
  - build

variables:
  PIP_CACHE_DIR: "${CI_PROJECT_DIR}/.cache/pip"

test:
  stage: test
  image: python:3.12-slim
  script:
    - pip install -r requirements.txt
    - pytest -q
  cache:
    paths:
      - .cache/pip

build:
  stage: build
  image: docker:24
  services:
    - docker:24-dind
  script:
    - docker build -t "${CI_REGISTRY_IMAGE}:${CI_COMMIT_SHA}" .
    - docker push "${CI_REGISTRY_IMAGE}:${CI_COMMIT_SHA}"
  only:
    - main
```

GitLab runners execute jobs; register shared or project-specific runners for private clusters.

15.7 11.7 Branch protection and environments

15.7.1 GitHub environments

Define **staging** and **production** with:

- Required reviewers before deploy job runs.
- Wait timers for change windows.
- Secrets scoped per environment.

```

jobs:
  deploy-prod:
    runs-on: ubuntu-latest
    environment: production
    needs: [test]
    steps:
      - run: ./deploy.sh production

```

15.7.2 GitLab protected environments

Similar via **protected branches**, **deployment tiers**, and **manual jobs** (when: manual).

15.8 11.8 Fork and open-source workflows

External contributors fork repos, push branches on their fork, and open PRs/MRs upstream.

Maintainers:

- Run CI on PRs from forks with **limited secrets** (GitHub uses `pull_request_target` carefully—security sensitive).
- Use **CLA bots** or DCO sign-off for legal compliance.
- Label `good first issue` to grow contributors.

15.9 11.9 Integrations DevOps teams use

Integration	Use case
Slack / Teams	PR and deploy notifications
Jira / Linear	Bi-directional issue sync
Dependabot / Renovate	Automated dependency PRs
CODE scanning	SAST on push
OIDC to cloud	Keyless AWS/GCP/Azure deploy from CI

Example OIDC pattern (GitHub → AWS): configure IAM role trust for `token.actions.githubusercontent.com`; workflow assumes role without long-lived AWS keys.

15.10 11.10 Comparing Actions vs GitLab CI (practical)

Dimension	GitHub Actions	GitLab CI
Marketplace actions	Huge ecosystem	Smaller; more shell/scripts
Self-hosted runners	Per repo/org	First-class runner manager
Monorepo	Path filters, reusable workflows	<code>rules:changes</code> , child pipelines
Container registry	GHCR integrated	Built into GitLab
Enterprise	GitHub Enterprise	GitLab dedicated/self-managed

Many organizations use **both**—product on GitHub, internal mirrors on GitLab—so learn YAML patterns for each.

15.11 11.11 CLI automation: `gh` and `glab`

```
# GitHub CLI
gh pr create --title "feat: retry" --body "See template"
gh pr checks
gh run list --workflow ci.yml

# GitLab CLI (glab)
glab mr create --title "feat: retry"
glab pipeline status
```

Scripting PR creation and CI status in onboarding docs reduces friction for new engineers.

15.12 11.12 Chapter summary

- GitHub **PRs** and GitLab **MRs** are the hub for review, CI, and audit.
 - Issues linked to PRs create traceability from idea to production.
 - **CODEOWNERS** routes review to the right experts automatically.
 - **GitHub Actions** and **GitLab CI** define pipelines as code in YAML.
 - Use **environments**, branch protection, and OIDC for safe deployments.
-

15.13 Lab 11.1 — First GitHub Actions workflow

1. Add `.github/workflows/ci.yml` to a practice repo running `shellcheck` on `scripts/`.
 2. Open a PR that breaks a script intentionally; confirm CI fails.
 3. Fix and merge; confirm CI passes on `main`.
 4. Add a job summary with `$GITHUB_STEP_SUMMARY`.
-

15.14 Lab 11.2 — GitLab mirror (optional)

1. If using GitLab, create `.gitlab-ci.yml` with lint + test stages.
 2. Configure a manual `deploy_staging` job only on `main`.
 3. Document required CI variables in README.
-

15.15 Review questions

1. What is the GitLab equivalent of a GitHub Pull Request?
 2. Where should CI workflow files live in GitHub vs GitLab repos?
 3. How does CODEOWNERS improve review routing?
 4. Why should production secrets be scoped to environments?
 5. Name two integrations that reduce toil for DevOps teams on these platforms.
-

Next: [Chapter 12 — OSI Model, TCP/IP, DNS Deep Dive](#)

16 Chapter 12: OSI Model, TCP/IP, DNS Deep Dive

DevOps Handbook — Part IV, Pages 201–225

16.1 12.1 Why models matter during incidents

When a user reports “the site is down,” you need a **mental stack** to isolate the layer at fault: physical link, IP routing, DNS, TLS, HTTP, or application logic. The **OSI model** (7 layers) and the practical **TCP/IP model** (4 layers) provide shared vocabulary for engineers and vendors.

You will not configure “Layer 5” in AWS—but you will say “this looks like L4” or “DNS is broken” and save hours.

16.2 12.2 OSI seven layers (reference)

Layer	Name	PDU	Examples
7	Application	Data	HTTP, DNS, SMTP
6	Presentation	Data	TLS encryption, compression
5	Session	Data	Session management (often folded into app)
4	Transport	Segment	TCP, UDP
3	Network	Packet	IP, ICMP, routing
2	Data Link	Frame	Ethernet, MAC, switches
1	Physical	Bits	Cables, fiber, Wi-Fi radio

Mnemonic: *All People Seem To Need Data Processing* (top-down) or *Please Do Not Throw Sausage Pizza Away* (bottom-up).

DevOps work concentrates on **Layers 3–7** in cloud environments; hardware L1/L2 is abstracted but still appears in hybrid/on-prem links.

16.3 12.3 TCP/IP four-layer model (operational)

TCP/IP layer	Maps to OSI	DevOps touchpoints
Application	5–7	HTTP APIs, DNS queries, gRPC
Transport	4	TCP ports, UDP, load balancer targets
Internet	3	IP addresses, routes, security groups
Network access	1–2	NIC, VPC, subnets, ARP

Most troubleshooting flows **top-down** (curl fails → dig → ping/trace → SG rules) or **bottom-up** when link issues suspected.

16.4 12.4 IP addressing and CIDR

IPv4 addresses are 32-bit dotted quads; subnets use **CIDR** notation:

```
10.0.1.25/24
|
└─ prefix length: 24 network bits, 8 host bits → 256 addresses
   └─ private RFC1918 range (common in VPCs)
```

Private ranges (RFC 1918):

Range	CIDR
10.0.0.0 – 10.255.255.255	10.0.0.0/8
172.16.0.0 – 172.31.255.255	172.16.0.0/12
192.168.0.0 – 192.168.255.255	192.168.0.0/16

NAT maps private IPs to public IPs at egress—understand this before debugging “works inside VPC, fails from internet.”

IPv6 adoption grows (128-bit addresses, /64 typical on subnets). Tools: `ip -6 addr`, AAAA records, dual-stack load balancers.

16.5 12.5 TCP vs UDP

Aspect	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless
Reliability	Retransmits, ordering	Best-effort
Use cases	HTTP, SSH, DB	DNS, QUIC, video, metrics
Flow control	Yes	No
DevOps example	HTTPS API	CoreDNS, StatsD

TCP connection establishment:

```
Client → SYN → Server
Client ← SYN-ACK ← Server
Client → ACK → Server
```

ss states like SYN-SENT, ESTAB, TIME-WAIT map to this lifecycle—**TIME-WAIT** accumulation can exhaust ports on high-churn load generators.

16.6 12.6 DNS: the distributed phone book

DNS resolves names to records. Critical for **every** deploy that changes IPs or uses CDNs.

16.6.1 Record types

Type	Purpose	Example
A	IPv4 address	api.example.com → 203.0.113.10
AAAA	IPv6 address	api.example.com → 2001:db8::1
CNAME	Alias to another name	www → example.com
MX	Mail server	Priority + hostname
TXT	Arbitrary text	SPF, DKIM, domain verification
NS	Delegates zone	Points to authoritative servers

SRV	Service location	Port + host for protocols
CAA	Certificate authority restriction	Security

16.6.2 Resolution chain

```

Browser/OS stub resolver
→ Recursive resolver (ISP, 8.8.8.8, corporate DNS)
  → Root (.)
    → TLD (.com)
      → Authoritative NS (example.com)
        → Answer (A record)

```

Results are **cached** at each hop per **TTL** (seconds). Low TTL (60–300) speeds cutovers; high TTL reduces query load but slows rollback.

16.7 12.7 DNS operations DevOps engineers perform

16.7.1 Cutover checklist

1. Lower TTL **24–48 hours** before change.
2. Apply new records on authoritative provider.
3. Verify with `dig @authoritative-ns` and public resolvers.
4. Monitor error rates and `curl` from multiple regions.
5. Restore TTL after stable.

16.7.2 Split-horizon and private zones

Internal services use **private DNS** (Route 53 private zones, Cloud DNS private, Azure Private DNS):

- `db.internal` resolves to `10.0.5.12` inside VPC.
- Same name may not resolve publicly—or resolves differently (**split brain** by design).

Document which resolvers apps use (`/etc/resolv.conf` , CoreDNS `forward` plugin).

16.8 12.8 ICMP and path discovery

ICMP supports diagnostics (ping, unreachable messages). Many firewalls block ICMP—“ping fails” does not always mean host down.

```

ping -c 4 203.0.113.10
traceroute 203.0.113.10
tracepath 203.0.113.10  # No root required on Linux
mtr 203.0.113.10        # Interactive (install mtr)

```

Use traceroute when latency spikes or routing loops suspected after VPC peering changes.

16.9 12.9 ARP and L2 in VPC context

ARP maps IPv4 to MAC on local subnet. In cloud VPCs, you rarely touch ARP directly—but **security groups** and **NACLs** behave like L3/L4 filters on virtual interfaces.

On-prem hybrid: VPN/ Direct Connect extends L3; understand whether broadcasts traverse (usually not).

16.10 12.10 Common failure patterns

Symptom	Likely layer	First checks
Unknown host	DNS	<code>dig</code> , resolver config
Connection timed out	L3/L4 firewall/route	SG, NACL, <code>ip route get</code>
Connection refused	L4 app not listening	<code>ss -tlnp</code> , bind address
SSL certificate error	L6/TLS	cert SANs, expiry, chain
HTTP 502/504	L7/L4 LB	target health, upstream timeout

16.11 12.11 Chapter summary

- OSI provides vocabulary; **TCP/IP** maps to daily cloud networking.
- **CIDR** and private IP planning prevent overlapping VPC peering disasters.
- **TCP** reliability vs **UDP** speed trade-offs drive protocol choice.
- **DNS TTL**, record types, and resolver chains explain most “works after 5 minutes” incidents.
- Troubleshoot **top-down** with `curl` , `dig` , `ss` , and cloud network ACLs.

16.12 Lab 12.1 — DNS TTL experiment

1. Create a test subdomain with TTL 60 on your DNS provider.
2. Point A record to IP A; verify with `dig +trace` or `dig @ns .`
3. Change to IP B; measure propagation time from three resolvers.
4. Repeat with TTL 3600 in a lab zone (document expected delay).

16.13 Lab 12.2 — Layer mapping drill

1. For five past incidents (or hypothetical scenarios), identify the failing layer.
2. Write the first diagnostic command you would run for each.
3. Peer-review with a colleague or checklist.

16.14 Review questions

1. Map HTTP, TCP, and IP to OSI layers.
2. What is the difference between A and CNAME records?
3. Why lower DNS TTL before a migration?
4. What does TCP `TIME-WAIT` indicate, and when is it a problem?
5. Explain split-horizon DNS in one paragraph.

Next: [Chapter 13 — HTTP/HTTPS, TLS, and Certificates](#)

17 Chapter 13: HTTP/HTTPS, TLS, and Certificates

DevOps Handbook — Part IV, Pages 226–245

17.1 13.1 HTTP: the application protocol of the web

HTTP (Hypertext Transfer Protocol) is a stateless request/response protocol at OSI Layer 7. DevOps engineers encounter HTTP in health checks, API gateways, ingress controllers, and CI cache headers daily.

17.1.1 Request structure

```
GET /api/v1/users HTTP/1.1
Host: api.example.com
Accept: application/json
Authorization: Bearer eyJhbGc...
User-Agent: curl/8.5.0
```

17.1.2 Response structure

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 42
Cache-Control: no-store

{"users":[]}
```

17.1.3 Common methods

Method	Idempotent	Typical use
GET	Yes	Read resource
POST	No	Create, actions
PUT	Yes	Replace resource
PATCH	No*	Partial update
DELETE	Yes	Remove resource
HEAD	Yes	Headers only (health)

*PATCH idempotency depends on implementation.

17.1.4 Status codes (DevOps essentials)

Code	Meaning	Action
200	OK	Success
301/302	Redirect	Check Location header
400	Bad request	Client payload issue
401/403	Authn/authz	Credentials or policy
404	Not found	Route or resource missing
429	Rate limited	Backoff, quotas
500	Server error	App bug or dependency
502/503/504	Gateway/upstream	LB, overload, timeout

17.2 13.2 HTTP/1.1 vs HTTP/2 vs HTTP/3

Version	Transport	Notes
HTTP/1.1	TCP	Head-of-line blocking on one connection
HTTP/2	TCP + TLS (common)	Multiplexed streams, header compression
HTTP/3	QUIC (UDP)	Faster handshakes, mobile lossy networks

Ingress controllers (nginx, Envoy, ALB) terminate HTTP/2 from clients and may speak HTTP/1.1 to legacy backends —know your **end-to-end protocol** when debugging streaming and timeouts.

17.3 13.3 HTTPS and TLS

HTTPS is HTTP over **TLS** (Transport Layer Security). TLS provides:

1. **Confidentiality** — encrypted payload.
2. **Integrity** — tamper detection.
3. **Authentication** — server (and optionally client) identity via certificates.

17.3.1 TLS handshake (simplified)

ClientHello (supported ciphers, SNI)
← ServerHello, certificate, key exchange
Client verifies cert chain → generates session keys
Encrypted application data (HTTP)

SNI (Server Name Indication) lets one IP host many HTTPS sites—virtual hosting at TLS layer. Missing SNI breaks multi-tenant ingress.

Inspect with:

```
openssl s_client -connect api.example.com:443 -servername api.example.com </dev/null 2>/dev/null \  
| openssl x509 -noout -subject -issuer -dates  
curl -vI https://api.example.com
```

17.4 13.4 Certificates and PKI

A **certificate** binds a public key to identities (DNS names in **SAN**—Subject Alternative Name). Browsers trust **Certificate Authorities (CAs)** rooted in the system trust store.

Component	Role
Private key	Stays secret on server or HSM
CSR	Certificate Signing Request → sent to CA
Leaf cert	Presented to clients
Intermediate + root	Chain of trust to a trusted root

17.4.1 Let's Encrypt and ACME

ACME automates issuance (cert-manager on Kubernetes, Caddy, Traefik). Certs are short-lived (90 days)—automation is mandatory.

```
# cert-manager Certificate (illustrative)
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: api-tls
spec:
  secretName: api-tls-secret
  dnsNames:
    - api.example.com
  issuerRef:
    name: letsencrypt-prod
    kind: ClusterIssuer
```

17.5 13.5 mTLS (mutual TLS)

Standard TLS authenticates the **server** to the client. **mTLS** also requires clients to present certificates—common in service meshes (Istio, Linkerd) and zero-trust internal APIs.

DevOps tasks:

- Rotate client certs before expiry.
- Store CA bundles in trust stores for sidecars.
- Monitor handshake failure metrics.

17.6 13.6 Cipher suites and hardening

Weak ciphers and old TLS versions (1.0, 1.1) are disabled in modern compliance baselines. Prefer **TLS 1.2+** and **TLS 1.3** where supported.

Test external posture:

```
# Using testssl.sh or SSL Labs API for audits
nmap --script ssl-enum-ciphers -p 443 api.example.com
```

Align ingress `ssl_protocols` / `ssl_ciphers` with security team policy—document exceptions.

17.7 13.7 HTTP headers DevOps should know

Header	Purpose
Host	Virtual host routing
X-Forwarded-For / Forwarded	Client IP through proxies
X-Request-ID	Correlation across services
Strict-Transport-Security	Force HTTPS (HSTS)
Content-Security-Policy	Mitigate XSS
Cache-Control	CDN and browser caching

Misconfigured **X-Forwarded-For** trust allows IP spoofing in rate limits—only trust headers from known L7 proxies.

17.8 13.8 Timeouts, keep-alive, and load balancers

Setting	Symptom if wrong
Client idle timeout	Premature connection close
LB idle timeout	502 during long requests
Upstream read timeout	504 gateway timeout
Keep-alive	Extra TCP/TLS handshakes if disabled

AWS ALB default idle timeout is 60s—long-running exports may need tuning on both app and LB.

17.9 13.9 Debugging TLS in production

Checklist:

1. **Expiry** — `openssl x509 -enddate`; alert 30 days ahead.
 2. **Chain** — missing intermediate → Android/old clients fail.
 3. **SAN** — cert must cover exact hostname (`api` vs `api.internal`).
 4. **Clock skew** — NTP drift causes “cert not yet valid.”
 5. **SNI** — curl without `-H 'Host: '` or wrong `-servername` .
- ```
echo | openssl s_client -connect wrong-host.example.com:443 -servername api.example.com 2>&1 | grep subject
```

## 17.10 13.10 Chapter summary

- HTTP methods and status codes drive **health checks and SLOs**.
- **TLS** encrypts and authenticates; understand handshake, SNI, and chains.
- Automate cert lifecycle with **ACME/cert-manager**—manual renewals fail at scale.
- **mTLS** secures east-west traffic in meshes and zero-trust designs.
- Align **timeouts and forwarded headers** between apps, proxies, and LBs.

## 17.11 Lab 13.1 — Inspect a live certificate

1. Run `openssl s_client` against three HTTPS sites you operate or public APIs.
2. Record issuer, SANs, and expiry dates in a table.
3. Simulate SNI: connect by IP with `-servername` and observe cert change on shared IP hosts.

## 17.12 Lab 13.2 — Broken chain simulation

1. In a lab nginx/ingress, serve a leaf cert without intermediate bundle.
2. Verify failure with `curl` on a minimal trust store container.
3. Fix by appending intermediate; document the full chain file order.

## 17.13 Review questions

1. What is the difference between HTTP 502 and 504?

2. What role does SNI play on shared load balancer IPs?
  3. Name the three security properties TLS provides.
  4. Why are short-lived Let's Encrypt certs acceptable?
  5. What header carries client IP through proxies, and what is the spoofing risk?
- 

Next: [\*Chapter 14 — Load Balancing, Proxies, and CDN\*](#)

# 18 Chapter 14: Load Balancing, Proxies, and CDN

DevOps Handbook — Part IV, Pages 246–265

## 18.1 14.1 Why load balancing exists

Single servers fail and saturate. **Load balancers** distribute traffic across healthy backends, terminate TLS, enforce routing rules, and provide a stable **virtual IP/DNS name** while instances churn behind them.

Goals:

| Goal            | Mechanism                                      |
|-----------------|------------------------------------------------|
| Availability    | Health checks remove bad targets               |
| Scalability     | Add backends horizontally                      |
| Maintainability | Rolling deploys without client reconfiguration |
| Security        | WAF, TLS offload, DDoS absorption (with CDN)   |

## 18.2 14.2 Layer 4 vs Layer 7 load balancing

| Type          | OSI layer   | Sees                        | Routing basis           | Examples                  |
|---------------|-------------|-----------------------------|-------------------------|---------------------------|
| L4 (NLB)      | Transport   | IP, port, TCP/UDP           | IP:port, flow hash      | AWS NLB, HAProxy TCP mode |
| L7 (ALB/HTTP) | Application | HTTP headers, path, cookies | URL path, Host, headers | AWS ALB, nginx, Envoy     |

**L4** is faster and protocol-agnostic—great for databases, gRPC-TLS passthrough, or extreme throughput.

**L7** enables path-based routing ( `/api` → service A ), host-based routing ( `api.` vs `www.` ), sticky sessions, and request manipulation.

Many architectures stack both: **CDN/L7 edge** → **L7 ingress** → **L4 service mesh**.

## 18.3 14.3 Load balancing algorithms

| Algorithm         | Behavior                   | Use when                   |
|-------------------|----------------------------|----------------------------|
| Round robin       | Cycle targets evenly       | Homogeneous backends       |
| Least connections | Send to least busy         | Long-lived connections     |
| Weighted          | Favor stronger instances   | Mixed instance sizes       |
| IP hash           | Same client → same backend | Simple session stickiness  |
| Random / Maglev   | Hash tables for stability  | Large clusters (Google LB) |

**Health checks** must reflect **application** health—not just TCP open port. Return 200 on `/health` with dependency checks (DB ping) when appropriate.



Unhealthy threshold and interval tune flapping vs slow detection:

```
interval: 10s
timeout: 5s
healthy_threshold: 2
unhealthy_threshold: 3
```

## 18.4 14.4 Reverse proxies vs forward proxies

|              | Reverse proxy                 | Forward proxy               |
|--------------|-------------------------------|-----------------------------|
| Sits         | In front of servers           | In front of clients         |
| Clients know | Backend exists (often hidden) | Proxy exists                |
| Examples     | nginx, Traefik, ALB           | Corporate HTTP proxy, Squid |
| DevOps use   | Ingress, API gateway          | Egress control from CI/VPC  |

A **reverse proxy** terminates client connections and opens new ones to upstreams—enabling TLS offload, buffering, and rate limits.

## 18.5 14.5 Common proxy software

### 18.5.1 nginx (ingress workhorse)

```
upstream api_backend {
 least_conn;
 server 10.0.1.10:8080 max_fails=3 fail_timeout=30s;
 server 10.0.1.11:8080 max_fails=3 fail_timeout=30s;
}

server {
 listen 443 ssl http2;
 server_name api.example.com;

 location / {
 proxy_pass http://api_backend;
 proxy_set_header Host $host;
 proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
 proxy_set_header X-Forwarded-Proto $scheme;
 }
}
```

### 18.5.2 HAProxy

Popular for TCP and HTTP with advanced health checks and stick tables.

### 18.5.3 Envoy

Data plane for Istio and modern API gateways; dynamic config via xDS.  
Choose based on team expertise and platform (managed cloud LB vs self-run on VMs/K8s).

## 18.6 14.6 Session persistence (stickiness)

Stateful apps without shared session store may need **sticky sessions**:

- **Cookie-based** (ALB insert cookie).

- **Source IP affinity** (breaks behind carrier NAT—fragile).

Prefer **external session store** (Redis) over stickiness when possible—stickiness complicates deploys and failover.

---

## 18.7 14.7 Content Delivery Networks (CDN)

A **CDN** caches static and cacheable dynamic content at **edge PoPs** close to users—reducing latency and origin load.

| Benefit            | Explanation                              |
|--------------------|------------------------------------------|
| <b>Latency</b>     | RTT to edge << RTT to origin region      |
| <b>Scale</b>       | Absorb traffic spikes at edge            |
| <b>DDoS</b>        | Distributed scrubbing (vendor-dependent) |
| <b>TLS at edge</b> | Terminate close to user                  |

Providers: CloudFront, Fastly, Cloudflare, Akamai, Google Cloud CDN.

### 18.7.1 Cache control

Origin sets policy via headers:

```
Cache-Control: public, max-age=3600
Cache-Control: no-store
ETag: "abc123"
```

**Cache key** often includes path, query string rules, and `Vary` headers—misconfiguration serves wrong content to wrong users (classic bug: caching authenticated pages without `private`).

### 18.7.2 Purge and invalidation

Deploy new `app.js` → purge `/static/*` or use **filename hashing** ( `app.a1b2c3.js` ) so cache busting is automatic.

---

## 18.8 14.8 Global load balancing and failover

**Geo DNS** or **anycast** routes users to nearest healthy region:

```
User in EU → GSLB → eu-west-1 (healthy)
Failover → us-east-1 if region unhealthy
```

Requires **state replication** or accept read-only failover; DNS TTL and health probe design determine RTO.

---

## 18.9 14.9 Kubernetes ingress pattern

```
Internet → Cloud LB → Ingress Controller (nginx/Envoy)
→ Service (ClusterIP) → Pod endpoints
```

`Ingress` resources define host/path rules; **Gateway API** is the evolving standard with clearer L4/L7 separation.

Service mesh adds **sidecar proxies** for east-west L7 policy without changing Service types.

---

## 18.10 14.10 Observability for load balancers

Monitor:

| Metric                      | Indicates                       |
|-----------------------------|---------------------------------|
| Request count / latency p99 | User experience                 |
| 5xx rate                    | Upstream or LB misconfig        |
| Healthy host count          | Deploy or health check failures |
| TLS handshake time          | Cert or cipher issues           |
| Cache hit ratio             | CDN efficiency                  |

Log **X-Request-ID** from edge through origin for distributed traces.

---

## 18.11 14.11 Chapter summary

- **L4** balances connections; **L7** routes HTTP semantics—choose per use case.
  - Health checks and algorithms must match **workload shape** (short vs long connections).
  - **Reverse proxies** enable TLS, routing, and security at the edge.
  - **CDNs** cache at PoPs—control with headers, hashed assets, and purge strategy.
  - Instrument LB and CDN metrics as part of **SLO dashboards**.
- 

## 18.12 Lab 14.1 — nginx upstream failover

1. Run two backend containers on ports 8081/8082 returning different bodies.
  2. Configure nginx upstream with one server marked down.
  3. curl through nginx; confirm traffic to healthy only.
  4. Stop healthy container; observe 502 until nginx marks unhealthy.
- 

## 18.13 Lab 14.2 — CDN cache headers

1. Serve a static file from a simple web server with `Cache-Control: max-age=60`.
  2. If CDN unavailable, use two curl requests and `Age` header via local cache proxy—or document expected CDN behavior.
  3. Change file content without purge; explain stale risk.
  4. Switch to content-hashed filename; show safe long `max-age`.
- 

## 18.14 Review questions

1. When would you choose L4 over L7 load balancing?
  2. Why is TCP-open health check insufficient for many apps?
  3. Difference between reverse and forward proxy?
  4. How do cache keys cause authenticated content leaks?
  5. What Kubernetes component typically replaces raw nginx on bare VMs?
- 

Next: [Chapter 15 — Cloud Networking Patterns \(VPC overview\)](#)

# 19 Chapter 15: Cloud Networking Patterns — VPC Overview

DevOps Handbook — Part IV, Pages 266–280

## 19.1 15.1 VPC: your private network in the cloud

A **Virtual Private Cloud (VPC)** is an isolated virtual network in AWS, GCP (**VPC**), or Azure (**VNet**). You define IP ranges, subnets, routing, firewalls, and attachments to on-prem or other clouds.

DevOps engineers provision VPCs via Terraform, deploy workloads into **private subnets**, and expose services through **controlled ingress**—not public IPs on every VM.

Conceptual map (AWS-centric; others analogous):

| AWS                   | GCP                     | Azure            |
|-----------------------|-------------------------|------------------|
| VPC                   | VPC                     | Virtual Network  |
| Subnet                | Subnet                  | Subnet           |
| Security Group        | Firewall rule (VPC)     | NSG              |
| NACL                  | — (less common pattern) | NACL (optional)  |
| Internet Gateway      | —                       | Internet Gateway |
| NAT Gateway           | Cloud NAT               | NAT Gateway      |
| Route 53 private zone | Cloud DNS private       | Private DNS zone |

## 19.2 15.2 CIDR design fundamentals

Plan **non-overlapping** CIDR blocks before peering or VPN:

```
VPC: 10.0.0.0/16
├─ public-a: 10.0.0.0/24 (AZ a) – LB, NAT
├─ public-b: 10.0.1.0/24 (AZ b)
├─ private-a: 10.0.10.0/24 (AZ a) – apps, workers
├─ private-b: 10.0.11.0/24 (AZ b)
└─ data-a/b: 10.0.20.0/24, 10.0.21.0/24 – databases
```

Rules of thumb:

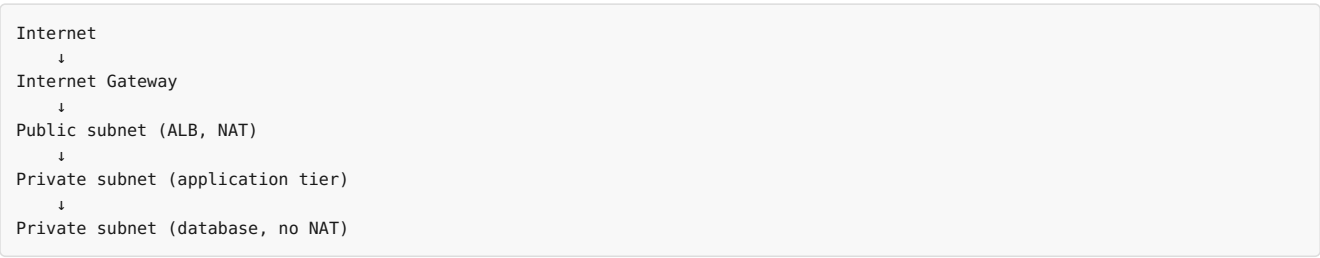
- Leave **spare capacity** in VPC CIDR for growth.
- **Multi-AZ** subnets for HA—one failure domain per AZ.
- Put **internet-facing LBs** in public subnets; **instances** in private subnets without public IPs.

## 19.3 15.3 Public vs private subnets

| Subnet type | Default route                      | Typical resources                    |
|-------------|------------------------------------|--------------------------------------|
| Public      | 0.0.0.0/0 → Internet Gateway       | NAT GW, public ALB, bastion (legacy) |
| Private     | 0.0.0.0/0 → NAT Gateway (optional) | App servers, EKS nodes, RDS          |

|  |  |  |
|--|--|--|
|  |  |  |
|--|--|--|

Private subnets **without NAT** cannot reach the internet—ideal for data layers with VPC endpoints only.



## 19.4 15.4 Security groups vs NACLs

### 19.4.1 Security groups (stateful)

- Instance/network interface level.
- **Allow rules only**; implicit deny.
- **Stateful**—return traffic automatically allowed.
- Example: allow 443 from 0.0.0.0/0 to ALB SG; allow 8080 from ALB SG to app SG.

### 19.4.2 Network ACLs (stateless, AWS)

- Subnet level; optional deny rules.
- **Stateless**—must allow ephemeral return ports explicitly if used.
- Less common in day-to-day DevOps than SGs.

**Defense in depth:** SG for fine-grained app rules; NACL for coarse subnet blocks if compliance requires.

## 19.5 15.5 NAT, egress, and VPC endpoints

**NAT Gateway** lets private instances initiate outbound internet (package updates, external APIs) without inbound exposure. Costs and bandwidth add up—monitor NAT metrics.

**VPC endpoints** (AWS **Interface** and **Gateway** endpoints) keep traffic on AWS backbone:

| Endpoint type | Example                   | Avoids               |
|---------------|---------------------------|----------------------|
| Gateway       | S3, DynamoDB              | Internet for AWS API |
| Interface     | ECR, Secrets Manager, STS | Public NAT egress    |

For EKS pulling images from ECR, **interface endpoints** reduce NAT dependency and improve security posture.

## 19.6 15.6 Hybrid and multi-VPC connectivity

| Pattern                     | Use case                                        |
|-----------------------------|-------------------------------------------------|
| <b>VPC peering</b>          | Connect two VPCs (non-transitive)               |
| <b>Transit Gateway</b>      | Hub-and-spoke many VPCs                         |
| <b>VPN / Direct Connect</b> | On-prem to cloud                                |
| <b>PrivateLink</b>          | Consumer access to service without full peering |

Peering requires **non-overlapping CIDRs**. Transitive routing pitfalls break naive “peer everything” designs—document a **network topology diagram**.

---

## 19.7 15.7 Kubernetes networking in VPC

Managed Kubernetes (EKS, GKE, AKS) integrates CNI plugins assigning pod IPs from VPC or overlay ranges.

DevOps concerns:

- **Pod CIDR vs VPC CIDR**—must not overlap peered networks.
  - **Security groups for pods** (where supported) replace blanket node SG rules.
  - **LoadBalancer Service** provisions cloud LB into public/private subnets per annotation.
  - **Internal-only services** use internal LB annotations and private DNS.
- 

## 19.8 15.8 IaC example (Terraform sketch)

```
resource "aws_vpc" "main" {
 cidr_block = "10.0.0.0/16"
 enable_dns_hostnames = true
 enable_dns_support = true
}

resource "aws_subnet" "private_a" {
 vpc_id = aws_vpc.main.id
 cidr_block = "10.0.10.0/24"
 availability_zone = "us-east-1a"
}

resource "aws_security_group" "app" {
 name = "app-sg"
 vpc_id = aws_vpc.main.id

 ingress {
 from_port = 8080
 to_port = 8080
 protocol = "tcp"
 security_groups = [aws_security_group.alb.id]
 }

 egress {
 from_port = 0
 to_port = 0
 protocol = "-1"
 cidr_blocks = ["0.0.0.0/0"]
 }
}
```

Store network diagrams alongside Terraform for reviewers who don’t read HCL daily.

---

## 19.9 15.9 Operational checklist

Before production launch:

1. **No 0.0.0.0/0 SSH** on app SGs—use SSM Session Manager or bastion with MFA.
  2. **Flow logs** enabled on VPC or key subnets for forensics.
  3. **Route tables** verified per subnet—accidental IGW on DB subnet is a data breach.
  4. **DNS** private zones linked to VPC; resolver rules for hybrid.
  5. **Quota limits** checked (EIPs, NAT GW per AZ, LB limits).
-

## 19.10 15.10 Chapter summary

- **VPC** isolates cloud resources with subnets, routes, and firewalls.
  - **Public subnets** expose ingress paths; **private subnets** protect workloads.
  - **Security groups** are stateful primary filters; know when **NACLs** apply.
  - **NAT vs VPC endpoints** trade cost, security, and simplicity for egress.
  - Plan **CIDR and peering** early; overlaps are painful to fix later.
- 

## 19.11 Lab 15.1 — Paper VPC design

1. Design a VPC for a three-tier app (web, app, DB) in two AZs.
  2. Assign CIDR blocks and list route table entries for each subnet type.
  3. Document SG rules as a matrix: source → destination → port.
  4. Identify which traffic uses IGW, NAT, and VPC endpoints.
- 

## 19.12 Lab 15.2 — Terraform mini-VPC (optional cloud)

1. Apply a module creating VPC + one public + one private subnet.
  2. Launch a test instance in private subnet without public IP.
  3. Verify outbound via NAT with `curl` and inbound denial from internet.
  4. Tear down to avoid charges.
- 

## 19.13 Review questions

1. What is the difference between public and private subnets at the routing layer?
  2. Why are security groups described as stateful?
  3. When would you use a VPC endpoint instead of NAT for S3 access?
  4. What problem does non-overlapping CIDR solve in peering?
  5. Where should an internet-facing ALB sit in a well-designed VPC?
- 

Next: [Chapter 16 — Python for DevOps](#)

# 20 Chapter 16: Python for DevOps — boto3, requests, pathlib

DevOps Handbook — Part V, Pages 281–305

## 20.1 16.1 Why Python dominates DevOps automation

Python balances readability, rich libraries, and fast iteration. It powers Ansible modules, AWS Lambda, glue scripts, data migration tools, and CI steps. You will use it alongside bash (Chapter 8) for anything beyond simple command wrapping.

Core libraries for this handbook:

| Library     | Purpose                     |
|-------------|-----------------------------|
| boto3       | AWS API SDK                 |
| requests    | HTTP client                 |
| pathlib     | Modern filesystem paths     |
| json / yaml | Config and API payloads     |
| subprocess  | Safe CLI invocation         |
| logging     | Structured operational logs |

## 20.2 16.2 Project layout and virtual environments

Never install packages globally on shared CI runners without pinning. Use **venv** per project:

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install boto3 requests
pip freeze > requirements.txt
```

Minimal repo layout:

```
tools/
aws_inventory/
 __init__.py
 main.py
 requirements.txt
 README.md
tests/
 test_inventory.py
```

Pin versions in `requirements.txt` for reproducible CI:

```
boto3==1.34.162
requests==2.32.3
```

## 20.3 16.3 pathlib: filesystem without footguns



`pathlib.Path` replaces brittle string path concatenation:

```
from pathlib import Path

root = Path(__file__).resolve().parent
config_dir = root / "config"
log_file = Path("/var/log/app") / "deploy.log"

if not config_dir.exists():
 config_dir.mkdir(parents=True, exist_ok=True)

for yml_file in config_dir.glob("*.yaml"):
 print(yml_file.read_text(encoding="utf-8")[:200])

Safe join – no double slashes
staging = root / ".." / "staging" / "inventory.json"
staging = staging.resolve()
```

Use `.read_text()`, `.write_text()`, `.iterdir()`, and `.glob()` instead of `os.path` when starting new scripts.

---

## 20.4 16.4 requests: HTTP for APIs and health checks

```
import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

session = requests.Session()
retries = Retry(total=3, backoff_factor=0.5, status_forcelist=[502, 503, 504])
session.mount("https://", HTTPAdapter(max_retries=retries))

def get_health(url: str, timeout: float = 5.0) -> bool:
 try:
 resp = session.get(url, timeout=timeout)
 resp.raise_for_status()
 return resp.json().get("status") == "ok"
 except requests.RequestException as exc:
 print(f"health check failed: {exc}")
 return False

if __name__ == "__main__":
 ok = get_health("https://api.example.com/health")
 raise SystemExit(0 if ok else 1)
```

Best practices:

- Always set **timeouts** ( `timeout=(connect, read)` ).
- Use `raise_for_status()` or explicit status handling.
- Reuse **Session** for connection pooling in loops.
- Never disable TLS verify in production ( `verify=False` is debug-only).

---

## 20.5 16.5 boto3: AWS automation

boto3 uses **botocore** for API calls. Credentials resolve via environment variables, `~/.aws/credentials`, or IAM role on EC2/EKS.

```
import boto3
from botocore.exceptions import ClientError

def list_stopped_instances(region: str = "us-east-1"):
 ec2 = boto3.client("ec2", region_name=region)
 paginator = ec2.get_paginator("describe_instances")
 stopped = []

 for page in paginator.paginate(
 Filters=[{"Name": "instance-state-name", "Values": ["stopped"]}]):
 pass
```

```

 for reservation in page["Reservations"]:
 for inst in reservation["Instances"]:
 name = next(
 (t["Value"] for t in inst.get("Tags", []) if t["Key"] == "Name"),
 "unnamed",
)
 stopped.append({"id": inst["InstanceId"], "name": name})
 return stopped

def stop_instance(instance_id: str, region: str = "us-east-1") -> None:
 ec2 = boto3.client("ec2", region_name=region)
 try:
 ec2.stop_instances(InstanceIds=[instance_id])
 except ClientError as e:
 raise RuntimeError(f"stop failed: {e}") from e

```

### 20.5.1 Resource vs client

| API      | Style                    | Use                              |
|----------|--------------------------|----------------------------------|
| client   | Low-level dict responses | Full API coverage                |
| resource | Object-oriented          | Convenient for EC2/S3 common ops |

```

s3 = boto3.resource("s3")
for bucket in s3.buckets.all():
 print(bucket.name)

```

Handle `ClientError` with error codes ( `AccessDenied` , `InvalidParameterValue` ) for actionable messages.

## 20.6 16.6 Idempotency and dry-run patterns

DevOps scripts should be safe to re-run:

```

def ensure_tag(ec2_client, instance_id: str, key: str, value: str) -> None:
 ec2_client.create_tags(
 Resources=[instance_id],
 Tags=[{"Key": key, "Value": value}],
)
 # create_tags is idempotent for same key/value on AWS

```

Add `--dry-run` CLI flags for destructive operations; boto3 supports `DryRun` on many calls to validate permissions without effect.

## 20.7 16.7 CLI entry points with argparse

```

import argparse
from pathlib import Path

def main() -> int:
 parser = argparse.ArgumentParser(description="Deploy artifact to S3")
 parser.add_argument("--bucket", required=True)
 parser.add_argument("--file", type=Path, required=True)
 parser.add_argument("--region", default="us-east-1")
 args = parser.parse_args()

 if not args.file.is_file():
 parser.error(f"file not found: {args.file}")

 # upload logic here
 return 0

if __name__ == "__main__":
 raise SystemExit(main())

```

Use `raise SystemExit(main())` so CI captures exit codes.

---

## 20.8 16.8 Logging instead of print

```
import logging

logging.basicConfig(
 level=logging.INFO,
 format="%(asctime)s %(levelname)s %(message)s",
)
log = logging.getLogger(__name__)

log.info("starting sync", extra={"bucket": "my-bucket"})
```

Structured JSON logging (for Loki/CloudWatch) can use `python-json-logger` in larger services.

---

## 20.9 16.9 Testing Python automation

```
tests/test_health.py
from unittest.mock import patch, MagicMock
import inventory # your module

def test_get_health_ok():
 mock_resp = MagicMock()
 mock_resp.json.return_value = {"status": "ok"}
 mock_resp.raise_for_status = MagicMock()
 with patch("inventory.session.get", return_value=mock_resp):
 assert inventory.get_health("http://test") is True
```

Run with `pytest`; mock external APIs—never hit production AWS in unit tests.

---

## 20.10 16.10 Security notes

- Store secrets in env vars or secret managers—not in code.
  - Use **least-privilege IAM** policies scoped to required actions.
  - Validate input paths to prevent `../..` escapes when operating on filesystems.
  - Scan dependencies with **pip-audit** or SCA in CI.
- 

## 20.11 16.11 Chapter summary

- Use **venv + pinned requirements** for reproducible automation.
  - **pathlib** simplifies safe filesystem operations.
  - **requests** needs timeouts, retries, and explicit error handling.
  - **boto3** clients/resources automate AWS—paginate and handle `ClientError`.
  - Add **argparse**, logging, dry-run, and **pytest** mocks for production-quality scripts.
- 

## 20.12 Lab 16.1 — EC2 inventory script

1. Write `list_ec2_by_tag.py --tag-key Environment --tag-value staging`.
2. Output JSON list of instance ID, type, state, private IP.
3. Run with read-only IAM; add `--output table` optional format.

4. Add pytest with mocked boto3 responses.
- 

## 20.13 Lab 16.2 — Health gate for CI

1. Implement `check_health.py --url URL --retries 5 --interval 10`.
  2. Exit non-zero on failure for use in GitHub Actions step.
  3. Document usage in README with example workflow snippet.
- 

## 20.14 Review questions

1. Why use `requests.Session` in a loop?
  2. Difference between boto3 client and resource?
  3. How does pathlib improve on string path joins?
  4. What three elements belong in every HTTP call in automation?
  5. How do you test boto3 code without calling AWS?
- 

Next: [Chapter 17 — Go for CLI Tools and Operators](#)

# 21 Chapter 17: Go for CLI Tools and Operators

DevOps Handbook — Part V, Pages 306–325

## 21.1 17.1 Why Go in the DevOps toolchain

Go (Golang) compiles to **single static binaries** with no runtime dependency—ideal for CLIs shipped to servers, minimal container images, and Kubernetes **operators**. The cloud-native ecosystem (Docker, Kubernetes, Terraform, Prometheus, Helm) is largely Go.

| Strength              | DevOps use                           |
|-----------------------|--------------------------------------|
| Fast compile          | Quick iteration in CI                |
| Static binary         | Copy to jump host, air-gapped env    |
| Concurrency           | Parallel checks across hosts/regions |
| Strong stdlib         | HTTP, TLS, encoding/json             |
| kubebuilder ecosystem | Custom controllers and operators     |

Python wins for quick scripts; Go wins for **distributable tools** and **control loops**.

## 21.2 17.2 Module setup

Go 1.21+ with modules (no `$GOPATH` src layout required):

```
mkdir -p ~/go/devops-check && cd ~/go/devops-check
go mod init github.com/you/devops-check
```

```
devops-check/
cmd/
 devops-check/
 main.go
internal/
 health/
 health.go
go.mod
go.sum
```

Build:

```
go build -o bin/devops-check ./cmd/devops-check
GOOS=linux GOARCH=amd64 go build -o bin/devops-check-linux ./cmd/devops-check
```

## 21.3 17.3 A minimal CLI with cobra (pattern)

Many projects use **spf13/cobra** for subcommands and flags:

```
// cmd/devops-check/main.go
package main

import (
 "fmt"
 "os"
```

```

 "github.com/spfl3/cobra"
 "github.com/you/devops-check/internal/health"
)

func main() {
 var url string
 var timeout int

 root := &cobra.Command{
 Use: "devops-check",
 Short: "Operational health utilities",
 }

 checkCmd := &cobra.Command{
 Use: "http",
 Short: "HTTP health check",
 RunE: func(cmd *cobra.Command, args []string) error {
 ok, err := health.CheckHTTP(url, timeout)
 if err != nil {
 return err
 }
 if !ok {
 return fmt.Errorf("unhealthy response from %s", url)
 }
 fmt.Println("OK")
 return nil
 },
 }
 checkCmd.Flags().StringVar(&url, "url", "", "URL to check")
 checkCmd.Flags().IntVar(&timeout, "timeout", 5, "timeout seconds")
 _ = checkCmd.MarkFlagRequired("url")

 root.AddCommand(checkCmd)
 if err := root.Execute(); err != nil {
 os.Exit(1)
 }
}

```

```

// internal/health/health.go
package health

import (
 "net/http"
 "time"
)

func CheckHTTP(url string, timeoutSec int) (bool, error) {
 client := &http.Client{Timeout: time.Duration(timeoutSec) * time.Second}
 resp, err := client.Get(url)
 if err != nil {
 return false, err
 }
 defer resp.Body.Close()
 return resp.StatusCode >= 200 && resp.StatusCode < 300, nil
}

```

## 21.4 17.4 Error handling and exit codes

Go uses explicit error returns—no exceptions:

```

if err := run(); err != nil {
 fmt.Fprintf(os.Stderr, "error: %v\n", err)
 os.Exit(1)
}

```

Wrap errors for context with Go 1.13+ `%w`:

```

return fmt.Errorf("fetch config: %w", err)

```

CI systems depend on **non-zero exit** for failures—never swallow errors silently.

---

## 21.5 17.5 Concurrency for parallel probes

Checking hundreds of endpoints sequentially is slow. Use goroutines with bounded concurrency:

```
sem := make(chan struct{}, 10) // max 10 concurrent
var wg sync.WaitGroup

for _, url := range urls {
 wg.Add(1)
 go func(u string) {
 defer wg.Done()
 sem <- struct{}{}
 defer func() { <-sem }()
 // check u
 }(url)
}
wg.Wait()
```

Always cap concurrency—unbounded goroutines against production can cause incidents.

---

## 21.6 17.6 Kubernetes operators (conceptual)

An **operator** extends Kubernetes with custom resources (CRDs) and a **control loop**:

```
Observe CR state → Compare desired vs actual → Act (create/update/delete) → Requeue
```

Built with **controller-runtime** or **kubebuilder**:

```
kubebuilder init --domain example.com --repo github.com/you/sidecar-operator
kubebuilder create api --group app --version v1 --kind SidecarConfig
```

DevOps engineers operate operators; platform teams author them for domain-specific automation (database failover, certificate renewal, tenant provisioning).

You don't need to write an operator on day one—**understand the pattern** when installing cert-manager, Prometheus operator, or custom internal controllers.

---

## 21.7 17.7 Testing Go CLIs

```
// internal/health/health_test.go
func TestCheckHTTP(t *testing.T) {
 srv := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 w.WriteHeader(http.StatusOK)
 }))
 defer srv.Close()

 ok, err := CheckHTTP(srv.URL, 2)
 if err != nil || !ok {
 t.Fatalf("expected ok, got ok=%v err=%v", ok, err)
 }
}
```

Run `go test ./...` in CI; use `httptest` for HTTP without network flakiness.

---

## 21.8 17.8 Release and distribution

---

| Method          | Notes                          |
|-----------------|--------------------------------|
| GitHub Releases | Attach cross-compiled binaries |
| Homebrew tap    | macOS/Linux developer installs |
| Container       | FROM scratch + static binary   |
| go install      | Private modules need GOPRIVATE |

Example multi-arch build with GoReleaser or raw `G00S` / `G0ARCH` matrix in Actions.

## 21.9 17.9 When not to use Go

- One-off 10-line file rename on a laptop → bash.
- Heavy data science or ML pipelines → Python.
- Team lacks Go experience and tool won't be maintained → Python with typed hints (mypy).

Choose Go when **shipping a binary** or **Kubernetes reconciliation** is the core requirement.

## 21.10 17.10 Chapter summary

- Go produces **static binaries** suited to CLIs and cloud-native tooling.
- Structure projects with `cmd/` and `internal/` packages.
- Use **cobra** (or flag stdlib) for UX; return proper **exit codes**.
- Bounded **concurrency** speeds checks without overloading targets.
- **Operators** implement control loops for Kubernetes custom resources.

## 21.11 Lab 17.1 — Multi-target HTTP checker

1. Implement `devops-check http --url` accepting multiple URLs or a file of URLs.
2. Run checks with max 5 concurrent workers; print failures to stderr.
3. Exit 1 if any fail; add `go test` with httptest server.

## 21.12 Lab 17.2 — Cross-compile release

1. Build linux/amd64 and darwin/arm64 binaries in CI (or locally).
2. Compute SHA256 checksums; upload as release artifacts in a practice repo.
3. Document install instructions in README.

## 21.13 Review questions

1. Name two reasons Go is popular for Kubernetes ecosystem tools.
2. What is the purpose of the `internal/` directory convention?
3. Why cap goroutines when probing many hosts?
4. Describe the operator control loop in four steps.
5. When is Python a better choice than Go for DevOps tasks?



Next: [Chapter 18 — JSON/YAML, Jinja2, and Templating](#)

## 22 Chapter 18: JSON/YAML, Jinja2, and Templating

---

DevOps Handbook — Part V, Pages 326–345

---

### 22.1 18.1 Structured data in DevOps

Modern infrastructure is **data**, not manual clicking: CI matrices, Kubernetes manifests, Terraform variables, Ansible inventories, and API payloads. **JSON** and **YAML** are the two dominant serialization formats; **Jinja2** (in Ansible and many generators) templates text from variables.

Misformatted YAML has caused production outages—treat config editing as code review with schema validation.

---

### 22.2 18.2 JSON essentials

JSON is strict: double quotes, no trailing commas, limited types (object, array, string, number, boolean, null).

```
{
 "service": "payments-api",
 "replicas": 3,
 "env": {
 "LOG_LEVEL": "info",
 "FEATURE_RETRY": true
 },
 "ports": [8080, 9090]
}
```

#### 22.2.1 CLI tooling

```
echo '{"a":1}' | jq '.a'
jq -r '.items[].name' deploy.json
yq -o=json '.' config.yaml # yq v4: YAML ↔ JSON
python3 -m json.tool < messy.json # Pretty-print
```

In Python:

```
import json
from pathlib import Path

data = json.loads(Path("config.json").read_text(encoding="utf-8"))
data["replicas"] = 5
Path("config.out.json").write_text(
 json.dumps(data, indent=2, sort_keys=True) + "\n",
 encoding="utf-8",
)
```

### 22.3 18.3 YAML essentials and pitfalls

YAML is human-friendly but ambiguous if abused.

```
service: payments-api
replicas: 3
env:
 LOG_LEVEL: info
 FEATURE_RETRY: true
ports:
 - 8080
 - 9090
```

### 22.3.1 Gotchas

| Issue          | Example                         | Fix                            |
|----------------|---------------------------------|--------------------------------|
| Norway problem | N0: off → false in some parsers | Quote "N0"                     |
| Octal numbers  | 0123 misread                    | Quote strings                  |
| Tabs           | Tab indentation                 | Spaces only (2 spaces common)  |
| Multi-document | --- separators                  | Split or use explicit loader   |
| Aliases        | &id, *id                        | Powerful; confusing in reviews |

Kubernetes and GitHub Actions use YAML extensively—run `yamllint` in CI.

Python safe load (never `yaml.load` without Loader):

```
import yaml

with open("deploy.yaml", encoding="utf-8") as f:
 doc = yaml.safe_load(f)
```

### 22.4 18.4 JSON vs YAML: when to use which

| Choose JSON              | Choose YAML                                                      |
|--------------------------|------------------------------------------------------------------|
| API request/response     | Kubernetes, Ansible, docker-compose                              |
| Machine-generated only   | Human-edited config with comments                                |
| Strict schema validation | Long nested config files                                         |
| jq everywhere            | Inline multiline strings ( <code>\ </code> , <code>&gt;</code> ) |

Many pipelines **author YAML, convert to JSON** for tools that only accept JSON.

### 22.5 18.5 Templating concepts

**Templates** separate **structure** from **values**:

```
template + variables → rendered config
```

Engines:

| Engine           | Context                           |
|------------------|-----------------------------------|
| Jinja2           | Ansible, Salt, some CI generators |
| Go text/template | Helm charts, some CLIs            |
| Terraform HCL    | Built-in interpolation            |
| envsubst         | Shell-level \$VAR replacement     |

Jinja2 syntax preview:

```
nginx.conf.j2
upstream {{ service_name }} {
 {% for backend in backends %}
```

```
server {{ backend.host }}:{{ backend.port }};
{% endfor %}
}
```

Rendered with variables:

```
from jinja2 import Environment, FileSystemLoader, select_autoescape

env = Environment(
 loader=FileSystemLoader("templates"),
 autoescape=select_autoescape(default=False),
)
template = env.get_template("nginx.conf.j2")
print(template.render(
 service_name="api",
 backends=[{"host": "10.0.1.10", "port": 8080}],
))
```

## 22.6 18.6 Jinja2 in Ansible

Ansible playbooks are YAML; **templates** live in `templates/` and deploy via `template` module:

```
- name: Deploy nginx config
 ansible.builtin.template:
 src: nginx.conf.j2
 dest: /etc/nginx/conf.d/api.conf
 mode: "0644"
 notify: Reload nginx
```

Variables come from inventory, group\_vars, host\_vars, and facts:

```
group_vars/api.yml
service_name: payments-api
backends:
 - host: 10.0.10.5
 port: 8080
 - host: 10.0.10.6
 port: 8080
```

Filters and tests ( `| default('')`, `when: env == 'prod'` ) keep templates readable—avoid business logic soup in Jinja.

## 22.7 18.7 Helm and Go templates (brief)

Helm charts template Kubernetes YAML:

```
templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: {{ include "mychart.fullname" . }}
spec:
 replicas: {{ .Values.replicaCount }}
```

`helm template` renders locally for review before apply—always render in CI and diff against cluster.

## 22.8 18.8 Schema validation

Prevent bad config from merging:

| Tool                   | Format                    |
|------------------------|---------------------------|
| <b>jsonschema</b>      | JSON Schema for JSON/YAML |
| <b>kubeconform</b>     | Kubernetes OpenAPI        |
| <b>checkov / tfsec</b> | IaC policies              |
| <b>ajv</b>             | JSON in Node CI           |

Example with Python jsonschema:

```
from jsonschema import validate

schema = {
 "type": "object",
 "required": ["service", "replicas"],
 "properties": {
 "service": {"type": "string"},
 "replicas": {"type": "integer", "minimum": 1},
 },
}
validate(instance=doc, schema=schema)
```

## 22.9 18.9 Secrets and templating

Never template **plaintext secrets** into Git-tracked files. Patterns:

- Reference `{{ vault_db_password }}` from Ansible Vault or External Secrets.
- Render secrets at deploy time from CI secret store.
- Use `envsubst` on runtime-only files in ephemeral CI job.

Redact rendered output in logs ( `no_log: true` in Ansible for sensitive tasks).

## 22.10 18.10 Chapter summary

- **JSON** is strict and universal for APIs; **YAML** powers K8s and Ansible with comment-friendly syntax.
- Watch YAML **gotchas** (types, tabs, unquoted `NO`).
- **Jinja2** separates templates from data—used heavily in Ansible.
- **Validate** structured config with schemas before apply.
- Keep **secrets out** of rendered artifacts in version control.

## 22.11 Lab 18.1 — Render nginx from Jinja2

1. Create `templates/upstream.conf.j2` with loop over backends.
2. Write `render.py` reading variables from `vars.yaml`.
3. Add jsonschema validation for `vars.yaml` structure.
4. Break validation intentionally; confirm CI/local script fails.

## 22.12 Lab 18.2 — YAML lint pipeline

1. Add `yamllint` config to a repo with sample K8s manifests.
2. Fix indentation and document-start issues.
3. Add GitHub Actions or GitLab job running yamllint on every PR.

---

## 22.13 Review questions

1. Why is `yaml.safe_load` preferred over bare `yaml.load`?
2. Name two YAML features that cause review confusion.
3. What problem does Jinja2 solve in Ansible?
4. How does Helm differ from plain Jinja2 templating?
5. Why validate config against a schema in CI?

---

Next: [\*Chapter 19 — Testing Automation Scripts\*](#)

# 23 Chapter 19: Testing Automation Scripts

DevOps Handbook — Part V, Pages 346–360

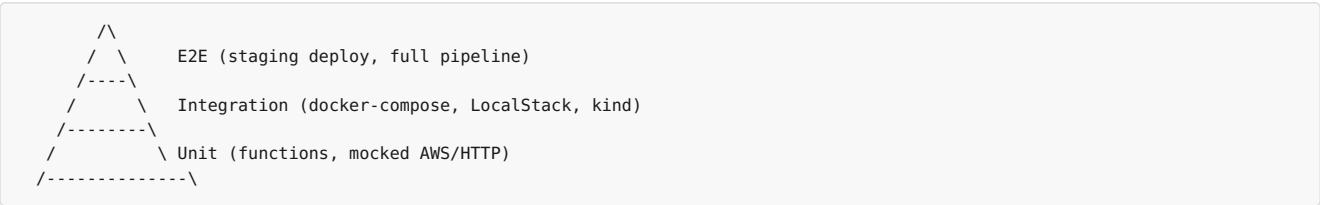
## 23.1 19.1 Why test automation code?

Pipeline scripts and operational tools **are production software**. Untested deploy scripts cause outages as surely as application bugs. Testing automation yields:

| Benefit                 | Example                           |
|-------------------------|-----------------------------------|
| Regression safety       | Refactor deploy.sh without fear   |
| Documentation           | Tests show expected behavior      |
| Faster review           | CI green reduces manual guesswork |
| Confidence in rollbacks | Verified restore procedures       |

The investment scales with **blast radius**—a one-off migration may need smoke tests only; a shared deploy tool needs full unit and integration coverage.

## 23.2 19.2 Test pyramid for DevOps scripts



| Layer       | Scope               | Speed  | Cost   |
|-------------|---------------------|--------|--------|
| Unit        | Pure logic, mocks   | Fast   | Low    |
| Integration | Real local services | Medium | Medium |
| E2E         | Staging environment | Slow   | High   |

Run **many unit**, **some integration**, **few E2E** in CI budgets.

## 23.3 19.3 Testing bash scripts

Tools:

| Tool       | Role                                      |
|------------|-------------------------------------------|
| bats       | Bash Automated Testing System             |
| shellcheck | Static analysis (not tests, but required) |
| shfmt      | Formatting consistency                    |

Example bats test:

```
#!/usr/bin/env bats
```

```
tests/healthcheck.bats

setup() {
 load '../scripts/healthcheck.sh'
}

@test "returns 0 for HTTP 200" {
 run curl_mock 200
 ["$status" -eq 0]
}
```

Pragmatic approach: extract logic to `lib/*.sh` functions, source in tests, mock external commands by placing stubs early in `PATH`:

```
tests/bin/curl - mock
#!/usr/bin/env bash
echo "200"
```

Run in CI:

```
- name: Shellcheck
 run: shellcheck scripts/*.sh
- name: Bats
 run: bats tests/
```

## 23.4 19.4 Testing Python automation

Use **pytest** with mocks for cloud and HTTP:

```
from unittest.mock import patch, MagicMock
import boto3
import mymodule

@patch("boto3.client")
def test_list_buckets(mock_client):
 mock_client.return_value.list_buckets.return_value = {
 "Buckets": [{"Name": "a"}, {"Name": "b"}]
 }
 names = mymodule.list_bucket_names()
 assert names == ["a", "b"]
```

Fixtures for temp filesystem:

```
import pytest
from pathlib import Path

@pytest.fixture
def tmp_config(tmp_path):
 p = tmp_path / "config.yaml"
 p.write_text("replicas: 2\n", encoding="utf-8")
 return p
```

Run `pytest --cov=mymodule --cov-report=term-missing` for coverage gaps on critical paths.

## 23.5 19.5 Testing Go CLIs

Go's `testing` package plus `httptest` cover most CLI logic (Chapter 17). Test `RunE` functions by extracting business logic from `main`.

Table-driven tests:

```
tests := []struct {
 name string
```



```

 code int
 wantOK bool
 }{
 {"ok", 200, true},
 {"fail", 500, false},
 }
 for _, tt := range tests {
 t.Run(tt.name, func(t *testing.T) {
 // ...
 })
 }
}

```

## 23.6 19.6 Integration testing with containers

**docker-compose** or **testcontainers** spin real dependencies:

```

docker-compose.test.yml
services:
 localstack:
 image: localstack/localstack:3
 environment:
 SERVICES: s3,ec2
 ports:
 - "4566:4566"

```

Point boto3 to `endpoint_url=http://localhost:4566` in integration tests marked `@pytest.mark.integration`—skip by default in fast CI jobs.

For Kubernetes manifests: `kind` cluster in CI, `kubectl apply --dry-run=server`, or **kubeconform** validation.

## 23.7 19.7 Testing Terraform and Ansible

| Tool             | Test approach                                                                                              |
|------------------|------------------------------------------------------------------------------------------------------------|
| <b>Terraform</b> | <code>terraform validate</code> , <code>tflint</code> , <code>checkov</code> , <b>terraform plan</b> in CI |
| <b>Ansible</b>   | <code>ansible-playbook --syntax-check</code> , <b>molecule</b> with docker driver                          |
| <b>Helm</b>      | <code>helm template</code> + kubeconform, chart unit tests                                                 |

**Terratest** (Go) applies modules in ephemeral AWS accounts—powerful but slow; use for module maintainers.

## 23.8 19.8 CI pipeline layout for script repos

```

jobs:
 lint:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - run: shellcheck scripts/*.sh || true
 - run: pip install ruff && ruff check .
 - run: yamllint .

 unit:
 needs: lint
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - run: pip install -r requirements-dev.txt
 - run: pytest -q --ignore=tests/integration

 integration:

```

```
needs: unit
runs-on: ubuntu-latest
services:
 localstack:
 image: localstack/localstack:3
 ports:
 - 4566:4566
steps:
 - run: pytest tests/integration -q
```

Gate merges on **lint + unit**; run integration on main or nightly if slow.

---

## 23.9 19.9 Test data and fixtures

- Store **sanitized** sample JSON/YAML in `tests/fixtures/`.
- Never commit real **secrets** or production dumps.
- Use **factories** for repetitive object graphs (Python `dataclasses`, Go structs).

Golden file testing: compare rendered template to committed expected output; update with explicit `--update-golden` flag when intentional.

---

## 23.10 19.10 When tests are enough

Minimum bar before merging shared automation:

1. **shellcheck / ruff / go vet** clean.
2. **Unit tests** for parsing, decision branches, error paths.
3. **Dry-run** or plan-only validation for infra changes.
4. **README** with usage and rollback notes.

For incident one-offs, a **checklist and peer review** may suffice—promote to tested tooling if reused twice.

---

## 23.11 19.11 Chapter summary

- Automation scripts deserve the same **quality gates** as application code.
  - Favor **unit tests with mocks**; add integration tests for cloud APIs via LocalStack/containers.
  - Use **bats**, **pytest**, and Go `testing` per language.
  - Validate **Terraform/Ansible/Helm** with domain-specific tools in CI.
  - Match test depth to **blast radius** and reuse frequency.
- 

## 23.12 Lab 19.1 — pytest suite for inventory script

1. Take the EC2 inventory script from Lab 16.1 (or a stub module).
  2. Add unit tests with mocked boto3 for empty, single, and paginated responses.
  3. Wire pytest into GitHub Actions; fail PR on coverage below 80% for module.
- 

## 23.13 Lab 19.2 — bats for bash healthcheck

1. Add bats tests for the healthcheck script from Lab 8.1.
2. Mock `curl` via PATH stub returning 200, 500, and connection failure.
3. Run shellcheck + bats in CI.

---

## 23.14 Review questions

1. Why are untested deploy scripts risky in DevOps?
2. What belongs in unit vs integration tests for a boto3 script?
3. How can you test bash that wraps curl without hitting real URLs?
4. Name two tools for validating Terraform in CI besides `apply`.
5. When is it acceptable to skip automated tests?

---

Next: [\*Chapter 20 — IaC Principles: State, Idempotency, Drift\*](#)

# 24 Chapter 20: IaC Principles — State, Idempotency, and Drift

DevOps Handbook — Part VI, Pages 361–380

## 24.1 20.1 What is Infrastructure as Code?

**Infrastructure as Code (IaC)** means defining servers, networks, databases, and cloud resources in **machine-readable files** stored in version control—treated like application code.

Benefits:

- **Reproducibility** — Identical dev/staging/prod environments
- **Auditability** — Git history shows who changed what
- **Speed** — Provision in minutes, not days
- **Documentation** — Code is the source of truth
- **Disaster recovery** — Rebuild from code

## 24.2 20.2 Declarative vs imperative

| Style       | You specify       | Example tools                                    |
|-------------|-------------------|--------------------------------------------------|
| Declarative | Desired end state | Terraform, CloudFormation, Kubernetes YAML       |
| Imperative  | Steps to execute  | Ansible (mostly), shell scripts, AWS CLI scripts |

**Declarative** tools compare desired state to actual state and compute a plan. **Imperative** tools run commands in sequence.

Most modern IaC is declarative with imperative escape hatches.

## 24.3 20.3 Idempotency

An operation is **idempotent** if running it multiple times produces the same result.

```
Ansible – idempotent: "ensure nginx is installed and running"
- name: Install nginx
 apt:
 name: nginx
 state: present

- name: Start nginx
 service:
 name: nginx
 state: started
```

Running the playbook twice does not install nginx twice or restart unnecessarily.

**Why it matters:** CI/CD pipelines re-run constantly; non-idempotent scripts cause drift and outages.

## 24.4 20.4 State management

Tools like **Terraform** track what they created in a **state file** ( `terraform.tfstate` ).

State maps resource addresses to real cloud IDs:

```
{
 "resources": [{
 "type": "aws_s3_bucket",
 "name": "logs",
 "instances": [{
 "attributes": { "id": "my-app-logs-bucket", "arn": "arn:aws:s3:::..." }
 }]
 }]
}
```

### 24.4.1 State best practices

1. **Remote state** — Store in S3 + DynamoDB lock (AWS), GCS, Terraform Cloud
2. **Never commit secrets** in state (may contain sensitive values)
3. **Locking** — Prevent concurrent `terraform apply` from corrupting state
4. **State encryption** at rest

## 24.5 20.5 Configuration drift

**Drift** occurs when live infrastructure diverges from IaC definitions (manual console changes, emergency fixes).

Detection:

- `terraform plan` shows drift before apply
- AWS Config rules
- Scheduled drift detection jobs

Remediation:

- **Reconcile** — Apply IaC to restore desired state
- **Import** — Bring manual resources under IaC management
- **Prevent** — Restrict console access; require IaC PRs

## 24.6 20.6 IaC workflow in teams

Developer → PR with .tf changes → CI runs `terraform plan` → Review → Merge  
→ CD runs `terraform apply` → Slack notification

### 24.6.1 Directory structure (example)

```
infrastructure/
├── modules/
│ ├── vpc/
│ ├── eks/
│ └── rds/
├── environments/
│ ├── dev/
│ ├── staging/
│ └── prod/
└── README.md
```

Each environment calls shared modules with different variables.

## 24.7 20.7 Testing IaC

| Level           | Tool                                                                        | What it checks              |
|-----------------|-----------------------------------------------------------------------------|-----------------------------|
| Lint            | <code>terraform fmt</code> ,<br><code>tflint</code> , <code>cfn-lint</code> | Syntax and style            |
| Static security | <code>checkov</code> , <code>tfsec</code> ,<br><code>kics</code>            | Misconfigurations           |
| Plan            | <code>terraform plan</code> in CI                                           | Expected changes            |
| Integration     | Terratest, kitchen-<br>terraform                                            | Real deploy in test account |

---

## 24.8 20.8 Chapter summary

- IaC makes infrastructure versioned, repeatable, and reviewable.
- Prefer **declarative**, **idempotent** tools.
- Manage **state** remotely with locking; detect and fix **drift**.

---

## 24.9 Lab 20.1

1. Manually change a tag on a Terraform-managed resource in the AWS console.
2. Run `terraform plan` and observe drift output.
3. Run `terraform apply` to reconcile.

---

Next: [Chapter 21 — Terraform](#)

# 25 Chapter 21: Terraform — HCL, Providers, and Modules

---

DevOps Handbook — Part VI, Pages 381–410

---

## 25.1 21.1 Terraform overview

**Terraform** (HashiCorp / OpenTofu fork) is the most widely used multi-cloud IaC tool. You write **HCL** (HashiCorp Configuration Language); Terraform calls cloud APIs via **providers**.

```
terraform init # Download providers, init backend
terraform fmt # Format code
terraform validate # Syntax check
terraform plan # Preview changes
terraform apply # Create/update/destroy resources
terraform destroy # Tear down (use carefully)
```

## 25.2 21.2 Basic configuration

main.tf:

```
terraform {
 required_version = ">= 1.5"

 required_providers {
 aws = {
 source = "hashicorp/aws"
 version = "~> 5.0"
 }
 }

 backend "s3" {
 bucket = "my-terraform-state"
 key = "prod/vpc/terraform.tfstate"
 region = "us-east-1"
 dynamodb_table = "terraform-locks"
 encrypt = true
 }
}

provider "aws" {
 region = var.aws_region
}

resource "aws_s3_bucket" "logs" {
 bucket = "${var.project_name}-logs-${var.environment}"

 tags = {
 Project = var.project_name
 Environment = var.environment
 ManagedBy = "terraform"
 }
}
```

variables.tf:

```
variable "aws_region" {
 type = string
 default = "us-east-1"
}

variable "project_name" {
 type = string
}
```

```

}

variable "environment" {
 type = string
 validation {
 condition = contains(["dev", "staging", "prod"], var.environment)
 error_message = "environment must be dev, staging, or prod."
 }
}

```

outputs.tf:

```

output "logs_bucket_name" {
 value = aws_s3_bucket.logs.id
 description = "Name of the logs S3 bucket"
}

```

Apply with: `terraform apply -var="project_name=myapp" -var="environment=dev"`

---

## 25.3 21.3 Resources, data sources, and dependencies

### 25.3.1 Data sources — read existing infrastructure

```

data "aws_ami" "amazon_linux" {
 most_recent = true
 owners = ["amazon"]

 filter {
 name = "name"
 values = ["al2023-ami-*-x86_64"]
 }
}

resource "aws_instance" "web" {
 ami = data.aws_ami.amazon_linux.id
 instance_type = "t3.micro"
}

```

### 25.3.2 Implicit vs explicit dependencies

Terraform infers dependencies from references. Use `depends_on` when ordering matters but no attribute reference exists.

---

## 25.4 21.4 Modules — reusable infrastructure

modules/vpc/main.tf:

```

resource "aws_vpc" "this" {
 cidr_block = var.cidr_block
 enable_dns_hostnames = true
 tags = { Name = var.name }
}

resource "aws_subnet" "public" {
 count = length(var.public_subnet_cidrs)
 vpc_id = aws_vpc.this.id
 cidr_block = var.public_subnet_cidrs[count.index]
 availability_zone = var.availability_zones[count.index]
 map_public_ip_on_launch = true
}

```

Using the module:

```

module "vpc" {

```



```

source = "../modules/vpc"

name = "prod-vpc"
cidr_block = "10.0.0.0/16"
public_subnet_cidrs = ["10.0.1.0/24", "10.0.2.0/24"]
availability_zones = ["us-east-1a", "us-east-1b"]
}

```

Publish modules to **Terraform Registry** or private registries for org-wide reuse.

---

## 25.5 21.5 Lifecycle meta-arguments

```

resource "aws_instance" "web" {
 ami = var.ami_id
 instance_type = "t3.micro"

 lifecycle {
 create_before_destroy = true
 prevent_destroy = false # Set true for prod databases
 ignore_changes = [tags["LastReboot"]] # Ignore specific drift
 }
}

```

## 25.6 21.6 Workspaces and environments

```

terraform workspace list
terraform workspace new staging
terraform workspace select prod

```

Workspaces share the same backend key by default—often **separate directories per environment** (with separate state keys) is clearer for production.

---

## 25.7 21.7 CI/CD integration

GitHub Actions example (plan on PR):

```

- name: Terraform Plan
 run: |
 terraform init -input=false
 terraform plan -input=false -no-color -out=tfplan
 env:
 AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
 AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }

```

Apply only from protected `main` branch after approval.

---

## 25.8 21.8 Chapter summary

- Terraform workflow: **init** → **plan** → **apply** with remote state.
  - Use **variables**, **outputs**, and **modules** for reuse.
  - Run `terraform plan` in CI on every PR; apply from trusted pipelines only.
- 

## 25.9 Lab 21.1

1. Write Terraform to create an S3 bucket with versioning and encryption.
  2. Extract a VPC module with 2 public subnets.
  3. Configure S3 backend for remote state.
  4. Run plan/apply in a free-tier AWS account.
- 

Next: [\*Chapter 26 — Docker\*](#)

# 26 Chapter 22: Terraform — Workspaces, Backends, and CI Integration

DevOps Handbook — Part VI, Pages 411–430

## 26.1 22.1 Why remote state and backends matter

Terraform’s **state file** is the source of truth mapping HCL resource addresses to real cloud IDs. Local state on a laptop does not scale for teams: it cannot be shared, locked, or audited. A **backend** configures where Terraform stores state and how it acquires **locks** during `plan` and `apply`.

| Backend type                        | Locking | Best for                     |
|-------------------------------------|---------|------------------------------|
| local                               | None    | Solo learning only           |
| s3 + DynamoDB                       | Yes     | AWS teams                    |
| azurerm                             | Yes     | Azure                        |
| gcs                                 | Yes     | GCP                          |
| remote (Terraform Cloud/Enterprise) | Yes     | Org policy, run UI, Sentinel |
| kubernetes                          | Via API | GitOps-native shops          |

Remote state enables collaboration, CI pipelines, and disaster recovery. Treat state as sensitive: it often contains secrets even when outputs are marked `sensitive`.

## 26.2 22.2 Configuring S3 backend with locking

backend.tf:

```
terraform {
 backend "s3" {
 bucket = "acme-terraform-state"
 key = "networking/vpc/terraform.tfstate"
 region = "us-east-1"
 dynamodb_table = "terraform-locks"
 encrypt = true
 kms_key_id = "arn:aws:kms:us-east-1:123456789012:key/abcd-1234"
 }
}
```

Bootstrap the backend itself with a one-time local state apply, or use a separate `bootstrap/` directory:

```
bootstrap/main.tf – run once with local backend
resource "aws_s3_bucket" "state" {
 bucket = "acme-terraform-state"
}

resource "aws_s3_bucket_versioning" "state" {
 bucket = aws_s3_bucket.state.id
 versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "state" {
 bucket = aws_s3_bucket.state.id
 rule {
 apply_server_side_encryption_by_default {
 sse_algorithm = "aws:kms"
 }
 }
}
```

```

}

resource "aws_dynamodb_table" "locks" {
 name = "terraform-locks"
 billing_mode = "PAY_PER_REQUEST"
 hash_key = "LockID"

 attribute {
 name = "LockID"
 type = "S"
 }
}

```

### 26.2.1 Partial backend configuration

Pass backend settings at `init` time for multi-environment reuse:

```

terraform init \
 -backend-config="bucket=acme-terraform-state" \
 -backend-config="key=prod/app/terraform.tfstate" \
 -backend-config="region=us-east-1" \
 -backend-config="dynamodb_table=terraform-locks"

```

Use `backend.hcl` files per environment and `-backend-config=backend-prod.hcl`.

## 26.3 22.3 Terraform workspaces

**Workspaces** partition a single configuration into multiple isolated state instances within the same backend. Each workspace gets its own state object (for S3 backends, the key becomes `env:/<workspace>/...`).

```

terraform workspace list
terraform workspace new staging
terraform workspace select prod
terraform workspace show

```

Reference workspace in HCL:

```

locals {
 environment = terraform.workspace

 instance_type = {
 dev = "t3.micro"
 staging = "t3.small"
 prod = "m6i.large"
 }[terraform.workspace]
}

resource "aws_instance" "app" {
 ami = data.aws_ami.amazon_linux.id
 instance_type = local.instance_type
 tags = {
 Environment = local.environment
 }
}

```

### 26.3.1 Workspaces vs directory-per-environment

| Approach                                                                         | Pros                          | Cons                             |
|----------------------------------------------------------------------------------|-------------------------------|----------------------------------|
| <b>Workspaces</b>                                                                | One codebase, quick switching | Easy to apply to wrong workspace |
| <b>Separate directories</b> ( <code>env/prod</code> , <code>env/staging</code> ) | Explicit, clearer in CI       | Duplication unless using modules |
| <b>Separate repos</b>                                                            | Strong blast-radius isolation | Harder to keep modules in sync   |

---

Production teams often prefer **separate state keys per environment** (directory or `-backend-config`) over workspaces alone, because `terraform workspace select` mistakes have caused real outages.

---

## 26.4 22.4 State operations and safety

```
terraform state list
terraform state show aws_instance.app
terraform state mv aws_instance.old aws_instance.new
terraform state rm aws_instance.orphan # Remove from state, not from cloud
terraform import aws_instance.app i-0abc123def456
```

`terraform state rm` does not destroy the resource—it only forgets it. `terraform import` adopts existing infrastructure into state.

For large refactors, use **moved blocks** (Terraform 1.1+):

```
moved {
 from = aws_instance.web
 to = module.compute.aws_instance.web
}
```

Enable **state encryption, versioning** on the S3 bucket, and restrict IAM to CI roles and break-glass admin roles only.

---

## 26.5 22.5 CI/CD integration patterns

### 26.5.1 Pull request: plan only

```
.github/workflows/terraform.yml
name: Terraform
on:
 pull_request:
 paths: ['infra/**']
 push:
 branches: [main]
 paths: ['infra/**']

permissions:
 id-token: write # OIDC
 contents: read
 pull-requests: write

jobs:
 plan:
 runs-on: ubuntu-latest
 defaults:
 run:
 working-directory: infra
 steps:
 - uses: actions/checkout@v4

 - uses: hashicorp/setup-terraform@v3
 with:
 terraform_version: 1.9.0

 - uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: arn:aws:iam::123456789012:role/github-terraform-plan
 aws-region: us-east-1

 - run: terraform init -input=false
 - run: terraform fmt -check -recursive
 - run: terraform validate
 - run: terraform plan -input=false -no-color -out=tfplan
```

```
- uses: actions/upload-artifact@v4
 with:
 name: tfplan
 path: infra/tfplan
```

Post plan output as a PR comment with `terraform show -no-color tfplan` or tools like **atlantis** or **terraform-cloud** run tasks.

## 26.5.2 Main branch: apply with approval

```
apply:
 if: github.ref == 'refs/heads/main' && github.event_name == 'push'
 needs: plan
 environment: production # Requires GitHub Environment approval
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: hashicorp/setup-terraform@v3
 - uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: arn:aws:iam::123456789012:role/github-terraform-apply
 aws-region: us-east-1
 - run: terraform init -input=false
 - run: terraform apply -input=false -auto-approve
```

## 26.5.3 OIDC over long-lived keys

Prefer **OIDC federation** ( `aws-actions/configure-aws-credentials` with `role-to-assume` ) instead of storing `AWS_ACCESS_KEY_ID` in GitHub Secrets. Scope IAM roles: plan role read-only, apply role write with conditions on branch and environment.

## 26.6 22.6 Atlantis and Terraform Cloud

**Atlantis** runs `terraform plan` on PRs inside your VPC:

```
atlantis.yaml
version: 3
projects:
 - name: prod-vpc
 dir: infra/networking
 workspace: default
 autoplan:
 when_modified: ["**/*.tf", "**/*.tfvars"]
 apply_requirements: [approved, mergeable]
```

**Terraform Cloud (TFC) / HCP Terraform** centralizes runs, variables, and policy (Sentinel/OPA). The `cloud` block replaces manual backend config:

```
terraform {
 cloud {
 organization = "acme-corp"
 workspaces {
 name = "prod-networking"
 }
 }
}
```

## 26.7 22.7 Testing and policy in the pipeline

| Tool | Purpose |
|------|---------|
|      |         |

|                              |                                            |
|------------------------------|--------------------------------------------|
| <b>tflint</b>                | Lint provider-specific mistakes            |
| <b>checkov / tfsec</b>       | Security misconfiguration scan             |
| <b>terratest</b>             | Go integration tests against real APIs     |
| <b>terraform test</b> (1.6+) | Native <code>.tftest.hcl</code> unit tests |

```
tests/vpc.tftest.hcl
run "valid_cidr" {
 command = plan
 variables {
 cidr_block = "10.0.0.0/16"
 }
 assert {
 condition = aws_vpc.this.cidr_block == "10.0.0.0/16"
 error_message = "VPC CIDR mismatch"
 }
}
```

Run `terraform test` in CI before `plan`.

## 26.8 22.8 Chapter summary

- Use **remote backends** with **locking**; never rely on local state in teams.
- **Workspaces** isolate state within one config; **separate state keys** per environment are often safer for production.
- CI should run **fmt**, **validate**, **test**, **scan**, and **plan** on every PR; **apply** only from protected branches with human approval.
- Prefer **OIDC** credentials and least-privilege IAM roles for pipeline identities.

## 26.9 Lab 22.1

1. Bootstrap an S3 bucket and DynamoDB table for Terraform state.
2. Migrate a Chapter 21 project from local to remote backend.
3. Create `dev` and `prod` workspaces (or separate backend keys) with different instance types.
4. Add a GitHub Actions workflow that runs `terraform plan` on PRs.
5. Import an existing S3 bucket into Terraform state.

## 26.10 Lab 22.2

1. Configure Atlantis (or TFC free tier) for automated PR plans.
2. Add `tflint` and `checkov` steps that fail the pipeline on high-severity findings.
3. Write a `terraform test` file validating a module output.

## 26.11 Review questions

1. What happens if two engineers run `terraform apply` simultaneously without state locking?
2. When would you choose separate directories over Terraform workspaces?
3. Why should the plan role in CI have fewer permissions than the apply role?
4. What is the difference between `terraform state rm` and `terraform destroy`?
5. How does OIDC improve on static AWS access keys in GitHub Actions?

Next: [Chapter 23 — Ansible Playbooks, Roles, Inventories](#)



# 27 Chapter 23: Ansible — Playbooks, Roles, and Inventories

DevOps Handbook — Part VI, Pages 431–450

## 27.1 23.1 Ansible in the IaC landscape

**Ansible** is an agentless configuration management and automation tool. It connects over SSH (Linux) or WinRM (Windows), pushes **modules** that declare desired state, and exits. Unlike Terraform, Ansible does not maintain a state file—it interrogates each host every run.

| Tool       | Primary use                            | State                       |
|------------|----------------------------------------|-----------------------------|
| Terraform  | Provision cloud resources              | Remote state file           |
| Ansible    | Configure OS, deploy apps, orchestrate | None (facts cache optional) |
| Packer     | Build machine images                   | Ephemeral                   |
| Kubernetes | Run container workloads                | etcd                        |

Ansible complements Terraform: Terraform creates the VPC and EC2 instances; Ansible installs nginx, agents, and application releases.

## 27.2 23.2 Inventory — defining targets

An **inventory** lists hosts and groups. Static INI format:

```
inventories/production/hosts.ini
[web]
web01.example.com ansible_host=10.0.1.10
web02.example.com ansible_host=10.0.1.11

[db]
db01.example.com ansible_host=10.0.2.10

[web:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/prod.pem

[production:children]
web
db
```

YAML inventory:

```
all:
 children:
 web:
 hosts:
 web01.example.com:
 ansible_host: 10.0.1.10
 vars:
 http_port: 80
 db:
 hosts:
 db01.example.com:
 ansible_host: 10.0.2.10
```

### 27.2.1 Dynamic inventory

Query cloud APIs at runtime:

```
AWS EC2 dynamic inventory (ansible-core 2.12+)
ansible-inventory -i aws_ec2.yml --graph
```

aws\_ec2.yml :

```
plugin: amazon.aws.aws_ec2
regions:
 - us-east-1
filters:
 tag:Environment: production
 instance-state-name: running
keyed_groups:
 - key: tags.Role
 prefix: role
hostnames:
 - private-ip-address
```

Run against a group: `ansible-playbook site.yml -i aws_ec2.yml --limit role_web`

---

## 27.3 23.3 Playbooks — ordered automation

A **playbook** is YAML listing **plays** (mapped to host groups) and **tasks** (module calls).

site.yml :

```

- name: Configure web tier
 hosts: web
 become: true
 vars:
 app_version: "2.4.1"

 pre_tasks:
 - name: Wait for SSH
 wait_for_connection:
 timeout: 300

 roles:
 - common
 - nginx
 - app_deploy

 post_tasks:
 - name: Verify health endpoint
 uri:
 url: "http://localhost/health"
 status_code: 200
 register: health
 retries: 5
 delay: 10
 until: health.status == 200
```

playbook.yml for a single concern:

```

- name: Patch all production servers
 hosts: production
 become: true
 serial: "25%" # Rolling update – 25% at a time
 max_fail_percentage: 0

 tasks:
 - name: Update packages (Debian)
 apt:
 update_cache: true
 upgrade: dist
 when: ansible_os_family == "Debian"
```

```

- name: Reboot if required
 reboot:
 msg: "Rebooting after patching"
 when: reboot_required_file.stat.exists

```

## 27.4 23.4 Modules, idempotency, and handlers

```

- name: Install and configure nginx
 hosts: web
 become: true

 tasks:
 - name: Install nginx package
 apt:
 name: nginx
 state: present
 update_cache: true

 - name: Deploy site config
 template:
 src: templates/nginx.conf.j2
 dest: /etc/nginx/sites-available/default
 validate: nginx -t -c %s
 notify: Reload nginx

 - name: Ensure nginx is running
 service:
 name: nginx
 state: started
 enabled: true

 handlers:
 - name: Reload nginx
 service:
 name: nginx
 state: reloaded

```

**Handlers** run once at the end of the play, only if notified. **Templates** use Jinja2 ( `{{ ansible_hostname }}` ). The `validate` parameter prevents deploying broken configs.

### 27.4.1 Check mode and diff

```

ansible-playbook site.yml --check --diff # Dry run with file diffs
ansible-playbook site.yml --tags deploy # Run tagged tasks only

```

## 27.5 23.5 Roles — reusable bundles

Role layout:

```

roles/nginx/
├── defaults/main.yml # Low precedence variables
├── vars/main.yml # Role-internal vars
├── tasks/main.yml
├── handlers/main.yml
├── templates/nginx.conf.j2
├── files/ssl.crt
├── meta/main.yml # Dependencies
└── README.md

```

`roles/nginx/tasks/main.yml`:

```

- name: Install nginx
 package:
 name: "{{ nginx_package }}"

```

```

state: present

- name: Configure worker processes
 lineinfile:
 path: /etc/nginx/nginx.conf
 regexp: '^worker_processes'
 line: "worker_processes {{ nginx_worker_processes }}"
 notify: Reload nginx

```

roles/nginx/defaults/main.yml :

```

nginx_package: nginx
nginx_worker_processes: auto

```

meta/main.yml declares dependencies:

```

dependencies:
- role: common

```

Install from Ansible Galaxy: `ansible-galaxy install geerlingguy.docker`

## 27.6 23.6 Variables and facts precedence

Ansible merges variables from many sources. Highest wins (simplified):

| Priority (high → low)          | Source                           |
|--------------------------------|----------------------------------|
| Extra vars ( <code>-e</code> ) | CLI                              |
| Task vars                      | <code>vars:</code> on task       |
| Block/play vars                | <code>vars:</code> in play       |
| Host facts / host_vars         | <code>host_vars/web01.yml</code> |
| Role defaults                  | <code>defaults/main.yml</code>   |

```

ansible-playbook site.yml -e "app_version=2.5.0"

```

Gather facts: `setup` module runs automatically. Disable with `gather_facts: false` for speed on large fleets.

Custom facts in `/etc/ansible/facts.d/*.fact` (JSON or executable).

## 27.7 23.7 Vault, collections, and project structure

Encrypt secrets with **Ansible Vault**:

```

ansible-vault create group_vars/production/vault.yml
ansible-vault edit group_vars/production/vault.yml
ansible-playbook site.yml --ask-vault-pass
Or --vault-password-file for CI

```

```

group_vars/production/vault.yml (encrypted)
db_password: "s3cr3t"

```

Reference in templates: `{{ db_password }}` — never commit plaintext secrets.

**Collections** package modules and roles ( `ansible-galaxy collection install amazon.aws` ).

Recommended layout:

```
ansible/
├─ ansible.cfg
├─ inventories/
│ └─ staging/
│ └─ production/
├─ group_vars/
├─ host_vars/
├─ roles/
├─ playbooks/
│ └─ site.yml
└─ requirements.yml
```

ansible.cfg:

```
[defaults]
inventory = inventories/production
roles_path = roles
host_key_checking = True
retry_files_enabled = False
interpreter_python = auto_silent

[privilege_escalation]
become = True
become_method = sudo
```

---

## 27.8 23.8 Ansible in CI/CD

```
GitHub Actions excerpt
- name: Run Ansible lint
 run: pip install ansible-lint && ansible-lint playbooks/

- name: Molecule test
 run: cd roles/nginx && molecule test

- name: Deploy to staging
 run: |
 ansible-playbook playbooks/site.yml \
 -i inventories/staging \
 --private-key "${{ secrets.SSH_KEY }}" \
 --vault-password-file vault-pass.sh
```

Use **Molecule** for role testing with Docker or cloud drivers. Limit production deploys to tagged releases and `--check` on canary hosts first.

---

## 27.9 23.9 Chapter summary

- **Inventory** defines where Ansible runs; use **dynamic inventory** for cloud fleets.
  - **Playbooks** orchestrate tasks; **roles** encapsulate reusable configuration.
  - Ansible is **idempotent**—second runs should change nothing if state is correct.
  - Protect secrets with **Ansible Vault**; test roles with **Molecule** and lint with **ansible-lint**.
- 

## 27.10 Lab 23.1

1. Create a static inventory with `web` and `db` groups (use Vagrant or multipass VMs).
  2. Write a playbook that installs nginx and deploys a custom `index.html` via template.
  3. Extract nginx configuration into a role with defaults and handlers.
  4. Encrypt a database password with Ansible Vault and use it in a template.
-

## 27.11 Lab 23.2

1. Configure AWS EC2 dynamic inventory and run `ansible all -m ping`.
  2. Add a `serial: 1` rolling restart play for a fake application service.
  3. Set up Molecule with the Docker driver to test your nginx role.
- 

## 27.12 Review questions

1. Why is Ansible described as “agentless,” and what are the trade-offs?
  2. What triggers a handler to run, and when do handlers execute?
  3. How does `--check` mode differ from running tasks with `state: absent`?
  4. Where should role defaults go versus play-level `vars`?
  5. How would you safely store and use production secrets in Ansible?
- 

Next: [Chapter 24 — Packer & Golden Images](#)

# 28 Chapter 24: Packer & Golden Images

DevOps Handbook — Part VI, Pages 451–465

## 28.1 24.1 Golden images and the immutable infrastructure pattern

A **golden image** (or **machine image**) is a pre-baked AMI, Azure image, or GCP image with the OS hardened, agents installed, and dependencies cached. New instances launch from the image instead of running lengthy bootstrap scripts on every boot.

| Approach                                          | Boot time | Drift risk              | Rollback               |
|---------------------------------------------------|-----------|-------------------------|------------------------|
| <b>Configure at boot</b><br>(cloud-init, Ansible) | Slower    | Higher (scripts change) | Harder                 |
| <b>Golden image</b> (Packer)                      | Fast      | Lower (versioned AMIs)  | Revert launch template |
| <b>Containers</b>                                 | Fastest   | Lowest for app layer    | Image tag/digest       |

**Packer** (HashiCorp) builds images across providers from a single template. Pair Packer with Terraform (provision infrastructure) and Ansible (configure during build) for a complete pipeline.

## 28.2 24.2 Packer workflow

```
packer init . # Install plugins
packer validate amazon-linux.pkr.hcl
packer build amazon-linux.pkr.hcl
packer build -var "region=us-west-2" amazon-linux.pkr.hcl
```

Packer launches a temporary builder instance, runs **provisioners**, creates an image snapshot, then terminates the builder.

## 28.3 24.3 AWS EBS builder example

amazon-linux.pkr.hcl:

```
packer {
 required_version = ">= 1.10.0"
 required_plugins {
 amazon = {
 version = ">= 1.2.8"
 source = "github.com/hashicorp/amazon"
 }
 ansible = {
 version = ">= 1.1.0"
 source = "github.com/hashicorp/ansible"
 }
 }
}

variable "region" {
 type = string
 default = "us-east-1"
}

variable "ami_prefix" {
 type = string
 default = "acme-base"
}
```

```

locals {
 timestamp = regex_replace(timestamp(), "[- TZ:]", "")
}

source "amazon-ebs" "base" {
 ami_name = "${var.ami_prefix}-${local.timestamp}"
 instance_type = "t3.small"
 region = var.region

 source_ami_filter {
 filters = {
 name = "al2023-ami-*-x86_64"
 root-device-type = "ebs"
 virtualization-type = "hvm"
 }
 owners = ["amazon"]
 most_recent = true
 }

 ssh_username = "ec2-user"

 tags = {
 Name = "${var.ami_prefix}-${local.timestamp}"
 BuiltBy = "packer"
 Environment = "shared"
 }
}

build {
 sources = ["source.amazon-ebs.base"]

 provisioner "shell" {
 inline = [
 "sudo dnf update -y",
 "sudo dnf install -y amazon-cloudwatch-agent jq"
]
 }

 provisioner "ansible" {
 playbook_file = "./ansible/base.yml"
 extra_arguments = [
 "--become",
 "-e", "ansible_python_interpreter=/usr/bin/python3"
]
 }

 provisioner "shell" {
 inline = [
 "sudo cloud-init clean --logs",
 "sudo rm -f /root/.ssh/authorized_keys /home/ec2-user/.ssh/authorized_keys",
 "sudo shred -u /etc/ssh/ssh_host_* || true"
]
 }
}

```

The final **cleanup provisioner** removes SSH host keys and authorized keys so clones do not share credentials.

## 28.4 24.4 Provisioners

| Provisioner                | Use case                                |
|----------------------------|-----------------------------------------|
| shell                      | Inline scripts, package installs        |
| ansible / ansible-local    | Configuration management during build   |
| file                       | Upload static files                     |
| windows-shell / powershell | Windows images                          |
| docker                     | Build Docker images (different builder) |

Order matters: install packages before Ansible, clean secrets last.



ansible/base.yml :

```

- hosts: default
 become: true
 tasks:
 - name: Harden sshd
 lineinfile:
 path: /etc/ssh/sshd_config
 regexp: '^PermitRootLogin'
 line: 'PermitRootLogin no'
 notify: restart sshd

 - name: Install node exporter
 unarchive:
 src: https://github.com/prometheus/node_exporter/releases/download/v1.8.2/node_exporter-1.8.2.linux-amd64.tar.gz
 dest: /usr/local/bin
 remote_src: true
 extra_opts: [--strip-components=1]
 include: ['node_exporter']

 - name: Enable node exporter systemd unit
 copy:
 dest: /etc/systemd/system/node_exporter.service
 content: |
 [Unit]
 Description=Node Exporter
 [Service]
 ExecStart=/usr/local/bin/node_exporter
 [Install]
 WantedBy=multi-user.target
 notify: daemon reload

 handlers:
 - name: restart sshd
 service: { name: sshd, state: restarted }
 - name: daemon reload
 systemd: { daemon_reload: true }
```

---

## 28.5 24.5 Multi-region and shared images

```
build {
 sources = [
 "source.amazon-ebs.us_east",
 "source.amazon-ebs.eu_west"
]
 # Same provisioners run in each region
}
```

Use **AMI sharing** across accounts via `ami_users` or AWS Organizations. In Terraform, data source the latest Packer AMI:

```
data "aws_ami" "app_base" {
 most_recent = true
 owners = ["self"]

 filter {
 name = "name"
 values = ["acme-base-*"]
 }

 filter {
 name = "tag:BuiltBy"
 values = ["packer"]
 }
}

resource "aws_launch_template" "app" {
 name = "app-lt"
 image_id = data.aws_ami.app_base.id
 instance_type = "t3.medium"
}
```

# 28.6 24.6 Versioning, promotion, and CI

Image naming strategy:

```
acme-base-20250901120000 # Timestamp build
acme-base-v1.4.2 # Semantic version tag
```

Pipeline stages:

- 1. **Build** on merge to `main` (Packer)
- 2. **Scan** AMI or exported container with **Inspector**, **Trivy**, or **ECR scan**
- 3. **Promote** to staging launch template
- 4. **Bake period** — run integration tests
- 5. **Promote** to production; keep last N AMIs for rollback

```
CI excerpt
- run: packer build -machine-readable . | tee build.log
- run: |
 AMI_ID=$(grep 'artifact,0,id' build.log | cut -d, -f6 | cut -d: -f2)
 aws ec2 create-tags --resources "$AMI_ID" --tags Key=Version,Value=${{ github.sha }}
```

Store Packer variables and plugin versions in Git; pin plugin versions in `required_plugins`.

# 28.7 24.7 Security and compliance

- **No secrets in images** — use instance roles and secrets managers at runtime
- **CIS benchmarks** — apply hardening roles (DevSec hardening collection)
- **SBOM** — document packages; scan for CVEs before promotion
- **Immutable updates** — new AMI + rolling instance refresh, not SSH patching in prod
- **Signed AMIs** — AWS EC2 Image Builder supports compliance pipelines

| Anti-pattern                       | Better approach                          |
|------------------------------------|------------------------------------------|
| Embedding API keys in image        | IAM instance profile                     |
| <code>:latest</code> untagged AMIs | Explicit version tags + lifecycle policy |
| Manual AMI copies                  | Automated pipeline with scan gates       |

# 28.8 24.8 Chapter summary

- **Golden images** reduce boot time and configuration drift; Packer automates their creation.
- Use **provisioners** in order: update OS, configure, then **sanitize** SSH keys and logs.
- Integrate Packer with **CI scan gates** and Terraform **launch templates** referencing latest approved AMI.
- Treat images as **versioned artifacts** with promotion and rollback paths.

# 28.9 Lab 24.1

- 1. Install Packer and initialize the Amazon plugin.
- 2. Build an Amazon Linux 2023 AMI with nginx and the CloudWatch agent.
- 3. Launch an EC2 instance from the AMI and verify services without extra user-data.

4. Add a cleanup provisioner that removes build-time SSH artifacts.
- 

## 28.10 Lab 24.2

1. Wire Packer to Ansible for CIS-style sshd hardening.
  2. Output the AMI ID from CI and pass it to a Terraform `aws_launch_template`.
  3. Implement AMI retention: keep last 5 images, deregister older AMIs.
- 

## 28.11 Review questions

1. Why should SSH host keys be regenerated (or removed) during the Packer build?
  2. When would you choose cloud-init bootstrap over golden images?
  3. How does Terraform discover the latest Packer-built AMI safely?
  4. What security risks exist if secrets are baked into an AMI?
  5. Describe a promotion pipeline from build to production launch template.
- 

Next: [Chapter 25 — Policy as Code](#)

# 29 Chapter 25: Policy as Code — OPA, Checkov, and Sentinel

DevOps Handbook — Part VI, Pages 466–480

## 29.1 25.1 Why policy as code?

Infrastructure and deployment mistakes scale with automation. **Policy as Code (PaC)** encodes guardrails—encryption required, no public S3 buckets, approved regions—as executable rules evaluated in CI, admission controllers, or Terraform runs **before** changes reach production.

| Without PaC                          | With PaC                         |
|--------------------------------------|----------------------------------|
| Manual checklist in PR template      | Automated deny/warn on violation |
| Post-incident “what broke the rule?” | Audit log of policy decisions    |
| Inconsistent interpretation          | Same rule in dev and prod        |

PaC complements but does not replace peer review and threat modeling.

## 29.2 25.2 Policy evaluation points

Developer → PR → [Lint/Scan: Checkov] → Plan → [OPA/Sentinel] → Apply → Runtime [Gatekeeper/Kyverno]

| Stage           | Tools                    | Input format            |
|-----------------|--------------------------|-------------------------|
| Static IaC scan | Checkov, tfsec, KICS     | .tf, .yaml, .dockerfile |
| Plan-time       | OPA (conftest), Sentinel | terraform plan JSON     |
| K8s admission   | OPA Gatekeeper, Kyverno  | Kubernetes API requests |
| General purpose | Open Policy Agent (OPA)  | JSON                    |

## 29.3 25.3 Checkov — static IaC scanning

**Checkov** (Bridgecrew/Prisma) scans Terraform, CloudFormation, Kubernetes, Dockerfiles, and more.

```
pip install checkov
checkov -d infra/ # Directory scan
checkov -f main.tf --framework terraform
checkov -d . --skip-check CKV_AWS_20 # Skip specific check
checkov -d . --soft-fail # Warn only (not for prod)
```

Example finding: S3 bucket without encryption or public access block.

Suppress false positives inline:

```
resource "aws_s3_bucket" "logs" {
 #checkov:skip=CKV_AWS_18:Access logging handled centrally
 bucket = "acme-logs"
}
```

`.checkov.yml` for CI:

```
quiet: false
compact: true
framework:
 - terraform
 - dockerfile
soft-fail: false
skip-check:
 - CKV_AWS_144 # Document why in SECURITY.md
```

GitHub Actions:

```
- name: Checkov scan
 uses: bridgecrewio/checkov-action@master
 with:
 directory: infra/
 framework: terraform
 soft_fail: false
```

---

## 29.4 25.4 Open Policy Agent (OPA) and Rego

**OPA** is a general-purpose policy engine. Policies are written in **Rego**. **conftest** runs Rego against JSON/YAML files.

Install: `brew install conftest` or download from [openpolicyagent.org](https://openpolicyagent.org).

Policy: `policy/s3.rego`

```
package terraform.s3

deny[msg] {
 resource := input.resource_changes[_]
 resource.type == "aws_s3_bucket"
 not resource.change.after.server_side_encryption_configuration
 msg := sprintf("S3 bucket %s must enable encryption", [resource.address])
}

deny[msg] {
 resource := input.resource_changes[_]
 resource.type == "aws_s3_bucket"
 resource.change.after.acl == "public-read"
 msg := sprintf("S3 bucket %s must not be public", [resource.address])
}
```

Generate plan JSON and test:

```
terraform plan -out=tfplan
terraform show -json tfplan > plan.json
conftest test plan.json -p policy/
```

OPA on Kubernetes (**Gatekeeper**):

```
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sRequiredLabels
metadata:
 name: require-team-label
spec:
 match:
 kinds:
 - apiGroups: [""]
 kinds: ["Pod"]
 parameters:
 labels: ["team", "cost-center"]
```

Violating pods are rejected at admission time.

## 29.5 25.5 HashiCorp Sentinel (Terraform Cloud/Enterprise)

**Sentinel** is HashiCorp’s policy language for Terraform Cloud/Enterprise. It integrates directly with the run pipeline.

Example: restrict instance types

```
policy/limit-instance-type.sentinel
import "tfplan/v2" as tfplan

allowed = ["t3.micro", "t3.small", "t3.medium"]

main = rule {
 all tfplan.resource_changes as _, rc {
 rc.type is not "aws_instance" or
 rc.change.after.instance_type in allowed
 }
}
```

Enforcement levels:

| Level          | Behavior                    |
|----------------|-----------------------------|
| advisory       | Log only                    |
| soft-mandatory | Overridable with permission |
| hard-mandatory | Cannot apply                |

Sentinel has first-class Terraform plan imports ( `tfplan` , `tfconfig` , `tfstate` ). OPA is more portable across tools; Sentinel is tightly coupled to HashiCorp ecosystem.

## 29.6 25.6 Comparison matrix

| Feature        | Checkov                 | OPA/conftest      | Sentinel                  |
|----------------|-------------------------|-------------------|---------------------------|
| Languages      | Python checks           | Rego              | Sentinel DSL              |
| Terraform plan | Indirect (HCL)          | Plan JSON         | Native                    |
| K8s admission  | Limited                 | Gatekeeper        | N/A                       |
| License        | Apache 2.0              | Apache 2.0        | BSL / TFC                 |
| Best for       | Quick security baseline | Multi-tool policy | TFC enterprise governance |

Use **Checkov** for broad static scans, **OPA** for custom plan logic and Kubernetes, **Sentinel** if you are standardized on Terraform Cloud with hierarchical policies.

## 29.7 25.7 Building a policy program

1. **Start with deny-list basics** — public exposure, unencrypted storage, open security groups
2. **Map to compliance** — tag each rule to CIS, SOC2, or internal standard
3. **Warn then enforce** — `soft-fail` for 2 weeks, then hard fail
4. **Exception process** — ticket ID in `#checkov:skip` or OPA `data.exceptions`
5. **Measure** — track violations per team; fix generators not just resources

`policy/exceptions.json` for OPA:

```
{
 "exceptions": [
 {
 "resource": "aws_s3_bucket.legacy_logs",
 "policy": "terraform.s3.deny",
 "reason": "TICKET-4521 migration Q4",
 "expires": "2026-12-31"
 }
]
}
```

```
exception[resource] {
 ex := data.exceptions[_]
 resource := ex.resource
 time.parse_rfc3339_ns(ex.expires) > time.now_ns()
}
```

---

## 29.8 25.8 CI pipeline integration

```
jobs:
 policy:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: hashicorp/setup-terraform@v3
 - run: terraform init && terraform plan -out=tfplan
 - run: terraform show -json tfplan > plan.json
 - run: checkov -d infra/ --quiet
 - run: conftest test plan.json -p policy/ --all-namespaces
```

Fail the build on any `deny` rule. Store policy repos separately for platform teams to version independently.

---

## 29.9 25.9 Chapter summary

- **Policy as Code** automates compliance and prevents recurring misconfigurations.
  - **Checkov** provides out-of-the-box IaC security checks with minimal setup.
  - **OPA/Rego** is flexible for Terraform plans, Kubernetes admission, and custom JSON APIs.
  - **Sentinel** integrates with Terraform Cloud for plan-time enforcement in enterprise workflows.
  - Combine static scan + plan-time policy + runtime admission for defense in depth.
- 

### 29.10 Lab 25.1

1. Run Checkov on a Terraform project; fix three high-severity findings.
  2. Write a Rego policy denying `0.0.0.0/0` ingress on security groups.
  3. Integrate Checkov and conftest into a GitHub Actions workflow.
- 

### 29.11 Lab 25.2

1. Install OPA Gatekeeper on a test cluster and deploy a `require-labels` constraint.
  2. Create an exception file with expiring waivers for one resource.
  3. Document a team policy: which checks are hard-fail vs advisory.
-

## 29.12 Review questions

1. What is the difference between static IaC scanning and plan-time policy evaluation?
2. When would you use Gatekeeper instead of only Checkov?
3. Why should policy exceptions have expiration dates?
4. How does Sentinel's `soft-mandatory` differ from `hard-mandatory`?
5. Name three resource types you would prioritize in a new PaC rollout.

---

Next: [Chapter 26 — Docker Fundamentals](#)



# 30 Chapter 26: Docker — Images, Dockerfile, and Multi-stage Builds

DevOps Handbook — Part VII, Pages 481–505

## 30.1 26.1 Containers in one paragraph

A **container** packages an application with its dependencies into an isolated process on a shared Linux kernel. Unlike VMs, containers share the host OS kernel—lightweight and fast to start.

**Docker** is the dominant tooling for building, shipping, and running containers locally and in CI.

## 30.2 26.2 Core concepts

| Concept    | Description                                   |
|------------|-----------------------------------------------|
| Image      | Read-only template (layers)                   |
| Container  | Running instance of an image                  |
| Dockerfile | Recipe to build an image                      |
| Registry   | Storage for images (Docker Hub, ECR)          |
| Volume     | Persistent data outside container lifecycle   |
| Network    | Connect containers to each other and the host |

## 30.3 26.3 Essential commands

```
docker pull nginx:1.25-alpine
docker images
docker run -d -p 8080:80 --name web nginx:1.25-alpine
docker ps
docker logs web
docker exec -it web sh
docker stop web && docker rm web
docker build -t myapp:1.0 .
docker tag myapp:1.0 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0
```

## 30.4 26.4 Dockerfile best practices

```
Use specific version tags, not :latest in production
FROM node:20-alpine AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production=false

COPY . .
RUN npm run build

Multi-stage: smaller final image
FROM node:20-alpine AS runtime
```

```

RUN addgroup -g 1001 -S app && adduser -u 1001 -S app -G app
WORKDIR /app
USER app

COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY package.json .

EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s CMD wget -q0- http://localhost:3000/health || exit 1

CMD ["node", "dist/server.js"]

```

### 30.4.1 Best practices checklist

- Use **multi-stage builds** to shrink images
- Run as **non-root** user
- Minimize layers; combine RUN commands where sensible
- Use `.dockerignore` (exclude `node_modules`, `.git`, etc.)
- Pin base image digests for supply chain security
- Add **HEALTHCHECK** for orchestrators

## 30.5 26.5 Docker Compose for local development

`docker-compose.yml` :

```

services:
 api:
 build: .
 ports:
 - "3000:3000"
 environment:
 DATABASE_URL: postgres://user:pass@db:5432/app
 depends_on:
 db:
 condition: service_healthy

 db:
 image: postgres:16-alpine
 environment:
 POSTGRES_USER: user
 POSTGRES_PASSWORD: pass
 POSTGRES_DB: app
 volumes:
 - pgdata:/var/lib/postgresql/data
 healthcheck:
 test: ["CMD-SHELL", "pg_isready -U user -d app"]
 interval: 5s
 timeout: 5s
 retries: 5

volumes:
 pgdata:

```

```

docker compose up -d
docker compose logs -f api
docker compose down -v # Remove volumes too

```

## 30.6 26.6 Security basics

- Scan images: `docker scout cves` or **Trivy**
- Do not store secrets in images or Dockerfile `ENV`
- Use read-only root filesystem where possible ( `--read-only` )

- Drop capabilities: `--cap-drop=ALL`
- 

## 30.7 26.7 Chapter summary

- Images are built from Dockerfiles; containers are ephemeral runtime instances.
  - **Multi-stage builds** and non-root users are production essentials.
  - **Docker Compose** orchestrates multi-container local stacks.
- 

## 30.8 Lab 26.1

1. Containerize a simple Node or Python web app.
  2. Add multi-stage build and health check.
  3. Run with Compose alongside PostgreSQL.
  4. Scan the image with Trivy.
- 

Next: [Chapter 37 — CI/CD Pipeline Design](#)

# 31 Chapter 27: Container Networking, Volumes, and Security

DevOps Handbook — Part VII, Pages 506–525

## 31.1 27.1 Container networking model

Containers share the host Linux kernel but need isolated network namespaces. **Docker** (and containerd/CRI-O underneath) creates virtual interfaces, bridges, and iptables/nftables rules so containers get IPs and can reach each other and the outside world.

| Namespace | Isolates                    |
|-----------|-----------------------------|
| Network   | Interfaces, routes, ports   |
| PID       | Process tree                |
| Mount     | Filesystem view             |
| UTS       | Hostname                    |
| IPC       | Inter-process communication |
| User      | UID/GID mapping (rootless)  |

Understanding namespaces explains why a process in a container sees its own `eth0` and port 80, while the host may map that to port 8080.

## 31.2 27.2 Docker network drivers

```
docker network ls
docker network inspect bridge
docker network create --driver bridge app-net
docker run -d --name api --network app-net nginx:alpine
docker run -d --name cache --network app-net redis:7-alpine
```

| Driver  | Scope              | Use case                                       |
|---------|--------------------|------------------------------------------------|
| bridge  | Single host        | Default; container-to-container on one machine |
| host    | Single host        | Container uses host network stack (no NAT)     |
| none    | Isolated           | No networking                                  |
| overlay | Multi-host (Swarm) | Legacy Swarm clustering                        |
| macvlan | L2 segment         | Containers appear as physical NICs on LAN      |

### 31.2.1 DNS on user-defined bridges

On the default `bridge`, containers resolve each other by IP only. On **user-defined bridge networks**, Docker’s embedded DNS resolves **container names**:

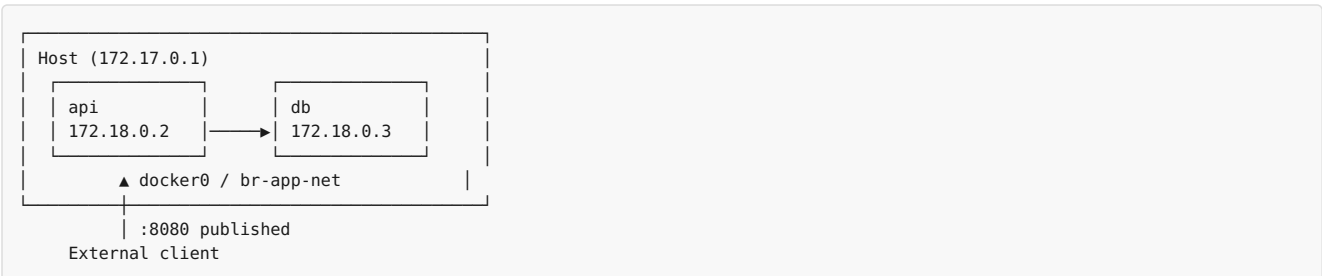
```
From api container:
docker exec api ping -c1 cache # Resolves to cache container IP
```

### 31.2.2 Published ports

```
docker run -p 127.0.0.1:8080:80 nginx # Bind host localhost only
docker run -p 8080:80 nginx # All interfaces (careful in prod)
```

`-p` sets DNAT rules forwarding host port → container port.

## 31.3 27.3 Container-to-container and host communication



**Host network mode** (`--network host`): container shares host ports directly—useful for high-performance proxies; breaks port isolation.

**Extra hosts** (`--add-host`): inject static `/etc/hosts` entries for legacy apps.

## 31.4 27.4 Volumes and persistence

Container filesystems are ephemeral. **Volumes** persist data outside the container lifecycle.

| Mechanism         | Location                              | Best for                     |
|-------------------|---------------------------------------|------------------------------|
| <b>Volume</b>     | <code>/var/lib/docker/volumes/</code> | Databases, shared data       |
| <b>Bind mount</b> | Host path                             | Dev hot-reload, config files |
| <b>tmpfs</b>      | Memory                                | Secrets, temp cache          |

```
docker volume create pgdata
docker run -d \
 --name postgres \
 -v pgdata:/var/lib/postgresql/data \
 -e POSTGRES_PASSWORD=secret \
 postgres:16-alpine

Bind mount for development
docker run -v $(pwd)/src:/app/src node:20-alpine npm run dev
```

Volume commands:

```
docker volume ls
docker volume inspect pgdata
docker run --rm -v pgdata:/data alpine ls /data
```

**Named volumes** are portable across hosts when using volume plugins or orchestrators (Kubernetes PVCs). **Bind mounts** tie data to a specific host path—avoid for production databases on single Docker hosts.

### 31.4.1 Read-only root filesystem

```
docker run --read-only \
 --tmpfs /tmp \
 -v app-cache:/var/cache \
 myapp:1.0
```

## 31.5 27.5 Linux capabilities and seccomp

Containers are not VMs. Root in a container may still be dangerous on the host without proper isolation. Docker drops most **Linux capabilities** by default.

```
Drop all, add only what's needed
docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE nginx:alpine

Never do this in production
docker run --privileged myapp # Near-root on host
```

| Capability   | Risk if granted               |
|--------------|-------------------------------|
| SYS_ADMIN    | Mount, cgroup manipulation    |
| NET_RAW      | Packet sniffing, ARP spoofing |
| DAC_OVERRIDE | Bypass file permissions       |

**seccomp** profiles filter syscalls. Docker uses a default profile; Kubernetes can set `seccompProfile: RuntimeDefault`.

**AppArmor** / **SELinux** add MAC labels:

```
docker run --security-opt apparmor=docker-default ...
```

## 31.6 27.6 Image and runtime security

### 31.6.1 Non-root user

```
RUN adduser -D -u 10001 appuser
USER appuser
```

### 31.6.2 Scanning

```
trivy image myapp:1.0
docker scout cves myapp:1.0
grype myapp:1.0
```

Fail CI if CRITICAL CVEs exist without accepted risk ticket.

### 31.6.3 Secrets management

| Anti-pattern                      | Fix                                   |
|-----------------------------------|---------------------------------------|
| ENV DB_PASSWORD=... in Dockerfile | Runtime env from vault/SSM            |
| Secrets in image layers           | BuildKit secrets, multi-stage discard |
| World-readable bind mounts        | tmpfs or secret driver                |

BuildKit secret mount:

```
syntax=docker/dockerfile:1
RUN --mount=type=secret,id=npnrc,target=/root/.npmrc npm ci
```

```
docker build --secret id=npnrc,src=$HOME/.npmrc .
```

---

## 31.7 27.7 Resource limits and health

```
docker run -d \
 --memory=512m \
 --cpus=1.5 \
 --pids-limit=100 \
 --restart=unless-stopped \
 myapp:1.0
```

Without limits, one container can exhaust host memory (**OOM killer** may kill random processes).

**Health checks** (Dockerfile or run flag):

```
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
 CMD wget -q0- http://localhost:8080/health || exit 1
```

Orchestrators use health status for rolling updates and traffic routing.

---

## 31.8 27.8 Debugging networking issues

```
docker exec api ip addr
docker exec api cat /etc/resolv.conf
docker exec api wget -q0- http://cache:6379
iptables -t nat -L -n | grep 8080 # Host: inspect port publishing
docker logs api --tail 50
```

Common problems:

- Wrong network → container not on same user-defined bridge
- Published port conflict → another process bound to host port
- DNS failure → using default bridge instead of named network
- Firewall on host blocking forwarded traffic

---

## 31.9 27.9 Chapter summary

- Containers use **network namespaces**; user-defined **bridge networks** provide DNS-based service discovery.
- **Volumes** persist data; prefer named volumes over bind mounts for production state.
- Harden with **non-root users**, **cap-drop**, **read-only rootfs**, and **image scanning**.
- Never run `--privileged` in production; inject secrets at runtime, not in images.

---

### 31.10 Lab 27.1

1. Create a user-defined bridge network with `api` and `redis` containers; verify name resolution.
2. Attach a named volume to PostgreSQL; delete and recreate the container without data loss.
3. Run nginx with `--read-only`, `--cap-drop=ALL`, and `--cap-add=NET_BIND_SERVICE`.

---

### 31.11 Lab 27.2

1. Publish a port bound to `127.0.0.1` only; confirm remote access is blocked.
2. Scan an image with Trivy; remediate one CVE by upgrading a base package.

3. Debug a deliberate misconfiguration (wrong network) using `docker inspect` and `exec`.
- 

## 31.12 Review questions

1. Why do containers on the default bridge network not resolve each other by name?
  2. What is the difference between a bind mount and a named volume?
  3. Which capability allows binding to ports below 1024 without running as root?
  4. Why is `--privileged` dangerous compared to adding specific capabilities?
  5. How does `-p 8080:80` affect host firewall and NAT rules?
- 

Next: [Chapter 28 — Docker Compose and Local Dev](#)



# 32 Chapter 28: Docker Compose and Local Dev Environments

---

DevOps Handbook — Part VII, Pages 526–540

---

## 32.1 28.1 What Docker Compose solves

**Docker Compose** (v2, integrated as `docker compose`) defines multi-container applications in a single YAML file. Developers run `docker compose up` to start databases, caches, APIs, and frontends with consistent networking and volumes—eliminating “works on my machine” setup drift.

Compose is ideal for **local development**, **integration tests**, and **demo stacks**. Production orchestration typically moves to Kubernetes, ECS, or Nomad, but Compose files often inform those definitions.

---

## 32.2 28.2 Compose file structure

`compose.yaml` :

```
name: bookstore

services:
 api:
 build:
 context: ./api
 dockerfile: Dockerfile
 target: dev
 ports:
 - "3000:3000"
 environment:
 NODE_ENV: development
 DATABASE_URL: postgres://app:app@db:5432/bookstore
 REDIS_URL: redis://cache:6379/0
 volumes:
 - ./api/src:/app/src:ro
 depends_on:
 db:
 condition: service_healthy
 cache:
 condition: service_started
 develop:
 watch:
 - action: sync
 path: ./api/src
 target: /app/src
 - action: rebuild
 path: ./api/package.json

 db:
 image: postgres:16-alpine
 environment:
 POSTGRES_USER: app
 POSTGRES_PASSWORD: app
 POSTGRES_DB: bookstore
 volumes:
 - pgdata:/var/lib/postgresql/data
 healthcheck:
 test: ["CMD-SHELL", "pg_isready -U app -d bookstore"]
 interval: 5s
 timeout: 5s
 retries: 5

 cache:
 image: redis:7-alpine
 command: redis-server --save "" --appendonly no
```

```

adminer:
 image: adminer:4
 ports:
 - "8080:8080"
 profiles:
 - tools

volumes:
 pgdata:

```

Key directives:

| Key                    | Purpose                                   |
|------------------------|-------------------------------------------|
| build / image          | Build locally or pull image               |
| ports                  | Publish container ports                   |
| environment / env_file | Configuration                             |
| volumes                | Persistence and dev bind mounts           |
| depends_on             | Startup ordering (with health conditions) |
| profiles               | Optional services ( --profile tools )     |
| develop.watch          | File sync / rebuild (Compose 2.22+)       |

## 32.3 28.3 Essential commands

```

docker compose up -d # Detached start
docker compose up --build # Rebuild images
docker compose ps
docker compose logs -f api
docker compose exec api sh
docker compose run --rm api npm test
docker compose stop
docker compose down # Stop and remove containers
docker compose down -v # Also remove named volumes
docker compose config # Validate and render merged YAML

```

Run one-off commands in the service context:

```
docker compose run --rm api npx prisma migrate deploy
```

## 32.4 28.4 Environment-specific overrides

compose.yaml — base stack

compose.override.yaml — auto-loaded local overrides (gitignored secrets)

compose.prod.yaml — production-like settings

```
docker compose -f compose.yaml -f compose.prod.yaml up -d
```

compose.override.yaml example:

```

services:
 api:
 ports:
 - "3000:3000"
 - "9229:9229" # Node debugger
 environment:
 LOG_LEVEL: debug

```

`.env` file (loaded automatically):

```
POSTGRES_USER=app
POSTGRES_PASSWORD=localdev
API_PORT=3000
```

Reference in compose: `${API_PORT}:3000`

---

## 32.5 28.5 Networking in Compose

Compose creates a **default network** named `{project}_default`. All services join it and resolve each other by **service name** (`db`, `cache`, `api`).

Custom network:

```
networks:
 frontend:
 backend:

services:
 web:
 networks: [frontend]
 api:
 networks: [frontend, backend]
 db:
 networks: [backend]
```

`db` is unreachable from `web` —only `api` bridges both networks. Useful for mimicking DMZ patterns locally.

---

## 32.6 28.6 Development workflows

### 32.6.1 Hot reload with bind mounts

Mount source code read-write; run dev server with file watcher inside container:

```
services:
 api:
 build: .
 command: npm run dev
 volumes:
 - ./api:/app
 - /app/node_modules # Anonymous volume preserves container node_modules
```

The anonymous volume prevents host `node_modules` from overwriting Linux binaries built in the image.

### 32.6.2 Compose Watch

```
develop:
 watch:
 - action: sync+restart
 path: ./config
 target: /app/config
```

`docker compose watch` syncs files and restarts affected services.

### 32.6.3 Seed data

```
services:
 db:
 volumes:
 - pgdata:/var/lib/postgresql/data
```

```
- ./scripts/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
```

PostgreSQL runs scripts in `docker-entrypoint-initdb.d` on first initialization only.

## 32.7 28.7 Testing with Compose

`compose.test.yaml` :

```
services:
 api:
 build: .
 environment:
 DATABASE_URL: postgres://test:test@db:5432/test
 command: npm test
 depends_on:
 db:
 condition: service_healthy

 db:
 image: postgres:16-alpine
 environment:
 POSTGRES_USER: test
 POSTGRES_PASSWORD: test
 POSTGRES_DB: test
 healthcheck:
 test: ["CMD-SHELL", "pg_isready -U test"]
 interval: 2s
 retries: 10
```

CI:

```
- run: docker compose -f compose.test.yaml up --build --abort-on-container-exit --exit-code-from api
- run: docker compose -f compose.test.yaml down -v
```

`--exit-code-from api` fails the job if tests fail.

## 32.8 28.8 Compose vs production

| Concern         | Local Compose                                     | Production K8s/ECS      |
|-----------------|---------------------------------------------------|-------------------------|
| Scaling         | <code>deploy.replicas</code> (Swarm only); manual | HPA, replica sets       |
| Secrets         | <code>.env</code> , Docker secrets                | Vault, K8s Secrets, SSM |
| Health routing  | None                                              | Load balancers, probes  |
| Rolling updates | Recreate containers                               | Controlled rollouts     |

Translate patterns, not files literally—tools like **Kompose** exist but output usually needs manual refinement.

## 32.9 28.9 Chapter summary

- **Compose** defines multi-service stacks with networking, volumes, and dependencies in YAML.
- Use **overrides**, `.env` , and **profiles** for flexible local vs CI environments.
- **Healthchecks** and `depends_on` conditions prevent race conditions on startup.
- **Compose Watch** and bind mounts enable fast inner-loop development.
- Run integration tests in CI with `docker compose up --abort-on-container-exit` .

## 32.10 Lab 28.1

1. Build a three-service stack: API, PostgreSQL, Redis.
  2. Add healthchecks and verify API waits for database readiness.
  3. Create a `tools` profile with Adminer or pgAdmin.
- 

## 32.11 Lab 28.2

1. Split `compose.yaml` and `compose.override.yaml` for debug ports and log level.
  2. Add Compose Watch or a bind mount for live code reload.
  3. Write a CI job that runs tests via `compose.test.yaml`.
- 

## 32.12 Review questions

1. How does Compose service discovery differ from linking containers manually?
  2. Why use an anonymous volume for `node_modules` in a bind-mounted Node app?
  3. What does `depends_on: condition: service_healthy` solve?
  4. When should you use `compose down -v` versus `compose down`?
  5. How do Compose profiles help keep optional services out of the default stack?
- 

Next: [Chapter 29 — Container Registries and Image Signing](#)

# 33 Chapter 29: Container Registries and Image Signing

DevOps Handbook — Part VII, Pages 541–560

## 33.1 29.1 Container registries overview

A **container registry** stores and distributes OCI-compatible **images** (manifests + layers). Registries are the artifact hub of the container supply chain—CI pushes images; clusters pull them to run workloads.

| Registry                         | Provider           | Notes                                          |
|----------------------------------|--------------------|------------------------------------------------|
| Docker Hub                       | Docker             | Public default; rate limits on anonymous pulls |
| Amazon ECR                       | AWS                | IAM auth, image scanning, lifecycle policies   |
| Google Artifact Registry         | GCP                | Multi-format (Docker, npm, maven)              |
| Azure ACR                        | Azure              | Geo-replication, content trust                 |
| GitHub Container Registry (GHCR) | GitHub             | ghcr.io/org/image                              |
| Harbor                           | CNCF / self-hosted | RBAC, replication, signing                     |

Images are referenced by **tag** ( myapp:1.2.3 ) or immutable **digest** ( myapp@sha256:abc123... ). Prefer digests in production manifests for immutability.

## 33.2 29.2 Image naming and tagging strategy

```
<registry>/<namespace>/<repository>:<tag>
ghcr.io/acme-corp/payments-api:v1.4.2
123456789.dkr.ecr.us-east-1.amazonaws.com/web:main-abc1234
```

Tagging conventions:

| Tag pattern  | Use                               |
|--------------|-----------------------------------|
| v1.2.3       | Release semver                    |
| main-abc1234 | Git SHA from branch               |
| 1.2.3-alpine | Base OS variant                   |
| latest       | Dev only—never prod deploy target |

**Never** reuse tags for different content. CI should push unique tags per build; promotion means updating deployment to point at a tested tag/digest.

## 33.3 29.3 Authenticating to registries

### 33.3.1 Amazon ECR

```
aws ecr get-login-password --region us-east-1 | \
 docker login --username AWS --password-stdin \
 123456789.dkr.ecr.us-east-1.amazonaws.com

docker tag myapp:1.0 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0
```

Create repository with Terraform:

```
resource "aws_ecr_repository" "myapp" {
 name = "myapp"
 image_tag_mutability = "IMMUTABLE"

 image_scanning_configuration {
 scan_on_push = true
 }
}

resource "aws_ecr_lifecycle_policy" "myapp" {
 repository = aws_ecr_repository.myapp.name
 policy = jsonencode({
 rules = [{
 rulePriority = 1
 description = "Keep last 30 images"
 selection = {
 tagStatus = "any"
 countType = "imageCountMoreThan"
 countNumber = 30
 }
 action = { type = "expire" }
 }]
 })
}
```

### 33.3.2 GHCR

```
echo $GITHUB_TOKEN | docker login ghcr.io -u USERNAME --password-stdin
docker tag myapp:1.0 ghcr.io/acme-corp/myapp:1.0
docker push ghcr.io/acme-corp/myapp:1.0
```

Use fine-grained PAT or `GITHUB_TOKEN` in Actions with `packages: write` permission.

## 33.4 29.4 Pull policies in Kubernetes and beyond

```
spec:
 containers:
 - name: api
 image: 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:v1.4.2@sha256:deadbeef...
 imagePullPolicy: IfNotPresent
```

| imagePullPolicy | Behavior                                                  |
|-----------------|-----------------------------------------------------------|
| Always          | Pull every pod start (required for <code>:latest</code> ) |
| IfNotPresent    | Pull if not cached locally                                |
| Never           | Local only (kind/minikube dev)                            |

Private registries need **imagePullSecrets** or cloud-specific workload identity (EKS, GKE).

### 33.5 29.5 Vulnerability scanning

Scan on push and on schedule:

```
trivy image ghcr.io/acme-corp/myapp:1.0
aws ecr describe-image-scan-findings \
 --repository-name myapp \
 --image-id imageTag=1.0
```

CI gate example:

```
- name: Build and push
 run: |
 docker build -t ghcr.io/acme-corp/myapp:${{ github.sha }} .
 docker push ghcr.io/acme-corp/myapp:${{ github.sha }}

- name: Scan image
 uses: aquasecurity/trivy-action@master
 with:
 image-ref: ghcr.io/acme-corp/myapp:${{ github.sha }}
 severity: CRITICAL,HIGH
 exit-code: 1
```

Maintain **SBOMs** (Software Bill of Materials) with `syft` or `trivy sbom` for compliance and faster CVE matching.

## 33.6 29.6 Image signing and verification

Signing proves an image was built by a trusted pipeline and was not tampered with in transit or at rest in the registry.

### 33.6.1 Docker Content Trust (Notary v1)

Legacy; being superseded by **Sigstore/cosign** in most ecosystems.

### 33.6.2 Cosign (Sigstore)

```
Generate key pair (prefer keyless in CI with OIDC)
cosign generate-key-pair

Sign after push
cosign sign --key cosign.key ghcr.io/acme-corp/myapp:1.0

Verify before deploy
cosign verify --key cosign.pub ghcr.io/acme-corp/myapp:1.0
```

**Keyless signing** in GitHub Actions:

```
permissions:
 id-token: write
 contents: read
 packages: write

- uses: sigstore/cosign-installer@main
- run: cosign sign ghcr.io/acme-corp/myapp@${{ steps.build.outputs.digest }}
 env:
 COSIGN_EXPERIMENTAL: "true"
```

Attach **SBOM** as attestation:

```
syft ghcr.io/acme-corp/myapp:1.0 -o spdx-json > sbom.spdx.json
cosign attest --predicate sbom.spdx.json --type spdx ghcr.io/acme-corp/myapp:1.0
```

### 33.6.3 Policy enforcement

**Kyverno** `verifyImages` policy:

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
```



```

metadata:
 name: verify-signed-images
spec:
 validationFailureAction: Enforce
 rules:
 - name: check-signature
 match:
 any:
 - resources:
 kinds: [Pod]
 verifyImages:
 - imageReferences: ["ghcr.io/acme-corp/*"]
 attestors:
 - entries:
 - keyless:
 subject: "https://github.com/acme-corp/myapp/.github/workflows/release.yml@refs/heads/main"
 issuer: "https://token.actions.githubusercontent.com"

```

Unsigned images are rejected at admission.

## 33.7 29.7 Registry operations

### 33.7.1 Mirroring and air-gapped pulls

Harbor and pull-through caches proxy Docker Hub to avoid rate limits and enable offline clusters.

### 33.7.2 Replication

Harbor and cloud registries replicate images across regions for disaster recovery and faster pulls.

### 33.7.3 Retention and garbage collection

Untagged manifests accumulate storage costs. Lifecycle policies expire old SHAs; run registry GC during maintenance windows.

### 33.7.4 RBAC

| Role          | Typical permissions      |
|---------------|--------------------------|
| CI robot      | Push to specific repos   |
| Cluster nodes | Pull only                |
| Developers    | Pull dev; no prod push   |
| Security      | Scan, quarantine, delete |

## 33.8 29.8 Supply chain best practices

1. **Pin base images** by digest in Dockerfile
2. **Build in CI** — developers do not push local images to prod registry
3. **Sign every release** artifact
4. **Verify signatures** in deploy pipeline and admission controller
5. **Immutable tags** on production repositories
6. **Separate registries** or repos per environment (dev vs prod)
7. **Audit logs** — who pushed/pulled what and when

## 33.9 29.9 Chapter summary

- Registries are the **distribution hub** for container images; use structured tagging and immutability.
  - Authenticate with **short-lived tokens** (ECR, OIDC) rather than long-lived passwords.
  - **Scan** images in CI and block critical CVEs; generate **SBOMs** for traceability.
  - **Sign** images with Cosign; **verify** at deploy time and with admission policies.
  - Apply **lifecycle policies** to control storage and reduce attack surface from stale images.
- 

## 33.10 Lab 29.1

1. Create an ECR or GHCR repository and push an image from CI.
  2. Enable scan-on-push and review findings for one image.
  3. Tag images with Git SHA and semver; deploy using digest in a K8s manifest.
- 

## 33.11 Lab 29.2

1. Sign an image with Cosign (keyless or key pair).
  2. Verify the signature in a deploy script before `kubectl apply`.
  3. Write a lifecycle policy retaining the last 20 images.
- 

## 33.12 Review questions

1. Why should production deployments reference digests instead of mutable tags?
  2. What is the difference between image signing and vulnerability scanning?
  3. How does ECR authentication differ from Docker Hub username/password?
  4. What does an immutable tag policy prevent?
  5. Where in the pipeline should unsigned images be rejected?
- 

Next: [Chapter 30 — Rootless Containers and Podman](#)

# 34 Chapter 30: Rootless Containers and Podman

DevOps Handbook — Part VII, Pages 561–580

## 34.1 30.1 Why rootless containers matter

Traditionally, Docker required a **root-owned daemon** ( `dockerd` ). Any user in the `docker` group effectively had root on the host—container escape or `-v /:/host` mounts could compromise the entire machine.

**Rootless containers** run the container runtime and workloads as an unprivileged user, mapping container UIDs to subordinate UID ranges on the host. Compromise of a container or runtime bug has **reduced blast radius**.

| Mode                   | Daemon user    | Container root maps to |
|------------------------|----------------|------------------------|
| Rootful Docker         | root           | host root (dangerous)  |
| Rootless Podman/Docker | user namespace | unprivileged host UID  |
| Kubernetes             | varies         | depends on PSA/ SCC    |

Production clusters still use rootful runtimes on nodes, but developers and CI increasingly adopt rootless tooling.

## 34.2 30.2 Podman overview

**Podman** (Red Hat / CNCF) is a daemonless OCI container engine. It pulls images, creates containers, and manages pods—compatible with Docker CLI for most workflows.

```
podman --version
podman run -d -p 8080:80 --name web docker.io/library/nginx:alpine
podman ps
podman logs web
podman exec -it web sh
podman stop web && podman rm web
```

### Docker-compatible aliases:

```
alias docker=podman
alias docker-compose=podman-compose
```

Podman stores data in `~/.local/share/containers/` for rootless users—no central daemon socket owned by root.

## 34.3 30.3 Rootless setup

On Fedora/RHEL/Ubuntu with upstream packages:

```
Install
sudo apt install podman slirp4netns fuse-overlayfs

Enable lingering for systemd user services (servers)
loginctl enable-linger $USER

Verify rootless
podman info --format '{{.Host.Security.Rootless}}'
```

`/etc/subuid` and `/etc/subgid` must allocate ranges:

```
testuser:100000:65536
```

Podman maps container UID 0 → host UID 100000, etc.

### 34.3.1 Limitations of rootless

| Feature                   | Rootless constraint                         |
|---------------------------|---------------------------------------------|
| Binding ports < 1024      | Needs <code>sysctl</code> or rootful helper |
| Some volume mounts        | Permission mapping complexity               |
| <code>host</code> network | Limited compared to rootful                 |
| cgroup devices            | May need delegated cgroup v2                |

Bind privileged ports rootlessly (Linux 5.7+):

```
echo "net.ipv4.ip_unprivileged_port_start=80" | \
sudo tee /etc/sysctl.d/99-unprivileged-ports.conf
sudo sysctl --system
```

## 34.4 30.4 Pods and systemd integration

Podman groups containers into **Pods** (shared network namespace, like a Kubernetes pod):

```
podman pod create --name app -p 8080:80
podman run -d --pod app --name web nginx:alpine
podman run -d --pod app --name metrics prom/prometheus
podman pod ps
```

Generate **systemd unit files** for production-like single-host deploys:

```
podman generate systemd --new --name web > container-web.service
systemctl --user enable --now container-web.service
journalctl --user -u container-web.service -f
```

`--new` recreates the container on each start from the image definition—closer to immutable infrastructure.

## 34.5 30.5 Podman Compose and Kubernetes YAML

**podman-compose:**

```
podman-compose -f compose.yaml up -d
```

**Play kube** — run Kubernetes manifests locally:

```
podman play kube deployment.yaml
podman play kube --down deployment.yaml
```

Useful for validating K8s YAML without a full cluster.

## 34.6 30.6 Rootless Docker (Moby)

Docker Engine 20.10+ supports **rootless mode** via `dockerd-rootless-setup.sh install`.

```
dockerd-rootless-setuptool.sh install
export DOCKER_HOST=unix:///${XDG_RUNTIME_DIR}/docker.sock
docker run hello-world
```

Rootless Docker still uses `containerd` and user namespaces but maintains the Docker CLI/API model. Podman is daemonless; choose based on team familiarity and RHEL vs Debian ecosystems.

## 34.7 30.7 Security comparison

Rootful Docker:  
Client → `/var/run/docker.sock` (group=docker) → root dockerd → container (root in cgroup)

Rootless Podman:  
Client → podman (same UID) → conmon + crun/runc → user namespace → mapped UIDs

Hardening tips for rootless dev:

- Do not `sudo podman` routinely—it defeats the model
- Use **SELinux** labels ( `:Z` on volume mounts on RHEL)
- Scan images same as production (**Trivy**)
- Prefer **read-only** rootfs and **cap-drop**

```
podman run --read-only --tmpfs /tmp --cap-drop=ALL myapp:1.0
```

## 34.8 30.8 Buildah and Skopeo

The **containers** ecosystem includes:

| Tool           | Role                                          |
|----------------|-----------------------------------------------|
| <b>Buildah</b> | Build OCI images without Dockerfile (or with) |
| <b>Skopeo</b>  | Copy/sign/inspect images between registries   |
| <b>Podman</b>  | Run containers and pods                       |
| <b>crun</b>    | OCI runtime (lightweight)                     |

Build without daemon:

```
buildah bud -t myapp:dev .
buildah push myapp:dev docker://ghcr.io/acme/myapp:dev
```

Skopeo copy (air-gapped mirror):

```
skopeo copy docker://docker.io/library/alpine:3.20 \
 docker://registry.internal/alpine:3.20
```

## 34.9 30.9 When to use Podman vs Docker vs Kubernetes

| Scenario                              | Recommendation                |
|---------------------------------------|-------------------------------|
| Developer laptop (security-conscious) | Rootless Podman               |
| Existing Docker Desktop team          | Docker with hardened settings |
|                                       |                               |

|                          |                                               |
|--------------------------|-----------------------------------------------|
| CI build agents          | <code>buildah / kaniko / docker buildx</code> |
| Production orchestration | Kubernetes (containerd/CRI-O runtime)         |
| Single-node edge         | Podman + systemd units                        |

Kubernetes nodes typically run **CRI-O** or **containerd**—not Podman—though Red Hat OpenShift uses CRI-O with similar roots.

## 34.10 30.10 Chapter summary

- **Rootless** containers reduce host compromise risk by eliminating a root-owned central daemon.
- **Podman** is daemonless, Docker CLI-compatible, and integrates with **systemd** and **pods**.
- Configure **subuid/subgid** ranges and understand rootless **port and volume** limitations.
- **Buildah** and **Skopeo** complete the toolchain for build and registry operations without Docker.
- Use rootless on workstations; enforce signing, scanning, and orchestrator policies in production.

## 34.11 Lab 30.1

1. Install Podman rootless; run nginx publishing port 8080.
2. Create a pod with two containers sharing a network namespace.
3. Generate a systemd user unit and manage the container with `systemctl --user`.

## 34.12 Lab 30.2

1. Convert a Docker Compose file to run under `podman-compose`.
2. Build an image with Buildah and push to GHCR with Skopeo or `podman push`.
3. Compare `podman info` security section between rootful and rootless modes.

## 34.13 Review questions

1. Why is membership in the `docker` group considered equivalent to root access?
2. How do user namespaces map container root to the host?
3. What problem do Podman pods solve compared to standalone containers?
4. Name two limitations of rootless mode when binding network ports.
5. When would you choose Buildah over `podman build`?

Next: [Chapter 31 — K8s Architecture](#)

# 35 Chapter 31: Kubernetes Architecture — Control Plane and Node Components

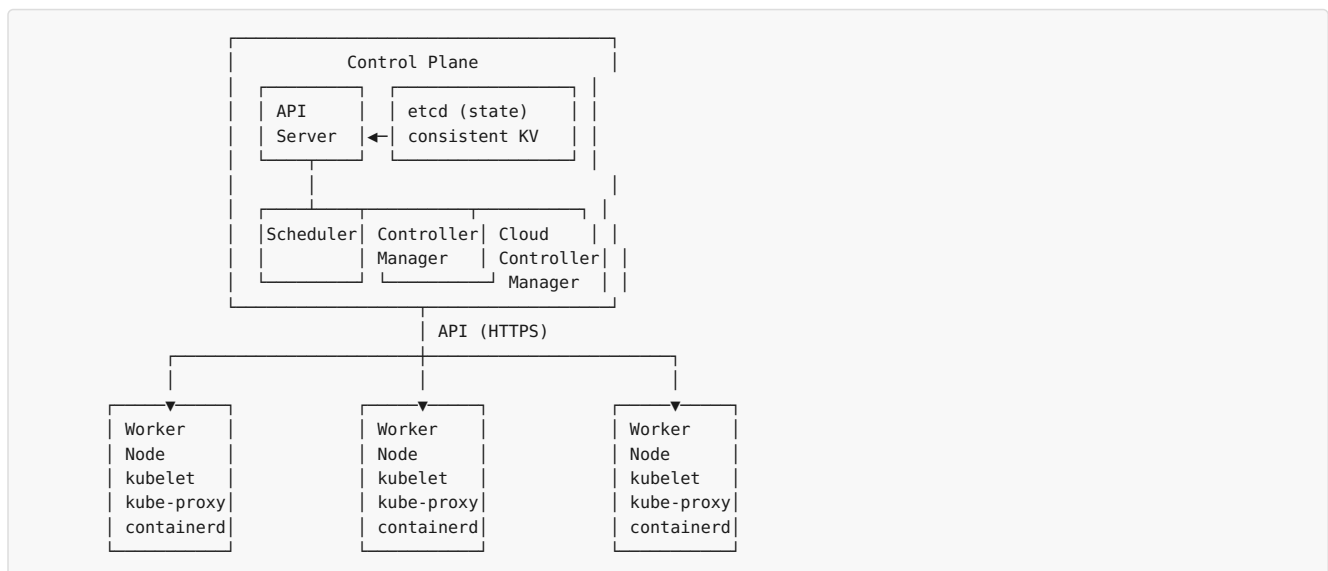
DevOps Handbook — Part VIII, Pages 581–605

## 35.1 31.1 What Kubernetes provides

**Kubernetes (K8s)** is a container orchestration platform that schedules workloads across a cluster, heals failed containers, scales replicas, rolls out updates, and exposes services. You declare **desired state** in YAML; controllers reconcile reality toward that state.

Kubernetes does not replace containers—it orchestrates **OCI images** running in **pods** on **nodes**, managed by a **control plane**.

## 35.2 31.2 Cluster architecture overview



## 35.3 31.3 Control plane components

### 35.3.1 API Server ( kube-apiserver )

The **front door** to the cluster. All tools ( `kubectl` , controllers, kubelet) talk to the API server over HTTPS. It validates requests, persists objects to etcd, and enforces authentication/authorization.

```
kubectl cluster-info
kubectl get --raw /healthz
kubectl api-resources
```

### 35.3.2 etcd

Distributed **key-value store** holding all cluster state: pods, services, secrets metadata, RBAC, etc. Production clusters run etcd with TLS, regular backups, and odd-numbered quorum (3 or 5 members).

```
Backup (on etcd member – simplified)
```

```
ETCDCTL_API=3 etcdctl snapshot save backup.db \
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key
```

Never expose etcd to the internet.

### 35.3.3 Scheduler ( kube-scheduler )

Watches unscheduled pods and assigns them to nodes based on resources, affinity, taints/tolerations, and topology spread.

### 35.3.4 Controller Manager ( kube-controller-manager )

Runs control loops: ReplicaSet, Deployment, Node, Job, EndpointSlice controllers, etc. Example: Deployment controller creates ReplicaSets; ReplicaSet controller creates pods.

### 35.3.5 Cloud Controller Manager (CCM)

Integrates with cloud APIs: node lifecycle, load balancers, routes, volumes. On EKS/GKE/AKS, the cloud provider manages much of this.

---

## 35.4 31.4 Node components

### 35.4.1 kubelet

Agent on each node. Registers the node, watches pod specs assigned to it, pulls images, starts containers via the **CRI** (containerd, CRI-O), reports status, and runs probes.

### 35.4.2 kube-proxy

Maintains network rules (iptables, ipvs, or eBPF) so **Services** get stable virtual IPs and load-balance traffic to pod endpoints.

### 35.4.3 Container runtime

Runs containers. **containerd** is the default in modern distros; Docker was removed as a direct runtime after K8s 1.24 (use cri-dockerd shim only if needed).

```
kubectctl get nodes -o wide
crictl ps # On node - CRI debugging
systemctl status kubelet
```

---

## 35.5 31.5 Kubernetes objects and namespaces

Everything is an API **resource** with `apiVersion`, `kind`, `metadata`, `spec`, `status`.

```
apiVersion: v1
kind: Pod
metadata:
 name: demo
 namespace: dev
 labels:
 app: demo
spec:
 containers:
 - name: nginx
 image: nginx:1.25-alpine
 ports:
```



```
- containerPort: 80
```

```
kubectl apply -f pod.yaml
kubectl get pods -n dev -o wide
kubectl describe pod demo -n dev
kubectl logs demo -n dev
kubectl delete pod demo -n dev
```

**Namespaces** isolate resources logically (not physically):

| Namespace            | Typical use            |
|----------------------|------------------------|
| kube-system          | Control plane addons   |
| kube-public          | Public cluster info    |
| default              | Avoid in prod          |
| dev / staging / prod | Environment separation |

## 35.6 31.6 kubectl and kubeconfig

~/.kube/config defines **clusters**, **users**, and **contexts**:

```
contexts:
- name: prod-eks
 context:
 cluster: prod-eks
 user: prod-eks-admin
 namespace: production
current-context: prod-eks
```

```
kubectl config get-contexts
kubectl config use-context staging-eks
kubectl config set-context --current --namespace=payments
```

Imperative vs declarative:

```
kubectl run nginx --image=nginx --port=80 # Imperative (avoid in prod)
kubectl apply -f deployment.yaml # Declarative (preferred)
kubectl diff -f deployment.yaml
```

## 35.7 31.7 Managed vs self-managed clusters

| Type                  | Examples            | You manage               |
|-----------------------|---------------------|--------------------------|
| Managed control plane | EKS, GKE, AKS       | Nodes, workloads, addons |
| Self-managed          | kubeadm, kops       | Control plane + nodes    |
| Local dev             | kind, minikube, k3d | Learning only            |

Managed services patch control plane versions; you still own upgrades, node AMIs, CNI, and workload security.

## 35.8 31.8 Addons and the CNCF landscape

Common cluster addons:

| Addon                                    | Purpose               |
|------------------------------------------|-----------------------|
| <b>CNI</b> (Calico, Cilium, AWS VPC CNI) | Pod networking        |
| <b>CoreDNS</b>                           | Service discovery DNS |
| <b>Metrics Server</b>                    | kubectl top, HPA      |
| <b>Ingress controller</b>                | HTTP routing          |
| <b>CSI driver</b>                        | Persistent volumes    |
| <b>cert-manager</b>                      | TLS certificates      |

Install with Helm or cloud marketplace; pin versions and test upgrades in staging.

---

## 35.9 31.9 High availability and etcd sizing

Production control plane:

- **3+ API server** instances behind load balancer
- **3 or 5 etcd** members across failure domains
- Regular **etcd backup** and restore drills
- **Separate** etcd from worker nodes

Worker nodes:

- Spread across **availability zones**
  - Use **PodDisruptionBudgets** during node drains
  - **Cluster Autoscaler** adds/removes nodes based on pending pods
- 

## 35.10 31.10 Chapter summary

- The **control plane** (API server, etcd, scheduler, controllers) manages desired state; **nodes** run workloads via kubelet and runtime.
  - All interaction flows through the **API server**; protect it with RBAC, audit logs, and network policies.
  - **etcd** is the source of truth—backup and secure it aggressively.
  - Use **declarative YAML** and **kubectl apply**; understand namespaces and contexts for multi-environment work.
- 

## 35.11 Lab 31.1

1. Create a local cluster with `kind` or `minikube`.
  2. Deploy a pod, inspect it with `describe`, and view logs.
  3. List control plane pods in `kube-system` and identify CoreDNS, kube-proxy.
- 

## 35.12 Lab 31.2

1. Configure two contexts (e.g., `kind` + `cloud sandbox`) and switch between them.
  2. Take an etcd snapshot on a kubeadm lab cluster (or document EKS backup approach).
  3. Draw your org's cluster diagram: control plane ownership, node pools, addons.
-

## 35.13 Review questions

1. Which component is the only one that writes to etcd for API requests?
2. What is the kubelet's relationship to the container runtime?
3. Why should etcd never be exposed publicly?
4. What is the difference between the scheduler and a controller?
5. Name three addons required for a minimally functional cluster network and DNS.

---

Next: [Chapter 32 — K8s Workloads](#)

# 36 Chapter 32: Kubernetes Workloads — Pod, Deployment, StatefulSet, DaemonSet

DevOps Handbook — Part VIII, Pages 606-630

## 36.1 32.1 Pods — the atomic unit

A **Pod** is the smallest deployable unit in Kubernetes: one or more containers sharing network and storage namespaces, scheduled together on one node.

```
apiVersion: v1
kind: Pod
metadata:
 name: web
 labels:
 app: web
spec:
 containers:
 - name: nginx
 image: nginx:1.25-alpine
 ports:
 - containerPort: 80
 resources:
 requests:
 cpu: 100m
 memory: 128Mi
 limits:
 cpu: 500m
 memory: 256Mi
 livenessProbe:
 httpGet:
 path: /
 port: 80
 initialDelaySeconds: 10
 periodSeconds: 10
 readinessProbe:
 httpGet:
 path: /
 port: 80
 periodSeconds: 5
```

| Probe     | Purpose                                  |
|-----------|------------------------------------------|
| liveness  | Restart if deadlocked                    |
| readiness | Include in Service endpoints             |
| startup   | Slow-start apps before liveness kicks in |

**Sidecar pattern** — helper container in same pod:

```
spec:
 containers:
 - name: app
 image: myapp:1.0
 volumeMounts:
 - name: logs
 mountPath: /var/log/app
 - name: log-shipper
 image: fluent/fluent-bit:2
 volumeMounts:
 - name: logs
 mountPath: /var/log/app
 volumes:
 - name: logs
 emptyDir: {}
```

Pods are ephemeral—do not manage them directly in production. Use controllers.

---

## 36.2 32.2 Deployments — stateless apps

A **Deployment** manages ReplicaSets to run N identical pod replicas with rolling updates and rollbacks.

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: api
 labels:
 app: api
spec:
 replicas: 3
 selector:
 matchLabels:
 app: api
 strategy:
 type: RollingUpdate
 rollingUpdate:
 maxSurge: 1
 maxUnavailable: 0
 template:
 metadata:
 labels:
 app: api
 spec:
 containers:
 - name: api
 image: ghcr.io/acme/api:v1.2.0
 ports:
 - containerPort: 8080
 env:
 - name: LOG_LEVEL
 value: info
```

```
kubectl apply -f deployment.yaml
kubectl rollout status deployment/api
kubectl set image deployment/api api=ghcr.io/acme/api:v1.3.0
kubectl rollout history deployment/api
kubectl rollout undo deployment/api
kubectl scale deployment/api --replicas=5
```

### 36.2.1 Rolling update flow

```
v1 v1 v1 → v1 v1 v2 → v1 v2 v2 → v2 v2 v2
```

`maxUnavailable: 0` ensures capacity during rollout; `maxSurge` allows extra pods temporarily.

---

## 36.3 32.3 ReplicaSets and labels

Deployments own **ReplicaSets**; ReplicaSets own **Pods**. Labels in `selector.matchLabels` must match pod template labels.

```
kubectl get rs
kubectl get pods --show-labels
```

Never change immutable selector fields on existing Deployments—create a new Deployment instead.

---

## 36.4 32.4 StatefulSets — stable identity and storage

**StatefulSets** assign stable network IDs ( `web-0` , `web-1` ) and persistent storage per replica. Use for databases, Kafka, ZooKeeper.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
 name: postgres
spec:
 serviceName: postgres-headless
 replicas: 3
 selector:
 matchLabels:
 app: postgres
 template:
 metadata:
 labels:
 app: postgres
 spec:
 containers:
 - name: postgres
 image: postgres:16
 ports:
 - containerPort: 5432
 volumeMounts:
 - name: data
 mountPath: /var/lib/postgresql/data
 volumeClaimTemplates:
 - metadata:
 name: data
 spec:
 accessModes: ["ReadWriteOnce"]
 storageClassName: gp3
 resources:
 requests:
 storage: 50Gi
```

Headless Service for stable DNS:

```
apiVersion: v1
kind: Service
metadata:
 name: postgres-headless
spec:
 clusterIP: None
 selector:
 app: postgres
 ports:
 - port: 5432
```

Pod DNS: `postgres-0.postgres-headless.default.svc.cluster.local`

StatefulSet pods are created/deleted in order (0, 1, 2...). Scaling and updates are slower but predictable.

---

## 36.5 32.5 DaemonSets — one pod per node

**DaemonSets** run a pod on every (or selected) node—agents, log collectors, CNI plugins, node exporters.

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
 name: node-exporter
spec:
 selector:
 matchLabels:
 app: node-exporter
 template:
 metadata:
 labels:
 app: node-exporter
 spec:
 hostNetwork: true
```

```

tolerations:
 - operator: Exists
containers:
 - name: exporter
 image: prom/node-exporter:v1.8.2
 ports:
 - containerPort: 9100

```

`tolerations: operator: Exists` schedules on tainted control plane nodes if needed.

## 36.6 32.6 Jobs and CronJobs

**Job** — run to completion:

```

apiVersion: batch/v1
kind: Job
metadata:
 name: db-migrate
spec:
 backoffLimit: 3
 template:
 spec:
 restartPolicy: OnFailure
 containers:
 - name: migrate
 image: ghcr.io/acme/api:v1.2.0
 command: ["npm", "run", "migrate"]

```

**CronJob** — scheduled jobs:

```

apiVersion: batch/v1
kind: CronJob
metadata:
 name: nightly-report
spec:
 schedule: "0 2 * * *"
 jobTemplate:
 spec:
 template:
 spec:
 restartPolicy: OnFailure
 containers:
 - name: report
 image: ghcr.io/acme/reporter:latest

```

## 36.7 32.7 Choosing a workload type

| Workload          | Controller  | When                        |
|-------------------|-------------|-----------------------------|
| HTTP API, workers | Deployment  | Stateless, horizontal scale |
| Postgres cluster  | StatefulSet | Stable ID + PVC per pod     |
| Log agent         | DaemonSet   | Per-node background service |
| Migration, batch  | Job         | Run once to completion      |
| Reports, backups  | CronJob     | Time-based                  |

## 36.8 32.8 Pod scheduling controls

```
spec:
```

```

affinity:
 podAntiAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
 - labelSelector:
 matchLabels:
 app: api
 topologyKey: kubernetes.io/hostname
 topologySpreadConstraints:
 - maxSkew: 1
 topologyKey: topology.kubernetes.io/zone
 whenUnsatisfiable: DoNotSchedule
 labelSelector:
 matchLabels:
 app: api

```

**Taints and tolerations** repel pods unless tolerated:

```

Node taint: dedicated=gpu:NoSchedule
spec:
 tolerations:
 - key: dedicated
 operator: Equal
 value: gpu
 effect: NoSchedule

```

---

## 36.9 32.9 Chapter summary

- **Pods** run containers; use probes and resource requests/limits.
  - **Deployments** manage stateless apps with rolling updates and rollbacks.
  - **StatefulSets** provide stable identity and storage for stateful systems.
  - **DaemonSets** run per-node agents; **Jobs/CronJobs** handle batch work.
  - Match workload type to data and scaling requirements—do not run databases as bare Deployments without understanding data loss risk.
- 

### 36.10 Lab 32.1

1. Deploy a 3-replica nginx Deployment; practice rollout, rollback, and scale.
  2. Add liveness and readiness probes; break the app and observe restarts vs traffic removal.
  3. Deploy a StatefulSet with `volumeClaimTemplates` and verify ordered pod names.
- 

### 36.11 Lab 32.2

1. Create a DaemonSet node-exporter and confirm one pod per node.
  2. Run a job migration container and inspect completion status.
  3. Add pod anti-affinity so replicas spread across nodes/zones.
- 

## 36.12 Review questions

1. What happens if readiness probe fails but liveness passes?
  2. Why do StatefulSets need a headless Service?
  3. When should you use a Job instead of a Deployment with `restartPolicy Never`?
  4. What do `maxSurge` and `maxUnavailable` control during rollouts?
  5. Why is running PostgreSQL as a single-replica Deployment risky?
-



Next: [Chapter 33 — K8s Services and Ingress](#)

# 37 Chapter 33: Kubernetes Services, Ingress, and Network Policies

DevOps Handbook — Part VIII, Pages 631–655

## 37.1 33.1 Kubernetes networking model

Every pod gets a cluster-routable **IP address**. Pods can talk to any other pod without NAT (CNI permitting). **Services** provide stable virtual IPs and DNS names in front of ephemeral pod IPs. **Ingress** (or Gateway API) routes HTTP/S from outside the cluster to Services.

Internet → Load Balancer → Ingress Controller → Service → Pod endpoints

## 37.2 33.2 Service types

```
apiVersion: v1
kind: Service
metadata:
 name: api
spec:
 type: ClusterIP
 selector:
 app: api
 ports:
 - port: 80
 targetPort: 8080
 protocol: TCP
```

| Type         | Reachability          | Use case                              |
|--------------|-----------------------|---------------------------------------|
| ClusterIP    | Internal only         | Default; microservice-to-microservice |
| NodePort     | Host port 30000–32767 | Dev, legacy exposure                  |
| LoadBalancer | Cloud LB → Service    | Public APIs on managed clouds         |
| ExternalName | DNS CNAME             | External SaaS alias                   |

DNS inside cluster: `api.default.svc.cluster.local` (short: `api` in same namespace).

```
kubectl get svc
kubectl get endpoints api
kubectl run -it --rm debug --image=busybox -- wget -qO- http://api
```

**Headless Service** ( `clusterIP: None` ) returns pod A records directly—used by StatefulSets.

## 37.3 33.3 Endpoints and EndpointSlices

The **Endpoints** (or **EndpointSlice**) controller populates backend pod IPs for Services matching `selector` . Readiness probe failure removes a pod from endpoints—traffic stops without deleting the pod.

```
kubectl get endpointslices -l kubernetes.io/service-name=api
```

Services without selectors can manually point to external IPs:

```
spec:
 ports:
 - port: 443
 type: ClusterIP

apiVersion: v1
kind: Endpoints
metadata:
 name: external-db
subsets:
 - addresses:
 - ip: 10.0.50.10
 ports:
 - port: 5432
```

## 37.4 33.4 Ingress — HTTP routing

**Ingress** requires an **Ingress Controller** (nginx, traefik, AWS ALB, etc.).

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: web-ingress
 annotations:
 cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
 ingressClassName: nginx
 tls:
 - hosts:
 - app.example.com
 secretName: app-tls
 rules:
 - host: app.example.com
 http:
 paths:
 - path: /
 pathType: Prefix
 backend:
 service:
 name: frontend
 port:
 number: 80
 - host: api.example.com
 http:
 paths:
 - path: /v1
 pathType: Prefix
 backend:
 service:
 name: api
 port:
 number: 80
```

| pathType               | Match                   |
|------------------------|-------------------------|
| Prefix                 | /api matches /api/users |
| Exact                  | Exact path only         |
| ImplementationSpecific | Controller-defined      |

Annotations configure TLS, redirects, rate limits, body size—varies by controller.

## 37.5 33.5 LoadBalancer and NodePort in cloud

EKS example — Service type LoadBalancer provisions an ELB:

```

apiVersion: v1
kind: Service
metadata:
 name: api-public
 annotations:
 service.beta.kubernetes.io/aws-load-balancer-type: "nlb"
 service.beta.kubernetes.io/aws-load-balancer-scheme: "internet-facing"
spec:
 type: LoadBalancer
 selector:
 app: api
 ports:
 - port: 443
 targetPort: 8080

```

Prefer **Ingress** or **Gateway API** over many LoadBalancers—cost and IP management.

---

## 37.6 33.6 Gateway API (modern alternative)

Gateway API separates **Gateway** (infrastructure) from **HTTPRoute** (application routing):

```

apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
 name: api-route
spec:
 parentRefs:
 - name: public-gateway
 hostnames:
 - api.example.com
 rules:
 - matches:
 - path:
 type: PathPrefix
 value: /v1
 backendRefs:
 - name: api
 port: 80

```

More expressive than Ingress; adoption growing on cloud providers.

---

## 37.7 33.7 Network Policies — zero trust inside the cluster

By default, all pods can reach all pods (**allow-all**). **NetworkPolicy** restricts traffic at L3/L4.

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: api-allow
 namespace: production
spec:
 podSelector:
 matchLabels:
 app: api
 policyTypes:
 - Ingress
 - Egress
 ingress:
 - from:
 - podSelector:
 matchLabels:
 app: frontend
 - namespaceSelector:
 matchLabels:
 name: ingress-nginx
 ports:
 - protocol: TCP
 port: 8080

```

```

egress:
 - to:
 - podSelector:
 matchLabels:
 app: postgres
 ports:
 - protocol: TCP
 port: 5432
 - to: # DNS
 - namespaceSelector: {}
 podSelector:
 matchLabels:
 k8s-app: kube-dns
 ports:
 - protocol: UDP
 port: 53

```

Requires a CNI that enforces NetworkPolicy (**Calico**, **Cilium**, **Weave**). AWS VPC CNI needs policy enforcement addon.

**Default deny pattern:**

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: default-deny-all
spec:
 podSelector: {}
 policyTypes: [Ingress, Egress]

```

Then add explicit allow policies per app.

## 37.8 33.8 Debugging connectivity

```

kubectl run netshoot --rm -it --image=nicolaka/netshoot -- bash
curl -v http://api.production.svc.cluster.local
nslookup api.production.svc.cluster.local
kubectl describe networkpolicy -n production
kubectl get ingress
kubectl logs -n ingress-nginx deploy/ingress-nginx-controller

```

| Symptom              | Check                                    |
|----------------------|------------------------------------------|
| 503 from Ingress     | Endpoints empty? Readiness failing?      |
| Timeout pod-to-pod   | NetworkPolicy deny? Wrong namespace DNS? |
| Works by IP not name | CoreDNS issue                            |
| External no route    | LB security groups, Ingress class        |

## 37.9 33.9 Chapter summary

- **Services** abstract pod IPs with ClusterIP, NodePort, and LoadBalancer types.
- **Ingress** (or **Gateway API**) routes HTTP host/path to Services; requires a controller.
- **NetworkPolicies** implement least-privilege pod communication—start with default deny in production namespaces.
- Always verify **endpoints** and **readiness** when traffic fails mysteriously.

## 37.10 Lab 33.1

1. Expose a Deployment with ClusterIP; reach it from a debug pod via DNS.
  2. Install nginx Ingress Controller; route two hostnames to two Services.
  3. Add TLS with cert-manager (or self-signed for lab).
- 

## 37.11 Lab 33.2

1. Apply default-deny NetworkPolicy in a namespace.
  2. Add allow rules so frontend → api → postgres works; verify blocked paths fail.
  3. Document required egress to kube-dns for DNS resolution.
- 

## 37.12 Review questions

1. How does a Service know which pods to send traffic to?
  2. What is the difference between ClusterIP and Headless Services?
  3. Why does Ingress require a separate controller?
  4. What happens to Service traffic when all pods fail readiness?
  5. Which CNIs enforce NetworkPolicy, and what breaks if yours does not?
- 

Next: [\*Chapter 34 — K8s Config and Storage\*](#)

# 38 Chapter 34: Kubernetes ConfigMaps, Secrets, and Storage (PV/PVC)

DevOps Handbook — Part VIII, Pages 656-675

## 38.1 34.1 Separating config from images

Twelve-factor apps store config in the environment, not in images. Kubernetes provides **ConfigMaps** for non-sensitive config and **Secrets** for sensitive data—mounted as files or injected as environment variables.

| Object    | Data       | Base64               | Encryption at rest  |
|-----------|------------|----------------------|---------------------|
| ConfigMap | Plain text | No                   | Optional            |
| Secret    | Sensitive  | Yes (not encryption) | Enable KMS provider |

Never commit plaintext Secrets to Git. Use **Sealed Secrets**, **External Secrets Operator**, or **SOPS**.

## 38.2 34.2 ConfigMaps

```
apiVersion: v1
kind: ConfigMap
metadata:
 name: api-config
data:
 LOG_LEVEL: debug
 APP_ENV: staging
 nginx.conf: |
 server {
 listen 80;
 location / { proxy_pass http://127.0.0.1:8080; }
 }
```

Reference in a Deployment:

```
spec:
 containers:
 - name: api
 image: ghcr.io/acme/api:1.0
 envFrom:
 - configMapRef:
 name: api-config
 env:
 - name: LOG_LEVEL
 valueFrom:
 configMapKeyRef:
 name: api-config
 key: LOG_LEVEL
 volumeMounts:
 - name: config-vol
 mountPath: /etc/nginx
 readOnly: true
 volumes:
 - name: config-vol
 configMap:
 name: api-config
 items:
 - key: nginx.conf
 path: nginx.conf
```

```
kubectl create configmap api-config --from-literal=LOG_LEVEL=info
kubectl create configmap app-config --from-file=./config/
```

```
kubectl get configmap api-config -o yaml
```

**Immutable ConfigMaps** (recommended): set `immutable: true` to prevent accidental updates causing rolling reload storms.

---

## 38.3 34.3 Secrets

```
apiVersion: v1
kind: Secret
metadata:
 name: db-credentials
type: Opaque
stringData: # Plain text in manifest – encode on apply
 username: appuser
 password: s3cr3t-db-pwd
```

```
kubectl create secret generic db-credentials \
 --from-literal=username=appuser \
 --from-literal=password='s3cr3t'

kubectl create secret docker-registry ghcr-creds \
 --docker-server=ghcr.io \
 --docker-username=user \
 --docker-password=$TOKEN
```

Mount as files (preferred for apps that read credential files):

```
volumeMounts:
- name: db-secret
 mountPath: /etc/secrets/db
 readOnly: true
volumes:
- name: db-secret
 secret:
 secretName: db-credentials
 defaultMode: 0400
```

**RBAC** restricts who reads Secrets. Enable **encryption at rest** with KMS (AWS, GCP, Azure integrations).

### 38.3.1 External Secrets Operator

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
 name: db-credentials
spec:
 refreshInterval: 1h
 secretStoreRef:
 name: aws-secrets-manager
 kind: ClusterSecretStore
 target:
 name: db-credentials
 data:
 - secretKey: password
 remoteRef:
 key: prod/bookstore/db
 property: password
```

Secrets sync from AWS Secrets Manager, Vault, or Parameter Store into native K8s Secrets.

---

## 38.4 34.4 Storage basics — PV, PVC, StorageClass

Pod filesystems are ephemeral. **PersistentVolume (PV)** is cluster storage; **PersistentVolumeClaim (PVC)** is a



pod’s request for storage.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
 name: gp3
provisioner: ebs.csi.aws.com
parameters:
 type: gp3
 encrypted: "true"
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: data-pvc
spec:
 accessModes:
 - ReadWriteOnce
 storageClassName: gp3
 resources:
 requests:
 storage: 20Gi
```

```
spec:
 containers:
 - name: postgres
 image: postgres:16
 volumeMounts:
 - name: data
 mountPath: /var/lib/postgresql/data
 volumes:
 - name: data
 persistentVolumeClaim:
 claimName: data-pvc
```

| Access mode         | Meaning                          |
|---------------------|----------------------------------|
| ReadWriteOnce (RWO) | One node read/write              |
| ReadOnlyMany (ROX)  | Many nodes read                  |
| ReadWriteMany (RWX) | Many nodes read/write (EFS, NFS) |

**WaitForFirstConsumer** delays volume provisioning until pod is scheduled—important for zonal disks.

### 38.5 34.5 StatefulSet storage pattern

StatefulSets use **volumeClaimTemplates** for per-replica PVCs ( `data-postgres-0` , `data-postgres-1` ).

Scaling down does **not** delete PVCs by default—data survives for rescheduling. Orphan PVC cleanup must be deliberate.

```
kubectl get pvc
kubectl describe pvc data-pvc
kubectl exec postgres-0 -- df -h /var/lib/postgresql/data
```

### 38.6 34.6 Config reload and updates

Changing a ConfigMap mounted as env var **does not** update running containers—restart required. Mounted **volumes** can refresh (with delay; subPath does not auto-update).

Patterns:

- **Reloader** operator watches ConfigMaps/Secrets and rolls Deployments
- Sidecar **inotify** watcher triggers app reload
- Immutable versions: `api-config-v3` → update deployment reference

For Secrets, prefer short-lived tokens and automatic rotation via External Secrets.

---

## 38.7 34.7 Backup and snapshots

CSI **VolumeSnapshot**:

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
 name: data-snap-20250901
spec:
 volumeSnapshotClassName: csi-aws-vsc
 source:
 persistentVolumeClaimName: data-pvc
```

Backup tools: **Velero** (cluster resources + PV snapshots), cloud-native backup services.

Test restores regularly—an untested backup is wishful thinking.

---

## 38.8 34.8 Security checklist

- No Secrets in container images or public Git
  - Enable **encryption at rest** for etcd Secrets
  - Restrict Secret get/list with RBAC
  - Use **readOnly** volume mounts where possible
  - Prefer **projected volumes** combining multiple sources
  - Scan manifests in CI for hardcoded credentials (**gitleaks**, **checkov**)
- 

## 38.9 34.9 Chapter summary

- **ConfigMaps** hold non-sensitive config; **Secrets** hold sensitive data—neither replaces a secrets manager at scale.
  - Mount config as **volumes** for file-based apps; use **env** for simple key-value.
  - **PVCs** request storage; **StorageClasses** enable dynamic provisioning.
  - Match **access modes** and **StorageClass** topology to workload (RWO for EBS, RWX for shared files).
  - Integrate **External Secrets** and **volume snapshots** for production operations.
- 

## 38.10 Lab 34.1

1. Create ConfigMap and Secret; deploy an app consuming both via env and volume mounts.
  2. Provision a PVC with dynamic StorageClass; verify data survives pod deletion.
  3. Expand a PVC if `allowVolumeExpansion` is enabled.
- 

## 38.11 Lab 34.2

1. Deploy External Secrets Operator (or Sealed Secrets) syncing one secret from a vault/cloud store.
  2. Take a VolumeSnapshot and restore to a new PVC.
  3. Add Reloader or manual rollout workflow when ConfigMap changes.
- 

## 38.12 Review questions

1. Why is base64 encoding in Secrets not considered encryption?
  2. What happens to a PVC when its pod is deleted?
  3. When should you use ReadWriteMany instead of ReadWriteOnce?
  4. Why do env-var ConfigMap changes not appear in running pods?
  5. What does `volumeBindingMode: WaitForFirstConsumer` prevent?
- 

Next: [\*Chapter 35 — Helm and GitOps\*](#)

# 39 Chapter 35: Helm, Kustomize, and GitOps (Argo CD)

---

DevOps Handbook — Part VIII, Pages 676-700

---

## 39.1 35.1 The packaging problem

Raw YAML does not scale: duplicated labels, repeated image tags across environments, and manual `kubectl apply` drift from Git. **Helm** packages charts; **Kustomize** overlays bases; **GitOps** (Argo CD, Flux) continuously reconciles cluster state from Git.

```
Git (desired state) → CI builds image → GitOps controller → Kubernetes (actual state)
```

## 39.2 35.2 Helm charts

A **chart** is a directory of templated Kubernetes manifests plus metadata.

```
mychart/
├── Chart.yaml
├── values.yaml
├── templates/
│ ├── deployment.yaml
│ ├── service.yaml
│ ├── ingress.yaml
│ └── _helpers.tpl
└── charts/ # Subchart dependencies
```

Chart.yaml :

```
apiVersion: v2
name: api
description: Bookstore API
type: application
version: 0.1.0
appVersion: "1.2.0"
```

values.yaml :

```
replicaCount: 2
image:
 repository: ghcr.io/acme/api
 tag: "1.2.0"
 pullPolicy: IfNotPresent
service:
 type: ClusterIP
 port: 80
ingress:
 enabled: true
 host: api.example.com
resources:
 requests:
 cpu: 100m
 memory: 128Mi
```

templates/deployment.yaml excerpt:

```
apiVersion: apps/v1
kind: Deployment
metadata:
```

```

name: {{ include "api.fullname" . }}
spec:
 replicas: {{ .Values.replicaCount }}
 template:
 spec:
 containers:
 - name: api
 image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
 ports:
 - containerPort: 8080

```

Commands:

```

helm repo add bitnami https://charts.bitnami.com/bitnami
helm search repo nginx
helm install my-nginx bitnami/nginx --namespace web --create-namespace
helm upgrade my-nginx bitnami/nginx -f values-prod.yaml
helm list
helm uninstall my-nginx
helm template api ./mychart -f values-staging.yaml # Render locally

```

Override per environment:

```

values-prod.yaml
replicaCount: 5
image:
 tag: "1.2.0"
ingress:
 host: api.example.com

```

## 39.3 35.3 Helm hooks and releases

**Hooks** run Jobs at pre/post install/upgrade ( `helm.sh/hook: pre-upgrade` ). Use for DB migrations—carefully, with rollback strategy.

**Release** = installed chart instance stored as Secrets in `sh.helm.release.v1.*` (history for rollback).

```

helm history api
helm rollback api 3

```

Helm 3 is **client-side only**—no Tiller. RBAC applies to the user or CI service account running Helm.

## 39.4 35.4 Kustomize — patch without templates

**Kustomize** is built into `kubectl apply -k`. No templating language—bases + overlays with patches and image transforms.

```

deploy/
├── base/
│ ├── kustomization.yaml
│ ├── deployment.yaml
│ └── service.yaml
└── overlays/
 ├── staging/
 │ ├── kustomization.yaml
 │ └── patch-replicas.yaml
 └── production/
 ├── kustomization.yaml
 └── patch-resources.yaml

```

base/kustomization.yaml :

```

apiVersion: kustomize.config.k8s.io/v1beta1

```

```
kind: Kustomization
resources:
- deployment.yaml
- service.yaml
commonLabels:
 app: api
```

overlays/production/kustomization.yaml :

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- ../../base
namespace: production
replicas:
- name: api
 count: 5
images:
- name: ghcr.io/acme/api
 newTag: v1.2.0
patches:
- path: patch-resources.yaml
configMapGenerator:
- name: api-config
 literals:
 - LOG_LEVEL=warn
```

```
kubectl apply -k deploy/overlays/production
kubectl diff -k deploy/overlays/staging
```

| Tool      | Strength                                | Weakness                |
|-----------|-----------------------------------------|-------------------------|
| Helm      | Packaging, versioning, ecosystem charts | Go templates complexity |
| Kustomize | Simple overlays, native kubectl         | Less packaging metadata |

Many teams use **Helm charts rendered by Kustomize** ( `helm template` | `kustomize` ) or Helm with post-render Kustomize.

## 39.5 35.5 GitOps principles

1. **Git is the source of truth** for desired cluster state
2. **Declarative** manifests (YAML, Helm, Kustomize)
3. **Automated reconciliation** — controller applies diffs
4. **Observable** — drift visible in UI/CLI

Anti-pattern: `kubectl apply` from laptops to production. GitOps makes changes via PR merge.

## 39.6 35.6 Argo CD

**Argo CD** watches Git repos and syncs Applications to clusters.

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
 name: bookstore-api
 namespace: argocd
spec:
 project: default
 source:
 repoURL: https://github.com/acme-corp/gitops.git
 targetRevision: main
 path: apps/bookstore/overlays/production
 destination:
```

```

server: https://kubernetes.default.svc
namespace: production
syncPolicy:
 automated:
 prune: true
 selfHeal: true
 syncOptions:
 - CreateNamespace=true

```

```

argocd app list
argocd app diff bookstore-api
argocd app sync bookstore-api
argocd app rollback bookstore-api

```

**App of Apps** pattern bootstraps many Applications from one root manifest.

### 39.6.1 Sync strategies

| Mode      | Behavior                          |
|-----------|-----------------------------------|
| Manual    | Human clicks Sync                 |
| Automated | Sync on Git change                |
| selfHeal  | Revert manual kubectl edits       |
| prune     | Delete resources removed from Git |

Use **prune** cautiously with shared resources; test in staging first.

## 39.7 35.7 Promotion pipeline

Typical flow:

1. CI builds image → pushes `ghcr.io/acme/api:abc123`
2. CI or bot opens PR updating `newTag` in `overlays/staging`
3. Argo CD syncs staging; run smoke tests
4. PR promotes same tag to `overlays/production`
5. Argo CD syncs prod (manual approval optional via Argo CD projects)

**Image updater** tools (Argo CD Image Updater, Flux image automation) can write back to Git when new tags appear—pin policies to semver or regex.

## 39.8 35.8 Flux (brief comparison)

**Flux CD** uses `GitRepository` + `Kustomization`/`HelmRelease` CRDs:

```

apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
 name: gitops
spec:
 url: https://github.com/acme-corp/gitops
 ref:
 branch: main

apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
 name: production
spec:
 sourceRef:
 kind: GitRepository
 name: gitops

```

```
path: ./apps/bookstore/overlays/production
interval: 5m
prune: true
```

|               | Argo CD         | Flux                    |
|---------------|-----------------|-------------------------|
| UI            | Rich web UI     | CLI / Weave GitOps UI   |
| Model         | Application CRD | Modular controllers     |
| Multi-cluster | ApplicationSet  | Cluster API integration |

Both are CNCF graduated/incubating; choice often organizational.

---

## 39.9 35.9 Secrets in GitOps

Never plain Secrets in Git. Options:

- **Sealed Secrets** — encrypt for cluster-only decryption
- **SOPS + KMS** — Mozilla SOPS with age/PGP/cloud KMS
- **External Secrets** — reference cloud vault; not stored in Git

Argo CD supports helm-secrets, ksops plugins for encrypted manifests.

---

## 39.10 35.10 Chapter summary

- **Helm** packages and parameterizes Kubernetes apps; **Kustomize** patches bases per environment.
  - **GitOps** reconciles cluster state from Git—eliminating ad-hoc kubectl changes.
  - **Argo CD** and **Flux** automate deploys with visibility into drift and sync status.
  - Protect secrets with encryption or external stores; promote images via Git PRs, not manual image bumps on nodes.
- 

## 39.11 Lab 35.1

1. Package a Deployment + Service as a Helm chart; install with staging and prod values files.
  2. Replicate the same app with Kustomize base + two overlays.
  3. Compare rendered YAML from `helm template` vs `kubectl kustomize`.
- 

## 39.12 Lab 35.2

1. Install Argo CD on a test cluster; deploy an Application from a Git repo.
  2. Change a replica count in Git; observe auto-sync (if enabled).
  3. Make a manual `kubectl scale` change; observe selfHeal revert it.
- 

## 39.13 Review questions

1. What is the difference between a Helm chart and a Helm release?
2. When would Kustomize be preferable to Helm?
3. What risks does `prune: true` introduce?
4. How does GitOps change the role of `kubectl apply` in production?



5. Name two approaches for storing secrets in a GitOps repository.

---

Next: [\*Chapter 36 — K8s RBAC and Security\*](#)

# 40 Chapter 36: Kubernetes RBAC, Pod Security, and Hardening

---

DevOps Handbook — Part VIII, Pages 701–720

---

## 40.1 36.1 Defense in depth for Kubernetes

Cluster security spans authentication (who), authorization (what they can do), admission control (what can be created), network policy (who can talk to whom), and workload hardening (how pods run). No single control is sufficient—layer them.

```
Request → AuthN (cert/OIDC) → AuthZ (RBAC) → Admission (PSA/OPA) → Pod sandbox → NetworkPolicy
```

---

## 40.2 36.2 Authentication overview

Human users should not use long-lived client certificates in production. Prefer **OIDC** integration (Google, Azure AD, Okta) via `kubect1` and cloud IAM for CI.

Service accounts authenticate **in-cluster** workloads:

```
apiVersion: v1
kind: ServiceAccount
metadata:
 name: api
 namespace: production
```

```
spec:
 serviceAccountName: api
 containers:
 - name: api
 image: ghcr.io/acme/api:1.0
```

Each ServiceAccount gets a mounted token (bound to audience in K8s 1.24+). Use **IRSA** (EKS), **Workload Identity** (GKE), or **Azure Workload Identity** to map SA → cloud IAM role—never static cloud keys in pods.

```
kubectl auth can-i create deployments --namespace production
kubectl auth can-i --list --as=system:serviceaccount:production:api
```

---

## 40.3 36.3 RBAC — Roles and Bindings

RBAC grants permissions via **Role** (namespace) or **ClusterRole** (cluster-wide), bound with **RoleBinding** or **ClusterRoleBinding**.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
 name: deployer
 namespace: staging
rules:
 - apiGroups: ["apps"]
 resources: ["deployments"]
 verbs: ["get", "list", "watch", "create", "update", "patch"]
 - apiGroups: [""]
 resources: ["pods", "pods/log"]
```

```

 verbs: ["get", "list", "watch"]

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
 name: ci-deployer
 namespace: staging
subjects:
- kind: User
 name: github-actions-ci
 apiGroup: rbac.authorization.k8s.io
roleRef:
 kind: Role
 name: deployer
 apiGroup: rbac.authorization.k8s.io

```

Least privilege verbs:

| Verb                  | Risk        |
|-----------------------|-------------|
| get, list, watch      | Read        |
| create, update, patch | Modify      |
| delete                | Destructive |
| *                     | Avoid       |

**ClusterRole** for read-only auditors:

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
 name: view-all
rules:
- apiGroups: ["*"]
 resources: ["*"]
 verbs: ["get", "list", "watch"]

```

Bind to group `viewers` from OIDC. Never grant `cluster-admin` casually—use break-glass accounts with MFA and audit logging.

## 40.4 36.4 Aggregated roles and default bindings

Built-in ClusterRoles: `view`, `edit`, `admin`, `cluster-admin`. Namespace `RoleBinding` to `cluster-admin` is still namespace-scoped for namespaced resources but powerful.

Audit bindings:

```

kubectl get rolebindings,clusterrolebindings --all-namespaces
kubectl describe clusterrolebinding cluster-admin

```

Remove default permissions for unauthenticated or overly broad groups in managed clusters.

## 40.5 36.5 Pod Security Admission (PSA)

**Pod Security Admission** replaces deprecated `PodSecurityPolicy`. Namespace labels enforce **privileged**, **baseline**, or **restricted** profiles.

```

apiVersion: v1
kind: Namespace
metadata:
 name: production
 labels:

```

```
pod-security.kubernetes.io/enforce: restricted
pod-security.kubernetes.io/enforce-version: latest
pod-security.kubernetes.io/audit: restricted
pod-security.kubernetes.io/warn: restricted
```

| Profile    | Constraints                                           |
|------------|-------------------------------------------------------|
| privileged | Unrestricted                                          |
| baseline   | No hostNetwork, hostPID, privileged containers        |
| restricted | Non-root, drop caps, seccomp, no privilege escalation |

Restricted-compliant pod excerpt:

```
spec:
 securityContext:
 runAsNonRoot: true
 seccompProfile:
 type: RuntimeDefault
 containers:
 - name: api
 image: ghcr.io/acme/api:1.0
 securityContext:
 allowPrivilegeEscalation: false
 readOnlyRootFilesystem: true
 capabilities:
 drop: ["ALL"]
 volumeMounts:
 - name: tmp
 mountPath: /tmp
 volumes:
 - name: tmp
 emptyDir: {}
```

Test violations:

```
kubectl label namespace production pod-security.kubernetes.io/enforce=restricted --overwrite
kubectl apply -f pod.yaml # Should fail if non-compliant
```

---

## 40.6 36.6 OPA Gatekeeper and Kyverno

Admission controllers enforce custom policies beyond PSA.

**Kyverno** — validate/mutate/generate:

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
 name: require-non-root
spec:
 validationFailureAction: Enforce
 rules:
 - name: check-runasnonroot
 match:
 any:
 - resources:
 kinds: [Pod]
 validate:
 message: "Containers must run as non-root"
 pattern:
 spec:
 containers:
 - securityContext:
 runAsNonRoot: true
```

**Gatekeeper** uses Rego (see Chapter 25). Start with PSA labels, add policy engines for org-specific rules (required labels, approved registries, resource limits).

---

## 40.7 36.7 Workload hardening checklist

| Control           | Implementation                                            |
|-------------------|-----------------------------------------------------------|
| Non-root user     | <code>runAsUser: 10001, runAsNonRoot: true</code>         |
| Read-only rootfs  | <code>readOnlyRootFilesystem: true</code> + tmpfs volumes |
| Drop capabilities | <code>capabilities.drop: [ALL]</code>                     |
| No privileged     | <code>privileged: false</code>                            |
| Resource limits   | Prevent noisy neighbor / DoS                              |
| Seccomp/AppArmor  | <code>seccompProfile: RuntimeDefault</code>               |
| Image pull policy | <code>IfNotPresent</code> + pinned tags/digests           |
| NetworkPolicy     | Default deny + explicit allow                             |

```
resources:
 requests:
 cpu: 100m
 memory: 128Mi
 limits:
 cpu: "1"
 memory: 512Mi
```

Pods without limits may be BestEffort QoS and evicted first under pressure.

---

## 40.8 36.8 Secrets, audit logs, and API security

Enable **audit logging** on API server (managed clouds: control plane logs). Ship to SIEM; alert on `secrets` access and `clusterrolebinding` changes.

Restrict **etcd** and **API server** network access. Disable anonymous auth. Rotate credentials.

### RBAC for Secrets:

```
rules:
- apiGroups: [""]
 resources: ["secrets"]
 verbs: ["get", "list"]
```

Limit to operators and controllers—not all developers.

---

## 40.9 36.9 Node and supply chain security

- **Minimal node OS** — Bottlerocket, COS, hardened AMIs
- **Auto-upgrade** node pools on CVE patches
- **Admission** — only signed images from approved registries (Chapter 29)
- **Falco** — runtime threat detection (syscall anomalies)
- **Vulnerability scanning** — workload images in CI and admission

Disable mounting Docker socket into pods ( `/var/run/docker.sock` )—instant cluster compromise vector.

---

## 40.10 36.10 Incident response primitives

```
Contain compromised pod
kubectl cordon node-name
kubectl delete pod compromised-pod --grace-period=0
kubectl scale deployment/suspicious --replicas=0

Investigate
kubectl auth can-i --list --as=system:serviceaccount:ns:sa
kubectl get events -A --sort-by='.lastTimestamp'
kubectl logs -n kube-system -l component=kube-apiserver
```

Maintain runbooks for credential rotation, etcd restore, and compromised service account token revocation.

---

## 40.11 36.11 Chapter summary

- **RBAC** enforces least privilege—prefer namespace Roles over cluster-admin.
  - **Service accounts** should map to cloud IAM via workload identity, not static keys.
  - **Pod Security Admission** enforces baseline/restricted profiles at namespace level.
  - Layer **admission policies**, **NetworkPolicies**, and **runtime monitoring** for production hardening.
  - Enable **audit logs** and test restricted pod specs before enforcing in production namespaces.
- 

## 40.12 Lab 36.1

1. Create a Role allowing deploy updates only in `staging`; bind to a test user or SA.
  2. Label a namespace `restricted` and fix a Deployment until it passes PSA.
  3. Run `kubectl auth can-i` checks validating your RBAC design.
- 

## 40.13 Lab 36.2

1. Install Kyverno or Gatekeeper; deny containers running as root cluster-wide.
  2. Apply default-deny NetworkPolicy plus explicit allows for one app stack.
  3. Review API audit logs (or simulate with `kubectl` commands) for sensitive resource access.
- 

## 40.14 Review questions

1. What is the difference between a Role and a ClusterRole?
  2. Which PSA profile should production app namespaces use?
  3. Why should pods not use cluster-admin ServiceAccounts?
  4. Name three fields required for a restricted Pod Security profile.
  5. How do admission controllers differ from RBAC?
- 

Next: [Chapter 37 — CI/CD Pipeline Design](#)

# 41 Chapter 37: CI/CD Concepts and Pipeline Design

DevOps Handbook — Part IX, Pages 721–745

## 41.1 37.1 Continuous Integration (CI)

CI means developers merge code to a shared branch **frequently** (at least daily), and each merge triggers an **automated build and test** pipeline.

Goals:

- Detect integration bugs early
- Keep `main` always deployable
- Reduce merge hell from long-lived branches

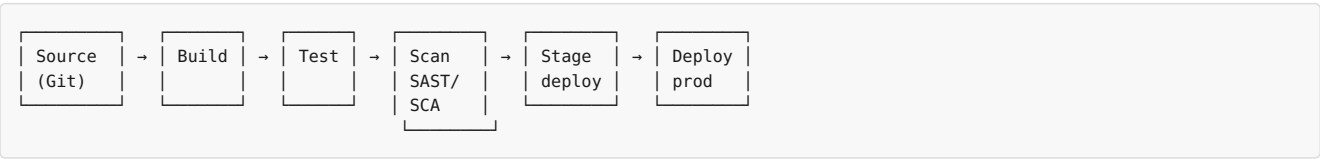
## 41.2 37.2 Continuous Delivery vs Continuous Deployment

| Practice              | Definition                                                                            |
|-----------------------|---------------------------------------------------------------------------------------|
| Continuous Delivery   | Every change is releasable; deploy to prod is a <b>manual</b> (or one-click) decision |
| Continuous Deployment | Every change that passes tests is <b>automatically</b> deployed to production         |

Most enterprises use **Continuous Delivery** with approval gates for production.

## 41.3 37.3 Pipeline stages

A typical pipeline:



| Stage        | Activities                                      |
|--------------|-------------------------------------------------|
| Source       | Webhook on push/PR                              |
| Build        | Compile, docker build, artifact upload          |
| Test         | Unit, integration, contract tests               |
| Scan         | SAST, dependency scan, container scan, IaC scan |
| Stage deploy | Deploy to staging; smoke tests                  |
| Prod deploy  | Blue/green, canary, or rolling; approval gate   |

## 41.4 37.4 GitHub Actions example

.github/workflows/ci.yml:

```
name: CI

on:
 push:
 branches: [main]
 pull_request:
 branches: [main]

jobs:
 build-and-test:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4

 - name: Setup Node
 uses: actions/setup-node@v4
 with:
 node-version: "20"
 cache: npm

 - run: npm ci
 - run: npm run lint
 - run: npm test -- --coverage

 - name: Build Docker image
 run: docker build -t myapp:${{ github.sha }} .

 - name: Scan image
 uses: aquasecurity/trivy-action@master
 with:
 image-ref: myapp:${{ github.sha }}
 severity: CRITICAL,HIGH
 exit-code: 1

 deploy-staging:
 needs: build-and-test
 if: github.ref == 'refs/heads/main'
 runs-on: ubuntu-latest
 environment: staging
 steps:
 - uses: actions/checkout@v4
 - name: Deploy to staging
 run: ./scripts/deploy.sh staging ${{ github.sha }}
```

## 41.5 37.5 Branching and trunk-based development

**Trunk-based development** (recommended for CI/CD):

- Short-lived feature branches (< 2 days)
- Frequent merges to `main`
- Feature flags hide incomplete work

Avoid long-running `develop` branches that diverge from production reality.

## 41.6 37.6 Deployment strategies

| Strategy   | How it works                              | Risk                |
|------------|-------------------------------------------|---------------------|
| Rolling    | Replace instances gradually               | Medium              |
| Blue/green | Switch traffic between two identical envs | Low downtime        |
| Canary     | Route small % traffic to new version      | Lowest blast radius |
| Recreate   | Tear down old, start new                  | Downtime            |



---

## 41.7 37.7 Pipeline secrets and artifacts

- Store secrets in **GitHub Secrets**, **Vault**, or cloud secret managers—never in YAML.
  - Pin action versions ( `@v4` , commit SHA) for supply chain security.
  - Upload build artifacts (JAR, Docker image digest) for downstream deploy jobs.
  - Sign artifacts (Sigstore/cosign) for provenance.
- 

## 41.8 37.8 DORA metrics in your pipeline

Track:

1. **Deployment frequency** — How often you deploy to prod
2. **Lead time for changes** — Commit to prod duration
3. **Change failure rate** — % of deploys causing incidents
4. **MTTR** — Time to restore after failure

Instrument pipelines to emit these metrics to your observability platform.

---

## 41.9 37.9 Chapter summary

- CI validates every merge; CD/CD automates release to staging/prod.
  - Design pipelines with **build** → **test** → **scan** → **deploy** stages.
  - Prefer trunk-based dev, pinned dependencies, and safe deployment strategies.
- 

## 41.10 Lab 37.1

Build a full GitHub Actions pipeline for a sample app: lint, test, Docker build, Trivy scan, and deploy to staging on merge to `main`.

---

*Continue: Chapter 38 — GitHub Actions Deep Dive (outline in TOC)*

# 42 Chapter 38: GitHub Actions — Workflows, Secrets, Runners

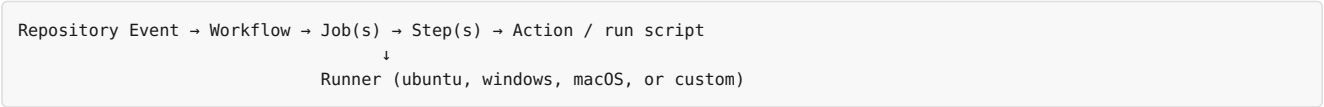
DevOps Handbook — Part IX, Pages 746–770

## 42.1 38.1 GitHub Actions architecture

**GitHub Actions** is GitHub’s native CI/CD platform. Workflows are triggered by repository events (push, pull request, schedule, manual dispatch) and execute as one or more **jobs** on **runners**—virtual machines or containers that run your steps.

Core components:

| Component   | Description                                                                                      |
|-------------|--------------------------------------------------------------------------------------------------|
| Workflow    | YAML file in <code>.github/workflows/</code> defining automation                                 |
| Event       | Trigger ( <code>push</code> , <code>pull_request</code> , <code>workflow_dispatch</code> , etc.) |
| Job         | A set of steps that run on the same runner                                                       |
| Step        | Individual task: run a script or use an <b>action</b>                                            |
| Action      | Reusable unit (checkout, setup-node, deploy)                                                     |
| Runner      | Execution environment (GitHub-hosted or self-hosted)                                             |
| Artifact    | Files persisted between jobs or after workflow completion                                        |
| Environment | Named deployment target with optional protection rules                                           |



GitHub Actions integrates tightly with GitHub: PR checks, branch protection, environments, and OIDC federation for cloud deployments without long-lived credentials.

## 42.2 38.2 Workflow syntax fundamentals

Every workflow begins with a name, trigger, and jobs block:

```
name: CI Pipeline

on:
 push:
 branches: [main, release/*]
 pull_request:
 branches: [main]
 workflow_dispatch:
 inputs:
 environment:
 description: Target environment
 required: true
 type: choice
 options: [staging, production]

concurrency:
 group: ${ github.workflow }-${ github.ref }
 cancel-in-progress: true

permissions:
```

```

 contents: read
 id-token: write # Required for OIDC to AWS/GCP/Azure

jobs:
 test:
 runs-on: ubuntu-latest
 timeout-minutes: 15
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-go@v5
 with:
 go-version: "1.22"
 cache: true
 - run: go test ./... -race -coverprofile=coverage.out
 - uses: actions/upload-artifact@v4
 with:
 name: coverage
 path: coverage.out

```

Key syntax elements:

| Key                                          | Purpose                                               |
|----------------------------------------------|-------------------------------------------------------|
| <code>on</code>                              | Events and filters (paths, branches, tags)            |
| <code>jobs.&lt;id&gt;.needs</code>           | Job dependency graph                                  |
| <code>jobs.&lt;id&gt;.if</code>              | Conditional execution                                 |
| <code>jobs.&lt;id&gt;.strategy.matrix</code> | Parallel builds across OS/versions                    |
| <code>\${{ }}</code>                         | Expression syntax for contexts and functions          |
| <code>env</code>                             | Environment variables at workflow, job, or step level |

**Contexts** provide runtime data: `github`, `env`, `job`, `steps`, `secrets`, `vars`. Use `github.sha`, `github.ref_name`, and `github.event_name` for dynamic behavior.

## 42.3 38.3 Reusable workflows and composite actions

Avoid duplicating pipeline logic across repositories with **reusable workflows** and **composite actions**.

Reusable workflow (called from another repo's workflow):

```

.github/workflows/reusable-test.yml
on:
 workflow_call:
 inputs:
 node-version:
 required: true
 type: string
 secrets:
 NPM_TOKEN:
 required: true
 outputs:
 image-tag:
 description: Built image tag
 value: ${{ jobs.build.outputs.tag }}

 jobs:
 build:
 runs-on: ubuntu-latest
 outputs:
 tag: ${{ steps.meta.outputs.tag }}
 steps:
 - uses: actions/checkout@v4
 - id: meta
 run: echo "tag=${{ github.sha }}" >> $GITHUB_OUTPUT

```

Caller workflow:

```

jobs:
 call-reusable:
 uses: ../github/workflows/reusable-test.yml
 with:
 node-version: "20"
 secrets:
 NPM_TOKEN: ${ secrets.NPM_TOKEN }

```

Composite actions bundle multiple steps into one `uses:` reference—ideal for org-wide lint or deploy patterns stored in a dedicated `actions` repository.

## 42.4 38.4 Secrets, variables, and security

Secrets must never appear in logs. GitHub automatically masks values registered as secrets.

| Secret scope         | Visibility                       |
|----------------------|----------------------------------|
| Repository secrets   | Single repo workflows            |
| Environment secrets  | Jobs targeting named environment |
| Organization secrets | Selected repos in org            |
| Dependabot secrets   | Dependabot workflows only        |

```

jobs:
 deploy:
 runs-on: ubuntu-latest
 environment: production
 steps:
 - name: Deploy with API key
 env:
 DEPLOY_KEY: ${ secrets.DEPLOY_API_KEY }
 run: ./scripts/deploy.sh

```

### Security best practices:

1. **Least privilege** — Set `permissions:` explicitly; default was historically overly broad.
2. **Pin actions** — Use commit SHA or version tag ( `@v4` ), not `@main`.
3. **OIDC over static keys** — Federate to AWS IAM, Azure, GCP without storing cloud credentials in GitHub.
4. **Environment protection** — Require reviewers for production; limit branches that can deploy.
5. **Fork PR safety** — Workflows from forks do not receive secrets unless explicitly configured (and carefully).

OIDC example for AWS:

```

permissions:
 id-token: write
 contents: read

steps:
 - uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: arn:aws:iam::123456789012:role/GitHubActionsDeploy
 aws-region: us-east-1

```

Repository **variables** ( `vars` ) store non-sensitive configuration (region names, feature flags) visible to collaborators with appropriate access.

## 42.5 38.5 Runners: hosted vs self-hosted

**GitHub-hosted runners** are managed by GitHub (Ubuntu, Windows, macOS). They are ephemeral, pre-installed with common tools, and billed by minute (included minutes on free/team plans, then usage-based).

**Self-hosted runners** run on your infrastructure—on-premises VMs, Kubernetes, or cloud instances you control.

| Aspect          | GitHub-hosted    | Self-hosted                   |
|-----------------|------------------|-------------------------------|
| Maintenance     | GitHub           | You                           |
| Custom software | Limited image    | Full control                  |
| Network access  | Public internet  | Private VPC, internal APIs    |
| Cost model      | Per-minute       | Your infra + runner software  |
| Isolation       | Fresh VM per job | Shared or dedicated per label |
| Scale           | Automatic        | You add runners               |

Self-hosted runner registration:

```
On your runner machine
mkdir actions-runner && cd actions-runner
curl -o actions-runner-linux-x64-2.311.0.tar.gz -L \
 https://github.com/actions/runner/releases/download/v2.311.0/actions-runner-linux-x64-2.311.0.tar.gz
tar xzf actions-runner-linux-x64-2.311.0.tar.gz
./config.sh --url https://github.com/myorg/myrepo --token RUNNER_TOKEN
sudo ./svc.sh install
sudo ./svc.sh start
```

Use **runner labels** ( `runs-on: [self-hosted, linux, gpu]` ) to route jobs. For Kubernetes, projects like **Actions Runner Controller (ARC)** scale ephemeral runners as pods.

**Runner groups** (organization level) restrict which repos can use which runners—critical for security when runners access internal networks.

## 42.6 38.6 Matrix builds, caching, and artifacts

Matrix strategies run the same job across multiple configurations:

```
jobs:
 test-matrix:
 strategy:
 fail-fast: false
 matrix:
 os: [ubuntu-latest, macos-latest]
 python: ["3.11", "3.12"]
 runs-on: ${ matrix.os }
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-python@v5
 with:
 python-version: ${ matrix.python }
 - run: pip install -r requirements.txt && pytest
```

**Caching** speeds dependency installation:

```
- uses: actions/cache@v4
 with:
 path: ~/.npm
 key: npm-${ runner.os }-${ hashFiles('**/package-lock.json') }
 restore-keys: npm-${ runner.os }-
```

**Artifacts** pass build outputs between jobs or retain them for download:

```
- uses: actions/upload-artifact@v4
 with:
 name: dist
 path: dist/
 retention-days: 7
```

Use artifacts for test reports, built binaries, or Terraform plans—not for large Docker images (push to a registry instead).

## 42.7 38.7 Environments, deployments, and approvals

**Environments** model deployment targets (staging, production) with optional rules:

- Required reviewers before job runs
- Wait timer (e.g., 5 minutes before prod deploy)
- Branch restrictions (only `main` deploys to prod)
- Environment-specific secrets

```
jobs:
 deploy-prod:
 needs: test
 runs-on: ubuntu-latest
 environment:
 name: production
 url: https://app.example.com
 steps:
 - uses: actions/github-script@v7
 with:
 script: |
 github.rest.repos.createDeployment({
 owner: context.repo.owner,
 repo: context.repo.repo,
 ref: context.sha,
 environment: 'production',
 auto_merge: false,
 required_contexts: []
 })
 - run: ./deploy.sh production
```

The **deployments API** surfaces deployment history in the GitHub UI, linking commits to running releases.

## 42.8 38.8 Debugging and troubleshooting

| Problem                 | Diagnostic approach                                                            |
|-------------------------|--------------------------------------------------------------------------------|
| Workflow not triggering | Check <code>on:</code> filters, branch names, path filters                     |
| Secret empty in fork PR | Expected behavior; use <code>pull_request_target</code> only with extreme care |
| Action version break    | Pin SHA; read action changelog                                                 |
| Runner offline          | Check self-hosted runner service, token expiry                                 |
| Job queued forever      | Org concurrency limits; no available runners                                   |
| Expression errors       | Enable debug logging: <code>ACTIONS_STEP_DEBUG: true</code>                    |

Enable workflow run logs retention in repo settings. Use **workflow visualization** in the Actions tab to inspect job graphs.

## 42.9 38.9 Chapter summary

- GitHub Actions workflows are event-driven YAML pipelines composed of jobs and steps on runners.
- Secure pipelines with scoped permissions, pinned actions, OIDC, and environment protection rules.
- Choose hosted runners for simplicity; self-hosted for private network access and custom tooling.
- Reusable workflows, matrix builds, caching, and artifacts scale CI across teams and tech stacks.

---

## 42.10 Lab 38.1 — Multi-environment deploy pipeline

1. Create `.github/workflows/deploy.yml` with jobs: `lint`, `test`, `build`, `deploy-staging`, `deploy-production`.
  2. Configure a `staging` environment with no approval and `production` with required reviewer.
  3. Add OIDC or repository secrets for your target cloud.
  4. Use a matrix to test on Node 18 and 20.
  5. Upload test coverage as an artifact; verify download after run.
  6. Trigger via `workflow_dispatch` and confirm environment gates block production until approval.
- 

## 42.11 Lab 38.2 — Self-hosted runner in Docker

1. Register a self-hosted runner in a test organization (or use a disposable repo).
  2. Run the runner in a Docker container with label `docker-runner`.
  3. Create a workflow targeting `runs-on: [self-hosted, docker-runner]` that accesses an internal mock service on `host.docker.internal`.
  4. Document cleanup: remove runner registration and revoke token.
- 

## 42.12 Review questions

1. What is the difference between a **job**, a **step**, and an **action**?
  2. Why should you prefer OIDC over storing AWS access keys in GitHub Secrets?
  3. When would you choose a self-hosted runner over a GitHub-hosted runner?
  4. Explain how `concurrency` groups prevent duplicate deploys.
  5. What security risk does `pull_request_target` introduce, and when is it justified?
  6. How do environment protection rules support Continuous Delivery without Continuous Deployment?
  7. Describe two ways to share pipeline logic across multiple repositories.
- 

*Continue: Chapter 39 — Jenkins Pipelines as Code*

# 43 Chapter 39: Jenkins Pipelines as Code

DevOps Handbook — Part IX, Pages 771–790

## 43.1 39.1 Jenkins in the modern CI/CD landscape

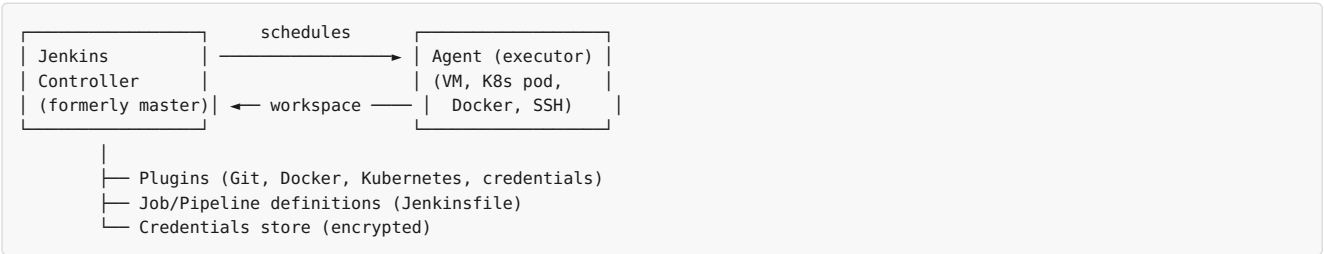
**Jenkins** is the longest-running open-source automation server in DevOps. While cloud-native platforms (GitHub Actions, GitLab CI) dominate greenfield projects, Jenkins remains entrenched in enterprises with thousands of plugins, existing investment, and air-gapped environments.

Modern Jenkins centers on **Pipeline as Code**: pipelines defined in `Jenkinsfile` files stored in Git, versioned alongside application code, and reviewed through pull requests—eliminating click-configured jobs that drift from documentation.

| Strength                   | Limitation                         |
|----------------------------|------------------------------------|
| 2,000+ plugins             | Plugin maintenance burden          |
| Self-hosted control        | You operate HA, upgrades, security |
| Flexible agents            | Complex agent provisioning         |
| Declarative & Scripted DSL | Groovy learning curve              |

This chapter focuses on **Declarative Pipeline** syntax—the recommended approach for readability and Jenkins validation.

## 43.2 39.2 Architecture: controller, agents, and plugins



- **Controller** — Orchestrates builds, serves UI, stores configuration. Never run heavy builds on the controller in production.
- **Agent** — Executes pipeline steps. Labels route jobs ( `agent { label 'linux && docker' }` ).
- **Plugins** — Extend functionality; pin versions and audit for CVEs.
- **Credentials** — Bound to folders or jobs; referenced by ID in pipelines.

For Kubernetes, the **Kubernetes plugin** creates ephemeral agent pods per build—isolating workloads and scaling horizontally.

## 43.3 39.3 Declarative Pipeline structure

A minimal `Jenkinsfile` in the repository root:

```
pipeline {
 agent any

 options {
```



```

 timestamps()
 timeout(time: 30, unit: 'MINUTES')
 buildDiscarder(logRotator(numToKeepStr: '20'))
 }

 environment {
 APP_NAME = 'order-service'
 REGISTRY = 'registry.example.com'
 }

 stages {
 stage('Checkout') {
 steps {
 checkout scm
 }
 }

 stage('Build') {
 steps {
 sh 'make build'
 }
 }

 stage('Test') {
 steps {
 sh 'make test'
 }
 post {
 always {
 junit '**/target/surefire-reports/*.xml'
 }
 }
 }

 stage('Docker Build & Push') {
 when {
 branch 'main'
 }
 steps {
 script {
 docker.withRegistry("https://${REGISTRY}", 'registry-credentials-id') {
 def image = docker.build("${APP_NAME}:${env.BUILD_NUMBER}")
 image.push('latest')
 image.push("${env.GIT_COMMIT.take(7)}")
 }
 }
 }
 }
 }

 post {
 success {
 slackSend channel: '#ci', message: "Build ${env.BUILD_URL} succeeded"
 }
 failure {
 slackSend channel: '#ci', message: "Build ${env.BUILD_URL} FAILED"
 }
 }
}

```

Declarative blocks:

| Block          | Role                                 |
|----------------|--------------------------------------|
| agent          | Where pipeline runs                  |
| environment    | Variables and credentials binding    |
| stages / stage | Logical pipeline phases              |
| steps          | Commands and plugin calls            |
| post           | Actions on success/failure/always    |
| when           | Conditional stage execution          |
| options        | Timeouts, concurrency, log retention |

---

## 43.4 39.4 Agents and parallelism

Agent directives control execution placement:

```
pipeline {
 agent none

 stages {
 stage('Parallel Tests') {
 parallel {
 stage('Unit') {
 agent { label 'linux' }
 steps { sh 'npm run test:unit' }
 }
 stage('Integration') {
 agent { label 'linux && docker' }
 steps { sh 'npm run test:integration' }
 }
 stage('Lint') {
 agent { label 'linux' }
 steps { sh 'npm run lint' }
 }
 }
 }
 }
}
```

**Docker agent** runs the entire pipeline inside a container:

```
pipeline {
 agent {
 docker {
 image 'node:20-alpine'
 args '-v /var/run/docker.sock:/var/run/docker.sock'
 }
 }
 stages {
 stage('Build') {
 steps { sh 'npm ci && npm run build' }
 }
 }
}
```

**Kubernetes agent** pod template (requires Kubernetes plugin):

```
pipeline {
 agent {
 kubernetes {
 yaml '''
apiVersion: v1
kind: Pod
spec:
 containers:
 - name: maven
 image: maven:3.9-eclipse-temurin-17
 command: ["sleep"]
 args: ["99d"]
 resources:
 limits:
 memory: "1Gi"
 cpu: "1000m"
 ...
 }
 }
 stages {
 stage('Maven Build') {
 steps {
 container('maven') {
 sh 'mvn -B clean verify'
 }
 }
 }
 }
}
```

```
}
```

## 43.5 39.5 Credentials, parameters, and shared libraries

Never hardcode secrets in Jenkinsfiles. Use the **Credentials Binding** plugin:

```
stage('Deploy') {
 steps {
 withCredentials([
 string(credentialsId: 'deploy-api-token', variable: 'TOKEN'),
 usernamePassword(credentialsId: 'git-deploy', usernameVariable: 'GIT_USER', passwordVariable: 'GIT_PASS')
]) {
 sh '''
 curl -H "Authorization: Bearer $TOKEN" \
 -X POST https://api.example.com/deploy
 '''
 }
 }
}
```

**Parameters** enable manual runs with choices:

```
parameters {
 choice(name: 'ENV', choices: ['staging', 'production'], description: 'Target')
 booleanParam(name: 'RUN_INTEGRATION', defaultValue: true)
}

stage('Deploy') {
 when {
 expression { params.ENV == 'production' }
 }
 steps {
 input message: 'Deploy to production?', ok: 'Deploy'
 sh "./deploy.sh ${params.ENV}"
 }
}
```

**Shared Libraries** centralize Groovy functions across pipelines. Define in **Manage Jenkins → Configure System → Global Pipeline Libraries**, then:

```
@Library('my-devops-lib@v2') _

pipeline {
 agent any
 stages {
 stage('Scan') {
 steps {
 devopsSecurity.scanImage('myapp:latest')
 }
 }
 }
}
```

Libraries live in a Git repo with `vars/` (global steps) and `src/` (classes).

## 43.6 39.6 Multibranch and organization folders

**Multibranch Pipeline** jobs automatically discover branches and PRs with a `Jenkinsfile`, creating child jobs per branch:

1. Connect GitHub/GitLab/Bitbucket via webhook or polling.
2. Jenkins scans repository for branches containing `Jenkinsfile`.
3. Each branch gets an isolated pipeline history.

**Organization Folder** scans an entire GitHub organization—ideal for microservice estates with one repo per service.

`Jenkinsfile` location can be customized ( `Script Path` ) for monorepos: `services/payments/Jenkinsfile` .

### 43.7 39.7 Pipeline durability and best practices

| Practice                                                                   | Rationale                              |
|----------------------------------------------------------------------------|----------------------------------------|
| Declarative over Scripted                                                  | Easier review, built-in validation     |
| Keep controller lightweight                                                | Agents run builds                      |
| Pin plugin versions                                                        | Reproducible builds, fewer surprises   |
| Use <code>options { disableConcurrentBuilds() }</code> on deploy pipelines | Prevent overlapping deploys            |
| Archive artifacts selectively                                              | Disk fills quickly on busy controllers |
| Blue Ocean or Stage View                                                   | Visualize parallel stages              |
| Backup <code>\$JEN_HOME</code>                                             | Jobs, credentials, config              |
| RBAC via Matrix Authorization                                              | Least privilege for Jenkins UI         |

**Scripted Pipeline** (full Groovy) remains for edge cases but should be wrapped in `script { }` blocks inside Declarative when needed.

### 43.8 39.8 Jenkins vs cloud-native CI (when to stay)

Stay on Jenkins when:

- Regulatory requirements mandate on-premises CI
- Heavy customization via niche plugins is required
- Existing pipeline investment exceeds migration cost

Migrate when:

- Plugin debt and controller HA consume disproportionate ops time
- Teams want CI tightly integrated with Git hosting
- Ephemeral cloud runners reduce agent maintenance

Hybrid patterns exist: Jenkins orchestrates legacy jobs while new services use GitHub Actions, unified by deployment tooling (Argo CD, Spinnaker).

### 43.9 39.9 Chapter summary

- Jenkins Pipeline as Code ( `Jenkinsfile` ) brings version control and review to CI/CD definitions.
- Declarative Pipeline provides structured stages, agents, credentials, and post-build actions.
- Agents on VMs, Docker, or Kubernetes execute work; the controller orchestrates only.
- Multibranch pipelines and shared libraries scale Jenkins across many services and teams.

### 43.10 Lab 39.1 — Declarative pipeline from scratch

1. Install Jenkins (Docker or local) with Git, Pipeline, Docker Pipeline, and JUnit plugins.
  2. Create a Multibranch Pipeline job connected to a sample repo.
  3. Add a `Jenkinsfile` with checkout, build, test, and Docker publish stages.
  4. Store registry credentials in Jenkins Credentials; reference via `withCredentials`.
  5. Add a `when { branch 'main' }` gate on publish; verify feature branches skip deploy.
  6. Configure Slack or email notification in `post { failure }`.
- 

## 43.11 Lab 39.2 — Kubernetes ephemeral agents

1. Deploy Jenkins with Kubernetes plugin on a test cluster.
  2. Define a pod template with `jnlp` and `maven` containers.
  3. Run a pipeline that builds a Java app inside the `maven` container.
  4. Confirm agent pod is deleted after build completes.
- 

## 43.12 Review questions

1. What problem does Pipeline as Code solve compared to UI-configured jobs?
  2. Why should production builds not run on the Jenkins controller?
  3. Explain the difference between Declarative and Scripted Pipeline.
  4. How do Multibranch Pipelines discover which branches to build?
  5. What is a Jenkins Shared Library, and when should you use one?
  6. Describe two ways to inject secrets safely into a pipeline step.
  7. What are the main operational costs of running Jenkins at enterprise scale?
- 

*Continue: Chapter 40 — GitLab CI and Multi-platform Comparison*

# 44 Chapter 40: GitLab CI and Multi-platform Comparison

---

DevOps Handbook — Part IX, Pages 791-810

---

## 44.1 40.1 GitLab CI/CD overview

**GitLab CI/CD** is built into GitLab—same platform for source control, merge requests, registry, and pipelines. Configuration lives in `.gitlab-ci.yml` at the repository root (or custom path). GitLab Runner executes jobs; GitLab Server (gitlab.com or self-managed) schedules pipelines.

Architecture:

```
.gitlab-ci.yml → GitLab CI/CD → Runner(s) → Job containers/shell
|
├─ Variables & CI/CD settings
├─ Environments & deployments
└─ Container Registry / Package Registry
```

GitLab emphasizes **DevOps platform** consolidation: one vendor for SCM, CI, security scanning (Ultimate tier), and deployment tracking.

---

## 44.2 40.2 `.gitlab-ci.yml` fundamentals

```
stages:
 - build
 - test
 - security
 - deploy

default:
 image: node:20-alpine
 cache:
 key: ${CI_COMMIT_REF_SLUG}
 paths:
 - node_modules/

variables:
 NODE_ENV: test

build:
 stage: build
 script:
 - npm ci
 - npm run build
 artifacts:
 paths:
 - dist/
 expire_in: 1 week

unit-test:
 stage: test
 script:
 - npm ci
 - npm run test:unit
 coverage: '/Lines\s*:\s*(\d+\.\d+)%/'

integration-test:
 stage: test
 services:
 - name: postgres:15
 alias: db
 variables:
```

```

 POSTGRES_DB: testdb
 POSTGRES_USER: test
 POSTGRES_PASSWORD: test
 DATABASE_URL: postgres://test:test@db:5432/testdb
script:
 - npm run test:integration

deploy-staging:
 stage: deploy
 environment:
 name: staging
 url: https://staging.example.com
 rules:
 - if: $CI_COMMIT_BRANCH == "main"
 script:
 - ./deploy.sh staging
 needs:
 - build
 - unit-test

```

Core concepts:

| Concept         | Description                                                  |
|-----------------|--------------------------------------------------------------|
| <b>Stage</b>    | Ordered phase; jobs in same stage run in parallel            |
| <b>Job</b>      | Unit of work with <code>script</code>                        |
| <b>Runner</b>   | Executes jobs (shared, group, project-specific)              |
| <b>Artifact</b> | Files passed between jobs                                    |
| <b>Cache</b>    | Speed up dependency installs across pipelines                |
| <b>Service</b>  | Sidecar containers (DB, Redis) for tests                     |
| <b>Rules</b>    | Modern conditional syntax replacing <code>only/except</code> |

## 44.3 40.3 Runners, executors, and isolation

**GitLab Runner** is a separate agent process registering to GitLab.

| Executor          | Use case                           |
|-------------------|------------------------------------|
| <b>Docker</b>     | Most common; job runs in container |
| <b>Kubernetes</b> | Ephemeral pods per job             |
| <b>Shell</b>      | Direct on VM; least isolation      |
| <b>SSH</b>        | Remote machine execution           |
| <b>Instance</b>   | Auto-scaled cloud VMs (Premium+)   |

Register a runner:

```

gitlab-runner register \
 --url https://gitlab.com/ \
 --registration-token PROJECT_TOKEN \
 --executor docker \
 --docker-image alpine:latest \
 --description "docker-runner-01" \
 --tag-list "docker,linux"

```

Use **tags** in jobs: `tags: [docker, linux]`. **Protected runners** run only on protected branches—critical for production deploy jobs accessing sensitive credentials.

# 44.4 40.4 Variables, environments, and child pipelines

CI/CD **variables** scope: project, group, environment. Mark sensitive values as **masked** and **protected**.

```
deploy-production:
 stage: deploy
 environment:
 name: production
 url: https://app.example.com
 deployment_tier: production
 rules:
 - if: $CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/
 script:
 - echo "Deploying $CI_COMMIT_TAG"
 variables:
 KUBE_CONTEXT: prod-cluster
```

**Manual jobs** ( `when: manual` ) implement approval gates. **Resource groups** serialize deploys to the same environment.

**Child pipelines** split monorepo CI:

```
trigger-backend:
 trigger:
 include: backend/.gitlab-ci.yml
 strategy: depend
 rules:
 - changes:
 - backend/**/*
```

**Merge request pipelines** run on MR events; **merged results pipelines** test the post-merge state—reducing “green MR, red main” surprises.

# 44.5 40.5 GitLab-specific features

| Feature              | Benefit                                            |
|----------------------|----------------------------------------------------|
| Auto DevOps          | Opinionated default pipeline for build/test/deploy |
| Review Apps          | Dynamic environments per MR                        |
| Container Registry   | Built-in image storage tied to project             |
| Dependency Proxy     | Cache upstream Docker images                       |
| Compliance pipelines | Enforce stages across org (Ultimate)               |
| DAST/SAST templates  | Security jobs from template catalog                |

Review App example:

```
review:
 stage: deploy
 environment:
 name: review/$CI_COMMIT_REF_SLUG
 url: https://$CI_ENVIRONMENT_SLUG.example.com
 on_stop: stop_review
 script:
 - deploy-review.sh
 rules:
 - if: $CI_MERGE_REQUEST_IID

stop_review:
 stage: deploy
 environment:
 name: review/$CI_COMMIT_REF_SLUG
 action: stop
 when: manual
```



```
script:
 - teardown-review.sh
```

## 44.6 40.6 Multi-platform CI/CD comparison

| Dimension         | GitHub Actions                       | GitLab CI                       | Jenkins                  |
|-------------------|--------------------------------------|---------------------------------|--------------------------|
| Config file       | <code>.github/workflows/*.yml</code> | <code>.gitlab-ci.yml</code>     | <code>Jenkinsfile</code> |
| Hosting           | GitHub cloud/Enterprise              | GitLab.com/self-managed         | Self-hosted (typical)    |
| Runners           | Hosted + self-hosted                 | Runner required (shared or own) | Agents                   |
| Marketplace       | Actions marketplace                  | CI templates, components        | Plugins                  |
| MR/PR integration | Native checks                        | Native pipelines                | Plugin-dependent         |
| OIDC to cloud     | Excellent                            | Supported                       | Plugin-dependent         |
| Learning curve    | Moderate YAML                        | Moderate YAML                   | Groovy + ops             |
| Cost              | Minutes-based                        | Per-user + compute              | Infra + maintenance      |

**Azure Pipelines** (YAML, Microsoft-hosted agents, Azure DevOps integration) fits Microsoft-centric orgs. **CircleCI**, **Buildkite**, and **Tekton** (Kubernetes-native) appear in cloud-native and high-scale scenarios.

Decision framework:

1. **Where is code hosted?** Align CI with SCM when possible.
2. **Compliance** — Data residency, air-gap, audit requirements.
3. **Runner strategy** — Shared vs dedicated, spot instances, K8s.
4. **Total cost** — License + compute + engineer maintenance.
5. **Ecosystem** — Required integrations (Jira, SonarQube, Vault).

## 44.7 40.7 Portable pipeline patterns

Regardless of platform, mature pipelines share structure:

```
lint → unit test → build artifact → security scan → integration test → deploy staging → deploy prod
```

Portable techniques:

- **Containerize builds** — Same Dockerfile locally and in CI
- **Make/task runners** — `make test` invoked identically everywhere
- **Composite/reusable units** — GitHub reusable workflows, GitLab `include:`, Jenkins shared libraries
- **Policy gates** — Fail on critical CVEs, coverage thresholds
- **Immutable artifacts** — Promote the same image digest staging → prod

Example GitLab `include` for shared org template:

```
include:
 - project: 'platform/ci-templates'
 ref: main
 file: '/nodejs/build-test-deploy.yml'
```

## 44.8 40.8 Migration considerations

Migrating Jenkins → GitLab or GitHub:

1. Inventory jobs, plugins, credentials, and agent labels.
2. Rewrite pipelines incrementally (strangler pattern)—one service at a time.
3. Map credentials to new secret stores; rotate during migration.
4. Parallel-run old and new pipelines until confidence is high.
5. Decommission Jenkins agents only after traffic cutover.

Tooling like **GitHub Actions Importer** and community converters assist but require manual validation for complex Groovy logic.

---

## 44.9 40.9 Chapter summary

- GitLab CI/CD uses `.gitlab-ci.yml` with stages, jobs, runners, artifacts, and environments in one platform.
  - Runners with Docker or Kubernetes executors provide flexible, isolated build environments.
  - Compare CI platforms on hosting, runner model, integration depth, security, and operational cost—not syntax alone.
  - Portable pipeline design (containers, shared templates, immutable artifacts) reduces vendor lock-in.
- 

## 44.10 Lab 40.1 — GitLab pipeline with services

1. Create a GitLab project (gitlab.com free tier or self-managed).
  2. Add `.gitlab-ci.yml` with Postgres `services` for integration tests.
  3. Configure CI/CD variables for a mock deploy token (masked).
  4. Add `deploy-staging` with `environment` and `rules` for `main` only.
  5. Create an MR and verify MR pipeline runs; merge and confirm staging deploy.
- 

## 44.11 Lab 40.2 — Platform comparison exercise

1. Implement the same three-stage pipeline (lint, test, docker build) in GitHub Actions and GitLab CI for one sample repo.
  2. Document: lines of config, runner setup time, secret handling, and MR/PR integration UX.
  3. Present a one-page recommendation for a hypothetical 50-developer org.
- 

## 44.12 Review questions

1. What is the role of GitLab Runner, and how does it differ from GitLab Server?
  2. Explain `rules` vs deprecated `only/except` in GitLab CI.
  3. How do **Review Apps** improve merge request workflows?
  4. Name three criteria for choosing between GitHub Actions and GitLab CI.
  5. What is a child pipeline, and when is it useful in monorepos?
  6. Why should production deploy runners be **protected**?
  7. What pipeline patterns remain portable across CI platforms?
-

# 45 Chapter 41: Deployment Strategies — Blue/Green, Canary, Feature Flags

DevOps Handbook — Part IX, Pages 811–825

## 45.1 41.1 Why deployment strategy matters

Releasing software is not binary—“deploy or don’t deploy.” How you roll out changes determines **downtime**, **blast radius**, **rollback speed**, and **confidence**. A pipeline that builds and tests flawlessly still causes outages if production traffic shifts incorrectly.

Deployment strategies sit at the intersection of **CI/CD**, **infrastructure**, and **application design**. This chapter covers rolling, recreate, blue/green, canary, and feature flags—with Kubernetes and load balancer examples.

| Strategy      | Downtime  | Rollback speed       | Infra cost        | Complexity  |
|---------------|-----------|----------------------|-------------------|-------------|
| Recreate      | Yes       | Slow (redploy)       | Low               | Low         |
| Rolling       | Minimal   | Moderate             | Low               | Low         |
| Blue/green    | Near zero | Fast (switch)        | 2× during cutover | Medium      |
| Canary        | None      | Fast (traffic shift) | Partial duplicate | High        |
| Feature flags | None      | Instant (toggle)     | Low               | App changes |

## 45.2 41.2 Rolling deployment

**Rolling** replaces instances incrementally: old and new versions run simultaneously until all instances update.

Kubernetes default Deployment strategy:

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: api
spec:
 replicas: 5
 strategy:
 type: RollingUpdate
 rollingUpdate:
 maxSurge: 1
 maxUnavailable: 0
 template:
 spec:
 containers:
 - name: api
 image: registry.example.com/api:v2.3.0
 readinessProbe:
 httpGet:
 path: /health
 port: 8080
```

| Parameter      | Effect                                        |
|----------------|-----------------------------------------------|
| maxSurge       | Extra pods above desired count during rollout |
| maxUnavailable | Pods that can be down during rollout          |

**Pros:** No duplicate full environment; built into K8s. **Cons:** Mixed versions during rollout—API compatibility

required; slow rollback if new image already propagated.

---

## 45.3 41.3 Recreate deployment

**Recreate** terminates all old instances before starting new ones—simple but causes downtime.

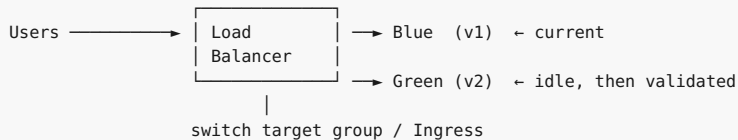
```
strategy:
 type: Recreate
```

Use for: stateful batch jobs, maintenance windows, or when only one version can run (license constraints). Avoid for user-facing services with SLA requirements.

---

## 45.4 41.4 Blue/green deployment

**Blue/green** maintains two identical environments—**blue** (current production) and **green** (new version). After validating green, traffic switches atomically.



AWS ALB example (conceptual):

1. Green ASG or target group runs v2 behind same ALB with zero weight.
2. Run smoke tests against green via internal DNS or test header.
3. Swap listener rules: green 100%, blue 0%.
4. Keep blue warm for quick rollback.

Kubernetes with two Deployments and Service selector swap—or use **Argo Rollouts** blue/green:

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
 name: api-rollout
spec:
 replicas: 5
 strategy:
 blueGreen:
 activeService: api-active
 previewService: api-preview
 autoPromotionEnabled: false
 selector:
 matchLabels:
 app: api
 template:
 metadata:
 labels:
 app: api
 spec:
 containers:
 - name: api
 image: registry.example.com/api:v2.3.0
```

**Pros:** Instant rollback (switch back to blue); clear validation window on preview URL. **Cons:** Double resource cost during transition; database schema migrations need careful handling (expand/contract pattern).

---

## 45.5 41.5 Canary deployment

**Canary** routes a small percentage of traffic to the new version, monitors metrics, then progressively increases traffic.

```
Traffic: 95% → stable (v1)
 5% → canary (v2) → 50/50 → 100% v2
```

Istio VirtualService example:

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
 name: api
spec:
 hosts:
 - api.example.com
 http:
 - route:
 - destination:
 host: api-stable
 weight: 95
 - destination:
 host: api-canary
 weight: 5
```

**Flagger** (Kubernetes) automates canary analysis using Prometheus metrics:

```
apiVersion: flagger.app/v1beta1
kind: Canary
metadata:
 name: api
spec:
 targetRef:
 apiVersion: apps/v1
 kind: Deployment
 name: api
 service:
 port: 8080
 analysis:
 interval: 1m
 threshold: 5
 maxWeight: 50
 stepWeight: 10
 metrics:
 - name: request-success-rate
 thresholdRange:
 min: 99
 interval: 1m
 - name: request-duration
 thresholdRange:
 max: 500
 interval: 1m
```

Canary success criteria typically include: error rate, latency p99, business KPIs (conversion rate). Automated rollback triggers when thresholds breach.

**Pros:** Smallest blast radius; data-driven promotion. **Cons:** Requires observability, traffic management, and often service mesh or advanced ingress.

---

## 45.6 41.6 Feature flags (dark launches)

**Feature flags** decouple **deployment** (code in production) from **release** (feature visible to users). Code ships disabled; operators enable for internal testers, then percentage of users, then everyone.

```
from flagsmith import Flagsmith

flagsmith = Flagsmith(environment_key=os.environ["FLAGSMITH_KEY"])

def checkout_handler(request):
```

```

flags = flagsmith.get_identity_flags(str(request.user.id))
if flags.is_feature_enabled("new_checkout_flow"):
 return new_checkout(request)
return legacy_checkout(request)

```

Flag management platforms: **LaunchDarkly**, **Flagsmith**, **Unleash**, **Split.io**, or open-source **GO Feature Flag**.

| Flag type              | Use case                              |
|------------------------|---------------------------------------|
| <b>Release</b>         | Gradual rollout of new UI/API         |
| <b>Ops/kill switch</b> | Disable expensive endpoint under load |
| <b>Experiment</b>      | A/B test variants                     |
| <b>Permission</b>      | Entitlements, beta access             |

Best practices:

- Flags are **temporary**—remove after full rollout (flag debt causes untestable combinatorial states).
- Default to **off** in production; explicit enablement.
- Audit flag changes; tie to incident runbooks for kill switches.
- Test both flag states in CI where feasible.

Feature flags complement canary: canary validates infrastructure and baseline metrics; flags control user-visible behavior within the same binary.

## 45.7 41.7 Database migrations and deployment coupling

Schema changes break naive blue/green:

| Phase               | Pattern                                   |
|---------------------|-------------------------------------------|
| <b>Expand</b>       | Add new column/table; old code ignores it |
| <b>Migrate data</b> | Backfill asynchronously                   |
| <b>Contract</b>     | Remove old column after all code uses new |

Never deploy code that requires a schema change **before** the schema is backward-compatible. Tools: **Flyway**, **Liquibase**, **Alembic**—run migrations as pipeline stage with rollback plans.

## 45.8 41.8 Choosing a strategy

Decision tree:

1. **Zero downtime required?** → Eliminate recreate.
2. **Strong observability + traffic control?** → Canary.
3. **Need instant rollback without redeploy?** → Blue/green or feature flags.
4. **Simple stateless app, K8s?** → Rolling often sufficient.
5. **Hide incomplete features on main?** → Feature flags + trunk-based development.

Many organizations combine: **rolling** for frequent small patches, **canary** for high-risk services, **feature flags** for product experimentation.

## 45.9 41.9 Chapter summary

- Deployment strategy determines outage risk and rollback speed—not just pipeline success.

- Rolling and recreate suit simpler cases; blue/green enables instant traffic reversal.
  - Canary reduces blast radius when metrics-driven automation validates new versions.
  - Feature flags separate code deployment from feature release; manage flag lifecycle diligently.
  - Coordinate database migrations with expand/contract to support parallel versions.
- 

## 45.10 Lab 41.1 — Kubernetes rolling vs recreate

1. Deploy a sample app with 3 replicas and `RollingUpdate (maxUnavailable: 0)`.
  2. Update image tag; watch `kubectl rollout status` and observe zero downtime with readiness probes.
  3. Change strategy to `Recreate`; repeat update and measure downtime with `curl` in a loop.
- 

## 45.11 Lab 41.2 — Feature flag rollout

1. Deploy open-source Unleash or Flagsmith locally.
  2. Wrap a sample endpoint with a feature flag defaulting to off.
  3. Enable for 10% of user IDs; verify split behavior.
  4. Document flag removal checklist for post-rollout cleanup.
- 

## 45.12 Review questions

1. Compare rolling and blue/green in terms of rollback speed and resource cost.
  2. What metrics would you use to automate canary promotion?
  3. Why must feature flags be removed after full rollout?
  4. Explain the expand/contract pattern for database migrations.
  5. When is recreate deployment acceptable?
  6. How do feature flags differ from canary deployments?
  7. What role does a service mesh play in canary releases?
- 

*Continue: Chapter 42 — DORA Metrics and Continuous Improvement*

# 46 Chapter 42: DORA Metrics and Continuous Improvement

DevOps Handbook — Part IX, Pages 826–840

## 46.1 42.1 What DORA measures

The **DevOps Research and Assessment (DORA)** program, led by Nicole Forsgren, Jez Humble, and Gene Kim, identified four metrics that distinguish high-performing software delivery organizations from low performers. Published in *Accelerate* (2018) and updated in annual **State of DevOps** reports, these metrics provide a **evidence-based** framework—not vanity metrics like lines of code or story points.

The four **DORA metrics**:

| Metric                                | Definition                           | Elite benchmark (indicative) |
|---------------------------------------|--------------------------------------|------------------------------|
| <b>Deployment frequency</b>           | How often code deploys to production | Multiple per day             |
| <b>Lead time for changes</b>          | Commit to production duration        | Less than one hour           |
| <b>Change failure rate</b>            | % of changes causing failure in prod | 0–15%                        |
| <b>Time to restore service (MTTR)</b> | Recovery time after incident         | Less than one hour           |

A fifth metric—**reliability** (availability, latency SLOs)—was added in later research as a companion to delivery speed: speed without stability is unsustainable.

## 46.2 42.2 Why these four metrics matter

Traditional KPIs often optimize local maxima: development velocity without operational stability, or uptime achieved by never deploying. DORA metrics capture **both** speed and stability—the CALMS balance from Part I.

High performers demonstrate:

- Small, frequent changes (lower risk per change)
- Fast feedback when something breaks
- Culture of learning, not blame

Low performers often show:

- Infrequent, large releases (“release trains” without automation)
- Long manual testing phases
- Heroics during outages instead of systematic recovery

**Important:** Metrics drive behavior. Gaming DORA (deploying trivial config changes to inflate frequency) destroys trust. Use metrics for **improvement**, not individual performance review.

## 46.3 42.3 Measuring deployment frequency



**Deployment frequency** counts production deployments over time—typically per day or per week per service or team.

What counts as a deployment:

| Counts                           | Does not count                                             |
|----------------------------------|------------------------------------------------------------|
| New app version to prod          | Deploy to staging only                                     |
| Config/IaC change affecting prod | Failed deploy that never served traffic                    |
| Hotfix rollout                   | Restart without version change (debatable—document policy) |

Instrumentation:

```
GitHub Actions – emit deployment event
- name: Record deployment
 if: success()
 run: |
 curl -X POST "${METRICS_API}/deployments" \
 -H "Authorization: Bearer ${ secrets.METRICS_TOKEN }" \
 -d '{
 "service": "order-api",
 "environment": "production",
 "commit_sha": "${ github.sha }",
 "timestamp": "$(date -u +%Y-%m-%dT%H:%M:%SZ)"
 }'
```

Sources: CI/CD deployment logs, GitHub Deployments API, GitLab environments, Argo CD sync events, Spinnaker pipeline history.

Aggregate by **team** or **service**—not entire company—so teams see actionable data.

## 46.4 42.4 Measuring lead time for changes

**Lead time for changes** is the elapsed time from **code committed** to **running successfully in production**.

```
Lead Time = prod_deploy_timestamp - first_commit_timestamp
```

For a feature spanning multiple commits, use the merge commit to `main` or the tag triggering release—**define consistently**.

Break down lead time into segments (value stream map):

| Segment             | Typical bottlenecks              |
|---------------------|----------------------------------|
| Code review wait    | Large PRs, reviewer availability |
| CI queue + duration | Slow tests, insufficient runners |
| Approval gates      | Manual change advisory boards    |
| Deploy execution    | Manual steps, change windows     |

Example calculation from Git + CI:

```
Pseudocode: compute lead time for a deployment
deploy = get_production_deployment(service="api", sha="abc123")
commit_time = git.commit(deploy.sha).committed_at
lead_time_hours = (deploy.finished_at - commit_time).total_seconds() / 3600
record_metric("lead_time_hours", lead_time_hours, labels={"service": "api"})
```

Reducing lead time usually means: trunk-based development, automated tests, parallel CI, automated promotion, and eliminating manual handoffs.

---

## 46.5 42.5 Change failure rate

**Change failure rate** = (deployments causing prod failure) / (total deployments) in a period.

Define **failure** explicitly:

- Rollback required
- Hotfix within 24 hours of deploy
- SEV-1/SEV-2 incident linked to a change
- SLO breach attributed to deployment (via incident tagging)

```
-- Example: weekly change failure rate per service
SELECT
 service,
 COUNT(*) FILTER (WHERE caused_incident) * 100.0 / COUNT(*) AS failure_rate_pct
FROM deployments
WHERE environment = 'production'
AND deployed_at >= NOW() - INTERVAL '7 days'
GROUP BY service;
```

Link deployments to incidents in your incident tool (PagerDuty, Jira, incident.io) via **deployment ID** or **commit SHA** in incident metadata.

Improve change failure rate with: automated testing, canary analysis, feature flags, and post-deploy smoke tests —not by deploying less frequently (that hides problems in bigger batches).

---

## 46.6 42.6 Time to restore service (MTTR)

**MTTR** (Mean Time to Restore) measures how quickly service returns to normal after an incident—distinct from **MTTD** (detect) and **MTTR** repair vs **MTBF** (between failures).

```
MTTR = incident_resolved_at - incident_started_at
```

Use **customer-impacting** start time (SLO breach or alert), not internal discovery time, for user-centric measurement.

Improve MTTR with:

- Runbooks and automated remediation
- Observability (Chapter 43–47)
- Blameless postmortems with action items
- Chaos engineering validating recovery paths
- On-call rotation and escalation policies

Track MTTR separately for **deployment-related** vs **infrastructure** incidents to target improvements.

---

## 46.7 42.7 Performance clusters and benchmarking

DORA research clusters organizations into **elite**, **high**, **medium**, **low** performers based on metric combinations—not single metrics in isolation.

| Cluster | Typical profile                                              |
|---------|--------------------------------------------------------------|
| Elite   | On-demand deploys, sub-hour lead time, low CFR, fast restore |
| High    | Daily-weekly deploys, day-scale lead time                    |
| Medium  | Monthly deploys, weeks lead time                             |

|     |                                      |
|-----|--------------------------------------|
| Low | Quarterly+ deploys, months lead time |
|-----|--------------------------------------|

Compare teams to **past self**, not blindly to FAANG elite—context (regulated industry, legacy monolith) matters. Use benchmarks to set **directional goals**.

## 46.8 42.8 Building a metrics program

Implementation roadmap:

1. **Define** — Document what counts as deploy, failure, production for each service.
2. **Instrument** — CI/CD hooks, deployment APIs, incident linkage.
3. **Visualize** — Dashboards (Grafana, Datadog, custom) per team.
4. **Review** — Weekly team reviews; monthly cross-team patterns.
5. **Improve** — Pick one bottleneck from lead time breakdown per quarter.
6. **Protect culture** — No individual DORA targets; blameless analysis.

Sample dashboard panels:

- Deploy frequency (7-day rolling)
- Lead time p50 / p95
- Change failure rate (30-day)
- MTTR (30-day)
- Deployments vs incidents overlay chart

## 46.9 42.9 Continuous improvement loops

DORA metrics feed **Plan-Do-Check-Act (PDCA)** and **Three Ways** (Part I):

| Three Way          | DORA connection                               |
|--------------------|-----------------------------------------------|
| Flow               | Lead time, deployment frequency               |
| Feedback           | Change failure rate, MTTR                     |
| Continual learning | Postmortems, experiment with pipeline changes |

Combine with **SPACE** framework (Developer productivity—Satisfaction, Performance, Activity, Communication, Efficiency) to avoid over-optimizing deploy count at developer experience expense.

Space for improvement without DORA data: value stream mapping workshops, identifying wait states, automating top three manual steps.

## 46.10 42.10 Chapter summary

- DORA’s four metrics—deployment frequency, lead time, change failure rate, MTTR—measure delivery speed and stability together.
- Define measurement policies consistently; instrument CI/CD and incident systems.
- Improve metrics through systemic changes (automation, testing, observability)—not by gaming numbers.
- Benchmark against past performance; use metrics in blameless continuous improvement cycles.

## 46.11 Lab 42.1 — Deployment event pipeline

1. Add a post-deploy step to your CI pipeline that POSTs deployment metadata to a spreadsheet, database, or metrics endpoint.
  2. Tag each deployment with service name, environment, commit SHA, and duration.
  3. Build a simple dashboard (Grafana or spreadsheet chart) showing deployment frequency over 2 weeks of test data.
- 

## 46.12 Lab 42.2 — Lead time value stream map

1. Pick one recent production release.
  2. Record timestamps: first commit, PR opened, PR merged, CI complete, staging deploy, prod deploy.
  3. Calculate segment durations; identify the largest wait state.
  4. Propose one automation or process change targeting that bottleneck.
- 

## 46.13 Review questions

1. Name the four core DORA metrics and what each measures.
  2. Why should DORA metrics not be used for individual performance evaluation?
  3. How do you define whether a deployment “failed” for change failure rate?
  4. What is the difference between lead time for changes and CI pipeline duration?
  5. How does MTTR relate to observability and incident response practices?
  6. What lead time segments would you investigate first in a team with 5-day lead time?
  7. How do elite performers differ from low performers in deployment frequency patterns?
- 

*Continue: Chapter 43 — Monitoring Fundamentals*

# 47 Chapter 43: Monitoring Fundamentals — Metrics, Logs, Traces

DevOps Handbook — Part X, Pages 841–865

## 47.1 43.1 From monitoring to observability

**Monitoring** asks: “Is the system up? Are known thresholds breached?” Traditional monitoring relies on predefined dashboards and alerts—CPU > 80%, error rate > 1%.

**Observability** asks: “Why is the system behaving this way?” A system is observable when you can infer internal state from **external outputs**—especially for novel failures nobody anticipated.

The three pillars of observability:

| Pillar  | Question it answers                    | Typical tools                   |
|---------|----------------------------------------|---------------------------------|
| Metrics | How much? How fast? Trend over time?   | Prometheus, Datadog, CloudWatch |
| Logs    | What happened? Context and narrative?  | ELK, Loki, CloudWatch Logs      |
| Traces  | Where did time go? Cross-service path? | Jaeger, Tempo, Zipkin, X-Ray    |

Metrics are cheap at scale; logs are verbose; traces connect distributed requests. Mature platforms **correlate** all three—click from alert → logs → trace span.

## 47.2 43.2 Metrics: types and best practices

**Metrics** are numeric measurements aggregated over time.

| Type      | Description              | Example                                                     |
|-----------|--------------------------|-------------------------------------------------------------|
| Counter   | Monotonically increasing | <code>http_requests_total</code>                            |
| Gauge     | Value that goes up/down  | <code>memory_usage_bytes</code> , queue depth               |
| Histogram | Distribution in buckets  | Request latency buckets                                     |
| Summary   | Quantiles (client-side)  | <code>http_request_duration_seconds{quantile="0.99"}</code> |

Prometheus exposition format:

```
HELP http_requests_total Total HTTP requests
TYPE http_requests_total counter
http_requests_total{method="GET",status="200",handler="/api/orders"} 15432
http_requests_total{method="POST",status="500",handler="/api/orders"} 12
```

**RED method** for services:

- **Rate** — Requests per second
- **Errors** — Failed requests per second
- **Duration** — Latency distribution

**USE method** for resources:

- **Utilization** — % busy
- **Saturation** — Queue length / wait
- **Errors** — Error count

Naming conventions: include unit suffix ( `_seconds` , `_bytes` , `_total` for counters). Avoid high-cardinality labels (user IDs, unbounded paths)—they explode storage and query cost.

```
from prometheus_client import Counter, Histogram, start_http_server

REQUEST_COUNT = Counter(
 "http_requests_total", "Total requests",
 ["method", "endpoint", "status"]
)
REQUEST_LATENCY = Histogram(
 "http_request_duration_seconds", "Latency",
 ["method", "endpoint"],
 buckets=[0.01, 0.05, 0.1, 0.5, 1.0, 2.5, 5.0]
)

def handle_request(method, endpoint, handler):
 with REQUEST_LATENCY.labels(method, endpoint).time():
 status = handler()
 REQUEST_COUNT.labels(method, endpoint, str(status)).inc()
```

## 47.3 43.3 Logs: structure and aggregation

**Logs** are discrete events with timestamps—historically unstructured text, now increasingly **structured** (JSON).

Unstructured (hard to query):

```
2026-09-01T14:32:01Z ERROR payment failed user=12345 order=987 reason=timeout
```

Structured (machine-parseable):

```
{
 "timestamp": "2026-09-01T14:32:01.123Z",
 "level": "ERROR",
 "service": "payment-api",
 "trace_id": "4bf92f3577b329da7c93630a8a323129",
 "span_id": "00f067aa0ba902b7",
 "message": "payment failed",
 "user_id": "12345",
 "order_id": "987",
 "error": "gateway timeout"
}
```

Best practices:

| Practice                 | Rationale                                        |
|--------------------------|--------------------------------------------------|
| Structured JSON          | Filter/aggregate in log platforms                |
| Consistent field names   | Cross-service queries                            |
| Include trace/span IDs   | Jump to distributed trace                        |
| Log levels (DEBUG–ERROR) | Control volume in prod                           |
| Never log secrets/PII    | Compliance and security                          |
| Centralize               | SSHing to grep individual servers does not scale |

Log levels guide:

- **DEBUG** — Development troubleshooting (usually off in prod)
- **INFO** — Normal lifecycle events (startup, deploy, key business events)
- **WARN** — Recoverable anomalies
- **ERROR** — Failures requiring attention

## 47.4 43.4 Traces: distributed request flow

In microservices, a single user request traverses many services. **Distributed tracing** assigns a **trace ID** to the request; each service creates **spans** (timed operations) linked by parent-child relationships.

```
Trace ID: abc123
├─ Span: API Gateway (50ms)
│ └─ Span: Auth Service (10ms)
│ └─ Span: Order Service (35ms)
│ └─ Span: Postgres query (5ms)
│ └─ Span: Inventory RPC (20ms)
```

OpenTelemetry span attributes:

```
from opentelemetry import trace

tracer = trace.get_tracer(__name__)

with tracer.start_as_current_span("process_order") as span:
 span.set_attribute("order.id", order_id)
 span.set_attribute("customer.tier", "premium")
 result = charge_payment(order_id)
 span.add_event("payment_completed", {"amount": result.amount})
```

Traces answer: Which downstream call caused latency? Did error originate in service A or B? What was the critical path?

Sample **tail-based sampling**: keep all error traces, sample 1% of success traces—balancing cost and debuggability.

## 47.5 43.5 The observability data pipeline



**OpenTelemetry (OTel)** is the vendor-neutral standard for instrumenting metrics, logs, and traces with one SDK and exporting to many backends.

Design principles:

- 1. **Instrument at boundaries** — HTTP handlers, RPC, DB calls, queue publish/consume.
- 2. **SLO-driven** — Alert on user-impacting symptoms, not every metric (Chapter 47).
- 3. **Cardinality control** — Label discipline prevents metric explosion.
- 4. **Retention tiers** — Hot (days), warm (weeks), cold (compliance archive).

## 47.6 43.6 Alerting philosophy

Alerts should be **actionable**, **urgent**, and **symptom-based**.

| Bad alert            | Good alert                           |
|----------------------|--------------------------------------|
| CPU > 70% on one pod | Error budget burn rate > 2x for 15m  |
| Disk 60% full        | Disk will fill in 4h at current rate |
| Any ERROR log        | Payment success rate < 99.5% for 5m  |

---

Alert routing:

- **Page** (wake someone) — Customer-impacting SLO breach
- **Ticket** — Degraded but not urgent
- **Dashboard** — Informational trend

Reduce **alert fatigue**—on-call engineers ignoring alerts is worse than no alerts.

---

## 47.7 43.7 Golden signals and SLIs

Google SRE **four golden signals**:

1. **Latency** — Time to serve request ( distinguish success vs error latency)
2. **Traffic** — Demand on system
3. **Errors** — Rate of failed requests
4. **Saturation** — How “full” the system is

Map golden signals to **Service Level Indicators (SLIs)**—quantifiable measure of service aspect (availability, latency, throughput). Chapter 47 expands SLOs and error budgets.

Example SLI:

```
Availability SLI = successful_requests / total_valid_requests
Latency SLI = proportion of requests < 300ms
```

---

## 47.8 43.8 Instrumentation ownership

| Role             | Responsibility                                   |
|------------------|--------------------------------------------------|
| Application team | Business metrics, RED metrics, trace propagation |
| Platform team    | Collectors, TSDB, dashboards, alert routing      |
| SRE              | SLO definition, on-call, incident process        |

“You build it, you run it” includes **you instrument it**. Platform provides plumbing; teams own service-level observability.

---

## 47.9 43.9 Chapter summary

- Observability combines metrics (aggregates), logs (events), and traces (request paths) to explain system behavior.
  - Use RED/USE methods, structured logging, and trace propagation with correlation IDs.
  - Control cardinality, retention, and alert quality to keep systems operable at scale.
  - OpenTelemetry unifies instrumentation; backend choice can evolve without re-instrumenting apps.
- 

## 47.10 Lab 43.1 — RED metrics for a sample API

1. Add Prometheus client to a sample HTTP service exposing `/metrics`.
2. Instrument request count, error count, and latency histogram per route.
3. Scrape with Prometheus; graph rate and p99 latency in Grafana.



4. Generate load with `hey` or `k6`; observe metrics change.
- 

## 47.11 Lab 43.2 — Correlate logs and traces

1. Instrument a two-service demo with OpenTelemetry.
  2. Emit JSON logs including `trace_id`.
  3. Trigger a request; find log line in Loki/Elasticsearch and open linked trace in Jaeger.
- 

## 47.12 Review questions

1. Distinguish monitoring from observability with an example novel failure.
  2. What are the four metric types in Prometheus, and when use each?
  3. Explain RED and USE methods and their target systems.
  4. Why is structured logging preferred over plain text in production?
  5. What is a trace span, and how do spans form a trace tree?
  6. Why is high cardinality dangerous in metric labels?
  7. What makes an alert “actionable” vs noise?
- 

*Continue: Chapter 44 — Prometheus & Grafana Stack*

# 48 Chapter 44: Prometheus & Grafana Stack

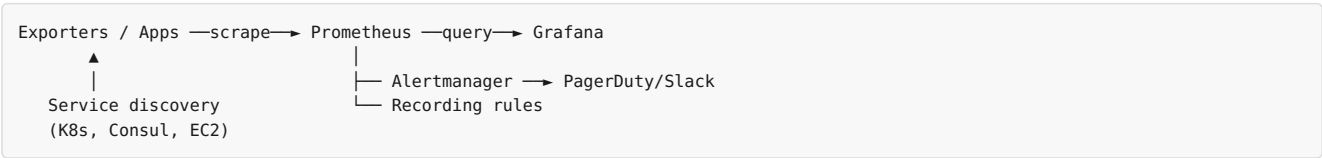
DevOps Handbook — Part X, Pages 866–890

## 48.1 44.1 The Prometheus ecosystem

**Prometheus** is an open-source monitoring system and time-series database (TSDB) designed for cloud-native environments. Created at SoundCloud, now a CNCF graduated project, it powers metrics collection for Kubernetes and microservices worldwide.

**Grafana** visualizes metrics from Prometheus (and 100+ other sources), providing dashboards, alerting UI, and exploration.

Typical stack:



Prometheus uses a **pull model**: it scrapes HTTP `/metrics` endpoints on intervals. Pushgateway exists for short-lived batch jobs but is not the default pattern.

## 48.2 44.2 Prometheus architecture

| Component         | Role                                                        |
|-------------------|-------------------------------------------------------------|
| Prometheus server | Scrapes, stores TSDB, evaluates rules                       |
| Exporters         | Expose third-party metrics (node_exporter, mysqld_exporter) |
| Alertmanager      | Dedupes, routes, silences alerts                            |
| Pushgateway       | Accept push for batch jobs                                  |
| Service discovery | Dynamic scrape targets                                      |

Data model:

- **Metric name + labels** identify a time series
- **Samples**: float value + millisecond timestamp
- Retention default ~15 days (configurable); long-term storage via Thanos, Cortex, Mimir

## 48.3 44.3 Prometheus configuration

`prometheus.yml`:

```
global:
 scrape_interval: 15s
 evaluation_interval: 15s
 external_labels:
 cluster: production
 region: us-east-1

alerting:
```

```

alertmanagers:
 - static_configs:
 - targets: ['alertmanager:9093']

rule_files:
 - /etc/prometheus/rules/*.yaml

scrape_configs:
 - job_name: prometheus
 static_configs:
 - targets: ['localhost:9090']

 - job_name: node
 static_configs:
 - targets: ['node-exporter:9100']

 - job_name: kubernetes-pods
 kubernetes_sd_configs:
 - role: pod
 relabel_configs:
 - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
 action: keep
 regex: true
 - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_port]
 action: replace
 target_label: __address__
 regex: (.+)
 replacement: $1

```

**Relabeling** filters and transforms targets before scrape—essential for Kubernetes pod discovery.

Pod annotation example:

```

metadata:
 annotations:
 prometheus.io/scrape: "true"
 prometheus.io/port: "8080"
 prometheus.io/path: "/metrics"

```

## 48.4 44.4 PromQL essentials

**PromQL** (Prometheus Query Language) queries time series.

Basic selectors:

```

Instant vector: current value
http_requests_total

Filter by label
http_requests_total{status="500", service="api"}

Rate over 5 minutes (counter → per-second rate)
rate(http_requests_total[5m])

Error ratio
sum(rate(http_requests_total{status=~"5.."}[5m]))
/
sum(rate(http_requests_total[5m]))

p99 latency from histogram
histogram_quantile(0.99,
 sum(rate(http_request_duration_seconds_bucket[5m])) by (le, service)
)

CPU usage (node exporter)
100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)

```

Common functions:

| Function | Use |
|----------|-----|
|          |     |

|                                   |                                    |
|-----------------------------------|------------------------------------|
| <code>rate()</code>               | Per-second rate for counters       |
| <code>increase()</code>           | Total increase over range          |
| <code>histogram_quantile()</code> | Percentiles from histogram buckets |
| <code>avg_over_time()</code>      | Gauge averages                     |
| <code>predict_linear()</code>     | Disk fill prediction               |
| <code>absent()</code>             | Detect missing metrics             |

## 48.5 44.5 Alerting rules and Alertmanager

Recording rules pre-compute expensive queries; alerting rules fire when conditions hold.

`rules/api.yml` :

```
groups:
- name: api_alerts
 interval: 30s
 rules:
 - record: job:http_requests:rate5m
 expr: sum(rate(http_requests_total[5m])) by (job, service)

 - alert: HighErrorRate
 expr: |
 (
 sum(rate(http_requests_total{status=~"5.."}[5m])) by (service)
 /
 sum(rate(http_requests_total[5m])) by (service)
) > 0.05
 for: 5m
 labels:
 severity: critical
 annotations:
 summary: "High 5xx rate on {{ $labels.service }}"
 description: "Error rate {{ $value | humanizePercentage }} for 5m"

 - alert: HighLatencyP99
 expr: |
 histogram_quantile(0.99,
 sum(rate(http_request_duration_seconds_bucket[5m])) by (le, service)
) > 1
 for: 10m
 labels:
 severity: warning
```

Alertmanager routing ( `alertmanager.yml` ):

```
route:
 receiver: default
 group_by: ['alertname', 'service']
 group_wait: 30s
 group_interval: 5m
 repeat_interval: 4h
 routes:
 - match:
 severity: critical
 receiver: pagerduty
 - match:
 severity: warning
 receiver: slack

receivers:
- name: pagerduty
 pagerduty_configs:
 - service_key: SECRET
- name: slack
 slack_configs:
 - api_url: SECRET
 channel: '#alerts'
```

```

- name: default
 slack_configs:
 - api_url: SECRET
 channel: '#alerts-low'

```

Use **inhibition rules** to suppress noisy dependent alerts (node down inhibits all pod alerts on that node).

## 48.6 44.6 Grafana dashboards

Install Grafana, add Prometheus data source ( `http://prometheus:9090` ), build dashboards.

Example panel queries:

| Panel        | PromQL                                                          |
|--------------|-----------------------------------------------------------------|
| Request rate | <code>sum(rate(http_requests_total[5m])) by (service)</code>    |
| Error rate   | ratio query above                                               |
| p50/p95/p99  | <code>histogram_quantile(0.95, ...)</code>                      |
| Saturation   | <code>process_resident_memory_bytes</code> , queue depth gauges |

Dashboard best practices:

- **Row per service** or golden signals template
- **Variables** — `$service` , `$environment` dropdowns via label values
- **Annotations** — Deploy markers from CI webhook
- **Version control** — Export JSON to Git (Grafana provisioning or Grafana-as-code tools like **Grafonnet**, **Terraform grafana provider**)

Provisioning example `datasources.yml` :

```

apiVersion: 1
datasources:
- name: Prometheus
 type: prometheus
 access: proxy
 url: http://prometheus:9090
 isDefault: true

```

## 48.7 44.7 Kubernetes deployment patterns

**kube-prometheus-stack** Helm chart bundles Prometheus, Alertmanager, Grafana, and default Kubernetes dashboards:

```

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install kube-prometheus prometheus-community/kube-prometheus-stack \
 --namespace monitoring --create-namespace

```

**Prometheus Operator** manages Prometheus, Alertmanager, ServiceMonitor, PodMonitor CRDs:

```

apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
 name: api-metrics
 namespace: monitoring
spec:
 selector:
 matchLabels:
 app: api
 namespaceSelector:
 matchNames:

```

```

- production
endpoints:
- port: metrics
 interval: 30s
 path: /metrics

```

**Thanos** or **Grafana Mimir** add long-term storage and global query across clusters.

## 48.8 44.8 Exporters and instrumentation

Common exporters:

| Exporter           | Metrics source                        |
|--------------------|---------------------------------------|
| node_exporter      | Linux host CPU, memory, disk, network |
| blackbox_exporter  | Probe HTTP/TCP/ICMP uptime            |
| kube-state-metrics | Kubernetes object state               |
| postgres_exporter  | PostgreSQL stats                      |
| redis_exporter     | Redis stats                           |

Application instrumentation via Prometheus client libraries (Go, Python, Java, etc.) or **OpenTelemetry** exporting to Prometheus format.

Blackbox probe config:

```

modules:
 http_2xx:
 prober: http
 timeout: 5s
 http:
 valid_status_codes: [200, 201, 204]
 preferred_ip_protocol: ip4

```

## 48.9 44.9 Operations and troubleshooting

| Issue           | Investigation                                           |
|-----------------|---------------------------------------------------------|
| Target down     | Prometheus → Status → Targets; check network, TLS, path |
| Missing metrics | Verify exposition format, label typos                   |
| High memory     | Reduce retention, cardinality, scrape frequency         |
| Alert storm     | Alertmanager grouping, inhibition, for: duration        |
| Query timeout   | Recording rules, optimize PromQL, scale Prometheus      |

Capacity planning: ~1-2 bytes per sample; estimate  $(\text{metrics} \times \text{labels} \times \text{scrape\_freq} \times \text{retention})$ .

Security: authenticate Grafana, restrict Prometheus admin API, TLS for scrape in zero-trust networks.

## 48.10 44.10 Chapter summary

- Prometheus scrapes and stores metrics; PromQL analyzes time series; Alertmanager routes alerts.
- Grafana visualizes Prometheus data with dashboards, variables, and provisioning-as-code.
- Kubernetes deployments use ServiceMonitors, kube-prometheus-stack, and optional long-term storage

(Thanos/Mimir).

- Control cardinality, use recording rules, and design actionable alerts tied to symptoms.
- 

## 48.11 Lab 44.1 — Local Prometheus + Grafana

1. Run Prometheus, node\_exporter, and Grafana via Docker Compose.
  2. Scrape node metrics; build a dashboard with CPU, memory, disk panels.
  3. Add an alerting rule for disk > 85%; route to a webhook receiver (use webhook.site for testing).
- 

## 48.12 Lab 44.2 — ServiceMonitor on Kubernetes

1. Deploy kube-prometheus-stack on a test cluster.
  2. Expose a sample app with `/metrics`; create ServiceMonitor.
  3. Confirm targets appear in Prometheus UI; import a RED metrics dashboard.
- 

## 48.13 Review questions

1. Why does Prometheus use pull-based scraping instead of push by default?
  2. Explain the difference between recording rules and alerting rules.
  3. Write PromQL for 5xx error rate as a percentage over 5 minutes.
  4. What problem does Alertmanager solve that Prometheus alerts alone do not?
  5. How do ServiceMonitors differ from static scrape configs?
  6. Why is high label cardinality a problem for Prometheus?
  7. When would you add Thanos or Mimir to the stack?
- 

*Continue: Chapter 45 — ELK/EFK Log Aggregation*

# 49 Chapter 45: ELK/EFK Log Aggregation

DevOps Handbook — Part X, Pages 891–910

## 49.1 45.1 Centralized logging rationale

Production systems generate millions of log lines across containers, VMs, and serverless functions. SSHing to individual hosts and running `grep` does not scale and fails when pods are ephemeral.

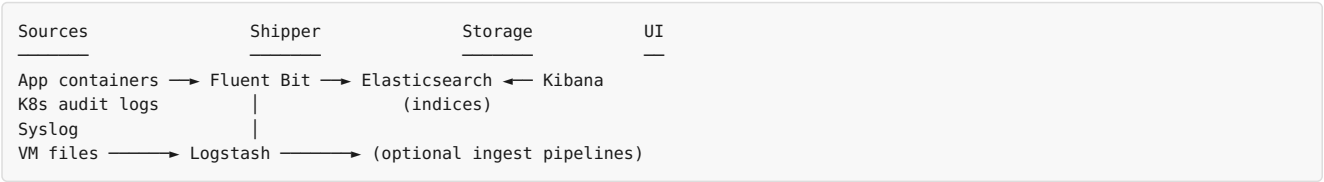
**Centralized log aggregation** collects, indexes, and searches logs from all sources in one platform—enabling correlation, compliance retention, and security analytics.

The dominant open-source stack:

| Stack | Components                                |
|-------|-------------------------------------------|
| ELK   | Elasticsearch, Logstash, Kibana           |
| EFK   | Elasticsearch, Fluent Bit/Fluentd, Kibana |
| ECK   | Elastic Cloud on Kubernetes (operator)    |

Alternatives: **Grafana Loki** (label-based, pairs with Prometheus), **OpenSearch** (AWS fork of Elasticsearch), cloud-native **CloudWatch Logs**, **Datadog**, **Splunk**.

## 49.2 45.2 Architecture overview



Data flow:

1. **Generate** — Apps write stdout/stderr (JSON preferred) or log files
2. **Ship** — Agents (Fluent Bit, Filebeat) forward logs
3. **Process** — Parse, enrich, filter (Logstash or ingest pipelines)
4. **Store** — Elasticsearch indexes documents for search
5. **Visualize** — Kibana Discover, Dashboards, alerting

## 49.3 45.3 Elasticsearch fundamentals

**Elasticsearch** is a distributed search and analytics engine storing logs as JSON documents in **indices**.

Concepts:

| Term     | Meaning                                   |
|----------|-------------------------------------------|
| Index    | Collection of documents (like a database) |
| Document | Single log entry (JSON)                   |
| Shard    | Horizontal partition of index             |
|          |                                           |



|         |                              |
|---------|------------------------------|
| Replica | Copy for HA and read scaling |
| Mapping | Schema (field types)         |

Index naming patterns:

```
logs-{service}-{yyyy.MM.dd}
filebeat-8.11.0-2026.09.01
```

Index Lifecycle Management (ILM) automates rollover, warm, cold, delete phases—controlling cost:

| Phase  | Action                              |
|--------|-------------------------------------|
| Hot    | Active writes, fast storage         |
| Warm   | Shrink shards, reduce replicas      |
| Cold   | Searchable snapshots, cheap storage |
| Delete | Remove after retention period       |

Example ILM policy (conceptual): hot 7 days → warm 30 days → delete at 90 days.

## 49.4 45.4 Fluent Bit vs Logstash vs Filebeat

| Tool       | Role                                              | Resource use |
|------------|---------------------------------------------------|--------------|
| Fluent Bit | Lightweight forwarder; K8s DaemonSet standard     | Very low     |
| Fluentd    | Richer routing; heavier                           | Medium       |
| Filebeat   | Elastic Beats shipper; simple file/container logs | Low          |
| Logstash   | Heavy transformation pipeline                     | High (JVM)   |

Fluent Bit Kubernetes config snippet:

```
apiVersion: v1
kind: ConfigMap
metadata:
 name: fluent-bit-config
data:
 fluent-bit.conf: |
 [SERVICE]
 Flush 5
 Log_Level info
 Daemon off

 [INPUT]
 Name tail
 Tag kube.*
 Path /var/log/containers/*.log
 Parser docker
 DB /var/log/flb_kube.db
 Mem_Buf_Limit 50MB

 [FILTER]
 Name kubernetes
 Match kube.*
 Kube_URL https://kubernetes.default.svc:443
 Merge_Log 0n
 Keep_Log 0ff
 K8S-Logging.Parser 0n

 [OUTPUT]
 Name es
```

|                    |                           |
|--------------------|---------------------------|
| Match              | *                         |
| Host               | elasticsearch.logging.svc |
| Port               | 9200                      |
| Index              | k8s-logs                  |
| Suppress_Type_Name | 0n                        |
| tls                | 0n                        |
| tls.verify         | 0n                        |

Deploy Fluent Bit as **DaemonSet**—one pod per node tailing `/var/log/containers/*.log`.

## 49.5 45.5 Logstash pipelines

**Logstash** receives, transforms, and outputs events:

```
input {
 beats {
 port => 5044
 }
}

filter {
 if [service] == "payment-api" {
 json {
 source => "message"
 }
 mutate {
 remove_field => ["message"]
 }
 }

 if [level] == "ERROR" {
 mutate {
 add_tag => ["needs_review"]
 }
 }

 geoip {
 source => "client_ip"
 }
}

output {
 elasticsearch {
 hosts => ["https://elasticsearch:9200"]
 user => "logstash_internal"
 password => "${LOGSTASH_PASSWORD}"
 index => "logs-%[service]}-%{+YYYY.MM.dd}"
 }
}
```

**Ingest pipelines** (Elasticsearch-native) reduce Logstash dependency for simpler transforms.

Prefer parsing at **application** (structured JSON on stdout)—shipper stays dumb, fast, reliable.

## 49.6 45.6 Kibana for search and dashboards

**Kibana** provides:

- **Discover** — Ad-hoc search with KQL (Kibana Query Language)
- **Visualize** — Charts, tables, maps
- **Dashboard** — Combined operational views
- **Alerting** — Log threshold rules (Elastic Stack alerting)

KQL examples:

```
service: "payment-api" and level: "ERROR"
```

```
trace_id: "4bf92f3577b329da7c93630a8a323129"
@timestamp >= "2026-09-01T00:00:00" and kubernetes.namespace: "production"
response.status >= 500
```

Link logs to traces: include `trace_id` and `span_id` fields matching OpenTelemetry/Jaeger.

Saved searches for on-call:

- Payment errors last hour
- OOMKilled container events
- Authentication failures spike

---

## 49.7 45.7 EFK on Kubernetes (ECK)

Elastic **ECK operator** manages Elasticsearch, Kibana, Beats CRDs:

```
apiVersion: elasticsearch.k8s.elastic.co/v1
kind: Elasticsearch
metadata:
 name: logging
spec:
 version: 8.11.0
 nodeSets:
 - name: default
 count: 3
 config:
 node.store.allow_mmap: false
 volumeClaimTemplates:
 - metadata:
 name: elasticsearch-data
 spec:
 accessModes: ["ReadWriteOnce"]
 resources:
 requests:
 storage: 100Gi
```

Sizing guidance:

| Daily log volume | Starting cluster                                          |
|------------------|-----------------------------------------------------------|
| < 50 GB/day      | 3 nodes, 8 GB heap each                                   |
| 50–500 GB/day    | Dedicated masters, hot/warm tiers                         |
| > 500 GB/day     | Index lifecycle, dedicated ingest, consult Elastic sizing |

Heap: stay below 50% RAM, max ~31 GB per node (compressed pointers).

---

## 49.8 45.8 Security and compliance

- **TLS everywhere** — Shipper to Elasticsearch, Kibana access
- **Authentication** — Native, SAML, OIDC integration
- **RBAC** — Index-level, document-level security (paid features in Elastic)
- **Audit logs** — Who searched what (compliance)
- **PII scrubbing** — Filter at ingest; never index credit cards, passwords

```
Logstash mutate — drop sensitive fields
filter {
 mutate {
 remove_field => ["password", "credit_card", "ssn"]
 }
}
```

Regulatory retention: ILM delete phase after required period; legal hold overrides.

## 49.9 45.9 Loki as an alternative

**Grafana Loki** indexes labels (like Prometheus), not full text—lower cost, less flexible search.

```
{namespace="production", app="api"} |= "error" | json | level="ERROR"
```

Choose Loki when already on Grafana/Prometheus and logs are structured. Choose ELK when full-text search, complex analytics, and Elastic ecosystem (SIEM) are required.

## 49.10 45.10 Operational best practices

| Practice                  | Benefit                                        |
|---------------------------|------------------------------------------------|
| JSON structured logs      | Faster queries, smaller pipelines              |
| Consistent @timestamp UTC | Cross-region correlation                       |
| Sample DEBUG in prod      | Cost control                                   |
| Dedicated logging cluster | Blast radius isolation from app cluster        |
| Monitor the monitors      | Alert on ingest lag, cluster health yellow/red |
| Test disaster recovery    | Snapshot/restore to S3 repository              |

Common failures:

- **Mapping explosions** — Dynamic fields from poorly structured logs
- **Hot shards** — Uneven index sizing; use rollover
- **Ingest backpressure** — Scale Fluent Bit buffers or Elasticsearch ingest nodes

## 49.11 45.11 Chapter summary

- ELK/EFK centralizes logs: shippers (Fluent Bit/Filebeat) → Elasticsearch → Kibana.
- Structure logs at the source; use ILM for retention and cost control.
- Fluent Bit DaemonSets are the Kubernetes standard for container log collection.
- Secure the stack with TLS, RBAC, and PII filtering; consider Loki for label-centric workloads.

## 49.12 Lab 45.1 — Docker Compose ELK

1. Deploy Elasticsearch, Logstash, Kibana, and Filebeat via Docker Compose.
2. Configure a sample app to log JSON to stdout; Filebeat ships to Logstash → ES.
3. In Kibana Discover, search by `level:ERROR` and create a visualization of errors over time.

## 49.13 Lab 45.2 — Fluent Bit on Kubernetes

1. Install Fluent Bit DaemonSet pointing to a managed or local Elasticsearch.
2. Deploy a pod that logs JSON with `trace_id` field.
3. Query logs in Kibana by `trace_id`; verify kubernetes metadata enrichment (pod name, namespace).

---

## 49.14 Review questions

1. Compare ELK and EFK—what component differs and why choose Fluent Bit?
2. What is an Elasticsearch index lifecycle policy, and why use it?
3. Why should applications emit structured JSON instead of plain text?
4. Explain the role of a DaemonSet log shipper in Kubernetes.
5. What causes mapping explosions, and how do you prevent them?
6. When would you choose Grafana Loki over Elasticsearch?
7. How do you correlate logs with distributed traces in Kibana?

---

*Continue: Chapter 46 — OpenTelemetry Distributed Tracing*

# 50 Chapter 46: OpenTelemetry Distributed Tracing

DevOps Handbook — Part X, Pages 911–930

## 50.1 46.1 The tracing problem

Microservices decompose monoliths into independently deployable units—each request may traverse API gateways, auth services, business logic, databases, caches, and message queues. When latency spikes or errors appear, **which hop failed?**

**Distributed tracing** records the path of a request across process boundaries. **OpenTelemetry (OTel)** is the CNCF standard unifying tracing, metrics, and logs instrumentation under one vendor-neutral API—replacing OpenTracing and OpenCensus.

Benefits:

- End-to-end latency breakdown
- Dependency mapping (service graph)
- Root cause analysis for errors
- Performance regression detection per release

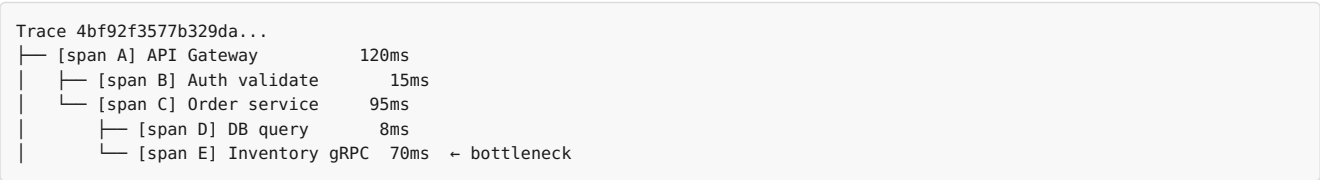
## 50.2 46.2 Traces, spans, and context propagation

| Concept             | Definition                                                |
|---------------------|-----------------------------------------------------------|
| Trace               | End-to-end journey of a request (shared trace ID)         |
| Span                | Single timed operation within a trace                     |
| Parent span         | Upstream operation that triggered child work              |
| Context propagation | Passing trace ID/span ID across HTTP/gRPC/message headers |

W3C **Trace Context** standard headers:

```
traceparent: 00-4bf92f3577b329da7c93630a8a323129-00f067aa0ba902b7-01
tracestate: congo=t61rcWkgMzE
```

Format: `version-trace_id-parent_span_id-flags`

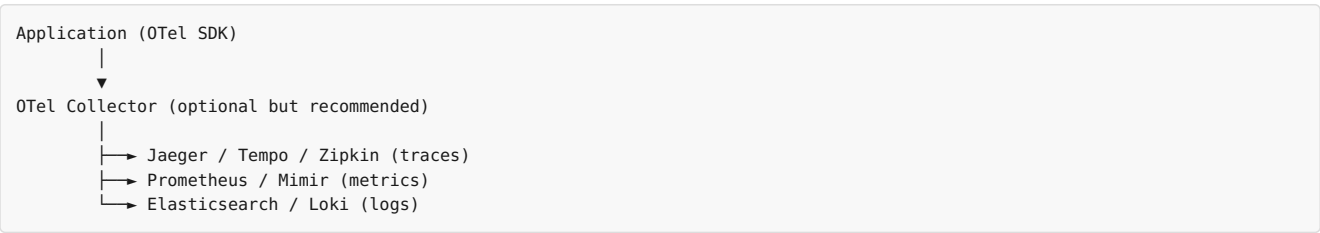


Span attributes (metadata):

- `http.method`, `http.status_code`
- `db.system`, `db.statement` (sanitized)
- `service.name`, `deployment.environment`

Span events: point-in-time annotations within a span (e.g., “cache miss”).

## 50.3 46.3 OpenTelemetry architecture



Components:

| Component                 | Role                                          |
|---------------------------|-----------------------------------------------|
| OTel API                  | Instrumentation interfaces in app code        |
| OTel SDK                  | Sampling, batching, export                    |
| Instrumentation libraries | Auto-instrument HTTP, gRPC, DB drivers        |
| OTel Collector            | Receive, process, export; vendor-agnostic hub |
| Backend                   | Storage and UI (Jaeger, Grafana Tempo)        |

## 50.4 46.4 Instrumentation: auto vs manual

**Auto-instrumentation** (agents) attaches without code changes:

```
Python example
opentelemetry-instrument \
 --traces_exporter otlp \
 --metrics_exporter otlp \
 --service_name order-api \
 --exporter_otlp_endpoint http://otel-collector:4317 \
 python app.py
```

Java: `-javaagent:opentelemetry-javaagent.jar`

**Manual instrumentation** for business logic:

```
from opentelemetry import trace
from opentelemetry.trace import Status, StatusCode

tracer = trace.get_tracer("order-service", "1.0.0")

def create_order(order_data):
 with tracer.start_as_current_span("create_order") as span:
 span.set_attribute("order.customer_id", order_data["customer_id"])
 try:
 validate(order_data)
 order_id = persist(order_data)
 span.set_attribute("order.id", order_id)
 return order_id
 except ValidationError as e:
 span.record_exception(e)
 span.set_status(Status(StatusCode.ERROR, str(e)))
 raise
```

Ensure **context propagation** across async boundaries (thread pools, asyncio, message queues)—dropped context breaks trace chains.

## 50.5 46.5 OTel Collector configuration

Collector pipelines: **receivers** → **processors** → **exporters**

```
receivers:
 otlp:
 protocols:
 grpc:
 endpoint: 0.0.0.0:4317
 http:
 endpoint: 0.0.0.0:4318

processors:
 batch:
 timeout: 5s
 send_batch_size: 1024
 memory_limiter:
 check_interval: 1s
 limit_mib: 512
 resource:
 attributes:
 - key: deployment.environment
 value: production
 action: upsert
 tail_sampling:
 decision_wait: 10s
 policies:
 - name: errors
 type: status_code
 status_code:
 status_codes: [ERROR]
 - name: latency
 type: latency
 latency:
 threshold_ms: 1000
 - name: probabilistic
 type: probabilistic
 probabilistic:
 sampling_percentage: 5

exporters:
 otlp/jaeger:
 endpoint: jaeger-collector:4317
 tls:
 insecure: true
 prometheus:
 endpoint: 0.0.0.0:8889

service:
 pipelines:
 traces:
 receivers: [otlp]
 processors: [memory_limiter, tail_sampling, batch, resource]
 exporters: [otlp/jaeger]
 metrics:
 receivers: [otlp]
 processors: [batch, resource]
 exporters: [prometheus]
```

**Tail sampling** at collector evaluates complete traces—keep all errors and slow traces, sample routine success paths.

## 50.6 46.6 Sampling strategies

Tracing every request at high QPS is expensive. Sampling reduces volume while preserving debuggability.

| Strategy           | Behavior                                            |
|--------------------|-----------------------------------------------------|
| Always on          | Dev/test only                                       |
| Probabilistic head | Random % at trace start; simple but may miss errors |
| Rate limiting      | Max traces per second                               |
| Tail sampling      | Decide after trace completes; keep errors/slow      |



Head sampling in SDK:

```
from opentelemetry.sdk.trace.sampling import TraceIdRatioBased

sampler = TraceIdRatioBased(0.1) # 10% of traces
```

Production recommendation: **tail sampling in Collector** with error and latency policies plus low baseline probabilistic sample.

---

## 50.7 46.7 Backends: Jaeger and Grafana Tempo

**Jaeger** (CNCF graduated): UI for trace search, dependency graphs, compare traces.

Deploy Jaeger on Kubernetes:

```
Simplified – use Jaeger Operator in production
apiVersion: apps/v1
kind: Deployment
metadata:
 name: jaeger
spec:
 template:
 spec:
 containers:
 - name: jaeger
 image: jaegertracing/all-in-one:1.52
 ports:
 - containerPort: 16686 # UI
 - containerPort: 4317 # OTLP gRPC
```

**Grafana Tempo**: object storage backend (S3/GCS), integrates natively with Grafana—trace-to-log correlation via `trace_id`.

Trace search:

- By trace ID (from logs or error message)
- By service + operation + duration
- By tags ( `http.status_code=500` )

---

## 50.8 46.8 Correlating traces, logs, and metrics

**Exemplars** link metrics to traces—a histogram bucket points to example trace ID.

Logs include trace context:

```
{
 "message": "order created",
 "trace_id": "4bf92f3577b329da7c93630a8a323129",
 "span_id": "00f067aa0ba902b7"
}
```

Grafana **Trace to logs** jump: from span → Loki query `{trace_id="..."}`.

Metrics derived from traces (span metrics connector in Collector): RED metrics per service from trace data without duplicate instrumentation.

---

## 50.9 46.9 Service mesh tracing

**Istio** and **Linkerd** generate spans automatically for mesh traffic—application instrumentation still needed for in-process detail.

Istio enables tracing via Telemetry API; spans exported to OTel Collector.

Mesh traces show network latency (TLS, retries); app traces show business logic—combine both.

---

## 50.10 46.10 Adoption roadmap

1. **Deploy Collector** — Central export point before picking backend
2. **Auto-instrument ingress** — API gateway, primary services
3. **Propagate context** — Verify cross-service trace continuity
4. **Add manual spans** — Critical business operations
5. **Tail sampling** — Cost control in production
6. **Integrate Grafana/Jaeger** — On-call workflows, dashboards
7. **SLO dashboards** — Latency from trace metrics

Anti-patterns:

- Instrumentation without sampling plan (cost surprise)
  - Broken context in async/message paths
  - Storing full SQL with PII in span attributes
- 

## 50.11 46.11 Chapter summary

- OpenTelemetry standardizes traces, metrics, and logs with vendor-neutral SDKs and Collector.
  - Traces consist of spans linked by context propagation (W3C Trace Context headers).
  - Use auto-instrumentation plus manual spans; tail sampling preserves errors while controlling cost.
  - Correlate traces with logs ( `trace_id` ) and metrics (exemplars) for full observability.
- 

## 50.12 Lab 46.1 — OTel + Jaeger locally

1. Run OTel Collector, Jaeger, and a two-service demo (e.g., frontend → backend).
  2. Auto-instrument both services with OTel agents.
  3. Generate traffic; verify end-to-end trace in Jaeger UI.
  4. Introduce artificial 2s delay in backend; identify span in critical path.
- 

## 50.13 Lab 46.2 — Tail sampling

1. Configure Collector tail sampling: 100% errors, 5% success.
  2. Generate 1000 successful requests and 50 failing requests.
  3. Compare trace counts in Jaeger; confirm error traces retained.
- 

## 50.14 Review questions

1. What is the difference between a trace and a span?
2. Explain W3C Trace Context propagation across HTTP services.
3. Why is tail sampling preferred over head sampling in production?
4. What role does the OpenTelemetry Collector play?

5. How do you correlate logs with traces in Grafana?
  6. When is manual instrumentation necessary beyond auto-instrumentation?
  7. What are exemplars, and how do they connect metrics to traces?
- 

*Continue: Chapter 47 — SLOs, Error Budgets, Incident Response*

# 51 Chapter 47: SLOs, Error Budgets, and Incident Response

DevOps Handbook — Part X, Pages 931–945

## 51.1 47.1 Reliability as a product decision

Uptime is not maximized at 100%—infinite reliability costs infinite money and slows feature delivery. **Site Reliability Engineering (SRE)** treats reliability as a **product feature** with an explicit target negotiated between engineering and business.

**Service Level Objective (SLO)** — Target reliability (e.g., 99.9% availability over 30 days).

**Service Level Indicator (SLI)** — Measured metric (e.g., successful requests / total requests).

**Service Level Agreement (SLA)** — Contract with customers (often looser than internal SLO).

**Error budget** — Allowed unreliability:  $1 - \text{SLO}$ . If SLO is 99.9%, budget is 0.1% (~43 minutes downtime/month).

When error budget is \*\* exhausted, **freeze risky releases and invest in reliability**. When budget is healthy\*\*, ship features aggressively.

## 51.2 47.2 Choosing SLIs and SLOs

SLIs must reflect **user experience**, not internal convenience.

| Service type         | Good SLI                 | Poor SLI              |
|----------------------|--------------------------|-----------------------|
| Request/response API | Success rate + latency   | CPU utilization       |
| Data pipeline        | Freshness, completeness  | Worker pod count      |
| Storage              | Durability, read success | Disk free space alone |

Example API SLIs:

```
Availability SLI (30-day window)
sum(rate(http_requests_total{status!~"5.."}[5m]))
/
sum(rate(http_requests_total[5m]))

Latency SLI: proportion under 300ms
sum(rate(http_request_duration_seconds_bucket{le="0.3"}[5m]))
/
sum(rate(http_request_duration_seconds_count[5m]))
```

SLO targets (illustrative):

| Tier             | Availability | Latency (p99) |
|------------------|--------------|---------------|
| Critical payment | 99.99%       | < 200ms       |
| Core API         | 99.9%        | < 500ms       |
| Internal tooling | 99.5%        | < 2s          |

Document SLOs in a **SLO document** per service: SLI definition, query, target, window, owners, escalation.

## 51.3 47.3 Error budgets in practice

Error budget calculation (30-day rolling):

```
Budget = (1 - SL0) × total_events
Consumed = failed_events or slow_events
Remaining = Budget - Consumed
```

Burn rate alerts predict budget exhaustion before window ends:

| Burn rate | Meaning                                   | Action       |
|-----------|-------------------------------------------|--------------|
| 1×        | On track to exhaust exactly at window end | Monitor      |
| 2×        | Exhaust in half the window                | Investigate  |
| 14.4×     | Exhaust in ~2 days at 99.9% monthly       | Page on-call |

Multi-window burn alert (Google SRE):

```
Conceptual – implement via Prometheus recording rules
Fast burn: 2% budget in 1 hour → page
Slow burn: 5% budget in 6 hours → ticket
```

Policy when budget depleted:

- 1. Stop non-critical releases
- 2. Focus engineering on reliability work
- 3. Executive visibility—product tradeoff discussion
- 4. Resume feature velocity when budget recovers

Error budgets **align** dev and ops: shared metric replaces “ops blocks deploys” vs “dev ignores stability.”

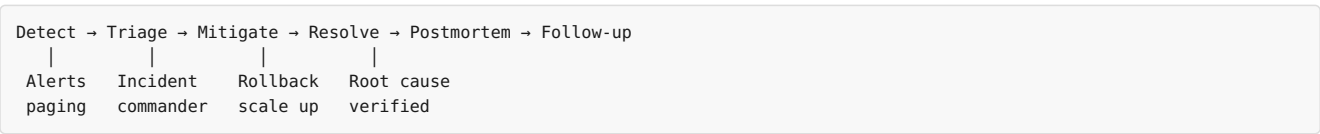
## 51.4 47.4 Incident severity and lifecycle

**Incident** — Unplanned interruption or degradation requiring coordinated response.

Severity levels (example):

| Severity | Impact                                 | Response                        |
|----------|----------------------------------------|---------------------------------|
| SEV-1    | Customer-facing outage, revenue impact | Immediate all-hands, exec comms |
| SEV-2    | Major degradation, workaround exists   | On-call + team lead within 15m  |
| SEV-3    | Minor impact, limited users            | Business hours response         |
| SEV-4    | Cosmetic, no user impact               | Backlog                         |

Incident lifecycle:



Roles:

| Role                    | Responsibility                         |
|-------------------------|----------------------------------------|
| Incident Commander (IC) | Coordinates response, decisions, comms |

|                            |                                  |
|----------------------------|----------------------------------|
| <b>Technical Lead</b>      | Drives debugging and mitigation  |
| <b>Communications Lead</b> | Status page, stakeholder updates |
| <b>Scribe</b>              | Timeline, actions log            |

## 51.5 47.5 On-call and escalation

Effective on-call requires:

- **Runbooks** — Step-by-step for known alerts
- **Dashboards** — Golden signals per service linked from alerts
- **Escalation policy** — Primary → secondary → manager
- **Handoff** — Document active issues between shifts
- **Toil limits** — On-call should not mean endless manual work

PagerDuty/Opsgenie escalation example:

1. Primary on-call (5 min)
2. Secondary on-call (10 min)
3. Engineering manager (15 min)
4. Incident commander pool (SEV-1)

**Alert quality** directly affects on-call burnout—every page must require human action (Chapter 43).

## 51.6 47.6 Incident response procedures

**Detection** — Automated alerts on SLO burn, synthetic probes, customer reports.

**Triage checklist:**

1. Confirm user impact (not false positive)
2. Assign severity and IC
3. Open incident channel ( `#inc-2026-09-01-api-outage` )
4. Start timeline document
5. Communicate status page update within 15 min (SEV-1)

**Mitigation over root cause** — Restore service first (rollback, failover, scale, feature flag off), investigate after.

Common mitigations:

| Symptom         | Fast mitigation                     |
|-----------------|-------------------------------------|
| Bad deploy      | Rollback to previous version        |
| Traffic spike   | Scale horizontally, rate limit      |
| Dependency down | Circuit breaker, degrade gracefully |
| Data corruption | Stop writes, restore from backup    |

**Resolution** — Verify SLI recovery, close incident, schedule postmortem within 5 business days.

## 51.7 47.7 Blameless postmortems

Postmortem documents **what happened**, **why** (contributing factors, not single root cause), and **action items**—without blaming individuals.

Template sections:

1. Summary (one paragraph)
2. Impact (duration, users affected, SLO breach)
3. Timeline (UTC timestamps)
4. Root cause analysis (5 whys, fishbone)
5. What went well / what went poorly
6. Action items (owner, due date, priority)
7. Lessons learned

Blameless culture encourages reporting near-misses—hidden failures repeat.

Action item tracking: treat reliability fixes like product backlog items with SLAs.

---

## 51.8 47.8 Status communication

**Status page** (Statuspage, Instatus, custom):

- Investigating → Identified → Monitoring → Resolved
- Update every 30–60 min during SEV-1
- Honest scope—avoid “all systems operational” when degraded

Internal vs external messaging:

- Internal: technical detail, hypothesis, commands run
  - External: user impact, workaround, ETA if confident
- 

## 51.9 47.9 Tooling integration

Integrate observability → incident → postmortem:

```
Prometheus alert → Alertmanager → PagerDuty → Slack #incidents
Deploy webhook → Timeline auto-entry
Grafana snapshot → Postmortem attachment
Jira/Linear → Action item tickets from postmortem template
```

Tag incidents with **deployment ID** and **trace IDs** for DORA change failure analysis.

---

## 51.10 47.10 Chapter summary

- SLOs define target reliability; SLIs measure it; error budgets balance feature velocity and stability.
  - Burn rate alerts provide early warning before budget exhaustion.
  - Incident response prioritizes mitigation, clear roles, and stakeholder communication.
  - Blameless postmortems drive systemic improvements with tracked action items.
- 

## 51.11 Lab 47.1 — Define SLOs for a sample service

1. Pick a sample HTTP API with Prometheus metrics.
  2. Write SLI queries for availability and latency.
  3. Set SLO target 99.9% / 30 days; calculate monthly error budget in minutes.
  4. Create Grafana dashboard showing budget remaining.
-

## 51.12 Lab 47.2 — Tabletop incident exercise

1. Scenario: deploy causes 10% 5xx rate spike.
  2. Assign roles (IC, tech lead, comms, scribe).
  3. Run 30-minute simulated response: triage, mitigation decision, status update draft.
  4. Write blameless postmortem with 3 action items.
- 

## 51.13 Review questions

1. Distinguish SLI, SLO, and SLA with an example.
  2. How is error budget calculated for 99.95% over 30 days?
  3. What should happen when error budget is exhausted mid-quarter?
  4. Why prioritize mitigation over root cause during an active incident?
  5. What makes a postmortem blameless, and why does it matter?
  6. Explain multi-window burn rate alerting.
  7. What SLIs would you choose for a batch data pipeline?
- 

*Continue: Chapter 48 — Chaos Engineering and Game Days*



# 52 Chapter 48: Chaos Engineering and Game Days

DevOps Handbook — Part X, Pages 946–960

## 52.1 48.1 Proactive resilience testing

Production fails in unexpected ways—despite staging environments, tests, and code review. **Chaos engineering** deliberately injects failures into systems to validate resilience **before** real outages expose gaps.

*“Chaos engineering is the discipline of experimenting on a system in order to build confidence in the system’s capability to withstand turbulent conditions in production.” — Principles of Chaos Engineering*

Not reckless destruction: chaos experiments are **hypothesis-driven, controlled, measured, and bounded** in blast radius.

Relationship to other practices:

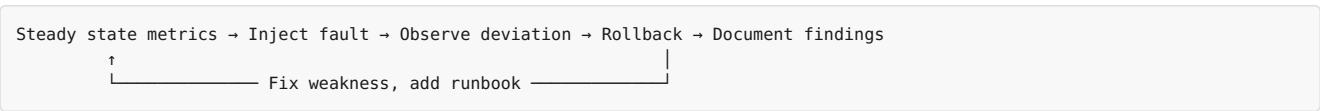
| Practice          | Focus                                         |
|-------------------|-----------------------------------------------|
| Testing           | Known expected behavior                       |
| Chaos engineering | Unknown failure modes, emergent behavior      |
| Game days         | Scheduled team exercises simulating incidents |
| DR drills         | Region/account failover                       |

Netflix **Chaos Monkey** (2011) randomly terminated production instances—proving redundancy worked. The practice evolved into comprehensive fault injection platforms.

## 52.2 48.2 Principles of chaos engineering

1. **Build a hypothesis around steady state** — Define normal (SLO metrics, error rate, latency).
2. **Vary real-world events** — Server failure, network partition, dependency timeout, disk full.
3. **Run experiments in production** — Staging lacks scale and traffic patterns (start in non-prod, graduate to prod with safeguards).
4. **Automate experiments** — Manual one-offs do not scale; continuous verification.
5. **Minimize blast radius** — Start small (one pod, one AZ, 1% traffic).

Experiment workflow:



## 52.3 48.3 Failure modes to inject

| Category       | Examples                                         |
|----------------|--------------------------------------------------|
| Infrastructure | Kill pod/VM, drain node, AZ failure              |
| Network        | Latency, packet loss, DNS failure, partition     |
| Dependency     | Upstream 503, slow responses, certificate expiry |
| Resource       | CPU/memory pressure, disk fill, connection pool  |

|             |                                                    |
|-------------|----------------------------------------------------|
|             | exhaustion                                         |
| Application | Exception injection, feature flag misconfiguration |
| Data        | Replica lag, split-brain simulation                |
| Human       | On-call unreachable (game day scenario)            |

Prioritize failures matching **real incident history**—postmortem action items often become chaos experiments.

## 52.4 48.4 Chaos tools landscape

| Tool               | Environment         | Capabilities                              |
|--------------------|---------------------|-------------------------------------------|
| Litmus Chaos       | Kubernetes          | CRD-based experiments, hub of scenarios   |
| Chaos Mesh         | Kubernetes          | Pod kill, network chaos, IO chaos, stress |
| Gremlin            | K8s, VM, bare metal | SaaS/enterprise fault injection           |
| AWS FIS            | AWS                 | EC2 stop, AZ impairment, RDS failover     |
| Azure Chaos Studio | Azure               | Managed chaos experiments                 |
| Toxiproxy          | Dev/test            | Network condition proxy                   |
| Pumba              | Docker              | Container chaos                           |

Litmus experiment example:

```
apiVersion: litmuschaos.io/v1alpha1
kind: ChaosEngine
metadata:
 name: api-pod-delete
 namespace: production
spec:
 appinfo:
 appns: production
 applabel: "app=api"
 appkind: deployment
 chaosServiceAccount: litmus-admin
 experiments:
 - name: pod-delete
 spec:
 components:
 env:
 - name: TOTAL_CHAOS_DURATION
 value: "60"
 - name: CHAOS_INTERVAL
 value: "10"
 - name: FORCE
 value: "false"
```

Chaos Mesh NetworkChaos (latency):

```
apiVersion: chaos-mesh.org/v1alpha1
kind: NetworkChaos
metadata:
 name: delay-inventory
spec:
 action: delay
 mode: one
 selector:
 namespaces:
 - production
 labelSelectors:
 app: order-api
 delay:
 latency: "500ms"
 correlation: "100"
```

```
jitter: "50ms"
direction: to
target:
 selector:
 namespaces:
 - production
 labelSelectors:
 app: inventory-service
```

## 52.5 48.5 Safe experimentation guardrails

Before running in production:

| Guardrail                        | Purpose                                    |
|----------------------------------|--------------------------------------------|
| Abort conditions                 | Auto-stop if error rate > threshold        |
| Business hours vs off-peak       | Limit user impact window                   |
| One variable at a time           | Isolate cause and effect                   |
| Rollback plan                    | Immediate experiment halt mechanism        |
| Stakeholder notification         | Support, leadership aware                  |
| Exclude critical paths initially | Payment during peak—defer                  |
| Observability first              | Cannot validate hypothesis without metrics |

Integrate with **OpenTelemetry/Prometheus** dashboards dedicated to experiment runs.

**Steady state hypothesis** example:

*“When one API pod is terminated, p99 latency remains < 500ms and error rate < 0.1% because Kubernetes reschedules within 30s and HPA maintains capacity.”*

If hypothesis fails: fix ( PDB, probes, HPA tuning), do not dismiss the experiment.

## 52.6 48.6 Game days

**Game days** are scheduled, cross-functional exercises—often simulating major incidents without continuous automated chaos.

Structure (4-hour example):

| Time      | Activity                                      |
|-----------|-----------------------------------------------|
| 0:00      | Briefing, roles assigned, rules of engagement |
| 0:15      | Inject scenario (facilitator triggers faults) |
| 0:15–2:00 | Team responds as real incident                |
| 2:00      | End injection, debrief hot wash               |
| 2:30      | Document gaps, update runbooks                |

Scenarios:

- Primary database failover
- Region loss (traffic shift drill)
- Certificate expiry on ingress
- DDoS simulation (with provider coordination)

- Supply chain compromise response

Facilitator **does not** fix—observes communication, decision quality, tooling gaps.

Include: engineering, SRE, support, comms, product—for SEV-1 realism.

Frequency: quarterly per critical service; annual org-wide DR game day.

---

## 52.7 48.7 From experiment to improvement

Chaos findings feed backlog:

| Finding                           | Remediation                   |
|-----------------------------------|-------------------------------|
| No PDB on critical Deployment     | Add PodDisruptionBudget       |
| HPA too slow                      | Tune metrics, reduce cooldown |
| Retry storm on dependency failure | Circuit breaker + bulkhead    |
| Alert did not fire                | Fix SLO burn alert            |
| Runbook outdated                  | Update with verified steps    |

Track **resilience score**: % of critical services with passing monthly chaos experiments.

Automate regression: experiment suite runs in staging on every release candidate.

---

## 52.8 48.8 Organizational maturity

| Maturity | Behavior                                               |
|----------|--------------------------------------------------------|
| Level 0  | No chaos; surprises in prod                            |
| Level 1  | Ad-hoc manual tests in staging                         |
| Level 2  | Scheduled game days, documented scenarios              |
| Level 3  | Automated chaos in staging CI                          |
| Level 4  | Continuous limited chaos in production with guardrails |
| Level 5  | Chaos-driven architecture; resilience SLOs             |

Start at Level 1–2; production chaos requires strong observability and org trust.

---

## 52.9 48.9 Chapter summary

- Chaos engineering validates resilience through controlled, hypothesis-driven failure injection.
- Tools like Litmus and Chaos Mesh inject infrastructure and network faults in Kubernetes.
- Guardrails (abort conditions, blast radius limits) make production experiments safe.
- Game days train teams on incident response; findings improve architecture, alerts, and runbooks.

---

## 52.10 Lab 48.1 — Pod delete experiment

1. Deploy sample app with 3 replicas, PDB, and HPA in test cluster.

2. Run Litmus or kubectl delete pod experiment during load test.
  3. Measure recovery time and error rate; document hypothesis pass/fail.
- 

## **52.11 Lab 48.2 — Mini game day**

1. Design a 1-hour scenario: dependency returns 503 for 10 minutes.
  2. Facilitate team response with IC and scribe roles.
  3. Debrief: what broke, what worked, 3 action items.
- 

## **52.12 Review questions**

1. State the five principles of chaos engineering.
  2. Why run experiments in production rather than only in staging?
  3. What abort conditions would you set for a pod-delete experiment?
  4. Compare chaos engineering and game days.
  5. Name three failure modes worth injecting for a microservices API.
  6. How should a failed chaos hypothesis be handled?
  7. What organizational prerequisites are needed before production chaos?
- 

*Continue: Chapter 49 — Shift-Left Security*

# 53 Chapter 49: Shift-Left Security in the Pipeline

DevOps Handbook — Part XI, Pages 961–980

## 53.1 49.1 DevSecOps and shift-left

**DevSecOps** integrates security into the software delivery lifecycle—not as a final gate before release, but as continuous practice from design through operation. **Shift-left** means moving security activities **earlier**—cheaper to fix vulnerabilities in IDE or PR than in production breach.

Traditional model:

```
Dev → Test → Security audit (weeks) → Deploy → Pen test → Incident
```

Shift-left model:

```
Threat model → Secure coding → Pre-commit → PR scan → CI scan → Deploy → Runtime monitor
```

Security becomes **everyone’s responsibility** with platform-enabling tooling—not solely a separate security team blocking releases.

## 53.2 49.2 Security activities across the SDLC

| Phase   | Activities                                                           |
|---------|----------------------------------------------------------------------|
| Design  | Threat modeling (STRIDE), security requirements, data classification |
| Develop | Secure coding standards, IDE plugins, pre-commit hooks               |
| Build   | SAST, SCA, secret scanning, IaC scan                                 |
| Test    | DAST, fuzzing, penetration test automation                           |
| Deploy  | Image signing, admission control, least-privilege IAM                |
| Operate | Runtime detection, WAF, audit logs, incident response                |
| Respond | Vulnerability disclosure, patch SLAs, forensics                      |

The **software supply chain** spans all phases—dependencies, build systems, registries, and deployment pipelines are attack surfaces (SolarWinds, Log4Shell lessons).

## 53.3 49.3 Threat modeling fundamentals

**STRIDE** categorizes threats:

| Letter | Threat                 | Example                            |
|--------|------------------------|------------------------------------|
| S      | Spoofing               | Fake JWT                           |
| T      | Tampering              | Modify API payload                 |
| R      | Repudiation            | Deny transaction without audit log |
| I      | Information disclosure | Log leaking PII                    |
| D      | Denial of service      | Unbounded query                    |

|          |                        |               |
|----------|------------------------|---------------|
| <b>E</b> | Elevation of privilege | IDOR to admin |
|----------|------------------------|---------------|

Process:

1. Draw data flow diagram (users, services, data stores)
2. Identify trust boundaries
3. Apply STRIDE per component
4. Prioritize mitigations (risk = likelihood × impact)
5. Track in backlog

Tools: **Microsoft Threat Modeling Tool**, **OWASP Threat Dragon**, Miro templates.

Output feeds security test cases and pipeline scan configuration.

## 53.4 49.4 Pipeline security gates

Integrate checks in CI without destroying developer velocity:

```
Commit → Secret scan → Lint/IaC policy → SAST → Unit tests → Build →
Container scan → SCA → Sign artifact → Deploy (policy) → DAST (staging)
```

Gate philosophy:

| Severity | Policy example                   |
|----------|----------------------------------|
| Critical | Block merge/deploy               |
| High     | Block deploy to prod; warn on PR |
| Medium   | Ticket required; SLA 30 days     |
| Low      | Informational                    |

**Policy as Code** (OPA, Conftest) enforces gates consistently:

```
deny deployment if critical CVEs in scan result
package deploy

deny[msg] {
 input.scan.critical_count > 0
 msg := sprintf("Critical vulnerabilities: %d", [input.scan.critical_count])
}
```

Balance **speed vs safety**: fast feedback on PR (< 10 min for secret/SAST); heavier scans async or nightly.

## 53.5 49.5 Secret scanning

Secrets in Git history are permanent until rotated. Scan **commits**, **PRs**, and **history**.

Tools:

| Tool                           | Integration      |
|--------------------------------|------------------|
| <b>GitLeaks</b>                | Pre-commit, CI   |
| <b>TruffleHog</b>              | CI, deep history |
| <b>GitHub secret scanning</b>  | Native on GitHub |
| <b>GitLab secret detection</b> | Native on GitLab |

Pre-commit hook:

```
.pre-commit-config.yaml
repos:
- repo: https://github.com/gitleaks/gitleaks
 rev: v8.18.0
 hooks:
 - id: gitleaks
```

CI step:

```
- name: Secret scan
 uses: gitleaks/gitleaks-action@v2
 env:
 GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```

On leak: **rotate immediately**, purge from history ( `git filter-repo` ) if exposed publicly—scanning alone is insufficient.

### 53.6 49.6 Secure defaults and platform guardrails

Platform engineering enables secure defaults:

- **Base images** — Hardened, scanned, regularly rebuilt
- **Template repos** — CI security jobs pre-configured
- **Namespace/network policies** — Deny-by-default in Kubernetes
- **IAM roles** — Short-lived credentials, no long-lived keys in repos
- **Dependency update bots** — Renovate, Dependabot with auto-merge for patches

**Golden paths** make the secure path the easy path—developers opt out explicitly, not opt in.

### 53.7 49.7 Security champions program

Scale security expertise without bottleneck:

| Role                         | Responsibility                                  |
|------------------------------|-------------------------------------------------|
| Central security             | Policy, tooling, compliance, escalation         |
| Security champion (per team) | Triage findings, threat model reviews, training |
| Developers                   | Fix findings, secure coding                     |

Champions attend monthly syncs, get early tool access, relay team friction to security org.

Metrics: mean time to remediate (MTTR) vulnerabilities by severity, not raw finding count (discourages hiding issues).

### 53.8 49.8 Compliance integration

Map shift-left controls to frameworks:

| Control area      | Pipeline automation           |
|-------------------|-------------------------------|
| Access control    | IAM review in IaC scan        |
| Change management | Git audit trail, approved PRs |



|                    |                                          |
|--------------------|------------------------------------------|
| Vulnerability mgmt | SCA/SAST gates, ticket integration       |
| Logging/monitoring | Mandatory observability checks           |
| Encryption         | TLS policy, secret management validation |

Evidence collection: export CI scan reports, deployment logs, and policy decisions for SOC 2, ISO 27001, PCI audits —**continuous compliance** vs annual scramble.

---

## 53.9 49.9 Culture and psychological safety

Developers must report security concerns without fear. Blameless response to leaked secrets encourages fast rotation.

Training:

- OWASP Top 10 awareness
- Secure coding labs (SQLi, XSS in sandbox)
- Annual phishing simulations

Security team as **enabler** (“here’s how to fix”) not **gatekeeper** (“no”).

---

## 53.10 49.10 Chapter summary

- Shift-left embeds security early: threat modeling, scanning, and policy gates in CI/CD.
  - Secret scanning, SAST/SCA/DAST (Chapters 50–52), and policy-as-code form layered defense.
  - Platform golden paths and security champions scale practices across teams.
  - Balance blocking gates with remediation SLAs and developer experience.
- 

## 53.11 Lab 49.1 — Pre-commit security stack

1. Add pre-commit with gitleaks and detect-private-key hooks to a sample repo.
  2. Attempt to commit a fake AWS key; confirm hook blocks.
  3. Add CI job duplicating secret scan for defense in depth.
- 

## 53.12 Lab 49.2 — Threat model exercise

1. Draw data flow for a three-tier web app.
  2. Apply STRIDE to API gateway and database boundary.
  3. Produce 5 mitigations mapped to pipeline controls (scan type or policy).
- 

## 53.13 Review questions

1. What does “shift-left” mean in DevSecOps?
2. List security activities appropriate at design vs deploy phases.
3. Explain STRIDE with one example threat per category.
4. How should CI gates differ for critical vs medium findings?
5. Why rotate secrets immediately when leaked to Git?
6. What is a security champion, and why use this model?

7. How does shift-left support compliance audits?

---

*Continue: Chapter 50 — SAST, DAST, SCA, Container Scanning*

# 54 Chapter 50: SAST, DAST, SCA, and Container Scanning

DevOps Handbook — Part XI, Pages 981-1000

## 54.1 50.1 Application security testing taxonomy

Modern applications combine custom code, open-source dependencies, container images, and infrastructure configuration—each layer requires specialized scanning.

| Acronym        | Full name                            | What it analyzes                | When                |
|----------------|--------------------------------------|---------------------------------|---------------------|
| SAST           | Static Application Security Testing  | Source code without running     | PR, CI build        |
| DAST           | Dynamic Application Security Testing | Running application (black-box) | Staging, pre-prod   |
| SCA            | Software Composition Analysis        | Dependencies, licenses, CVEs    | CI, daily scheduled |
| IAST           | Interactive AST                      | Instrumented runtime + code     | Optional hybrid     |
| Container scan | Image/filesystem CVE scan            | OS packages, layers             | After docker build  |
| IaC scan       | Infrastructure as Code               | Terraform, K8s manifests        | PR, CI              |

No single tool covers all—**defense in depth** layers findings with deduplication and prioritization.

## 54.2 50.2 SAST — static analysis

SAST parses source code and bytecode for vulnerability patterns: SQL injection, XSS, path traversal, hardcoded secrets, insecure crypto.

Popular tools:

| Tool                 | Languages                    | Notes              |
|----------------------|------------------------------|--------------------|
| Semgrep              | Multi-language, custom rules | Fast, CI-friendly  |
| SonarQube            | 30+ languages                | Quality + security |
| CodeQL               | Multi (GitHub)               | Semantic analysis  |
| Bandit               | Python                       | Python-specific    |
| gosec                | Go                           | Go-specific        |
| Checkmarx / Veracode | Enterprise                   | Broad coverage     |

Semgrep CI example:

```
- name: Semgrep SAST
 uses: returntocorp/semgrep-action@v1
 with:
 config: >-
 p/security-audit
 p/owasp-top-ten
```

Custom Semgrep rule:

```
rules:
- id: sql-injection-concat
 patterns:
 - pattern: |
 $QUERY = "... " + $USER_INPUT + "... "
 - pattern: execute($QUERY)
 message: Possible SQL injection via string concatenation
 severity: ERROR
 languages: [python, javascript]
```

SAST limitations:

- **False positives** — Requires tuning and baselines
- **No runtime context** — Cannot see config-dependent bugs
- **Late custom rules** — Novel vulns need rule updates

Triage workflow: suppress with justification ( `nosemgrep` comment + ticket), fix true positives within SLA.

## 54.3 50.3 DAST — dynamic testing

DAST probes **running** applications like an attacker—HTTP requests, form fuzzing, authentication bypass attempts.

Tools:

| Tool              | Type                           |
|-------------------|--------------------------------|
| <b>OWASP ZAP</b>  | Open-source proxy + automation |
| <b>Burp Suite</b> | Manual + automated (pro)       |
| <b>Nuclei</b>     | Template-based scanner         |
| <b>StackHawk</b>  | CI-integrated DAST             |
| <b>Acunetix</b>   | Commercial                     |

ZAP baseline scan in CI (against staging):

```
- name: OWASP ZAP Baseline
 uses: zaproxy/action-baseline@v0.12.0
 with:
 target: https://staging.example.com
 rules_file_name: .zap/rules.tsv
 cmd_options: -a
```

DAST requires:

- Deployed environment with realistic data (sanitized)
- Authentication handling (ZAP contexts, scripts)
- Scope limits—never scan production without approval

Finds: misconfigured headers, exposed admin panels, reflected XSS, CSRF gaps—issues SAST may miss.

## 54.4 50.4 SCA — dependency analysis

**SCA** inventories direct and transitive dependencies, matches against CVE databases (NVD, GitHub Advisory), flags license violations.

Tools:

| Tool                   | Integration                  |
|------------------------|------------------------------|
| Dependabot             | GitHub native PRs            |
| Renovate               | Multi-platform, configurable |
| Snyk                   | SCA + container + IaC        |
| Grype                  | Anchore ecosystem            |
| OWASP Dependency-Check | Jenkins, CLI                 |

GitHub Dependabot config:

```
.github/dependabot.yml
version: 2
updates:
 - package-ecosystem: npm
 directory: "/"
 schedule:
 interval: daily
 open-pull-requests-limit: 10
 groups:
 production-dependencies:
 patterns: ["*"]
 exclude-patterns: ["@types/*"]
```

Syft + Grype pipeline:

```
syft packages dir:. -o cyclonedx-json > sbom.json
grype sbom:sbom.json --fail-on critical
```

Prioritize by **exploitability** (EPSS score), **reachability** (is vulnerable code path used?), and **severity**—not all CVEs are exploitable in your context.

## 54.5 50.5 Container and image scanning

Container images bundle OS packages and application dependencies—both need scanning.

| Scanner             | Focus                                  |
|---------------------|----------------------------------------|
| Trivy               | OS + language deps, misconfig, secrets |
| Grype               | CVE matching via SBOM                  |
| Clair               | Layer-based analysis                   |
| Snyk Container      | Integrated platform                    |
| Amazon ECR scanning | Native AWS                             |

Trivy in GitHub Actions:

```
- name: Build image
 run: docker build -t myapp:${{ github.sha }} .

- name: Trivy scan
 uses: aquasecurity/trivy-action@0.24.0
 with:
 image-ref: myapp:${{ github.sha }}
 format: sarif
 output: trivy-results.sarif
 severity: CRITICAL,HIGH
 exit-code: 1

- name: Upload SARIF
```

```
uses: github/codeql-action/upload-sarif@v3
with:
 sarif_file: trivy-results.sarif
```

Scan **base images** regularly—pin to digest, not `:latest`. Rebuild golden images on CVE patches.

Distroless/minimal bases reduce attack surface and scan noise.

## 54.6 50.6 IaC and Kubernetes scanning

| Tool                 | Targets                                    |
|----------------------|--------------------------------------------|
| Checkov              | Terraform, CloudFormation, K8s, Dockerfile |
| tfsec / Trivy config | Terraform                                  |
| kube-score           | Kubernetes best practices                  |
| Polaris              | K8s policy validation                      |
| Kubescape            | NSA/CIS benchmarks                         |

Checkov example:

```
checkov -d terraform/ --framework terraform \
--check CKV_AWS_79,CKV_AWS_80 \
--soft-fail-on LOW
```

Common findings: public S3 buckets, overly permissive security groups, containers running as root, missing resource limits.

## 54.7 50.7 Unified vulnerability management

Centralize findings from SAST, DAST, SCA, container scans:

```
CI tools → SARIF / API → DefectDojo / GitHub Security / Snyk dashboard
|
Prioritization
SLA tracking
Jira tickets
```

**SARIF** (Static Analysis Results Interchange Format) standardizes results across tools for GitHub Advanced Security integration.

Remediation SLAs (example):

| Severity               | Fix SLA      |
|------------------------|--------------|
| Critical (exploitable) | 7 days       |
| High                   | 30 days      |
| Medium                 | 90 days      |
| Low                    | Next quarter |

## 54.8 50.8 Reducing noise

|  |  |
|--|--|
|  |  |
|--|--|

| Technique              | Benefit                       |
|------------------------|-------------------------------|
| Baseline accepted risk | Stop re-alerting known issues |
| Reachability analysis  | Focus on used vulnerable code |
| EPSS prioritization    | Exploit probability scoring   |
| Consolidate tools      | One SCA source of truth       |
| Fix upstream           | Patch base image vs per-app   |

Developer experience: surface findings in PR comments with fix guidance—not email-only reports ignored for weeks.

## 54.9 50.9 Chapter summary

- SAST analyzes source; DAST probes running apps; SCA tracks dependency CVEs; container scans cover image layers.
- Integrate scans in CI with severity gates; export SARIF for unified tracking.
- Combine tools for defense in depth; prioritize by exploitability and reachability.
- Maintain base images, dependency bots, and remediation SLAs for sustainable security.

### 54.10 Lab 50.1 — Full scan pipeline

1. Add Semgrep, Trivy (filesystem + image), and Dependabot to a sample Node.js + Docker project.
2. Introduce deliberate vulnerable dependency ( `npm install lodash@4.17.20` ); confirm SCA flags CVE.
3. Fix via upgrade; verify clean scan.

### 54.11 Lab 50.2 — DAST against staging

1. Deploy sample app to staging with intentional missing security header.
2. Run OWASP ZAP baseline; document finding.
3. Add header middleware; re-scan to confirm resolution.

### 54.12 Review questions

1. Compare SAST and DAST—what can each detect that the other cannot?
2. Why scan container images after build, not only application dependencies?
3. What is SARIF, and why use it?
4. How do EPSS and reachability improve SCA prioritization?
5. Name three common Checkov findings in Terraform.
6. What are typical false positive sources in SAST, and how handle them?
7. Why should DAST run against staging, not production?

*Continue: Chapter 51 — Secrets Management*

# 55 Chapter 51: Secrets Management — Vault, SOPS, External Secrets

DevOps Handbook — Part XI, Pages 1001–1020

## 55.1 51.1 The secrets problem

Applications need credentials—database passwords, API keys, TLS certificates, cloud IAM tokens. Storing them in Git, environment files on laptops, or plaintext Kubernetes Secrets violates basic security hygiene and compliance.

Requirements for secrets management:

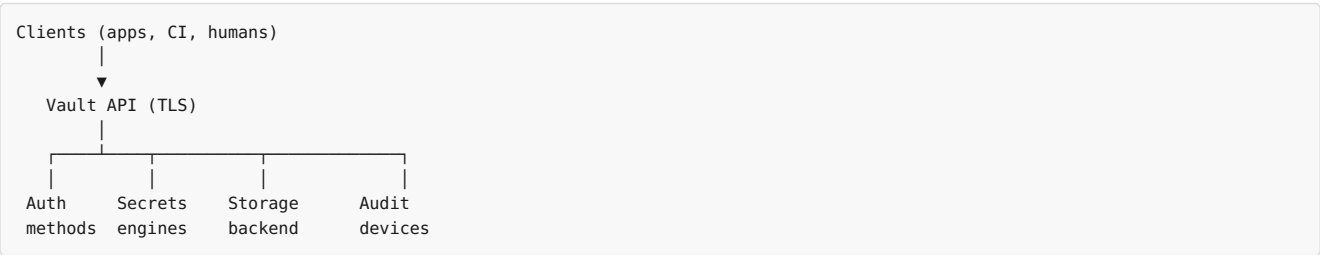
| Requirement           | Description                                      |
|-----------------------|--------------------------------------------------|
| Encryption at rest    | Secrets stored encrypted, not plaintext on disk  |
| Encryption in transit | TLS for all secret API calls                     |
| Access control        | Least privilege, audit who read what             |
| Rotation              | Periodic and emergency rotation without downtime |
| Dynamic secrets       | Short-lived credentials generated on demand      |
| Audit trail           | Immutable log of access events                   |

This chapter covers **HashiCorp Vault**, **Mozilla SOPS**, and **External Secrets Operator**—common patterns in cloud-native DevOps.

## 55.2 51.2 HashiCorp Vault architecture

**Vault** is a secrets engine and identity-aware broker.

Components:



| Concept         | Role                                                         |
|-----------------|--------------------------------------------------------------|
| Seal/Unseal     | Master key protects data; Shamir shards or auto-unseal (KMS) |
| Auth methods    | Kubernetes, AWS IAM, OIDC, AppRole, userpass                 |
| Secrets engines | KV, database, PKI, AWS, transit encryption                   |
| Policies        | HCL paths controlling read/write                             |
| Namespaces      | Multi-tenancy (Enterprise)                                   |

Vault CLI basics:



```

export VAULT_ADDR=https://vault.example.com:8200
vault login -method=oidc

Read static secret
vault kv get secret/data/myapp/database

Write secret
vault kv put secret/myapp/api key=NEW_VALUE

Generate dynamic DB creds (TTL 1h)
vault read database/creds/myapp-role

```

Dynamic database credentials: Vault creates temporary DB user, returns username/password, revokes on lease expiry—eliminating long-lived DB passwords in apps.

## 55.3 51.3 Vault on Kubernetes

**Vault Agent Injector** sidecar mounts secrets into pods:

```

apiVersion: apps/v1
kind: Deployment
metadata:
 name: api
 annotations:
 vault.hashicorp.com/agent-inject: "true"
 vault.hashicorp.com/role: "api"
 vault.hashicorp.com/agent-inject-secret-db: "secret/data/api/database"
 vault.hashicorp.com/agent-inject-template-db: |
 {{- with secret "secret/data/api/database" -}}
 DATABASE_URL=postgres://{{ .Data.data.username }}:{{ .Data.data.password }}@db:5432/app
 {{- end -}}
spec:
 template:
 spec:
 serviceAccountName: api
 containers:
 - name: api
 image: myapp:latest

```

Kubernetes auth binds ServiceAccount → Vault role → policy.

**Vault Secrets Operator** (newer) syncs Vault secrets to Kubernetes Secret objects natively.

Operational concerns:

- **HA deployment** — Raft integrated storage or Consul backend
- **Auto-unseal** — AWS KMS, GCP CKMS, Azure Key Vault
- **Backup** — Snapshot Raft; test restore
- **Upgrade path** — Read release notes; staged rollout

## 55.4 51.4 Mozilla SOPS — Git-friendly encryption

**SOPS** (Secrets OPERATIONs) encrypts YAML/JSON/env files **in Git**—only encrypted blobs committed; decryption keys from KMS/PGP/age.

Use case: team wants secrets versioned alongside config with PR review, without plaintext in repo.

Encrypted file example ( `secrets.enc.yaml` ):

```

db_password: ENC[AES256_GCM,data:xxx,iv:yyy,tag:zzz,type:str]
sops:
 kms:
 - arn: arn:aws:kms:us-east-1:123456789012:key/abc-123
 created_at: "2026-09-01T12:00:00Z"

```

```
enc: ...
lastmodified: "2026-09-01T12:00:00Z"
version: 3.8.1
```

Commands:

```
Encrypt
sops --encrypt --kms arn:aws:kms:us-east-1:123:key/abc secrets.yaml > secrets.enc.yaml

Edit in place (decrypt-edit-reencrypt)
sops secrets.enc.yaml

Decrypt to stdout for deploy
sops --decrypt secrets.enc.yaml > /tmp/secrets.yaml
```

CI integration:

```
- name: Decrypt secrets
 env:
 AWS_REGION: us-east-1
 run: |
 sops --decrypt deploy/secrets.enc.yaml > deploy/secrets.yaml
 kubectl apply -f deploy/
```

**age** keys for teams without cloud KMS:

```
age-keygen -o key.txt
sops --encrypt --age age1xxx secrets.yaml > secrets.enc.yaml
```

SOPS tradeoffs: secrets in Git history remain encrypted but **key compromise** exposes all; rotate keys periodically.

---

## 55.5 51.5 External Secrets Operator (ESO)

**External Secrets Operator** syncs secrets from external providers into Kubernetes Secrets—bridging Vault, AWS Secrets Manager, GCP Secret Manager, Azure Key Vault, Parameter Store.

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
 name: api-db-credentials
 namespace: production
spec:
 refreshInterval: 1h
 secretStoreRef:
 name: aws-secrets-manager
 kind: ClusterSecretStore
 target:
 name: api-db-credentials
 creationPolicy: Owner
 data:
 - secretKey: DATABASE_URL
 remoteRef:
 key: production/api/database
 property: url
```

ClusterSecretStore for AWS:

```
apiVersion: external-secrets.io/v1beta1
kind: ClusterSecretStore
metadata:
 name: aws-secrets-manager
spec:
 provider:
 aws:
 service: SecretsManager
 region: us-east-1
```

```

auth:
 jwt:
 serviceAccountRef:
 name: external-secrets
 namespace: external-secrets

```

Benefits:

- No secrets in Git
- Central rotation in AWS/Vault propagates to K8s on refresh interval
- IRSA (IAM Roles for Service Accounts) for authentication—no static AWS keys

## 55.6 51.6 Cloud-native secret managers

| Provider | Service                              | Integration                |
|----------|--------------------------------------|----------------------------|
| AWS      | Secrets Manager, SSM Parameter Store | ESO, Lambda rotation       |
| GCP      | Secret Manager                       | ESO, GKE workload identity |
| Azure    | Key Vault                            | ESO, managed identity      |

AWS Secrets Manager rotation Lambda template rotates RDS passwords automatically.

Compare Vault vs cloud-native:

| Factor           | Vault                  | Cloud SM           |
|------------------|------------------------|--------------------|
| Multi-cloud      | Yes                    | Per-cloud          |
| Dynamic secrets  | Rich engines           | Limited            |
| Operational load | Higher                 | Managed            |
| Cost             | Self-host + HCP option | Per-secret pricing |

Many orgs: Vault for dynamic/complex; cloud SM for simple static secrets.

## 55.7 51.7 CI/CD secrets patterns

| Anti-pattern                            | Better approach                  |
|-----------------------------------------|----------------------------------|
| Long-lived cloud keys in GitHub Secrets | OIDC federation to IAM role      |
| Same secret all environments            | Separate secrets per env         |
| Manual rotation                         | Automated rotation + ESO refresh |
| Echo secrets in CI logs                 | Mask variables; use secret refs  |

GitHub OIDC → AWS (no static key):

```

permissions:
 id-token: write
steps:
 - uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: arn:aws:iam::123456789012:role/GHA-Deploy
 aws-region: us-east-1
 - run: aws secretsmanager get-secret-value --secret-id prod/api

```

Short-lived tokens reduce blast radius of CI compromise.

---

## 55.8 51.8 Rotation and emergency response

Rotation checklist:

1. Generate new secret in manager
2. Dual-write period (app accepts old + new) if applicable
3. Update consumers via ESO refresh or redeploy
4. Revoke old secret after validation
5. Audit access logs for anomalies

Emergency: leaked secret → revoke immediately, rotate, scan Git history, review Vault/AWS audit logs for unauthorized access.

---

## 55.9 51.9 Chapter summary

- Vault provides dynamic secrets, fine-grained policies, and audit for enterprise secret management.
  - SOPS enables encrypted secrets in Git with KMS/age keys for reviewable configuration.
  - External Secrets Operator syncs cloud/Vault secrets into Kubernetes without plaintext in repos.
  - Prefer OIDC and short-lived credentials in CI; automate rotation and maintain audit trails.
- 

### 55.10 Lab 51.1 — SOPS with age

1. Generate age key pair; encrypt a YAML file with DB credentials.
  2. Commit encrypted file to Git; verify plaintext never appears in history.
  3. Decrypt in CI script simulating deploy.
- 

### 55.11 Lab 51.2 — External Secrets on Kubernetes

1. Store secret in AWS Secrets Manager (or LocalStack for local test).
  2. Install ESO; configure ClusterSecretStore and ExternalSecret.
  3. Mount synced K8s Secret in pod; verify app reads value.
- 

## 55.12 Review questions

1. Compare static secrets in K8s Secrets vs Vault dynamic database credentials.
  2. When is SOPS appropriate vs External Secrets Operator?
  3. What is Vault seal/unseal, and why use auto-unseal with KMS?
  4. How does IRSA enable ESO to access AWS Secrets Manager without static keys?
  5. Describe a safe secret rotation procedure for a database password.
  6. Why use OIDC in GitHub Actions instead of storing AWS access keys?
  7. What audit capabilities should a secrets platform provide?
- 

*Continue: Chapter 52 — Supply Chain, SBOM, Sigstore, SLSA*

# 56 Chapter 52: Supply Chain Security — SBOM, Sigstore, SLSA

DevOps Handbook — Part XI, Pages 1021–1040

## 56.1 52.1 Software supply chain threats

Modern software is assembled from thousands of components—open-source libraries, base images, CI plugins, Terraform modules. Attackers target the **supply chain** because compromising one dependency affects thousands of downstream users.

Notable incidents:

| Incident                  | Mechanism                                    |
|---------------------------|----------------------------------------------|
| SolarWinds (2020)         | Build system compromise, trojanized updates  |
| Log4Shell (2021)          | Ubiquitous vulnerable dependency             |
| Codecov (2021)            | CI script compromise exfiltrating secrets    |
| npm package typosquatting | Malicious packages mimicking popular names   |
| 3CX (2023)                | Signed malware in legitimate software update |

**Supply chain security** verifies **provenance** (where artifacts came from), **integrity** (unchanged since build), and **trust** (who built and signed).

## 56.2 52.2 SBOM — Software Bill of Materials

An **SBOM** is a machine-readable inventory of components in a software artifact—like nutrition labels for code.

| Format    | Maintainer                    |
|-----------|-------------------------------|
| CycloneDX | OWASP / ECMA standard         |
| SPDX      | Linux Foundation ISO standard |

SBOM enables:

- Rapid CVE response (“are we affected by CVE-2024-XXXX?”)
- License compliance audits
- Vendor risk assessment
- Regulatory requirements (US EO 14028, EU CRA)

Generate SBOM with **Syft**:

```
Filesystem
syft dir:. -o cyclonedx-json > sbom.json

Container image
syft registry.example.com/myapp:v1.2.3 -o spdx-json > sbom.spdx.json

Attach to image attestation
syft attest registry.example.com/myapp:v1.2.3 -o cyclonedx-json
```

CI integration:

```

- name: Generate SBOM
 uses: anchore/sbom-action@v0
 with:
 image: myapp:${{ github.sha }}
 format: cyclonedx-json
 output-file: sbom.cdx.json

- name: Upload SBOM artifact
 uses: actions/upload-artifact@v4
 with:
 name: sbom
 path: sbom.cdx.json

```

Store SBOMs alongside artifacts in registry (OCI referrer) or artifact repository—queryable when new CVEs emerge.

**VEX** (Vulnerability Exploitability eXchange) documents why listed CVEs are not exploitable in context—reduces false panic.

## 56.3 52.3 SLSA — supply chain levels

**SLSA** (Supply-chain Levels for Software Artifacts) defines progressive security levels for build pipelines.

| Level         | Requirements (summary)                              |
|---------------|-----------------------------------------------------|
| <b>SLSA 1</b> | Provenance exists (build documented)                |
| <b>SLSA 2</b> | Hosted build service, signed provenance             |
| <b>SLSA 3</b> | Hardened build platform, non-falsifiable provenance |
| <b>SLSA 4</b> | Two-person review, reproducible builds              |

Provenance attestation (conceptual):

```

{
 "_type": "https://in-toto.io/Statement/v1",
 "subject": [{ "name": "myapp:v1.2.3", "digest": { "sha256": "abc..." } }],
 "predicateType": "https://slsa.dev/provenance/v1",
 "predicate": {
 "buildDefinition": {
 "buildType": "https://github.com/Attestations/GitHubActionsWorkflow@v1",
 "externalParameters": {
 "workflow": { "ref": "refs/heads/main", "repository": "https://github.com/org/myapp" }
 }
 },
 "runDetails": {
 "builder": { "id": "https://github.com/org/myapp/.github/workflows/build.yml@refs/heads/main" },
 "metadata": { "invocationId": "1234567890" }
 }
 }
}

```

GitHub Actions can generate SLSA provenance with **slsa-github-generator** workflow.

## 56.4 52.4 Sigstore and cosign

**Sigstore** provides free, automated code signing and verification—**cosign** signs container images and blobs; **Fulcio** issues short-lived certificates bound to OIDC identity; **Rekor** is transparency log.

Sign image after build:

```

Keyless signing (OIDC identity from CI)
cosign sign registry.example.com/myapp:v1.2.3

```

```
Verify
cosign verify registry.example.com/myapp:v1.2.3 \
 --certificate-identity-regexp="https://github.com/myorg/myapp/.github/workflows/*.*" \
 --certificate-oidc-issuer="https://token.actions.githubusercontent.com"
```

GitHub Actions:

```
- name: Install cosign
 uses: sigstore/cosign-installer@v3

- name: Sign image
 run: cosign sign --yes registry.example.com/myapp:${{ github.sha }}
 env:
 COSIGN_EXPERIMENTAL: "true"
```

**Kubernetes admission** — Verify signatures before deploy:

```
Policy Controller / Kyverno verifyImages
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
 name: verify-signed-images
spec:
 validationFailureAction: Enforce
 rules:
 - name: verify-cosign-signature
 match:
 any:
 - resources:
 kinds: [Pod]
 verifyImages:
 - imageReferences:
 - "registry.example.com/myapp:*"
 attestors:
 - count: 1
 entries:
 - keyless:
 subject: "https://github.com/myorg/myapp/.github/workflows/*"
 issuer: "https://token.actions.githubusercontent.com"
```

Only signed images from trusted CI workflows deploy to production.

## 56.5 52.5 Dependency and build hardening

| Control                       | Implementation                                   |
|-------------------------------|--------------------------------------------------|
| <b>Pin dependencies</b>       | Lock files, digest-pinned base images            |
| <b>Verify checksums</b>       | npm/yarn integrity, Go sum                       |
| <b>Private registry proxy</b> | Pull-through cache, vulnerability gate           |
| <b>Minimal CI permissions</b> | permissions: contents: read default              |
| <b>Signed commits/tags</b>    | GPG or Sigstore gitsign                          |
| <b>Branch protection</b>      | Required reviews, status checks                  |
| <b>Hermetic builds</b>        | Reproducible, no network in compile (Bazel, Nix) |

**Dependabot/Renovate** with grouped updates and CI validation before merge.

Block unpinned actions in CI:

```
Policy: actions must use SHA pin
Semgrep or custom linter on workflow files
uses: actions/checkout@b4ffde65f46336ab88eb53be808477a3936bae11 # v4.1.1
```

---

## 56.6 52.6 in-toto and attestations

**in-toto** framework chains attestations through supply chain steps: design → source → build → test → deploy.

Each step produces signed metadata; final deployment verifies entire chain.

**GitHub Artifact Attestations** (2024+) attach build provenance to artifacts in GitHub UI and API.

OCI **referrers** attach SBOM and signatures to container images in registry—Cosign, ORAS CLI.

```
oras attach registry.example.com/myapp:v1.2.3 \
 --artifact-type application/vnd.cyclonedx+json \
 --file sbom.json
```

---

## 56.7 52.7 Organizational supply chain program

1. **Inventory** — SBOM for every production artifact
2. **Sign** — cosign keyless in CI for all release images
3. **Verify** — Admission policy in K8s clusters
4. **Monitor** — Alert on new CVEs affecting SBOM components
5. **Respond** — Playbook linking CVE → affected services via SBOM query
6. **Vendor** — Require SBOM + SLSA level from third-party software

Metrics:

- % artifacts with SBOM + signature
- Mean time to identify affected services for new CVE
- % deployments blocked by unsigned image policy (should be 0 in prod bypass)

---

## 56.8 52.8 Chapter summary

- SBOMs inventory software components for CVE response and compliance.
- SLSA defines progressive build hardening levels with provenance attestations.
- Sigstore/cosign enables keyless signing and Kubernetes verification policies.
- Pin dependencies, harden CI, and maintain organizational SBOM + signing program.

---

## 56.9 Lab 52.1 — SBOM generation and CVE lookup

1. Build Docker image for sample app; generate CycloneDX SBOM with Syft.
2. Scan SBOM with Gripe; document findings.
3. Upgrade vulnerable package; regenerate SBOM; confirm clean scan.

---

## 56.10 Lab 52.2 — Sign and verify with cosign

1. Build and push image to registry (GHCR or local).
  2. Sign with cosign keyless in GitHub Actions (or local key).
  3. Configure Kyverno or conftest policy rejecting unsigned images.
  4. Attempt deploy unsigned image; confirm rejection.
-



## 56.11 Review questions

1. What is an SBOM, and why generate it for every release?
2. Compare CycloneDX and SPDX formats.
3. Explain SLSA Level 2 vs Level 3 requirements at high level.
4. How does cosign keyless signing bind identity to CI workflow?
5. What is VEX, and when use it?
6. Name three supply chain attacks and their mechanisms.
7. How do Kubernetes admission policies enforce signed images?

---

*Continue: Chapter 53 — Runtime Security*

# 57 Chapter 53: Runtime Security — Falco and OPA Gatekeeper

DevOps Handbook — Part XI, Pages 1041–1060

## 57.1 53.1 Defense beyond the pipeline

Shift-left security catches vulnerabilities before deploy—but **runtime** threats remain: zero-days, insider abuse, container escapes, cryptomining, lateral movement after breach. **Runtime security** detects and responds to malicious behavior in executing systems.

Two complementary approaches:

| Approach           | Mechanism                                  | Example tools                                   |
|--------------------|--------------------------------------------|-------------------------------------------------|
| Threat detection   | Monitor syscalls, K8s audit events         | Falco, Sysdig, Aqua                             |
| Policy enforcement | Block non-compliant resources at admission | OPA Gatekeeper, Kyverno, Pod Security Admission |

Detection alerts; admission prevents. Use both.

## 57.2 53.2 Falco — cloud-native runtime detection

**Falco** (CNCF) detects unexpected behavior via **kernel events** (syscalls via eBPF or kernel module) and **Kubernetes audit logs**.

Detection examples:

- Shell spawned in container
- Unexpected outbound connection
- Sensitive file read ( `/etc/shadow` )
- Privilege escalation attempt
- Write below `/etc` or `/usr/bin`

Architecture:



Install on Kubernetes (Helm):

```
helm repo add falcosecurity https://falcosecurity.github.io/charts
helm install falco falcosecurity/falco \
 --namespace falco --create-namespace \
 --set driver.kind=ebpf
```

Custom rule:

```
/etc/falco/rules.d/custom-rules.yaml
- rule: Terminal shell in container
 desc: Detect shell spawned in running container
 condition: >
 spawned_process and container and
 proc.name in (bash, sh, zsh) and
 not proc.pname in (bash, sh, docker-init, tini)
```

```

output: >
 Shell spawned in container
 (user=%user.name container=%container.name
 image=%container.image.repository cmdline=%proc.cmdline)
priority: WARNING
tags: [container, shell, mitre_execution]

- rule: Outbound connection to suspicious port
desc: Unexpected outbound connection on non-standard port
condition: >
 outbound and container and
 fd.sport != 443 and fd.sport != 80 and
 not fd.sip in (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16)
output: Suspicious outbound (container=%container.name connection=%fd.name)
priority: NOTICE

```

Tune rules to reduce false positives—baseline normal behavior in staging first.

Response integration via **Falcosidekick**:

```

config:
slack:
 webhookurl: "https://hooks.slack.com/..."
 minimumpriority: warning
pagerduty:
 routingkey: SECRET
 minimumpriority: error

```

## 57.3 53.3 Falco response actions

Detection alone is insufficient—define response playbooks:

| Alert                   | Response                                       |
|-------------------------|------------------------------------------------|
| Shell in prod container | Kill pod, page on-call, capture forensics      |
| Cryptomining outbound   | NetworkPolicy block, isolate node              |
| Sensitive file access   | Audit user/service account, revoke credentials |

**Falco Response Engine** (evolving) can trigger Kubernetes actions (delete pod, label node) via webhooks—use cautiously with human approval for destructive actions.

Combine with **NetworkPolicies** and **seccomp/AppArmor** profiles to limit blast radius when detection triggers.

## 57.4 53.4 OPA and Gatekeeper

**Open Policy Agent (OPA)** evaluates policy decisions via Rego language. **Gatekeeper** is OPA’s Kubernetes admission controller—validates and mutates resources at create/update.

```

kubectrl apply → API Server → Gatekeeper (OPA) → Allow/Deny
 |
 ConstraintTemplate
 Constraint

```

Install Gatekeeper:

```
kubectrl apply -f https://raw.githubusercontent.com/open-policy-agent/gatekeeper/v3.14.0/deploy/gatekeeper.yaml
```

ConstraintTemplate (require labels):

```
apiVersion: templates.gatekeeper.sh/v1
```

```

kind: ConstraintTemplate
metadata:
 name: k8srequiredlabels
spec:
 crd:
 spec:
 names:
 kind: K8sRequiredLabels
 validation:
 openAPIV3Schema:
 type: object
 properties:
 labels:
 type: array
 items:
 type: string
 targets:
 - target: admission.k8s.gatekeeper.sh
 rego: |
 package k8srequiredlabels
 violation[{"msg": msg}] {
 required := input.parameters.labels
 provided := input.review.object.metadata.labels
 missing := required[_]
 not provided[missing]
 msg := sprintf("Missing required label: %v", [missing])
 }

```

Constraint (enforce):

```

apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sRequiredLabels
metadata:
 name: require-team-label
spec:
 match:
 kinds:
 - apiGroups: [""]
 kinds: ["Pod"]
 namespaces: ["production"]
 parameters:
 labels: ["team", "app"]

```

Block privileged containers:

```

package k8spsprivileged

violation[{"msg": msg}] {
 input.review.object.spec.containers[_].securityContext.privileged
 msg := "Privileged containers are not allowed"
}

```

Gatekeeper **mutation** (assign labels, defaults) via AssignMetadata, Assign mutations.

## 57.5 53.5 Policy libraries and Pod Security

Use community policy libraries:

- **Gatekeeper library** — CIS benchmarks, PSA equivalents
- **Kyverno policies** — Alternative YAML-native policy engine
- **Pod Security Admission (PSA)** — Built-in K8s 1.25+ ( restricted , baseline , privileged )

Namespace labels for PSA:

```

apiVersion: v1
kind: Namespace
metadata:
 name: production
 labels:
 pod-security.kubernetes.io/enforce: restricted

```

Compare:

| Engine         | Language | Strength                      |
|----------------|----------|-------------------------------|
| Gatekeeper/OPA | Rego     | Flexible, complex policies    |
| Kyverno        | YAML     | K8s-native, easier onboarding |
| PSA            | Built-in | Baseline pod hardening        |

Many orgs: PSA for baseline + Gatekeeper/Kyverno for org-specific rules.

## 57.6 53.6 Runtime security program

Layered Kubernetes security:

1. Pod Security Admission (restricted)
2. Gatekeeper/Kyverno admission policies
3. NetworkPolicies (default deny)
4. Falco runtime detection
5. Audit logging → SIEM
6. Regular policy review and game days

Metrics:

- Admission deny rate (misconfigurations vs attacks)
- Falco alert volume and true positive rate
- Time to remediate policy violations

## 57.7 53.7 Chapter summary

- Falco detects anomalous runtime behavior via kernel events and K8s audit with customizable rules.
- OPA Gatekeeper enforces Rego policies at Kubernetes admission—preventing non-compliant workloads.
- Combine admission control (prevent) with runtime detection (detect/respond).
- Use PSA, NetworkPolicies, and policy libraries for defense in depth.

## 57.8 Lab 53.1 — Falco shell detection

1. Install Falco on test cluster (eBPF driver).
2. Deploy nginx pod; exec bash inside container.
3. Confirm Falco alert in logs or Falcosidekick webhook.
4. Add exception rule for approved debug namespace.

## 57.9 Lab 53.2 — Gatekeeper privileged block

1. Install Gatekeeper; apply ConstraintTemplate blocking privileged pods.
2. Attempt `kubectl run` with `--privileged`; confirm denial.
3. Deploy compliant pod; confirm success.

## 57.10 Review questions

1. Distinguish runtime detection from admission control with examples.
2. How does Falco obtain visibility into container behavior?
3. What are ConstraintTemplate and Constraint in Gatekeeper?
4. Write Rego logic conceptually to deny containers running as root.
5. Compare OPA Gatekeeper and Kyverno for Kubernetes policy.
6. How do Pod Security Admission levels differ?
7. What tuning is needed before Falco alerts page on-call?

---

*Continue: Chapter 54 — Compliance Automation*

# 58 Chapter 54: Compliance Automation and Audit Trails

DevOps Handbook — Part XI, Pages 1061–1080

## 58.1 54.1 Compliance in continuous delivery

Regulated industries—finance, healthcare, government—must demonstrate **controls** are designed and operating effectively: SOC 2, ISO 27001, PCI DSS, HIPAA, FedRAMP, GDPR. Traditional compliance meant annual audits with manual evidence collection. **Compliance automation** embeds controls in pipelines and infrastructure, producing **continuous evidence**.

DevOps alignment:

| Traditional compliance   | Automated compliance              |
|--------------------------|-----------------------------------|
| Annual spreadsheet audit | Continuous control monitoring     |
| Manual change tickets    | Git history + approved PRs        |
| Screenshot of firewall   | IaC + drift detection             |
| Quarterly access review  | IAM analytics + automated reports |

Goal: **audit-ready by default**—evidence generated as a side effect of normal operations.

## 58.2 54.2 Control frameworks mapping

Map technical controls to framework requirements:

| Framework domain         | Technical implementation                   |
|--------------------------|--------------------------------------------|
| Access control           | IAM, RBAC, MFA, SSO                        |
| Change management        | Git PR approval, CI gates, deployment logs |
| Encryption               | TLS, KMS, encrypted volumes                |
| Logging/monitoring       | Centralized logs, immutable audit trails   |
| Vulnerability management | SCA/SAST gates, patch SLAs                 |
| Backup/DR                | Automated backups, tested restore          |
| Incident response        | On-call, postmortems, retention            |

**SOC 2 Trust Service Criteria** (Security, Availability, Processing Integrity, Confidentiality, Privacy) map to pipeline and platform controls. **PCI DSS** requires segmentation, logging, vulnerability scanning for cardholder data environments.

Maintain a **control matrix** spreadsheet/database linking control ID → implementation → evidence source → owner.

## 58.3 54.3 Audit trails — what to log

Immutable audit logs for:

| Event type                   | Examples                           |
|------------------------------|------------------------------------|
| <b>Authentication</b>        | Login, MFA, failed attempts        |
| <b>Authorization</b>         | IAM policy change, role assignment |
| <b>Data access</b>           | Admin read of customer data        |
| <b>Infrastructure change</b> | Terraform apply, kubectl apply     |
| <b>Deployment</b>            | Who deployed what commit when      |
| <b>Secret access</b>         | Vault/AWS Secrets Manager read     |
| <b>Security findings</b>     | Scan results, waiver approvals     |

Kubernetes audit policy:

```
apiVersion: audit.k8s.io/v1
kind: Policy
rules:
 - level: Metadata
 omitStages: [RequestReceived]
 - level: RequestResponse
 verbs: ["create", "update", "patch", "delete"]
 resources:
 - group: ""
 resources: ["secrets", "configmaps"]
 - level: RequestResponse
 verbs: ["create", "update", "patch", "delete"]
 resources:
 - group: "rbac.authorization.k8s.io"
 resources: ["roles", "rolebindings", "clusterroles", "clusterrolebindings"]
```

Ship audit logs to SIEM (Elastic Security, Splunk) with **WORM** storage (S3 Object Lock) for tamper evidence.

CloudTrail (AWS) / Cloud Audit Logs (GCP) / Activity Log (Azure)—enable organization trail, log integrity validation.

## 58.4 54.4 Policy as Code for compliance

Encode compliance rules as executable policy:

**Checkov** CIS benchmarks:

```
checkov -d terraform/ --framework terraform --check CKV_AWS_* \
 --download-external-modules true \
 --output sarif --output-file-path checkov.sarif
```

**OPA/Conftest** custom org policies:

```
require S3 bucket encryption
package terraform.s3

deny[msg] {
 resource := input.resource.aws_s3_bucket[name]
 not input.resource.aws_s3_bucket_server_side_encryption_configuration[name]
 msg := sprintf("S3 bucket %s missing encryption config", [name])
}
```

**Cloud Custodian** — AWS/GCP/Azure compliance automation:

```
policies:
 - name: s3-require-encryption
 resource: aws.s3
 filters:
 - type: bucket-encryption
 state: false
 actions:
```



```
- type: notify
- type: mark-for-op
op: encrypt
```

Run in CI (prevent) and scheduled scans (detect drift).

## 58.5 54.5 Continuous control monitoring

CCM tools aggregate evidence:

| Tool        | Function                         |
|-------------|----------------------------------|
| Vanta       | Automated SOC 2 evidence         |
| Drata       | Continuous compliance monitoring |
| Secureframe | Control tracking                 |
| OpenSCAP    | NIST/CIS scanning                |
| InSpec      | Compliance profiles as code      |

Custom dashboard example metrics:

- % production resources passing Checkov CIS
- Mean patch time for critical CVEs
- % employees with MFA enabled
- PR approval compliance rate (no self-merge to main)

Automated evidence collection:

```
Weekly compliance report job
- name: Collect evidence
 run: |
 checkov -d infra/ -o json > evidence/checkov-$(date +%F).json
 aws iam generate-credential-report
 aws iam get-credential-report --query Content --output text | base64 -d > evidence/iam-$(date +%F).csv
 aws s3 cp evidence/ s3://compliance-evidence-bucket/$(date +%Y/%m/%d)/ --recursive
```

## 58.6 54.6 Change management automation

Auditors ask: “How do you control production changes?”

Automated answer:

1. All changes via **Git PR** with required reviewers
2. Branch protection: status checks (CI, security scans) must pass
3. Deployment linked to commit SHA in CI logs and GitHub Deployments API
4. **Immutable infrastructure** — no SSH manual edits
5. Emergency break-glass procedure documented with retroactive PR

Terraform Cloud / Atlantis audit trail:

```
PR #1234 → plan posted → approved → apply → state updated → audit log entry
```

Separation of duties: developer proposes (PR), different person approves, CI applies—SOD for SOC 2.

## 58.7 54.7 Data privacy and retention

GDPR/CCPA requirements:

- **Data inventory** — Map PII flows (SBOM-like for data)
- **Retention policies** — ILM on logs (Chapter 45)
- **Right to erasure** — Procedures in runbooks
- **DPIA** — Data Protection Impact Assessment for new features

Log redaction at ingest—never store full credit card numbers in application logs.

Retention schedule:

| Data type        | Retention          | Rationale       |
|------------------|--------------------|-----------------|
| Application logs | 90 days            | Operations      |
| Audit logs       | 7 years            | Regulatory      |
| CI artifacts     | 1 year             | Reproducibility |
| Backups          | Per RPO/RTO policy | DR              |

---

## 58.8 54.8 Audit preparation workflow

Before external audit:

1. **Control matrix review** — Update implementations
2. **Evidence spot check** — Sample 30 days of deployment logs, access reviews
3. **Exception documentation** — Risk acceptance for waived findings
4. **Personnel interviews** — Engineers explain actual process matches documentation
5. **Pen test / vuln scan reports** — Current within policy window

During audit: provide read-only access to evidence repository—not production systems.

Post-audit: remediate findings, update automation to prevent recurrence.

---

## 58.9 54.9 Chapter summary

- Compliance automation maps framework controls to pipeline, IaC, and platform implementations.
  - Immutable audit trails (K8s audit, CloudTrail, Git, Vault logs) provide continuous evidence.
  - Policy-as-code (Checkov, OPA, Cloud Custodian) prevents and detects drift.
  - CCM platforms and scheduled evidence jobs reduce audit scramble; maintain control matrix ownership.
- 

## 58.10 Lab 54.1 — Kubernetes audit logging

1. Enable audit policy on test cluster logging RequestResponse for Secrets and RBAC.
  2. Create and delete a Secret; query audit logs for event chain.
  3. Forward logs to Elasticsearch; create Kibana dashboard of admin actions.
- 

## 58.11 Lab 54.2 — Compliance evidence pipeline

1. Run Checkov against Terraform sample with intentional misconfigurations.
2. Export SARIF/JSON to `evidence/` directory with timestamp.
3. Document control matrix entry linking CIS benchmark to Checkov check ID and evidence file.

---

## 58.12 Review questions

1. How does Git-based change management satisfy audit change control requirements?
2. What events must Kubernetes audit logs capture for security compliance?
3. Compare continuous compliance monitoring vs annual audit preparation.
4. What is separation of duties in CI/CD, and why implement it?
5. How do ILM policies support GDPR retention requirements?
6. Name three tools for policy-as-code compliance scanning.
7. What evidence would you provide for a “vulnerability management” control?

---

*Continue: Chapter 55 — Platform Engineering*

# 59 Chapter 55: Platform Engineering and Internal Developer Platforms

DevOps Handbook — Part XII, Pages 1081–1100

## 59.1 55.1 From DevOps to platform engineering

As organizations scale, each product team rebuilding CI templates, Kubernetes manifests, and observability stacks creates **cognitive load** and inconsistency. **Platform engineering** builds an **Internal Developer Platform (IDP)**—a curated set of tools, templates, and self-service capabilities that enable developers to ship software efficiently without becoming infrastructure experts.

Platform team mission (CNCF definition):

*Treat platform as a product; optimize for developer experience and organizational outcomes.*

Platform engineering vs related roles:

| Role                 | Focus                                     |
|----------------------|-------------------------------------------|
| DevOps               | Culture + automation bridging dev and ops |
| SRE                  | Reliability, SLOs, toil reduction         |
| Platform engineering | Self-service golden paths, IDP product    |
| Infrastructure team  | Raw compute, network, storage             |

Platform teams succeed when developers **choose** the platform because it accelerates work—not because mandates force compliance.

## 59.2 55.2 Internal Developer Platform components

Typical IDP layers:

|                                                                                             |                   |                     |                   |
|---------------------------------------------------------------------------------------------|-------------------|---------------------|-------------------|
| Developer Portal (Backstage, Cortex, Port)<br>Service catalog, docs, scaffolder, scorecards |                   |                     |                   |
| Golden paths: deploy web app, add cron job, expose API                                      |                   |                     |                   |
| CI/CD templates                                                                             | Runtime (K8s/EKS) | Observability stack | Security scanning |
| Infrastructure: Terraform modules, networking, IAM                                          |                   |                     |                   |

Key capabilities:

| Capability               | Purpose                                     |
|--------------------------|---------------------------------------------|
| Service catalog          | Discoverable services, owners, dependencies |
| Software templates       | Cookie-cutter repo + CI + infra             |
| Environment provisioning | Dev/staging/prod namespaces or accounts     |
| Deployment abstraction   | GitOps, Helm charts, deploy buttons         |
| Secrets integration      | Vault/ESO wired by default                  |

|                             |                                       |
|-----------------------------|---------------------------------------|
| <b>Observability bundle</b> | Dashboards, alerts pre-configured     |
| <b>Documentation</b>        | TechDocs, runbooks linked from portal |

## 59.3 55.3 Backstage — reference IDP portal

**Backstage** (CNCF, Spotify-origin) is the most adopted open-source developer portal.

Features:

- **Software Catalog** — YAML entity descriptors ( `catalog-info.yaml` )
- **Software Templates (Scaffolder)** — Create repos from templates
- **TechDocs** — Docs-as-code in repo
- **Plugins** — Kubernetes, Argo CD, Grafana, CI visibility

Catalog entity:

```
catalog-info.yaml
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
 name: order-api
 description: Order processing microservice
 annotations:
 github.com/project-slug: myorg/order-api
 backstage.io/techdocs-ref: dir:.
 tags:
 - python
 - kubernetes
spec:
 type: service
 lifecycle: production
 owner: team-checkout
 system: commerce
 dependsOn:
 - resource:default/postgres-orders
 providesApis:
 - order-api-rest
```

Scaffolder template (simplified):

```
apiVersion: scaffolder.backstage.io/v1beta3
kind: Template
metadata:
 name: python-microservice
 title: Python Microservice on EKS
spec:
 parameters:
 - title: Service info
 required: [name, owner]
 properties:
 name:
 type: string
 owner:
 type: string
 ui:field: OwnerPicker
 steps:
 - id: fetch
 action: fetch:template
 input:
 url: ./skeleton
 values:
 name: ${ parameters.name }
 owner: ${ parameters.owner }
 - id: publish
 action: publish:github
 input:
 repoUrl: github.com?repo=${ parameters.name }&owner=myorg
 - id: register
 action: catalog:register
 input:
```

```
repoContentsUrl: ${ steps.publish.output.repoContentsUrl }
```

## 59.4 55.4 Golden paths and paved roads

**Golden path** — Opinionated, supported workflow for common tasks:

“Create a REST API → get CI, EKS deployment, Prometheus metrics, and security scans automatically.”

**Paved road vs unpaved road:**

| Paved (golden)               | Unpaved (escape hatch)     |
|------------------------------|----------------------------|
| Supported by platform team   | Team owns fully            |
| Fast onboarding              | Flexibility for edge cases |
| Security/compliance built-in | Custom review required     |
| SLAs on platform uptime      | Best effort                |

Document escape hatches—forcing all workloads through inadequate abstractions drives shadow IT.

Measure golden path adoption: % new services created via scaffolder vs manual.

## 59.5 55.5 Platform as a product

Apply product management to platform:

| Product practice   | Platform application                    |
|--------------------|-----------------------------------------|
| User research      | Developer interviews, surveys           |
| Roadmap            | Quarterly platform priorities           |
| Metrics            | DORA, time-to-first-deploy, CSAT        |
| SLAs               | IDP availability, template freshness    |
| Support            | #platform-support Slack, office hours   |
| Deprecation policy | Notice before breaking template changes |

**Team Topologies** (Part I): platform team is an **enabling team** or **platform team** interacting via **X-as-a-Service** with stream-aligned product teams.

Avoid **Ivory tower platform**—embed platform engineers in product teams periodically; consume your own golden paths (“eat your own dog food”).

## 59.6 55.6 Infrastructure abstractions

Platform hides complexity behind APIs:

| Raw infrastructure | Platform abstraction                |
|--------------------|-------------------------------------|
| Terraform modules  | “Create environment” button         |
| Kubernetes YAML    | Helm chart with values              |
| IAM policies       | Pre-approved roles per service type |
|                    |                                     |

|                  |                                    |
|------------------|------------------------------------|
| Network policies | Automatic based on service catalog |
|------------------|------------------------------------|

**Humanitec, Crossplane, Kratix** — platform orchestration patterns composing infra resources from service definitions.

Crossplane Composite Resource:

```

apiVersion: platform.example.com/v1alpha1
kind: XServiceEnvironment
metadata:
 name: order-api-staging
spec:
 serviceName: order-api
 environment: staging
 region: us-east-1
 # Platform controller provisions RDS, EKS namespace, secrets, DNS

```

Developers declare intent; platform controllers reconcile infrastructure.

## 59.7 55.7 Developer experience metrics

| Metric                        | Meaning                    |
|-------------------------------|----------------------------|
| Time to first PR              | New hire productivity      |
| Time to production            | Golden path efficiency     |
| Template usage rate           | Platform adoption          |
| Developer satisfaction (CSAT) | Qualitative health         |
| Toil tickets to platform      | Gaps in self-service       |
| Cognitive load survey         | Team Topologies assessment |

Improve metrics iteratively—platform backlog prioritized by developer pain, not loudest stakeholder.

## 59.8 55.8 Anti-patterns

| Anti-pattern                 | Consequence                    |
|------------------------------|--------------------------------|
| Platform without users       | Wasted investment              |
| Mandatory only, no DX        | Shadow infrastructure          |
| Endless customization        | Unmaintainable templates       |
| No versioning                | Breaking all consumers at once |
| Ops team renamed “platform”  | No product mindset change      |
| Ignoring security/compliance | Golden path bypassed for speed |

## 59.9 55.9 Chapter summary

- Platform engineering delivers an IDP with catalog, templates, and self-service golden paths.
- Backstage exemplifies developer portals integrating CI, runtime, docs, and ownership.
- Treat platform as a product with metrics, SLAs, and paved roads plus documented escape hatches.
- Infrastructure abstractions (Crossplane, GitOps) hide complexity while enforcing standards.

---

## 59.10 Lab 55.1 — Backstage software catalog

1. Deploy Backstage locally or via demo instance.
2. Register three sample services with `catalog-info.yaml`.
3. Enable TechDocs plugin for one service; view rendered docs.

---

## 59.11 Lab 55.2 — Golden path template

1. Create scaffolder template: Node.js app + Dockerfile + GitHub Actions CI.
2. Scaffold new service; verify repo contains working pipeline.
3. Document golden path README with onboarding steps (< 30 min to first deploy).

---

## 59.12 Review questions

1. How does platform engineering differ from traditional DevOps team?
2. What components belong in an Internal Developer Platform?
3. Explain golden paths and why offer unpaved road escape hatches.
4. What Backstage features support service discovery and documentation?
5. How should platform teams measure success?
6. What is Team Topologies' view of platform teams?
7. Name two anti-patterns in platform engineering initiatives.

---

*Continue: Chapter 56 — Multi-Cloud and Hybrid Patterns*



# 60 Chapter 56: Multi-Cloud and Hybrid Patterns

DevOps Handbook — Part XII, Pages 1101–1115

## 60.1 56.1 Why multi-cloud and hybrid exist

Organizations rarely start with a single cloud strategy. **Multi-cloud** uses two or more public clouds (AWS + GCP + Azure). **Hybrid cloud** combines on-premises data centers with public cloud. Drivers include:

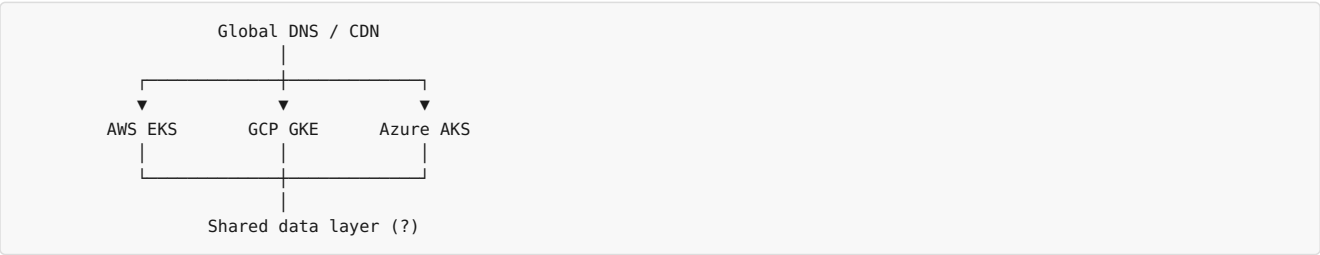
| Driver             | Example                                            |
|--------------------|----------------------------------------------------|
| Acquisition        | Merged companies on different clouds               |
| Regulatory         | Data residency requires specific regions/providers |
| Vendor negotiation | Avoid lock-in for pricing leverage                 |
| Best-of-breed      | GCP for ML, AWS for mature managed services        |
| Legacy             | Mainframe/on-prem cannot migrate quickly           |
| Disaster recovery  | Secondary region on different provider             |

Multi-cloud is **hard**—operational complexity often exceeds benefits without deliberate architecture. Prefer **single cloud + multi-region** unless a clear requirement forces multi-cloud.

## 60.2 56.2 Architecture patterns

### 60.2.1 Active-active multi-cloud

Traffic served from multiple clouds simultaneously—highest complexity:



Data consistency across clouds is the hard problem—avoid unless necessary.

### 60.2.2 Active-passive DR

Primary on AWS; GCP/Azure standby for failover:

- Replicate data asynchronously (RDS cross-region, object replication)
- DNS failover (Route 53, Cloud DNS health checks)
- Regular DR drills (Chapter 48)

### 60.2.3 Hybrid edge

On-prem core systems + cloud burst for scale:

|          |                                     |
|----------|-------------------------------------|
| On-prem: | ERP, legacy DB                      |
| Cloud:   | Web tier, analytics, ML training    |
| Link:    | Direct Connect / ExpressRoute / VPN |

### 60.2.4 Cloud-adjacent services

Keep sensitive data on-prem; process in cloud via encrypted links—**data gravity** pattern.

## 60.3 56.3 Portability strategies

| Layer         | Portability approach                                   |
|---------------|--------------------------------------------------------|
| Application   | Containers, 12-factor apps, standard APIs              |
| Orchestration | Kubernetes (EKS, GKE, AKS similar but not identical)   |
| IaC           | Terraform/Pulumi with provider modules                 |
| CI/CD         | Platform-agnostic pipelines (GitHub Actions, GitLab)   |
| Observability | OpenTelemetry, Prometheus, Grafana                     |
| Identity      | OIDC/SAML federation; avoid cloud-specific auth in app |
| Data          | Hardest—managed DB services differ significantly       |

**Abstraction cost:** Every portability layer adds indirection. Abstract **interfaces**, not every service feature.

Terraform multi-provider example:

```
modules/kubernetes-cluster - cloud-agnostic interface
variable "cluster_name" {}
variable "node_count" {}
variable "region" {}

AWS implementation
module "eks" {
 source = "../aws-eks"
 count = var.cloud == "aws" ? 1 : 0
 # ...
}

GCP implementation
module "gke" {
 source = "../gcp-gke"
 count = var.cloud == "gcp" ? 1 : 0
 # ...
}
```

## 60.4 56.4 Networking across clouds and on-prem

Hybrid connectivity options:

| Connection                                         | Bandwidth         | Latency  | Use case                  |
|----------------------------------------------------|-------------------|----------|---------------------------|
| VPN                                                | Moderate          | Higher   | Dev/test, small workloads |
| Direct Connect / ExpressRoute / Cloud Interconnect | High              | Low      | Production hybrid         |
| SD-WAN                                             | Optimized routing | Variable | Multi-site enterprises    |

Design considerations:

- **IP addressing** — Non-overlapping CIDR plans across clouds and on-prem
- **DNS** — Split-horizon or unified private zones
- **Firewall rules** — Centralized policy management

- **Latency** — Cross-cloud synchronous calls degrade UX

Service mesh (Istio multi-cluster) spans clusters across clouds—complex operational overhead.

## 60.5 56.5 GitOps and multi-cluster management

**GitOps** (Argo CD, Flux) scales to multi-cluster:

```
ApplicationSet – deploy to all clusters
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
 name: api-all-clusters
spec:
 generators:
 - clusters:
 selector:
 matchLabels:
 env: production
 template:
 metadata:
 name: 'api-{{name}}'
 spec:
 project: default
 source:
 repoURL: https://github.com/myorg/k8s-manifests
 path: apps/api/overlays/{{metadata.labels.cloud}}
 destination:
 server: '{{server}}'
 namespace: production
```

**Cluster API** provisions clusters consistently across providers.

**Rancher, Anthos, Azure Arc** — multi-cluster management planes with varying lock-in.

## 60.6 56.6 Data and state in multi-cloud

| Pattern                           | Description                                                 |
|-----------------------------------|-------------------------------------------------------------|
| <b>Single primary DB</b>          | One cloud owns data; others read via API                    |
| <b>Async replication</b>          | Cross-cloud DB replication (lag, conflict resolution)       |
| <b>Per-cloud data</b>             | Regional data stays local; aggregate via analytics pipeline |
| <b>Object storage replication</b> | S3 CRR, GCS transfer, Azure Blob replication                |

Avoid dual-write across clouds without strong consistency model—leads to split-brain.

Event-driven sync:

```
Cloud A DB change → Kafka/EventBridge → Cloud B consumer updates read replica
```

## 60.7 56.7 Operational model

| Challenge            | Mitigation                                         |
|----------------------|----------------------------------------------------|
| Different IAM models | Central identity (Okta) + cloud role federation    |
| Skill fragmentation  | Platform golden paths per cloud + shared K8s layer |

|                   |                                                |
|-------------------|------------------------------------------------|
| Cost visibility   | FinOps tagging standards (Chapter 57)          |
| Incident response | Unified on-call, cross-cloud runbooks          |
| Compliance        | Consistent policy-as-code adapted per provider |

**Cloud center of excellence (CCoE)** — Governance without blocking teams: standards, modules, training—not approval for every resource.

---

## 60.8 56.8 When NOT to multi-cloud

Avoid multi-cloud when:

- Single region cloud meets all requirements
- Team lacks operational maturity for one cloud
- Motivation is vague “lock-in fear” without cost/benefit analysis
- Application is tightly coupled to one cloud’s proprietary services (DynamoDB, BigQuery ML)

**Portable where it matters** (compute, containers); **embrace managed services** where velocity wins.

---

## 60.9 56.9 Chapter summary

- Multi-cloud and hybrid architectures address DR, compliance, legacy, and vendor strategy—at operational cost.
  - Portability via containers, Kubernetes, Terraform, and OpenTelemetry reduces lock-in but adds abstraction.
  - GitOps and cluster management tools scale deployments across cloud boundaries.
  - Prefer single-cloud multi-region unless requirements clearly mandate multi-cloud; treat data as hardest portability layer.
- 

## 60.10 Lab 56.1 — Terraform multi-provider module

1. Create Terraform module with `cloud` variable selecting AWS or GCP backend.
  2. Provision test cluster on one provider; switch variable; plan second (destroy first if budget limited).
  3. Document interface variables shared across implementations.
- 

## 60.11 Lab 56.2 — Argo CD ApplicationSet

1. Register two clusters (or two namespaces simulating clusters) in Argo CD.
  2. Deploy ApplicationSet syncing same app to both.
  3. Update Git manifest; verify both destinations sync.
- 

## 60.12 Review questions

1. Distinguish multi-cloud, hybrid cloud, and multi-region single cloud.
2. Why is data the hardest layer to make portable across clouds?
3. Name three legitimate business drivers for multi-cloud architecture.
4. How does GitOps help manage multi-cluster deployments?
5. What networking options connect on-premises to public cloud?
6. When should an organization avoid multi-cloud strategy?

7. What role does a Cloud Center of Excellence play?

---

*Continue: Chapter 57 — FinOps*

# 61 Chapter 57: FinOps and Cost-Aware Engineering

DevOps Handbook — Part XII, Pages 1116–1130

## 61.1 57.1 Cloud cost as engineering concern

Cloud’s pay-as-you-go model shifts infrastructure from CapEx (predictable depreciation) to OpEx (variable, opaque bills). Without discipline, costs scale faster than revenue—the **FinOps** discipline brings **financial accountability** to cloud spending through collaboration between engineering, finance, and business.

FinOps Foundation principles:

- 1. **Teams need to collaborate** — Finance, engineering, leadership share ownership
- 2. **Everyone takes ownership** — Engineers see and optimize their spend
- 3. **Reports should be accessible and timely** — Near real-time visibility
- 4. **Decisions are driven by business value** — Cost vs performance tradeoffs explicit
- 5. **Take advantage of the variable cost model** — Scale down, spot instances, right-size

FinOps is a **continuous cycle**: Inform → Optimize → Operate.

## 61.2 57.2 Cost visibility and allocation

You cannot optimize what you cannot attribute.

**Tagging strategy** (mandatory tags enforced by policy):

| Tag         | Purpose                 |
|-------------|-------------------------|
| team        | Cost center owner       |
| service     | Application attribution |
| environment | dev/staging/prod        |
| cost-center | Finance chargeback      |
| project     | Initiative tracking     |

AWS SCP / Tag Policies enforce tags at organization level. Kubernetes labels mirror tags for in-cluster cost tools.

Tools:

| Tool                          | Scope                               |
|-------------------------------|-------------------------------------|
| AWS Cost Explorer / CUR       | AWS detailed billing                |
| GCP Billing Export + BigQuery | GCP analysis                        |
| Azure Cost Management         | Azure spend                         |
| Kubecost / OpenCost           | Kubernetes cost by namespace/pod    |
| CloudHealth / Cloudability    | Multi-cloud FinOps platforms        |
| Infracost                     | Terraform plan cost estimates in CI |

OpenCost on Kubernetes:

```
apiVersion: apps/v1
kind: Deployment
metadata:
```

```

name: opencost
namespace: opencost
spec:
 template:
 spec:
 containers:
 - name: opencost
 image: opencost/opencost:latest
 env:
 - name: PROMETHEUS_SERVER_ENDPOINT
 value: "http://prometheus:9090"

```

Dashboard shows cost per namespace, deployment, label—allocating cluster overhead by CPU/RAM usage.

## 61.3 57.3 Right-sizing and waste elimination

Common waste sources:

| Waste                         | Detection                              | Remediation                         |
|-------------------------------|----------------------------------------|-------------------------------------|
| Idle resources                | CloudWatch/GCP metrics low utilization | Stop dev instances nights/weekends  |
| Oversized instances           | CPU < 20% sustained                    | Downsize instance type              |
| Unattached EBS volumes        | Cost anomaly reports                   | Snapshot and delete                 |
| Old snapshots                 | Age-based report                       | Lifecycle policy                    |
| Over-provisioned K8s requests | Kubecost efficiency score              | Reduce requests/limits              |
| Forgotten environments        | Tag + age audit                        | TTL automation                      |
| Data transfer                 | Network cost breakdown                 | CDN, VPC endpoints, region locality |

Right-sizing workflow:

1. Identify top 20 cost drivers (Pareto: 80% spend in 20% resources)
2. Analyze 30-day utilization metrics
3. Test smaller size in staging
4. Apply change with rollback plan
5. Measure cost delta next billing cycle

**AWS Compute Optimizer, GCP Recommender, Azure Advisor** — automated right-sizing suggestions.

## 61.4 57.4 Pricing models and savings

| Model                              | When to use                                         |
|------------------------------------|-----------------------------------------------------|
| On-demand                          | Variable, unpredictable workloads                   |
| Reserved Instances / Savings Plans | Steady-state baseline (1-3 year commit)             |
| Spot / Preemptible                 | Fault-tolerant batch, CI runners, stateless workers |
| Committed use discounts            | GCP CUD, Azure RI                                   |

Spot instance pattern for CI:

```

Kubernetes node pool – spot for CI workloads
nodeSelector:
 workload-type: ci-batch

```

```
tolerations:
- key: "spot"
 operator: "Equal"
 value: "true"
 effect: "NoSchedule"
```

Mix: 70% reserved for baseline, 30% on-demand/spot for burst—model in spreadsheet before committing.

## 61.5 57.5 Cost in CI/CD and IaC

Shift cost awareness left:

**Infracost** in PR comments:

```
- name: Infracost
 uses: infracost/actions/setup@v3
- run: |
 infracost breakdown --path terraform/ --format json \
 --out-file infracost-base.json
 infracost diff --path terraform/ \
 --compare-to infracost-base.json
```

Developers see “+\$450/month” before merging Terraform expanding RDS instance.

Policies:

- Budget alerts at 80%, 100%, 120% of forecast
- Auto-shutdown Lambda for non-prod tagged `auto-shutdown=true`
- Maximum instance size guardrails in Terraform policy (OPA: deny `p4d.24xlarge` in dev)

## 61.6 57.6 Unit economics

Connect cloud spend to business metrics:

| Metric               | Formula                             |
|----------------------|-------------------------------------|
| Cost per customer    | Monthly infra / active customers    |
| Cost per transaction | Infra / transactions                |
| Cost per API request | Service cost / request volume       |
| Margin per feature   | Revenue - allocated infra - support |

Engineering decisions framed in unit economics:

“Caching reduces DB cost \$2K/month and improves latency—ROI positive in 2 weeks.”

Finance partnership: monthly **FinOps review** with engineering managers reviewing team dashboards—not surprise bills.

## 61.7 57.7 FinOps maturity stages

| Stage | Characteristics                                       |
|-------|-------------------------------------------------------|
| Crawl | Basic billing visibility, manual reports              |
| Walk  | Tagging enforced, team dashboards, anomaly alerts     |
| Run   | Unit economics, automated optimization, CI cost gates |



|            |                                                                  |
|------------|------------------------------------------------------------------|
| <b>Fly</b> | Predictive forecasting, autoscaling cost-aware, culture embedded |
|------------|------------------------------------------------------------------|

## 61.8 57.8 Chapter summary

- FinOps aligns engineering, finance, and business on cloud cost accountability.
- Tagging and tools (Cost Explorer, Kubecost, Infracost) enable visibility and allocation.
- Right-sizing, reserved capacity, and spot instances optimize spend without sacrificing reliability.
- Unit economics and CI cost feedback integrate financial awareness into daily engineering.

## 61.9 Lab 57.1 — Cost allocation tags

1. Define mandatory tag schema for Terraform modules.
2. Apply OPA/Checkov policy denying untagged resources.
3. Export cost report grouped by `team` tag (or simulate with labeled resources).

## 61.10 Lab 57.2 — Infracost in CI

1. Add Infracost to Terraform repo CI on pull requests.
2. Open PR changing instance type; review cost diff comment.
3. Document team policy for cost increase approval threshold.

## 61.11 Review questions

1. What are the three FinOps lifecycle phases?
2. Why is tagging essential for cloud cost allocation?
3. Compare reserved instances and spot instances use cases.
4. How does Kubecost attribute Kubernetes cluster costs?
5. What is unit economics in cloud engineering context?
6. Name three common sources of cloud waste.
7. How can IaC CI pipelines prevent costly infrastructure changes?

*Continue: Chapter 58 — MLOps Overview*

# 62 Chapter 58: MLOps Overview for DevOps Engineers

DevOps Handbook — Part XII, Pages 1131–1145

## 62.1 58.1 Why DevOps engineers need MLOps

Machine learning systems are **software systems**—they require versioning, testing, deployment, monitoring, and incident response. Unlike traditional apps, ML adds **data**, **models**, and **non-deterministic behavior**. **MLOps** (Machine Learning Operations) applies DevOps principles to the ML lifecycle.

DevOps engineers increasingly support data science teams—or operate ML platforms. Understanding MLOps prevents treating models as “black box scripts” outside standard engineering practices.

ML system components:

Data sources → Feature pipeline → Training → Model registry →  
Deployment → Inference service → Monitoring (data + model drift)

## 62.2 58.2 ML lifecycle vs traditional SDLC

| Traditional app             | ML system                                  |
|-----------------------------|--------------------------------------------|
| Code is primary artifact    | Code + data + model weights                |
| Tests verify logic          | Tests verify data schema, model metrics    |
| Deploy binary/container     | Deploy model version + feature pipeline    |
| Monitor errors/latency      | Monitor + data drift + model performance   |
| Rollback = previous release | Rollback = previous model + feature schema |

**Technical debt in ML** accumulates silently—training-serving skew, stale features, label leakage—requiring explicit governance.

## 62.3 58.3 Key MLOps concepts

| Concept             | Definition                                                       |
|---------------------|------------------------------------------------------------------|
| Feature store       | Centralized, versioned features for training and serving         |
| Model registry      | Versioned model artifacts with metadata and stage (staging/prod) |
| Experiment tracking | Log params, metrics, artifacts per training run                  |
| Training pipeline   | Automated, reproducible training workflow                        |
| Serving             | Real-time (REST/gRPC) or batch inference                         |
| Data validation     | Schema and statistical checks on input data                      |
| Model monitoring    | Drift detection, performance decay alerts                        |

Tools landscape:

|  |  |
|--|--|
|  |  |
|--|--|

| Category            | Tools                                       |
|---------------------|---------------------------------------------|
| Experiment tracking | MLflow, Weights & Biases, Neptune           |
| Orchestration       | Kubeflow, Metaflow, Airflow, Argo Workflows |
| Feature store       | Feast, Tecton, SageMaker Feature Store      |
| Model registry      | MLflow Model Registry, SageMaker, Vertex AI |
| Serving             | KServe, Seldon Core, TorchServe, Triton     |
| Monitoring          | Evidently AI, WhyLabs, Arize                |

## 62.4 58.4 Reproducible training pipeline

Training must be **reproducible**: same data snapshot + code + hyperparameters = same model (within stochastic variance).

MLflow training example:

```
import mlflow
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

with mlflow.start_run():
 mlflow.log_param("n_estimators", 100)
 mlflow.log_param("max_depth", 10)

 model = RandomForestClassifier(n_estimators=100, max_depth=10)
 model.fit(X_train, y_train)

 preds = model.predict(X_test)
 accuracy = accuracy_score(y_test, preds)
 mlflow.log_metric("accuracy", accuracy)

 mlflow.sklearn.log_model(model, "model")
```

Pipeline orchestration (Kubeflow Pipelines):

```
@dsl.pipeline(name="Training pipeline")
def training_pipeline(data_path: str):
 validate_op = validate_data(data_path)
 train_op = train_model(validate_op.outputs["validated_data"])
 evaluate_op = evaluate_model(train_op.outputs["model"])
 with dsl.Condition(evaluate_op.outputs["accuracy"] > 0.85):
 deploy_op = register_model(train_op.outputs["model"])
```

CI for ML:

- Unit tests on feature engineering functions
- Data validation on sample datasets (Great Expectations)
- Model performance threshold gates before registry promotion

## 62.5 58.5 Model deployment patterns

| Pattern               | Description                                        |
|-----------------------|----------------------------------------------------|
| <b>Embedded model</b> | Model loaded in app process (sklearn pickle, ONNX) |
| <b>Model server</b>   | Separate inference service (Triton, TF Serving)    |
| <b>Sidecar</b>        | Model container alongside app pod in K8s           |
| <b>Serverless</b>     | Lambda/Cloud Functions for low-traffic inference   |

|       |                                         |
|-------|-----------------------------------------|
| Batch | Scheduled jobs scoring datasets offline |
|-------|-----------------------------------------|

KServe InferenceService:

```

apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
 name: fraud-detector
spec:
 predictor:
 sklearn:
 storageUri: s3://models/fraud-detector/v3
 resources:
 requests:
 cpu: "500m"
 memory: "512Mi"
 limits:
 cpu: "1"
 memory: "1Gi"

```

**Canary model rollout:** route 5% traffic to new model version; compare prediction latency and business KPI before full promotion—same patterns as Chapter 41.

## 62.6 58.6 Feature stores and training-serving skew

**Training-serving skew** — Features computed differently in training vs production—silent accuracy degradation.

Feature store solves:

- **Single definition** — Feature logic defined once
- **Point-in-time correctness** — Training uses historical feature values as they existed
- **Online/offline serving** — Same features for batch training and real-time inference

Feast example:

```

from feast import FeatureStore

store = FeatureStore(repo_path=".")

Training: historical features
training_df = store.get_historical_features(
 entity_df=entity_df,
 features=["customer_features:avg_order_value"],
).to_df()

Serving: online features
features = store.get_online_features(
 features=["customer_features:avg_order_value"],
 entity_rows=[{"customer_id": 12345}],
).to_dict()

```

DevOps role: operate Feast infrastructure, monitor feature pipeline latency, integrate with data warehouse connectors.

## 62.7 58.7 Monitoring ML in production

Monitor four layers:

| Layer          | Signals                                                |
|----------------|--------------------------------------------------------|
| Infrastructure | CPU/GPU, memory, request rate (standard observability) |
| Service        | Latency, errors, throughput                            |

|              |                                                               |
|--------------|---------------------------------------------------------------|
| <b>Data</b>  | Input distribution drift, missing features, schema violations |
| <b>Model</b> | Prediction distribution drift, accuracy on labeled feedback   |

Evidently drift report (conceptual):

```
from evidently.report import Report
from evidently.metric_preset import DataDriftPreset

report = Report(metrics=[DataDriftPreset()])
report.run(reference_data=training_sample, current_data=production_sample)
report.save_html("drift_report.html")
```

Alert when:

- Feature drift exceeds statistical threshold
- Model accuracy on holdout feedback drops
- Inference latency SLO breached

Retrain triggers: scheduled (weekly) + event-driven (drift alert).

## 62.8 58.8 Governance and responsible ML

- **Model cards** — Document intended use, limitations, bias evaluation
- **Lineage** — Track data sources, transformations, model dependencies
- **Access control** — Sensitive training data restricted
- **Audit** — Who promoted model to production
- **Rollback** — Previous model version in registry ready to deploy

Regulatory contexts (credit, healthcare) require explainability (SHAP, LIME) and audit trails—integrate with compliance automation (Chapter 54).

## 62.9 58.9 DevOps + MLOps collaboration

| DevOps provides             | Data science provides        |
|-----------------------------|------------------------------|
| K8s, CI/CD, observability   | Models, features, metrics    |
| GPU node pools, autoscaling | Training job requirements    |
| Secrets, networking         | Data access patterns         |
| SLOs, on-call for serving   | Model performance thresholds |

Platform golden path for ML: “Deploy sklearn model from registry to KServe with monitoring”—Backstage template scaffolds InferenceService + Grafana dashboard.

## 62.10 58.10 Chapter summary

- MLOps extends DevOps to data, training, model registry, serving, and drift monitoring.
- Reproducible pipelines, experiment tracking, and feature stores reduce training-serving skew.
- Deploy models like services—with canaries, SLOs, and rollback via model registry.
- DevOps engineers operate ML infrastructure; data scientists own model quality—collaborate via platform golden paths.

## 62.11 Lab 58.1 — MLflow end-to-end

1. Train simple sklearn model; log to local MLflow.
  2. Register model; promote to Staging.
  3. Serve model via `mlflow models serve`; send test predictions.
- 

## 62.12 Lab 58.2 — Model monitoring dashboard

1. Collect week of inference inputs (CSV).
  2. Run Evidently drift report against training baseline.
  3. Document retrain trigger policy based on drift score.
- 

## 62.13 Review questions

1. How does an ML system differ from a traditional application in terms of artifacts?
  2. What is training-serving skew, and how do feature stores help?
  3. Name three components of an MLOps platform stack.
  4. Compare real-time and batch inference deployment patterns.
  5. What should ML monitoring cover beyond standard RED metrics?
  6. How do canary deployments apply to model rollouts?
  7. What is a model registry, and why version models?
- 

*Continue: Chapter 59 — Capstone Production System Design*

# 63 Chapter 59: Capstone — End-to-End Production System Design

DevOps Handbook — Part XII, Pages 1146–1180

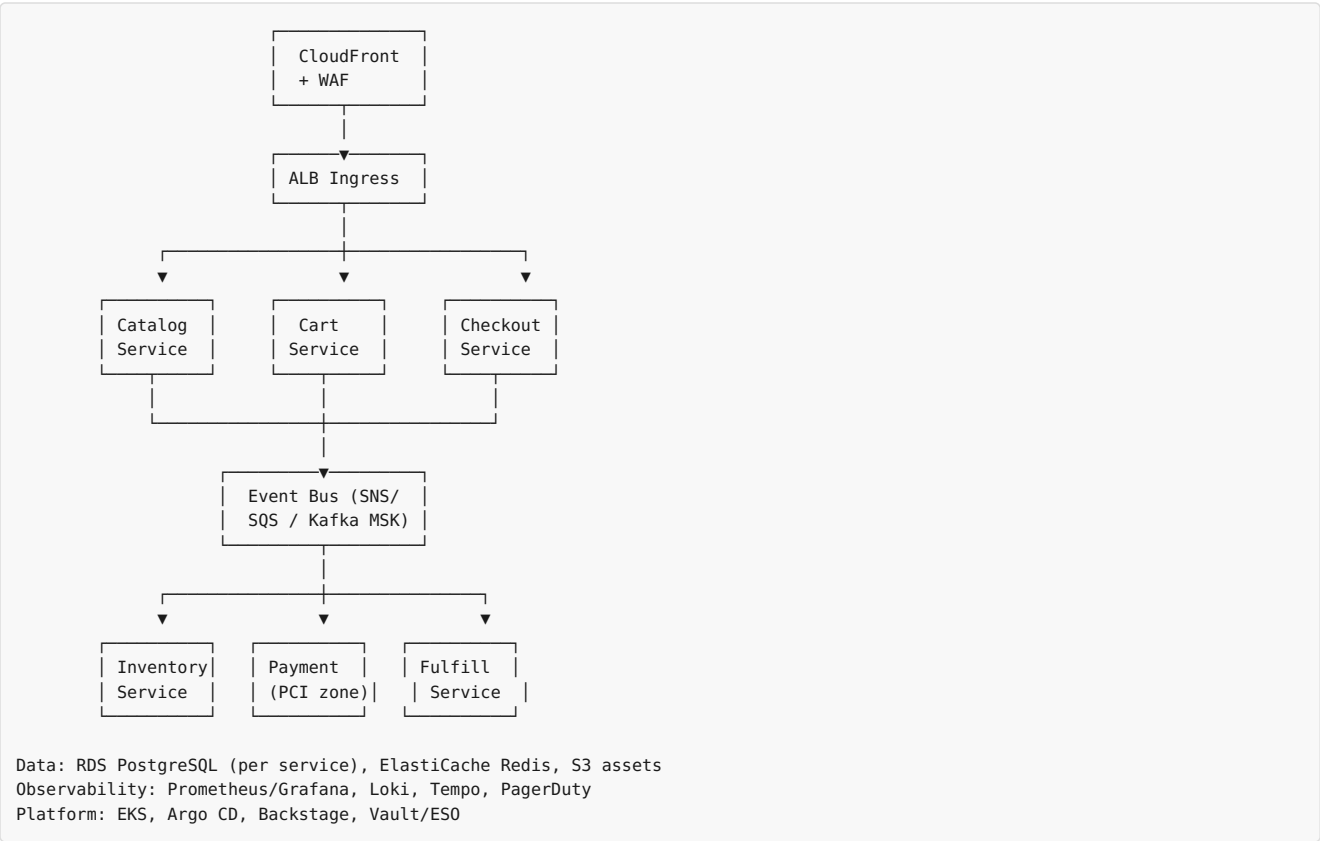
## 63.1 59.1 Capstone objectives

This chapter synthesizes the handbook into a **reference architecture** for a production-grade SaaS platform. You will design a system that embodies DevOps principles: automated delivery, observable operations, security by default, cost awareness, and reliable incident response.

Capstone scenario: **ShopStream** — a B2C e-commerce API handling product catalog, cart, checkout, and order fulfillment. Requirements:

| Requirement      | Target                                                 |
|------------------|--------------------------------------------------------|
| Availability SLO | 99.9% (43 min/month error budget)                      |
| Latency SLO      | p99 < 500ms for checkout API                           |
| Scale            | 10K RPS peak, 50 microservices over 2 years            |
| Compliance       | PCI DSS for payment scope; SOC 2 Type II               |
| Team             | 8 stream-aligned teams + 1 platform team               |
| Cloud            | AWS primary (reference AWS Handbook for service depth) |

## 63.2 59.2 Architecture overview



Design principles applied:

- **Domain-driven service boundaries** — Checkout team owns checkout SLO
- **Async where possible** — Order events via queue, not synchronous chains
- **PCI segmentation** — Payment service in isolated subnet; no PAN in logs
- **Fail closed** — Auth failures reject; circuit breakers on dependencies

## 63.3 59.3 Repository and team structure

Team Topologies mapping:

| Team        | Services                           | Interaction             |
|-------------|------------------------------------|-------------------------|
| Catalog     | product-api, search-indexer        | Stream-aligned          |
| Checkout    | cart, checkout, payment-gateway    | Stream-aligned          |
| Fulfillment | inventory, shipping, notifications | Stream-aligned          |
| Platform    | IDP, clusters, shared modules      | Platform X-as-a-Service |
| Security    | Policies, scanning, audit          | Enabling                |

Repository strategy: **polyrepo** with shared templates (one repo per service)—scaffolded via Backstage golden path.

Each service repo contains:

```
service-name/
├── src/
├── Dockerfile
├── helm/
├── terraform/ # Service-specific infra module
├── .github/workflows/
│ ├── ci.yml
│ └── deploy.yml
├── catalog-info.yaml
├── docs/
└── RUNBOOK.md
```

Monorepo optional for tightly coupled early-stage; migrate to polyrepo as teams scale.

## 63.4 59.4 CI/CD pipeline design

Per-service pipeline (GitHub Actions):

```
PR opened:
 lint → unit test → SAST (Semgrep) → SCA → IaC scan → build image →
 Trivy scan → push to ECR (SHA tag) → deploy preview (Review App)

Merge to main:
 integration tests → sign image (cosign) → generate SBOM →
 deploy staging (Argo CD sync) → DAST (ZAP baseline) →
 manual approval → canary 5% prod → metrics gate → full prod
```

Key configurations:

```
Excerpt — production deploy with canary via Argo Rollouts
deploy-production:
 needs: [build, scan, staging-smoke]
 environment: production
 steps:
 - uses: aws-actions/configure-aws-credentials@v4
```



```

with:
 role-to-assume: arn:aws:iam::ACCOUNT:role/GHA-Deploy-Prod
- run: |
 cosign verify ${ env.IMAGE } --certificate-oidc-issuer=...
 argocd app set checkout-api -p image.tag=${ github.sha }
 argocd app sync checkout-api --prune

```

DORA instrumentation: deployment webhook to metrics backend on every prod sync.

Deployment strategies by service criticality:

| Service  | Strategy                       |
|----------|--------------------------------|
| Catalog  | Rolling                        |
| Checkout | Canary with automated analysis |
| Payment  | Blue/green + manual approval   |

Feature flags (LaunchDarkly) for checkout UI experiments decoupled from deploy cadence.

## 63.5 59.5 Infrastructure as Code

**Terraform** layered modules:

```

terraform/
├─ modules/
│ ├─ vpc/
│ ├─ eks/
│ ├─ rds/
│ └─ service-deployment/
├─ environments/
│ ├─ staging/
│ └─ production/
└─ global/
 └─ iam/

```

State: S3 backend + DynamoDB lock per environment. CI runs `terraform plan` on PR, `apply` on merge to environment branch.

**GitOps** (Argo CD) reconciles Kubernetes from `k8s-manifests` repo:

```

apps/
├─ checkout-api/
│ ├─ base/
│ └─ overlays/
│ ├─ staging/
│ └─ production/
└─ catalog-api/
 └─ ...

```

Platform provisions EKS, networking, shared RDS proxy; teams own service overlays.

Policy gates: Checkov in CI; Gatekeeper in cluster (no privileged pods, required labels, signed images).

## 63.6 59.6 Security architecture

Defense in depth layers:

| Layer    | Implementation                                            |
|----------|-----------------------------------------------------------|
| Edge     | WAF, rate limiting, TLS 1.3                               |
| Identity | OAuth2/OIDC (Cognito/Auth0), mTLS service mesh (optional) |

|              |                                                        |
|--------------|--------------------------------------------------------|
| Network      | VPC segmentation, NetworkPolicies, PCI isolated subnet |
| Secrets      | Vault + ESO; dynamic DB creds for payment service      |
| Supply chain | SBOM, cosign, Kyverno verifyImages                     |
| Runtime      | Falco rules, PSA restricted                            |
| Compliance   | Audit logs → SIEM; quarterly access reviews            |

PCI scope minimization: payment service tokenizes via Stripe—ShopStream never stores PAN; SAQ A scope where possible.

Security scanning integrated per Chapter 49–54; critical CVE blocks prod deploy.

## 63.7 59.7 Observability and SRE

Per-service SLOs documented in `slos/checkout-api.yaml`:

```
service: checkout-api
slos:
 - name: availability
 target: 99.9
 window: 30d
 sli:
 query: |
 sum(rate(http_requests_total{service="checkout",status!~"5.."}[5m]))
 / sum(rate(http_requests_total{service="checkout"}[5m]))
 - name: latency-p99
 target: 99.0 # 99% under 500ms
 window: 30d
 sli:
 query: |
 histogram_quantile(0.99, sum(rate(http_duration_bucket[5m])) by (le))
 < 0.5
```

Stack:

- **Metrics** — Prometheus + Grafana (kube-prometheus-stack)
- **Logs** — Fluent Bit → Loki (or OpenSearch for full-text SIEM)
- **Traces** — OpenTelemetry SDK → Collector → Tempo
- **Alerting** — Alertmanager → PagerDuty; multi-window burn alerts

Runbooks in repo `RUNBOOK.md` linked from Backstage. On-call rotation per team with escalation to platform for infra issues.

Incident process: SEV definitions, blameless postmortems within 5 days, action items in Jira.

Chaos: monthly Litmus pod-delete on staging; quarterly game day simulating payment dependency failure.

## 63.8 59.8 Data, DR, and scaling

| Component      | Pattern                                      |
|----------------|----------------------------------------------|
| PostgreSQL     | RDS Multi-AZ; read replicas for catalog      |
| Redis          | ElastiCache cluster mode for session cache   |
| Object storage | S3 + CloudFront for product images           |
| Events         | MSK Kafka for order stream; DLQ for failures |
| Search         | OpenSearch fed by CDC from catalog DB        |

**DR:** RPO 1 hour, RTO 4 hours—warm standby in second region (us-west-2); Route 53 failover drill quarterly.

**Autoscaling:** HPA on CPU/RPS custom metrics; Karpenter for node provisioning; scale-to-zero for dev namespaces (FinOps).

---

## 63.9 59.9 FinOps and platform metrics

Mandatory tags: `team`, `service`, `environment`, `cost-center`.

Kubecost dashboards per team; monthly FinOps review with unit economics:

- Cost per order processed
- Cost per active user
- Idle resource report automated weekly

Infracost on Terraform PRs; budget alerts at org and team level.

Platform team metrics: golden path adoption %, developer CSAT, time-to-first-deploy for new services.

---

## 63.10 59.10 Capstone delivery roadmap

Phased implementation (technical sequencing, not calendar estimates):

| Phase             | Deliverables                                              |
|-------------------|-----------------------------------------------------------|
| 1 — Foundation    | EKS, VPC, Terraform modules, Argo CD, observability stack |
| 2 — Platform      | Backstage, golden path template, ESO, CI templates        |
| 3 — First service | Catalog API end-to-end: CI, deploy, SLO, runbook          |
| 4 — Critical path | Checkout + payment with PCI segmentation, canary          |
| 5 — Hardening     | Security gates, chaos, DR drill, compliance evidence      |
| 6 — Scale         | Event bus, additional services, FinOps optimization       |

Each phase ends with **definition of done**: deployed to staging, SLO dashboard live, runbook reviewed, security scan clean.

---

## 63.11 59.11 Design review checklist

Before production launch, verify:

- ☐ SLOs defined with error budget alerts
  - ☐ CI: lint, test, SAST, SCA, container scan, IaC scan
  - ☐ Signed images + admission policy enforced
  - ☐ Secrets not in Git; ESO/Vault operational
  - ☐ Centralized logs with trace correlation
  - ☐ Runbooks and on-call rotation active
  - ☐ DR failover tested within RTO target
  - ☐ Tagging and cost dashboards per team
  - ☐ Postmortem template and incident roles documented
  - ☐ DORA metrics emitted from pipeline
-

## 63.12 59.12 Chapter summary

- ShopStream capstone integrates microservices, EKS, GitOps, CI/CD security, observability, and FinOps.
  - Team Topologies and golden paths scale organizationally; polyrepo + templates balance autonomy and consistency.
  - Layered security, SLO-driven operations, and phased delivery reduce risk while enabling continuous deployment.
  - Use the design review checklist as a template for your own production systems.
- 

## 63.13 Capstone Lab — Mini ShopStream

Build a reduced capstone (3 services: catalog, cart, checkout):

1. Scaffold services via template with shared CI pipeline.
  2. Deploy to local k3d or cloud EKS with Argo CD.
  3. Implement Prometheus metrics, one SLO dashboard, and alert.
  4. Add Trivy scan gate and cosign sign (optional keyless).
  5. Write RUNBOOK.md and conduct 30-minute game day (kill checkout pod).
  6. Document architecture decision record (ADR) for one major choice.
- 

## 63.14 Review questions

1. How do you decompose ShopStream into team-aligned services?
  2. Why isolate payment in a separate network segment?
  3. Describe the full CI/CD path from PR to production canary.
  4. Which observability signals form checkout SLOs?
  5. How does the platform team enable stream-aligned teams without bottleneck?
  6. What DR strategy meets RPO 1h / RTO 4h?
  7. Walk through the capstone design review checklist for catalog-api.
- 

*Continue: Chapter 60 — Appendix, Cheatsheets, Glossary*

# 64 Chapter 60: Appendix — Cheatsheets, Glossary, and Index

---

DevOps Handbook — Part XII, Pages 1181–1200

---

## 64.1 60.1 kubectl quick reference

```
Context and namespaces
kubectl config use-context prod
kubectl get pods -n production -o wide

Deployments
kubectl rollout status deployment/api -n production
kubectl rollout undo deployment/api -n production
kubectl scale deployment/api --replicas=5

Debugging
kubectl logs -f deployment/api -c api --tail=100
kubectl exec -it pod/api-xxx -- /bin/sh
kubectl describe pod api-xxx
kubectl get events --sort-by=.metadata.creationTimestamp

Resources
kubectl top pods -n production
kubectl apply -f manifest.yaml --dry-run=server
kubectl diff -f manifest.yaml

Port forward
kubectl port-forward svc/api 8080:80
```

## 64.2 60.2 Docker quick reference

```
Build and run
docker build -t myapp:v1 .
docker run -d -p 8080:80 --name myapp myapp:v1
docker exec -it myapp sh

Cleanup
docker system prune -a --volumes # caution: removes unused data
docker logs -f myapp

Multi-stage build pattern
FROM golang:1.22 AS builder
FROM gcr.io/distroless/static-debian12
```

## 64.3 60.3 Terraform quick reference

```
terraform init
terraform plan -out=tfplan
terraform apply tfplan
terraform state list
terraform import aws_instance.web i-0abc123
terraform fmt -recursive
terraform validate

Workspace
terraform workspace select staging
terraform workspace list
```

---

## 64.4 60.4 Git essentials

```
git status
git log --oneline --graph -20
git diff main...feature-branch
git rebase -i HEAD~3
git stash push -m "wip"
git cherry-pick abc1234

Recover
git reflog
git reset --hard HEAD@{2}
```

---

## 64.5 60.5 Linux troubleshooting

```
Processes
ps aux | grep nginx
systemctl status nginx
journalctl -u nginx -f --since "10 min ago"

Network
ss -tlnp
curl -vI https://example.com
dig +short example.com
traceroute example.com

Disk
df -h
du -sh /var/log/*
lsof +D /path/to/dir

Resources
top -o %CPU
free -h
vmstat 1 5
iostat -x 1 3
```

---

## 64.6 60.6 PromQL cheatsheet

```
Rate and errors
rate(http_requests_total[5m])
sum by (service) (rate(http_requests_total{status=~"5.."}[5m]))

Latency percentiles
histogram_quantile(0.99, sum(rate(http_duration_bucket[5m])) by (le, service))

Resource usage
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)

Predict disk full (4h)
predict_linear(node_filesystem_free_bytes[1h], 4*3600) < 0

Absent metric alert helper
absent(up{job="api"})
```

---

## 64.7 60.7 CI/CD YAML snippets

GitHub Actions concurrency:

```
concurrency:
```

```
group: deploy-${github.ref}
cancel-in-progress: true
```

GitLab rules:

```
rules:
- if: $CI_PIPELINE_SOURCE == "merge_request_event"
- if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

Jenkins declarative post:

```
post {
 always { cleanWs() }
 failure { emailx subject: "Failed", body: "${env.BUILD_URL}" }
}
```

## 64.8 60.8 Security tool commands

```
Secret scan
gitLeaks detect --source . -v

Container scan
trivy image myapp:latest
trivy fs --severity CRITICAL,HIGH .

SBOM
syft packages dir:. -o cyclonedx-json > sbom.json
gype sbom:sbom.json

Sign and verify
cosign sign myregistry/myapp:v1
cosign verify myregistry/myapp:v1

IaC scan
checkov -d terraform/ --framework terraform
```

## 64.9 60.9 Observability checklist

| Signal     | Minimum instrumentation                       |
|------------|-----------------------------------------------|
| Metrics    | RED per service; USE per node                 |
| Logs       | JSON with timestamp, level, service, trace_id |
| Traces     | W3C traceparent on all HTTP/gRPC exits        |
| Alerts     | SLO burn + symptom-based pages only           |
| Dashboards | Golden signals per service + infra overview   |

## 64.10 60.10 SLO error budget reference

| SLO    | Monthly downtime budget | Quarterly budget |
|--------|-------------------------|------------------|
| 99%    | ~7.2 hours              | ~21.6 hours      |
| 99.9%  | ~43 minutes             | ~2.16 hours      |
| 99.95% | ~22 minutes             | ~1.08 hours      |
|        |                         |                  |

|        |              |             |
|--------|--------------|-------------|
| 99.99% | ~4.3 minutes | ~13 minutes |
|--------|--------------|-------------|

Formula: `budget = (1 - SLO) × period_seconds`

## 64.11 60.11 Glossary

| Term                 | Definition                                                   |
|----------------------|--------------------------------------------------------------|
| <b>Alert fatigue</b> | Desensitization from excessive non-actionable alerts         |
| <b>Argo CD</b>       | GitOps continuous delivery tool for Kubernetes               |
| <b>Blue/green</b>    | Deployment with two environments; traffic switch for cutover |
| <b>Canary</b>        | Gradual traffic shift to new version with monitoring         |
| <b>CI/CD</b>         | Continuous Integration / Continuous Delivery or Deployment   |
| <b>Container</b>     | Lightweight isolated process bundle with filesystem          |
| <b>DAST</b>          | Dynamic Application Security Testing on running apps         |
| <b>DevSecOps</b>     | Integrating security into DevOps lifecycle                   |
| <b>DORA metrics</b>  | Deployment frequency, lead time, change failure rate, MTTR   |
| <b>Error budget</b>  | Allowed unreliability: $1 - SLO$                             |
| <b>Feature flag</b>  | Runtime toggle decoupling deploy from release                |
| <b>FinOps</b>        | Cloud financial management discipline                        |
| <b>GitOps</b>        | Declarative infra/apps with Git as source of truth           |
| <b>Golden path</b>   | Supported self-service workflow in platform engineering      |
| <b>Helm</b>          | Kubernetes package manager                                   |
| <b>IaC</b>           | Infrastructure as Code                                       |
| <b>IDP</b>           | Internal Developer Platform                                  |
| <b>Ingress</b>       | Kubernetes HTTP routing into cluster                         |
| <b>Kubernetes</b>    | Container orchestration platform                             |
| <b>MLOps</b>         | ML lifecycle automation and operations                       |
| <b>MTTR</b>          | Mean Time to Restore service                                 |
| <b>Observability</b> | Inferring internal state from external outputs               |
| <b>OIDC</b>          | OpenID Connect federated identity                            |
| <b>OPA</b>           | Open Policy Agent for policy decisions                       |
| <b>Pod</b>           | Smallest deployable unit in Kubernetes                       |
| <b>Prometheus</b>    | Open-source metrics TSDB and scraper                         |
| <b>Rego</b>          | OPA policy language                                          |
| <b>Runbook</b>       | Step-by-step operational procedure                           |
| <b>SAML</b>          | Security Assertion Markup Language for SSO                   |
| <b>SAST</b>          | Static Application Security Testing                          |
| <b>SBOM</b>          | Software Bill of Materials                                   |
| <b>SCA</b>           | Software Composition Analysis for dependencies               |



|                                |                                                |
|--------------------------------|------------------------------------------------|
| <b>Shift-left</b>              | Moving activities earlier in SDLC              |
| <b>SLO/SLI/SLA</b>             | Service Level Objective/Indicator/Agreement    |
| <b>SLSA</b>                    | Supply-chain Levels for Software Artifacts     |
| <b>SRE</b>                     | Site Reliability Engineering                   |
| <b>Terraform</b>               | IaC tool by HashiCorp                          |
| <b>Toil</b>                    | Manual repetitive operational work             |
| <b>Trunk-based development</b> | Short-lived branches merged frequently to main |
| <b>Vault</b>                   | HashiCorp secrets management                   |
| <b>WAF</b>                     | Web Application Firewall                       |

## 64.12 60.12 Handbook chapter index

| Part | Chapters | Topics                                                 |
|------|----------|--------------------------------------------------------|
| I    | 1–4      | DevOps culture, CALMS, careers                         |
| II   | 5–8      | Linux, systemd, networking, bash                       |
| III  | 9–11     | Git, branching, collaboration                          |
| IV   | 12–15    | Networking, HTTP/TLS, load balancing                   |
| V    | 16–19    | Python, Go, templating, testing                        |
| VI   | 20–25    | IaC, Terraform, Ansible, Packer, policy                |
| VII  | 26–30    | Docker, compose, registries, Podman                    |
| VIII | 31–36    | Kubernetes, Helm, GitOps, RBAC                         |
| IX   | 37–42    | CI/CD, Actions, Jenkins, GitLab, deploy, DORA          |
| X    | 43–48    | Observability, Prometheus, ELK, OTel, SLO, chaos       |
| XI   | 49–54    | DevSecOps, scanning, secrets, supply chain, compliance |
| XII  | 55–60    | Platform eng, multi-cloud, FinOps, MLOps, capstone     |

Cross-reference: [AWS Handbook](#) for cloud-specific service depth.

## 64.13 60.13 Recommended reading

| Book                                | Author(s)                   | Topic               |
|-------------------------------------|-----------------------------|---------------------|
| <i>The DevOps Handbook</i>          | Kim, Humble, Debois, Willis | DevOps practices    |
| <i>Accelerate</i>                   | Forsgren, Humble, Kim       | DORA research       |
| <i>Site Reliability Engineering</i> | Google                      | SRE principles      |
| <i>The Phoenix Project</i>          | Kim, Spafford, Behr         | DevOps narrative    |
| <i>Team Topologies</i>              | Skelton, Pais               | Organization design |
| <i>Kubernetes Up &amp; Running</i>  | Hightower, Burns, Beda      | K8s operations      |
| <i>Terraform: Up &amp; Running</i>  | Brikman                     | IaC patterns        |
| <i>Security DevOps</i>              | Bellomo et al.              | DevSecOps           |

## 64.14 60.14 Certification paths (optional)

| Certification                        | Focus                               |
|--------------------------------------|-------------------------------------|
| AWS Solutions Architect / DevOps Pro | AWS platform                        |
| CKA / CKAD / CKS                     | Kubernetes admin, app dev, security |
| Terraform Associate                  | IaC                                 |
| LFCS / LFCE                          | Linux                               |
| GIAC / Security+                     | Security foundations                |

Certifications validate knowledge; hands-on labs in this handbook build practical skill.

---

## 64.15 60.15 Final review questions (handbook-wide)

1. Explain the Three Ways and how Parts IX–X implement the Second and Third Ways.
  2. Design a CI pipeline listing stages from commit to production with security gates.
  3. How do SLOs, error budgets, and DORA metrics interact in organizational decision-making?
  4. Compare Terraform, Kubernetes, and GitOps responsibilities in platform architecture.
  5. Describe a supply chain security program covering SBOM, signing, and admission control.
  6. When would you choose Elasticsearch vs Loki for logging?
  7. What capabilities belong in an Internal Developer Platform vs team-owned services?
- 

## 64.16 60.16 Chapter summary

- This appendix provides command cheatsheets for daily DevOps operations.
  - The glossary defines key terms used throughout the handbook.
  - The chapter index and reading list support continued learning beyond these pages.
-

# 65 AWS Handbook — Master Table of Contents

---

**Target length:** ~800 pages (~320,000 words)  
**Audience:** Beginner → Advanced AWS practitioners  
**Companion:** [DevOps Handbook](#)

---

## 65.1 Part I — Cloud Foundations (Pages 1-80)

| Ch | Title                                         | Pages |
|----|-----------------------------------------------|-------|
| 1  | Introduction to AWS & Shared Responsibility   | 1-18  |
| 2  | AWS Global Infrastructure: Regions, AZs, Edge | 19-35 |
| 3  | AWS Account Setup, Billing, and Cost Basics   | 36-55 |
| 4  | AWS CLI, CloudShell, and SDK Overview         | 56-80 |

**Written:** All 42 chapters complete

---

## 65.2 Part II — Identity & Access Management (Pages 81-160)

| Ch | Title                                        | Pages   |
|----|----------------------------------------------|---------|
| 5  | IAM Users, Groups, Roles, and Policies       | 81-105  |
| 6  | IAM Best Practices & Permission Boundaries   | 106-125 |
| 7  | IAM Identity Center (SSO) & Federation       | 126-145 |
| 8  | AWS Organizations & Service Control Policies | 146-160 |

**Written:** [Chapter 5](#)

---

## 65.3 Part III — Networking (Pages 161-280)

| Ch | Title                                         | Pages   |
|----|-----------------------------------------------|---------|
| 9  | VPC Fundamentals: CIDR, Subnets, Route Tables | 161-185 |
| 10 | Internet Gateway, NAT, and Routing            | 186-205 |
| 11 | Security Groups vs NACLs                      | 206-220 |
| 12 | Load Balancers: ALB, NLB, GWLB                | 221-245 |
| 13 | Route 53: DNS Records and Routing Policies    | 246-265 |
| 14 | VPC Peering, Transit Gateway, PrivateLink     | 266-280 |

## 65.4 Part IV — Compute (Pages 281-400)

| Ch | Title | Pages   |
|----|-------|---------|
| 15 |       | 281-305 |

|    |                                                   |         |
|----|---------------------------------------------------|---------|
|    | EC2: Instance Types, AMIs, Launch Templates       |         |
| 16 | Auto Scaling & Elastic Load Balancing Integration | 306–325 |
| 17 | Lambda: Events, Concurrency, and Layers           | 326–350 |
| 18 | ECS & Fargate                                     | 351–375 |
| 19 | EKS: Managed Kubernetes on AWS                    | 376–400 |

---

## 65.5 Part V — Storage (Pages 401–480)

| Ch | Title                                        | Pages   |
|----|----------------------------------------------|---------|
| 20 | S3: Buckets, Objects, Versioning, Lifecycle  | 401–425 |
| 21 | S3 Security, Encryption, and Static Websites | 426–445 |
| 22 | EBS & EFS                                    | 446–465 |
| 23 | AWS Backup & Disaster Recovery Patterns      | 466–480 |

---

## 65.6 Part VI — Databases (Pages 481–560)

| Ch | Title                              | Pages   |
|----|------------------------------------|---------|
| 24 | RDS & Aurora                       | 481–505 |
| 25 | DynamoDB: Tables, Indexes, Streams | 506–525 |
| 26 | ElastiCache & Caching Patterns     | 526–540 |
| 27 | Database Migration (DMS) & DR      | 541–560 |

---

## 65.7 Part VII — Application Integration (Pages 561–620)

| Ch | Title                     | Pages   |
|----|---------------------------|---------|
| 28 | API Gateway               | 561–580 |
| 29 | SQS, SNS, and EventBridge | 581–605 |
| 30 | Step Functions Workflows  | 606–620 |

---

## 65.8 Part VIII — DevOps on AWS (Pages 621–700)

| Ch | Title                               | Pages   |
|----|-------------------------------------|---------|
| 31 | CloudFormation Deep Dive            | 621–645 |
| 32 | AWS CDK                             | 646–665 |
| 33 | CodePipeline, CodeBuild, CodeDeploy | 666–685 |
| 34 | Systems Manager & Parameter Store   | 686–700 |

---

## 65.9 Part IX — Observability & Operations (Pages 701-760)

| Ch | Title                                  | Pages   |
|----|----------------------------------------|---------|
| 35 | CloudWatch Metrics, Alarms, Dashboards | 701-720 |
| 36 | CloudWatch Logs & Insights             | 721-735 |
| 37 | X-Ray & Distributed Tracing            | 736-750 |
| 38 | CloudTrail & AWS Config                | 751-760 |

## 65.10 Part X — Security & Compliance (Pages 761-800)

| Ch | Title                                     | Pages   |
|----|-------------------------------------------|---------|
| 39 | KMS, Secrets Manager, ACM                 | 761-775 |
| 40 | GuardDuty, Security Hub, WAF, Shield      | 776-790 |
| 41 | Well-Architected Review & Capstone Design | 791-800 |

## 65.11 Expansion status

| Part               | Status              |
|--------------------|---------------------|
| I — Foundations    | Complete (Ch 1-4)   |
| II — IAM           | Complete (Ch 5-8)   |
| III — Networking   | Complete (Ch 9-14)  |
| IV — Compute       | Complete (Ch 15-19) |
| V — Storage        | Complete (Ch 20-23) |
| VI — Databases     | Complete (Ch 24-27) |
| VII — Integration  | Complete (Ch 28-30) |
| VIII — DevOps      | Complete (Ch 31-34) |
| IX — Observability | Complete (Ch 35-38) |
| X — Security       | Complete (Ch 39-41) |

# 66 Chapter 1: Introduction to AWS & the Shared Responsibility Model

AWS Handbook — Part I, Pages 1-18

## 66.1 1.1 What is Amazon Web Services (AWS)?

**Amazon Web Services (AWS)** is a comprehensive **cloud computing platform** offering on-demand compute, storage, databases, networking, machine learning, and 200+ other services. Instead of buying physical servers, you provision resources in minutes and pay primarily for what you use.

### 66.1.1 Why AWS matters for DevOps

- **API-driven** — Every service is controllable via CLI, SDK, and IaC.
- **Global scale** — Deploy in multiple regions for resilience and latency.
- **Managed services** — Offload undifferentiated heavy lifting (RDS, EKS, Lambda).
- **Ecosystem** — Largest cloud market share; extensive documentation and community.

## 66.2 1.2 Cloud service models

| Model | You manage          | Provider manages              | AWS examples                  |
|-------|---------------------|-------------------------------|-------------------------------|
| IaaS  | OS, apps, data      | Hardware, network, hypervisor | EC2, VPC, EBS                 |
| PaaS  | Apps, data          | Runtime, OS, infra            | Elastic Beanstalk, App Runner |
| SaaS  | Configuration, data | Everything else               | WorkSpaces, Chime             |
| FaaS  | Function code       | Everything else               | Lambda                        |

**Containers** blur lines: ECS/EKS give you orchestration (between IaaS and PaaS).

## 66.3 1.3 The Shared Responsibility Model

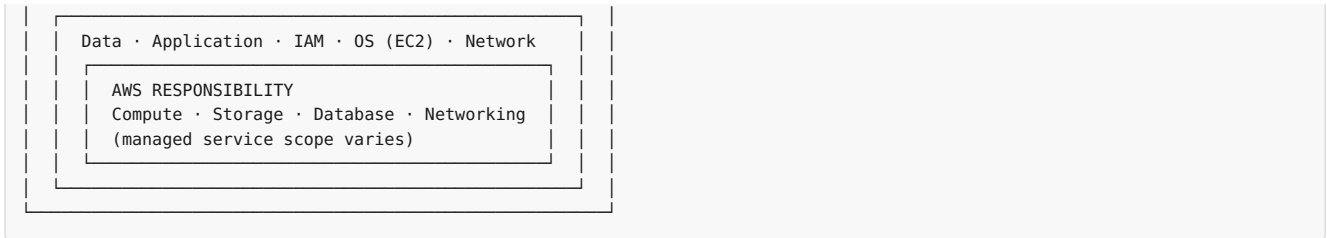
Security and compliance are **shared** between AWS and the customer. The boundary shifts by service type.

### 66.3.1 AWS is responsible for Security OF the Cloud

- Physical data centers
- Hardware and global network
- Virtualization layer
- Managed service patching (for fully managed services)

### 66.3.2 You are responsible for Security IN the Cloud

- Data classification and encryption choices
- IAM users, roles, and policies
- Operating system patches (on EC2)
- Network configuration (security groups, NACLs)
- Application code security



### 66.3.3 Example: RDS vs EC2 MySQL

| Task                      | EC2 + self-managed MySQL | Amazon RDS          |
|---------------------------|--------------------------|---------------------|
| Patch MySQL               | <b>You</b>               | <b>AWS</b>          |
| Patch OS                  | <b>You</b>               | <b>AWS</b>          |
| Configure security groups | <b>You</b>               | <b>You</b>          |
| Encrypt data at rest      | <b>You</b> (configure)   | <b>You</b> (enable) |
| Physical security         | <b>AWS</b>               | <b>AWS</b>          |

## 66.4 1.4 Core AWS concepts

| Concept                       | Definition                                                               |
|-------------------------------|--------------------------------------------------------------------------|
| <b>Region</b>                 | Geographic area (e.g., <code>us-east-1</code> , <code>eu-west-1</code> ) |
| <b>Availability Zone (AZ)</b> | Isolated data center within a region                                     |
| <b>Account</b>                | Billing and security boundary (12-digit ID)                              |
| <b>Resource</b>               | Any AWS entity (EC2 instance, S3 bucket, etc.)                           |
| <b>ARN</b>                    | Amazon Resource Name — unique identifier                                 |
| <b>Tag</b>                    | Key-value metadata on resources                                          |

Example ARN:

```
arn:aws:s3:::my-bucket
arn:aws:ec2:us-east-1:123456789012:instance/i-0abcd1234
```

## 66.5 1.5 AWS console, CLI, and IaC

Three ways to interact with AWS:

1. **Management Console** — Web UI; great for learning and ad-hoc tasks.
2. **AWS CLI** — Scriptable; essential for automation.
3. **Infrastructure as Code** — CloudFormation, CDK, Terraform for reproducible environments.

DevOps engineers spend most time in **CLI + IaC**, using the console for debugging.

## 66.6 1.6 Pricing fundamentals

- **On-Demand** — Pay per hour/second; no commitment.
- **Reserved Instances / Savings Plans** — 1-3 year commitment; lower cost.

- **Spot Instances** — Unused capacity; cheap but can be interrupted.
- **Free Tier** — Limited free usage for 12 months (and some always-free services).

**Always set billing alarms** (AWS Budgets) on new accounts.

---

## 66.7 1.7 Well-Architected Framework (preview)

AWS defines six pillars for good architecture (covered throughout this handbook):

1. Operational Excellence
  2. Security
  3. Reliability
  4. Performance Efficiency
  5. Cost Optimization
  6. Sustainability
- 

## 66.8 1.8 Chapter summary

- AWS provides on-demand cloud infrastructure and managed services.
  - **Shared responsibility** splits security between AWS and you—know your side.
  - Learn **regions, accounts, ARNs, and tags** early.
  - Use CLI and IaC for production; console for exploration.
- 

## 66.9 Lab 1.1 — Create your AWS account

1. Sign up at <https://aws.amazon.com> (use a dedicated email).
  2. Enable **MFA** on the root account; do not use root daily.
  3. Create an **IAM admin user** for daily work.
  4. Set a **billing budget** alert at \$10 or your comfort threshold.
  5. Explore the console: EC2, S3, IAM dashboards.
- 

## 66.10 Review questions

1. What is the difference between IaaS and PaaS? Give one AWS example of each.
  2. Who patches the OS on an EC2 instance?
  3. What is an ARN used for?
  4. Name two AWS pricing models.
- 

Next: [Chapter 2 — Global Infrastructure](#)



# 67 Chapter 2: AWS Global Infrastructure — Regions, AZs, and Edge

AWS Handbook — Part I, Pages 19–35

## 67.1 2.1 Global infrastructure overview

AWS operates a worldwide network of data centers organized into:

Region (e.g., us-east-1)

- Availability Zone A (us-east-1a)
- Availability Zone B (us-east-1b)
- Availability Zone C (us-east-1c)
- ... (typically 3+ AZs per region)

Edge Locations (200+) — CloudFront, Route 53, etc.

## 67.2 2.2 Regions

A **Region** is a separate geographic area. Each region is **fully isolated** from others for most services.

### 67.2.1 Choosing a region

| Factor               | Guidance                                             |
|----------------------|------------------------------------------------------|
| Latency              | Place resources near users                           |
| Compliance           | Data residency (GDPR, etc.) may require EU regions   |
| Service availability | New services launch in select regions first          |
| Cost                 | Pricing varies by region ( us-east-1 often cheapest) |
| Disaster recovery    | Use a second region for critical workloads           |

**Naming:** us-east-1 (N. Virginia), eu-west-1 (Ireland), ap-southeast-1 (Singapore).

### 67.2.2 Regional vs global services

| Regional (must specify region) | Global (single endpoint)  |
|--------------------------------|---------------------------|
| EC2, RDS, VPC, Lambda          | IAM, Route 53, CloudFront |
| S3 (bucket tied to region)     | AWS Organizations         |

## 67.3 2.3 Availability Zones (AZs)

An **AZ** is one or more discrete data centers with independent power, networking, and cooling within a region. AZs are connected by **low-latency private fiber**.

- Deploy across **multiple AZs** for high availability.
- AZ names like us-east-1a are **account-specific mappings**— 1a in your account may not be the same physical DC as 1a in another account.

### 67.3.1 Multi-AZ patterns

| Pattern         | Use case                          |
|-----------------|-----------------------------------|
| Active-passive  | RDS Multi-AZ failover             |
| Active-active   | ALB spanning AZs with EC2 in each |
| Stretch cluster | EKS node groups per AZ            |

## 67.4 2.4 Local Zones, Wavelength, and Outposts

| Type        | Purpose                                                 |
|-------------|---------------------------------------------------------|
| Local Zones | Extend region closer to metro areas (ultra-low latency) |
| Wavelength  | 5G edge at carrier locations                            |
| Outposts    | AWS hardware on-premises; hybrid cloud                  |

These are specialized; start with standard regions + AZs.

## 67.5 2.5 Edge locations and CloudFront

**Edge locations** cache content close to users via **Amazon CloudFront** (CDN). Route 53 and AWS Shield also use the edge network.

Benefits: - Lower latency for static assets - Reduced origin load - DDoS protection at edge (with Shield)

## 67.6 2.6 Fault isolation design principles

1. **Never rely on a single AZ** for production.
2. **Design for regional failure** for mission-critical systems (multi-region active-active or pilot light).
3. **Use managed services** that handle AZ redundancy (RDS Multi-AZ, S3 cross-AZ durability).
4. **Test failover** regularly.

### 67.6.1 S3 durability vs availability

- **S3 Standard** — 99.999999999% (11 nines) durability across AZs in a region.
- **Availability** — Different metric; rare regional outages can still occur.

## 67.7 2.7 Service quotas (limits)

Each account has **default quotas** per region (e.g., EC2 vCPUs, VPCs per region). Request increases via **Service Quotas** console. Plan capacity before large deployments.

## 67.8 2.8 Chapter summary

- **Regions** are geographic; **AZs** are isolated DCs within a region.
- Deploy production workloads across **multiple AZs**.
- **Edge locations** power CloudFront and global services.
- Know **regional vs global** services when designing architecture.

---

## 67.9 Lab 2.1 — Explore regions in CLI

```
aws configure # Set default region
aws ec2 describe-availability-zones --region us-east-1
aws ec2 describe-regions --output table
```

Document which region you will use for labs and why.

---

## 67.10 Review questions

1. How many AZs should a highly available web app use at minimum?
  2. Is IAM a regional or global service?
  3. What AWS service uses edge locations for caching?
  4. Why might you choose `eu-central-1` over `us-east-1`?
- 

Next: Part II — [Chapter 5: IAM Fundamentals](#)

# 68 Chapter 3: AWS Account Setup, Billing, and Cost Basics

AWS Handbook — Part I, Pages 36-55

## 68.1 3.1 Creating a secure AWS account

### 68.1.1 Step-by-step

1. **Sign up** at <https://aws.amazon.com> with a dedicated email (not personal if possible for work).
2. **Root account** — Full admin access. Enable **MFA immediately**.
3. **Do not use root** for daily tasks.
4. Create an **IAM admin user** or use **IAM Identity Center** for workforce access.
5. Set up **billing alerts** before launching resources.

### 68.1.2 Root account checklist

- ☐ MFA enabled (hardware key preferred)
- ☐ Strong unique password in password manager
- ☐ No access keys on root
- ☐ Contact info and alternate contacts set
- ☐ AWS Budgets alert configured

## 68.2 3.2 AWS Free Tier

| Type          | Examples                                           |
|---------------|----------------------------------------------------|
| 12-month free | 750 hrs/month t2/t3.micro EC2, 5 GB S3, RDS limits |
| Always free   | Lambda 1M requests/month, DynamoDB 25 GB           |
| Short trials  | Some services have limited trials                  |

**Warning:** Free tier expires; resources left running can incur charges. Always set billing alarms.

## 68.3 3.3 Billing and Cost Management

| Tool                      | Purpose                                     |
|---------------------------|---------------------------------------------|
| Billing Dashboard         | Current month spend                         |
| Cost Explorer             | Historical analysis, forecasts, filters     |
| AWS Budgets               | Email/SNS alerts at thresholds              |
| Cost & Usage Report (CUR) | Granular export to S3 for analysis          |
| Pricing Calculator        | Estimate architecture costs before building |

### 68.3.1 Budget example

Create a monthly budget of \$20 with alerts at 50%, 80%, and 100%:

```
aws budgets create-budget \
```

```
--account-id 123456789012 \
--budget file://budget.json \
--notifications-with-subscribers file://notifications.json
```

---

## 68.4 3.4 Cost optimization fundamentals

1. **Tag everything** — Environment, Project, Owner, CostCenter
2. **Right-size** — Use Compute Optimizer recommendations
3. **Stop dev resources** — Schedule EC2/RDS shutdown nights/weekends
4. **Use Spot** for fault-tolerant batch workloads
5. **S3 lifecycle** — Move old data to IA/Glacier
6. **Reserved capacity** — 1-3 year Savings Plans for steady workloads
7. **Delete unused** — EBS volumes, Elastic IPs, old snapshots

---

## 68.5 3.4 Consolidated billing with Organizations

**AWS Organizations** lets you manage multiple accounts under one payer account:

- **Consolidated billing** — Single invoice, volume discounts
- **SCPs** — Guardrails on what accounts can do
- **Separate accounts** per environment (dev/staging/prod) or per team

Recommended for any serious AWS usage.

---

## 68.6 3.5 Support plans (overview)

| Plan       | Best for                                        |
|------------|-------------------------------------------------|
| Basic      | Free; account and billing support               |
| Developer  | Business hours email support                    |
| Business   | 24/7 phone, < 1 hr response for production down |
| Enterprise | TAM, < 15 min critical response                 |

---

## 68.7 3.6 Chapter summary

- Secure the **root account** with MFA; use IAM for daily work.
- Set **budgets and alerts** on day one.
- **Tag** resources and review Cost Explorer weekly during learning.

---

## 68.8 Lab 3.1

1. Enable MFA on root.
  2. Create IAM admin user `admin-yourname`.
  3. Create a \$10 monthly budget with email alert.
  4. Enable Cost Explorer (may take 24h for data).
-

Next: [Chapter 6 — IAM Best Practices](#)

# 69 Chapter 4: AWS CLI, CloudShell, and SDK Overview

AWS Handbook — Part I, Pages 56–80

## 69.1 4.1 Why the CLI matters for DevOps

The **AWS Management Console** is excellent for exploration, but production workflows demand **repeatability**, **version control**, and **automation**. The **AWS Command Line Interface (CLI)** is the universal tool that every AWS service exposes through a consistent interface. Combined with **CloudShell** (browser-based shell) and **SDKs** (programmatic libraries), you can script deployments, integrate CI/CD pipelines, and build applications that call AWS APIs directly.

DevOps engineers typically spend 70–80% of their AWS interaction time in CLI and IaC tools, reserving the console for debugging and one-off investigations.

## 69.2 4.2 Installing and configuring the AWS CLI

### 69.2.1 Installation options

| Platform    | Method                                                 |
|-------------|--------------------------------------------------------|
| Linux/macOS | <code>curl</code> installer or package manager         |
| Windows     | MSI installer or <code>winget</code>                   |
| CI/CD       | Pre-installed on GitHub Actions runners; use in Docker |
| CloudShell  | Pre-installed; no setup required                       |

### 69.2.2 AWS CLI v2 vs v1

Always use **CLI v2**. It supports SSO, improved pagination, `aws s3` performance improvements, and interactive `aws configure sso`.

### 69.2.3 Initial configuration

```
Interactive setup with long-term access keys (avoid for daily use; prefer SSO)
aws configure
AWS Access Key ID: AKIA...
AWS Secret Access Key: ...
Default region name: us-east-1
Default output format: json

Verify identity
aws sts get-caller-identity
```

Configuration files live at:

| File                            | Purpose                             |
|---------------------------------|-------------------------------------|
| <code>~/.aws/credentials</code> | Access keys per profile             |
| <code>~/.aws/config</code>      | Region, output format, SSO settings |

### 69.2.4 Named profiles

```
aws configure --profile dev
aws s3 ls --profile dev

Environment variable override
export AWS_PROFILE=prod
aws ec2 describe-instances --query 'Reservations[].Instances[].InstanceId'
```

### 69.2.5 SSO configuration (recommended for humans)

```
aws configure sso
SSO session name: my-org
SSO start URL: https://my-org.awsapps.com/start
SSO region: us-east-1
CLI default client Region: us-east-1

aws sso login --profile dev-admin
aws sts get-caller-identity --profile dev-admin
```

## 69.3 4.3 AWS CloudShell

**AWS CloudShell** is a browser-based shell environment pre-authenticated with your console credentials. It includes the AWS CLI v2, Python, Node.js, Git, and 1 GB persistent storage per region.

### 69.3.1 When to use CloudShell

| Use CloudShell               | Use local CLI              |
|------------------------------|----------------------------|
| Quick API calls from console | Daily development          |
| No local install available   | CI/CD pipelines            |
| Temporary troubleshooting    | Scripts in version control |
| Learning and labs            | Large file transfers       |

### 69.3.2 CloudShell features

- **Automatic credentials** — Uses your current console session IAM permissions.
- **Persistent home directory** — Files survive across sessions (per region).
- **Network access** — Can reach public AWS APIs; VPC access requires additional setup.
- **Limitations** — 1 GB storage, no inbound connections, session timeout after inactivity.

```
Example: list S3 buckets from CloudShell
aws s3 ls

Install a Python package (persists in home directory)
pip install --user boto3
```

## 69.4 4.4 CLI essentials: structure and patterns

Every AWS CLI command follows this pattern:

```
aws <service> <operation> [--parameters] [--global-options]
```

### 69.4.1 Global options

| Option   | Purpose                 |
|----------|-------------------------|
| --region | Override default region |



|                             |                                                                                |
|-----------------------------|--------------------------------------------------------------------------------|
| <code>--profile</code>      | Use named profile                                                              |
| <code>--output</code>       | <code>json</code> , <code>yaml</code> , <code>text</code> , <code>table</code> |
| <code>--query</code>        | JMESPath filter on response                                                    |
| <code>--dry-run</code>      | Validate without executing (where supported)                                   |
| <code>--no-cli-pager</code> | Disable pager for scripts                                                      |

### 69.4.2 JMESPath query examples

```
List running EC2 instance IDs only
aws ec2 describe-instances \
 --filters "Name=instance-state-name,Values=running" \
 --query 'Reservations[].Instances[].InstanceId' \
 --output text

Get bucket names and creation dates as a table
aws s3api list-buckets \
 --query 'Buckets[].{Name:Name,Created:CreationDate}' \
 --output table
```

### 69.4.3 Pagination

```
CLI v2 auto-paginates most list operations
aws ec2 describe-instances --output json

Manual control
aws s3api list-objects-v2 --bucket my-bucket --max-items 100
```

### 69.4.4 Waiters

```
aws ec2 run-instances --image-id ami-0c55b159cbfafa1f0 --instance-type t3.micro ...

aws ec2 wait instance-running --instance-ids i-0abc123
aws ec2 wait instance-status-ok --instance-ids i-0abc123
```

## 69.5 4.5 AWS SDK overview

SDKs wrap AWS APIs in language-native libraries. They handle request signing, retries, and pagination.

| SDK                              | Language | Package                                   |
|----------------------------------|----------|-------------------------------------------|
| <b>Boto3</b>                     | Python   | <code>pip install boto3</code>            |
| <b>AWS SDK for JavaScript v3</b> | Node.js  | <code>@aws-sdk/client-s3</code>           |
| <b>AWS SDK for Go v2</b>         | Go       | <code>github.com/aws/aws-sdk-go-v2</code> |
| <b>AWS SDK for .NET</b>          | C#       | <code>AWSSDK.S3</code>                    |

### 69.5.1 Credential chain (SDK and CLI)

Both resolve credentials in this order:

1. Environment variables ( `AWS_ACCESS_KEY_ID` , `AWS_SECRET_ACCESS_KEY` )
2. Shared credentials file ( `~/.aws/credentials` )
3. IAM role (EC2 instance profile, ECS task role, Lambda execution role)
4. SSO cached credentials

**Best practice:** Applications on AWS use **IAM roles**, never hard-coded keys.

### 69.5.2 Boto3 example

```
import boto3

s3 = boto3.client("s3", region_name="us-east-1")
response = s3.list_buckets()
for bucket in response["Buckets"]:
 print(bucket["Name"])
```

### 69.5.3 SDK with assumed role

```
import boto3

sts = boto3.client("sts")
assumed = sts.assume_role(
 RoleArn="arn:aws:iam::123456789012:role/CrossAccountRole",
 RoleSessionName="my-session",
)
creds = assumed["Credentials"]
client = boto3.client(
 "s3",
 aws_access_key_id=creds["AccessKeyId"],
 aws_secret_access_key=creds["SecretAccessKey"],
 aws_session_token=creds["SessionToken"],
)
```

## 69.6 4.6 Terraform provider configuration

Terraform uses the AWS provider, which shares the same credential chain as the CLI.

```
terraform {
 required_providers {
 aws = {
 source = "hashicorp/aws"
 version = "~> 5.0"
 }
 }
}

provider "aws" {
 region = "us-east-1"
 profile = "dev"
}

resource "aws_s3_bucket" "logs" {
 bucket = "my-app-logs-${data.aws_caller_identity.current.account_id}"
}

data "aws_caller_identity" "current" {}
```

## 69.7 4.7 CLI security best practices

| Practice                                                                 | Rationale                           |
|--------------------------------------------------------------------------|-------------------------------------|
| Use SSO, not long-term access keys                                       | Keys can leak; SSO sessions expire  |
| Never commit credentials to Git                                          | Use IAM roles in CI/CD              |
| Use <code>--profile</code> or <code>AWS_PROFILE</code>                   | Separate dev/staging/prod           |
| Enable MFA for sensitive operations                                      | Extra layer for destructive actions |
| Rotate any remaining access keys                                         | Limit blast radius                  |
| Use <code>aws sts get-caller-identity</code> before destructive commands | Confirm correct account             |

---

## 69.8 4.8 Useful CLI productivity tips

```
Auto-complete (bash)
complete -C '/usr/local/bin/aws_completer' aws

Generate CLI skeleton for complex commands
aws ec2 run-instances --generate-cli-skeleton input > run-instance.json
Edit JSON, then:
aws ec2 run-instances --cli-input-json file://run-instance.json

Filter help by service
aws ec2 help | grep describe

Export credentials for third-party tools (short-lived)
eval "$(aws configure export-credentials --profile dev --format env)"
```

---

## 69.9 4.9 Chapter summary

- The **AWS CLI** is the foundation of automation; configure profiles and prefer **SSO**.
- **CloudShell** provides instant CLI access from the console without local setup.
- **SDKs** (Boto3, etc.) use the same credential chain; applications on AWS should use **IAM roles**.
- Master **JMESPath queries**, **waiters**, and **pagination** for efficient scripting.
- Terraform and other IaC tools inherit CLI credential configuration.

---

## 69.10 Lab 4.1 — CLI and CloudShell workflow

1. Install AWS CLI v2 locally (or open **CloudShell** in the console).
2. Configure an SSO profile: `aws configure sso`.
3. Run `aws sts get-caller-identity` and note your account ID and ARN.
4. List all regions: `aws ec2 describe-regions --query 'Regions[].RegionName' --output table`.
5. Create a test S3 bucket (unique name): `aws s3 mb s3://cli-lab-$(aws sts get-caller-identity --query Account --output text)-$(date +%s)`.
6. Upload a file: `echo "hello" > test.txt && aws s3 cp test.txt s3://YOUR-BUCKET/`.
7. Delete the bucket and objects when done.

## 69.11 Lab 4.2 — Boto3 script

1. On an EC2 instance with an instance profile (or locally with configured credentials), write a Python script that lists all EC2 instances and prints ID, type, and state.
2. Run it with `python3 list_instances.py`.
3. Extend the script to tag instances missing an `Owner` tag.

---

## 69.12 Review questions

1. What is the difference between `~/.aws/credentials` and `~/.aws/config`?
  2. How does the SDK credential chain resolve credentials on an EC2 instance with an instance profile?
  3. What JMESPath query would return only S3 bucket names from `aws s3api list-buckets`?
  4. When should you use CloudShell instead of a local CLI installation?
  5. Why is SSO preferred over long-term access keys for human users?
-

Next: [Chapter 5 — IAM Fundamentals](#)

# 70 Chapter 5: IAM Users, Groups, Roles, and Policies

AWS Handbook — Part II, Pages 81–105

## 70.1 5.1 What is IAM?

**AWS Identity and Access Management (IAM)** controls **who** can access **which** AWS resources and **how**. Every API call to AWS is authenticated and authorized through IAM (except public resources like anonymous S3 access). IAM is **global**—users and roles exist at the account level, not per region.

## 70.2 5.2 IAM identity types

| Identity  | Use case                                                             |
|-----------|----------------------------------------------------------------------|
| Root user | Account owner; created at signup. <b>Avoid for daily use.</b>        |
| IAM user  | Long-term credentials for humans or legacy apps                      |
| IAM role  | Temporary credentials; assumed by users, services, or other accounts |
| IAM group | Collection of users with shared permissions                          |

**Best practice:** Humans use **IAM Identity Center (SSO)** or roles with MFA. Applications and EC2/Lambda use **roles**, not access keys on users.

## 70.3 5.3 Policies — the core of permissions

A **policy** is a JSON document defining **Allow** or **Deny** for **actions** on **resources** under **conditions**.

### 70.3.1 Policy structure

```
{
 "Version": "2012-10-17",
 "Statement": [
 {
 "Sid": "AllowS3Read",
 "Effect": "Allow",
 "Action": [
 "s3:GetObject",
 "s3:ListBucket"
],
 "Resource": [
 "arn:aws:s3:::my-app-bucket",
 "arn:aws:s3:::my-app-bucket/*"
],
 "Condition": {
 "IpAddress": {
 "aws:SourceIp": "203.0.113.0/24"
 }
 }
 }
]
}
```

### 70.3.2 Policy types

|  |  |
|--|--|
|  |  |
|--|--|

| Type                  | Attached to                         |
|-----------------------|-------------------------------------|
| Identity-based        | Users, groups, roles                |
| Resource-based        | S3 buckets, SNS topics, etc.        |
| Permission boundaries | Max permissions for identity        |
| SCP                   | AWS Organizations — account ceiling |
| Session policies      | Limit role session scope            |

**Evaluation logic:** Explicit **Deny** always wins. Default is implicit deny until Allow.

## 70.4 5.4 Users and groups

### 70.4.1 Create a developer group (example workflow)

1. Create group `Developers`.
2. Attach policy `PowerUserAccess` (or custom least-privilege policy).
3. Create users; add to group.
4. Enable **MFA** for console access.
5. Provide **programmatic access** only if needed (prefer roles).

```
aws iam create-group --group-name Developers
aws iam attach-group-policy --group-name Developers \
 --policy-arn arn:aws:iam::aws:policy/PowerUserAccess
aws iam create-user --user-name alice
aws iam add-user-to-group --user-name alice --group-name Developers
```

## 70.5 5.5 Roles and trust policies

A **role** has two policies:

1. **Trust policy** — Who can assume the role
2. **Permission policy** — What the role can do

### 70.5.1 EC2 instance role example

Trust policy (allows EC2 service to assume):

```
{
 "Version": "2012-10-17",
 "Statement": [
 {
 "Effect": "Allow",
 "Principal": { "Service": "ec2.amazonaws.com" },
 "Action": "sts:AssumeRole"
 }
]
}
```

Attach permission policy allowing `s3:GetObject` on a specific bucket. The EC2 instance receives **temporary credentials** via instance metadata—no hardcoded keys.

### 70.5.2 Assume role (cross-account or CLI)

```
aws sts assume-role \
 --role-arn arn:aws:iam::123456789012:role/DeployRole \
 --role-session-name deploy-session
```

---

## 70.6 5.6 AWS managed vs customer managed policies

| Type             | Description                                                                          |
|------------------|--------------------------------------------------------------------------------------|
| AWS managed      | Maintained by AWS ( <code>ReadOnlyAccess</code> , <code>AdministratorAccess</code> ) |
| Customer managed | You create and reuse across identities                                               |
| Inline           | Embedded in single user/group/role; harder to reuse                                  |

Prefer **customer managed** policies for team standards; avoid `AdministratorAccess` except break-glass accounts.

---

## 70.7 5.7 Least privilege in practice

1. Start with **read-only** access; add permissions as needed.
2. Use **IAM Access Analyzer** to find overly broad access.
3. Scope resources with ARNs, not `"Resource": "*" .`
4. Use **conditions** (MFA present, source IP, tag-based).
5. Rotate or eliminate **access keys**; prefer roles.

### 70.7.1 Tag-based access (ABAC)

```
"Condition": {
 "StringEquals": {
 "aws:PrincipalTag/Department": "${aws:ResourceTag/Department}"
 }
}
```

Users tagged `Department=Engineering` access only resources tagged the same.

---

## 70.8 5.8 Common IAM actions for DevOps

| Action prefix                 | Service                           |
|-------------------------------|-----------------------------------|
| <code>ec2:*</code>            | EC2                               |
| <code>s3:*</code>             | S3                                |
| <code>iam:PassRole</code>     | Allow attaching roles to services |
| <code>sts:AssumeRole</code>   | Switch roles                      |
| <code>cloudformation:*</code> | Stack operations                  |

`iam:PassRole` is often required when CI/CD deploys resources that need an instance/task role.

---

## 70.9 5.9 IAM Access Analyzer & CloudTrail

- **Access Analyzer** — Identifies resources shared externally.
- **CloudTrail** — Logs every IAM API call for audit.

Enable CloudTrail in all regions for security accounts.

---

## 70.10 5.10 Chapter summary

- IAM controls authentication and authorization globally.
  - Use **groups** for humans, **roles** for services and temporary access.
  - Policies are JSON with Effect, Action, Resource, Condition.
  - Apply **least privilege**; avoid root and long-lived access keys.
- 

## 70.11 Lab 5.1 — IAM hands-on

1. Create IAM user `lab-user` with MFA.
  2. Create customer managed policy allowing `s3:ListBucket` and `s3:GetObject` on one bucket.
  3. Create role `EC2-S3-ReadRole` with trust for EC2 and the S3 read policy.
  4. Launch EC2 with the role; verify `aws s3 ls` works without configured keys.
  5. Review effective permissions in **IAM Policy Simulator**.
- 

## 70.12 Review questions

1. What is the difference between an IAM user and an IAM role?
  2. What happens when Allow and Deny conflict?
  3. Why should EC2 instances use roles instead of access keys?
  4. What does `sts:AssumeRole` do?
- 

*Continue: Chapter 6 — IAM Best Practices (outline in TOC)*



# 71 Chapter 6: IAM Best Practices and Permission Boundaries

AWS Handbook — Part II, Pages 106–125

## 71.1 6.1 The IAM security baseline

Follow these rules in every AWS account:

- 1. **Root account** — MFA only; no access keys; break-glass use only
- 2. **No long-lived access keys** for humans — use SSO + temporary credentials
- 3. **Least privilege** — Grant minimum permissions required
- 4. **MFA** — Required for console access and sensitive API calls
- 5. **Rotate keys** — If keys must exist, rotate every 90 days max
- 6. **Audit** — CloudTrail enabled in all regions; review Access Analyzer findings

## 71.2 6.2 Prefer roles over users for applications

| Identity            | Use for                                |
|---------------------|----------------------------------------|
| IAM user + keys     | Legacy; avoid for new projects         |
| IAM role            | EC2, Lambda, ECS, cross-account, CI/CD |
| IAM Identity Center | Human workforce access                 |

### 71.2.1 CI/CD role pattern

```
{
 "Version": "2012-10-17",
 "Statement": [{
 "Effect": "Allow",
 "Principal": { "Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.githubusercontent.com" },
 "Action": "sts:AssumeRoleWithWebIdentity",
 "Condition": {
 "StringEquals": {
 "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
 },
 "StringLike": {
 "token.actions.githubusercontent.com:sub": "repo:myorg/myrepo:*"
 }
 }
 }]
}
```

GitHub Actions assumes this role—no static AWS keys in GitHub Secrets.

## 71.3 6.3 Permission boundaries

A **permissions boundary** sets the **maximum** permissions an identity can have—even if policies grant more.

Use for:

- Delegating IAM admin to developers without full account access
- Sandbox accounts with caps

```
aws iam put-user-permissions-boundary \
--user-name developer \
--permissions-boundary arn:aws:iam::aws:policy/PowerUserAccess
```

---

## 71.4 6.4 Policy writing tips

### 71.4.1 Start with AWS managed, refine to customer managed

```
{
 "Version": "2012-10-17",
 "Statement": [{
 "Sid": "DeployToSpecificBucket",
 "Effect": "Allow",
 "Action": ["s3:PutObject", "s3:GetObject"],
 "Resource": "arn:aws:s3::my-app-artifacts-prod/*"
 }]
}
```

### 71.4.2 Use IAM Policy Simulator

Test policies before deploying: AWS Console → IAM → Policy Simulator.

### 71.4.3 Avoid these anti-patterns

- "Action": "\*" on production roles
- "Resource": "\*" when ARNs are known
- Attaching AdministratorAccess to CI roles
- Sharing one IAM user across a team

---

## 71.5 6.5 Service-linked roles

AWS creates **service-linked roles** for services (e.g., AWSServiceRoleForElasticLoadBalancing). Do not delete these unless AWS docs say it's safe.

---

## 71.6 6.6 Cross-account access

**Pattern:** Account A (workload) trusts Account B (CI/CD) via role assumption.

```
From CI in account B
aws sts assume-role \
--role-arn arn:aws:iam::WORKLOAD_ACCOUNT:role/DeployRole \
--role-session-name github-deploy
```

Use **AWS Organizations** SCPs to prevent privilege escalation across accounts.

---

## 71.7 6.7 Chapter summary

- Humans: **SSO + MFA**. Apps: **roles**. CI: **OIDC federation**.
  - Write **scoped customer-managed policies**; test with Policy Simulator.
  - Use **permission boundaries** and **SCPs** for guardrails.
-

## 71.8 Lab 6.1

1. Create a CI/CD role trust policy for GitHub OIDC (or a second AWS account).
2. Attach a least-privilege policy for S3 deploy only.
3. Run Access Analyzer and remediate any `Public` findings.

---

Next: [\*Chapter 9 — VPC Fundamentals\*](#)

# 72 Chapter 7: IAM Identity Center (SSO) & Federation

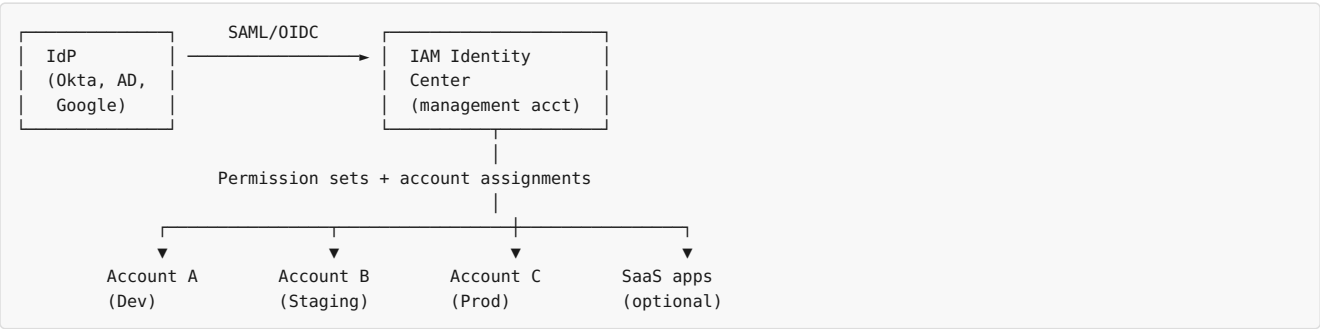
AWS Handbook — Part II, Pages 126–145

## 72.1 7.1 The problem with IAM users at scale

Managing hundreds of **IAM users** across dozens of AWS accounts does not scale. Each user needs credentials, MFA enrollment, password rotation, and offboarding when they leave. **IAM Identity Center** (formerly AWS Single Sign-On) centralizes workforce access: users authenticate once through your corporate identity provider and receive temporary credentials to AWS accounts and applications.

For organizations with multiple accounts, Identity Center is the **recommended** human access pattern, replacing per-account IAM users.

## 72.2 7.2 IAM Identity Center architecture



### 72.2.1 Key components

| Component                | Description                                         |
|--------------------------|-----------------------------------------------------|
| Identity Center instance | Deployed in a home region; one per organization     |
| Identity source          | Built-in directory, or federation with external IdP |
| Permission set           | Template of IAM policies assigned to users/groups   |
| Account assignment       | Maps user/group + permission set → AWS account      |
| Access portal            | <code>https://d-xxxxxxxxx.awsapps.com/start</code>  |

## 72.3 7.3 Permission sets

A **permission set** is a collection of IAM policies that define what a user can do in a target account. When assigned, Identity Center creates an **IAM role** in the target account (prefixed `AWSReservedSSO_`).

### 72.3.1 Common permission sets

| Permission set                   | Typical use                |
|----------------------------------|----------------------------|
| <code>AdministratorAccess</code> | Platform team, break-glass |
|                                  |                            |

|                          |                             |
|--------------------------|-----------------------------|
| PowerUserAccess          | Developers (no IAM changes) |
| ReadOnlyAccess           | Auditors, observers         |
| Custom (least privilege) | Application-specific access |

### 72.3.2 Custom permission set example (Terraform)

```
resource "aws_ssoadmin_permission_set" "developer" {
 name = "DeveloperAccess"
 instance_arn = tolist(data.aws_ssoadmin_instances.main.arns)[0]
 session_duration = "PT4H" # ISO 8601 duration: 4 hours
}

resource "aws_ssoadmin_managed_policy_attachment" "developer_poweruser" {
 instance_arn = tolist(data.aws_ssoadmin_instances.main.arns)[0]
 permission_set_arn = aws_ssoadmin_permission_set.developer.arn
 managed_policy_arn = "arn:aws:iam::aws:policy/PowerUserAccess"
}

resource "aws_ssoadmin_account_assignment" "dev_team" {
 instance_arn = tolist(data.aws_ssoadmin_instances.main.arns)[0]
 permission_set_arn = aws_ssoadmin_permission_set.developer.arn
 principal_id = aws_identitystore_group.developers.group_id
 principal_type = "GROUP"
 target_id = "123456789012" # Dev account ID
 target_type = "AWS_ACCOUNT"
}
```

## 72.4 7.4 Federation with external identity providers

### 72.4.1 SAML 2.0 federation

Most common for enterprise IdPs (Okta, Azure AD, PingFederate):

1. Configure SAML application in your IdP.
2. In Identity Center, set identity source to **External identity provider**.
3. Exchange metadata (IdP → AWS and AWS → IdP).
4. Map IdP attributes to Identity Center users and groups.

### 72.4.2 Attribute mapping

| IdP attribute | Identity Center field                 |
|---------------|---------------------------------------|
| email         | Username                              |
| givenName     | First name                            |
| sn            | Last name                             |
| memberOf      | Group membership (via SCIM or manual) |

### 72.4.3 SCIM provisioning

**System for Cross-domain Identity Management (SCIM)** automates user and group lifecycle:

- User created in Okta → automatically appears in Identity Center
- User deactivated in Okta → access revoked in AWS
- Group membership changes → permission set assignments update

Enable SCIM in Identity Center settings and configure your IdP's provisioning connector.

## 72.5 7.5 Accessing AWS with Identity Center

### 72.5.1 AWS access portal

Users visit the portal URL, authenticate via IdP, and see assigned accounts and roles. Clicking an account/role opens the console or provides CLI credentials.

### 72.5.2 CLI access

```
aws configure sso
SSO start URL: https://d-xxxxxxxxx.awsapps.com/start
SSO region: us-east-1
Account: 123456789012
Role: DeveloperAccess

aws sso login --profile dev
aws sts get-caller-identity --profile dev
```

### 72.5.3 Programmatic access from CI/CD

CI/CD should **not** use human SSO sessions. Instead:

- Use **IAM roles** with OIDC federation (GitHub Actions → AWS)
- Use **IAM roles** for EC2/ECS/Lambda
- Use short-lived credentials via `sts:AssumeRole`

## 72.6 7.6 IAM federation without Identity Center

For single-account or legacy setups, you can federate directly:

| Method                 | Use case                                  |
|------------------------|-------------------------------------------|
| SAML 2.0               | Enterprise IdP → IAM role                 |
| Web Identity (OIDC)    | Google, Facebook, Amazon login            |
| Cognito Identity Pools | Mobile/web apps with federated identities |

### 72.6.1 SAML role trust policy example

```
{
 "Version": "2012-10-17",
 "Statement": [
 {
 "Effect": "Allow",
 "Principal": {
 "Federated": "arn:aws:iam::123456789012:saml-provider/MyCorp"
 },
 "Action": "sts:AssumeRoleWithSAML",
 "Condition": {
 "StringEquals": {
 "SAML:aud": "https://signin.aws.amazon.com/saml"
 }
 }
 }
]
}
```

## 72.7 7.7 Multi-account access patterns

| Pattern | Description |
|---------|-------------|
|---------|-------------|

|                                |                                               |
|--------------------------------|-----------------------------------------------|
| <b>Account per environment</b> | dev, staging, prod as separate accounts       |
| <b>Account per team</b>        | Team A owns accounts for their services       |
| <b>Account per workload</b>    | PCI workloads isolated from general workloads |
| <b>Management account</b>      | Identity Center, Organizations, billing only  |

### 72.7.1 Session duration

Permission sets support session duration from **1 hour to 12 hours**. Shorter sessions reduce risk; longer sessions improve developer experience. Production admin access should use shorter durations (1–2 hours).

## 72.8 7.8 Security best practices

| Practice                                       | Why                                 |
|------------------------------------------------|-------------------------------------|
| Use Identity Center for all human access       | Centralized audit, MFA, offboarding |
| Assign permissions via groups, not individuals | Easier lifecycle management         |
| Enable SCIM provisioning                       | Automatic deprovisioning            |
| Use least-privilege permission sets            | Limit blast radius per account      |
| Require MFA at the IdP level                   | Defense in depth                    |
| Audit assignments regularly                    | Remove stale access                 |
| Separate break-glass admin                     | Emergency access with extra logging |

### 72.8.1 Auditing SSO access

```
CloudTrail logs AssumeRoleWithSAML and SSO portal logins
aws cloudtrail lookup-events \
 --lookup-attributes AttributeKey=EventName,AttributeValue=AssumeRoleWithSAML \
 --max-results 10
```

## 72.9 7.9 Chapter summary

- **IAM Identity Center** replaces per-account IAM users with centralized SSO across accounts.
- **Permission sets** define what users can do; **account assignments** map them to specific accounts.
- **SAML/OIDC federation** connects corporate IdPs; **SCIM** automates user lifecycle.
- CLI access via `aws configure sso` and `aws sso login`.
- CI/CD and services should use **IAM roles**, not SSO sessions.

## 72.10 Lab 7.1 — Identity Center setup

1. In the management account, enable **IAM Identity Center** (if not already enabled).
2. Create a permission set `LabReadOnly` with the `ViewOnlyAccess` AWS managed policy.
3. Create a user (or use your IdP) and assign them to a dev account with `LabReadOnly`.
4. Log in via the access portal and verify you can view but not modify resources.
5. Configure CLI SSO: `aws configure sso` and run `aws s3 ls` with the new profile.

## 72.11 Lab 7.2 — Audit permission assignments

1. List all account assignments in Identity Center (console or CLI).
  2. Identify any assignments with `AdministratorAccess` to production accounts.
  3. Document a plan to replace them with least-privilege permission sets.
- 

## 72.12 Review questions

1. What is the difference between a permission set and an account assignment?
  2. How does SCIM improve security compared to manual user management?
  3. Why should CI/CD pipelines not use Identity Center SSO sessions?
  4. What IAM role prefix does Identity Center create in target accounts?
  5. Where should MFA be enforced: the IdP, Identity Center, or both?
- 

Next: [\*Chapter 8 — AWS Organizations & SCPs\*](#)



# 73 Chapter 8: AWS Organizations & Service Control Policies

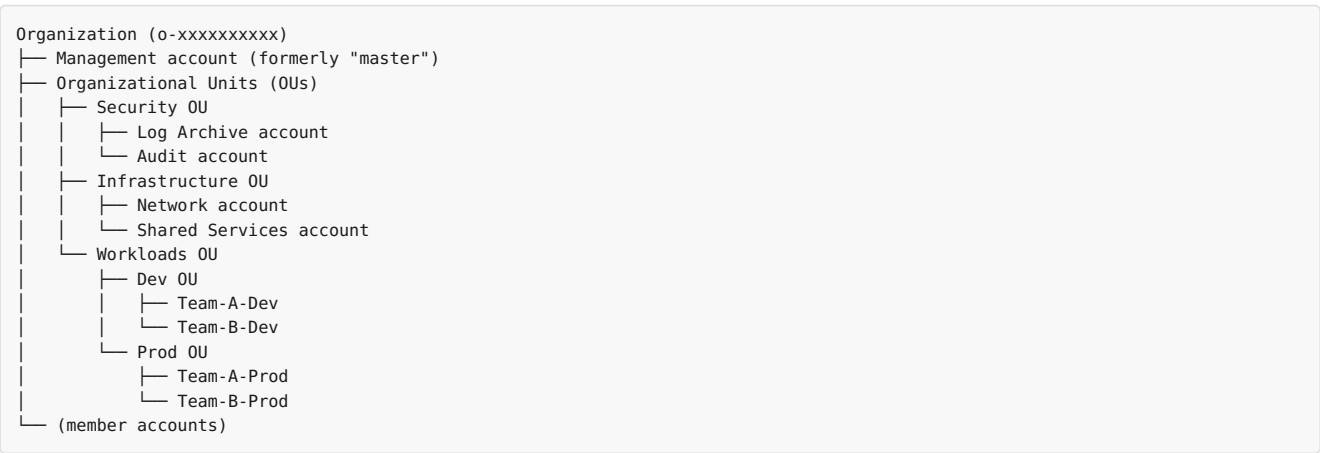
AWS Handbook — Part II, Pages 146-160

## 73.1 8.1 Why AWS Organizations?

A single AWS account works for small teams, but enterprises need **multiple accounts** for security isolation, blast-radius containment, billing separation, and compliance boundaries. **AWS Organizations** is a free service that centrally manages a hierarchy of accounts under one **organization**.

Without Organizations, you cannot use Service Control Policies, consolidated billing, or automated account vending.

## 73.2 8.2 Organization structure



### 73.2.1 Key concepts

| Concept                  | Description                                                        |
|--------------------------|--------------------------------------------------------------------|
| Management account       | Pays for all member accounts; hosts Organizations, Identity Center |
| Member account           | Standard AWS account within the organization                       |
| Organizational Unit (OU) | Grouping of accounts for policy application                        |
| Root                     | Top of the hierarchy; contains all OUs and accounts                |

## 73.3 8.3 Consolidated billing

Organizations provides a **single bill** for all accounts:

- **Volume discounts** aggregate across accounts (e.g., S3, Data Transfer).
- **Cost allocation tags** propagate for chargeback.
- **AWS Budgets** and **Cost Explorer** work across the organization.
- **Reserved Instances and Savings Plans** can be shared within the org.

# List all accounts in the organization

```
aws organizations list-accounts \
 --query 'Accounts[].{Id:Id,Name:Name,Status:Status}' \
 --output table
```

## 73.4 8.4 Service Control Policies (SCPs)

**SCPs** are guardrails that set the **maximum permissions** for accounts or OUs. They do not grant permissions—they **filter** what identity-based policies can allow.

### 73.4.1 SCP evaluation

Effective permissions = Identity policies  $\cap$  SCPs (and permission boundaries)

- **Explicit Deny** in an SCP always wins.
- If no SCP allows an action, it is implicitly denied (for SCP scope).
- SCPs affect all principals **except** the management account root user and certain service-linked roles.

### 73.4.2 Example: Deny leaving the organization

```
{
 "Version": "2012-10-17",
 "Statement": [
 {
 "Sid": "DenyLeaveOrganization",
 "Effect": "Deny",
 "Action": "organizations:LeaveOrganization",
 "Resource": "*"
 }
]
}
```

### 73.4.3 Example: Restrict regions

```
{
 "Version": "2012-10-17",
 "Statement": [
 {
 "Sid": "DenyUnapprovedRegions",
 "Effect": "Deny",
 "NotAction": [
 "iam:*",
 "organizations:*",
 "route53:*",
 "support:*",
 "cloudfront:*",
 "globalaccelerator:*"
],
 "Resource": "*",
 "Condition": {
 "StringNotEquals": {
 "aws:RequestedRegion": ["us-east-1", "eu-west-1"]
 }
 }
 }
]
}
```

### 73.4.4 Example: Deny root user actions

```
{
 "Version": "2012-10-17",
 "Statement": [
 {
 "Sid": "DenyRootUser",
 "Effect": "Deny",
 "Action": "*",

```

```

 "Resource": "*",
 "Condition": {
 "StringLike": {
 "aws:PrincipalArn": "arn:aws:iam::*:root"
 }
 }
 }
]
}

```

## 73.5 8.5 Common SCP strategies

| SCP purpose                         | Approach                                                                                  |
|-------------------------------------|-------------------------------------------------------------------------------------------|
| <b>Region lock</b>                  | Deny all actions outside approved regions                                                 |
| <b>Prevent privilege escalation</b> | Deny <code>iam:CreateUser</code> , <code>iam:AttachUserPolicy</code> in workload accounts |
| <b>Require encryption</b>           | Deny <code>S3 PutObject</code> without <code>s3:x-amz-server-side-encryption</code>       |
| <b>Block expensive services</b>     | Deny <code>ec2:RunInstances</code> for large instance types in dev                        |
| <b>Protect security services</b>    | Deny disabling GuardDuty, CloudTrail, Config                                              |

### 73.5.1 Terraform SCP attachment

```

resource "aws_organizations_policy" "deny_root" {
 name = "DenyRootUser"
 type = "SERVICE_CONTROL_POLICY"
 content = jsonencode({
 Version = "2012-10-17"
 Statement = [{
 Sid = "DenyRootUser"
 Effect = "Deny"
 Action = "*"
 Resource = "*"
 Condition = {
 StringLike = {
 "aws:PrincipalArn" = "arn:aws:iam::*:root"
 }
 }
 }]
 })
}

resource "aws_organizations_policy_attachment" "workloads" {
 policy_id = aws_organizations_policy.deny_root.id
 target_id = aws_organizations_organizational_unit.workloads.id
}

```

## 73.6 8.6 Account vending with Control Tower

**AWS Control Tower** builds on Organizations to provide:

- **Account Factory** — Automated account creation with baseline guardrails.
- **Guardrails** — Pre-built SCPs (mandatory and elective).
- **Account Factory for Terraform (AFT)** — Terraform-based account vending.

Typical landing zone accounts:

| Account | Purpose |
|---------|---------|
|         |         |

|                 |                                                  |
|-----------------|--------------------------------------------------|
| Management      | Organizations, Identity Center, billing          |
| Log Archive     | Centralized CloudTrail, Config logs              |
| Audit           | Security tooling (GuardDuty admin, Security Hub) |
| Network         | Transit Gateway, VPN, Direct Connect             |
| Shared Services | CI/CD, artifact repos, DNS                       |

## 73.7 8.7 Delegated administration

Certain security services can be administered from a **delegated admin account** instead of the management account:

| Service          | Delegated admin use           |
|------------------|-------------------------------|
| GuardDuty        | Central threat detection      |
| Security Hub     | Aggregated findings           |
| AWS Config       | Aggregator for compliance     |
| CloudTrail       | Organization trail management |
| Firewall Manager | WAF rules across accounts     |

```
aws organizations enable-aws-service-access \
 --service-principal guardduty.amazonaws.com

aws guardduty enable-organization-admin-account \
 --admin-account-id 111111111111
```

## 73.8 8.8 SCP troubleshooting

| Symptom                                      | Likely cause                         |
|----------------------------------------------|--------------------------------------|
| AccessDeniedException for allowed IAM policy | SCP denies the action                |
| Works in management account, fails in member | SCP attached to member OU            |
| Service-linked role fails                    | SCP may need exception for that role |
| Cannot enable a service                      | SCP blocks the enable API            |

### 73.8.1 Testing SCP impact

```
Use IAM Policy Simulator with Organizations context
Or: create a test role in a sandbox account with the SCP attached
aws iam simulate-principal-policy \
 --policy-source-arn arn:aws:iam::123456789012:role/TestRole \
 --action-names ec2:RunInstances \
 --resource-arns "*" "
```

## 73.9 8.9 Chapter summary

- **AWS Organizations** manages multi-account hierarchies with consolidated billing.

- **OUs** group accounts for policy application.
  - **SCPs** set permission guardrails; they filter but do not grant access.
  - **Control Tower** provides a landing zone with automated account vending.
  - **Delegated administration** centralizes security services in a dedicated account.
- 

## 73.10 Lab 8.1 — Explore your organization

1. Run `aws organizations describe-organization` and note the organization ID.
2. List all accounts: `aws organizations list-accounts`.
3. Identify which OUs exist and which accounts belong to each.
4. List SCPs: `aws organizations list-policies --filter SERVICE_CONTROL_POLICY`.

## 73.11 Lab 8.2 — Create a sandbox SCP

1. In a **sandbox OU** (never production first), create an SCP that denies `ec2:RunInstances` for instance types larger than `xlarge`.
  2. Attach the SCP to the sandbox OU.
  3. Attempt to launch a `m5.2xlarge` — verify denial.
  4. Launch a `t3.micro` — verify success.
  5. Detach and delete the SCP when done.
- 

## 73.12 Review questions

1. Do SCPs grant permissions or restrict them?
  2. Which account is exempt from SCP restrictions?
  3. How do consolidated billing volume discounts work across accounts?
  4. What is the difference between an OU and an AWS account?
  5. Why should you test SCPs in a sandbox before applying to production OUs?
- 

Next: [Chapter 9 — VPC Fundamentals](#)

# 74 Chapter 9: VPC Fundamentals — CIDR, Subnets, and Route Tables

AWS Handbook — Part III, Pages 161–185

## 74.1 9.1 What is a VPC?

A **Virtual Private Cloud (VPC)** is your isolated network in AWS. You control IP ranges, subnets, routing, and firewalls.

Every AWS account has a **default VPC** per region (with public subnets). Production workloads use **custom VPCs** with explicit design.

## 74.2 9.2 CIDR and IP planning

**CIDR** (Classless Inter-Domain Routing) defines IP ranges:

| CIDR        | Addresses | Usable (AWS reserves 5) |
|-------------|-----------|-------------------------|
| 10.0.0.0/16 | 65,536    | ~65,531                 |
| 10.0.1.0/24 | 256       | ~251                    |
| 10.0.1.0/28 | 16        | ~11                     |

### 74.2.1 Example VPC design

```
VPC: 10.0.0.0/16

Public subnets (ALB, NAT Gateway):
 10.0.1.0/24 - us-east-1a
 10.0.2.0/24 - us-east-1b

Private subnets (app servers, ECS, Lambda):
 10.0.10.0/24 - us-east-1a
 10.0.11.0/24 - us-east-1b

Database subnets (RDS, no internet):
 10.0.20.0/24 - us-east-1a
 10.0.21.0/24 - us-east-1b
```

**Tip:** Plan for growth; changing VPC CIDR later is painful. Use `/16` for VPC, `/24` for subnets.

## 74.3 9.3 Subnets

Subnets live in **one Availability Zone**. Types:

| Type     | Route to IGW          | Typical workloads         |
|----------|-----------------------|---------------------------|
| Public   | Yes (via route table) | ALB, NAT Gateway, bastion |
| Private  | No direct IGW         | App servers, workers      |
| Isolated | No NAT either         | Databases                 |

`map_public_ip_on_launch = true` assigns public IPs to instances in public subnets.

## 74.4 9.4 Route tables

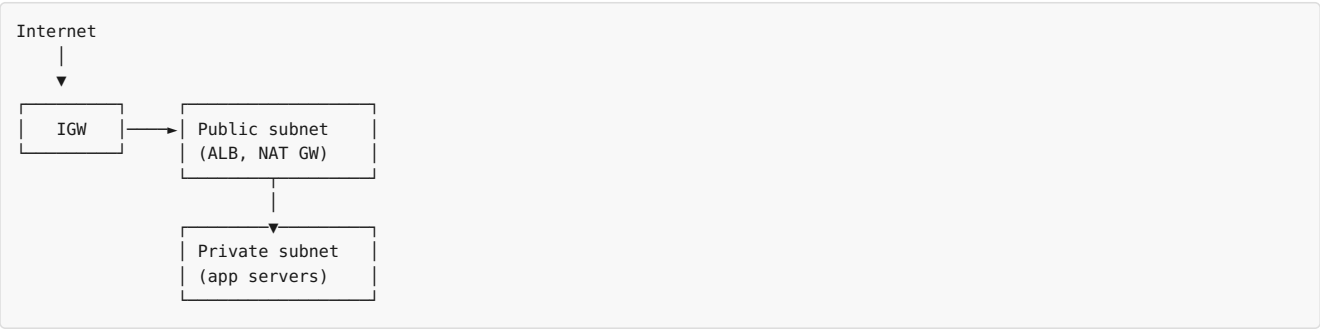
A **route table** defines where traffic goes:

| Destination | Target       | Meaning                                   |
|-------------|--------------|-------------------------------------------|
| 10.0.0.0/16 | local        | Traffic within VPC                        |
| 0.0.0.0/0   | igw-xxxxx    | Internet via Internet Gateway             |
| 0.0.0.0/0   | nat-xxxxx    | Internet via NAT Gateway (private subnet) |
| pl-xxxxx    | VPC Endpoint | Private AWS service access                |

Each subnet associates with **one** route table.

## 74.5 9.5 Internet Gateway and NAT (preview)

- **Internet Gateway (IGW)** — VPC component for bidirectional internet access
- **NAT Gateway** — Allows private subnet instances to reach internet (outbound only); place in public subnet



## 74.6 9.6 Security Groups vs NACLs

|         | Security Group   | NACL           |
|---------|------------------|----------------|
| Level   | Instance/ENI     | Subnet         |
| State   | Stateful         | Stateless      |
| Rules   | Allow only       | Allow and deny |
| Default | Deny all inbound | Allow all      |

**Best practice:** Use security groups as primary firewall; NACLs for subnet-level deny rules if needed.

Example security group for web tier:

- Inbound: 443 from ALB security group
- Outbound: 5432 to database security group

## 74.7 9.7 Terraform VPC snippet

```
resource "aws_vpc" "main" {
 cidr_block = "10.0.0.0/16"
 enable_dns_hostnames = true
```

```
enable_dns_support = true
tags = { Name = "prod-vpc" }
}

resource "aws_subnet" "public" {
 vpc_id = aws_vpc.main.id
 cidr_block = "10.0.1.0/24"
 availability_zone = "us-east-1a"
 map_public_ip_on_launch = true
}
```

---

## 74.8 9.8 Chapter summary

- Design VPCs with **public**, **private**, and **database** subnet tiers across **multiple AZs**.
- **Route tables** control internet and local routing.
- **Security groups** are stateful instance firewalls; default deny inbound.

---

## 74.9 Lab 9.1

1. Create a custom VPC with 2 public and 2 private subnets in different AZs.
2. Attach an Internet Gateway and configure route tables.
3. Launch an EC2 in a private subnet (no public IP) and verify no direct internet access.

---

Next: [Chapter 10 — NAT and Routing](#)



# 75 Chapter 10: Internet Gateway, NAT Gateway, and Routing

AWS Handbook — Part III, Pages 186-205

## 75.1 10.1 Internet Gateway (IGW)

An **Internet Gateway** is a horizontally scaled, redundant VPC component that allows:

- Instances with public IPs to communicate with the internet
- Internet to initiate connections to instances with public IPs

Attach one IGW per VPC (for internet-connected VPCs).

```
aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway --internet-gateway-id igw-xxx --vpc-id vpc-xxx
```

Public subnet route table:

| Destination | Target      |
|-------------|-------------|
| 0.0.0.0/0   | igw-xxxxxxx |
| 10.0.0.0/16 | local       |

## 75.2 10.2 NAT Gateway

Private subnet instances should **not** have public IPs. They reach the internet (for updates, API calls) via a **NAT Gateway** in a public subnet.

```
Private subnet instance (10.0.10.5)
 → NAT Gateway (public subnet, elastic IP)
 → Internet Gateway
 → Internet
```

### 75.2.1 NAT Gateway facts

- **Managed** by AWS (no patching)
- **Charged** per hour + per GB processed
- **AZ-specific** — deploy one NAT per AZ for HA, or accept cross-AZ failover tradeoffs
- **Alternative:** NAT instances (self-managed, not recommended)

Private subnet route table:

| Destination | Target      |
|-------------|-------------|
| 0.0.0.0/0   | nat-xxxxxxx |
| 10.0.0.0/16 | local       |

## 75.3 10.3 VPC Endpoints — avoid NAT costs

For AWS service traffic (S3, DynamoDB, etc.), use **VPC endpoints** to keep traffic on the AWS network—no NAT charges.

| Type | Services | Cost |
|------|----------|------|
|------|----------|------|

|                  |                     |                 |
|------------------|---------------------|-----------------|
| <b>Gateway</b>   | S3, DynamoDB        | Free            |
| <b>Interface</b> | Most other services | Per hour + data |

```
resource "aws_vpc_endpoint" "s3" {
 vpc_id = aws_vpc.main.id
 service_name = "com.amazonaws.us-east-1.s3"
 route_table_ids = [aws_route_table.private.id]
}
```

## 75.4 10.4 Elastic IP (EIP)

A **static public IPv4** address. Required for NAT Gateway.

- **Charge** applies if EIP is allocated but not attached to a running instance
- Release unused EIPs to avoid fees

## 75.5 10.5 Troubleshooting connectivity

| Problem                               | Check                                                    |
|---------------------------------------|----------------------------------------------------------|
| Can't SSH to public instance          | SG allows 22; has public IP; IGW attached; correct route |
| Private instance can't reach internet | NAT in public subnet; private RT has 0.0.0.0/0 → NAT     |
| Can't reach S3 from private subnet    | VPC endpoint or NAT route                                |
| Inter-subnet blocked                  | NACL rules; SG rules                                     |

### 75.5.1 Useful commands

```
aws ec2 describe-route-tables --filters "Name=vpc-id,Values=vpc-xxx"
aws ec2 describe-network-interfaces --filters "Name=subnet-id,Values=subnet-xxx"
From instance:
curl -s https://checkip.amazonaws.com # Public IP test
```

## 75.6 10.6 High availability networking

Production checklist:

- ☐ Multi-AZ subnets
- ☐ NAT Gateway per AZ (or accept risk)
- ☐ ALB spans multiple AZs
- ☐ VPC endpoints for S3/DynamoDB/ECR/API
- ☐ No single-AZ dependencies for critical paths

## 75.7 10.7 Chapter summary

- **IGW** enables public internet for public subnets.
- **NAT Gateway** provides outbound-only internet for private subnets.
- Use **VPC endpoints** to reduce NAT costs and improve security for AWS APIs.

---

## 75.8 Lab 10.1

1. Deploy NAT Gateway in public subnet with EIP.
2. Configure private subnet route table.
3. Launch EC2 in private subnet; `curl` an external API successfully.
4. Add S3 gateway endpoint; verify S3 access without NAT.

---

Next: [Chapter 15 — EC2](#)

# 76 Chapter 11: Security Groups vs Network ACLs

AWS Handbook — Part III, Pages 206–220

## 76.1 11.1 Two layers of VPC security

Amazon VPC provides **two complementary firewall mechanisms** for controlling traffic:

- 1. **Security Groups (SGs)** — Stateful, instance-level (or ENI-level) firewalls.
- 2. **Network Access Control Lists (NACLs)** — Stateless, subnet-level firewalls.

Understanding when and how to use each is essential for designing secure, compliant network architectures. Most day-to-day rules are security groups; NACLs add a subnet-wide safety net.

## 76.2 11.2 Security groups in depth

A **security group** acts as a virtual firewall for EC2 instances, RDS databases, Lambda functions in a VPC, ELBs, and other resources. It controls **inbound** and **outbound** traffic at the **elastic network interface (ENI)** level.

### 76.2.1 Key characteristics

| Property   | Security Group                                         |
|------------|--------------------------------------------------------|
| State      | <b>Stateful</b> — return traffic automatically allowed |
| Scope      | ENI / resource level                                   |
| Rules      | Allow only (no Deny rules)                             |
| Evaluation | All rules evaluated before decision                    |
| Default    | Deny all inbound; allow all outbound                   |
| Attachment | One or more SGs per ENI                                |

### 76.2.2 Stateful behavior example

If you allow inbound TCP 443, the response traffic on ephemeral ports is **automatically permitted** without an explicit outbound rule. This simplifies rule management significantly.

### 76.2.3 Security group rules

Each rule specifies:

| Field                  | Example                       |
|------------------------|-------------------------------|
| Type                   | HTTP, HTTPS, SSH, Custom TCP  |
| Protocol               | TCP, UDP, ICMP, all           |
| Port range             | 443, 22, 1024-65535           |
| Source (inbound)       | CIDR, another SG, prefix list |
| Destination (outbound) | CIDR, another SG, prefix list |

### 76.2.4 Referencing security groups

Instead of IP addresses, reference another security group:

Inbound: TCP 3306 from sg-0abc123 (app-tier SG)

This allows any instance in the app-tier SG to reach the database, regardless of IP address. When instances scale, rules remain valid.

### 76.2.5 CLI example

```
Create a web tier security group
aws ec2 create-security-group \
 --group-name web-tier-sg \
 --description "Allow HTTP/HTTPS from ALB" \
 --vpc-id vpc-0abc123

Allow HTTPS from ALB security group
aws ec2 authorize-security-group-ingress \
 --group-id sg-0web123 \
 --protocol tcp \
 --port 443 \
 --source-group sg-0alb456

Allow SSH from bastion only
aws ec2 authorize-security-group-ingress \
 --group-id sg-0web123 \
 --protocol tcp \
 --port 22 \
 --source-group sg-0bastion789
```

### 76.2.6 Terraform example

```
resource "aws_security_group" "web" {
 name = "web-tier"
 description = "Web server security group"
 vpc_id = aws_vpc.main.id

 ingress {
 description = "HTTPS from ALB"
 from_port = 443
 to_port = 443
 protocol = "tcp"
 security_groups = [aws_security_group.alb.id]
 }

 egress {
 from_port = 0
 to_port = 0
 protocol = "-1"
 cidr_blocks = ["0.0.0.0/0"]
 }

 tags = { Name = "web-tier-sg" }
}
```

## 76.3 11.3 Network ACLs in depth

A **Network ACL (NACL)** is an optional layer of security that acts as a firewall for **subnets**. Each subnet is associated with exactly one NACL; a NACL can be associated with multiple subnets.

### 76.3.1 Key characteristics

| Property   | NACL                                                     |
|------------|----------------------------------------------------------|
| State      | <b>Stateless</b> — must allow both directions explicitly |
| Scope      | Subnet level                                             |
| Rules      | Allow <b>and</b> Deny                                    |
| Evaluation | Lowest rule number first; first match wins               |

|            |                                |
|------------|--------------------------------|
| Default    | Allow all inbound and outbound |
| Attachment | One NACL per subnet            |

### 76.3.2 Stateless behavior

If you allow inbound TCP 443, you must **also** allow outbound traffic on ephemeral ports (1024–65535) for the response to return. Forgetting the outbound rule is the most common NACL mistake.

### 76.3.3 Rule numbering

NACL rules are numbered (e.g., 100, 200, 300). Lower numbers are evaluated first. A `*` rule at the end denies everything not explicitly allowed (in custom NACLs).

### 76.3.4 Example: Restrict SSH to corporate IP

| Rule # | Type     | Protocol | Port       | Source         | Allow/Deny |
|--------|----------|----------|------------|----------------|------------|
| 100    | Inbound  | TCP      | 22         | 203.0.113.0/24 | ALLOW      |
| 110    | Inbound  | TCP      | 22         | 0.0.0.0/0      | DENY       |
| 100    | Outbound | TCP      | 1024-65535 | 0.0.0.0/0      | ALLOW      |

### 76.3.5 CLI example

```
aws ec2 create-network-acl --vpc-id vpc-0abc123

aws ec2 create-network-acl-entry \
 --network-acl-id acl-0abc123 \
 --rule-number 100 \
 --protocol tcp \
 --port-range From=22,To=22 \
 --cidr-block 203.0.113.0/24 \
 --rule-action allow \
 --ingress

aws ec2 replace-network-acl-association \
 --association-id aclassoc-0abc123 \
 --network-acl-id acl-0abc123
```

## 76.4 11.4 Security groups vs NACLs — comparison

| Feature         | Security Group         | NACL                       |
|-----------------|------------------------|----------------------------|
| Level           | Instance/ENI           | Subnet                     |
| Stateful        | Yes                    | No                         |
| Deny rules      | No                     | Yes                        |
| Rule evaluation | All rules              | Ordered (first match)      |
| Default action  | Deny inbound           | Allow all                  |
| Affects         | Resources with ENI     | All traffic to/from subnet |
| Use case        | Primary access control | Subnet-wide guardrails     |

### 76.4.1 When to use which

| Scenario                          | Use                               |
|-----------------------------------|-----------------------------------|
| Allow app → database on port 3306 | Security group referencing app SG |

|                                                         |                        |
|---------------------------------------------------------|------------------------|
| Block a known malicious IP range                        | NACL Deny rule         |
| Allow ALB → web tier on 443                             | Security group         |
| Block all inbound except specific CIDRs at subnet level | NACL                   |
| Lambda in VPC accessing RDS                             | Security group on both |

## 76.5 11.5 Defense in depth pattern

Production architectures typically use **both**:

```
Internet → NACL (subnet) → Security Group (ALB) → Security Group (web) → NACL (private subnet) → Security Group (DB)
```

- 1. **Public subnet NACL** — Deny known bad IPs, allow 80/443 from internet.
- 2. **ALB security group** — Allow 80/443 from 0.0.0.0/0.
- 3. **Web tier SG** — Allow 443 only from ALB SG.
- 4. **Private subnet NACL** — Deny all inbound from internet CIDRs.
- 5. **Database SG** — Allow 3306 only from web tier SG.

## 76.6 11.6 Common mistakes and troubleshooting

| Mistake                              | Symptom                                     | Fix                                |
|--------------------------------------|---------------------------------------------|------------------------------------|
| NACL missing outbound ephemeral rule | Connection hangs after SYN                  | Add outbound allow for 1024-65535  |
| SG allows wrong source CIDR          | Unexpected access or blocked access         | Use SG references instead of CIDRs |
| Default SG used in production        | Overly permissive (same-SG traffic allowed) | Create dedicated SGs per tier      |
| NACL deny blocks health checks       | ALB marks targets unhealthy                 | Allow health check IPs/CIDRs       |
| Forgot IPv6 rules                    | IPv6 traffic blocked                        | Add ::/0 rules where needed        |

### 76.6.1 Connectivity troubleshooting order

- 1. Check **security group** inbound rules on the target.
- 2. Check **security group** outbound rules on the source.
- 3. Check **NACL** inbound on the target subnet.
- 4. Check **NACL** outbound on the source subnet.
- 5. Check **route tables** (covered in Chapter 10).
- 6. Use **VPC Reachability Analyzer** for automated path analysis.

```
aws ec2 describe-security-groups --group-ids sg-0abc123
aws ec2 describe-network-acls --filters "Name=association.subnet-id,Values=subnet-0abc123"
```

## 76.7 11.7 Security group limits and quotas

| Resource                | Default limit |
|-------------------------|---------------|
| Security groups per VPC | 2,500         |

|                                     |                          |
|-------------------------------------|--------------------------|
| Rules per security group            | 60 inbound + 60 outbound |
| Security groups per ENI             | 5 (can request increase) |
| Referenced security groups per rule | 1                        |

Use **prefix lists** to reduce rule count when allowing access from many CIDRs.

## 76.8 11.8 Chapter summary

- **Security groups** are stateful, instance-level, allow-only firewalls—the primary access control mechanism.
- **NACLs** are stateless, subnet-level, support deny rules—a secondary guardrail.
- Reference security groups instead of IP addresses for scalable architectures.
- Always allow **return traffic** in NACLs (ephemeral ports outbound).
- Use both layers for defense in depth; troubleshoot connectivity systematically.

## 76.9 Lab 11.1 — Security group tiers

1. Create three security groups in your VPC: `alb-sg`, `web-sg`, `db-sg`.
2. Configure: ALB allows 80/443 from internet; web allows 443 from ALB SG; db allows 3306 from web SG.
3. Launch a test EC2 in a private subnet with `web-sg`.
4. Verify you cannot SSH directly from the internet.
5. Verify web tier can reach a test database port using `nc` or `telnet`.

## 76.10 Lab 11.2 — NACL deny rule

1. Create a custom NACL for a public subnet.
2. Add a Deny rule for a specific IP (your test IP).
3. Verify HTTP access is blocked from that IP but works from another.
4. Remove the deny rule and confirm access is restored.

## 76.11 Review questions

1. Why are security groups called “stateful” and NACLs “stateless”?
2. Can you create a Deny rule in a security group?
3. If a security group allows inbound TCP 80, is an outbound rule needed for the response?
4. What happens when NACL rule 100 allows traffic but rule 110 denies it?
5. Why should you reference security groups instead of IP addresses between tiers?

Next: [Chapter 12 — Load Balancers](#)



# 77 Chapter 12: Load Balancers — ALB, NLB, and GWLB

AWS Handbook — Part III, Pages 221-245

## 77.1 12.1 Why load balancers?

In production, a single server cannot handle all traffic or survive failures. **Elastic Load Balancing (ELB)** distributes incoming traffic across multiple targets (EC2 instances, containers, IP addresses, Lambda functions) and performs health checks to route around unhealthy targets.

AWS offers three load balancer types, each optimized for different layers of the OSI model and use cases.

## 77.2 12.2 Load balancer comparison

| Feature       | ALB                                   | NLB                                             | GWLB                       |
|---------------|---------------------------------------|-------------------------------------------------|----------------------------|
| Layer         | Layer 7 (HTTP/HTTPS)                  | Layer 4 (TCP/UDP/TLS)                           | Layer 3 (IP)               |
| Protocols     | HTTP, HTTPS, gRPC, WebSocket          | TCP, UDP, TLS                                   | IP packets                 |
| Targets       | Instances, IPs, Lambda                | Instances, IPs, ALB                             | Appliance instances        |
| Static IP     | No (use Global Accelerator)           | Yes (per AZ)                                    | N/A                        |
| Cross-zone LB | Optional (default on)                 | Optional (default off)                          | N/A                        |
| Use case      | Web apps, microservices, path routing | Extreme performance, static IP, TLS passthrough | Inline security appliances |
| Pricing       | Per LCU-hour                          | Per NLCU-hour                                   | Per GLCU-hour              |

## 77.3 12.3 Application Load Balancer (ALB)

The **ALB** operates at Layer 7, making routing decisions based on HTTP content: host header, path, query string, and headers.

### 77.3.1 Key features

- **Listener rules** — Route by path ( `/api/*` → API target group), host ( `api.example.com` ), or headers.
- **Target groups** — Collections of targets with health checks and stickiness settings.
- **SSL/TLS termination** — Decrypt at the ALB using ACM certificates.
- **WebSocket and HTTP/2** — Native support.
- **AWS WAF integration** — Attach WAF Web ACLs for application-layer protection.
- **Authentication** — Built-in OIDC/Cognito authentication at the ALB.

### 77.3.2 ALB architecture

```

Internet → ALB (public subnets, multi-AZ)
├── Listener :443 (HTTPS)
│ ├── Rule: /api/* → Target Group A (ECS tasks)
│ ├── Rule: /admin/* → Target Group B (EC2 instances)
│ └── Default → Target Group C (static site)
└── Listener :80 (redirect to 443)

```

### 77.3.3 Health checks

| Setting             | Typical value           |
|---------------------|-------------------------|
| Protocol            | HTTP                    |
| Path                | /health                 |
| Interval            | 30 seconds              |
| Healthy threshold   | 2 consecutive successes |
| Unhealthy threshold | 3 consecutive failures  |
| Success codes       | 200                     |

### 77.3.4 Terraform ALB example

```

resource "aws_lb" "app" {
 name = "app-alb"
 internal = false
 load_balancer_type = "application"
 security_groups = [aws_security_group.alb.id]
 subnets = aws_subnet.public[*].id

 tags = { Name = "app-alb" }
}

resource "aws_lb_target_group" "web" {
 name = "web-tg"
 port = 80
 protocol = "HTTP"
 vpc_id = aws_vpc.main.id

 health_check {
 path = "/health"
 healthy_threshold = 2
 unhealthy_threshold = 3
 timeout = 5
 interval = 30
 matcher = "200"
 }
}

resource "aws_lb_listener" "https" {
 load_balancer_arn = aws_lb.app.arn
 port = 443
 protocol = "HTTPS"
 ssl_policy = "ELBSecurityPolicy-TLS13-1-2-2021-06"
 certificate_arn = aws_acm_certificate.app.arn

 default_action {
 type = "forward"
 target_group_arn = aws_lb_target_group.web.arn
 }
}

resource "aws_lb_listener_rule" "api" {
 listener_arn = aws_lb_listener.https.arn
 priority = 100

 action {
 type = "forward"
 target_group_arn = aws_lb_target_group.api.arn
 }

 condition {
 path_pattern { values = ["/api/*"] }
 }
}

```

```
}
}
```

### 77.3.5 CLI example

```
aws elbv2 create-load-balancer \
 --name app-alb \
 --subnets subnet-aaa subnet-bbb \
 --security-groups sg-alb \
 --scheme internet-facing \
 --type application

aws elbv2 create-target-group \
 --name web-tg \
 --protocol HTTP \
 --port 80 \
 --vpc-id vpc-0abc123 \
 --health-check-path /health
```

## 77.4 12.4 Network Load Balancer (NLB)

The **NLB** operates at Layer 4, handling millions of requests per second with ultra-low latency. It preserves the client source IP and supports static IP addresses per Availability Zone.

### 77.4.1 When to choose NLB

| Scenario                          | Why NLB                                   |
|-----------------------------------|-------------------------------------------|
| Non-HTTP protocols (MQTT, gaming) | Layer 4, any TCP/UDP                      |
| Extreme throughput requirements   | Millions of RPS                           |
| Static IP needed                  | Elastic IP per AZ                         |
| TLS passthrough                   | NLB forwards encrypted traffic to targets |
| PrivateLink service provider      | NLB as endpoint service                   |
| ALB as target                     | NLB → ALB for static IP + L7 routing      |

### 77.4.2 NLB characteristics

- **Connection-based** — Not request-based like ALB.
- **Cross-zone load balancing** — Off by default (enable for even distribution).
- **Preserve source IP** — Targets see the real client IP (unlike ALB which adds `X-Forwarded-For`).
- **Target types** — Instance, IP, ALB.

### 77.4.3 Health checks for NLB

NLB supports TCP, HTTP, and HTTPS health checks. TCP checks only verify port openness; HTTP/HTTPS checks validate response codes.

## 77.5 12.5 Gateway Load Balancer (GWLb)

The **GWLb** distributes traffic to **third-party virtual appliances** (firewalls, IDS/IPS, deep packet inspection) in a transparent, bump-in-the-wire fashion.

### 77.5.1 Use case

```
VPC traffic → GWLB Endpoint → GWLB → Firewall appliance → GWLB → destination
```

- Deploy security appliances as GWLB targets.
- Traffic is encapsulated using **GENEVE** protocol (port 6081).
- Appliances inspect and forward traffic transparently.
- Common with vendors: Palo Alto, Fortinet, Check Point.

### 77.5.2 GWLB components

| Component            | Role                                          |
|----------------------|-----------------------------------------------|
| <b>GWLB</b>          | Distributes flows to appliance fleet          |
| <b>Target group</b>  | Appliance instances (IP or instance type)     |
| <b>GWLB endpoint</b> | VPC endpoint that routes traffic through GWLB |
| <b>Route table</b>   | Directs subnet traffic to the GWLB endpoint   |

## 77.6 12.6 Shared ELB concepts

### 77.6.1 Listeners and rules

- **Listener** — Checks for connection requests on a port/protocol.
- **Rules** — ALB only; evaluated by priority (lower number first).
- **Default action** — Catches traffic not matched by any rule.

### 77.6.2 Target groups

| Setting                     | Description                                                 |
|-----------------------------|-------------------------------------------------------------|
| <b>Target type</b>          | instance, ip, lambda, alb                                   |
| <b>Protocol/port</b>        | How the LB communicates with targets                        |
| <b>Deregistration delay</b> | Drain time before removing unhealthy targets (default 300s) |
| <b>Stickiness</b>           | Session affinity via cookie (ALB) or source IP (NLB)        |

### 77.6.3 Stickiness

| LB  | Method                                            |
|-----|---------------------------------------------------|
| ALB | Application cookie (LB cookie or app cookie)      |
| NLB | Source IP hash (5-tuple for TCP, 3-tuple for UDP) |

### 77.6.4 Connection draining

When a target is deregistered or becomes unhealthy, the LB stops sending **new** connections but allows **in-flight** connections to complete during the deregistration delay.

## 77.7 12.7 SSL/TLS with load balancers

| Approach                 | LB       | Description                            |
|--------------------------|----------|----------------------------------------|
| <b>Terminate at LB</b>   | ALB, NLB | LB decrypts; backend can use HTTP      |
| <b>TLS passthrough</b>   | NLB      | Encrypted traffic forwarded to targets |
| <b>Mutual TLS (mTLS)</b> | ALB      | Client certificate validation          |
|                          |          |                                        |

|                    |            |                    |
|--------------------|------------|--------------------|
| <b>Certificate</b> | ACM (free) | Attach to listener |
|--------------------|------------|--------------------|

**Best practice:** Terminate TLS at the ALB with ACM certificates. Use HTTPS between ALB and targets for end-to-end encryption if required.

## 77.8 12.8 Monitoring and troubleshooting

| Tool                      | Purpose                                            |
|---------------------------|----------------------------------------------------|
| <b>CloudWatch metrics</b> | RequestCount, TargetResponseTime, HealthyHostCount |
| <b>Access logs</b>        | S3-stored per-request logs (enable explicitly)     |
| <b>Connection logs</b>    | NLB connection-level logs                          |
| <b>Target health</b>      | <code>aws elbv2 describe-target-health</code>      |

```
aws elbv2 describe-target-health \
 --target-group-arn arn:aws:elasticloadbalancing:us-east-1:123456789012:targetgroup/web-tg/abc123

aws elbv2 describe-load-balancer-attributes \
 --load-balancer-arn arn:aws:elasticloadbalancing:us-east-1:123456789012:loadbalancer/app/app-alb/abc123
```

### 77.8.1 Common issues

| Symptom                 | Cause                                              |
|-------------------------|----------------------------------------------------|
| 502 Bad Gateway         | Target not responding or security group blocks ALB |
| 503 Service Unavailable | No healthy targets in target group                 |
| Health check failing    | Wrong path, port, or security group                |
| Slow responses          | Target overloaded; check TargetResponseTime metric |

## 77.9 12.9 Chapter summary

- **ALB** — Layer 7; path/host routing, SSL termination, WAF, authentication.
- **NLB** — Layer 4; extreme performance, static IPs, non-HTTP protocols.
- **GWLB** — Inline security appliances with transparent traffic inspection.
- Configure **health checks**, **security groups**, and **target groups** carefully.
- Use **ACM** for certificates; enable **access logs** for troubleshooting.

### 77.10 Lab 12.1 — Deploy an ALB

1. Create an ALB in public subnets with an HTTPS listener (ACM certificate).
2. Create a target group pointing to EC2 instances in private subnets.
3. Configure a health check on `/health`.
4. Add a listener rule: `/api/*` forwards to a separate target group.
5. Verify routing with `curl` against both paths.

### 77.11 Lab 12.2 — NLB with static IP

1. Create an NLB with TCP listener on port 5432.
2. Note the static Elastic IP assigned per AZ.

3. Register RDS or an EC2 PostgreSQL instance as target.
  4. Connect from a client using the NLB static IP.
- 

## 77.12 Review questions

1. At which OSI layer does an ALB operate, and what routing decisions can it make?
  2. When would you choose NLB over ALB?
  3. What is the purpose of a GWLB?
  4. Why might ALB health checks fail even when the application is running?
  5. What is connection draining, and why does it matter during deployments?
- 

Next: [\*Chapter 13 — Route 53 DNS Routing\*](#)

# 78 Chapter 13: Route 53 — DNS Records and Routing Policies

AWS Handbook — Part III, Pages 246-265

## 78.1 13.1 What is Amazon Route 53?

**Amazon Route 53** is AWS’s highly available and scalable **Domain Name System (DNS)** web service. It translates human-readable domain names (e.g., `www.example.com`) into IP addresses and provides advanced routing policies for traffic management, failover, and latency optimization.

Route 53 also offers **domain registration**, **health checks**, and **traffic flow** visual policies.

## 78.2 13.2 Core DNS concepts

| Term             | Definition                                        |
|------------------|---------------------------------------------------|
| Hosted zone      | Container for DNS records for a domain            |
| Record           | Maps a name to a value (A, CNAME, MX, etc.)       |
| TTL              | Time-to-live; how long resolvers cache the answer |
| Name server (NS) | Authoritative DNS servers for the zone            |
| Resolver         | Client-side DNS lookup (Route 53 Resolver in VPC) |

### 78.2.1 Hosted zone types

| Type                | Cost                   | Use case                                |
|---------------------|------------------------|-----------------------------------------|
| Public hosted zone  | \$0.50/month + queries | Internet-facing domains                 |
| Private hosted zone | \$0.50/month + queries | Internal VPC DNS (associated with VPCs) |

## 78.3 13.3 DNS record types

| Type  | Purpose                                 | Example value                                      |
|-------|-----------------------------------------|----------------------------------------------------|
| A     | IPv4 address                            | <code>203.0.113.10</code>                          |
| AAAA  | IPv6 address                            | <code>2001:db8::1</code>                           |
| CNAME | Alias to another name                   | <code>app.example.com → d123.cloudfront.net</code> |
| MX    | Mail server                             | <code>10 mail.example.com</code>                   |
| TXT   | Text (SPF, DKIM, verification)          | <code>"v=spf1 include:..."</code>                  |
| NS    | Name servers                            | <code>ns-123.awsdns-45.com</code>                  |
| SOA   | Start of authority                      | Zone metadata                                      |
| SRV   | Service location                        | <code>_http._tcp.example.com</code>                |
| Alias | Route 53-specific; maps to AWS resource | ALB, CloudFront, S3, API Gateway                   |

### 78.3.1 Alias records (Route 53 specific)

**Alias records** are Route 53’s extension of CNAME with advantages:

- Can be used at the **zone apex** ( `example.com` — CNAME cannot).
- No charge for alias queries to AWS resources.
- Automatically tracks IP changes of the target resource.

Supported alias targets: ALB, CloudFront, S3 website, API Gateway, Elastic Beanstalk, VPC interface endpoints, another Route 53 record.

## 78.4 13.4 Routing policies

Route 53 supports multiple routing policies beyond simple DNS:

### 78.4.1 Simple routing

One record with one or more values. Route 53 returns **all** values in random order (client picks one).

```
aws route53 change-resource-record-sets \
 --hosted-zone-id Z1234567890ABC \
 --change-batch '{
 "Changes": [{
 "Action": "CREATE",
 "ResourceRecordSet": {
 "Name": "www.example.com",
 "Type": "A",
 "TTL": 300,
 "ResourceRecords": [{"Value": "203.0.113.10"}]
 }
 }]
 }'
```

### 78.4.2 Weighted routing

Distribute traffic by **weight** (percentage). Useful for A/B testing and blue/green deployments.

| Record                      | Weight | Traffic share |
|-----------------------------|--------|---------------|
| blue.example.com → 1.2.3.4  | 90     | 90%           |
| green.example.com → 5.6.7.8 | 10     | 10%           |

```
resource "aws_route53_record" "blue" {
 zone_id = aws_route53_zone.main.zone_id
 name = "app.example.com"
 type = "A"
 ttl = 60
 records = ["203.0.113.10"]

 set_identifier = "blue"
 weighted_routing_policy {
 weight = 90
 }
}

resource "aws_route53_record" "green" {
 zone_id = aws_route53_zone.main.zone_id
 name = "app.example.com"
 type = "A"
 ttl = 60
 records = ["203.0.113.20"]

 set_identifier = "green"
 weighted_routing_policy {
 weight = 10
 }
}
```



```
}
}
```

### 78.4.3 Latency-based routing

Route traffic to the region with the **lowest latency** for the user. Requires one record per region with a `region` identifier.

| Region    | Record        | Latency               |
|-----------|---------------|-----------------------|
| us-east-1 | 203.0.113.10  | ~20ms (US East users) |
| eu-west-1 | 198.51.100.10 | ~15ms (EU users)      |

### 78.4.4 Failover routing

**Active-passive** failover. Pair a primary record with a secondary; Route 53 health checks determine which is active.

```
Primary (active) → ALB in us-east-1 (health check: /health)
Secondary (passive) → S3 static failover page (no health check needed)
```

### 78.4.5 Geolocation routing

Route based on the user's **geographic location** (continent, country, or US state). Useful for content localization and compliance (e.g., EU data stays in EU).

### 78.4.6 Geoproximity routing

Route based on **geographic location and bias**. Unlike geolocation, you can shift traffic by applying a **bias** value to expand or shrink a region's coverage area. Requires Route 53 Traffic Flow.

### 78.4.7 Multi-value answer routing

Return **multiple healthy** IP addresses. Similar to simple routing but only returns values that pass health checks. Up to 8 healthy records.

## 78.5 13.5 Health checks

Route 53 health checks monitor endpoint health and integrate with routing policies (failover, weighted, multi-value).

| Type             | Monitors                                          |
|------------------|---------------------------------------------------|
| Endpoint         | HTTP, HTTPS, or TCP to an IP/hostname             |
| CloudWatch alarm | Any CloudWatch metric (e.g., DynamoDB throttling) |
| Calculated       | Combination of child health checks (AND/OR)       |

### 78.5.1 Health check settings

| Setting           | Typical value                    |
|-------------------|----------------------------------|
| Protocol          | HTTPS                            |
| Path              | /health                          |
| Interval          | 30 seconds (10 seconds for fast) |
| Failure threshold | 3                                |

|         |                                 |
|---------|---------------------------------|
| Regions | Minimum 3 for reliable failover |
|---------|---------------------------------|

```
aws route53 create-health-check \
 --health-check-config '{
 "IPAddress": "203.0.113.10",
 "Port": 443,
 "Type": "HTTPS",
 "ResourcePath": "/health",
 "FullyQualifiedDomainName": "app.example.com",
 "RequestInterval": 30,
 "FailureThreshold": 3
 }'
```

### 78.5.2 Health check billing note

Health checks cost \$0.50/month each (first 50 AWS-endpoint checks free). Fast interval checks cost more.

## 78.6 13.6 Private DNS and hybrid resolution

### 78.6.1 Private hosted zones

Associate a hosted zone with one or more VPCs. Internal names (e.g., `db.internal.example.com`) resolve only within those VPCs.

### 78.6.2 Route 53 Resolver

For hybrid cloud (on-premises + AWS):

| Feature            | Purpose                                 |
|--------------------|-----------------------------------------|
| Inbound endpoints  | On-premises resolves AWS private DNS    |
| Outbound endpoints | AWS VPC resolves on-premises DNS        |
| Resolver rules     | Forward specific domains to on-premises |

On-premises DNS → Inbound Resolver Endpoint → Private hosted zone (`db.internal`)  
 VPC instances → Outbound Resolver Endpoint → On-premises DNS (`corp.example.com`)

## 78.7 13.7 Route 53 with other AWS services

| Integration        | Pattern                                 |
|--------------------|-----------------------------------------|
| ALB/NLB            | Alias A record → load balancer DNS name |
| CloudFront         | Alias A/AAAA → CloudFront distribution  |
| S3 static site     | Alias A → S3 website endpoint           |
| API Gateway        | Alias A → API Gateway custom domain     |
| Global Accelerator | Alias A → accelerator DNS name          |
| ACM                | DNS validation via CNAME records        |

## 78.8 13.8 DNS security

|  |  |
|--|--|
|  |  |
|--|--|

| Feature                        | Purpose                                |
|--------------------------------|----------------------------------------|
| DNSSEC                         | Cryptographic signing of DNS responses |
| Route 53 Resolver DNS Firewall | Block malicious domains                |
| Query logging                  | Log all DNS queries to CloudWatch Logs |

---

## 78.9 13.9 Chapter summary

- **Route 53** provides DNS hosting, domain registration, and advanced routing policies.
  - **Alias records** map to AWS resources without CNAME limitations at the zone apex.
  - **Routing policies** — simple, weighted, latency, failover, geolocation, geoproximity, multi-value.
  - **Health checks** enable automatic failover and traffic shifting.
  - **Private hosted zones** and **Resolver** support hybrid DNS architectures.
- 

### 78.10 Lab 13.1 — Public hosted zone

1. Create a public hosted zone for a domain you control.
2. Update your registrar's NS records to point to Route 53 name servers.
3. Create an A record (alias) pointing to an ALB or CloudFront distribution.
4. Verify resolution: `dig www.yourdomain.com`.

### 78.11 Lab 13.2 — Weighted routing

1. Create two EC2 instances in different AZs with simple web pages ("Server A" / "Server B").
2. Create weighted A records (50/50) for the same hostname.
3. Run `dig` or `curl` multiple times and observe distribution.
4. Shift weight to 90/10 and verify the change.

### 78.12 Lab 13.3 — Failover routing

1. Set up a primary record with a health check on a healthy endpoint.
  2. Create a secondary record pointing to an S3 static failover page.
  3. Stop the primary endpoint and verify DNS failover within 1–2 minutes.
- 

## 78.13 Review questions

1. What is the advantage of an alias record over a CNAME at the zone apex?
  2. How does weighted routing differ from latency-based routing?
  3. What happens when a failover primary record's health check fails?
  4. How do private hosted zones differ from public hosted zones?
  5. Why should health checks use at least three AWS regions?
- 

Next: [Chapter 14 — VPC Peering, Transit Gateway, PrivateLink](#)

# 79 Chapter 14: VPC Peering, Transit Gateway, and PrivateLink

AWS Handbook — Part III, Pages 266–280

## 79.1 14.1 Connecting VPCs and services

As organizations grow, workloads spread across multiple VPCs, accounts, and regions. AWS provides several connectivity options, each with different trade-offs for complexity, scalability, and cost.

| Option          | Scope                       | Transitive    | Use case                                  |
|-----------------|-----------------------------|---------------|-------------------------------------------|
| VPC Peering     | Same or cross-region        | No            | Simple 1:1 VPC connections                |
| Transit Gateway | Hub-and-spoke               | Yes (via TGW) | Many VPCs, on-premises, multi-account     |
| PrivateLink     | Service consumer → provider | N/A           | Private access to SaaS/services           |
| VPC Endpoints   | VPC → AWS service           | N/A           | Private access to AWS APIs (S3, DynamoDB) |

## 79.2 14.2 VPC Peering

A **VPC peering connection** is a networking connection between two VPCs that enables routing using private IP addresses.

### 79.2.1 Peering rules

- Peering is **non-transitive**: if A↔B and B↔C, A cannot reach C through B.
- **No overlapping CIDR blocks** between peered VPCs.
- Peering works **same-region** and **cross-region**.
- **No single point of failure** — AWS backbone handles the connection.
- You control access via **route tables** and **security groups** (cross-VPC SG referencing supported in same region).

### 79.2.2 Peering setup steps

1. Create peering connection (requester → acceptor).
2. Accept the peering connection (same or different account).
3. Update **route tables** in both VPCs to route peer CIDR through the peering connection.
4. Update **security groups** to allow traffic from peer VPC CIDR or SG.

```
VPC A (10.0.0.0/16) ← Peering → VPC B (10.1.0.0/16)

Route table A: 10.1.0.0/16 → pcx-0abc123
Route table B: 10.0.0.0/16 → pcx-0abc123
```

### 79.2.3 CLI example

```
Create peering connection
aws ec2 create-vpc-peering-connection \
 --vpc-id vpc-0aaa \
 --peer-vpc-id vpc-0bbb \
```

```

--peer-region us-west-2 # omit for same region

Accept (in acceptor account/region)
aws ec2 accept-vpc-peering-connection \
 --vpc-peering-connection-id pcx-0abc123

Add routes
aws ec2 create-route \
 --route-table-id rtb-0aaa \
 --destination-cidr-block 10.1.0.0/16 \
 --vpc-peering-connection-id pcx-0abc123

```

### 79.2.4 Terraform example

```

resource "aws_vpc_peering_connection" "peer" {
 vpc_id = aws_vpc.a.id
 peer_vpc_id = aws_vpc.b.id
 peer_region = "us-west-2"
 auto_accept = false

 tags = { Name = "a-to-b-peering" }
}

resource "aws_route" "a_to_b" {
 route_table_id = aws_route_table.a.id
 destination_cidr_block = aws_vpc.b.cidr_block
 vpc_peering_connection_id = aws_vpc_peering_connection.peer.id
}

```

### 79.2.5 When peering breaks down

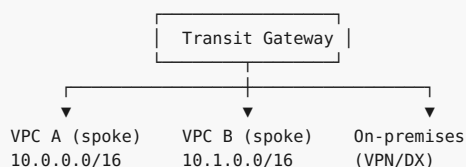
| VPCs | Peering connections needed  |
|------|-----------------------------|
| 2    | 1                           |
| 5    | 10 ( $n \times (n-1) / 2$ ) |
| 10   | 45                          |
| 50   | 1,225                       |

Peering does not scale beyond a handful of VPCs. Use **Transit Gateway** instead.

## 79.3 14.3 AWS Transit Gateway

**Transit Gateway (TGW)** is a regional network hub that connects VPCs, VPN connections, and Direct Connect gateways in a **hub-and-spoke** model.

### 79.3.1 Architecture



### 79.3.2 Key concepts

| Component              | Description                                |
|------------------------|--------------------------------------------|
| <b>Transit Gateway</b> | Regional hub                               |
| <b>Attachment</b>      | VPC, VPN, DX, peering, Connect             |
| <b>Route table</b>     | Controls which attachments can reach which |

|                    |                                                 |
|--------------------|-------------------------------------------------|
|                    |                                                 |
| <b>Association</b> | Links attachment to a route table               |
| <b>Propagation</b> | Auto-adds routes from attachment to route table |

### 79.3.3 Route table segmentation

Use multiple TGW route tables for isolation:

| Route table            | Attachments         | Routes to                        |
|------------------------|---------------------|----------------------------------|
| <b>Production</b>      | Prod VPCs, prod VPN | Other prod VPCs, shared services |
| <b>Development</b>     | Dev VPCs            | Dev VPCs only (not prod)         |
| <b>Shared Services</b> | Shared VPC          | All environments (one-way)       |

### 79.3.4 Cross-region peering

Connect TGWs in different regions with **inter-region peering** for global connectivity.

### 79.3.5 Terraform TGW example

```
resource "aws_ec2_transit_gateway" "main" {
 description = "Organization TGW"
 default_route_table_association = "disable"
 default_route_table_propagation = "disable"
 auto_accept_shared_attachments = "enable"

 tags = { Name = "org-tgw" }
}

resource "aws_ec2_transit_gateway_vpc_attachment" "app" {
 subnet_ids = aws_subnet.private[*].id
 transit_gateway_id = aws_ec2_transit_gateway.main.id
 vpc_id = aws_vpc.app.id

 tags = { Name = "app-vpc-attachment" }
}

resource "aws_ec2_transit_gateway_route_table" "prod" {
 transit_gateway_id = aws_ec2_transit_gateway.main.id
 tags = { Name = "prod-rt" }
}

resource "aws_ec2_transit_gateway_route_table_association" "app" {
 transit_gateway_attachment_id = aws_ec2_transit_gateway_vpc_attachment.app.id
 transit_gateway_route_table_id = aws_ec2_transit_gateway_route_table.prod.id
}
```

### 79.3.6 TGW pricing

- **Attachment hourly charge** per VPC/VPN/DX attachment.
- **Data processing charge** per GB transited.
- For many VPCs, TGW is more cost-effective and manageable than full-mesh peering.

## 79.4 14.4 AWS PrivateLink

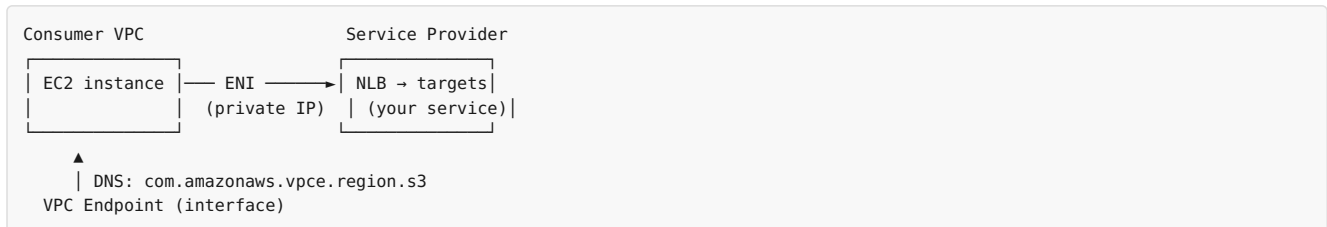
**PrivateLink** enables private connectivity from your VPC to services (AWS, partner, or your own) without traversing the public internet.

### 79.4.1 Two flavors

| Type | Direction | Example |
|------|-----------|---------|
|      |           |         |

|                                    |                                           |                                  |
|------------------------------------|-------------------------------------------|----------------------------------|
| <b>Interface VPC Endpoint</b>      | Your VPC → AWS service or partner service | S3 (interface), third-party SaaS |
| <b>Endpoint Service (Provider)</b> | Consumer VPC → your service               | Expose internal API to customers |

### 79.4.2 Interface endpoint architecture



### 79.4.3 Key points

- Creates **Elastic Network Interfaces (ENIs)** in your subnets.
- Uses **private IP addresses** from your subnet CIDR.
- Security groups apply to the endpoint ENIs.
- **Endpoint policies** control which principals can use the endpoint.

### 79.4.4 Gateway vs Interface endpoints

| Feature        | Gateway Endpoint                | Interface Endpoint                         |
|----------------|---------------------------------|--------------------------------------------|
| Services       | S3, DynamoDB                    | Most other AWS services + partner services |
| Cost           | Free                            | Hourly + data charges                      |
| Access control | Endpoint policy + bucket policy | SG + endpoint policy                       |
| DNS            | Route table entry               | Private DNS (optional)                     |
| IP             | Prefix list in route table      | ENI with private IP                        |

```
Create interface endpoint for a service
aws ec2 create-vpc-endpoint \
 --vpc-id vpc-0abc123 \
 --service-name com.amazonaws.us-east-1.execute-api \
 --vpc-endpoint-type Interface \
 --subnet-ids subnet-0aaa subnet-0bbb \
 --security-group-ids sg-0endpoint

Create gateway endpoint for S3
aws ec2 create-vpc-endpoint \
 --vpc-id vpc-0abc123 \
 --service-name com.amazonaws.us-east-1.s3 \
 --route-table-ids rtb-0aaa rtb-0bbb
```

## 79.5 14.5 Choosing the right connectivity

| Requirement                                 | Solution                       |
|---------------------------------------------|--------------------------------|
| Two VPCs, same region, simple               | VPC Peering                    |
| 10+ VPCs, hub-and-spoke                     | Transit Gateway                |
| Cross-account service consumption           | PrivateLink (endpoint service) |
| Private S3/DynamoDB access                  | Gateway VPC Endpoint           |
| Private access to AWS APIs (EC2, SSM, etc.) | Interface VPC Endpoint         |

|                              |                             |
|------------------------------|-----------------------------|
| On-premises to AWS           | VPN or Direct Connect → TGW |
| SaaS provider private access | PrivateLink (consumer)      |

## 79.6 14.6 Security considerations

| Control           | Applies to                                               |
|-------------------|----------------------------------------------------------|
| Route tables      | Peering, TGW — control which CIDRs are reachable         |
| Security groups   | Peering (same region SG ref), endpoints, TGW attachments |
| NACLs             | All traffic paths                                        |
| Endpoint policies | VPC endpoints — IAM-style access control                 |
| TGW route tables  | Segment environments (prod cannot reach dev)             |
| Resource policies | S3 bucket policies for gateway endpoints                 |

## 79.7 14.7 Chapter summary

- **VPC Peering** connects two VPCs directly; non-transitive; does not scale beyond a few VPCs.
- **Transit Gateway** provides hub-and-spoke connectivity for many VPCs, VPN, and Direct Connect.
- **PrivateLink** enables private access to services without public internet exposure.
- **Gateway endpoints** (S3, DynamoDB) are free; **interface endpoints** create ENIs with hourly charges.
- Choose connectivity based on scale, transitivity needs, and security segmentation requirements.

## 79.8 Lab 14.1 — VPC Peering

1. Create two VPCs with non-overlapping CIDRs (10.0.0.0/16 and 10.1.0.0/16).
2. Establish a peering connection and accept it.
3. Update route tables in both VPCs.
4. Launch an instance in each VPC and verify ping/SSH across the peering connection.

## 79.9 Lab 14.2 — S3 Gateway Endpoint

1. Create a gateway VPC endpoint for S3 in your VPC.
2. Attach it to private subnet route tables.
3. From a private instance (no NAT), run `aws s3 ls` and verify it works without internet access.

## 79.10 Lab 14.3 — Transit Gateway (advanced)

1. Create a Transit Gateway with two VPC attachments.
2. Create a TGW route table and associate both attachments.
3. Verify instances in both VPCs can communicate through the TGW.

## 79.11 Review questions

1. Why is VPC peering non-transitive, and what problem does Transit Gateway solve?
2. Can two peered VPCs have overlapping CIDR blocks?



3. What is the difference between a gateway endpoint and an interface endpoint?
  4. How does Transit Gateway route table segmentation improve security?
  5. When is VPC peering preferred over Transit Gateway?
- 

Next: [\*Chapter 15 — EC2 Fundamentals\*](#)

# 80 Chapter 15: EC2 — Instance Types, AMIs, and Launch Templates

AWS Handbook — Part IV, Pages 281–305

## 80.1 15.1 Amazon EC2 overview

Elastic Compute Cloud (EC2) provides resizable virtual machines in the cloud. You choose:

- **Instance type** — CPU, memory, network, storage
- **AMI** — Operating system image
- **Networking** — VPC, subnet, security groups
- **Storage** — EBS volumes
- **IAM role** — Permissions without access keys

## 80.2 15.2 Instance families

| Family               | Use case             | Examples           |
|----------------------|----------------------|--------------------|
| <b>T</b> (burstable) | Dev, low-traffic web | t3.micro, t3.small |
| <b>M</b> (general)   | Balanced workloads   | m6i.large          |
| <b>C</b> (compute)   | CPU-intensive        | c6i.xlarge         |
| <b>R</b> (memory)    | Databases, caches    | r6i.large          |
| <b>G/P</b> (GPU)     | ML, graphics         | g5.xlarge          |
| <b>I</b> (storage)   | High IOPS databases  | i3.large           |

Naming: `m6i.large` = family **m**, gen **6**, Intel **i**, size **large**.

### 80.2.1 Purchasing options

| Option                   | Cost           | Interruption                        |
|--------------------------|----------------|-------------------------------------|
| On-Demand                | Highest        | Never                               |
| Reserved / Savings Plans | 30–70% less    | Never                               |
| Spot                     | Up to 90% less | Can be terminated with 2-min notice |
| Dedicated Host           | Compliance     | Never                               |

## 80.3 15.3 AMIs and launch templates

**AMI (Amazon Machine Image)** — Template for root volume: OS + optional software.

Sources:

- AWS-provided (Amazon Linux 2023, Ubuntu)
- Marketplace
- Custom (built with EC2 Image Builder or Packer)

**Launch Template** — Reusable EC2 configuration (instance type, AMI, SG, user data, IAM role). Preferred over launch configurations.

```
aws ec2 run-instances \
 --image-id ami-0c55b159cbfafef1f0 \
 --instance-type t3.micro \
 --key-name my-key \
 --security-group-ids sg-xxx \
 --subnet-id subnet-xxx \
 --iam-instance-profile Name=EC2-S3-ReadRole \
 --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=web-01}]'
```

---

## 80.4 15.4 User data — bootstrap scripts

Run commands on first boot:

```
#!/bin/bash
yum update -y
yum install -y nginx
systemctl enable nginx
systemctl start nginx
echo "<h1>Hello from $(hostname)</h1>" > /usr/share/nginx/html/index.html
```

Pass via console, CLI `--user-data`, or launch template. Logs: `/var/log/cloud-init-output.log`.

---

## 80.5 15.5 Auto Scaling Groups (ASG)

ASG maintains desired instance count and replaces unhealthy instances.

```
resource "aws_autoscaling_group" "web" {
 name = "web-asg"
 vpc_zone_identifier = [aws_subnet.public_a.id, aws_subnet.public_b.id]
 target_group_arns = [aws_lb_target_group.web.arn]
 health_check_type = "ELB"
 min_size = 2
 max_size = 10
 desired_capacity = 2

 launch_template {
 id = aws_launch_template.web.id
 version = "$Latest"
 }
}
```

Pair with **ALB** for traffic distribution and **scaling policies** on CPU or request count.

---

## 80.6 15.6 EC2 best practices

- Use **IAM roles**, never embed access keys
- **Session Manager** instead of SSH where possible (no open port 22)
- **IMDSv2** required (`HttpTokens: required`) — prevents SSRF credential theft
- **Encrypted EBS** volumes by default
- **Patch** via SSM Patch Manager or golden AMI pipelines
- **Right-size** with CloudWatch metrics and Compute Optimizer

---

## 80.7 15.7 Chapter summary

- Choose instance **family and size** based on workload profile.
  - Use **launch templates** and **user data** for repeatable deployments.
  - **ASG + ALB** provides scalable, self-healing web tiers.
- 

## 80.8 Lab 15.1

1. Launch Amazon Linux 2023 t3.micro with nginx user data.
  2. Attach IAM role for S3 read.
  3. Create launch template and ASG (min 1, max 2) behind an ALB.
- 

Next: [Chapter 20 — S3](#)

# 81 Chapter 16: Auto Scaling & Elastic Load Balancing Integration

AWS Handbook — Part IV, Pages 306–325

## 81.1 16.1 Why Auto Scaling?

Manual capacity management cannot keep pace with variable demand. **Amazon EC2 Auto Scaling** automatically adjusts the number of EC2 instances (or other resources) based on conditions you define. Combined with **Elastic Load Balancing**, you get a self-healing, elastic application tier that scales out during traffic spikes and scales in during quiet periods—optimizing both performance and cost.

## 81.2 16.2 Auto Scaling components

| Component                       | Description                                                 |
|---------------------------------|-------------------------------------------------------------|
| Launch template / configuration | AMI, instance type, security groups, user data              |
| Auto Scaling group (ASG)        | Collection of EC2 instances with min, max, desired capacity |
| Scaling policies                | Rules that trigger scale-out or scale-in                    |
| Health checks                   | EC2 status checks and/or ELB health checks                  |
| Lifecycle hooks                 | Custom actions during instance launch/terminate             |

### 81.2.1 ASG capacity settings

| Setting | Meaning                          |
|---------|----------------------------------|
| Minimum | Floor; ASG never goes below this |
| Desired | Target number of instances       |
| Maximum | Ceiling; ASG never exceeds this  |

## 81.3 16.3 Launch templates vs launch configurations

**Launch templates** are the modern approach (launch configurations are legacy):

| Feature                               | Launch Template | Launch Configuration |
|---------------------------------------|-----------------|----------------------|
| Multiple versions                     | Yes             | No                   |
| Mix instance types (Spot + On-Demand) | Yes             | No                   |
| Latest features                       | Yes             | Deprecated           |
| Modify without replacement            | Yes             | No                   |

### 81.3.1 Launch template example (Terraform)

```
resource "aws_launch_template" "web" {
 name_prefix = "web-"
 image_id = data.aws_ami.amazon_linux.id
```

```

instance_type = "t3.micro"

vpc_security_group_ids = [aws_security_group.web.id]

user_data = base64encode(<<-EOF
#!/bin/bash
yum install -y httpd
systemctl start httpd
echo "<h1>$(hostname)</h1>" > /var/www/html/index.html
EOF
)

tag_specifications {
 resource_type = "instance"
 tags = { Name = "web-asg-instance" }
}
}

```

## 81.4 16.4 Auto Scaling group with ALB

```

Internet → ALB → Target Group ← ASG registers instances
 ↑
 Health checks (HTTP /health)

```

### 81.4.1 Terraform ASG + ALB integration

```

resource "aws_autoscaling_group" "web" {
 name = "web-asg"
 vpc_zone_identifier = aws_subnet.private[*].id
 target_group_arns = [aws_lb_target_group.web.arn]
 health_check_type = "ELB"
 health_check_grace_period = 300

 min_size = 2
 max_size = 10
 desired_capacity = 2

 launch_template {
 id = aws_launch_template.web.id
 version = "$Latest"
 }

 tag {
 key = "Name"
 value = "web-asg"
 propagate_at_launch = true
 }
}

```

#### Critical settings:

- `health_check_type = "ELB"` — ASG replaces instances that fail ALB health checks, not just EC2 status checks.
- `health_check_grace_period` — Time (seconds) before health checks count after launch. Allow your app to start.

## 81.5 16.5 Scaling policies

### 81.5.1 Target tracking (recommended)

Maintain a metric at a target value. AWS automatically creates scale-out and scale-in policies.

| Metric                   | Use case                  |
|--------------------------|---------------------------|
| ASGAverageCPUUtilization | General compute workloads |
|                          |                           |

|                          |                                             |
|--------------------------|---------------------------------------------|
| ALBRequestCountPerTarget | Request-driven scaling                      |
| Custom CloudWatch metric | Application-specific (queue depth, latency) |

```
resource "aws_autoscaling_policy" "cpu_target" {
 name = "cpu-target-tracking"
 autoscaling_group_name = aws_autoscaling_group.web.name
 policy_type = "TargetTrackingScaling"

 target_tracking_configuration {
 predefined_metric_specification {
 predefined_metric_type = "ASGAverageCPUUtilization"
 }
 target_value = 60.0
 }
}
```

### 81.5.2 Step scaling

Scale by steps based on alarm breach magnitude:

| CPU utilization | Action          |
|-----------------|-----------------|
| 50-70%          | Add 1 instance  |
| 70-90%          | Add 2 instances |
| > 90%           | Add 4 instances |

### 81.5.3 Simple scaling

Single adjustment per alarm. Less flexible; prefer target tracking or step scaling.

### 81.5.4 Scheduled scaling

Pre-define capacity for known patterns:

```
aws autoscaling put-scheduled-update-group-action \
 --auto-scaling-group-name web-asg \
 --scheduled-action-name scale-up-morning \
 --recurrence "0 8 * * MON-FRI" \
 --desired-capacity 10 \
 --time-zone "America/New_York"
```

### 81.5.5 Predictive scaling

Machine learning predicts traffic patterns and proactively scales. Works alongside dynamic policies.

## 81.6 16.6 Instance refresh

**Instance refresh** replaces instances in an ASG with new ones (e.g., after AMI update) without manual intervention.

| Setting                       | Description                                          |
|-------------------------------|------------------------------------------------------|
| <b>Min healthy percentage</b> | Minimum healthy instances during refresh (e.g., 90%) |
| <b>Instance warmup</b>        | Time before new instances count toward capacity      |
| <b>Checkpoint</b>             | Pause refresh at percentage milestones               |

```
aws autoscaling start-instance-refresh \
 --auto-scaling-group-name web-asg \
 --preferences '{
 "MinHealthyPercentage": 90,
```

```
"InstanceWarmup": 300,
"CheckpointPercentages": [50, 100]
}'
```

## 81.7 16.7 Lifecycle hooks

Lifecycle hooks pause instance launch or termination for custom actions:

| Hook        | Use case                                                  |
|-------------|-----------------------------------------------------------|
| Launching   | Install software, pull config, register with service mesh |
| Terminating | Drain connections, deregister from service discovery      |

```
Instance launching → Hook (wait) → Custom action (Lambda/SSM) → Continue → InService
Instance terminating → Hook (wait) → Drain logs, deregister → Continue → Terminated
```

Default heartbeat timeout: 3600 seconds (1 hour). Extend or complete via `complete-lifecycle-action`.

## 81.8 16.8 Mixed instances and Spot

ASGs support **mixed instance policies** combining On-Demand and Spot:

```
resource "aws_autoscaling_group" "mixed" {
 mixed_instances_policy {
 launch_template {
 launch_template_specification {
 launch_template_id = aws_launch_template.web.id
 version = "$Latest"
 }
 override { instance_type = "t3.micro" }
 override { instance_type = "t3.small" }
 }
 instances_distribution {
 on_demand_base_capacity = 2
 on_demand_percentage_above_base_capacity = 25
 spot_allocation_strategy = "capacity-optimized"
 }
 }
}
```

This keeps 2 On-Demand instances as baseline, with 25% On-Demand above that and the rest Spot.

## 81.9 16.9 Warm pools

**Warm pools** pre-initialize instances in `Stopped` or `Running:Wait` state for faster scale-out:

- Instances in `Stopped` state incur EBS charges but no compute charges.
- Instances in `Running:Wait` incur compute charges but scale out in seconds.

## 81.10 16.10 Monitoring and troubleshooting

| Metric                  | Alarm threshold idea |
|-------------------------|----------------------|
| GroupDesiredCapacity    | Track scaling events |
| GroupInServiceInstances | Should match desired |



|                                        |                       |
|----------------------------------------|-----------------------|
| <code>GroupInServiceInstances</code>   | Should match desired  |
| <code>GroupTerminatingInstances</code> | Spike during scale-in |
| <code>WarmPoolDesiredCapacity</code>   | Warm pool sizing      |

```
aws autoscaling describe-auto-scaling-groups \
 --auto-scaling-group-names web-asg

aws autoscaling describe-scaling-activities \
 --auto-scaling-group-name web-asg \
 --max-records 10
```

### 81.10.1 Common issues

| Symptom                         | Cause                                                       |
|---------------------------------|-------------------------------------------------------------|
| Instances launch then terminate | Failing health checks; check grace period                   |
| ASG won't scale out             | Max capacity reached; scaling policy cooldown               |
| Uneven AZ distribution          | Check <code>AZRebalance</code> and subnet configuration     |
| Spot interruptions              | Use capacity-optimized allocation; diversify instance types |

## 81.11 16.11 Chapter summary

- **Auto Scaling groups** maintain desired capacity with min/max bounds.
- Integrate with **ALB** using `health_check_type = "ELB"` and target group registration.
- **Target tracking** scaling policies are the simplest and most effective for most workloads.
- **Instance refresh** enables rolling AMI updates without downtime.
- **Mixed instances** and **warm pools** optimize cost and scale-out speed.

## 81.12 Lab 16.1 — ASG with ALB

1. Create a launch template with a simple web server and `/health` endpoint.
2. Create an ASG (min 2, max 6) in private subnets attached to an ALB target group.
3. Add a target tracking policy on `ASGAverageCPUUtilization` at 50%.
4. Generate load with `ab` or `hey` and watch instances scale out.
5. Stop load and observe scale-in after cooldown.

## 81.13 Lab 16.2 — Instance refresh

1. Update the launch template AMI or user data.
2. Start an instance refresh with 50% checkpoint.
3. Monitor `describe-scaling-activities` until complete.
4. Verify all instances serve the updated content.

## 81.14 Review questions

1. What is the difference between `health_check_type = "EC2"` and `"ELB"`?
2. Why is a health check grace period important?
3. How does target tracking scaling differ from step scaling?
4. What is the purpose of lifecycle hooks?

5. How do warm pools improve scale-out speed?

---

Next: [\*Chapter 17 — Lambda\*](#)

# 82 Chapter 17: Lambda — Events, Concurrency, and Layers

AWS Handbook — Part IV, Pages 326–350

## 82.1 17.1 What is AWS Lambda?

**AWS Lambda** is a serverless compute service that runs code in response to events without provisioning or managing servers. You upload your function code, configure triggers, and AWS handles scaling, patching, and availability. You pay only for compute time consumed (per millisecond) and number of requests.

Lambda is ideal for event-driven workloads: API backends, data processing, scheduled tasks, and microservices glue.

## 82.2 17.2 Lambda fundamentals

| Concept            | Description                                          |
|--------------------|------------------------------------------------------|
| Function           | Your code + runtime + configuration                  |
| Handler            | Entry point (e.g., <code>index.handler</code> )      |
| Runtime            | Language environment (Python 3.12, Node.js 20, etc.) |
| Trigger            | Event source that invokes the function               |
| Execution role     | IAM role granting AWS service permissions            |
| Deployment package | .zip or container image (up to 10 GB for containers) |

### 82.2.1 Limits (selected)

| Resource                        | Limit                                       |
|---------------------------------|---------------------------------------------|
| Memory                          | 128 MB – 10,240 MB (CPU scales with memory) |
| Timeout                         | 900 seconds (15 minutes)                    |
| Deployment package (zip)        | 50 MB (direct upload), 250 MB (S3)          |
| Environment variables           | 4 KB total                                  |
| /tmp storage                    | 512 MB – 10,240 MB                          |
| Concurrent executions (default) | 1,000 per region (can increase)             |

## 82.3 17.3 Creating a Lambda function

### 82.3.1 AWS CLI

```
Create execution role
aws iam create-role \
 --role-name lambda-basic-role \
 --assume-role-policy-document '{
 "Version": "2012-10-17",
 "Statement": [{
 "Effect": "Allow",
```

```

 "Principal": {"Service": "lambda.amazonaws.com"},
 "Action": "sts:AssumeRole"
 }]
}'

aws iam attach-role-policy \
 --role-name lambda-basic-role \
 --policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole

Create function
aws lambda create-function \
 --function-name hello-world \
 --runtime python3.12 \
 --handler index.handler \
 --role arn:aws:iam::123456789012:role/lambda-basic-role \
 --zip-file fileb://function.zip

```

### 82.3.2 Python handler example

```

import json

def handler(event, context):
 name = event.get("name", "World")
 return {
 "statusCode": 200,
 "body": json.dumps(f"Hello, {name}!"),
 }

```

### 82.3.3 Terraform

```

resource "aws_lambda_function" "api" {
 function_name = "api-handler"
 runtime = "python3.12"
 handler = "app.handler"
 role = aws_iam_role.lambda.arn
 filename = "lambda.zip"
 source_code_hash = filebase64sha256("lambda.zip")

 memory_size = 256
 timeout = 30

 environment {
 variables = {
 TABLE_NAME = aws_dynamodb_table.items.name
 }
 }

 vpc_config {
 subnet_ids = aws_subnet.private[*].id
 security_group_ids = [aws_security_group.lambda.id]
 }
}

```

## 82.4 17.4 Event sources and triggers

Lambda integrates with 200+ AWS services and custom applications:

| Trigger                 | Pattern                       |
|-------------------------|-------------------------------|
| <b>API Gateway</b>      | HTTP API requests             |
| <b>S3</b>               | Object created/deleted events |
| <b>SQS</b>              | Message queue processing      |
| <b>SNS</b>              | Pub/sub notifications         |
| <b>EventBridge</b>      | Event bus rules               |
| <b>DynamoDB Streams</b> | Change data capture           |

|                          |                                 |
|--------------------------|---------------------------------|
| <b>Kinesis</b>           | Stream record processing        |
| <b>CloudWatch Events</b> | Scheduled (cron) execution      |
| <b>ALB</b>               | HTTP requests to Lambda targets |

### 82.4.1 S3 trigger example

```
resource "aws_s3_bucket_notification" "trigger" {
 bucket = aws_s3_bucket.uploads.id

 lambda_function {
 lambda_function_arn = aws_lambda_function.processor.arn
 events = ["s3:ObjectCreated:*"]
 filter_prefix = "uploads/"
 filter_suffix = ".csv"
 }
}
```

### 82.4.2 Scheduled execution

```
resource "aws_cloudwatch_event_rule" "daily" {
 name = "daily-cleanup"
 schedule_expression = "cron(0 6 * * ? *)" # 6 AM UTC daily
}

resource "aws_cloudwatch_event_target" "lambda" {
 rule = aws_cloudwatch_event_rule.daily.name
 target_id = "cleanup"
 arn = aws_lambda_function.cleanup.arn
}
```

## 82.5 17.5 Concurrency

**Concurrency** is the number of simultaneous executions of your function.

### 82.5.1 Types

| Type               | Description                                       |
|--------------------|---------------------------------------------------|
| <b>Unreserved</b>  | Shared pool; competes with other functions        |
| <b>Reserved</b>    | Guaranteed concurrency for a specific function    |
| <b>Provisioned</b> | Pre-warmed execution environments (no cold start) |

### 82.5.2 Concurrency formula

```
Account concurrency limit = 1,000 (default, per region)
Reserved concurrency for function X = 100
Unreserved pool = 1,000 - sum(all reserved)
```

### 82.5.3 Reserved concurrency

```
aws lambda put-function-concurrency \
 --function-name api-handler \
 --reserved-concurrent-executions 50
```

**Use reserved concurrency to:** - Guarantee capacity for critical functions. - Limit maximum concurrency (acts as a throttle). - Prevent one function from consuming all account concurrency.

### 82.5.4 Provisioned concurrency

Eliminates cold starts by keeping execution environments warm:

```
aws lambda put-provisioned-concurrency-config \
 --function-name api-handler \
 --provisioned-concurrent-executions 10 \
 --qualifier live
```

Costs: provisioned concurrency is billed even when idle. Use for latency-sensitive APIs.

## 82.6 17.6 Cold starts and performance

A **cold start** occurs when Lambda creates a new execution environment:

| Factor                  | Impact                                            |
|-------------------------|---------------------------------------------------|
| Runtime                 | Java/.NET slower than Python/Node.js              |
| Package size            | Larger = slower init                              |
| VPC                     | Adds ENI creation time (improved with Hyperplane) |
| Provisioned concurrency | Eliminates cold starts                            |

### 82.6.1 Optimization tips

- Keep deployment packages small; use **Lambda layers** for dependencies.
- Initialize SDK clients **outside** the handler (reused across invocations).
- Use **ARM64 (Graviton2)** for 20% better price-performance.
- Choose the right memory (more memory = more CPU = potentially faster).
- Use **SnapStart** for Java functions.

## 82.7 17.7 Lambda layers

**Layers** package libraries, custom runtimes, or other dependencies separately from your function code:

```
Function code (handler.py) + Layer (boto3, pandas, numpy) = Deployment
```

### 82.7.1 Benefits

- Share dependencies across multiple functions.
- Keep deployment packages small.
- Separate dependency updates from function code changes.

### 82.7.2 Creating a layer

```
mkdir -p python/lib/python3.12/site-packages
pip install requests -t python/lib/python3.12/site-packages/
zip -r layer.zip python

aws lambda publish-layer-version \
 --layer-name python-requests \
 --zip-file fileb://layer.zip \
 --compatible-runtimes python3.12
```

### 82.7.3 Attaching a layer

```
resource "aws_lambda_function" "api" {
 # ...
 layers = [
```

```

aws_lambda_layer_version.deps.arn,
"arn:aws:lambda:us-east-1:770693421928:layer:Klayers-p312-requests:1" # public layer
}
}

```

## 82.8 17.8 Lambda in a VPC

Place Lambda in a VPC to access private resources (RDS, ElastiCache, internal APIs):

```

vpc_config {
 subnet_ids = aws_subnet.private[*].id
 security_group_ids = [aws_security_group.lambda.id]
}

```

**Requirements:** - Subnets must have routes to the resource (not necessarily NAT for VPC-internal traffic). - Security groups on Lambda and target must allow communication. - Hyperplane ENIs reduce cold start penalty but still add some latency.

## 82.9 17.9 Error handling and retries

| Trigger type                        | Retry behavior                                            |
|-------------------------------------|-----------------------------------------------------------|
| Synchronous (API GW, ALB)           | Caller handles retry                                      |
| Asynchronous (S3, SNS, EventBridge) | 2 automatic retries, then DLQ                             |
| Stream (Kinesis, DynamoDB)          | Retry until data expires; bisect on failure               |
| SQS                                 | Retry based on visibility timeout; DLQ after max receives |

### 82.9.1 Dead letter queue (DLQ)

```

resource "aws_lambda_function_event_invoke_config" "api" {
 function_name = aws_lambda_function.api.function_name

 destination_config {
 on_failure {
 destination = aws_sqs_queue.dlq.arn
 }
 }
}

```

## 82.10 17.10 Monitoring

| Tool                      | Purpose                                                        |
|---------------------------|----------------------------------------------------------------|
| <b>CloudWatch Metrics</b> | Invocations, Duration, Errors, Throttles, ConcurrentExecutions |
| <b>CloudWatch Logs</b>    | Function stdout/stderr (auto-created log group)                |
| <b>X-Ray</b>              | Distributed tracing (enable active tracing)                    |
| <b>Lambda Insights</b>    | Enhanced metrics (init duration, memory usage)                 |

```

aws logs filter-log-events \
 --log-group-name /aws/lambda/api-handler \
 --filter-pattern "ERROR" \
 --start-time $(date -d '1 hour ago' +%s000)

```

---

## 82.11 17.11 Chapter summary

- **Lambda** runs event-driven code without server management; pay per invocation and duration.
  - **Triggers** connect Lambda to 200+ event sources (API Gateway, S3, SQS, schedules).
  - **Concurrency** controls — reserved (guarantee/limit), provisioned (no cold start).
  - **Layers** separate dependencies from function code for reuse and smaller packages.
  - Configure **DLQs**, **VPC access**, and **monitoring** for production reliability.
- 

## 82.12 Lab 17.1 — S3-triggered processor

1. Create a Lambda function that processes CSV files uploaded to S3.
2. Configure an S3 event trigger for `uploads/*.csv`.
3. Upload a test CSV and verify the function logs and processes it.
4. Add error handling and a DLQ for failures.

## 82.13 Lab 17.2 — API with provisioned concurrency

1. Create a Lambda function behind API Gateway HTTP API.
  2. Measure cold start latency with `curl -w "%{time_total}"`.
  3. Enable provisioned concurrency (5) and measure again.
  4. Compare p50 and p99 latency.
- 

## 82.14 Review questions

1. What is the maximum Lambda timeout, and when would you need it?
  2. How does reserved concurrency differ from provisioned concurrency?
  3. Why should SDK clients be initialized outside the handler function?
  4. What happens to failed asynchronous invocations after retries are exhausted?
  5. What are the trade-offs of running Lambda in a VPC?
- 

Next: [Chapter 18 — ECS & Fargate](#)



# 83 Chapter 18: ECS & Fargate

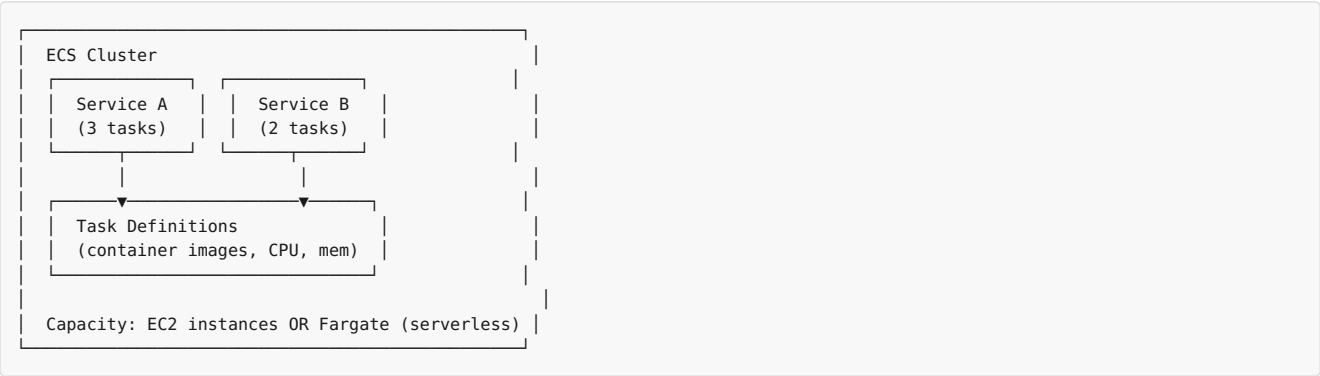
AWS Handbook — Part IV, Pages 351–375

## 83.1 18.1 Container orchestration on AWS

**Amazon Elastic Container Service (ECS)** is AWS’s native container orchestration platform. It schedules, deploys, and manages Docker containers across a cluster of EC2 instances or serverlessly via **AWS Fargate**. ECS integrates deeply with AWS networking, IAM, load balancing, and monitoring.

For teams that want AWS-native container management without the complexity of Kubernetes, ECS (especially with Fargate) is the recommended choice.

## 83.2 18.2 ECS architecture



### 83.2.1 Key components

| Component          | Description                                              |
|--------------------|----------------------------------------------------------|
| Cluster            | Logical grouping of tasks/services                       |
| Task definition    | Blueprint: image, CPU, memory, ports, env vars, IAM role |
| Task               | Running instance of a task definition (1+ containers)    |
| Service            | Maintains desired count of tasks; integrates with ALB    |
| Container instance | EC2 host running the ECS agent (EC2 launch type only)    |

## 83.3 18.3 EC2 vs Fargate launch types

| Feature        | EC2 Launch Type                      | Fargate Launch Type            |
|----------------|--------------------------------------|--------------------------------|
| Infrastructure | You manage EC2 instances             | AWS manages infrastructure     |
| Pricing        | EC2 instance hours                   | Per vCPU/memory per task hour  |
| Scaling        | ASG for instances + service scaling  | Service scaling only           |
| Control        | Host access, GPU, custom AMIs        | No host access                 |
| Best for       | Cost optimization, GPU, daemon tasks | Simplicity, variable workloads |

**Recommendation:** Start with Fargate. Move to EC2 if you need GPU, persistent host storage, or significant cost optimization at scale.

## 83.4 18.4 Task definitions

A **task definition** is a JSON blueprint specifying how containers run:

```
{
 "family": "web-app",
 "networkMode": "awsvpc",
 "requiresCompatibilities": ["FARGATE"],
 "cpu": "256",
 "memory": "512",
 "executionRoleArn": "arn:aws:iam::123456789012:role/ecsTaskExecutionRole",
 "taskRoleArn": "arn:aws:iam::123456789012:role/webAppTaskRole",
 "containerDefinitions": [
 {
 "name": "web",
 "image": "123456789012.dkr.ecr.us-east-1.amazonaws.com/web-app:latest",
 "portMappings": [
 { "containerPort": 8080, "protocol": "tcp" }
],
 "environment": [
 { "name": "ENV", "value": "production" }
],
 "secrets": [
 { "name": "DB_PASSWORD", "valueFrom": "arn:aws:secretsmanager:us-east-1:123456789012:secret:db-pass" }
],
 "logConfiguration": {
 "logDriver": "awslogs",
 "options": {
 "awslogs-group": "/ecs/web-app",
 "awslogs-region": "us-east-1",
 "awslogs-stream-prefix": "web"
 }
 },
 "healthCheck": {
 "command": ["CMD-SHELL", "curl -f http://localhost:8080/health || exit 1"],
 "interval": 30,
 "timeout": 5,
 "retries": 3
 }
 }
]
}
```

### 83.4.1 IAM roles

| Role                | Purpose                                              |
|---------------------|------------------------------------------------------|
| Task execution role | Pull images from ECR, write logs, fetch secrets      |
| Task role           | Permissions for the application (S3, DynamoDB, etc.) |

## 83.5 18.5 ECS services and load balancing

An **ECS service** maintains a desired number of running tasks and optionally registers them with a load balancer.

### 83.5.1 Terraform ECS Fargate service

```
resource "aws_ecs_service" "web" {
 name = "web-service"
 cluster = aws_ecs_cluster.main.id
 task_definition = aws_ecs_task_definition.web.arn
 desired_count = 3
 launch_type = "FARGATE"
```

```

network_configuration {
 subnets = aws_subnet.private[*].id
 security_groups = [aws_security_group.ecs_tasks.id]
 assign_public_ip = false
}

load_balancer {
 target_group_arn = aws_lb_target_group.web.arn
 container_name = "web"
 container_port = 8080
}

deployment_configuration {
 maximum_percent = 200
 minimum_healthy_percent = 100
}

depends_on = [aws_lb_listener.https]
}

```

### 83.5.2 Deployment types

| Type                  | Description                                         |
|-----------------------|-----------------------------------------------------|
| <b>Rolling update</b> | Replace tasks incrementally (default)               |
| <b>Blue/green</b>     | Via CodeDeploy; shift traffic between target groups |
| <b>Canary</b>         | Via CodeDeploy; gradual traffic shift               |

## 83.6 18.6 Auto Scaling ECS services

Scale tasks based on CloudWatch metrics:

```

resource "aws_appautoscaling_target" "ecs" {
 max_capacity = 10
 min_capacity = 2
 resource_id = "service/${aws_ecs_cluster.main.name}/${aws_ecs_service.web.name}"
 scalable_dimension = "ecs:service:DesiredCount"
 service_namespace = "ecs"
}

resource "aws_appautoscaling_policy" "cpu" {
 name = "cpu-scaling"
 policy_type = "TargetTrackingScaling"
 resource_id = aws_appautoscaling_target.ecs.resource_id
 scalable_dimension = aws_appautoscaling_target.ecs.scalable_dimension
 service_namespace = aws_appautoscaling_target.ecs.service_namespace

 target_tracking_scaling_policy_configuration {
 predefined_metric_specification {
 predefined_metric_type = "ECSServiceAverageCPUUtilization"
 }
 target_value = 70.0
 }
}

```

## 83.7 18.7 Amazon ECR

**Elastic Container Registry (ECR)** stores Docker images:

```

Authenticate
aws ecr get-login-password --region us-east-1 | \
 docker login --username AWS --password-stdin 123456789012.dkr.ecr.us-east-1.amazonaws.com

Build and push

```

```
docker build -t web-app .
docker tag web-app:latest 123456789012.dkr.ecr.us-east-1.amazonaws.com/web-app:latest
docker push 123456789012.dkr.ecr.us-east-1.amazonaws.com/web-app:latest
```

### 83.7.1 ECR features

| Feature                          | Purpose                                      |
|----------------------------------|----------------------------------------------|
| Image scanning                   | Vulnerability scanning on push               |
| Lifecycle policies               | Auto-delete old images                       |
| Cross-account/region replication | DR and multi-account                         |
| Pull-through cache               | Cache upstream registries (Docker Hub, etc.) |

## 83.8 18.8 Networking modes

| Mode   | Use case                                                |
|--------|---------------------------------------------------------|
| awsvpc | Required for Fargate; each task gets its own ENI and IP |
| bridge | EC2 only; Docker bridge networking (legacy)             |
| host   | EC2 only; task uses host network directly               |

Fargate requires `awsvpc` — plan IP address capacity in subnets (each task consumes one IP).

## 83.9 18.9 Service Connect and Service Discovery

| Feature                     | Purpose                                                          |
|-----------------------------|------------------------------------------------------------------|
| Cloud Map service discovery | DNS-based discovery ( <code>api.local</code> )                   |
| ECS Service Connect         | Built-in service mesh; observability, retries, traffic splitting |

Service Connect simplifies inter-service communication without managing a full service mesh.

## 83.10 18.10 ECS Exec

ECS Exec provides interactive shell access to running containers (like `kubectl exec`):

```
aws ecs execute-command \
 --cluster my-cluster \
 --task abc123 \
 --container web \
 --interactive \
 --command "/bin/bash"
```

Requires SSM permissions on the task role and `enableExecuteCommand` on the service.

## 83.11 18.11 Chapter summary

- **ECS** orchestrates containers on EC2 or **Fargate** (serverless).
- **Task definitions** define container images, resources, and IAM roles.

- **Services** maintain desired task count and integrate with ALB.
  - **ECR** stores container images with scanning and lifecycle policies.
  - Use **awsvpc** networking, **auto scaling**, and **ECS Exec** for production operations.
- 

## 83.12 Lab 18.1 — Fargate web service

1. Create an ECR repository and push a simple web container.
2. Create an ECS cluster, task definition, and Fargate service (2 tasks).
3. Attach an ALB and verify HTTP access.
4. Scale to 4 tasks and observe load distribution.

## 83.13 Lab 18.2 — ECS Exec debugging

1. Enable ECS Exec on your service.
  2. Connect to a running container and inspect logs, env vars, and network.
  3. Use ECS Exec to troubleshoot a misconfigured environment variable.
- 

## 83.14 Review questions

1. What is the difference between a task definition and a task?
  2. When would you choose EC2 launch type over Fargate?
  3. What are the two IAM roles in an ECS task definition?
  4. Why does Fargate require awsvpc networking mode?
  5. How does ECS service auto scaling differ from EC2 Auto Scaling?
- 

Next: [Chapter 19 — EKS](#)

# 84 Chapter 19: EKS — Managed Kubernetes on AWS

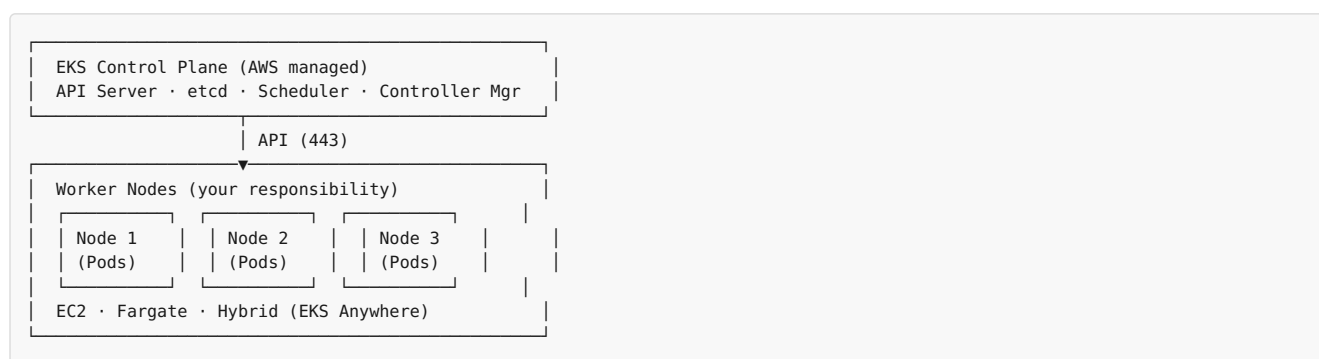
AWS Handbook — Part IV, Pages 376–400

## 84.1 19.1 Why Amazon EKS?

**Amazon Elastic Kubernetes Service (EKS)** is AWS’s managed Kubernetes offering. Kubernetes has become the de facto standard for container orchestration at scale, and EKS provides a certified, managed control plane with deep AWS integration.

Choose EKS when you need Kubernetes portability, a rich ecosystem (Helm, operators, service mesh), multi-cloud strategy, or your team already has Kubernetes expertise.

## 84.2 19.2 EKS architecture



### 84.2.1 What AWS manages

- Kubernetes control plane (API server, etcd, scheduler, controllers)
- Control plane high availability across 3 AZs
- Control plane patching and upgrades
- Integrated logging and monitoring

### 84.2.2 What you manage

- Worker nodes (EC2, Fargate, or hybrid)
- Node scaling and patching
- Application deployments
- Networking (CNI plugin)
- Add-ons and cluster configuration

## 84.3 19.3 Cluster creation

### 84.3.1 eksctl (recommended for getting started)

```
Install eksctl
curl -sLO "https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_${uname -s}_amd64.tar.gz"
tar -xzf eksctl_*.tar.gz -C /tmp && sudo mv /tmp/eksctl /usr/local/bin

Create cluster with managed node group
eksctl create cluster \
 --name production \
 --region us-east-1 \
```

```
--version 1.29 \
--nodegroup-name standard-workers \
--node-type t3.medium \
--nodes 3 \
--nodes-min 2 \
--nodes-max 6 \
--managed
```

## 84.3.2 Terraform

```
module "eks" {
 source = "terraform-aws-modules/eks/aws"
 version = "~> 20.0"

 cluster_name = "production"
 cluster_version = "1.29"

 vpc_id = module.vpc.vpc_id
 subnet_ids = module.vpc.private_subnets

 eks_managed_node_groups = {
 general = {
 min_size = 2
 max_size = 10
 desired_size = 3
 instance_types = ["t3.medium"]
 }
 }

 enable_cluster_creator_admin_permissions = true
}
```

## 84.4 19.4 Node groups

| Type                      | Description                                           |
|---------------------------|-------------------------------------------------------|
| <b>Managed node group</b> | AWS manages node lifecycle (launch, drain, terminate) |
| <b>Self-managed</b>       | You manage ASG and node provisioning                  |
| <b>Fargate profile</b>    | Serverless pods; no nodes to manage                   |

### 84.4.1 Managed node group features

- Automatic AMI updates via EKS optimized AMIs
- Integrated with Cluster Autoscaler and Karpenter
- Rolling updates with drain and cordon
- Launch templates for customization

### 84.4.2 Karpenter (recommended autoscaler)

**Karpenter** provisions nodes dynamically based on pending pod requirements—faster and more cost-efficient than Cluster Autoscaler:

```
apiVersion: karpenter.sh/v1beta1
kind: NodePool
metadata:
 name: default
spec:
 template:
 spec:
 requirements:
 - key: karpenter.sh/capacity-type
 operator: In
 values: ["spot", "on-demand"]
 - key: node.kubernetes.io/instance-type
 operator: In
 values: ["t3.medium", "t3.large", "m5.large"]
```

```
limits:
 cpu: 100
disruption:
 consolidationPolicy: WhenUnderutilized
```

## 84.5 19.5 Networking

### 84.5.1 VPC CNI

EKS uses the **Amazon VPC CNI plugin** — each pod gets a real VPC IP address:

- Plan IP capacity: each pod consumes an IP from the subnet.
- Use **prefix delegation** or **custom networking** for large clusters.
- Security groups can be applied directly to pods (Security Groups for Pods).

### 84.5.2 Service types

| Type         | AWS integration                                  |
|--------------|--------------------------------------------------|
| ClusterIP    | Internal only                                    |
| NodePort     | Exposes on node port                             |
| LoadBalancer | Creates ALB/NLB via AWS Load Balancer Controller |
| Ingress      | ALB Ingress Controller for HTTP routing          |

### 84.5.3 AWS Load Balancer Controller

```
helm repo add eks https://aws.github.io/eks-charts
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
 -n kube-system \
 --set clusterName=production \
 --set serviceAccount.create=false \
 --set serviceAccount.name=aws-load-balancer-controller
```

## 84.6 19.6 IAM integration

### 84.6.1 IRSA (IAM Roles for Service Accounts)

Pods assume IAM roles via OIDC federation — no need for node-level credentials:

```
apiVersion: v1
kind: ServiceAccount
metadata:
 name: app-sa
 annotations:
 eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/app-role

apiVersion: apps/v1
kind: Deployment
spec:
 template:
 spec:
 serviceAccountName: app-sa
 containers:
 - name: app
 image: my-app:latest
```

### 84.6.2 EKS Pod Identity (newer)

Simpler alternative to IRSA; assigns IAM roles to pods via the EKS API without OIDC provider setup.



## 84.7 19.7 Essential add-ons

| Add-on                         | Purpose                         |
|--------------------------------|---------------------------------|
| CoreDNS                        | Cluster DNS                     |
| kube-proxy                     | Network proxy on nodes          |
| VPC CNI                        | Pod networking                  |
| EBS CSI Driver                 | Persistent volumes (EBS)        |
| EFS CSI Driver                 | Shared persistent volumes (EFS) |
| Metrics Server                 | HPA resource metrics            |
| Cluster Autoscaler / Karpenter | Node autoscaling                |

```
aws eks create-addon \
 --cluster-name production \
 --addon-name aws-ebs-csi-driver \
 --service-account-role-arn arn:aws:iam::123456789012:role/AmazonEKS_EBS_CSI_DriverRole
```

## 84.8 19.8 Storage

| Type           | Use case                      | Provisioner    |
|----------------|-------------------------------|----------------|
| EBS (gp3)      | Single-pod persistent storage | EBS CSI Driver |
| EFS            | Shared storage across pods    | EFS CSI Driver |
| FSx for Lustre | High-performance computing    | FSx CSI Driver |
| emptyDir       | Ephemeral pod storage         | Built-in       |

### 84.8.1 PersistentVolumeClaim example

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: app-data
spec:
 accessModes: [ReadWriteOnce]
 storageClassName: gp3
 resources:
 requests:
 storage: 20Gi
```

## 84.9 19.9 Security

| Layer            | Control                                       |
|------------------|-----------------------------------------------|
| Control plane    | Private endpoint, public endpoint, or both    |
| Network policies | Calico or VPC CNI network policies            |
| Pod security     | Pod Security Standards (restricted, baseline) |
| Secrets          | External Secrets Operator + Secrets Manager   |
| Image scanning   | ECR scanning, admission controllers           |

|                      |                                      |
|----------------------|--------------------------------------|
| <b>Audit logging</b> | EKS control plane logs to CloudWatch |
|----------------------|--------------------------------------|

```
aws eks update-cluster-config \
 --name production \
 --logging '{"clusterLogging":[{"types":["api","audit","authenticator"],"enabled":true}]}'
```

## 84.10 19.10 Upgrades and operations

### 84.10.1 Cluster upgrade path

1. Upgrade control plane to target Kubernetes version.
2. Upgrade add-ons (CoreDNS, kube-proxy, VPC CNI).
3. Upgrade node groups (rolling replacement).
4. Update application manifests for deprecated APIs.

```
Check deprecated APIs
kubectl pluto detect-deprecated-apis --target-versions k8s=v1.29.0

Upgrade control plane
aws eks update-cluster-version --name production --kubernetes-version 1.29
```

### 84.10.2 Useful kubectl commands

```
kubectl get nodes -o wide
kubectl top nodes
kubectl get pods --all-namespaces -o wide
kubectl describe pod <name> -n <namespace>
kubectl logs -f <pod> -c <container>
```

## 84.11 19.11 EKS vs ECS decision matrix

| Factor                      | Choose EKS | Choose ECS |
|-----------------------------|------------|------------|
| Team Kubernetes expertise   | Yes        | No         |
| Multi-cloud portability     | Yes        | No         |
| AWS-native simplicity       | No         | Yes        |
| Ecosystem (Helm, operators) | Yes        | Limited    |
| Operational overhead        | Higher     | Lower      |
| Fargate support             | Yes        | Yes        |

## 84.12 19.12 Chapter summary

- **EKS** provides a managed Kubernetes control plane with AWS integrations.
- **Managed node groups**, **Fargate**, and **Karpenter** handle worker capacity.
- **IRSA** and **Pod Identity** grant fine-grained IAM permissions to pods.
- **AWS Load Balancer Controller** and **EBS/EFS CSI drivers** are essential add-ons.
- Plan **IP capacity**, **upgrades**, and **security** from day one.

## 84.13 Lab 19.1 — EKS cluster deployment

1. Create an EKS cluster with eksctl (3-node managed node group).
2. Deploy a sample application with a LoadBalancer service.
3. Configure IRSA for an S3-accessing pod.
4. Install Metrics Server and verify `kubectl top nodes`.

## 84.14 Lab 19.2 — Persistent storage

1. Install the EBS CSI driver add-on.
  2. Create a StorageClass, PVC, and Pod mounting the volume.
  3. Write data to the volume, delete the pod, recreate it, and verify data persists.
- 

## 84.15 Review questions

1. What components of Kubernetes does AWS manage in EKS?
  2. How does IRSA enable pods to access AWS services securely?
  3. Why is IP address planning critical for EKS clusters?
  4. What is the recommended order for EKS cluster upgrades?
  5. When would you choose ECS over EKS?
- 

Next: [Chapter 20 — S3 Fundamentals](#)

# 85 Chapter 20: S3 — Buckets, Objects, Versioning, and Lifecycle

AWS Handbook — Part V, Pages 401–425

## 85.1 20.1 Amazon S3 overview

**Simple Storage Service (S3)** is object storage: store files (objects) in containers (buckets). Virtually unlimited scale; 99.999999999% durability.

Use cases:

- Static websites and assets
- Data lakes and backups
- Application uploads
- Log archives
- Terraform state files

## 85.2 20.2 Core concepts

| Concept | Description                                        |
|---------|----------------------------------------------------|
| Bucket  | Global unique name; lives in a region              |
| Object  | File + metadata; up to 5 TB                        |
| Key     | Object path (e.g., <code>images/photo.jpg</code> ) |
| Prefix  | Simulated folders via key prefixes                 |

```
aws s3 mb s3://my-unique-bucket-name-12345
aws s3 cp report.pdf s3://my-bucket/reports/2026/
aws s3 ls s3://my-bucket/reports/
aws s3 sync ./local-dir s3://my-bucket/backup/
aws s3 rm s3://my-bucket/old/ --recursive
```

## 85.3 20.3 Storage classes

| Class                | Use case                   | Retrieval                     |
|----------------------|----------------------------|-------------------------------|
| Standard             | Frequent access            | Immediate                     |
| Standard-IA          | Infrequent access          | Immediate, min storage charge |
| Glacier Instant      | Archive, instant retrieval | Immediate                     |
| Glacier Flexible     | Archive                    | Minutes to hours              |
| Glacier Deep Archive | Long-term archive          | 12–48 hours                   |
| Intelligent-Tiering  | Unknown access patterns    | Auto-moves tiers              |

## 85.4 20.4 Versioning and lifecycle

### 85.4.1 Versioning

Protects against accidental deletes; keeps all versions of objects.

```
aws s3api put-bucket-versioning \
 --bucket my-bucket \
 --versioning-configuration Status=Enabled
```

### 85.4.2 Lifecycle rules

Automatically transition or expire objects:

```
{
 "Rules": [{
 "ID": "archive-old-logs",
 "Status": "Enabled",
 "Filter": { "Prefix": "logs/" },
 "Transitions": [{
 "Days": 30,
 "StorageClass": "STANDARD_IA"
 }, {
 "Days": 90,
 "StorageClass": "GLACIER"
 }],
 "Expiration": { "Days": 365 }
 }]
}
```

## 85.5 20.5 Security

### 85.5.1 Block Public Access (account and bucket level)

**Enable by default** on all buckets unless hosting a public static site intentionally.

### 85.5.2 Encryption

| Method      | Description            |
|-------------|------------------------|
| SSE-S3      | AWS-managed keys       |
| SSE-KMS     | KMS keys; audit trail  |
| SSE-C       | Customer-provided keys |
| Client-side | Encrypt before upload  |

### 85.5.3 Bucket policies vs ACLs

- **Bucket policies** (JSON) — Preferred; resource-based IAM
- **ACLs** — Legacy; avoid for new projects

Example: allow CloudFront OAC to read objects (not public internet).

## 85.6 20.6 Static website hosting

```
Enable static website hosting on bucket
aws s3 website s3://my-bucket/ --index-document index.html
```

Front with **CloudFront** + **ACM certificate** for HTTPS. Do not expose bucket directly to public.

---

## 85.7 20.7 S3 performance tips

- Use **multipart upload** for files > 100 MB
  - Prefix keys for parallelism (S3 scales automatically)
  - **S3 Transfer Acceleration** for long-distance uploads
  - **S3 Inventory** for auditing object metadata
- 

## 85.8 20.8 Chapter summary

- S3 stores **objects in buckets** with global-unique names.
  - Enable **versioning** and **lifecycle** for data protection and cost control.
  - **Block public access**; use bucket policies and CloudFront for controlled delivery.
- 

## 85.9 Lab 20.1

1. Create bucket with versioning and default encryption (SSE-S3).
  2. Upload files; delete one; restore previous version.
  3. Add lifecycle rule to transition `logs/` to IA after 30 days.
  4. Host static `index.html` behind CloudFront.
- 

Next: [Chapter 35 — CloudWatch](#)

# 86 Chapter 21: S3 Security, Encryption, and Static Websites

AWS Handbook — Part V, Pages 426–445

## 86.1 21.1 S3 security model

Amazon S3 security operates at multiple layers: **identity-based policies** (IAM), **resource-based policies** (bucket policies), **ACLs** (legacy), **block public access** settings, and **encryption**. Understanding how these layers interact is critical for preventing data breaches—the majority of S3 incidents involve misconfigured public access.

## 86.2 21.2 Block Public Access

**S3 Block Public Access** is the first line of defense. Enable it at the **account level** and verify it on every bucket:

| Setting               | Effect                                           |
|-----------------------|--------------------------------------------------|
| BlockPublicAcls       | Blocks new public ACLs                           |
| IgnorePublicAcls      | Ignores existing public ACLs                     |
| BlockPublicPolicy     | Blocks bucket policies granting public access    |
| RestrictPublicBuckets | Restricts access to buckets with public policies |

```
aws s3control put-public-access-block \
 --account-id 123456789012 \
 --public-access-block-configuration \
 BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
```

**Best practice:** Enable account-level block public access. Only disable for buckets that intentionally serve public content (static websites, CDN origins).

## 86.3 21.3 Bucket policies

Bucket policies are **resource-based** JSON policies attached to buckets:

```
{
 "Version": "2012-10-17",
 "Statement": [
 {
 "Sid": "AllowCloudFrontOAC",
 "Effect": "Allow",
 "Principal": {
 "Service": "cloudfront.amazonaws.com"
 },
 "Action": "s3:GetObject",
 "Resource": "arn:aws:s3::my-static-site/*",
 "Condition": {
 "StringEquals": {
 "AWS:SourceArn": "arn:aws:cloudfront::123456789012:distribution/E1234567890"
 }
 }
 }
]
}
```

### 86.3.1 Common policy patterns

| Pattern                  | Principal                                       | Use case               |
|--------------------------|-------------------------------------------------|------------------------|
| Cross-account access     | Another account's IAM role                      | Shared data lake       |
| CloudFront OAC           | cloudfront.amazonaws.com                        | Static website via CDN |
| VPC endpoint restriction | "*" with aws:SourceVpce condition               | Private S3 access      |
| Deny unencrypted uploads | "*" with Deny + s3:x-amz-server-side-encryption | Compliance             |

### 86.3.2 Deny insecure transport

```
{
 "Sid": "DenyInsecureTransport",
 "Effect": "Deny",
 "Principal": "*",
 "Action": "s3:*",
 "Resource": [
 "arn:aws:s3::my-bucket",
 "arn:aws:s3::my-bucket/*"
],
 "Condition": {
 "Bool": {
 "aws:SecureTransport": "false"
 }
 }
}
```

## 86.4 21.4 IAM policies for S3

Identity-based policies attach to IAM users, groups, or roles:

```
{
 "Version": "2012-10-17",
 "Statement": [
 {
 "Effect": "Allow",
 "Action": [
 "s3:GetObject",
 "s3:PutObject"
],
 "Resource": "arn:aws:s3::app-data-${aws:username}/*"
 }
]
}
```

### 86.4.1 Policy evaluation for S3

Request → Block Public Access → Bucket Policy → IAM Policy → ACL → Allow/Deny

Explicit **Deny** in any policy wins. Both identity and resource policies must allow for cross-account access.

## 86.5 21.5 Encryption

S3 supports encryption at rest and in transit.

### 86.5.1 Server-side encryption (SSE)

| Type | Key management | Use case |
|------|----------------|----------|
|------|----------------|----------|



|                 |                                             |                                          |
|-----------------|---------------------------------------------|------------------------------------------|
| <b>SSE-S3</b>   | AWS-managed (AES-256)                       | Default; no key management               |
| <b>SSE-KMS</b>  | AWS KMS customer-managed or AWS-managed key | Audit trail, key rotation, cross-account |
| <b>SSE-C</b>    | Customer-provided key                       | You manage keys outside AWS              |
| <b>DSSE-KMS</b> | Dual-layer KMS encryption                   | Highest security requirements            |

### 86.5.2 Default encryption

```
aws s3api put-bucket-encryption \
 --bucket my-bucket \
 --server-side-encryption-configuration '{
 "Rules": [{
 "ApplyServerSideEncryptionByDefault": {
 "SSEAlgorithm": "aws:kms",
 "KMSEMasterKeyID": "arn:aws:kms:us-east-1:123456789012:key/abc-123"
 },
 "BucketKeyEnabled": true
 }]
 }'
```

**Bucket Key** reduces KMS API calls (and costs) by using a bucket-level key that wraps object keys.

### 86.5.3 Client-side encryption

Encrypt data before uploading with the AWS Encryption SDK or your own encryption library. You manage keys entirely.

### 86.5.4 Encryption in transit

- All S3 API calls use HTTPS by default.
- Enforce with bucket policy `aws:SecureTransport` condition.
- TLS 1.2+ required.

## 86.6 21.6 Access Points and Object Lambda

### 86.6.1 S3 Access Points

Named network endpoints with dedicated bucket policies—simplify access for shared buckets:

```
Shared bucket → Access Point "finance" (finance team policy)
 → Access Point "engineering" (engineering team policy)
```

### 86.6.2 S3 Object Lambda

Transform objects on retrieval (resize images, redact PII, convert formats) without modifying the stored object.

## 86.7 21.7 Static website hosting

S3 can host static websites (HTML, CSS, JS, images):

### 86.7.1 Setup steps

1. Enable static website hosting on the bucket.
2. Set `index.html` as index document and `error.html` as error document.
3. Upload website files.
4. Configure bucket policy for public read (or use CloudFront OAC for private bucket).

```
aws s3 website s3://my-static-site/ \
--index-document index.html \
--error-document error.html
```

## 86.7.2 Production architecture (recommended)

User → Route 53 → CloudFront → S3 (private bucket via OAC)

↓  
ACM certificate (HTTPS)  
WAF (optional)

**Never expose S3 directly to the internet in production.** Use CloudFront with Origin Access Control (OAC) to keep the bucket private.

## 86.7.3 Terraform static site

```
resource "aws_s3_bucket" "site" {
 bucket = "my-static-site"
}

resource "aws_s3_bucket_website_configuration" "site" {
 bucket = aws_s3_bucket.site.id
 index_document { suffix = "index.html" }
 error_document { key = "error.html" }
}

resource "aws_cloudfront_distribution" "site" {
 origin {
 domain_name = aws_s3_bucket.site.bucket_regional_domain_name
 origin_id = "s3"
 origin_access_control_id = aws_cloudfront_origin_access_control.site.id
 }

 enabled = true
 default_root_object = "index.html"

 default_cache_behavior {
 allowed_methods = ["GET", "HEAD"]
 cached_methods = ["GET", "HEAD"]
 target_origin_id = "s3"
 viewer_protocol_policy = "redirect-to-https"
 forwarded_values {
 query_string = false
 cookies { forward = "none" }
 }
 }

 viewer_certificate {
 acm_certificate_arn = aws_acm_certificate.site.arn
 ssl_support_method = "sni-only"
 minimum_protocol_version = "TLSv1.2_2021"
 }

 restrictions {
 geo_restriction { restriction_type = "none" }
 }
}
```

## 86.8 21.8 S3 access logging and monitoring

| Feature                | Purpose                                   |
|------------------------|-------------------------------------------|
| Server access logging  | Log all bucket requests to another bucket |
| CloudTrail data events | API-level logging (GetObject, PutObject)  |
| S3 Storage Lens        | Organization-wide storage analytics       |
| S3 Inventory           | Scheduled reports of objects and metadata |

```
aws cloudtrail put-event-selectors \
 --trail-name org-trail \
 --event-selectors '[{
 "ReadWriteType": "All",
 "IncludeManagementEvents": true,
 "DataResources": [{
 "Type": "AWS::S3::Object",
 "Values": ["arn:aws:s3:::sensitive-bucket/"]
 }]
 }]
```

## 86.9 21.9 Pre-signed URLs and temporary access

Grant time-limited access without making objects public:

```
import boto3

s3 = boto3.client("s3")
url = s3.generate_presigned_url(
 "get_object",
 Params={"Bucket": "my-bucket", "Key": "report.pdf"},
 ExpiresIn=3600, # 1 hour
)
```

Use cases: download links, upload forms, sharing files with external parties.

## 86.10 21.10 Chapter summary

- Enable **Block Public Access** at the account level; only disable for intentional public buckets.
- Use **bucket policies** and **IAM policies** together for fine-grained access control.
- Encrypt with **SSE-KMS** for audit trails; enable **Bucket Key** to reduce KMS costs.
- Host static websites behind **CloudFront + OAC**, not direct S3 public access.
- Monitor with **CloudTrail data events**, **access logging**, and **Macie**.

## 86.11 Lab 21.1 — Secure bucket

1. Create a bucket with default SSE-KMS encryption and block public access.
2. Add a bucket policy denying unencrypted uploads and insecure transport.
3. Attempt to upload without encryption — verify denial.
4. Upload with `--server-side-encryption aws:kms` — verify success.

## 86.12 Lab 21.2 — Static website with CloudFront

1. Create an S3 bucket with static website files.
2. Create a CloudFront distribution with OAC (bucket stays private).
3. Configure Route 53 alias record and ACM certificate.
4. Verify HTTPS access through CloudFront.

## 86.13 Review questions

1. What are the four S3 Block Public Access settings?
  2. When would you use SSE-KMS over SSE-S3?
  3. Why should production static sites use CloudFront OAC instead of public bucket policies?
  4. How does a pre-signed URL provide temporary access?
  5. What is the purpose of S3 Bucket Key?
- 

Next: [\*Chapter 22 — EBS & EFS\*](#)

# 87 Chapter 22: EBS & EFS

AWS Handbook — Part V, Pages 446–465

## 87.1 22.1 Block vs file storage on AWS

AWS provides two primary storage services for compute workloads:

| Service    | Type               | Attachment                        | Use case                                      |
|------------|--------------------|-----------------------------------|-----------------------------------------------|
| Amazon EBS | Block storage      | Single EC2 instance (per volume)  | Boot volumes, databases, single-instance apps |
| Amazon EFS | File storage (NFS) | Multiple instances simultaneously | Shared files, content management, web serving |

Understanding when to use each—and their performance, durability, and cost characteristics—is essential for architecting reliable applications.

## 87.2 22.2 Amazon EBS deep dive

**Elastic Block Store (EBS)** provides persistent block-level storage volumes for EC2 instances. Think of EBS as a virtual hard drive.

### 87.2.1 Volume types

| Type              | Storage | IOPS                        | Throughput       | Use case                        |
|-------------------|---------|-----------------------------|------------------|---------------------------------|
| gp3               | SSD     | 3,000–16,000 (configurable) | 125–1,000 MB/s   | General purpose (default)       |
| gp2               | SSD     | 3–16,000 (burst/scaled)     | 128–250 MB/s     | Legacy general purpose          |
| io2 Block Express | SSD     | Up to 256,000               | Up to 4,000 MB/s | Mission-critical databases      |
| io1/io2           | SSD     | Up to 64,000                | Up to 1,000 MB/s | High-IOPS databases             |
| st1               | HDD     | 500 baseline                | Up to 500 MB/s   | Throughput-intensive (big data) |
| sc1               | HDD     | 250 baseline                | Up to 250 MB/s   | Cold data, infrequent access    |

**Recommendation:** Use **gp3** for most workloads. It offers better price-performance than gp2 with independently configurable IOPS and throughput.

### 87.2.2 Key characteristics

- **AZ-bound** — A volume exists in one AZ; snapshot and restore to another AZ.
- **Durability** — 99.999% (replicated within AZ).
- **Encryption** — All volume types support encryption (default with account setting).
- **Size** — 1 GB to 64 TB per volume.
- **Multi-attach** — io1/io2 support attachment to multiple instances (cluster file systems only).

### 87.2.3 Creating and attaching volumes

```

Create a gp3 volume
aws ec2 create-volume \
 --availability-zone us-east-1a \
 --size 100 \
 --volume-type gp3 \
 --iops 3000 \
 --throughput 125 \
 --encrypted \
 --tag-specifications 'ResourceType=volume,Tags=[{Key=Name,Value=data-vol}]'

Attach to instance
aws ec2 attach-volume \
 --volume-id vol-0abc123 \
 --instance-id i-0def456 \
 --device /dev/sdf

Format and mount (on the instance)
sudo mkfs -t xfs /dev/xvdf
sudo mkdir /data && sudo mount /dev/xvdf /data
echo '/dev/xvdf /data xfs defaults,nofail 0 2' | sudo tee -a /etc/fstab

```

## 87.2.4 Terraform

```

resource "aws_ebs_volume" "data" {
 availability_zone = "us-east-1a"
 size = 100
 type = "gp3"
 iops = 3000
 throughput = 125
 encrypted = true
 kms_key_id = aws_kms_key.ebs.arn

 tags = { Name = "app-data" }
}

resource "aws_volume_attachment" "data" {
 device_name = "/dev/sdf"
 volume_id = aws_ebs_volume.data.id
 instance_id = aws_instance.app.id
}

```

## 87.3 22.3 EBS snapshots

**Snapshots** are point-in-time backups stored in S3 (managed by AWS):

| Feature                      | Detail                                         |
|------------------------------|------------------------------------------------|
| <b>Incremental</b>           | Only changed blocks since last snapshot        |
| <b>Cross-region</b>          | Copy snapshots for DR                          |
| <b>Cross-account</b>         | Share snapshots with other accounts            |
| <b>Fast Snapshot Restore</b> | Pre-warm snapshots for immediate IOPS (costly) |
| <b>Lifecycle</b>             | Automate with Data Lifecycle Manager (DLM)     |

```

Create snapshot
aws ec2 create-snapshot \
 --volume-id vol-0abc123 \
 --description "Daily backup" \
 --tag-specifications 'ResourceType=snapshot,Tags=[{Key=Name,Value=daily-backup}]'

Create AMI from snapshot (for golden images)
aws ec2 register-image \
 --name "app-ami-v2" \
 --block-device-mappings '[{
 "DeviceName": "/dev/xvda",
 "Ebs": { "SnapshotId": "snap-0abc123", "VolumeSize": 20, "VolumeType": "gp3" }
}]'

```

### 87.3.1 DLM policy example

```
resource "aws_dlm_lifecycle_policy" "ebs_backup" {
 description = "Daily EBS snapshots"
 execution_role_arn = aws_iam_role.dlm.arn
 state = "ENABLED"

 policy_details {
 resource_types = ["VOLUME"]
 schedule {
 name = "daily"
 create_rule { interval = 24; interval_unit = "HOURS"; times = ["03:00"] }
 retain_rule { count = 7 }
 copy_tags = true
 }
 target_tags = { Backup = "true" }
 }
}
```

## 87.4 22.4 Amazon EFS deep dive

**Elastic File System (EFS)** provides scalable, elastic NFS file storage accessible from multiple EC2 instances, ECS tasks, and Lambda functions simultaneously.

### 87.4.1 Performance modes

| Mode            | Throughput                  | Use case                           |
|-----------------|-----------------------------|------------------------------------|
| General Purpose | Low latency                 | Web serving, CMS, dev environments |
| Max I/O         | Higher aggregate throughput | Big data, media processing         |

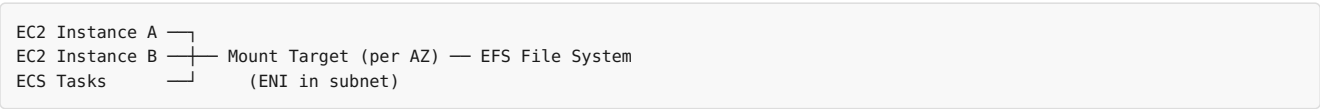
### 87.4.2 Throughput modes

| Mode              | Behavior                                    |
|-------------------|---------------------------------------------|
| Bursting          | Scales with storage size; burst credits     |
| Provisioned       | Set throughput independent of storage size  |
| Elastic (default) | Automatically scales throughput up and down |

### 87.4.3 Storage classes

| Class                  | Description                               |
|------------------------|-------------------------------------------|
| Standard               | Frequently accessed files                 |
| Infrequent Access (IA) | Lifecycle policy moves files after N days |
| Archive                | Rarely accessed; lowest cost              |
| One Zone               | Single AZ; lower cost, less resilience    |

### 87.4.4 EFS architecture



### 87.4.5 Creating EFS

```
Create file system
aws efs create-file-system \
```

```

--performance-mode generalPurpose \
--throughput-mode elastic \
--encrypted \
--tags Key=Name,Value=shared-data

Create mount targets in each AZ
aws efs create-mount-target \
--file-system-id fs-0abc123 \
--subnet-id subnet-0aaa \
--security-groups sg-0efs

Mount on EC2 (Amazon Linux)
sudo yum install -y amazon-efs-utils
sudo mount -t efs -o tls fs-0abc123:/ /mnt/efs

```

## 87.4.6 Terraform

```

resource "aws_efs_file_system" "shared" {
 creation_token = "shared-data"
 performance_mode = "generalPurpose"
 throughput_mode = "elastic"
 encrypted = true

 lifecycle_policy {
 transition_to_ia = "AFTER_30_DAYS"
 }

 tags = { Name = "shared-efs" }
}

resource "aws_efs_mount_target" "az" {
 count = length(var.private_subnet_ids)
 file_system_id = aws_efs_file_system.shared.id
 subnet_id = var.private_subnet_ids[count.index]
 security_groups = [aws_security_group.efs.id]
}

```

## 87.5 22.5 EBS vs EFS comparison

| Feature     | EBS                                        | EFS                 |
|-------------|--------------------------------------------|---------------------|
| Type        | Block                                      | File (NFS)          |
| Access      | Single instance (multi-attach for io1/io2) | Multiple instances  |
| AZ          | Single AZ                                  | Regional (multi-AZ) |
| Max size    | 64 TB                                      | Petabyte scale      |
| Latency     | Sub-millisecond                            | Low (NFS overhead)  |
| Cost        | Lower per GB                               | Higher per GB       |
| Boot volume | Yes                                        | No                  |
| Backup      | Snapshots                                  | AWS Backup          |

## 87.6 22.6 Instance Store (ephemeral)

**Instance store** volumes are physically attached to the host:

- **Highest IOPS** and lowest latency.
- **Ephemeral** — data lost on stop/terminate or hardware failure.
- Use for temporary data: caches, buffers, scratch space.
- Cannot be detached and reattached.



---

## 87.7 22.7 EBS optimization and monitoring

| Metric                             | What to watch                              |
|------------------------------------|--------------------------------------------|
| VolumeQueueLength                  | I/O wait; sustained > 1 may need more IOPS |
| VolumeReadBytes / VolumeWriteBytes | Throughput utilization                     |
| BurstBalance (gp2)                 | Burst credit depletion                     |
| PercentIOLimit (io1/io2)           | IOPS utilization                           |

### 87.7.1 Right-sizing tips

- Start with gp3; increase IOPS/throughput independently.
- Use CloudWatch metrics to identify I/O-bound volumes.
- Consider io2 Block Express for databases requiring > 64K IOPS.

---

## 87.8 22.8 Chapter summary

- **EBS** provides block storage for single-instance workloads; **gp3** is the default choice.
- **Snapshots** enable backup, AMI creation, and cross-region DR.
- **EFS** provides shared NFS file storage across multiple instances and AZs.
- Use **DLM** for automated snapshot lifecycle management.
- Choose EBS for databases and boot volumes; EFS for shared file access.

---

## 87.9 Lab 22.1 — EBS volume lifecycle

1. Launch an EC2 instance and attach a 20 GB gp3 volume.
2. Format, mount, and write test data.
3. Create a snapshot, terminate the instance, launch a new instance, and restore from snapshot.
4. Verify data persistence.

## 87.10 Lab 22.2 — Shared EFS

1. Create an EFS file system with mount targets in two AZs.
2. Launch EC2 instances in both AZs and mount the EFS.
3. Write a file from Instance A and read it from Instance B.
4. Configure a lifecycle policy to move files to IA after 30 days.

---

## 87.11 Review questions

1. What is the key difference between EBS and EFS access patterns?
2. Why is gp3 preferred over gp2 for new workloads?
3. What happens to instance store data when an EC2 instance is stopped?
4. How do EBS snapshots support disaster recovery?
5. When would you use EFS Infrequent Access storage class?

---

Next: [Chapter 23 — AWS Backup & DR](#)

# 88 Chapter 23: AWS Backup & Disaster Recovery Patterns

AWS Handbook — Part V, Pages 466–480

## 88.1 23.1 Why backup and disaster recovery matter

Data loss—from accidental deletion, ransomware, regional outages, or application bugs—can destroy businesses. AWS provides native backup capabilities and architectural patterns to protect workloads with defined **Recovery Time Objective (RTO)** and **Recovery Point Objective (RPO)** targets.

| Metric | Definition                          | Example                              |
|--------|-------------------------------------|--------------------------------------|
| RPO    | Maximum acceptable data loss (time) | 1 hour = lose at most 1 hour of data |
| RTO    | Maximum acceptable downtime         | 4 hours = back online within 4 hours |

## 88.2 23.2 AWS Backup

**AWS Backup** is a centralized backup service that automates and manages backups across AWS services:

| Supported resource | Backup method                  |
|--------------------|--------------------------------|
| EC2 / EBS          | EBS snapshots                  |
| RDS / Aurora       | DB snapshots                   |
| DynamoDB           | On-demand or continuous backup |
| EFS                | EFS backup                     |
| FSx                | FSx backup                     |
| S3                 | S3 backup (newer)              |
| Storage Gateway    | Volume backup                  |

### 88.2.1 Backup components

| Component        | Description                                        |
|------------------|----------------------------------------------------|
| Backup vault     | Container for recovery points; encryption with KMS |
| Backup plan      | Schedule, lifecycle, and rules                     |
| Backup rule      | Frequency, retention, target vault, copy actions   |
| Recovery point   | A restorable backup at a point in time             |
| Backup selection | Resources matched by tag or ID                     |

### 88.2.2 Terraform backup plan

```
resource "aws_backup_vault" "primary" {
 name = "primary-vault"
 kms_key_arn = aws_kms_key.backup.arn
}
```

```
resource "aws_backup_plan" "daily" {
 name = "daily-backup-plan"

 rule {
 rule_name = "daily-ec2-rds"
 target_vault_name = aws_backup_vault.primary.name
 schedule = "cron(0 5 * * ? *)" # 5 AM UTC daily

 lifecycle {
 delete_after = 30 # days
 }

 copy_action {
 destination_vault_arn = aws_backup_vault.dr.arn
 lifecycle {
 delete_after = 90
 }
 }
 }
}

resource "aws_backup_selection" "tagged" {
 name = "production-resources"
 plan_id = aws_backup_plan.daily.id
 iam_role_arn = aws_iam_role.backup.arn

 selection_tag {
 type = "STRINGEQUALS"
 key = "Backup"
 value = "daily"
 }
}
```

### 88.2.3 CLI

```
Start on-demand backup
aws backup start-backup-job \
 --backup-vault-name primary-vault \
 --resource-arn arn:aws:ec2:us-east-1:123456789012:volume/vol-0abc123 \
 --iam-role-arn arn:aws:iam::123456789012:role/AWSBackupDefaultServiceRole

List recovery points
aws backup list-recovery-points-by-backup-vault \
 --backup-vault-name primary-vault

Restore
aws backup start-restore-job \
 --recovery-point-arn arn:aws:backup:us-east-1:123456789012:recovery-point:abc123 \
 --iam-role-arn arn:aws:iam::123456789012:role/AWSBackupDefaultServiceRole \
 --metadata file://restore-metadata.json
```

## 88.3 23.3 DR strategies

AWS defines four disaster recovery strategies, ordered by increasing RTO/RPO and cost:

| Cost / Complexity | →           |              |                          |
|-------------------|-------------|--------------|--------------------------|
| Backup & Restore  | Pilot Light | Warm Standby | Multi-Site Active/Active |
| (highest RT0)     |             |              | (lowest RT0)             |
| (highest RPO)     |             |              | (lowest RPO)             |

### 88.3.1 Backup and Restore

- **RPO:** Hours to days | **RTO:** Hours to days
- Back up data to S3/EBS snapshots; restore in DR region when needed.
- Lowest cost; acceptable for non-critical workloads.

### 88.3.2 Pilot Light

- **RPO:** Minutes to hours | **RTO:** Hours
- Core infrastructure (database replica, AMIs) running at minimal scale in DR region.
- Scale up on failover.

### 88.3.3 Warm Standby

- **RPO:** Minutes | **RTO:** Minutes to hours
- Reduced-capacity version of production running in DR region.
- Scale up ASG/ECS on failover.

### 88.3.4 Multi-Site Active/Active

- **RPO:** Near zero | **RTO:** Near zero
- Full production in multiple regions simultaneously.
- Route 53 health checks shift traffic on failure.
- Highest cost and complexity.

## 88.4 23.4 Cross-region and cross-account backup

### 88.4.1 Cross-region copy

Protect against regional failures by copying backups to a DR region:

```
copy_action {
 destination_vault_arn = aws_backup_vault.dr_region.arn
 lifecycle {
 cold_storage_after = 30
 delete_after = 365
 }
}
```

### 88.4.2 Cross-account backup

Protect against account compromise (ransomware, insider threat):

1. Create backup vault in a separate **log archive** or **security** account.
2. Configure cross-account copy in the backup plan.
3. Apply vault access policies restricting who can delete recovery points.

```
{
 "Version": "2012-10-17",
 "Statement": [{
 "Effect": "Allow",
 "Principal": {"AWS": "arn:aws:iam::PROD_ACCOUNT:root"},
 "Action": "backup:CopyIntoBackupVault",
 "Resource": "*"
 }]
}
```

## 88.5 23.5 Service-specific DR patterns

### 88.5.1 RDS / Aurora

| Feature                     | RPO                             | Use case               |
|-----------------------------|---------------------------------|------------------------|
| Automated backups           | 5 minutes<br>(transaction logs) | Point-in-time recovery |
| Read replica (cross-region) | Seconds                         | DR with promotion      |

|                        |            |                                 |
|------------------------|------------|---------------------------------|
| Aurora Global Database | < 1 second | Multi-region with fast failover |
| Snapshot copy          | Hours      | Long-term retention             |

```
Create cross-region read replica
aws rds create-db-instance-read-replica \
 --db-instance-identifier prod-replica-dr \
 --source-db-instance-identifier arn:aws:rds:us-east-1:123456789012:db:prod \
 --region us-west-2
```

### 88.5.2 S3

| Feature                        | Purpose                                    |
|--------------------------------|--------------------------------------------|
| Cross-Region Replication (CRR) | Replicate objects to DR region             |
| S3 Versioning                  | Recover from accidental overwrites/deletes |
| MFA Delete                     | Prevent deletion without MFA               |
| Object Lock                    | WORM compliance; ransomware protection     |

### 88.5.3 DynamoDB

| Feature                  | RPO                                   |
|--------------------------|---------------------------------------|
| On-demand backup         | Point-in-time                         |
| Continuous backup (PITR) | 35 days, per-second recovery          |
| Global Tables            | Near-zero; multi-region active-active |

## 88.6 23.6 Failover automation

### 88.6.1 Route 53 failover

Health checks on primary region; automatic DNS failover to DR region ALB/CloudFront.

### 88.6.2 Database failover

```
Promote Aurora Global Database secondary
aws rds failover-global-cluster \
 --global-cluster-identifier my-global-cluster \
 --target-db-cluster-identifier dr-cluster
```

### 88.6.3 Application failover

- ASG/ECS desired count increase in DR region (via Lambda + CloudWatch alarm).
- Step Functions orchestrate multi-step failover runbooks.

## 88.7 23.7 Testing DR

**Untested backups are not backups.** Regular DR testing is required:

| Test type           | Frequency          | Scope                         |
|---------------------|--------------------|-------------------------------|
| Backup verification | Weekly (automated) | Restore random recovery point |
| Tabletop exercise   | Quarterly          | Walk through failover         |

|                         |               |                             |
|-------------------------|---------------|-----------------------------|
|                         |               | runbook                     |
| <b>Partial failover</b> | Semi-annually | Fail over one service to DR |
| <b>Full DR drill</b>    | Annually      | Complete regional failover  |

Document RTO/RPO achieved during each test and update runbooks.

## 88.8 23.8 Compliance and retention

| Requirement         | AWS feature                                   |
|---------------------|-----------------------------------------------|
| Immutable backups   | S3 Object Lock, Backup Vault Lock             |
| Long-term retention | Glacier via lifecycle; Backup cold storage    |
| Audit trail         | CloudTrail logs all backup API calls          |
| Encryption          | KMS keys per vault; separate keys per account |

### 88.8.1 Backup Vault Lock

Prevent deletion of recovery points before their retention period expires—even by root account.

```
aws backup put-backup-vault-lock-configuration \
 --backup-vault-name compliance-vault \
 --min-retention-days 90 \
 --changeable-for-days 3
```

## 88.9 23.9 Chapter summary

- **AWS Backup** centralizes backup policies across EC2, RDS, EFS, DynamoDB, and more.
- DR strategies range from **backup & restore** (cheapest) to **active/active** (fastest recovery).
- Use **cross-region** and **cross-account** copies for comprehensive protection.
- Define **RPO** and **RTO** targets and test DR plans regularly.
- Enable **Vault Lock** and **Object Lock** for ransomware-resistant backups.

## 88.10 Lab 23.1 — AWS Backup plan

1. Create a backup vault with KMS encryption.
2. Create a backup plan with daily backups and 14-day retention.
3. Tag an EC2 instance with `Backup=daily` and verify it is included.
4. Trigger an on-demand backup and restore to a new volume.

## 88.11 Lab 23.2 — Cross-region replication

1. Enable S3 versioning and CRR on a production bucket.
2. Upload objects and verify replication to the DR region.
3. Simulate primary region failure by blocking access and reading from DR.

## 88.12 Review questions

1. What is the difference between RPO and RTO?
2. Which DR strategy offers the lowest RTO and highest cost?
3. Why is cross-account backup important for ransomware protection?
4. How does Aurora Global Database achieve sub-second RPO?
5. Why should DR plans be tested regularly?

---

Next: [\*Chapter 24 — RDS & Aurora\*](#)

# 89 Chapter 24: RDS & Aurora

AWS Handbook — Part VI, Pages 481–505

## 89.1 24.1 Managed relational databases

**Amazon RDS** provides managed relational databases (MySQL, PostgreSQL, MariaDB, Oracle, SQL Server) with automated patching, backups, monitoring, and Multi-AZ failover. **Amazon Aurora** is AWS’s cloud-native database engine—MySQL and PostgreSQL compatible—with superior performance, storage auto-scaling, and global distribution.

For most new workloads, **Aurora PostgreSQL** or **Aurora MySQL** is the recommended choice over standard RDS.

## 89.2 24.2 RDS fundamentals

### 89.2.1 Supported engines

| Engine     | Versions                             | Licensing                |
|------------|--------------------------------------|--------------------------|
| PostgreSQL | 12-16                                | Open source              |
| MySQL      | 5.7, 8.0                             | Open source              |
| MariaDB    | 10.x                                 | Open source              |
| Oracle     | 19c, 21c                             | BYOL or License Included |
| SQL Server | 2019, 2022                           | License Included         |
| Aurora     | MySQL 5.7/8.0, PostgreSQL compatible | AWS proprietary          |

### 89.2.2 Instance classes

| Class      | Use case                         |
|------------|----------------------------------|
| db.t3/t4g  | Dev/test, burstable              |
| db.m6g/m7g | General purpose, Graviton        |
| db.r6g/r7g | Memory-optimized, large datasets |
| db.x2g     | Extreme memory (Aurora)          |

### 89.2.3 Storage types

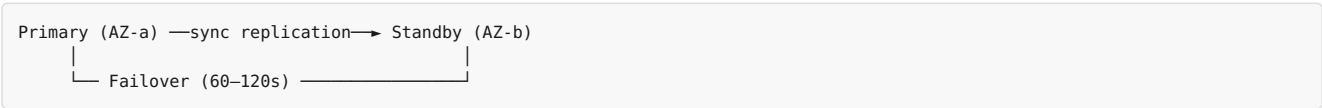
| Type     | IOPS          | Use case                  |
|----------|---------------|---------------------------|
| gp3      | 3,000–16,000  | General purpose (default) |
| io2      | Up to 256,000 | I/O-intensive workloads   |
| magnetic | Legacy        | Deprecated                |

## 89.3 24.3 High availability

### 89.3.1 Multi-AZ deployment



RDS synchronously replicates to a **standby instance** in another AZ:



- Automatic failover on primary failure.
- Same endpoint DNS (points to new primary after failover).
- Standby not readable (except Aurora).

89.3.2 Read replicas

Asynchronous replication for **read scaling** and DR:

| Feature      | Multi-AZ      | Read Replica           |
|--------------|---------------|------------------------|
| Purpose      | HA / failover | Read scaling, DR       |
| Replication  | Synchronous   | Asynchronous           |
| Endpoint     | Same          | Separate read endpoint |
| Cross-region | No            | Yes                    |
| Promotion    | Automatic     | Manual                 |

```
aws rds create-db-instance-read-replica \
 --db-instance-identifier mydb-replica-1 \
 --source-db-instance-identifier mydb \
 --db-instance-class db.t3.medium
```

89.4 24.4 Amazon Aurora

89.4.1 Aurora architecture

Unlike RDS (local storage per instance), Aurora uses a **distributed, shared storage layer** across 3 AZs:



89.4.2 Aurora advantages over RDS

| Feature           | Aurora                   | RDS                  |
|-------------------|--------------------------|----------------------|
| Storage scaling   | Automatic (10 GB-128 TB) | Manual (max 64 TB)   |
| Read replicas     | Up to 15, < 10ms lag     | Up to 5              |
| Failover          | ~30 seconds              | 60-120 seconds       |
| Storage copies    | 6 across 3 AZs           | 2 (Multi-AZ)         |
| Serverless v2     | Yes (auto-scaling ACUs)  | No                   |
| Global Database   | < 1 second cross-region  | Manual replica setup |
| Backtrack (MySQL) | Rewind to point in time  | No                   |

### 89.4.3 Aurora Serverless v2

Automatically scales compute capacity (ACUs) based on demand:

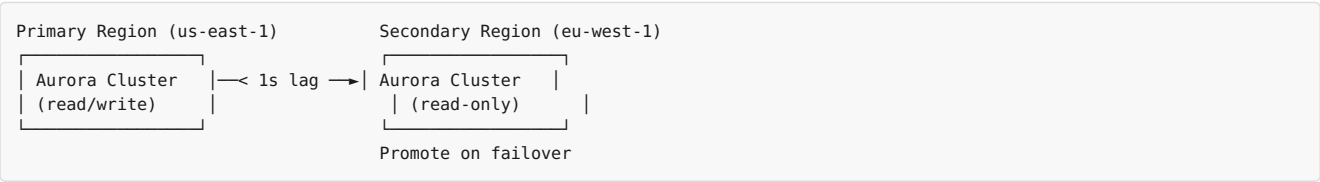
```
resource "aws_rds_cluster" "aurora" {
 engine = "aurora-postgresql"
 engine_mode = "provisioned"
 engine_version = "15.4"
 database_name = "appdb"
 master_username = "admin"
 master_password = var.db_password

 serverlessv2_scaling_configuration {
 min_capacity = 0.5 # ACUs
 max_capacity = 16
 }
}

resource "aws_rds_cluster_instance" "aurora" {
 cluster_identifier = aws_rds_cluster.aurora.id
 instance_class = "db.serverless"
 engine = aws_rds_cluster.aurora.engine
}
```

### 89.4.4 Aurora Global Database

Cross-region replication with < 1 second lag:



## 89.5 24.5 Backups and recovery

| Feature                | RDS                  | Aurora                 |
|------------------------|----------------------|------------------------|
| Automated backups      | 1-35 days retention  | 1-35 days retention    |
| Manual snapshots       | Yes                  | Yes (cluster snapshot) |
| Point-in-time recovery | 5-minute granularity | 5-minute granularity   |
| Backtrack (MySQL)      | No                   | Rewind without restore |

```
Restore to point in time
aws rds restore-db-instance-to-point-in-time \
 --source-db-instance-identifier mydb \
 --target-db-instance-identifier mydb-restored \
 --restore-time 2024-06-15T10:30:00Z
```

## 89.6 24.6 Security

| Control               | Implementation                                      |
|-----------------------|-----------------------------------------------------|
| Network               | Deploy in private subnets; security groups          |
| Encryption at rest    | KMS (enabled by default on new instances)           |
| Encryption in transit | SSL/TLS connections (force via parameter group)     |
| IAM authentication    | Token-based auth (no password in connection string) |

|                        |                                                        |
|------------------------|--------------------------------------------------------|
|                        |                                                        |
| <b>Secrets Manager</b> | Auto-rotate database credentials                       |
| <b>Audit</b>           | PostgreSQL log exports, MySQL audit plugin, CloudTrail |

### 89.6.1 IAM database authentication

```
import boto3

rds = boto3.client("rds")
token = rds.generate_db_auth_token(
 DBHostname="mydb.abc123.us-east-1.rds.amazonaws.com",
 Port=5432,
 DBUsername="iam_user",
 Region="us-east-1",
)
Use token as password in connection string (valid 15 minutes)
```

## 89.7 24.7 Parameter and option groups

| Group                  | Purpose                                                |
|------------------------|--------------------------------------------------------|
| <b>Parameter group</b> | Engine configuration (max_connections, shared_buffers) |
| <b>Option group</b>    | Optional features (Oracle OEM, SQL Server SSIS)        |

```
aws rds modify-db-parameter-group \
 --db-parameter-group-name custom-pg \
 --parameters "ParameterName=max_connections,ParameterValue=200,ApplyMethod=immediate"
```

## 89.8 24.8 Monitoring

| Tool                        | Metrics                                           |
|-----------------------------|---------------------------------------------------|
| <b>CloudWatch</b>           | CPU, connections, IOPS, replica lag, free storage |
| <b>Performance Insights</b> | Top SQL, wait events, database load               |
| <b>Enhanced Monitoring</b>  | OS-level metrics (1-second granularity)           |
| <b>Event subscriptions</b>  | SNS notifications for failover, maintenance       |

Key alarms: - CPUUtilization > 80% - FreeableMemory < 256 MB - DatabaseConnections approaching max\_connections - ReplicaLag > 30 seconds

## 89.9 24.9 Terraform example

```
resource "aws_db_subnet_group" "main" {
 name = "main"
 subnet_ids = aws_subnet.private[*].id
}

resource "aws_rds_cluster" "aurora" {
 cluster_identifier = "app-aurora"
 engine = "aurora-postgresql"
 engine_version = "15.4"
 database_name = "appdb"
 master_username = "admin"
 manage_master_user_password = true # Secrets Manager
```

```
db_subnet_group_name = aws_db_subnet_group.main.name
vpc_security_group_ids = [aws_security_group.rds.id]

backup_retention_period = 14
preferred_backup_window = "03:00-04:00"
storage_encrypted = true
kms_key_id = aws_kms_key.rds.arn

enabled_cloudwatch_logs_exports = ["postgresql"]

tags = { Name = "app-aurora" }
}

resource "aws_rds_cluster_instance" "aurora" {
 count = 2
 identifier = "app-aurora-${count.index}"
 cluster_identifier = aws_rds_cluster.aurora.id
 instance_class = "db.r6g.large"
 engine = aws_rds_cluster.aurora.engine
 publicly_accessible = false
}
```

---

## 89.10 24.10 Chapter summary

- **RDS** provides managed relational databases; **Aurora** offers superior performance and scaling.
- **Multi-AZ** for HA; **read replicas** for read scaling and cross-region DR.
- **Aurora Serverless v2** auto-scales compute; **Global Database** for multi-region.
- Encrypt at rest and in transit; use **IAM authentication** and **Secrets Manager**.
- Monitor with **Performance Insights** and CloudWatch alarms.

---

## 89.11 Lab 24.1 — Aurora cluster

1. Create an Aurora PostgreSQL cluster with one writer and one reader.
2. Connect via psql from a private EC2 instance.
3. Insert test data and query from the reader endpoint.
4. Simulate failover and verify application reconnects.

## 89.12 Lab 24.2 — Point-in-time recovery

1. Enable automated backups (7-day retention).
2. Create and delete a test table.
3. Restore to a point before the deletion.
4. Verify the table exists in the restored instance.

---

## 89.13 Review questions

1. What is the difference between Multi-AZ and read replicas?
2. How does Aurora storage differ from RDS storage?
3. What is an ACU in Aurora Serverless v2?
4. How does Aurora Global Database achieve sub-second cross-region replication?
5. Why should databases be deployed in private subnets?

---

Next: [Chapter 25 — DynamoDB](#)

# 90 Chapter 25: DynamoDB – Tables, Indexes, and Streams

AWS Handbook — Part VI, Pages 506–525

## 90.1 25.1 What is DynamoDB?

**Amazon DynamoDB** is a fully managed, serverless NoSQL database providing single-digit millisecond performance at any scale. Unlike RDS, there are no servers to provision, patch, or scale—DynamoDB handles capacity automatically (on-demand mode) or lets you provision read/write capacity units.

DynamoDB excels at key-value and document data models with predictable access patterns.

## 90.2 25.2 Core concepts

| Concept       | Description                                                   |
|---------------|---------------------------------------------------------------|
| Table         | Collection of items (like a table without schema enforcement) |
| Item          | A record (up to 400 KB)                                       |
| Attribute     | A field within an item (name-value pair)                      |
| Primary key   | Uniquely identifies each item                                 |
| Partition key | Hash key (required)                                           |
| Sort key      | Range key (optional; enables queries on range)                |

### 90.2.1 Primary key types

| Type      | Structure                | Example            |
|-----------|--------------------------|--------------------|
| Simple    | Partition key only       | UserId             |
| Composite | Partition key + sort key | UserId + OrderDate |

## 90.3 25.3 Capacity modes

| Mode                       | Billing                               | Use case                             |
|----------------------------|---------------------------------------|--------------------------------------|
| On-demand                  | Per request                           | Unpredictable traffic, new apps      |
| Provisioned                | Per RCU/WCU per hour                  | Predictable, cost-optimized at scale |
| Provisioned + Auto Scaling | Provisioned with automatic adjustment | Variable but bounded traffic         |

### 90.3.1 Capacity units

| Unit                      | Definition                                   |
|---------------------------|----------------------------------------------|
| RCU (Read Capacity Unit)  | 1 strongly consistent read/sec for 4 KB item |
| WCU (Write Capacity Unit) | 1 write/sec for 1 KB item                    |

**Eventually consistent reads** cost half the RCUs of strongly consistent reads.

## 90.4 25.4 Indexes

### 90.4.1 Global Secondary Index (GSI)

- Different partition key and sort key from the base table.
- Eventually consistent (by default).
- Separate provisioned or on-demand capacity.
- Up to 20 GSIs per table.

```
Base table: PK=UserId, SK=OrderDate
GSI: PK=ProductId, SK=OrderDate → Query orders by product
```

### 90.4.2 Local Secondary Index (LSI)

- Same partition key, different sort key.
- Must be created at table creation time.
- Shares table capacity.
- Up to 5 LSIs per table.

### 90.4.3 When to use which

| Need                                         | Index type |
|----------------------------------------------|------------|
| Query by different partition key             | GSI        |
| Query by different sort key (same partition) | LSI        |
| Add index after table creation               | GSI only   |
| Strongly consistent reads on index           | LSI only   |

## 90.5 25.5 Query and Scan

### 90.5.1 Query (efficient)

Access items by partition key (and optionally sort key condition):

```
import boto3

dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table("Orders")

response = table.query(
 KeyConditionExpression="UserId = :uid AND OrderDate > :date",
 ExpressionAttributeValues={
 ":uid": "user-123",
 ":date": "2024-01-01",
 },
)
```

### 90.5.2 Scan (expensive)

Reads every item in the table. Avoid in production; use GSIs instead.

### 90.5.3 Best practices

- Design access patterns first, then schema.

- Use composite keys to model one-to-many relationships.
- Prefer Query over Scan.
- Use `ProjectionExpression` to return only needed attributes.
- Paginate with `LastEvaluatedKey`.

## 90.6 25.6 DynamoDB Streams

**Streams** capture item-level changes (insert, update, delete) in near real-time:

| View type          | Data captured               |
|--------------------|-----------------------------|
| KEYS_ONLY          | Partition and sort key only |
| NEW_IMAGE          | Entire item after change    |
| OLD_IMAGE          | Entire item before change   |
| NEW_AND_OLD_IMAGES | Both (most common)          |

### 90.6.1 Stream consumers

| Consumer                   | Use case                     |
|----------------------------|------------------------------|
| Lambda                     | Event-driven processing, CDC |
| Kinesis Data Streams       | Extended retention, fan-out  |
| DynamoDB (Kinesis adapter) | Legacy integration           |

### 90.6.2 Lambda trigger example

```
resource "aws_lambda_event_source_mapping" "dynamo" {
 event_source_arn = aws_dynamodb_table.orders.stream_arn
 function_name = aws_lambda_function.processor.arn
 starting_position = "LATEST"
 batch_size = 100

 filter_criteria {
 filter {
 pattern = jsonencode({
 eventName = ["INSERT", "MODIFY"]
 })
 }
 }
}
```

## 90.7 25.7 Advanced features

### 90.7.1 DynamoDB Accelerator (DAX)

In-memory cache for microsecond read latency. Transparent to application (drop-in replacement for DynamoDB endpoint).

### 90.7.2 Global Tables

Multi-region, multi-active replication:

```
us-east-1 table ←sync→ eu-west-1 table
 | |
 Application A Application B
 (read/write) (read/write)
```

- Conflict resolution: last writer wins.
- RPO: sub-second; RTO: near-zero (DNS failover).

### 90.7.3 Transactions

`TransactWriteItems` and `TransactGetItems` for ACID operations across up to 100 items (same or different tables).

### 90.7.4 TTL (Time to Live)

Automatically delete items after a specified timestamp attribute. No WCU consumed for TTL deletions.

```
aws dynamodb update-time-to-live \
 --table-name Sessions \
 --time-to-live-specification "Enabled=true,AttributeName=ExpiresAt"
```

### 90.7.5 Point-in-time recovery (PITR)

Continuous backup with per-second restore capability (35-day window).

## 90.8 25.8 Single-table design

DynamoDB best practice: model multiple entity types in one table using composite keys:

| PK          | SK             | Attributes    |
|-------------|----------------|---------------|
| USER#123    | PROFILE        | name, email   |
| USER#123    | ORDER#2024-001 | total, status |
| USER#123    | ORDER#2024-002 | total, status |
| PRODUCT#ABC | METADATA       | name, price   |
| PRODUCT#ABC | REVIEW#001     | rating, text  |

GSI for alternate access patterns:

```
GSI1PK=PRODUCT#ABC GSI1SK=REVIEW#001 → All reviews for a product
```

## 90.9 25.9 Terraform example

```
resource "aws_dynamodb_table" "orders" {
 name = "Orders"
 billing_mode = "PAY_PER_REQUEST"
 hash_key = "UserId"
 range_key = "OrderDate"

 attribute {
 name = "UserId"
 type = "S"
 }
 attribute {
 name = "OrderDate"
 type = "S"
 }
 attribute {
 name = "ProductId"
 type = "S"
 }

 global_secondary_index {
 name = "ProductIndex"
 hash_key = "ProductId"
 range_key = "OrderDate"
 projection_type = "ALL"
 }

 stream_enabled = true
}
```



```

stream_view_type = "NEW_AND_OLD_IMAGES"

point_in_time_recovery { enabled = true }
server_side_encryption { enabled = true }

ttl {
 attribute_name = "ExpiresAt"
 enabled = true
}

tags = { Name = "orders-table" }
}

```

## 90.10 25.10 Monitoring

| Metric                     | Alarm threshold idea            |
|----------------------------|---------------------------------|
| ConsumedReadCapacityUnits  | Approaching provisioned limit   |
| ConsumedWriteCapacityUnits | Approaching provisioned limit   |
| ThrottledRequests          | Any throttling → scale capacity |
| SuccessfulRequestLatency   | p99 > 10ms                      |
| UserErrors                 | 4xx errors from client          |

Enable **Contributor Insights** for per-item access pattern analysis.

## 90.11 25.11 Chapter summary

- **DynamoDB** is serverless NoSQL with single-digit ms latency at any scale.
- Design **access patterns first**; use composite keys and **GSIs** for query flexibility.
- **Streams** enable event-driven architectures and change data capture.
- **Global Tables** provide multi-region active-active replication.
- Prefer **on-demand** for unpredictable workloads; **single-table design** for complex models.

## 90.12 Lab 25.1 — Orders table

1. Create a DynamoDB table with composite key (UserId + OrderDate).
2. Add a GSI on ProductId.
3. Insert 20 orders across 5 users and 3 products.
4. Query orders for one user; query orders for one product via GSI.

## 90.13 Lab 25.2 — Streams + Lambda

1. Enable DynamoDB Streams on your table.
2. Create a Lambda function triggered by stream events.
3. Insert/update items and verify Lambda processes changes.
4. Add a filter for INSERT events only.

## 90.14 Review questions

1. What is the difference between a GSI and an LSI?

2. Why should you avoid Scan operations in production?
  3. How do DynamoDB Streams enable change data capture?
  4. What is single-table design and why is it recommended?
  5. When would you choose provisioned capacity over on-demand?
- 

Next: [\*Chapter 26 — ElastiCache\*](#)

# 91 Chapter 26: ElastiCache & Caching Patterns

AWS Handbook — Part VI, Pages 526–540

## 91.1 26.1 Why caching?

Every database query that hits disk (or even memory-optimized databases) adds latency and load. **Caching** stores frequently accessed data in a fast, in-memory layer, reducing database pressure and improving response times from milliseconds to microseconds.

**Amazon ElastiCache** provides fully managed in-memory caching with two engines: **Redis** and **Memcached**.

## 91.2 26.2 Redis vs Memcached

| Feature         | Redis                                                       | Memcached                      |
|-----------------|-------------------------------------------------------------|--------------------------------|
| Data structures | Strings, lists, sets, sorted sets, hashes, streams, bitmaps | Strings only (key-value)       |
| Persistence     | RDB snapshots, AOF                                          | None (pure cache)              |
| Replication     | Primary + up to 5 replicas                                  | None (client-side sharding)    |
| Multi-AZ        | Automatic failover                                          | No                             |
| Clustering      | Cluster mode (sharding)                                     | Client-side consistent hashing |
| Pub/Sub         | Yes                                                         | No                             |
| Lua scripting   | Yes                                                         | No                             |
| Backup/restore  | Yes                                                         | No                             |
| Use case        | Sessions, leaderboards, real-time analytics, queues         | Simple object caching          |

**Recommendation:** Choose **Redis** unless you need only simple key-value caching with horizontal scaling via client-side sharding.

## 91.3 26.3 ElastiCache deployment options

| Option                | Description                           |
|-----------------------|---------------------------------------|
| Single node           | Dev/test only; no HA                  |
| Multi-AZ with replica | Primary + replica; automatic failover |
| Cluster mode disabled | One shard; up to 5 read replicas      |
| Cluster mode enabled  | Multiple shards; horizontal scaling   |
| Serverless            | Auto-scaling memory; pay per use      |

### 91.3.1 Node types

| Family | Use case |
|--------|----------|
|--------|----------|

|                      |                                  |
|----------------------|----------------------------------|
| <b>cache.t3/t4g</b>  | Dev/test, burstable              |
| <b>cache.m6g/m7g</b> | General purpose                  |
| <b>cache.r6g/r7g</b> | Memory-optimized, large datasets |

## 91.4 26.4 Common caching patterns

### 91.4.1 Cache-aside (lazy loading)

Application checks cache first; on miss, reads from database and populates cache:

1. App → Cache: GET key
2. Cache miss → App → DB: SELECT
3. App → Cache: SET key (with TTL)
4. App returns data

```
import redis
import json

cache = redis.Redis(host="my-cluster.abc123.cache.amazonaws.com", port=6379)

def get_user(user_id):
 cached = cache.get(f"user:{user_id}")
 if cached:
 return json.loads(cached)

 user = db.query("SELECT * FROM users WHERE id = %s", user_id)
 cache.setex(f"user:{user_id}", 3600, json.dumps(user)) # TTL 1 hour
 return user
```

### 91.4.2 Write-through

Application writes to cache and database simultaneously:

```
App → Cache: SET key
App → DB: INSERT/UPDATE
```

Ensures cache is always up to date but adds write latency.

### 91.4.3 Write-behind (write-back)

Application writes to cache; cache asynchronously writes to database:

```
App → Cache: SET key
Cache → DB: async write (batched)
```

Highest write performance but risk of data loss if cache fails before DB write.

### 91.4.4 Read-through

Cache itself loads data from database on miss (requires cache library support):

```
App → Cache: GET key
Cache miss → Cache → DB: load and return
```

## 91.5 26.5 Redis data structure use cases

| Structure | Use case | Example |
|-----------|----------|---------|
|           |          |         |

|                    |                          |                                |
|--------------------|--------------------------|--------------------------------|
| <b>String</b>      | Simple cache, counters   | Session token, page view count |
| <b>Hash</b>        | Object storage           | User profile fields            |
| <b>List</b>        | Queues, timelines        | Recent activity feed           |
| <b>Set</b>         | Unique items             | Online users, tags             |
| <b>Sorted Set</b>  | Leaderboards, rankings   | Game scores, trending          |
| <b>Stream</b>      | Event log                | Activity stream, messaging     |
| <b>Bitmap</b>      | Feature flags, analytics | Daily active users             |
| <b>HyperLogLog</b> | Cardinality estimation   | Unique visitor count           |

### 91.5.1 Leaderboard example

```
Add score
cache.zadd("leaderboard:2024", {"player-1": 1500, "player-2": 2300})

Top 10
top_10 = cache.zrevrange("leaderboard:2024", 0, 9, withscores=True)

Player rank
rank = cache.zrevrank("leaderboard:2024", "player-1")
```

## 91.6 26.6 Session management

Redis is the standard for distributed session storage:

```
import uuid

def create_session(user_id):
 session_id = str(uuid.uuid4())
 cache.setex(f"session:{session_id}", 86400, json.dumps({
 "user_id": user_id,
 "created": time.time(),
 }))
 return session_id

def get_session(session_id):
 data = cache.get(f"session:{session_id}")
 return json.loads(data) if data else None
```

Benefits over sticky sessions: - Any app server can handle any request. - Sessions survive server restarts. - Easy session invalidation (delete key).

## 91.7 26.7 ElastiCache security

| Control                      | Implementation                                       |
|------------------------------|------------------------------------------------------|
| <b>Network</b>               | Deploy in private subnets; security groups           |
| <b>Encryption in transit</b> | TLS (required for compliance)                        |
| <b>Encryption at rest</b>    | KMS                                                  |
| <b>AUTH token</b>            | Redis password (required with encryption in transit) |
| <b>RBAC</b>                  | Redis ACLs for fine-grained access (Redis 6+)        |

```
resource "aws_elasticache_replication_group" "redis" {
 replication_group_id = "app-cache"
```

```

description = "Application Redis cluster"
node_type = "cache.r6g.large"
num_cache_clusters = 2
automatic_failover_enabled = true
multi_az_enabled = true

engine = "redis"
engine_version = "7.0"
port = 6379
parameter_group_name = "default.redis7"

subnet_group_name = aws_elasticache_subnet_group.main.name
security_group_ids = [aws_security_group.redis.id]

at_rest_encryption_enabled = true
transit_encryption_enabled = true
auth_token = var.redis_auth_token

snapshot_retention_limit = 7
snapshot_window = "03:00-05:00"
}

```

## 91.8 26.8 Monitoring and maintenance

| Metric                        | Action                                     |
|-------------------------------|--------------------------------------------|
| CPUUtilization                | Scale up node type if > 75%                |
| DatabaseMemoryUsagePercentage | Scale up or evict; risk of OOM             |
| CacheHitRate                  | Low hit rate → review TTLs and key design  |
| Evictions                     | Memory pressure; scale up or reduce TTLs   |
| ReplicationLag                | Replica falling behind; check network/load |

### 91.8.1 Maintenance windows

ElastiCache applies engine patches during maintenance windows. Enable **auto minor version upgrade** for security patches.

## 91.9 26.9 Caching anti-patterns

| Anti-pattern                   | Problem                                     | Solution                                  |
|--------------------------------|---------------------------------------------|-------------------------------------------|
| Cache everything               | Memory waste, stale data                    | Cache only hot data with TTLs             |
| No TTL                         | Stale data forever                          | Always set expiration                     |
| Cache stampede                 | Many requests hit DB on simultaneous expiry | Jittered TTLs, request coalescing         |
| Large objects                  | Memory pressure, slow serialization         | Compress or split large values            |
| Caching errors                 | Error responses cached                      | Don't cache 4xx/5xx responses             |
| No cache invalidation strategy | Stale data after updates                    | Event-driven invalidation via streams/SNS |

## 91.10 26.10 Chapter summary

- **ElastiCache** provides managed Redis and Memcached for in-memory caching.
  - **Redis** supports rich data structures, persistence, replication, and pub/sub.
  - **Cache-aside** is the most common pattern; always set **TTLs**.
  - Use Redis for sessions, leaderboards, and real-time features.
  - Deploy in **private subnets** with encryption in transit and at rest.
- 

## 91.11 Lab 26.1 — Redis cache-aside

1. Create an ElastiCache Redis cluster (single node for lab).
2. Write a Python app that implements cache-aside for a DynamoDB table.
3. Measure response time with and without cache.
4. Verify cache hit rate in CloudWatch.

## 91.12 Lab 26.2 — Leaderboard

1. Use Redis sorted sets to build a game leaderboard.
  2. Add scores for 20 players.
  3. Query top 10 and individual player rank.
  4. Implement a “scores in last hour” feature using TTL keys.
- 

## 91.13 Review questions

1. When would you choose Memcached over Redis?
  2. How does the cache-aside pattern handle cache misses?
  3. What is cache stampede and how do you prevent it?
  4. Why is Redis preferred for session management over sticky sessions?
  5. What CloudWatch metric indicates your cache is too small?
- 

Next: [Chapter 27 — DMS & DR](#)

# 92 Chapter 27: Database Migration (DMS) & DR

AWS Handbook — Part VI, Pages 541–560

## 92.1 27.1 Database migration challenges

Migrating databases to AWS—whether from on-premises, another cloud, or between AWS engines—is one of the highest-risk activities in cloud adoption. **AWS Database Migration Service (DMS)** simplifies heterogeneous and homogeneous migrations with minimal downtime.

Combined with **Schema Conversion Tool (SCT)** for schema translation and **native replication features** for ongoing DR, AWS provides a complete migration toolkit.

## 92.2 27.2 AWS DMS overview

**DMS** replicates data between source and target databases continuously:

| Migration type  | Description                                 |
|-----------------|---------------------------------------------|
| Full load       | One-time copy of all data                   |
| Full load + CDC | Initial copy + ongoing change data capture  |
| CDC only        | Ongoing replication (source already loaded) |

### 92.2.1 Supported sources and targets

| Source                                                     | Target                                       |
|------------------------------------------------------------|----------------------------------------------|
| On-premises Oracle, SQL Server, MySQL, PostgreSQL, MongoDB | RDS, Aurora, Redshift, S3, DynamoDB, Kinesis |
| EC2-hosted databases                                       | Same as above                                |
| RDS / Aurora                                               | Cross-engine (e.g., Oracle → PostgreSQL)     |
| S3                                                         | RDS, Redshift                                |

### 92.2.2 DMS components

| Component            | Description                                   |
|----------------------|-----------------------------------------------|
| Replication instance | EC2 instance running DMS engine               |
| Source endpoint      | Connection to source database                 |
| Target endpoint      | Connection to target database                 |
| Replication task     | Defines what to migrate and how               |
| Table mappings       | Rules for schema selection and transformation |

## 92.3 27.3 Migration workflow

1. Assess → SCT analyzes source schema; identifies incompatibilities
  2. Convert → SCT converts schema to target engine (if heterogeneous)



3. Provision → Create target RDS/Aurora instance
4. Replicate → DMS full load + CDC
5. Cutover → Switch application to target; stop DMS task
6. Validate → Data validation and application testing

### 92.3.1 Heterogeneous vs homogeneous

| Type          | Example                     | SCT needed? |
|---------------|-----------------------------|-------------|
| Homogeneous   | MySQL → Aurora MySQL        | No          |
| Heterogeneous | Oracle → Aurora PostgreSQL  | Yes         |
| Homogeneous   | SQL Server → RDS SQL Server | No          |
| Heterogeneous | MongoDB → DynamoDB          | Partial     |

## 92.4 27.4 DMS replication instance

| Size           | vCPU | Memory | Use case                          |
|----------------|------|--------|-----------------------------------|
| dms.t3.medium  | 2    | 4 GB   | Small databases (< 100 GB)        |
| dms.r5.large   | 2    | 16 GB  | Medium workloads                  |
| dms.r5.4xlarge | 16   | 128 GB | Large, high-throughput migrations |
| dms.c5.9xlarge | 36   | 72 GB  | Maximum throughput                |

**Multi-AZ** replication instances provide automatic failover for ongoing CDC tasks.

### 92.4.1 Terraform example

```

resource "aws_dms_replication_subnet_group" "main" {
 replication_subnet_group_id = "dms-subnet-group"
 replication_subnet_group_description = "DMS subnet group"
 subnet_ids = aws_subnet.private[*].id
}

resource "aws_dms_replication_instance" "main" {
 replication_instance_id = "dms-replication"
 replication_instance_class = "dms.r5.large"
 allocated_storage = 100
 multi_az = true
 replication_subnet_group_id = aws_dms_replication_subnet_group.main.id
 vpc_security_group_ids = [aws_security_group.dms.id]
}

resource "aws_dms_endpoint" "source" {
 endpoint_id = "source-oracle"
 endpoint_type = "source"
 engine_name = "oracle"
 server_name = "on-prem-oracle.example.com"
 port = 1521
 database_name = "ORCL"
 username = "dms_user"
 password = var.source_password
}

resource "aws_dms_endpoint" "target" {
 endpoint_id = "target-aurora-pg"
 endpoint_type = "target"
 engine_name = "aurora-postgresql"
 server_name = aws_rds_cluster.aurora.endpoint
 port = 5432
 database_name = "appdb"
 username = "admin"
 password = var.target_password
}

```

```

resource "aws_dms_replication_task" "migration" {
 replication_task_id = "oracle-to-aurora-pg"
 migration_type = "full-load-and-cdc"
 replication_instance_arn = aws_dms_replication_instance.main.replication_instance_arn
 source_endpoint_arn = aws_dms_endpoint.source.endpoint_arn
 target_endpoint_arn = aws_dms_endpoint.target.endpoint_arn

 table_mappings = file("${path.module}/table-mappings.json")

 replication_task_settings = jsonencode({
 TargetMetadata = { TargetSchema = "public" }
 FullLoadSettings = { TargetTablePrepMode = "DROP_AND_CREATE" }
 ChangeProcessingDdlHandlingPolicy = {
 HandleSourceTableDropped = true
 HandleSourceTableTruncated = true
 }
 })
}

```

## 92.5 27.5 Schema Conversion Tool (SCT)

**SCT** automates schema conversion for heterogeneous migrations:

1. Connect to source database.
2. Analyze schema objects (tables, views, stored procedures, triggers).
3. Convert to target engine syntax.
4. Report incompatible objects for manual review.
5. Apply converted schema to target.

### 92.5.1 Common conversion challenges

| Source (Oracle)   | Target (PostgreSQL) | SCT action           |
|-------------------|---------------------|----------------------|
| VARCHAR2          | VARCHAR             | Auto-convert         |
| NUMBER            | NUMERIC             | Auto-convert         |
| PL/SQL procedures | PL/pgSQL            | Partial auto-convert |
| Packages          | Schemas + functions | Manual review        |
| Sequences         | SERIAL / sequences  | Auto-convert         |

## 92.6 27.6 Migration strategies

### 92.6.1 Big bang

Migrate everything in one maintenance window:

- **Pros:** Simple, clean cutover.
- **Cons:** Extended downtime; high risk.
- **When:** Small databases, tolerant downtime windows.

### 92.6.2 Trickle (parallel run)

DMS replicates continuously; application reads from both during transition:

```

Phase 1: DMS full load + CDC running
Phase 2: Application writes to source; reads from source
Phase 3: Application writes to source; reads from target (validate)
Phase 4: Application writes to target; reads from target
Phase 5: Decommission source

```

- **Pros:** Minimal downtime (minutes for cutover).
- **Cons:** Complex application changes; dual-write period.

### 92.6.3 Zero-ETL integrations

AWS-native integrations that eliminate DMS for supported paths:

| Integration                 | Source → Target             |
|-----------------------------|-----------------------------|
| Aurora zero-ETL to Redshift | Aurora → Redshift analytics |
| S3 zero-ETL to Redshift     | S3 → Redshift               |
| OpenSearch zero-ETL         | Aurora/DMS → OpenSearch     |

## 92.7 27.7 Data validation

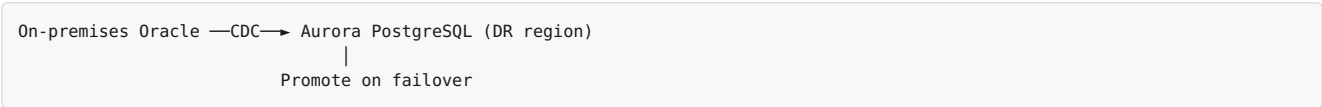
DMS provides built-in data validation:

```
{
 "ValidationSettings": {
 "EnableValidation": true,
 "ValidationMode": "ROW_LEVEL",
 "ThreadCount": 5,
 "FailureMaxCount": 10000
 }
}
```

Additionally: - Row count comparison between source and target. - Checksum validation for critical tables. - Application-level integration testing.

## 92.8 27.8 Ongoing replication for DR

DMS is not just for migration—it enables ongoing replication for DR:



| Feature              | Benefit                        |
|----------------------|--------------------------------|
| CDC                  | Near-real-time replication     |
| Multi-AZ instance    | DMS failover protection        |
| Table filtering      | Replicate only critical tables |
| Transformation rules | Mask PII in DR copy            |

For AWS-native DR, prefer **Aurora Global Database** or **RDS cross-region read replicas** over DMS when source is already on AWS.

## 92.9 27.9 Troubleshooting

| Issue          | Cause                           | Fix                     |
|----------------|---------------------------------|-------------------------|
| Full load slow | Undersized replication instance | Scale up instance class |
|                |                                 |                         |

|                      |                                    |                                                   |
|----------------------|------------------------------------|---------------------------------------------------|
| CDC lag increasing   | Network or target write bottleneck | Scale instance; optimize target                   |
| LOB columns failing  | LOB settings too small             | Increase <code>LobMaxSize</code> in task settings |
| FK constraint errors | Load order wrong                   | Disable FK checks during load                     |
| Connection timeout   | Security group or network ACL      | Verify SG rules and route tables                  |

```
aws dms describe-replication-tasks \
 --filters Name=replication-task-id,Values=oracle-to-aurora-pg

aws dms describe-table-statistics \
 --replication-task-arn arn:aws:dms:us-east-1:123456789012:task:abc123
```

## 92.10 27.10 Chapter summary

- **DMS** migrates databases with full load and ongoing CDC for minimal downtime.
- **SCT** converts schemas for heterogeneous migrations (Oracle → PostgreSQL, etc.).
- Use **trickle migration** for near-zero-downtime cutovers.
- DMS also supports **ongoing replication** for DR scenarios.
- Validate data thoroughly before and after cutover.

## 92.11 Lab 27.1 — Homogeneous migration

1. Create a source MySQL RDS instance with sample data.
2. Create a target Aurora MySQL cluster.
3. Configure DMS endpoints and a full-load + CDC replication task.
4. Verify data replication; insert new rows on source and confirm CDC delivery.

## 92.12 Lab 27.2 — Migration assessment

1. Install SCT on an EC2 instance.
2. Connect to a sample database and run an assessment report.
3. Review conversion warnings and manual action items.
4. Document a migration plan with estimated downtime.

## 92.13 Review questions

1. What is the difference between full load and CDC in DMS?
2. When is SCT required for a database migration?
3. What is trickle migration and how does it minimize downtime?
4. How does DMS support ongoing DR replication?
5. What factors determine the size of a DMS replication instance?

Next: [Chapter 28 — API Gateway](#)

# 93 Chapter 28: API Gateway

AWS Handbook — Part VII, Pages 561–580

## 93.1 28.1 What is Amazon API Gateway?

**Amazon API Gateway** is a fully managed service for creating, publishing, securing, and monitoring HTTP, REST, and WebSocket APIs at any scale. It acts as the front door for serverless backends (Lambda), container services (ECS/EKS), and legacy applications.

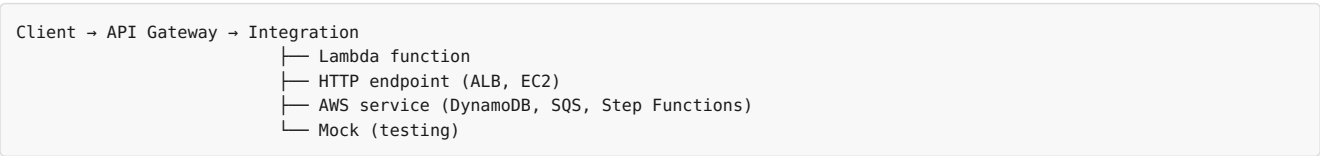
API Gateway handles request routing, authentication, throttling, caching, and API versioning—freeing backend services from these cross-cutting concerns.

## 93.2 28.2 API types

| Type             | Protocol  | Use case                                                   |
|------------------|-----------|------------------------------------------------------------|
| REST API         | HTTP/REST | Full-featured; request/response transformation             |
| HTTP API         | HTTP      | Simpler, cheaper, lower latency; Lambda/proxy integrations |
| WebSocket API    | WebSocket | Real-time bidirectional communication                      |
| Private REST API | HTTP/REST | Internal APIs via VPC endpoint                             |

**Recommendation:** Use **HTTP API** for new Lambda-based APIs (70% cheaper, lower latency). Use **REST API** when you need request validation, API keys, or advanced transformation.

## 93.3 28.3 REST API architecture



### 93.3.1 Key components

| Component   | Description                                                         |
|-------------|---------------------------------------------------------------------|
| Resource    | URL path segment ( <code>/users</code> , <code>/users/{id}</code> ) |
| Method      | HTTP verb (GET, POST, PUT, DELETE)                                  |
| Integration | Backend that processes the request                                  |
| Stage       | Deployment environment (dev, staging, prod)                         |
| Deployment  | Snapshot of API configuration                                       |

## 93.4 28.4 HTTP API with Lambda

### 93.4.1 Terraform example

```
resource "aws_apigatewayv2_api" "api" {
 name = "app-api"
 protocol_type = "HTTP"

 cors_configuration {
 allow_origins = ["https://app.example.com"]
 allow_methods = ["GET", "POST", "PUT", "DELETE"]
 allow_headers = ["Content-Type", "Authorization"]
 max_age = 3600
 }
}

resource "aws_apigatewayv2_integration" "lambda" {
 api_id = aws_apigatewayv2_api.api.id
 integration_type = "AWS_PROXY"
 integration_uri = aws_lambda_function.api.invoke_arn
 payload_format_version = "2.0"
}

resource "aws_apigatewayv2_route" "get_users" {
 api_id = aws_apigatewayv2_api.api.id
 route_key = "GET /users"
 target = "integrations/${aws_apigatewayv2_integration.lambda.id}"
}

resource "aws_apigatewayv2_route" "post_users" {
 api_id = aws_apigatewayv2_api.api.id
 route_key = "POST /users"
 target = "integrations/${aws_apigatewayv2_integration.lambda.id}"
}

resource "aws_apigatewayv2_stage" "prod" {
 api_id = aws_apigatewayv2_api.api.id
 name = "prod"
 auto_deploy = true

 access_log_settings {
 destination_arn = aws_cloudwatch_log_group.api.arn
 format = jsonencode({
 requestId = "$context.requestId"
 ip = "$context.identity.sourceIp"
 method = "$context.httpMethod"
 path = "$context.path"
 status = "$context.status"
 responseTime = "$context.responseLatency"
 })
 }

 default_route_settings {
 throttling_burst_limit = 100
 throttling_rate_limit = 50
 }
}

resource "aws_lambda_permission" "api" {
 statement_id = "AllowAPIGateway"
 action = "lambda:InvokeFunction"
 function_name = aws_lambda_function.api.function_name
 principal = "apigateway.amazonaws.com"
 source_arn = "${aws_apigatewayv2_api.api.execution_arn}/*/*"
}
```

## 93.5 28.5 Authentication and authorization

| Method             | Description                          |
|--------------------|--------------------------------------|
| IAM                | SigV4 signing; service-to-service    |
| Cognito User Pools | JWT tokens for user-facing APIs      |
| Lambda authorizer  | Custom auth logic (any token format) |

|                                  |                                         |
|----------------------------------|-----------------------------------------|
| <b>JWT authorizer</b> (HTTP API) | Validate JWT from any OIDC provider     |
| <b>API keys</b>                  | Simple key-based access (REST API only) |
| <b>Resource policy</b>           | IP-based or VPC-based restrictions      |

### 93.5.1 Lambda authorizer example

```
def handler(event, context):
 token = event["authorizationToken"]
 if not validate_token(token):
 raise Exception("Unauthorized") # Returns 401

 return {
 "principalId": "user-123",
 "policyDocument": {
 "Version": "2012-10-17",
 "Statement": [{
 "Action": "execute-api:Invoke",
 "Effect": "Allow",
 "Resource": event["methodArn"],
 }],
 },
 "context": {
 "userId": "user-123",
 "role": "admin",
 },
 }
```

## 93.6 28.6 Throttling and quotas

API Gateway protects backends with built-in throttling:

| Level                | Default                               |
|----------------------|---------------------------------------|
| <b>Account</b>       | 10,000 requests/second (can increase) |
| <b>Per API/stage</b> | Configurable burst and rate limits    |
| <b>Per method</b>    | Usage plans with API keys             |

### 93.6.1 Usage plans

```
resource "aws_api_gateway_usage_plan" "basic" {
 name = "basic-plan"

 api_stages {
 api_id = aws_api_gateway_rest_api.api.id
 stage = aws_api_gateway_stage.prod.stage_name
 }

 throttle_settings {
 burst_limit = 50
 rate_limit = 25
 }

 quota_settings {
 limit = 10000
 period = "MONTH"
 }
}
```

## 93.7 28.7 Request/response transformation

REST API supports **mapping templates** (VTL) for request/response transformation:

Client sends JSON → API Gateway transforms → DynamoDB PutItem format  
DynamoDB response → API Gateway transforms → Client JSON response

HTTP API passes the request directly to Lambda (proxy integration) with less transformation capability but better performance.

### 93.7.1 Request validation (REST API)

```
{
 "type": "object",
 "required": ["name", "email"],
 "properties": {
 "name": { "type": "string", "minLength": 1 },
 "email": { "type": "string", "format": "email" }
 }
}
```

## 93.8 28.8 Caching

API Gateway can cache responses at the stage level:

| Setting    | Value                     |
|------------|---------------------------|
| TTL        | 0-3600 seconds            |
| Cache key  | Query parameters, headers |
| Encryption | In-transit and at-rest    |
| Cost       | Per-hour cache capacity   |

Enable caching for read-heavy endpoints with infrequently changing data. Invalidate cache on deployments.

## 93.9 28.9 Custom domains and TLS

```
resource "aws_apigatewayv2_domain_name" "api" {
 domain_name = "api.example.com"

 domain_name_configuration {
 certificate_arn = aws_acm_certificate.api.arn
 endpoint_type = "REGIONAL"
 security_policy = "TLS_1_2"
 }
}

resource "aws_apigatewayv2_api_mapping" "api" {
 api_id = aws_apigatewayv2_api.api.id
 domain_name = aws_apigatewayv2_domain_name.api.id
 stage = aws_apigatewayv2_stage.prod.id
}
```

Route 53 alias record points `api.example.com` to the API Gateway domain.

## 93.10 28.10 WebSocket API

For real-time applications (chat, gaming, live dashboards):

Client  $\longleftrightarrow$  API Gateway  $\longleftrightarrow$  Lambda/DynamoDB  
\$connect                      Route by route key



```
$disconnect $default, custom routes
$default
```

```
resource "aws_apigatewayv2_api" "websocket" {
 name = "chat-api"
 protocol_type = "WEBSOCKET"
 route_selection_expression = "$request.body.action"
}
```

## 93.11 28.11 Monitoring

| Tool               | Data                                                        |
|--------------------|-------------------------------------------------------------|
| CloudWatch Metrics | Count, Latency, 4XXError, 5XXError, IntegrationLatency      |
| CloudWatch Logs    | Access logs, execution logs                                 |
| X-Ray              | Distributed tracing through API Gateway → Lambda → DynamoDB |

Key alarms: - 5XXError > 1% of requests - Latency p99 > 3000ms - Count approaching throttle limits

## 93.12 28.12 Chapter summary

- **API Gateway** is the managed front door for HTTP, REST, and WebSocket APIs.
- Use **HTTP API** for new Lambda backends; **REST API** for advanced features.
- Secure with **Cognito**, **Lambda authorizers**, or **IAM**.
- Configure **throttling**, **caching**, and **custom domains** for production.
- Monitor with CloudWatch metrics, access logs, and X-Ray tracing.

## 93.13 Lab 28.1 — HTTP API + Lambda CRUD

1. Create an HTTP API with Lambda integration for a DynamoDB-backed CRUD API.
2. Implement GET /items, POST /items, GET /items/{id}, DELETE /items/{id}.
3. Add a JWT authorizer (Cognito or custom).
4. Configure access logging and test with `curl`.

## 93.14 Lab 28.2 — Throttling and usage plans

1. Create a REST API with API key requirement.
2. Create a usage plan with 100 requests/day quota.
3. Exceed the quota and verify 429 responses.
4. Review CloudWatch metrics for throttled requests.

## 93.15 Review questions

1. When should you choose REST API over HTTP API?
2. How does a Lambda authorizer work?
3. What is the difference between burst limit and rate limit?
4. How do you attach a custom domain with TLS to API Gateway?
5. What CloudWatch metric indicates backend integration problems?

---

Next: [Chapter 29 — SQS, SNS, and EventBridge](#)

# 94 Chapter 29: SQS, SNS, and EventBridge

AWS Handbook — Part VII, Pages 581–605

## 94.1 29.1 Event-driven architecture on AWS

Modern applications decouple components using asynchronous messaging. AWS provides three core services for event-driven patterns:

| Service     | Pattern           | Delivery                        |
|-------------|-------------------|---------------------------------|
| SQS         | Message queuing   | Pull-based; consumer polls      |
| SNS         | Pub/sub fan-out   | Push-based; subscribers receive |
| EventBridge | Event bus routing | Push-based; rules route events  |

Understanding when to use each—and how to combine them—is fundamental to building scalable, resilient systems.

## 94.2 29.2 Amazon SQS

**Simple Queue Service (SQS)** is a fully managed message queue. Producers send messages; consumers poll and process them.

### 94.2.1 Queue types

| Type     | Throughput                          | Delivery                            | Use case                                 |
|----------|-------------------------------------|-------------------------------------|------------------------------------------|
| Standard | Unlimited                           | At-least-once; best-effort ordering | Most workloads                           |
| FIFO     | 3,000 msg/s (with batching: 30,000) | Exactly-once; strict ordering       | Order processing, financial transactions |

### 94.2.2 Key concepts

| Concept                 | Description                                                 |
|-------------------------|-------------------------------------------------------------|
| Message                 | Up to 256 KB (larger via S3 pointer)                        |
| Visibility timeout      | Message hidden after receive; must delete or it reappears   |
| Retention               | 1 minute to 14 days (default 4 days)                        |
| Dead letter queue (DLQ) | Captures messages that fail processing repeatedly           |
| Long polling            | Wait up to 20 seconds for messages (reduces empty receives) |

### 94.2.3 Standard queue workflow



### 94.2.4 Terraform

```

resource "aws_sqs_queue" "dlq" {
 name = "orders-dlq"
}

resource "aws_sqs_queue" "orders" {
 name = "orders-queue"
 visibility_timeout_seconds = 300
 message_retention_seconds = 1209600 # 14 days
 receive_wait_time_seconds = 20 # long polling

 redrive_policy = jsonencode({
 deadLetterTargetArn = aws_sqs_queue.dlq.arn
 maxReceiveCount = 3
 })
}

```

### 94.2.5 Lambda + SQS integration

```

resource "aws_lambda_event_source_mapping" "sqs" {
 event_source_arn = aws_sqs_queue.orders.arn
 function_name = aws_lambda_function.processor.arn
 batch_size = 10
 maximum_batching_window_in_seconds = 5

 scaling_config {
 maximum_concurrency = 50
 }
}

```

### 94.2.6 CLI

```

aws sqs send-message \
 --queue-url https://sqs.us-east-1.amazonaws.com/123456789012/orders-queue \
 --message-body '{"orderId": "12345", "action": "process"}'

aws sqs receive-message \
 --queue-url https://sqs.us-east-1.amazonaws.com/123456789012/orders-queue \
 --wait-time-seconds 20 \
 --max-number-of-messages 10

```

## 94.3 29.3 Amazon SNS

**Simple Notification Service (SNS)** is a pub/sub messaging service. Publishers send messages to a **topic**; subscribers (SQS, Lambda, HTTP, email, SMS) receive copies.

### 94.3.1 SNS + SQS fan-out pattern

```

 ┌ SQS Queue A → Lambda (process orders)
Publisher → SNS Topic ─┬─ SQS Queue B → Lambda (send email)
 └─ SQS Queue C → Lambda (update analytics)

```

Benefits: - Decouple publisher from consumers. - Each consumer processes at its own pace. - Add/remove consumers without changing the publisher.

```

resource "aws_sns_topic" "orders" {
 name = "order-events"
}

resource "aws_sns_topic_subscription" "email_queue" {
 topic_arn = aws_sns_topic.orders.arn
 protocol = "sqs"
 endpoint = aws_sqs_queue.email.arn
}

resource "aws_sqs_queue_policy" "email" {
 queue_url = aws_sqs_queue.email.id
 policy = jsonencode({

```

```
Statement = [{
 Effect = "Allow"
 Principal = { Service = "sns.amazonaws.com" }
 Action = "sqs:SendMessage"
 Resource = aws_sqs_queue.email.arn
 Condition = {
 ArnEquals = { "aws:SourceArn" = aws_sns_topic.orders.arn }
 }
}]
})
}
```

### 94.3.2 Message filtering

SNS subscription filter policies route only matching messages:

```
{
 "eventType": ["order.created", "order.updated"],
 "amount": [{ "numeric": [">", 100] }]
}
```

## 94.4 29.4 Amazon EventBridge

**EventBridge** is a serverless event bus that routes events from AWS services, SaaS applications, and custom applications.

### 94.4.1 Event buses

| Bus            | Events                                |
|----------------|---------------------------------------|
| <b>Default</b> | All AWS service events in the account |
| <b>Custom</b>  | Application-specific events           |
| <b>Partner</b> | SaaS events (Zendesk, Auth0, etc.)    |

### 94.4.2 Rules and targets

```
Event source → Rule (pattern match) → Target(s)
 |
 |— Lambda
 |— SQS
 |— SNS
 |— Step Functions
 |— API Gateway
 |— 20+ other targets
```

### 94.4.3 Event pattern example

```
{
 "source": ["aws.ec2"],
 "detail-type": ["EC2 Instance State-change Notification"],
 "detail": {
 "state": ["stopped", "terminated"]
 }
}
```

### 94.4.4 Terraform

```
resource "aws_cloudwatch_event_rule" "ec2_state" {
 name = "ec2-state-change"
 description = "Capture EC2 state changes"

 event_pattern = jsonencode({
 source = ["aws.ec2"]
 detail-type = ["EC2 Instance State-change Notification"]
 })
}
```

```

 detail = {
 state = ["stopped", "terminated"]
 }
})
}

resource "aws_cloudwatch_event_target" "lambda" {
 rule = aws_cloudwatch_event_rule.ec2_state.name
 target_id = "notify-slack"
 arn = aws_lambda_function.notify.arn
}

resource "aws_cloudwatch_event_bus" "app" {
 name = "app-events"
}

resource "aws_cloudwatch_event_rule" "order_created" {
 name = "order-created"
 event_bus_name = aws_cloudwatch_event_bus.app.name

 event_pattern = jsonencode({
 source = ["app.orders"]
 detail-type = ["OrderCreated"]
 })
}

```

### 94.4.5 Custom events

```

import boto3, json

events = boto3.client("events")
events.put_events(Entries=[{
 "Source": "app.orders",
 "DetailType": "OrderCreated",
 "Detail": json.dumps({"orderId": "12345", "amount": 99.99}),
 "EventBusName": "app-events",
}])

```

## 94.5 29.5 Choosing the right service

| Scenario                                     | Service                 |
|----------------------------------------------|-------------------------|
| Decouple two services; consumer polls        | SQS                     |
| Fan-out to multiple consumers                | SNS (+ SQS for each)    |
| React to AWS service events                  | EventBridge             |
| Custom application events with routing rules | EventBridge             |
| Guaranteed ordering                          | SQS FIFO                |
| Exactly-once processing                      | SQS FIFO                |
| Cross-account event routing                  | EventBridge             |
| Real-time push to HTTP endpoints             | SNS (HTTP subscription) |
| Scheduled events                             | EventBridge Scheduler   |

### 94.5.1 EventBridge vs SNS

| Feature          | EventBridge         | SNS                          |
|------------------|---------------------|------------------------------|
| Event filtering  | Content-based rules | Subscription filter policies |
| Targets per rule | 5 (default)         | Many subscribers per topic   |
| Schema registry  | Yes                 | No                           |

|                    |               |                  |
|--------------------|---------------|------------------|
| Archive and replay | Yes           | No               |
| Cross-account      | Native        | Via topic policy |
| Delivery           | At-least-once | At-least-once    |

## 94.6 29.6 Error handling patterns

### 94.6.1 SQS retry with DLQ

Receive → Process (fail) → Visibility timeout expires → Re-deliver  
→ Fail 3 times → Move to DLQ → Alert + manual review

### 94.6.2 SNS retry

SNS retries failed deliveries to HTTP/Lambda targets with exponential backoff. Configure DLQ on the subscription for persistent failures.

### 94.6.3 EventBridge retry

EventBridge retries failed target invocations for 24 hours with exponential backoff. Configure DLQ or retry policy per target.

## 94.7 29.7 Monitoring

| Service            | Key metrics                                                           |
|--------------------|-----------------------------------------------------------------------|
| <b>SQS</b>         | ApproximateNumberOfMessagesVisible ,<br>ApproximateAgeOfOldestMessage |
| <b>SNS</b>         | NumberOfMessagesPublished , NumberOfNotificationsFailed               |
| <b>EventBridge</b> | FailedInvocations , Invocations , ThrottledRules                      |

Alarm on DLQ message count > 0 and ApproximateAgeOfOldestMessage > threshold (processing lag).

## 94.8 29.8 Chapter summary

- **SQS** provides reliable message queuing with standard and FIFO options.
- **SNS** enables pub/sub fan-out to multiple subscribers.
- **EventBridge** routes events from AWS services and custom applications with content-based filtering.
- Combine **SNS + SQS** for fan-out with independent consumer scaling.
- Configure **DLQs** and monitor queue depth and message age.

## 94.9 Lab 29.1 — Order processing pipeline

1. Create an SNS topic for order events.
2. Subscribe two SQS queues (email notifications, inventory update).
3. Publish order events and verify both queues receive messages.
4. Process messages with Lambda and configure DLQs.

## 94.10 Lab 29.2 — EventBridge automation

1. Create a rule that triggers on S3 `ObjectCreated` events.
  2. Route matching events to a Lambda that processes uploaded files.
  3. Create a custom event bus and publish application events.
  4. Archive events and replay from a specific timestamp.
- 

## 94.11 Review questions

1. What is the difference between SQS standard and FIFO queues?
  2. How does the SNS + SQS fan-out pattern work?
  3. When would you use EventBridge instead of SNS?
  4. What is a dead letter queue and when should you configure one?
  5. What does the SQS visibility timeout control?
- 

Next: [Chapter 30 — Step Functions](#)



# 95 Chapter 30: Step Functions Workflows

AWS Handbook — Part VII, Pages 606–620

## 95.1 30.1 What are Step Functions?

**AWS Step Functions** is a serverless orchestration service that coordinates multiple AWS services into visual workflows. Instead of writing complex retry/error logic in application code, you define workflows as state machines that handle sequencing, parallel execution, error handling, and human approval steps.

Step Functions is ideal for order processing, ETL pipelines, ML workflows, and multi-step business processes.

## 95.2 30.2 Workflow types

| Type     | Language                           | Use case                                             |
|----------|------------------------------------|------------------------------------------------------|
| Standard | Amazon States Language (JSON/YAML) | Long-running (up to 1 year), exactly-once            |
| Express  | ASL                                | High-volume, short-duration (< 5 min), at-least-once |

### 95.2.1 Standard vs Express

| Feature         | Standard                | Express                         |
|-----------------|-------------------------|---------------------------------|
| Duration        | Up to 1 year            | Up to 5 minutes                 |
| Execution model | Exactly-once            | At-least-once                   |
| Pricing         | Per state transition    | Per execution + duration        |
| Use case        | Business workflows, ETL | IoT, streaming, high-throughput |

## 95.3 30.3 State types

| State    | Purpose                                        |
|----------|------------------------------------------------|
| Task     | Execute work (Lambda, ECS, DynamoDB, etc.)     |
| Choice   | Branch based on conditions                     |
| Parallel | Execute branches concurrently                  |
| Map      | Iterate over array items (dynamic parallelism) |
| Wait     | Delay for fixed time or until timestamp        |
| Pass     | Pass input to output (transform)               |
| Succeed  | Terminal success                               |
| Fail     | Terminal failure                               |

## 95.4 30.4 Example: Order processing workflow

```

Start → ValidateOrder → Choice (valid?)
├── Yes → ProcessPayment → Choice (paid?)
│ ├── Yes → FulfillOrder → Succeed
│ └── No → NotifyFailure → Fail
└── No → NotifyInvalid → Fail

```

### 95.4.1 ASL definition

```

{
 "Comment": "Order processing workflow",
 "StartAt": "ValidateOrder",
 "States": {
 "ValidateOrder": {
 "Type": "Task",
 "Resource": "arn:aws:lambda:us-east-1:123456789012:function:validate-order",
 "Next": "IsOrderValid",
 "Retry": [{
 "ErrorEquals": ["Lambda.ServiceException", "Lambda.TooManyRequestsException"],
 "IntervalSeconds": 2,
 "MaxAttempts": 3,
 "BackoffRate": 2
 }],
 "Catch": [{
 "ErrorEquals": ["ValidationError"],
 "Next": "NotifyInvalid"
 }]
 },
 "IsOrderValid": {
 "Type": "Choice",
 "Choices": [{
 "Variable": "$.valid",
 "BooleanEquals": true,
 "Next": "ProcessPayment"
 }],
 "Default": "NotifyInvalid"
 },
 "ProcessPayment": {
 "Type": "Task",
 "Resource": "arn:aws:states:::lambda:invoke",
 "Parameters": {
 "FunctionName": "process-payment",
 "Payload.$": "$"
 },
 "ResultPath": "$.payment",
 "Next": "IsPaymentSuccessful"
 },
 "IsPaymentSuccessful": {
 "Type": "Choice",
 "Choices": [{
 "Variable": "$.payment.Payload.success",
 "BooleanEquals": true,
 "Next": "FulfillOrder"
 }],
 "Default": "NotifyFailure"
 },
 "FulfillOrder": {
 "Type": "Task",
 "Resource": "arn:aws:states:::sqs:sendMessage.waitForTaskToken",
 "Parameters": {
 "QueueUrl": "https://sqs.us-east-1.amazonaws.com/123456789012/fulfillment",
 "MessageBody": {
 "orderId.$": "$.orderId",
 "taskToken.$": "$$.Task.Token"
 }
 },
 "End": true
 },
 "NotifyInvalid": {
 "Type": "Task",
 "Resource": "arn:aws:lambda:us-east-1:123456789012:function:notify-invalid",
 "Next": "OrderFailed"
 },
 "NotifyFailure": {
 "Type": "Task",
 "Resource": "arn:aws:lambda:us-east-1:123456789012:function:notify-failure",
 "Next": "OrderFailed"
 }
 }
}

```

```

 "OrderFailed": {
 "Type": "Fail",
 "Error": "OrderProcessingFailed",
 "Cause": "Order could not be processed"
 }
 }
 }
}

```

## 95.5 30.5 Service integrations

Step Functions integrates with 220+ AWS services via **optimized integrations** (no Lambda wrapper needed):

| Integration pattern      | Example                                                         |
|--------------------------|-----------------------------------------------------------------|
| <b>Request Response</b>  | Invoke Lambda, call DynamoDB GetItem                            |
| <b>Run a Job</b>         | Submit Batch job, start ECS task, start Glue job                |
| <b>Wait for Callback</b> | Send SQS message with task token; resume when callback received |

### 95.5.1 Direct DynamoDB integration

```

{
 "Type": "Task",
 "Resource": "arn:aws:states:::dynamodb:putItem",
 "Parameters": {
 "TableName": "Orders",
 "Item": {
 "OrderId": { "S.$": "$.orderId" },
 "Status": { "S": "PROCESSING" },
 "Amount": { "N.$": "$.amount" }
 }
 },
 "Next": "ProcessPayment"
}

```

### 95.5.2 ECS/Fargate task

```

{
 "Type": "Task",
 "Resource": "arn:aws:states:::ecs:runTask.sync",
 "Parameters": {
 "LaunchType": "FARGATE",
 "Cluster": "processing-cluster",
 "TaskDefinition": "data-processor",
 "NetworkConfiguration": {
 "AwsVpcConfiguration": {
 "Subnets": ["subnet-aaa", "subnet-bbb"],
 "SecurityGroups": ["sg-processor"]
 }
 }
 },
 "Next": "CheckResults"
}

```

## 95.6 30.6 Error handling

### 95.6.1 Retry

```

"Retry": [{

```

```

 "ErrorEquals": ["States.TaskFailed"],
 "IntervalSeconds": 1,
 "MaxAttempts": 3,
 "BackoffRate": 2.0,
 "JitterStrategy": "FULL"
 }]
}

```

## 95.6.2 Catch

```

"Catch": [{
 "ErrorEquals": ["PaymentDeclined"],
 "ResultPath": "$.error",
 "Next": "HandlePaymentFailure"
}]

```

## 95.6.3 Error types

| Error              | Source                                                            |
|--------------------|-------------------------------------------------------------------|
| States.Timeout     | Task exceeded timeout                                             |
| States.TaskFailed  | Task returned error                                               |
| States.Permissions | IAM permission denied                                             |
| Custom errors      | Application-defined (via <code>Fail</code> state or Lambda throw) |

# 95.7 30.7 Parallel and Map states

## 95.7.1 Parallel

Execute multiple branches simultaneously:

```

{
 "Type": "Parallel",
 "Branches": [
 { "StartAt": "SendEmail", "States": { "SendEmail": { "Type": "Task", "Resource": "...", "End": true } } },
 { "StartAt": "UpdateInventory", "States": { "UpdateInventory": { "Type": "Task", "Resource": "...", "End": true } } },
 { "StartAt": "LogAnalytics", "States": { "LogAnalytics": { "Type": "Task", "Resource": "...", "End": true } } }
],
 "Next": "OrderComplete"
}

```

## 95.7.2 Map (dynamic parallelism)

Process each item in an array concurrently:

```

{
 "Type": "Map",
 "ItemsPath": "$.orders",
 "MaxConcurrency": 10,
 "Iterator": {
 "StartAt": "ProcessSingleOrder",
 "States": {
 "ProcessSingleOrder": {
 "Type": "Task",
 "Resource": "arn:aws:lambda:...:function:process-order",
 "End": true
 }
 }
 },
 "Next": "AllOrdersProcessed"
}

```

**Distributed Map** (newer) supports large-scale processing with S3 item sources and child executions.

---

# 95.8 30.8 Terraform

```
resource "aws_sfn_state_machine" "order_processing" {
 name = "order-processing"
 role_arn = aws_iam_role.step_functions.arn

 definition = templatefile("${path.module}/state-machine.json", {
 validate_lambda_arn = aws_lambda_function.validate.arn
 payment_lambda_arn = aws_lambda_function.payment.arn
 })

 logging_configuration {
 level = "ALL"
 include_execution_data = true
 log_destination = "${aws_cloudwatch_log_group.sfn.arn}:*"
 }

 tracing_configuration {
 enabled = true
 }
}
```

---

# 95.9 30.9 Monitoring and debugging

| Tool               | Purpose                                            |
|--------------------|----------------------------------------------------|
| Execution history  | Visual trace of each state transition              |
| CloudWatch Logs    | Detailed execution data                            |
| X-Ray              | Distributed tracing across services                |
| CloudWatch Metrics | ExecutionsStarted, ExecutionsFailed, ExecutionTime |

```
aws stepfunctions start-execution \
 --state-machine-arn arn:aws:states:us-east-1:123456789012:stateMachine:order-processing \
 --input '{"orderId": "12345", "amount": 99.99}'

aws stepfunctions describe-execution \
 --execution-arn arn:aws:states:us-east-1:123456789012:execution:order-processing:abc123
```

---

# 95.10 30.10 Chapter summary

- **Step Functions** orchestrates multi-step workflows with built-in error handling and retries.
- **Standard workflows** for long-running processes; **Express** for high-volume, short tasks.
- Use **service integrations** to call AWS services directly without Lambda wrappers.
- **Map** and **Parallel** states enable dynamic and static concurrency.
- Monitor with execution history, CloudWatch Logs, and X-Ray tracing.

---

# 95.11 Lab 30.1 — Order workflow

1. Create a Step Functions state machine with validate → pay → fulfill steps.
2. Implement Lambda functions for each step.
3. Test successful and failed order paths.
4. Add retry logic for transient failures.

## 95.12 Lab 30.2 — Parallel processing

1. Create a Map state that processes an array of 10 items concurrently.
  2. Set `MaxConcurrency` to 3 and observe throttled execution.
  3. Review execution history in the Step Functions console.
- 

## 95.13 Review questions

1. What is the difference between Standard and Express workflows?
  2. How do Retry and Catch differ in Step Functions error handling?
  3. What is the purpose of the Map state?
  4. How does the “wait for callback” pattern work with task tokens?
  5. When would you use a direct service integration vs a Lambda task?
- 

Next: [Chapter 31 — CloudFormation](#)

# 96 Chapter 31: CloudFormation Deep Dive

AWS Handbook — Part VIII, Pages 621–645

## 96.1 31.1 Infrastructure as Code with CloudFormation

**AWS CloudFormation** is AWS’s native Infrastructure as Code (IaC) service. You define resources in YAML or JSON templates; CloudFormation provisions, updates, and deletes them as a single **stack**. Changes are tracked, rollback is automatic on failure, and drift detection identifies manual modifications.

For AWS-only environments, CloudFormation provides the deepest service integration and zero licensing cost.

## 96.2 31.2 Template structure

```
AWSTemplateFormatVersion: "2010-09-09"
Description: Web application stack

Parameters:
 Environment:
 Type: String
 Default: dev
 AllowedValues: [dev, staging, prod]

Mappings:
 RegionMap:
 us-east-1:
 AMI: ami-0c55b159cbfafa1f0
 eu-west-1:
 AMI: ami-0d71ea30463e0ff8d

Conditions:
 IsProd: !Equals [!Ref Environment, prod]

Resources:
 WebServer:
 Type: AWS::EC2::Instance
 Properties:
 ImageId: !FindInMap [RegionMap, !Ref "AWS::Region", AMI]
 InstanceType: !If [IsProd, t3.medium, t3.micro]

Outputs:
 PublicIP:
 Value: !GetAtt WebServer.PublicIp
 Export:
 Name: !Sub "${AWS::StackName}-PublicIP"
```

### 96.2.1 Intrinsic functions

| Function                | Purpose                         | Example                                        |
|-------------------------|---------------------------------|------------------------------------------------|
| <code>!Ref</code>       | Reference parameter or resource | <code>!Ref WebServer</code>                    |
| <code>!GetAtt</code>    | Get resource attribute          | <code>!GetAtt WebServer.PublicIp</code>        |
| <code>!Sub</code>       | String substitution             | <code>!Sub "arn:aws:s3:::\${Bucket}"</code>    |
| <code>!Join</code>      | Concatenate strings             | <code>!Join ["-", [a, b]]</code>               |
| <code>!Select</code>    | Pick from list                  | <code>!Select [0, !GetAZs ""]</code>           |
| <code>!If</code>        | Conditional value               | <code>!If [IsProd, t3.medium, t3.micro]</code> |
| <code>!FindInMap</code> | Lookup in mapping               | <code>!FindInMap [RegionMap, !Ref</code>       |

## 96.3 31.3 Stack operations

```
Validate template
aws cloudformation validate-template --template-body file://template.yaml

Create stack
aws cloudformation create-stack \
 --stack-name web-app-dev \
 --template-body file://template.yaml \
 --parameters ParameterKey=Environment,ParameterValue=dev \
 --capabilities CAPABILITY_IAM

Update stack
aws cloudformation update-stack \
 --stack-name web-app-dev \
 --template-body file://template.yaml \
 --parameters ParameterKey=Environment,ParameterValue=staging

Delete stack (removes all resources)
aws cloudformation delete-stack --stack-name web-app-dev

Describe events (troubleshooting)
aws cloudformation describe-stack-events --stack-name web-app-dev
```

### 96.3.1 Change sets

Preview changes before applying:

```
aws cloudformation create-change-set \
 --stack-name web-app-dev \
 --template-body file://template-v2.yaml \
 --change-set-name update-to-v2

aws cloudformation describe-change-set \
 --stack-name web-app-dev \
 --change-set-name update-to-v2

aws cloudformation execute-change-set \
 --stack-name web-app-dev \
 --change-set-name update-to-v2
```

## 96.4 31.4 Nested stacks

Break large templates into reusable modules:

```
Resources:
 NetworkStack:
 Type: AWS::CloudFormation::Stack
 Properties:
 TemplateURL: https://s3.amazonaws.com/templates/network.yaml
 Parameters:
 VpcCidr: 10.0.0.0/16

 AppStack:
 Type: AWS::CloudFormation::Stack
 DependsOn: NetworkStack
 Properties:
 TemplateURL: https://s3.amazonaws.com/templates/app.yaml
 Parameters:
 VpcId: !GetAtt NetworkStack.Outputs.VpcId
 SubnetIds: !GetAtt NetworkStack.Outputs.PrivateSubnetIds
```

### 96.4.1 Cross-stack references



Export values from one stack and import in another:

```
Stack A (network)
Outputs:
 VpcId:
 Value: !Ref MyVpc
 Export:
 Name: !Sub "${AWS::StackName}-VpcId"

Stack B (application)
Resources:
 WebServer:
 Properties:
 VpcId: !ImportValue network-stack-VpcId
```

## 96.5 31.5 StackSets

Deploy stacks across multiple accounts and regions from a single template:

```
aws cloudformation create-stack-set \
 --stack-set-name baseline-security \
 --template-body file://security-baseline.yaml \
 --permission-model SERVICE_MANAGED \
 --auto-deployment Enabled=true

aws cloudformation create-stack-instances \
 --stack-set-name baseline-security \
 --deployment-targets OrganizationalUnitIds=["ou-abc-123"] \
 --regions us-east-1 eu-west-1
```

Use cases: organization-wide guardrails, logging, security baselines.

## 96.6 31.6 IAM in CloudFormation

### 96.6.1 CAPABILITY flags

| Flag                   | When required                        |
|------------------------|--------------------------------------|
| CAPABILITY_IAM         | Template creates IAM resources       |
| CAPABILITY_NAMED_IAM   | Template creates named IAM resources |
| CAPABILITY_AUTO_EXPAND | Template uses macros (SAM, CDK)      |

### 96.6.2 Least-privilege IAM example

```
Resources:
 LambdaRole:
 Type: AWS::IAM::Role
 Properties:
 AssumeRolePolicyDocument:
 Version: "2012-10-17"
 Statement:
 - Effect: Allow
 Principal:
 Service: lambda.amazonaws.com
 Action: sts:AssumeRole
 Policies:
 - PolicyName: DynamoDBAccess
 PolicyDocument:
 Version: "2012-10-17"
 Statement:
 - Effect: Allow
 Action:
 - dynamodb:GetItem
```

```
- dynamodb:PutItem
Resource: !GetAtt OrdersTable.Arn
```

## 96.7 31.7 Drift detection

Manual changes outside CloudFormation create **drift**:

```
aws cloudformation detect-stack-drift --stack-name web-app-dev

aws cloudformation describe-stack-drift-detection-status \
 --stack-drift-detection-id abc-123

aws cloudformation describe-stack-resource-drifts \
 --stack-name web-app-dev
```

**Best practice:** All infrastructure changes go through CloudFormation. Use SCPs to deny direct resource creation where possible.

## 96.8 31.8 Custom resources

When CloudFormation lacks native support, use **custom resources** backed by Lambda:

```
Resources:
 CustomDNS:
 Type: Custom::DNSRecord
 Properties:
 ServiceToken: !GetAtt DNSLambda.Arn
 HostedZoneId: Z1234567890
 Name: app.example.com
 Value: !GetAtt WebServer.PublicIp
```

The Lambda handles Create, Update, and Delete requests and sends a response to CloudFormation's pre-signed S3 URL.

## 96.9 31.9 CloudFormation Hooks

**Hooks** validate resources before provisioning (proactive compliance):

| Hook         | Purpose                                 |
|--------------|-----------------------------------------|
| Guard Hooks  | Evaluate templates against policy rules |
| Lambda Hooks | Custom validation logic                 |

```
aws cloudformation activate-type \
 --type HOOK \
 --type-name MyOrg::S3::EncryptionCheck \
 --publisher-id 123456789012
```

## 96.10 31.10 Best practices

| Practice                                       | Rationale                           |
|------------------------------------------------|-------------------------------------|
| Use parameters for environment-specific values | Reuse templates across environments |
|                                                |                                     |

|                                              |                                     |
|----------------------------------------------|-------------------------------------|
| Use mappings for region-specific constants   | AMI IDs, instance types             |
| Use conditions for optional resources        | Prod-only resources (WAF, Multi-AZ) |
| Tag all resources                            | Cost allocation, automation         |
| Use nested stacks for modularity             | Manageable template sizes           |
| Enable termination protection on prod stacks | Prevent accidental deletion         |
| Use change sets for production updates       | Preview before apply                |
| Store templates in S3 with versioning        | Audit trail, rollback               |
| Run drift detection regularly                | Catch manual changes                |

---

## 96.11 31.11 Chapter summary

- **CloudFormation** provisions AWS infrastructure from declarative YAML/JSON templates.
  - **Stacks** group related resources; **nested stacks** and **StackSets** enable modularity and multi-account deployment.
  - Use **change sets** to preview updates; **drift detection** catches manual changes.
  - **Custom resources** extend CloudFormation with Lambda-backed logic.
  - Follow IaC best practices: parameters, conditions, tagging, and termination protection.
- 

## 96.12 Lab 31.1 — Multi-tier stack

1. Write a CloudFormation template with VPC, subnets, security groups, ALB, and EC2.
2. Deploy with parameters for environment (dev/prod).
3. Update the template to add an RDS instance (prod only, using conditions).
4. Create a change set and review before applying.

## 96.13 Lab 31.2 — Drift detection

1. Deploy a stack and manually modify a security group in the console.
  2. Run drift detection and identify the drifted resource.
  3. Update the stack to reconcile the drift.
- 

## 96.14 Review questions

1. What is the purpose of CloudFormation change sets?
  2. How do nested stacks improve template organization?
  3. What capability flag is required when creating IAM roles in a template?
  4. What is stack drift and how do you detect it?
  5. When would you use StackSets instead of individual stacks?
- 

Next: [Chapter 32 — AWS CDK](#)

# 97 Chapter 32: AWS CDK

AWS Handbook — Part VIII, Pages 646–665

## 97.1 32.1 What is the AWS CDK?

The **AWS Cloud Development Kit (CDK)** lets you define cloud infrastructure using familiar programming languages (TypeScript, Python, Java, C#, Go). CDK synthesizes your code into CloudFormation templates, combining the expressiveness of a general-purpose language with CloudFormation’s deployment engine.

CDK is ideal for teams that find YAML/JSON templates limiting and want reusable constructs, unit tests, and IDE support.

## 97.2 32.2 CDK concepts

| Concept    | Description                                                  |
|------------|--------------------------------------------------------------|
| App        | Root of the CDK application                                  |
| Stack      | Unit of deployment (maps to CloudFormation stack)            |
| Construct  | Reusable component (L1, L2, L3)                              |
| Synthesis  | Generate CloudFormation template from code                   |
| Deployment | <code>cdk deploy</code> creates/updates CloudFormation stack |

### 97.2.1 Construct levels

| Level    | Description                                       | Example                                            |
|----------|---------------------------------------------------|----------------------------------------------------|
| L1 (Cfn) | Direct CloudFormation resource                    | <code>CfnBucket</code>                             |
| L2       | Curated construct with sensible defaults          | <code>s3.Bucket</code>                             |
| L3       | Opinionated patterns combining multiple resources | <code>ApplicationLoadBalancedFargateService</code> |

## 97.3 32.3 Getting started (TypeScript)

```
npm install -g aws-cdk
cdk init app --language typescript
npm install @aws-cdk/aws-lambda @aws-cdk/aws-apigatewayv2-integrations
```

### 97.3.1 Basic stack

```
import * as cdk from "aws-cdk-lib";
import * as s3 from "aws-cdk-lib/aws-s3";
import * as lambda from "aws-cdk-lib/aws-lambda";
import * as apigwv2 from "aws-cdk-lib/aws-apigatewayv2";
import * as integrations from "aws-cdk-lib/aws-apigatewayv2-integrations";

export class AppStack extends cdk.Stack {
 constructor(scope: cdk.App, id: string, props?: cdk.StackProps) {
 super(scope, id, props);

 const bucket = new s3.Bucket(this, "DataBucket", {
```

```

 encryption: s3.BucketEncryption.S3_MANAGED,
 blockPublicAccess: s3.BlockPublicAccess.BLOCK_ALL,
 versioned: true,
 removalPolicy: cdk.RemovalPolicy.RETAIN,
 });

 const fn = new lambda.Function(this, "ApiHandler", {
 runtime: lambda.Runtime.PYTHON_3_12,
 handler: "app.handler",
 code: lambda.Code.fromAsset("lambda"),
 environment: { BUCKET_NAME: bucket.bucketName },
 });

 bucket.grantReadWrite(fn);

 const api = new apigwv2.HttpApi(this, "Api", {
 defaultIntegration: new integrations.HttpLambdaIntegration(
 "LambdaIntegration", fn
),
 });

 new cdk.CfnOutput(this, "ApiUrl", { value: api.url! });
}

const app = new cdk.App();
new AppStack(app, "AppStack", { env: { account: "123456789012", region: "us-east-1" } });

```

### 97.3.2 Python equivalent

```

from aws_cdk import (
 App, Stack, CfnOutput, RemovalPolicy,
 aws_s3 as s3,
 aws_lambda as lambda_,
)
from constructs import Construct

class AppStack(Stack):
 def __init__(self, scope: Construct, id: str, **kwargs):
 super().__init__(scope, id, **kwargs)

 bucket = s3.Bucket(
 self, "DataBucket",
 encryption=s3.BucketEncryption.S3_MANAGED,
 block_public_access=s3.BlockPublicAccess.BLOCK_ALL,
 versioned=True,
 removal_policy=RemovalPolicy.RETAIN,
)

 fn = lambda_.Function(
 self, "Processor",
 runtime=lambda_.Runtime.PYTHON_3_12,
 handler="app.handler",
 code=lambda_.Code.from_asset("lambda"),
 environment={"BUCKET_NAME": bucket.bucket_name},
)
 bucket.grant_read_write(fn)

app = App()
AppStack(app, "AppStack")
app.synth()

```

## 97.4 32.4 CDK workflow

```

cdk synth # Generate CloudFormation template
cdk diff # Compare deployed stack with current code
cdk deploy # Deploy stack(s)
cdk destroy # Delete stack(s)
cdk ls # List stacks in the app
cdk bootstrap # Prepare account/region (one-time)

```

### 97.4.1 Bootstrapping

CDK requires an S3 bucket and ECR repo for assets (Lambda code, Docker images):

```
cdk bootstrap aws://123456789012/us-east-1
```

## 97.5 32.5 Reusable constructs

### 97.5.1 Creating a construct

```
import { Construct } from "constructs";
import * as ec2 from "aws-cdk-lib/aws-ec2";
import * as rds from "aws-cdk-lib/aws-rds";

interface DatabaseProps {
 vpc: ec2.IVpc;
 instanceType: ec2.InstanceType;
}

export class Database extends Construct {
 public readonly endpoint: string;

 constructor(scope: Construct, id: string, props: DatabaseProps) {
 super(scope, id);

 const cluster = new rds.DatabaseCluster(this, "Cluster", {
 engine: rds.DatabaseClusterEngine.auroraPostgres({
 version: rds.AuroraPostgresEngineVersion.VER_15_4,
 }),
 vpc: props.vpc,
 writer: rds.ClusterInstance.provisioned("Writer", {
 instanceType: props.instanceType,
 }),
 readers: [
 rds.ClusterInstance.provisioned("Reader", {
 instanceType: props.instanceType,
 }),
],
 storageEncrypted: true,
 });

 this.endpoint = cluster.clusterEndpoint.hostname;
 }
}
```

### 97.5.2 Using the construct

```
const db = new Database(this, "AppDatabase", {
 vpc,
 instanceType: ec2.InstanceType.of(ec2.InstanceClass.R6G, ec2.InstanceSize.LARGE),
});
```

## 97.6 32.6 CDK Pipelines

CDK Pipelines is a high-level construct for CI/CD:

```
import { pipelines } from "aws-cdk-lib";

const pipeline = new pipelines.CodePipeline(this, "Pipeline", {
 pipelineName: "AppPipeline",
 synth: new pipelines.ShellStep("Synth", {
 input: pipelines.CodePipelineSource.gitHub("org/repo", "main"),
 commands: ["npm ci", "npm run build", "npx cdk synth"],
 }),
});
```

```
pipeline.addStage(new AppStage(this, "Prod", { env: prodEnv }));
```

Stages deploy to different environments with automatic approvals between them.

## 97.7 32.7 Testing CDK

### 97.7.1 Snapshot tests

```
import { Template } from "aws-cdk-lib/assertions";

test("S3 bucket is encrypted", () => {
 const stack = new AppStack(app, "TestStack");
 const template = Template.fromStack(stack);

 template.hasResourceProperties("AWS::S3::Bucket", {
 BucketEncryption: {
 ServerSideEncryptionConfiguration: [{
 ServerSideEncryptionByDefault: { SSEAlgorithm: "AES256" },
 }],
 },
 });
});
```

### 97.7.2 Fine-grained assertions

```
template.resourceCountIs("AWS::S3::Bucket", 1);
template.hasResource("AWS::IAM::Role", {
 Properties: { AssumeRolePolicyDocument: { /* ... */ } },
});
```

## 97.8 32.8 CDK vs CloudFormation vs Terraform

| Feature              | CDK                      | CloudFormation    | Terraform           |
|----------------------|--------------------------|-------------------|---------------------|
| Language             | TypeScript, Python, etc. | YAML/JSON         | HCL                 |
| AWS integration      | Deepest                  | Native            | Via provider        |
| Multi-cloud          | No                       | No                | Yes                 |
| State management     | CloudFormation           | CloudFormation    | Remote state        |
| Constructs/libraries | Rich ecosystem           | Modules (limited) | Modules (extensive) |
| Learning curve       | Moderate                 | Low               | Moderate            |

## 97.9 32.9 Best practices

| Practice                                  | Rationale                           |
|-------------------------------------------|-------------------------------------|
| Use L2 constructs over L1                 | Sensible defaults, less boilerplate |
| Create reusable L3 constructs             | Consistency across teams            |
| Use <code>cdk diff</code> before deploy   | Preview changes                     |
| Set <code>removalPolicy</code> explicitly | Prevent accidental data loss        |
|                                           |                                     |

|                                        |                                         |
|----------------------------------------|-----------------------------------------|
| Use aspects for cross-cutting concerns | Tagging, encryption, compliance         |
| Write snapshot tests                   | Catch unintended infrastructure changes |
| Pin CDK version                        | Reproducible builds                     |
| Use environments for multi-account     | <code>env: { account, region }</code>   |

---

## 97.10 32.10 Chapter summary

- **CDK** defines infrastructure in TypeScript, Python, or other languages; synthesizes to CloudFormation.
  - **Constructs** (L1/L2/L3) provide reusable building blocks.
  - **CDK Pipelines** enables CI/CD for infrastructure deployments.
  - Test with **snapshot assertions** and fine-grained resource checks.
  - Bootstrap accounts once; use `cdk diff` and `cdk deploy` for safe updates.
- 

## 97.11 Lab 32.1 — CDK web application

1. Initialize a CDK TypeScript project.
2. Create a stack with S3, Lambda, and HTTP API.
3. Run `cdk synth` and inspect the generated CloudFormation template.
4. Deploy with `cdk deploy` and test the API endpoint.

## 97.12 Lab 32.2 — Reusable construct

1. Create a `VpcStack` construct with public/private subnets.
  2. Create an `AppStack` that imports the VPC and deploys ECS Fargate.
  3. Write a snapshot test verifying the VPC has 2 AZs and 4 subnets.
- 

## 97.13 Review questions

1. What is the relationship between CDK and CloudFormation?
  2. What is the difference between L1, L2, and L3 constructs?
  3. Why is `cdk bootstrap` required before the first deployment?
  4. How do CDK snapshot tests help prevent infrastructure regressions?
  5. When would you choose CDK over raw CloudFormation templates?
- 

Next: [Chapter 33 — CodePipeline, CodeBuild, CodeDeploy](#)



# 98 Chapter 33: CodePipeline, CodeBuild, and CodeDeploy

AWS Handbook — Part VIII, Pages 666–685

## 98.1 33.1 CI/CD on AWS

Continuous Integration and Continuous Delivery automate the path from code commit to production deployment. AWS provides a native CI/CD suite:

| Service      | Role                                                 |
|--------------|------------------------------------------------------|
| CodeCommit   | Git repository (being deprecated; use GitHub/GitLab) |
| CodeBuild    | Build and test code                                  |
| CodeDeploy   | Deploy to EC2, ECS, Lambda, on-premises              |
| CodePipeline | Orchestrate the full pipeline                        |
| CodeArtifact | Package repository (npm, Maven, PyPI)                |

## 98.2 33.2 CodePipeline

**CodePipeline** orchestrates stages from source to deployment:

Source → Build → Test → Deploy (Staging) → Approval → Deploy (Production)

### 98.2.1 Pipeline stages

| Stage    | Actions                                     |
|----------|---------------------------------------------|
| Source   | GitHub, CodeCommit, S3, ECR                 |
| Build    | CodeBuild, Jenkins, custom                  |
| Test     | CodeBuild, third-party                      |
| Deploy   | CodeDeploy, CloudFormation, ECS, Lambda, S3 |
| Approval | Manual approval gate                        |

### 98.2.2 Terraform pipeline

```
resource "aws_codepipeline" "app" {
 name = "app-pipeline"
 role_arn = aws_iam_role.pipeline.arn

 artifact_store {
 location = aws_s3_bucket.artifacts.bucket
 type = "S3"
 }

 stage {
 name = "Source"
 action {
 name = "GitHub"
 category = "Source"
 owner = "AWS"
 }
 }
}
```

```

 provider = "CodeStarSourceConnection"
 version = "1"
 output_artifacts = ["source_output"]
 configuration = {
 ConnectionArn = aws_codestarconnections_connection.github.arn
 FullRepositoryId = "org/app-repo"
 BranchName = "main"
 }
 }
}

stage {
 name = "Build"
 action {
 name = "Build"
 category = "Build"
 owner = "AWS"
 provider = "CodeBuild"
 version = "1"
 input_artifacts = ["source_output"]
 output_artifacts = ["build_output"]
 configuration = {
 ProjectName = aws_codebuild_project.app.name
 }
 }
}

stage {
 name = "Deploy"
 action {
 name = "DeployECS"
 category = "Deploy"
 owner = "AWS"
 provider = "ECS"
 version = "1"
 input_artifacts = ["build_output"]
 configuration = {
 ClusterName = aws_ecs_cluster.main.name
 ServiceName = aws_ecs_service.web.name
 FileName = "imagedefinitions.json"
 }
 }
}
}

```

## 98.3 33.3 CodeBuild

**CodeBuild** compiles source code, runs tests, and produces deployment artifacts:

### 98.3.1 buildspec.yml

```

version: 0.2

env:
 variables:
 NODE_ENV: production
 parameter-store:
 DB_PASSWORD: /app/db-password
 secrets-manager:
 API_KEY: app/api-key

phases:
 install:
 runtime-versions:
 nodejs: 20
 commands:
 - npm ci

 pre_build:
 commands:
 - npm run lint
 - npm run test:unit

 build:

```

```

 commands:
 - npm run build
 - docker build -t $IMAGE_REPO_NAME:$IMAGE_TAG .

 post_build:
 commands:
 - aws ecr get-login-password | docker login --username AWS --password-stdin $ECR_URI
 - docker push $IMAGE_REPO_NAME:$IMAGE_TAG
 - printf '{{"name":"web","imageUri":"%s"}}' $IMAGE_REPO_NAME:$IMAGE_TAG > imagedefinitions.json

 artifacts:
 files:
 - imagedefinitions.json
 - dist/**/*
 discard-paths: no

 cache:
 paths:
 - node_modules/**/*
 - .npm/**/*

```

### 98.3.2 CodeBuild features

| Feature       | Purpose                               |
|---------------|---------------------------------------|
| Custom images | Docker-based build environments       |
| VPC support   | Access private resources during build |
| Caching       | S3 or local cache for dependencies    |
| Reports       | Test and coverage reports             |
| Batch builds  | Matrix builds across configurations   |
| Arm builds    | Graviton-based build environments     |

```

resource "aws_codebuild_project" "app" {
 name = "app-build"
 service_role = aws_iam_role.codebuild.arn
 build_timeout = 15

 artifacts { type = "CODEPIPELINE" }
 source {
 type = "CODEPIPELINE"
 buildspec = "buildspec.yml"
 }

 environment {
 compute_type = "BUILD_GENERAL1_MEDIUM"
 image = "aws/codebuild/amazonlinux2-x86_64-standard:5.0"
 type = "LINUX_CONTAINER"
 privileged_mode = true # required for Docker builds
 image_pull_credentials_type = "CODEBUILD"

 environment_variable {
 name = "ECR_URI"
 value = aws_ecr_repository.app.repository_url
 }
 }

 cache {
 type = "S3"
 location = "${aws_s3_bucket.cache.bucket}/codebuild-cache"
 }
}

```

## 98.4 33.4 CodeDeploy

**CodeDeploy** automates application deployments to various compute platforms:

|  |  |
|--|--|
|  |  |
|--|--|

| Target                   | Deployment type                      |
|--------------------------|--------------------------------------|
| <b>EC2 / On-premises</b> | In-place or blue/green               |
| <b>ECS</b>               | Blue/green with ALB traffic shifting |
| <b>Lambda</b>            | Canary or linear traffic shifting    |

### 98.4.1 ECS blue/green deployment

ALB Target Group A (blue) ← current production  
ALB Target Group B (green) ← new version

1. Deploy new task definition to green target group
2. Shift traffic: 10% → 50% → 100% to green
3. Monitor CloudWatch alarms
4. Auto-rollback if alarms trigger
5. Terminate blue tasks

### 98.4.2 appspec.yml (ECS)

```
version: 0.0
Resources:
 - TargetService:
 Type: AWS::ECS::Service
 Properties:
 TaskDefinition: arn:aws:ecs:us-east-1:123456789012:task-definition/web:5
 LoadBalancerInfo:
 ContainerName: web
 ContainerPort: 8080
 PlatformVersion: LATEST
 NetworkConfiguration:
 AwsVpcConfiguration:
 Subnets: [subnet-aaa, subnet-bbb]
 SecurityGroups: [sg-ecs]
 AssignPublicIp: DISABLED
```

### 98.4.3 Lambda canary deployment

```
version: 0.0
Resources:
 - MyFunction:
 Type: AWS::Lambda::Function
 Properties:
 Name: api-handler
 Alias: live
 CurrentVersion: "3"
 TargetVersion: "4"
```

Traffic shifting: 10% every 5 minutes to the new version, with automatic rollback on alarm.

## 98.5 33.5 Deployment strategies comparison

| Strategy             | Downtime        | Rollback speed    | Complexity |
|----------------------|-----------------|-------------------|------------|
| <b>In-place</b>      | Brief (rolling) | Slow (redploy)    | Low        |
| <b>Blue/green</b>    | None            | Fast (switch ALB) | Medium     |
| <b>Canary</b>        | None            | Fast (shift back) | Medium     |
| <b>Linear</b>        | None            | Fast              | Medium     |
| <b>Rolling (ECS)</b> | None            | Moderate          | Low        |

## 98.6 33.6 GitHub Actions alternative

Many teams use GitHub Actions with OIDC federation instead of CodePipeline:

```
.github/workflows/deploy.yml
name: Deploy
on:
 push:
 branches: [main]

permissions:
 id-token: write
 contents: read

jobs:
 deploy:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: aws-actions/configure-aws-credentials@v4
 with:
 role-to-assume: arn:aws:iam::123456789012:role/GitHubActionsRole
 aws-region: us-east-1
 - run: npm ci && npm test && npm run build
 - run: |
 aws ecr get-login-password | docker login --username AWS --password-stdin $ECR_URI
 docker build -t $ECR_URI:$GITHUB_SHA .
 docker push $ECR_URI:$GITHUB_SHA
 aws ecs update-service --cluster app --service web --force-new-deployment
```

## 98.7 33.7 Security best practices

| Practice                                   | Implementation                             |
|--------------------------------------------|--------------------------------------------|
| Least-privilege IAM roles                  | Separate roles for pipeline, build, deploy |
| Encrypted artifacts                        | S3 SSE-KMS on artifact bucket              |
| No long-term credentials                   | OIDC federation for GitHub/GitLab          |
| Secrets in Parameter Store/Secrets Manager | Never in buildspec or code                 |
| VPC for builds accessing private resources | CodeBuild VPC configuration                |
| Manual approval for production             | CodePipeline approval stage                |

## 98.8 33.8 Chapter summary

- **CodePipeline** orchestrates CI/CD from source to deployment.
- **CodeBuild** compiles, tests, and packages applications with `buildspec.yml`.
- **CodeDeploy** automates deployments to EC2, ECS, and Lambda with blue/green and canary strategies.
- Use **OIDC federation** instead of long-term credentials for external Git providers.
- Choose deployment strategy based on downtime tolerance and rollback requirements.

## 98.9 Lab 33.1 — CodePipeline for ECS

1. Create a CodeBuild project with a Docker buildspec.
2. Create a CodePipeline: GitHub source → CodeBuild → ECS deploy.
3. Push a code change and watch the pipeline execute.
4. Verify the new container version is running in ECS.

## 98.10 Lab 33.2 — Blue/green ECS deployment

1. Configure CodeDeploy for ECS blue/green with ALB.
  2. Deploy a new task definition version.
  3. Observe traffic shifting from blue to green target group.
  4. Trigger a CloudWatch alarm and verify automatic rollback.
- 

## 98.11 Review questions

1. What is the role of each service in the CodePipeline → CodeBuild → CodeDeploy flow?
  2. What is the purpose of `buildspec.yml`?
  3. How does ECS blue/green deployment minimize downtime?
  4. Why should CI/CD use OIDC instead of access keys?
  5. What deployment strategy provides the fastest rollback?
- 

Next: [\*Chapter 34 — Systems Manager & Parameter Store\*](#)

# 99 Chapter 34: Systems Manager & Parameter Store

AWS Handbook — Part VIII, Pages 686–700

## 99.1 34.1 AWS Systems Manager overview

**AWS Systems Manager** is a unified operations hub for managing AWS and hybrid resources. It eliminates the need for SSH bastion hosts, manual patching, and ad-hoc scripting by providing centralized visibility, automation, and configuration management.

Key capabilities include **Session Manager** (browser-based shell access), **Patch Manager**, **Run Command**, **State Manager**, and integration with **Parameter Store** and **Secrets Manager**.

## 99.2 34.2 Session Manager

**Session Manager** provides secure, auditable shell access to EC2 instances and on-premises servers without opening inbound SSH ports:

Engineer → AWS Console/CLI → Session Manager → SSM Agent → EC2 instance

### 99.2.1 Advantages over SSH

| Feature         | SSH                    | Session Manager                    |
|-----------------|------------------------|------------------------------------|
| Inbound port 22 | Required               | Not required                       |
| Bastion host    | Often needed           | Not needed                         |
| Key management  | SSH keys               | IAM permissions                    |
| Audit logging   | Manual                 | CloudWatch Logs + S3 automatically |
| Access control  | Security groups + keys | IAM policies                       |

### 99.2.2 Requirements

1. **SSM Agent** installed on the instance (pre-installed on Amazon Linux, Windows AMIs).
2. **IAM instance profile** with `AmazonSSMManagedInstanceCore` policy.
3. **Network path** to SSM endpoints (internet, NAT, or VPC endpoints).

```
Start a session
aws ssm start-session --target i-0abc123

Port forwarding (access RDS through instance)
aws ssm start-session \
 --target i-0abc123 \
 --document-name AWS-StartPortForwardingSessionToRemoteHost \
 --parameters '{"host":["mydb.abc.rds.amazonaws.com"],"portNumber":["5432"],"localPortNumber":["5432"]}'
```

### 99.2.3 Session logging

```
resource "aws_ssm_document" "session_prefs" {
 name = "SSM-SessionManagerRunShell"
 document_type = "Session"
 document_format = "JSON"
 content = jsonencode({
 schemaVersion = "1.0"
 sessionType = "Standard_Stream"
 inputs = {
```

```
s3BucketName = aws_s3_bucket.session_logs.bucket
s3KeyPrefix = "session-logs/"
cloudWatchLogGroupName = aws_cloudwatch_log_group.sessions.name
cloudWatchEncryptionEnabled = true
 }
 })
}
```

## 99.3 34.3 Run Command

Execute commands across fleets without SSH:

```
Run command on tagged instances
aws ssm send-command \
 --document-name "AWS-RunShellScript" \
 --targets "Key=tag:Environment,Values=production" \
 --parameters 'commands=["yum update -y", "systemctl restart httpd"]' \
 --comment "Patch and restart web servers"

Check command status
aws ssm list-command-invocations \
 --command-id abc-123 \
 --details
```

### 99.3.1 Common documents

| Document                | Purpose                    |
|-------------------------|----------------------------|
| AWS-RunShellScript      | Run shell commands (Linux) |
| AWS-RunPowerShellScript | Run PowerShell (Windows)   |
| AWS-UpdateSSMAgent      | Update SSM Agent           |
| AWS-ConfigureAWSPackage | Install SSM packages       |

## 99.4 34.4 Patch Manager

Automate OS patching across your fleet:

```
Maintenance Window → Patch Baseline → Scan → Install → Reboot (if needed)
```

```
resource "aws_ssm_patch_baseline" "amazon_linux" {
 name = "amazon-linux-baseline"
 operating_system = "AMAZON_LINUX_2"

 approval_rule {
 approve_after_days = 7
 patch_filter {
 key = "CLASSIFICATION"
 values = ["Security", "Critical"]
 }
 }
}

resource "aws_ssm_maintenance_window" "patch" {
 name = "patch-window"
 schedule = "cron(0 2 ? * SUN *)" # Sundays at 2 AM
 duration = 4
 cutoff = 1
}
```



# 99.5 34.5 State Manager

**State Manager** maintains desired configuration state on instances (similar to Ansible/Chef):

```
Association: ensure CloudWatch agent is installed
schemaVersion: "2.2"
description: Install and configure CloudWatch agent
mainSteps:
 - action: aws:configurePackage
 name: installCWAgent
 inputs:
 action: Install
 name: AmazonCloudWatchAgent
 - action: aws:runCommand
 name: configureCWAgent
 inputs:
 documentType: SSMDocument
 documentPath: AmazonCloudWatch-ManageAgent
 parameters:
 action: configure
 mode: ec2
```

Associations run on a schedule or at instance launch, ensuring compliance.

# 99.6 34.6 Parameter Store

**Systems Manager Parameter Store** provides hierarchical storage for configuration data and secrets:

| Type         | Cost                     | Use case                      |
|--------------|--------------------------|-------------------------------|
| String       | Free                     | Configuration values, AMI IDs |
| StringList   | Free                     | Comma-separated values        |
| SecureString | Free (KMS charges apply) | Passwords, API keys, tokens   |

## 99.6.1 Hierarchy

```
/app/prod/database/host
/app/prod/database/port
/app/prod/database/password (SecureString)
/app/dev/database/host
/shared/ami/amazon-linux-2023
```

## 99.6.2 Creating parameters

```
aws ssm put-parameter \
 --name "/app/prod/database/host" \
 --value "mydb.abc.us-east-1.rds.amazonaws.com" \
 --type String \
 --tags "Key=Environment,Value=prod"

aws ssm put-parameter \
 --name "/app/prod/database/password" \
 --value "super-secret" \
 --type SecureString \
 --key-id alias/app-secrets
```

## 99.6.3 Reading parameters

```
Single parameter
aws ssm get-parameter \
 --name "/app/prod/database/password" \
 --with-decryption

By path (all under /app/prod/)
```

```
aws ssm get-parameters-by-path \
 --path "/app/prod/" \
 --recursive \
 --with-decryption
```

## 99.6.4 In application code

```
import boto3

ssm = boto3.client("ssm")
response = ssm.get_parameters_by_path(
 Path="/app/prod/",
 Recursive=True,
 WithDecryption=True,
)
config = {p["Name"].split("/")[1]: p["Value"] for p in response["Parameters"]}
```

## 99.6.5 Terraform

```
resource "aws_ssm_parameter" "db_host" {
 name = "/app/${var.environment}/database/host"
 type = "String"
 value = aws_rds_cluster.main.endpoint
}

resource "aws_ssm_parameter" "db_password" {
 name = "/app/${var.environment}/database/password"
 type = "SecureString"
 key_id = aws_kms_key.app.id
 value = var.db_password
}
```

## 99.7 34.7 Parameter Store vs Secrets Manager

| Feature       | Parameter Store                       | Secrets Manager                              |
|---------------|---------------------------------------|----------------------------------------------|
| Cost          | Free (Standard); KMS for SecureString | \$0.40/secret/month + API calls              |
| Rotation      | Manual or Lambda                      | Built-in rotation for RDS                    |
| Cross-account | Via RAM                               | Via resource policy                          |
| Max size      | 4 KB (advanced: 8 KB)                 | 64 KB                                        |
| Versioning    | Yes                                   | Yes                                          |
| Use case      | Config, non-rotating secrets          | Database credentials, API keys with rotation |

**Guideline:** Use Parameter Store for configuration and non-rotating secrets. Use Secrets Manager for credentials requiring automatic rotation.

## 99.8 34.8 Inventory and Compliance

| Feature    | Purpose                                                     |
|------------|-------------------------------------------------------------|
| Inventory  | Collect OS, application, and custom metadata from instances |
| Compliance | Track patch compliance against baselines                    |
| OpsCenter  | Investigate and remediate operational issues                |

```
aws ssm get-inventory \
```

```
--filters '[{"Key":"AWS:InstanceInformation.PlatformName","Values":["Amazon Linux"],"Type":"Equal"}]'
```

## 99.9 34.9 VPC endpoints for SSM

For instances without internet access, create VPC interface endpoints:

| Endpoint                         | Purpose                 |
|----------------------------------|-------------------------|
| com.amazonaws.region.ssm         | Systems Manager API     |
| com.amazonaws.region.ssmmessages | Session Manager         |
| com.amazonaws.region.ec2messages | SSM Agent communication |

```
resource "aws_vpc_endpoint" "ssm" {
 vpc_id = aws_vpc.main.id
 service_name = "com.amazonaws.${var.region}.ssm"
 vpc_endpoint_type = "Interface"
 subnet_ids = aws_subnet.private[*].id
 security_group_ids = [aws_security_group.vpc_endpoints.id]
 private_dns_enabled = true
}
```

## 99.10 34.10 Chapter summary

- **Session Manager** replaces SSH with IAM-controlled, auditable shell access.
- **Run Command** and **Patch Manager** automate fleet operations.
- **Parameter Store** provides hierarchical configuration and secret storage.
- Use **Secrets Manager** for credentials requiring automatic rotation.
- Deploy **VPC endpoints** for SSM in private subnets without internet access.

## 99.11 Lab 34.1 — Session Manager access

1. Launch an EC2 instance with SSM instance profile (no SSH key, no port 22).
2. Connect via Session Manager in the console.
3. Enable session logging to S3 and CloudWatch Logs.
4. Review the session log after disconnecting.

## 99.12 Lab 34.2 — Parameter Store configuration

1. Create a hierarchy of parameters under `/app/dev/`.
2. Write a script that reads all parameters and prints configuration.
3. Update a Lambda function to read database credentials from Parameter Store.
4. Rotate a SecureString parameter and verify the application picks up the new value.

## 99.13 Review questions

1. What are the advantages of Session Manager over traditional SSH?
2. What IAM policy must an EC2 instance have for Session Manager?
3. When should you use Secrets Manager instead of Parameter Store?
4. How does State Manager differ from Run Command?
5. Which VPC endpoints are required for Session Manager in a private subnet?

---

Next: [Chapter 35 — CloudWatch Fundamentals](#)

# 100 Chapter 35: CloudWatch — Metrics, Alarms, and Dashboards

AWS Handbook — Part IX, Pages 701–720

## 100.1 35.1 Amazon CloudWatch overview

**CloudWatch** is AWS’s observability service for **metrics**, **logs**, **alarms**, and **dashboards**. Nearly every AWS service emits CloudWatch metrics automatically.

## 100.2 35.2 Metrics

A **metric** is a time-series datapoint (e.g., `CPUUtilization` for EC2).

| Concept     | Description                                                |
|-------------|------------------------------------------------------------|
| Namespace   | Grouping (AWS/EC2, AWS/S3, custom)                         |
| Metric name | e.g., <code>CPUUtilization</code>                          |
| Dimensions  | <code>InstanceId</code> , <code>FunctionName</code> , etc. |
| Resolution  | Standard 1 min; high-resolution down to 1 sec              |

### 100.2.1 View metrics

```
aws cloudwatch list-metrics --namespace AWS/EC2
aws cloudwatch get-metric-statistics \
 --namespace AWS/EC2 \
 --metric-name CPUUtilization \
 --dimensions Name=InstanceId,Value=i-0abc123 \
 --start-time 2026-09-01T00:00:00Z \
 --end-time 2026-09-01T12:00:00Z \
 --period 300 \
 --statistics Average
```

### 100.2.2 Custom metrics

```
import boto3
cloudwatch = boto3.client('cloudwatch')
cloudwatch.put_metric_data(
 Namespace='MyApp/Orders',
 MetricData=[{
 'MetricName': 'OrdersProcessed',
 'Value': 42,
 'Unit': 'Count'
 }]
)
```

## 100.3 35.3 Alarms

Alarms trigger actions when metrics breach thresholds.

```
aws cloudwatch put-metric-alarm \
 --alarm-name high-cpu-web-server \
 --metric-name CPUUtilization \
```

```
--namespace AWS/EC2 \
--statistic Average \
--period 300 \
--threshold 80 \
--comparison-operator GreaterThanThreshold \
--evaluation-periods 2 \
--alarm-actions arn:aws:sns:us-east-1:123456789012:ops-alerts \
--dimensions Name=InstanceId,Value=i-0abc123
```

### 100.3.1 Alarm states

| State             | Meaning               |
|-------------------|-----------------------|
| OK                | Within threshold      |
| ALARM             | Breached threshold    |
| INSUFFICIENT_DATA | Not enough datapoints |

### 100.3.2 Composite alarms

Combine multiple alarms with AND/OR logic to reduce alert noise.

## 100.4 35.4 Dashboards

Visualize metrics in the console or via API. Example Terraform:

```
resource "aws_cloudwatch_dashboard" "main" {
 dashboard_name = "production-overview"
 dashboard_body = jsonencode({
 widgets = [{
 type = "metric"
 width = 12
 height = 6
 properties = {
 metrics = [
 ["AWS/EC2", "CPUUtilization", "InstanceId", "i-0abc123"],
 [".", "NetworkIn", ".", "."]
]
 period = 300
 stat = "Average"
 region = "us-east-1"
 title = "EC2 Web Server"
 }
 }]
 })
}
```

## 100.5 35.5 CloudWatch Logs (overview)

| Feature              | Purpose                               |
|----------------------|---------------------------------------|
| Log groups / streams | Organize application logs             |
| Logs Insights        | SQL-like query language               |
| Subscription filters | Stream to Lambda, Kinesis, OpenSearch |
| Metric filters       | Turn log patterns into metrics        |

```
aws logs tail /aws/lambda/my-function --follow
```

## 100.6 35.6 Observability best practices on AWS

1. **Alarm on symptoms** — Error rate, latency p99, not just CPU
  2. **SNS → PagerDuty/Slack** for on-call routing
  3. **Dashboards per service** + one executive overview
  4. **Log retention** — Set retention policies (cost control)
  5. **X-Ray** for distributed tracing (complements CloudWatch)
  6. Use **Container Insights** / **Lambda Insights** for managed runtimes
- 

## 100.7 35.7 Chapter summary

- CloudWatch collects **metrics** from AWS and custom apps.
  - **Alarms** notify via SNS when thresholds breach.
  - **Dashboards** and **Logs** complete the operational picture.
- 

## 100.8 Lab 35.1

1. Create SNS topic `ops-alerts` (email subscription).
  2. Create CPU alarm on an EC2 instance.
  3. Build a dashboard with EC2 CPU, ALB request count, and RDS connections.
  4. Run a Logs Insights query on a Lambda log group.
- 

*Continue: Chapter 36 — CloudWatch Logs Deep Dive (outline in TOC)*

# 101 Chapter 36: CloudWatch Logs & Insights

AWS Handbook — Part IX, Pages 721–735

## 101.1 36.1 CloudWatch Logs overview

**Amazon CloudWatch Logs** centralizes log data from AWS services, applications, and on-premises servers. Every Lambda function, API Gateway stage, ECS task, and VPC Flow Log can stream to CloudWatch Logs for search, analysis, alerting, and long-term retention.

Effective log management is the foundation of operational troubleshooting and security investigation.

## 101.2 36.2 Log groups and streams

| Concept    | Description                                                                     |
|------------|---------------------------------------------------------------------------------|
| Log group  | Container for related log streams (e.g., <code>/aws/lambda/api-handler</code> ) |
| Log stream | Sequence of events from a single source (e.g., one Lambda instance)             |
| Log event  | Single log entry with timestamp and message                                     |
| Retention  | 1 day to 10 years (default: never expire)                                       |

### 101.2.1 Common log group paths

| Service       | Log group pattern                               |
|---------------|-------------------------------------------------|
| Lambda        | <code>/aws/lambda/&lt;function-name&gt;</code>  |
| API Gateway   | <code>/aws/apigateway/&lt;api-name&gt;</code>   |
| ECS           | <code>/ecs/&lt;task-definition&gt;</code>       |
| RDS           | <code>/aws/rds/instance/&lt;id&gt;/error</code> |
| VPC Flow Logs | Custom name                                     |
| CloudTrail    | Custom name                                     |

```
Set retention policy
aws logs put-retention-policy \
 --log-group-name /aws/lambda/api-handler \
 --retention-in-days 30

Create log group
aws logs create-log-group --log-group-name /app/production
```

## 101.3 36.3 Ingesting logs

### 101.3.1 AWS service integration

Most AWS services send logs automatically when enabled:

```
resource "aws_cloudwatch_log_group" "lambda" {
 name = "/aws/lambda/api-handler"
 retention_in_days = 30
}
```



```

kms_key_id = aws_kms_key.logs.arn
}

resource "aws_lambda_function" "api" {
 # ...
 logging_config {
 log_format = "JSON"
 application_log_level = "INFO"
 system_log_level = "WARN"
 }
}

```

### 101.3.2 CloudWatch Agent (EC2/on-premises)

```

{
 "logs": {
 "logs_collected": {
 "files": {
 "collect_list": [
 {
 "file_path": "/var/log/application.log",
 "log_group_name": "/app/production",
 "log_stream_name": "{instance_id}/application",
 "timezone": "UTC"
 }
]
 }
 }
 }
}

```

### 101.3.3 Subscription filters

Stream logs to Lambda, Kinesis, or Firehose for real-time processing:

```

resource "aws_cloudwatch_log_subscription_filter" "errors" {
 name = "error-filter"
 log_group_name = aws_cloudwatch_log_group.app.name
 filter_pattern = "ERROR"
 destination_arn = aws_lambda_function.alert.arn
}

```

## 101.4 36.4 CloudWatch Logs Insights

**Logs Insights** is a purpose-built query language for analyzing log data at scale without provisioning servers.

### 101.4.1 Query syntax

```

fields @timestamp, @message, @logStream
| filter @message like /ERROR/
| sort @timestamp desc
| limit 50

```

### 101.4.2 Common queries

#### Error rate by function (Lambda):

```

fields @timestamp, @message
| filter @message like /ERROR|Exception|error/
| stats count() as errorCount by bin(5m)

```

#### Slow API requests:

```

fields @timestamp, @message
| filter @type = "REPORT"

```

```
| filter @duration > 1000
| sort @duration desc
| limit 20
```

### Top IP addresses (VPC Flow Logs):

```
fields srcAddr, dstAddr, bytes
| filter action = "ACCEPT"
| stats sum(bytes) as totalBytes by srcAddr
| sort totalBytes desc
| limit 10
```

### Parse structured JSON logs:

```
fields @timestamp, level, message, userId, duration
| filter level = "ERROR"
| sort @timestamp desc
| limit 100
```

### 101.4.3 CLI query

```
aws logs start-query \
 --log-group-names /aws/lambda/api-handler \
 --start-time $(date -d '1 hour ago' +%s) \
 --end-time $(date +%s) \
 --query-string 'fields @timestamp, @message | filter @message like /ERROR/ | sort @timestamp desc | limit 20'
```

## 101.5 36.5 Metric filters

Convert log patterns into CloudWatch metrics for alarming:

```
resource "aws_cloudwatch_log_metric_filter" "errors" {
 name = "api-errors"
 log_group_name = aws_cloudwatch_log_group.api.name
 pattern = "ERROR"

 metric_transformation {
 name = "ApiErrorCount"
 namespace = "App/Errors"
 value = "1"
 default_value = "0"
 }
}

resource "aws_cloudwatch_metric_alarm" "error_rate" {
 alarm_name = "high-api-errors"
 comparison_operator = "GreaterThanThreshold"
 evaluation_periods = 2
 metric_name = "ApiErrorCount"
 namespace = "App/Errors"
 period = 300
 statistic = "Sum"
 threshold = 10
 alarm_actions = [aws_sns_topic.alerts.arn]
}
```

## 101.6 36.6 Log encryption and access control

| Control               | Implementation        |
|-----------------------|-----------------------|
| Encryption at rest    | KMS key on log group  |
| Encryption in transit | TLS for all API calls |

|                   |                                                               |
|-------------------|---------------------------------------------------------------|
| Resource policies | Cross-account and service access                              |
| IAM policies      | logs:CreateLogStream, logs:PutLogEvents, logs:FilterLogEvents |

### 101.6.1 Cross-account log sharing

```
{
 "Version": "2012-10-17",
 "Statement": [{
 "Effect": "Allow",
 "Principal": { "AWS": "arn:aws:iam::AUDIT_ACCOUNT:root" },
 "Action": ["logs:CreateLogStream", "logs:PutLogEvents"],
 "Resource": "arn:aws:logs:us-east-1:123456789012:log-group:/security/*"
 }]
}
```

## 101.7 36.7 Log export and archival

| Method              | Destination                      | Use case                            |
|---------------------|----------------------------------|-------------------------------------|
| Export to S3        | S3 bucket                        | Long-term archival, Athena analysis |
| Subscription filter | Kinesis Firehose → S3/OpenSearch | Real-time streaming                 |
| Logs Insights       | Query results                    | Ad-hoc analysis                     |

```
aws logs create-export-task \
 --log-group-name /aws/lambda/api-handler \
 --from $(date -d '7 days ago' +%s000) \
 --to $(date +%s000) \
 --destination my-log-archive-bucket \
 --destination-prefix lambda-logs/
```

## 101.8 36.8 Best practices

| Practice                               | Rationale                                    |
|----------------------------------------|----------------------------------------------|
| Set retention policies                 | Control costs; 30-90 days for most workloads |
| Use structured JSON logging            | Enables field-based queries in Insights      |
| Include correlation IDs                | Trace requests across services               |
| Create metric filters for errors       | Proactive alarming                           |
| Encrypt sensitive log groups           | KMS for compliance                           |
| Avoid logging secrets/PII              | Redact or mask sensitive data                |
| Use log levels appropriately           | DEBUG in dev, INFO/WARN in prod              |
| Centralize logs in a dedicated account | Security team access, tamper resistance      |

### 101.8.1 Structured logging example

```
import json, logging

logger = logging.getLogger()
logger.setLevel(logging.INFO)

def handler(event, context):
 logger.info(json.dumps({
 "level": "INFO",
 "message": "Processing order",
```

```
"orderId": event["orderId"],
"requestId": context.aws_request_id,
"duration_ms": None,
}))
```

---

## 101.9 36.9 Chapter summary

- **CloudWatch Logs** centralizes log data from AWS services and applications.
  - **Logs Insights** provides a powerful query language for log analysis.
  - **Metric filters** convert log patterns into alarmable CloudWatch metrics.
  - Use **structured JSON logging** and **retention policies** for effective operations.
  - **Subscription filters** enable real-time log processing and alerting.
- 

## 101.10 Lab 36.1 — Logs Insights queries

1. Deploy a Lambda function that logs structured JSON (level, message, requestId).
2. Invoke it multiple times with success and error paths.
3. Write Insights queries to find errors, calculate average duration, and count by log level.

## 101.11 Lab 36.2 — Error alerting pipeline

1. Create a metric filter for ERROR log entries.
  2. Create a CloudWatch alarm on the metric (threshold > 5 in 5 minutes).
  3. Connect the alarm to an SNS topic.
  4. Trigger errors and verify the alert fires.
- 

## 101.12 Review questions

1. What is the difference between a log group and a log stream?
  2. How do metric filters bridge logs and alarms?
  3. Why is structured JSON logging recommended over plain text?
  4. What is a subscription filter used for?
  5. How do you control CloudWatch Logs costs?
- 

Next: [Chapter 37 — X-Ray Distributed Tracing](#)

# 102 Chapter 37: X-Ray & Distributed Tracing

AWS Handbook — Part IX, Pages 736–750

## 102.1 37.1 Why distributed tracing?

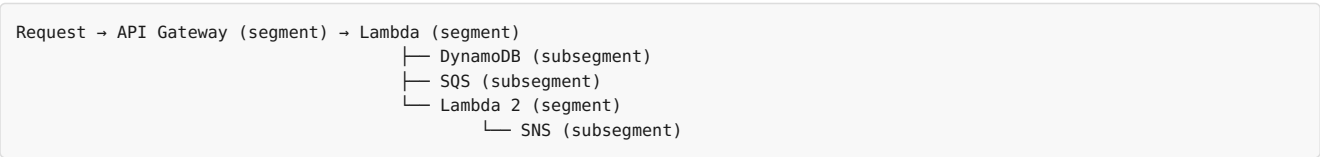
In microservice architectures, a single user request traverses multiple services: API Gateway → Lambda → DynamoDB → SQS → another Lambda → SNS. When latency spikes or errors occur, traditional logging cannot show the full request path.

**AWS X-Ray** provides end-to-end tracing, showing the complete journey of a request with timing, errors, and dependencies visualized as a service map.

## 102.2 37.2 X-Ray concepts

| Concept     | Description                                                            |
|-------------|------------------------------------------------------------------------|
| Trace       | Complete path of a request through services                            |
| Segment     | Work done by a single service (e.g., Lambda execution)                 |
| Subsegment  | Downstream call within a segment (e.g., DynamoDB query)                |
| Annotation  | Indexed key-value pairs for filtering (e.g., <code>userId=123</code> ) |
| Metadata    | Non-indexed data attached to segments                                  |
| Service map | Visual graph of service dependencies                                   |
| Trace ID    | Unique identifier propagated across services                           |

## 102.3 37.3 How X-Ray works



1. First service generates a **trace ID** and **segment**.
2. Trace ID propagates via HTTP header ( `X-Amzn-Trace-Id` ) or message attribute.
3. Each downstream service creates its own segment linked to the trace.
4. Segments are sent to the X-Ray daemon (or SDK) and stored by the X-Ray service.

## 102.4 37.4 Enabling X-Ray

### 102.4.1 AWS service integration

| Service     | Enable method                                   |
|-------------|-------------------------------------------------|
| Lambda      | <code>tracing_config { mode = "Active" }</code> |
| API Gateway | Stage tracing enabled                           |

|                          |                                                       |
|--------------------------|-------------------------------------------------------|
| <b>ECS</b>               | X-Ray daemon sidecar container                        |
| <b>EKS</b>               | ADOT (AWS Distro for OpenTelemetry) collector         |
| <b>Elastic Beanstalk</b> | Enable in environment config                          |
| <b>Step Functions</b>    | <code>tracing_configuration { enabled = true }</code> |

### 102.4.2 Lambda with active tracing

```
resource "aws_lambda_function" "api" {
 # ...
 tracing_config {
 mode = "Active"
 }
}

resource "aws_iam_role_policy" "xray" {
 role = aws_iam_role.lambda.id
 policy = jsonencode({
 Statement = [{
 Effect = "Allow"
 Action = ["xray:PutTraceSegments", "xray:PutTelemetryRecords"]
 Resource = "*"
 }]
 })
}
```

### 102.4.3 API Gateway

```
resource "aws_apigatewayv2_stage" "prod" {
 # ...
 default_route_settings {
 detailed_metrics_enabled = true
 throttling_burst_limit = 100
 throttling_rate_limit = 50
 }
}
```

## 102.5 37.5 SDK instrumentation

For custom applications, use the X-Ray SDK:

### 102.5.1 Python

```
from aws_xray_sdk.core import xray_recorder, patch_all
from aws_xray_sdk.core import lambda_launcher

patch_all() # auto-instrument boto3, requests, etc.

@xray_recorder.capture("process_order")
def process_order(order_id):
 xray_recorder.put_annotation("orderId", order_id)
 xray_recorder.put_metadata("order", {"id": order_id, "source": "api"})

 # Subsegment for DynamoDB call (auto-patched by patch_all)
 table.put_item(Item={"orderId": order_id, "status": "processing"})

def handler(event, context):
 process_order(event["orderId"])
 return {"statusCode": 200}
```

### 102.5.2 Node.js

```
const AWSXRay = require("aws-xray-sdk");
const AWS = AWSXRay.captureAWS(require("aws-sdk"));
```

```

exports.handler = async (event) => {
 const segment = AWSXRay.getSegment();
 segment.addAnnotation("orderId", event.orderId);

 const subsegment = segment.addNewSubsegment("validateOrder");
 try {
 // business logic
 subsegment.close();
 } catch (err) {
 subsegment.addError(err);
 subsegment.close();
 throw err;
 }
};

```

## 102.6 37.6 Service map

The **service map** visualizes dependencies and health:

```

[Client] → [API Gateway] → [Lambda: api-handler] → [DynamoDB: Orders]
 ↓ (error rate 2%)
 [SQS: orders-queue] → [Lambda: processor]

```

- **Green** edges: healthy (< 5% error rate).
- **Red/yellow** edges: elevated errors or latency.
- **Node size**: proportional to request volume.

Use the service map to identify bottlenecks, error hotspots, and unexpected dependencies.

## 102.7 37.7 Trace analysis

### 102.7.1 Filtering traces

| Filter           | Example                                 |
|------------------|-----------------------------------------|
| By annotation    | annotation.orderId = "12345"            |
| By response time | responsetime > 3                        |
| By error         | error = true                            |
| By service       | service("api-handler")                  |
| Combined         | service("api-handler") { error = true } |

### 102.7.2 Trace detail view

Each trace shows: - **Timeline** — waterfall of segments with duration. - **Segments** — per-service breakdown with metadata. - **Exceptions** — stack traces for errors. - **Annotations** — searchable key-value pairs.

```

aws xray get-trace-summaries \
 --start-time $(date -d '1 hour ago' +%s) \
 --end-time $(date +%s) \
 --filter-expression 'service("api-handler") { error = true }'

```

## 102.8 37.8 X-Ray with OpenTelemetry

AWS supports **OpenTelemetry (OTel)** as the modern standard for instrumentation:

| Component | AWS implementation                    |
|-----------|---------------------------------------|
| SDK       | OpenTelemetry SDK (language-specific) |
| Collector | ADOT (AWS Distro for OpenTelemetry)   |
| Backend   | X-Ray, CloudWatch, Prometheus, Jaeger |

ADOT collector on ECS/EKS/EC2 receives OTLP traces and exports to X-Ray:

```
receivers:
 otlp:
 protocols:
 grpc:
 endpoint: 0.0.0.0:4317

exporters:
 awsxray:
 region: us-east-1

service:
 pipelines:
 traces:
 receivers: [otlp]
 exporters: [awsxray]
```

# 102.9 37.9 Sampling

X-Ray uses sampling to control costs—by default, 1 request/second plus 5% of additional requests are traced.

## 102.9.1 Custom sampling rules

```
{
 "version": 2,
 "rules": [{
 "description": "Trace all errors",
 "host": "*",
 "http_method": "*",
 "url_path": "*",
 "fixed_target": 0,
 "rate": 1.0,
 "priority": 100,
 "service_name": "*",
 "service_type": "*",
 "resource_arn": "*"
 }, {
 "description": "Sample 10% of API calls",
 "host": "*",
 "http_method": "*",
 "url_path": "/api/*",
 "fixed_target": 1,
 "rate": 0.1,
 "priority": 200,
 "service_name": "api-handler",
 "service_type": "AWS::Lambda::Function",
 "resource_arn": "*"
 }]
}
```

# 102.10 37.10 X-Ray groups and insights

| Feature  | Purpose                                                  |
|----------|----------------------------------------------------------|
| Groups   | Filter traces by criteria; track error rates and latency |
| Insights | Automatically detect anomalies in trace data             |



---

## 102.11 37.11 Chapter summary

- **X-Ray** provides end-to-end distributed tracing across AWS services and custom applications.
  - **Segments** and **subsegments** capture timing and errors at each service boundary.
  - Enable active tracing on Lambda, API Gateway, and ECS; use SDKs for custom instrumentation.
  - The **service map** visualizes dependencies and identifies bottlenecks.
  - Use **sampling rules** to balance observability with cost; consider **OpenTelemetry** for modern instrumentation.
- 

## 102.12 Lab 37.1 — Lambda tracing

1. Enable active X-Ray tracing on a Lambda function behind API Gateway.
2. Invoke the API with success and error scenarios.
3. View the service map and trace timeline in the X-Ray console.
4. Filter traces by error and identify the failing subsegment.

## 102.13 Lab 37.2 — Custom annotations

1. Add X-Ray SDK annotations ( `userId` , `orderId` ) to a Lambda function.
  2. Generate traffic with different user IDs.
  3. Filter traces by annotation in the X-Ray console.
  4. Create an X-Ray group for high-latency traces (> 2 seconds).
- 

## 102.14 Review questions

1. What is the difference between a segment and a subsegment?
  2. How does the trace ID propagate between services?
  3. Why is sampling important for X-Ray cost management?
  4. What does the X-Ray service map show?
  5. How does OpenTelemetry relate to X-Ray?
- 

Next: [Chapter 38 — CloudTrail & AWS Config](#)

# 103 Chapter 38: CloudTrail & AWS Config

AWS Handbook — Part IX, Pages 751–760

## 103.1 38.1 Audit and compliance foundations

Two services form the backbone of AWS governance and compliance:

| Service    | Purpose                         | Question answered                               |
|------------|---------------------------------|-------------------------------------------------|
| CloudTrail | API activity logging            | Who did what, when, and from where?             |
| AWS Config | Resource configuration tracking | What is the current/desired state of resources? |

Together, they provide the audit trail and configuration baseline required for SOC 2, PCI DSS, HIPAA, and internal security policies.

## 103.2 38.2 AWS CloudTrail

**CloudTrail** records AWS API calls and delivers log files to S3 (and optionally CloudWatch Logs):

### 103.2.1 Event types

| Type              | Description                    | Example                                       |
|-------------------|--------------------------------|-----------------------------------------------|
| Management events | Control plane operations       | CreateVpc, RunInstances, DeleteBucket         |
| Data events       | Data plane operations (opt-in) | S3 GetObject, Lambda Invoke, DynamoDB GetItem |
| Insights events   | Unusual API activity           | Spike in DeleteSecurityGroup calls            |

### 103.2.2 Trail types

| Type               | Scope                                                             |
|--------------------|-------------------------------------------------------------------|
| Organization trail | All accounts in the organization (deploy from management account) |
| Account trail      | Single account                                                    |

**Best practice:** One organization trail in the management account, logging to a centralized S3 bucket in the log archive account.

### 103.2.3 Terraform organization trail

```
resource "aws_cloudtrail" "org" {
 name = "organization-trail"
 s3_bucket_name = aws_s3_bucket.trail.id
 include_global_service_events = true
 is_multi_region_trail = true
 is_organization_trail = true
 enable_log_file_validation = true
 kms_key_id = aws_kms_key.trail.arn

 event_selector {
 read_write_type = "All"
 }
}
```

```

include_management_events = true

data_resource {
 type = "AWS::S3::Object"
 values = ["arn:aws:s3:::sensitive-bucket/"]
}

insight_selector {
 insight_type = "ApiCallRateInsight"
}
}

```

### 103.2.4 CloudTrail log entry structure

```

{
 "eventTime": "2024-06-15T10:30:00Z",
 "eventName": "RunInstances",
 "eventSource": "ec2.amazonaws.com",
 "awsRegion": "us-east-1",
 "sourceIPAddress": "203.0.113.50",
 "userAgent": "aws-cli/2.15.0",
 "userIdentity": {
 "type": "AssumedRole",
 "principalId": "AROAI23:session-name",
 "arn": "arn:aws:sts:123456789012:assumed-role/AdminRole/session-name"
 },
 "requestParameters": {
 "instanceType": "t3.micro",
 "minCount": 1,
 "maxCount": 1
 },
 "responseElements": {
 "instancesSet": { "items": [{ "instanceId": "i-0abc123" }] }
 }
}

```

### 103.2.5 Querying CloudTrail

```

Recent events
aws cloudtrail lookup-events \
 --lookup-attributes AttributeKey=EventName,AttributeValue=DeleteBucket \
 --max-results 10

CloudTrail Lake (SQL queries)
aws cloudtrail create-event-data-store \
 --name security-events \
 --retention-period 365

Athena (S3-based trail logs)
Create Athena table on trail S3 bucket and run SQL queries

```

## 103.3 38.3 CloudTrail security

| Control             | Purpose                                                        |
|---------------------|----------------------------------------------------------------|
| Log file validation | Detect tampering with digest files                             |
| S3 bucket policy    | Restrict access to security team                               |
| KMS encryption      | Encrypt trail logs at rest                                     |
| S3 Object Lock      | Prevent deletion (WORM compliance)                             |
| Organization trail  | Ensure all accounts are logged (cannot be disabled by members) |

### 103.3.1 Protecting the trail bucket

```

{
 "Sid": "DenyTrailDeletion",
 "Effect": "Deny",
 "Principal": "*",
 "Action": ["s3:DeleteObject", "s3:DeleteBucket"],
 "Resource": [
 "arn:aws:s3::org-cloudtrail-logs",
 "arn:aws:s3::org-cloudtrail-logs/*"
],
 "Condition": {
 "StringNotEquals": {
 "aws:PrincipalArn": "arn:aws:iam::LOG_ACCOUNT:role/CloudTrailAdmin"
 }
 }
}

```

## 103.4 38.4 AWS Config

**AWS Config** continuously records resource configuration changes and evaluates compliance against rules:

### 103.4.1 How Config works

```

Resource change → Config records configuration item → Rules evaluate → Compliance status
 ↓
 Non-compliant → SNS/SSM remediation

```

### 103.4.2 Configuration items

Each CI includes: - Resource type, ID, and ARN. - Current configuration (JSON). - Relationships to other resources. - Configuration change timeline.

### 103.4.3 Config rules

| Type                   | Example                                              |
|------------------------|------------------------------------------------------|
| <b>AWS managed</b>     | s3-bucket-public-read-prohibited , encrypted-volumes |
| <b>Custom (Lambda)</b> | Company-specific policies                            |
| <b>Custom (Guard)</b>  | Policy-as-code (Cedar language)                      |

```

resource "aws_config_config_rule" "s3_public_read" {
 name = "s3-bucket-public-read-prohibited"

 source {
 owner = "AWS"
 source_identifier = "S3_BUCKET_PUBLIC_READ_PROHIBITED"
 }
}

resource "aws_config_config_rule" "encrypted_volumes" {
 name = "encrypted-volumes"

 source {
 owner = "AWS"
 source_identifier = "ENCRYPTED_VOLUMES"
 }

 scope {
 compliance_resource_types = ["AWS::EC2::Volume"]
 }
}

```

### 103.4.4 Remediation

Automatically fix non-compliant resources:

```
resource "aws_config_remediation_configuration" "encrypt_volume" {
 config_rule_name = aws_config_config_rule.encrypted_volumes.name
 target_type = "SSM_DOCUMENT"
 target_identifier = "AWS-EnableEBSEncryption"
 automatic = true
 maximum_automatic_attempts = 3

 parameter {
 name = "AutomationAssumeRole"
 static_value = aws_iam_role.remediation.arn
 }
}
```

## 103.5 38.5 Config aggregators

For multi-account environments, **Config aggregators** collect configuration and compliance data from multiple accounts and regions:

```
resource "aws_config_configuration_aggregator" "org" {
 name = "organization-aggregator"

 organization_aggregation_source {
 all_regions = true
 role_arn = aws_iam_role.config_aggregator.arn
 }
}
```

Deploy the aggregator in the security/audit account for centralized compliance dashboards.

## 103.6 38.6 CloudTrail vs Config

| Aspect           | CloudTrail                        | Config                          |
|------------------|-----------------------------------|---------------------------------|
| Records          | API calls (actions)               | Resource configurations (state) |
| Question         | Who deleted the bucket?           | Is the bucket encrypted?        |
| Trigger          | Every API call                    | Configuration change            |
| Retention        | S3 lifecycle policies             | Config retention period         |
| Compliance rules | No (use CloudTrail Lake + Athena) | Yes (managed and custom rules)  |

**Use both:** CloudTrail for “who did it” and Config for “is it compliant.”

## 103.7 38.7 Compliance reporting

| Tool                              | Output                                           |
|-----------------------------------|--------------------------------------------------|
| <b>Config conformance packs</b>   | Pre-built rule collections (PCI, CIS, NIST)      |
| <b>Config compliance timeline</b> | Historical compliance status                     |
| <b>Security Hub</b>               | Aggregates Config, GuardDuty, Inspector findings |
| <b>Athena on CloudTrail</b>       | Custom compliance queries                        |

```
resource "aws_config_conformance_pack" "cis" {
 name = "CIS-AWS-Foundations-Benchmark"
 template_body = file("cis-conformance-pack.yaml")
}
```

---

## 103.8 38.8 Chapter summary

- **CloudTrail** logs all API activity; deploy an **organization trail** to a centralized, protected S3 bucket.
  - **AWS Config** tracks resource configurations and evaluates **compliance rules**.
  - Use **Config aggregators** for multi-account compliance visibility.
  - Enable **log file validation**, **KMS encryption**, and **Object Lock** on trail buckets.
  - Combine CloudTrail (who) and Config (what state) for complete governance.
- 

## 103.9 Lab 38.1 — CloudTrail investigation

1. Enable a multi-region CloudTrail with S3 and CloudWatch Logs delivery.
2. Perform several API actions (create/delete security group).
3. Use `lookup-events` and CloudWatch Logs Insights to find the events.
4. Identify the IAM principal that performed each action.

## 103.10 Lab 38.2 — Config compliance

1. Enable AWS Config with the `s3-bucket-public-read-prohibited` managed rule.
  2. Create a bucket with public read access.
  3. Verify the rule reports non-compliance.
  4. Fix the bucket and confirm compliance is restored.
- 

## 103.11 Review questions

1. What is the difference between CloudTrail management events and data events?
  2. Why should organization trails be deployed from the management account?
  3. How does AWS Config differ from CloudTrail?
  4. What is a Config conformance pack?
  5. How do you protect CloudTrail logs from tampering?
- 

Next: [Chapter 39 — KMS, Secrets Manager, ACM](#)

# 104 Chapter 39: KMS, Secrets Manager, and ACM

AWS Handbook — Part X, Pages 761–775

## 104.1 39.1 Encryption and certificate management

AWS provides three core services for protecting data and communications:

| Service         | Purpose                          |
|-----------------|----------------------------------|
| KMS             | Encryption key management        |
| Secrets Manager | Secret storage and rotation      |
| ACM             | TLS/SSL certificate provisioning |

Understanding when to use each—and how they integrate—is essential for a secure AWS architecture.

## 104.2 39.2 AWS Key Management Service (KMS)

KMS creates and manages encryption keys used to encrypt data across AWS services.

### 104.2.1 Key types

| Type                   | Description                             | Use case                       |
|------------------------|-----------------------------------------|--------------------------------|
| AWS managed            | Created and managed by AWS per service  | Default S3, EBS encryption     |
| Customer managed (CMK) | You create and control                  | Cross-service, custom policies |
| AWS owned              | Shared across accounts; you cannot view | Some default encryption        |
| External (BYOK)        | Import your own key material            | Regulatory requirements        |

### 104.2.2 Key concepts

| Concept             | Description                                           |
|---------------------|-------------------------------------------------------|
| CMK                 | Customer Master Key — never leaves KMS                |
| Data key            | Symmetric key generated by CMK; used to encrypt data  |
| Envelope encryption | Encrypt data with data key; encrypt data key with CMK |
| Key policy          | Resource policy controlling key access                |
| Grant               | Delegated permission to use a key                     |
| Alias               | Friendly name for a key ( alias/app-secrets )         |

### 104.2.3 Creating a CMK

```
aws kms create-key \
 --description "Application encryption key" \
 --key-usage ENCRYPT_DECRYPT \
 --origin AWS_KMS

aws kms create-alias \
 --alias-name alias/app-encryption \
 --target-key-id <key-id>
```

### 104.2.4 Terraform

```
resource "aws_kms_key" "app" {
 description = "Application encryption key"
 deletion_window_in_days = 30
 enable_key_rotation = true

 policy = jsonencode({
 Version = "2012-10-17"
 Statement = [
 {
 Sid = "Enable IAM policies"
 Effect = "Allow"
 Principal = { AWS = "arn:aws:iam::123456789012:root" }
 Action = "kms:*"
 Resource = "*"
 },
 {
 Sid = "Allow service use"
 Effect = "Allow"
 Principal = { Service = ["s3.amazonaws.com", "rds.amazonaws.com"] }
 Action = ["kms:Encrypt", "kms:Decrypt", "kms:GenerateDataKey*"]
 Resource = "*"
 }
]
 })
}

resource "aws_kms_alias" "app" {
 name = "alias/app-encryption"
 target_key_id = aws_kms_key.app.key_id
}
```

### 104.2.5 Envelope encryption

```
import boto3

kms = boto3.client("kms")

Generate a data key
response = kms.generate_data_key(KeyId="alias/app-encryption", KeySpec="AES_256")
plaintext_key = response["Plaintext"] # use to encrypt data
encrypted_key = response["CiphertextBlob"] # store alongside encrypted data

Decrypt the data key when needed
decrypted = kms.decrypt(CiphertextBlob=encrypted_key)
plaintext_key = decrypted["Plaintext"]
```

### 104.2.6 Multi-Region Keys (MRK)

Replicate keys across regions for DR without re-encrypting data:

```
Primary key (us-east-1) ↔ Replica key (eu-west-1)
Same key ID, same key material, regional endpoints
```

## 104.3 39.3 AWS Secrets Manager

**Secrets Manager** stores and rotates secrets (database credentials, API keys, tokens):

### 104.3.1 Features

| Feature              | Description                         |
|----------------------|-------------------------------------|
| Automatic rotation   | Lambda-based rotation on a schedule |
| Cross-account access | Resource policies for sharing       |



|                    |                                                    |
|--------------------|----------------------------------------------------|
| <b>Versioning</b>  | AWSCURRENT and AWSPENDING versions during rotation |
| <b>Integration</b> | RDS, Redshift, DocumentDB native integration       |

### 104.3.2 Creating a secret

```
aws secretsmanager create-secret \
 --name app/prod/database \
 --description "Production database credentials" \
 --secret-string '{"username":"admin","password":"initial-password","host":"mydb.abc.rds.amazonaws.com","port":5432}'

Enable rotation (every 30 days)
aws secretsmanager rotate-secret \
 --secret-id app/prod/database \
 --rotation-lambda-arn arn:aws:lambda:us-east-1:123456789012:function:SecretsManagerRDSPostgreSQLRotation \
 --rotation-rules AutomaticallyAfterDays=30
```

### 104.3.3 Terraform

```
resource "aws_secretsmanager_secret" "db" {
 name = "app/${var.environment}/database"
 kms_key_id = aws_kms_key.secrets.arn
}

resource "aws_secretsmanager_secret_version" "db" {
 secret_id = aws_secretsmanager_secret.db.id
 secret_string = jsonencode({
 username = "admin"
 password = random_password.db.result
 host = aws_rds_cluster.main.endpoint
 port = 5432
 engine = "postgres"
 })
}

resource "aws_secretsmanager_secret_rotation" "db" {
 secret_id = aws_secretsmanager_secret.db.id
 rotation_lambda_arn = aws_lambda_function.rotation.arn
 rotation_rules {
 automatically_after_days = 30
 }
}
```

### 104.3.4 Retrieving secrets in applications

```
import boto3, json

client = boto3.client("secretsmanager")
response = client.get_secret_value(SecretId="app/prod/database")
secret = json.loads(response["SecretString"])
secret["username"], secret["password"], secret["host"]
```

## 104.4 39.4 AWS Certificate Manager (ACM)

ACM provisions, manages, and renews TLS/SSL certificates for free:

| Feature     | Detail                                                 |
|-------------|--------------------------------------------------------|
| Cost        | Free for use with AWS services                         |
| Validation  | DNS (recommended) or email                             |
| Renewal     | Automatic                                              |
| Integration | ALB, CloudFront, API Gateway, NLB                      |
| Private CA  | ACM Private CA for internal certificates (\$400/month) |

### 104.4.1 Requesting a certificate

```
aws acm request-certificate \
 --domain-name "*.example.com" \
 --subject-alternative-names "example.com" \
 --validation-method DNS
```

### 104.4.2 DNS validation (Terraform)

```
resource "aws_acm_certificate" "app" {
 domain_name = "app.example.com"
 subject_alternative_names = ["*.app.example.com"]
 validation_method = "DNS"

 lifecycle {
 create_before_destroy = true
 }
}

resource "aws_route53_record" "cert_validation" {
 for_each = {
 for dvo in aws_acm_certificate.app.domain_validation_options : dvo.domain_name => {
 name = dvo.resource_record_name
 record = dvo.resource_record_value
 type = dvo.resource_record_type
 }
 }
 zone_id = aws_route53_zone.main.zone_id
 name = each.value.name
 type = each.value.type
 records = [each.value.record]
 ttl = 60
}

resource "aws_acm_certificate_validation" "app" {
 certificate_arn = aws_acm_certificate.app.arn
 validation_record_fqdns = [for record in aws_route53_record.cert_validation : record.fqdn]
}
```

### 104.4.3 Certificate limitations

| Limitation      | Detail                                                          |
|-----------------|-----------------------------------------------------------------|
| <b>Export</b>   | Cannot export private keys (use with AWS services only)         |
| <b>Region</b>   | CloudFront requires certificates in us-east-1                   |
| <b>Wildcard</b> | *.example.com covers one level only                             |
| <b>On EC2</b>   | Cannot install ACM certs directly; use ALB/CloudFront or import |

## 104.5 39.5 Choosing the right service

| Need                                  | Service                                           |
|---------------------------------------|---------------------------------------------------|
| Encrypt EBS, S3, RDS at rest          | KMS                                               |
| Store database password with rotation | Secrets Manager                                   |
| Store API key (no rotation)           | Parameter Store (SecureString) or Secrets Manager |
| TLS for ALB/CloudFront                | ACM                                               |
|                                       |                                                   |

|                              |                         |
|------------------------------|-------------------------|
| Internal mTLS certificates   | ACM Private CA          |
| Application-level encryption | KMS envelope encryption |

---

## 104.6 39.6 Security best practices

| Practice                                     | Rationale                                          |
|----------------------------------------------|----------------------------------------------------|
| Use customer managed CMKs                    | Control key policies, enable rotation, audit usage |
| Enable automatic key rotation                | Annual rotation of CMK material                    |
| Separate keys per environment                | Limit blast radius                                 |
| Use Secrets Manager for rotating credentials | Automatic rotation reduces exposure window         |
| DNS validation for ACM                       | More reliable than email validation                |
| Monitor KMS API calls in CloudTrail          | Detect unauthorized key usage                      |
| Least-privilege key policies                 | Grant only required principals and actions         |

### 104.6.1 CloudTrail KMS events to monitor

- `Decrypt` — unauthorized data access attempts
- `DisableKey` / `ScheduleKeyDeletion` — potential sabotage
- `PutKeyPolicy` — policy tampering

---

## 104.7 39.7 Chapter summary

- **KMS** manages encryption keys; use **customer managed CMKs** with automatic rotation.
- **Secrets Manager** stores and rotates credentials; integrates with RDS and Lambda.
- **ACM** provides free TLS certificates with automatic renewal for AWS services.
- Use **envelope encryption** for application-level data protection.
- Monitor key usage via **CloudTrail** and enforce **least-privilege key policies**.

---

## 104.8 Lab 39.1 — KMS encryption

1. Create a customer managed CMK with automatic rotation.
2. Encrypt a file using envelope encryption (generate data key, encrypt, store).
3. Decrypt and verify the original content.
4. Review CloudTrail for KMS Decrypt events.

## 104.9 Lab 39.2 — Secrets Manager rotation

1. Create an RDS instance with Secrets Manager-managed credentials.
2. Enable automatic rotation (30-day cycle).
3. Retrieve the current secret and connect to the database.
4. Trigger a manual rotation and verify the password changes.

---

## 104.10 Review questions

1. What is envelope encryption and why is it used?
  2. When should you use Secrets Manager vs Parameter Store SecureString?
  3. Why must CloudFront certificates be in us-east-1?
  4. What is the difference between AWS managed and customer managed CMKs?
  5. How does Secrets Manager automatic rotation work?
- 

Next: [\*Chapter 40 — GuardDuty, Security Hub, WAF, Shield\*](#)

# 105 Chapter 40: GuardDuty, Security Hub, WAF, and Shield

AWS Handbook — Part X, Pages 776–790

## 105.1 40.1 AWS security services landscape

AWS provides a layered security stack for threat detection, compliance aggregation, and attack mitigation:

| Layer            | Service          | Function                                |
|------------------|------------------|-----------------------------------------|
| Detection        | GuardDuty        | ML-based threat detection               |
| Aggregation      | Security Hub     | Centralized findings dashboard          |
| Prevention       | WAF              | Application-layer firewall              |
| DDoS protection  | Shield           | Network/transport layer DDoS mitigation |
| Vulnerability    | Inspector        | Software vulnerability scanning         |
| Network firewall | Network Firewall | VPC-level stateful firewall             |

This chapter focuses on the four most critical services for production workloads.

## 105.2 40.2 Amazon GuardDuty

**GuardDuty** is a threat detection service that continuously monitors for malicious activity using machine learning, anomaly detection, and threat intelligence feeds.

### 105.2.1 Data sources

| Source             | Detects                                      |
|--------------------|----------------------------------------------|
| CloudTrail         | Unusual API calls, credential exfiltration   |
| VPC Flow Logs      | Malicious IPs, anomalous traffic patterns    |
| DNS Logs           | Compromised instances, crypto-mining domains |
| S3 Data Events     | Unusual data access patterns                 |
| EKS Audit Logs     | Kubernetes-specific threats                  |
| Malware Protection | Malicious files on EC2 and EBS               |

### 105.2.2 Finding types (examples)

| Severity | Finding                                                   | Description                             |
|----------|-----------------------------------------------------------|-----------------------------------------|
| High     | UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration | Credentials used from unusual IP        |
| High     | CryptoCurrency:EC2/BitcoinTool                            | Instance communicating with mining pool |
| Medium   | Recon:EC2/PortProbeUnprotectedPort                        | Port scanning detected                  |
| Low      | Policy:IAMUser/RootCredentialUsage                        | Root account API usage                  |

### 105.2.3 Enabling GuardDuty

```
Enable in a single account
aws guardduty create-detector --enable

Organization-wide (from delegated admin account)
aws guardduty enable-organization-admin-account \
 --admin-account-id 111111111111

aws organizations enable-aws-service-access \
 --service-principal guardduty.amazonaws.com

aws guardduty create-members \
 --detector-id abc123 \
 --account-details AccountId=222222222222,Email=team@example.com
```

### 105.2.4 Terraform

```
resource "aws_guardduty_detector" "main" {
 enable = true

 datasources {
 s3_logs { enable = true }
 kubernetes {
 audit_logs { enable = true }
 }
 malware_protection {
 scan_ec2_instance_with_findings {
 ebs_volumes { enable = true }
 }
 }
 }
}
```

### 105.2.5 Response workflow

```
GuardDuty finding → EventBridge rule → Lambda (auto-remediate) / SNS (alert SOC)
├─ Isolate instance (modify SG)
├─ Snapshot EBS for forensics
└─ Disable compromised IAM credentials
```

## 105.3 40.3 AWS Security Hub

**Security Hub** aggregates, organizes, and prioritizes security findings from multiple AWS services and third-party tools:

### 105.3.1 Integrated services

| Service             | Findings                            |
|---------------------|-------------------------------------|
| GuardDuty           | Threat detections                   |
| Inspector           | Vulnerability findings              |
| Config              | Compliance violations               |
| IAM Access Analyzer | External access findings            |
| Macie               | Sensitive data exposure             |
| Firewall Manager    | WAF rule violations                 |
| Third-party         | CrowdStrike, Palo Alto, Trend Micro |

### 105.3.2 Security standards

|  |  |
|--|--|
|  |  |
|--|--|

| Standard                                        | Framework                |
|-------------------------------------------------|--------------------------|
| <b>AWS Foundational Security Best Practices</b> | AWS-recommended controls |
| <b>CIS AWS Foundations Benchmark</b>            | CIS critical controls    |
| <b>PCI DSS</b>                                  | Payment card industry    |
| <b>NIST 800-53</b>                              | US government framework  |

```
resource "aws_securityhub_account" "main" {}

resource "aws_securityhub_standards_subscription" "fsbp" {
 standards_arn = "arn:aws:securityhub:us-east-1::standards/aws-foundational-security-best-practices/v/1.0.0"
 depends_on = [aws_securityhub_account.main]
}

resource "aws_securityhub_standards_subscription" "cis" {
 standards_arn = "arn:aws:securityhub:::ruleset/cis-aws-foundations-benchmark/v/1.4.0"
 depends_on = [aws_securityhub_account.main]
}
```

### 105.3.3 Finding workflow

| Status            | Meaning                    |
|-------------------|----------------------------|
| <b>NEW</b>        | Just received              |
| <b>NOTIFIED</b>   | Sent to integration        |
| <b>SUPPRESSED</b> | Acknowledged/accepted risk |
| <b>RESOLVED</b>   | Remediated                 |

### 105.3.4 Cross-account aggregation

Designate a **delegated administrator** account to aggregate findings from all organization accounts into a single dashboard.

## 105.4 40.4 AWS WAF

**AWS WAF** is a web application firewall that protects HTTP/HTTPS applications from common exploits:

### 105.4.1 Attach points

| Resource           | Use case                        |
|--------------------|---------------------------------|
| <b>CloudFront</b>  | Global edge protection          |
| <b>ALB</b>         | Regional application protection |
| <b>API Gateway</b> | API-specific rules              |
| <b>App Runner</b>  | Containerized app protection    |

### 105.4.2 Rule types

| Type                      | Description                                |
|---------------------------|--------------------------------------------|
| <b>Regular rule</b>       | Custom conditions (IP, headers, body, URI) |
| <b>Managed rule group</b> | AWS or marketplace pre-built rules         |
| <b>Rate-based rule</b>    | Block IPs exceeding request rate           |

### 105.4.3 AWS managed rule groups

| Group                                 | Protects against         |
|---------------------------------------|--------------------------|
| AWSManagedRulesCommonRuleSet          | OWASP Top 10 (SQLi, XSS) |
| AWSManagedRulesKnownBadInputs         | Log4j, Spring4Shell      |
| AWSManagedRulesSQLiRuleSet            | SQL injection            |
| AWSManagedRulesBotControlRuleSet      | Bot traffic              |
| AWSManagedRulesAmazonIpReputationList | Known malicious IPs      |

### 105.4.4 Terraform WAF

```
resource "aws_wafv2_web_acl" "main" {
 name = "app-waf"
 scope = "REGIONAL"

 default_action { allow {} }

 rule {
 name = "RateLimit"
 priority = 1
 action { block {} }
 statement {
 rate_based_statement {
 limit = 2000
 aggregate_key_type = "IP"
 }
 }
 visibility_config {
 cloudwatch_metrics_enabled = true
 metric_name = "RateLimit"
 sampled_requests_enabled = true
 }
 }

 rule {
 name = "AWSManagedRulesCommon"
 priority = 2
 override_action { none {} }
 statement {
 managed_rule_group_statement {
 name = "AWSManagedRulesCommonRuleSet"
 vendor_name = "AWS"
 }
 }
 visibility_config {
 cloudwatch_metrics_enabled = true
 metric_name = "CommonRules"
 sampled_requests_enabled = true
 }
 }

 visibility_config {
 cloudwatch_metrics_enabled = true
 metric_name = "AppWAF"
 sampled_requests_enabled = true
 }
}

resource "aws_wafv2_web_acl_association" "alb" {
 resource_arn = aws_lb.app.arn
 web_acl_arn = aws_wafv2_web_acl.main.arn
}
```

### 105.4.5 WAF logging

Send WAF logs to S3, CloudWatch Logs, or Kinesis Firehose for analysis:

```
aws wafv2 put-logging-configuration \
 --logging-configuration '{
```



```
"ResourceArn": "arn:aws:wafv2:us-east-1:123456789012:regional/webacl/app-waf/abc123",
"LogDestinationConfigs": ["arn:aws:firehose:us-east-1:123456789012:deliverystream/waf-logs"]
}'
```

## 105.5 40.5 AWS Shield

**Shield** provides DDoS protection for AWS resources:

| Tier            | Cost                          | Protection                                                               |
|-----------------|-------------------------------|--------------------------------------------------------------------------|
| Shield Standard | Free (automatic)              | Network/transport layer (L3/L4) for all AWS customers                    |
| Shield Advanced | \$3,000/month + data transfer | Enhanced L3/L4 + L7 protection, 24/7 DDoS Response Team, cost protection |

### 105.5.1 Shield Standard (automatic)

- Protects CloudFront, Route 53, ALB, NLB, Global Accelerator.
- No configuration required.
- Mitigates common SYN/UDP floods, reflection attacks.

### 105.5.2 Shield Advanced

| Feature                  | Benefit                                              |
|--------------------------|------------------------------------------------------|
| Advanced detection       | More sophisticated attack detection                  |
| DDoS Response Team (DRT) | 24/7 access to AWS DDoS experts                      |
| Cost protection          | Credits for scaling costs during attacks             |
| WAF included             | WAF fees waived for protected resources              |
| Real-time metrics        | Enhanced CloudWatch metrics and attack notifications |

### 105.5.3 DDoS mitigation architecture

```
Attacker → Shield Standard/Advanced → CloudFront (WAF) → ALB → Application
 ↓
 Automatic mitigation
 (SYN flood, UDP reflection, etc.)
```

## 105.6 40.6 Defense-in-depth architecture

```
Internet
 |
 ▼
Route 53 (Shield)
 |
 ▼
CloudFront + WAF (Shield + managed rules)
 |
 ▼
ALB + WAF (regional rules)
 |
 ▼
ECS/EC2 (Security Groups + GuardDuty monitoring)
 |
 ▼
RDS (encrypted, private subnet, Security Hub compliance)
```

| Layer | Service | Protects against |
|-------|---------|------------------|
|-------|---------|------------------|

|            |                           |                                         |
|------------|---------------------------|-----------------------------------------|
| DNS        | Shield                    | DNS amplification                       |
| Edge       | CloudFront + WAF + Shield | DDoS, OWASP Top 10                      |
| Network    | Security Groups, NACLs    | Unauthorized access                     |
| Detection  | GuardDuty                 | Compromised instances, credential theft |
| Compliance | Security Hub, Config      | Misconfigurations                       |
| Data       | KMS, Macie                | Encryption, data exposure               |

## 105.7 40.7 Incident response integration

```
resource "aws_cloudwatch_event_rule" "guardduty" {
 name = "guardduty-findings"
 event_pattern = jsonencode({
 source = ["aws.guardduty"]
 detail-type = ["GuardDuty Finding"]
 detail = {
 severity = [7, 7.0, 7.1, 7.2, 7.3, 7.4, 7.5, 7.6, 7.7, 7.8, 7.9,
 8, 8.0, 8.1, 8.2, 8.3, 8.4, 8.5, 8.6, 8.7, 8.8, 8.9]
 }
 })
}

resource "aws_cloudwatch_event_target" "sns" {
 rule = aws_cloudwatch_event_rule.guardduty.name
 target_id = "SendToSOC"
 arn = aws_sns_topic.security_alerts.arn
}
```

## 105.8 40.8 Chapter summary

- **GuardDuty** detects threats using ML across CloudTrail, VPC Flow Logs, and DNS logs.
- **Security Hub** aggregates findings and maps to compliance standards (CIS, PCI, NIST).
- **WAF** protects web applications with managed and custom rules at CloudFront and ALB.
- **Shield Standard** is free and automatic; **Shield Advanced** adds DRT and cost protection.
- Integrate findings with **EventBridge** for automated incident response.

## 105.9 Lab 40.1 — GuardDuty and Security Hub

1. Enable GuardDuty and Security Hub with the AWS Foundational standard.
2. Generate a test finding (e.g., unusual API call from a new IP).
3. Review the finding in GuardDuty and Security Hub.
4. Create an EventBridge rule to alert on high-severity findings.

## 105.10 Lab 40.2 — WAF protection

1. Create a WAF Web ACL with rate limiting and AWS managed rules.
2. Associate it with an ALB.
3. Test with a rate-exceeding script and verify blocking.
4. Review WAF sampled requests in the console.

## 105.11 Review questions

1. What data sources does GuardDuty analyze?
2. How does Security Hub differ from GuardDuty?
3. Where can AWS WAF be attached?
4. What is the difference between Shield Standard and Advanced?
5. How would you automate response to a high-severity GuardDuty finding?

---

Next: [\*Chapter 41 — Well-Architected Review & Capstone\*](#)

# 106 Chapter 41: Well-Architected Review & Capstone Design

AWS Handbook — Part X, Pages 791–800

## 106.1 41.1 The AWS Well-Architected Framework

Throughout this handbook, we have covered individual AWS services and patterns. The **AWS Well-Architected Framework** ties everything together into six pillars that define what “good” cloud architecture looks like. This capstone chapter applies the framework to a realistic design exercise, demonstrating how to evaluate and improve an architecture holistically.

## 106.2 41.2 The six pillars

| Pillar                 | Focus                           | Key questions                                    |
|------------------------|---------------------------------|--------------------------------------------------|
| Operational Excellence | Run and monitor systems         | Can you deploy, respond to events, and improve?  |
| Security               | Protect information and systems | Is data protected? Are threats detected?         |
| Reliability            | Recover from failures           | Can the system recover and meet demand?          |
| Performance Efficiency | Use resources efficiently       | Is the right technology used at the right scale? |
| Cost Optimization      | Avoid unnecessary costs         | Are you paying only for what you need?           |
| Sustainability         | Minimize environmental impact   | Are resources used efficiently to reduce carbon? |

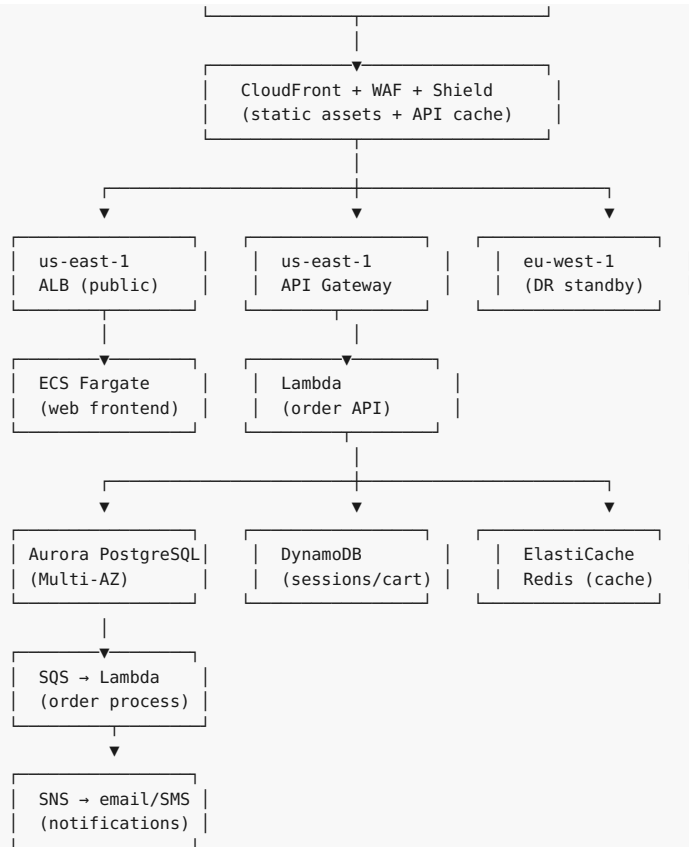
## 106.3 41.3 Capstone scenario

Design a **multi-tier e-commerce platform** with these requirements:

| Requirement  | Detail                                            |
|--------------|---------------------------------------------------|
| Traffic      | 10,000 concurrent users; 1M orders/month          |
| Availability | 99.95% uptime SLA                                 |
| Data         | Customer PII, payment references (PCI scope)      |
| Regions      | Primary: us-east-1; DR: eu-west-1                 |
| Team         | 5 developers, 2 DevOps engineers                  |
| Budget       | Optimize for cost without sacrificing reliability |

## 106.4 41.4 Proposed architecture





## 106.5 41.5 Pillar-by-pillar review

### 106.5.1 Operational Excellence

| Design decision                | Rationale                                     |
|--------------------------------|-----------------------------------------------|
| Infrastructure as Code (CDK)   | Reproducible environments; version-controlled |
| CI/CD via CodePipeline         | Automated testing and deployment              |
| CloudWatch dashboards + alarms | Centralized monitoring                        |
| Runbooks in Systems Manager    | Documented incident response                  |
| ECS Exec for debugging         | No SSH bastion needed                         |

**Review questions:** - [ ] Are deployments automated with rollback capability? - [ ] Are runbooks documented for common incidents? - [ ] Is infrastructure defined in code (not console clicks)? - [ ] Are changes tracked in version control?

### 106.5.2 Security

| Design decision                         | Rationale                         |
|-----------------------------------------|-----------------------------------|
| IAM Identity Center for human access    | Centralized SSO, MFA, audit       |
| Private subnets for all compute/data    | No direct internet exposure       |
| KMS encryption (RDS, S3, DynamoDB, EBS) | Data at rest protection           |
| Secrets Manager for DB credentials      | Automatic rotation                |
| WAF + Shield on CloudFront              | OWASP protection, DDoS mitigation |

|                                |                                                        |
|--------------------------------|--------------------------------------------------------|
| GuardDuty + Security Hub       | Threat detection and compliance                        |
| CloudTrail organization trail  | API audit logging                                      |
| VPC endpoints for AWS services | No internet traversal for API calls                    |
| PCI scope reduction            | No card data stored; tokenization via payment provider |

**Review questions:** - ☐ Is all data encrypted at rest and in transit? - ☐ Are security groups following least-privilege (SG references)? - ☐ Is GuardDuty enabled with automated response? - ☐ Are secrets rotated automatically? - ☐ Is the root account secured with MFA and not used daily?

### 106.5.3 Reliability

| Design decision                        | Rationale                               |
|----------------------------------------|-----------------------------------------|
| Multi-AZ Aurora                        | Automatic failover (< 30 seconds)       |
| ECS Fargate across 3 AZs               | Container-level HA                      |
| ALB health checks                      | Unhealthy targets removed automatically |
| SQS with DLQ                           | Order processing resilience             |
| Aurora Global Database (DR)            | Cross-region failover capability        |
| Auto Scaling (ECS + Aurora Serverless) | Handle traffic spikes                   |
| Route 53 health checks + failover      | DNS-level DR                            |
| AWS Backup for RDS and EBS             | Point-in-time recovery                  |

**Review questions:** - ☐ Are all tiers deployed across multiple AZs? - ☐ Is there a tested DR plan with defined RTO/RPO? - ☐ Are health checks configured on all load balancers? - ☐ Do async workflows use DLQs? - ☐ Has failover been tested in the last 6 months?

### 106.5.4 Performance Efficiency

| Design decision                          | Rationale                          |
|------------------------------------------|------------------------------------|
| CloudFront CDN                           | Static asset caching at edge       |
| ElastiCache Redis                        | Database query caching             |
| DynamoDB on-demand                       | Auto-scaling for cart/session data |
| Aurora Serverless v2 for variable load   | Right-size database compute        |
| Lambda for spiky order processing        | Scale to zero, pay per request     |
| Graviton instances (ARM) where supported | 20% better price-performance       |

**Review questions:** - ☐ Is caching used at multiple layers (CDN, application, database)? - ☐ Are instance types right-sized based on metrics? - ☐ Is serverless used for variable/spiky workloads? - ☐ Are Graviton instances evaluated for cost savings?

### 106.5.5 Cost Optimization

| Design decision                | Rationale                             |
|--------------------------------|---------------------------------------|
| Fargate (no idle EC2)          | Pay per task, not per instance        |
| Aurora Serverless v2           | Scale down during off-peak            |
| S3 lifecycle policies          | Move old data to IA/Glacier           |
| Reserved capacity for baseline | Savings Plans for predictable compute |

|                                   |                                     |
|-----------------------------------|-------------------------------------|
| CloudWatch cost anomaly detection | Alert on unexpected spend           |
| Right-sizing reviews quarterly    | Identify over-provisioned resources |

**Estimated monthly cost (simplified):**

| Service                          | Estimate            |
|----------------------------------|---------------------|
| ECS Fargate (4 tasks)            | \$120               |
| Aurora Serverless v2 (2-8 ACU)   | \$200               |
| DynamoDB on-demand               | \$50                |
| ElastiCache (cache.r6g.large)    | \$150               |
| CloudFront + WAF                 | \$100               |
| Lambda + API Gateway             | \$30                |
| Other (S3, SQS, SNS, monitoring) | \$50                |
| <b>Total</b>                     | <b>~\$700/month</b> |

**Review questions:** - [ ] Are unused resources identified and removed? - [ ] Are Savings Plans or Reserved Instances used for baseline? - [ ] Are S3 lifecycle policies configured? - [ ] Is cost allocation tagging enforced?

**106.5.6 Sustainability**

| Design decision              | Rationale                                       |
|------------------------------|-------------------------------------------------|
| Graviton (ARM) instances     | Lower energy per computation                    |
| Serverless (Lambda, Fargate) | No idle resource waste                          |
| S3 lifecycle to IA/Glacier   | Reduce storage energy footprint                 |
| Right-sizing                 | Fewer resources = less energy                   |
| Region selection             | Choose regions with renewable energy commitment |

**106.6 41.6 Well-Architected Tool review process**

AWS provides a free **Well-Architected Tool** in the console:

- 1. **Create a workload** — Define the architecture scope.
- 2. **Answer pillar questions** — ~50 questions across six pillars.
- 3. **Review risks** — High and medium risks identified with remediation guidance.
- 4. **Improve** — Implement changes and re-review.

```
Well-Architected Tool is console-based; use the API for automation
aws wellarchitected create-workload \
 --workload-name "E-Commerce Platform" \
 --description "Production e-commerce workload" \
 --environment PRODUCTION \
 --lenses "wellarchitected" \
 --review-owner "platform-team@example.com" \
 --aws-regions us-east-1 eu-west-1
```

**106.6.1 Review cadence**

| Trigger                   | Action                           |
|---------------------------|----------------------------------|
| New workload launch       | Initial review before production |
| Major architecture change | Re-review affected pillars       |

|               |                                               |
|---------------|-----------------------------------------------|
| Quarterly     | Scheduled review of all production workloads  |
| Post-incident | Review reliability and operational excellence |

## 106.7 41.7 Anti-patterns to avoid

| Anti-pattern                      | Risk                              | Fix                             |
|-----------------------------------|-----------------------------------|---------------------------------|
| Single AZ deployment              | AZ failure = outage               | Multi-AZ everything             |
| Public RDS/EC2                    | Data breach                       | Private subnets + SG references |
| Root account for daily work       | Credential compromise             | IAM Identity Center             |
| No monitoring/alarms              | Silent failures                   | CloudWatch alarms on all tiers  |
| Manual deployments                | Human error, slow recovery        | CI/CD pipeline                  |
| Hardcoded secrets                 | Credential leak                   | Secrets Manager                 |
| No backup testing                 | Backup exists but doesn't restore | Quarterly DR drills             |
| Over-permissive IAM               | Privilege escalation              | Least privilege + SCPs          |
| Ignoring costs until bill arrives | Budget overrun                    | Budgets + anomaly detection     |

## 106.8 41.8 Handbook journey recap

| Part                      | Topics covered                                               |
|---------------------------|--------------------------------------------------------------|
| <b>I — Foundations</b>    | Shared responsibility, global infrastructure, CLI, billing   |
| <b>II — IAM</b>           | Users, roles, policies, Identity Center, Organizations, SCPs |
| <b>III — Networking</b>   | VPC, NAT, security groups, NACLs, ALB/NLB, Route 53, TGW     |
| <b>IV — Compute</b>       | EC2, Auto Scaling, Lambda, ECS, EKS                          |
| <b>V — Storage</b>        | S3, EBS, EFS, backup and DR                                  |
| <b>VI — Databases</b>     | RDS, Aurora, DynamoDB, ElastiCache, DMS                      |
| <b>VII — Integration</b>  | API Gateway, SQS/SNS/EventBridge, Step Functions             |
| <b>VIII — DevOps</b>      | CloudFormation, CDK, CodePipeline, Systems Manager           |
| <b>IX — Observability</b> | CloudWatch, Logs Insights, X-Ray, CloudTrail, Config         |
| <b>X — Security</b>       | KMS, Secrets Manager, ACM, GuardDuty, WAF, Shield            |

## 106.9 41.9 Chapter summary

- The **Well-Architected Framework** provides six pillars for evaluating cloud architectures.
- Apply the framework systematically: design, review, improve, repeat.



- The capstone e-commerce design demonstrates multi-pillar thinking in practice.
  - Use the **Well-Architected Tool** for structured reviews with risk identification.
  - Avoid common **anti-patterns** that compromise security, reliability, and cost.
- 

## 106.10 Lab 41.1 — Well-Architected review

1. Open the AWS Well-Architected Tool in the console.
2. Create a workload for a project you have built (or the capstone design).
3. Complete the review for all six pillars.
4. Document the top 5 high-risk items and propose remediation.

## 106.11 Lab 41.2 — Architecture documentation

1. Draw the architecture diagram for your capstone (or real project).
  2. Annotate each component with its pillar contributions.
  3. Identify one improvement per pillar.
  4. Estimate the cost impact of each improvement.
- 

## 106.12 Review questions

1. Name the six Well-Architected Framework pillars.
  2. What is the target availability of 99.95% in downtime per year?
  3. How does the capstone design address the security pillar?
  4. Why should Well-Architected reviews be repeated after major changes?
  5. What is the most common anti-pattern you have seen in AWS environments?
- 

## 106.13 Final thoughts

You have completed the AWS Handbook—from cloud foundations through security and compliance. The AWS ecosystem evolves continuously; subscribe to AWS blogs, re:Invent sessions, and service announcements to stay current. The Well-Architected Framework is your compass for every new design decision.

Build securely. Automate everything. Monitor relentlessly. Review regularly.

— *End of AWS Handbook* —